

Core SQL Library — 500 Questions

SQL Interview Prep — 2026 Edition · Learn & Excel

This is the systematic deep-prep file of the pack. 10 chapters across the SQL surface area — fundamentals through modern SQL features. Each q follows the same format: tags · question · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up.

Table of Chapters

- Chapter 1 — SQL Fundamentals & Syntax
- Chapter 2, Part A — Joins
- Chapter 2, Part B — Subqueries
- Chapter 3, Part A — Window Functions
- Chapter 3, Part B — Common Table Expressions (CTEs)
- Chapter 4, Part A — Index Types & Design
- Chapter 4, Part B — Query Plans & Optimization
- Chapter 5, Part A — ACID & Isolation Levels
- Chapter 5, Part B — Locking & MVCC
- Chapter 5, Part C — Deadlocks & Transaction Conflicts
- Chapter 6 — Normalization & Schema Design
- Chapter 7 — Aggregations & Grouping
- Chapter 8 — Set Operations & Advanced SQL
- Chapter 9 — Stored Procedures, Functions & Triggers
- Chapter 10, Part A — JSON in SQL
- Chapter 10, Part B — MERGE, UPSERT, LATERAL & RETURNING
- Chapter 10, Part C — Modern Aggregation & NULL Handling
- Chapter 10, Part D — Generated Columns, Identity, Partitioning & Materialized Views

Chapter 1 — SQL Fundamentals & Syntax

This is the bedrock chapter. Every SQL interview — TCS fresher pool, Infosys lateral, Razorpay phone screen, JP Morgan GCC panel, Swiggy data role — opens with at least three or four questions from this set, and Indian service-MNC fresher rounds lean almost entirely here. The questions are not glamorous, but the candidates who fumble DDL-vs-DML, NULL semantics, or GROUP BY rules get cut in the first ten minutes. Master these twenty and you'll never lose a round to a "basic" question again. Each page below gives you the 30-second answer first, then the layered reasoning, then the one edge case the interviewer is waiting to hear you say out loud.

Section A — Statement Categories & Schema Basics

Q1 · DDL vs DML vs DCL vs TCL — What Belongs Where

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Accenture · Topic: SQL Fundamentals

The question (interviewer's verbatim phrasing):

Can you tell me what DDL, DML, DCL and TCL stand for, and give me at least two example statements for each category?

The 30-second answer: DDL is Data Definition Language — CREATE, ALTER, DROP, TRUNCATE — it changes schema. DML is Data Manipulation Language — INSERT, UPDATE, DELETE, (and SELECT in many lists) — it changes rows. DCL is Data Control Language — GRANT, REVOKE — it changes permissions. TCL is Transaction Control Language — COMMIT, ROLLBACK, SAVEPOINT — it controls transaction boundaries.

Step-by-step thought process: 1. Open with the one-line definition of each category — interviewers are mentally ticking off four boxes. 2. Pair each acronym with at least two real statements; don't just expand the acronym and stop. 3. Mention the grey areas: SELECT is technically DQL (Data Query Language) in strict classifications but most Indian textbooks bucket it under DML, so say both and move on. 4. Be ready for the auto-commit trap: in MySQL and Oracle, DDL statements implicitly COMMIT — so you cannot ROLLBACK a CREATE TABLE.

Edge cases to mention: - TRUNCATE is DDL, not DML, even though it removes rows — that's why it is faster and cannot be rolled back in MySQL. - Postgres treats DDL as transactional, so you can wrap CREATE TABLE inside BEGIN...ROLLBACK — unique among the big engines. - GRANT and REVOKE are sometimes called DCL and sometimes lumped under DDL by older study guides; mention both names.

What NOT to say: - Do not say "DML changes the database" — DDL also changes the database; DML specifically changes rows. - Do not forget SAVEPOINT under TCL — Indian campus panels love asking for the third TCL statement after COMMIT and ROLLBACK.

Common follow-up: *"If I run TRUNCATE inside a transaction and then ROLLBACK, what happens in MySQL versus Postgres?"* — MySQL: TRUNCATE auto-commits, ROLLBACK does nothing. Postgres: TRUNCATE is transactional, ROLLBACK restores all rows.

```
-- DDL
CREATE TABLE payments (id BIGINT PRIMARY KEY, amount DECIMAL(12,2));
-- DML
INSERT INTO payments VALUES (1, 499.00);
-- DCL
GRANT SELECT ON payments TO analyst_role;
-- TCL
BEGIN; UPDATE payments SET amount = 599 WHERE id = 1; ROLLBACK;
```

Q2 · TRUNCATE vs DELETE vs DROP

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wells Fargo, Walmart Labs, Cognizant · Topic: SQL Fundamentals

The question (interviewer's verbatim phrasing):

Walk me through the difference between TRUNCATE, DELETE and DROP. Which one resets the auto-increment counter and which one is the fastest?

The 30-second answer: DELETE is DML — removes rows one at a time, fires triggers, can be rolled back, keeps the table. TRUNCATE is DDL — deallocates pages, no per-row triggers, resets the identity counter, much faster, usually cannot be rolled back. DROP is DDL — removes the table itself, schema and all.

Step-by-step thought process: 1. State the three categories first (DML vs DDL) — that anchors every other difference. 2. Walk through speed: DELETE writes to the WAL/redo log row by row; TRUNCATE just marks pages as free. On a 50-million row orders table, DELETE can take an hour, TRUNCATE finishes in seconds. 3. Mention the identity reset — TRUNCATE resets AUTO_INCREMENT in MySQL and IDENTITY in SQL Server back to the seed; DELETE does not. 4. Foreign key gotcha: TRUNCATE fails on a table referenced by an FK with rows in the child table; DELETE works because it cascades.

Edge cases to mention: - DELETE without a WHERE clause and TRUNCATE both empty the table — but only DELETE fires per-row triggers and audit hooks. - In Oracle and Postgres, TRUNCATE can be made transactional; in MySQL it is implicit-commit and irreversible. - DROP can be reversed only if you

have a backup or if your engine supports flashback (Oracle) — there is no ROLLBACK after DROP in standard SQL.

What NOT to say: - Do not say "TRUNCATE and DELETE FROM are the same with no WHERE clause" — they are not, especially for triggers, identity counters, and FK behavior. - Do not claim DROP is recoverable via ROLLBACK in MySQL — it is not.

Common follow-up: "You truncated the wrong table in production. What is your recovery path?" — Restore from the latest backup plus replay binary logs / WAL up to the truncate point; there is no undo button.

```
DELETE FROM orders WHERE status = 'cancelled'; -- selective, logged, slow
TRUNCATE TABLE orders;                        -- whole table, fast, no rollback (MySQL)
DROP TABLE orders;                            -- table gone, schema gone
```

Q3 · ALTER TABLE — Add, Modify, Drop Columns Without Killing Production

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred, Meesho · Topic: Schema Changes

The question (interviewer's verbatim phrasing):

I need to add a new `gst_number` column to the merchants table which has 80 million rows. Walk me through what ALTER TABLE will do, the risks, and how you would do it safely.

The 30-second answer: ALTER TABLE ADD COLUMN with a nullable column and no default is usually metadata-only in modern Postgres and MySQL 8 — instant. ADD COLUMN with a NOT NULL default rewrites the whole table in older versions, locks it, and can take hours. The safe path: add nullable, backfill in batches, then add the NOT NULL constraint.

Step-by-step thought process: 1. State the basic syntax — ADD COLUMN, MODIFY/ALTER COLUMN, DROP COLUMN — then immediately move to the performance side. 2. Distinguish metadata-only changes (instant) from table rewrites (slow, locking). In MySQL 8 and Postgres 11+, adding a nullable column with no default is metadata-only. 3. Walk through the safe four-step recipe: ADD nullable column → backfill in chunked UPDATES of 10k rows → add NOT NULL constraint via VALIDATE (Postgres) or two-step (MySQL) → deploy app code. 4. Mention online DDL tools — pt-online-schema-change for MySQL, pg_repack for Postgres — when the table is genuinely too hot for native ALTER.

Edge cases to mention: - MODIFY COLUMN to change a VARCHAR(50) to VARCHAR(100) is usually metadata-only; shrinking it forces a full table scan to validate. - DROP COLUMN in Postgres is metadata-only (column marked dropped, space reclaimed on next VACUUM FULL); in MySQL it rewrites the table. - Adding an FK constraint with CHECK FULL re-scans the child table and takes a strong lock — use NOT VALID then VALIDATE on Postgres.

What NOT to say: - Do not say "ALTER TABLE is instant" — that is only true for specific cases on specific engines and versions. - Do not propose running ALTER on an 80M-row table during peak traffic without explaining the locking implications.

Common follow-up: "How would you backfill the `gst_number` column without locking the table?" — UPDATE in batches of 5–10k rows by primary key range, with a short sleep between batches; commit each batch; monitor replication lag.

```
-- Safe pattern
ALTER TABLE merchants ADD COLUMN gst_number VARCHAR(15) NULL;
-- backfill in chunks (pseudo)
-- UPDATE merchants SET gst_number = lookup(id) WHERE id BETWEEN ? AND ?;
ALTER TABLE merchants MODIFY gst_number VARCHAR(15) NOT NULL; -- MySQL
```

Section B — Data Types & Constraints

Q4 · Choosing Numeric Types — INT, BIGINT, DECIMAL, NUMERIC, FLOAT

Tags: Difficulty: Easy · Asked at: TCS, Infosys, JP Morgan, Goldman, Razorpay · Topic: Data Types

The question (interviewer's verbatim phrasing):

I'm designing a payments table. What data type would you use for the amount in rupees, for the order id, and why?

The 30-second answer: Amount — DECIMAL(12,2) or NUMERIC(12,2); never FLOAT for money because binary floating point cannot represent 0.10 exactly and you will see paise-level rounding errors. Order id — BIGINT, because INT tops out at ~2.1 billion and a fast-growing Indian D2C company can hit that ceiling within a couple of years.

Step-by-step thought process: 1. Start with the money rule: DECIMAL/NUMERIC for currency, always. Mention $0.1 + 0.2 \neq 0.3$ in FLOAT — that is the canonical example. 2. For ids, default to BIGINT in any new design — the extra 4 bytes per row is irrelevant; the cost of migrating INT to BIGINT later is enormous. 3. INT for counts and small enums (`status_code`, `retry_count`) where ~2.1B is comfortably more than you will ever need. 4. Mention SMALLINT and TINYINT only when storage matters at scale — analytic warehouses, narrow lookup tables.

Edge cases to mention: - DECIMAL and NUMERIC are synonyms in the ANSI standard; in some engines they are minutely different — call them out as effectively identical for the interview. - FLOAT and DOUBLE store more digits but cannot represent decimal fractions exactly — fine for sensor data, deadly for money. - BIGINT signed maxes at $\sim 9.2 \times 10^{18}$ — effectively unlimited for an id column.

What NOT to say: - Do not say "I'll use FLOAT because amounts can be fractional." That is the exact answer that fails the question. - Do not say "INT is enough for ids, BIGINT wastes space." It is 4 bytes per row — irrelevant against the migration cost.

Common follow-up: "What is the difference between `DECIMAL(10,2)` and `DECIMAL(12,2)`?" — `DECIMAL(10,2)` holds up to 99,999,999.99 (10 total digits, 2 after the point); `DECIMAL(12,2)` goes up to 9,999,999,999.99 — pick based on the largest realistic amount plus headroom.

```
CREATE TABLE payments (  
  id BIGINT PRIMARY KEY,  
  order_id BIGINT NOT NULL,  
  amount_inr DECIMAL(12,2) NOT NULL,    -- never FLOAT for money  
  status SMALLINT NOT NULL  
);
```

Q5 · VARCHAR vs CHAR vs TEXT — When to Use Which

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Cognizant · Topic: Data Types

The question (interviewer's verbatim phrasing):

What is the difference between CHAR, VARCHAR and TEXT, and when would you use TEXT over VARCHAR?

The 30-second answer: CHAR is fixed-length and pads with spaces; VARCHAR is variable-length up to a defined limit; TEXT is variable-length with no practical limit but is stored off-row in most engines, which makes it slower for filtering and indexing. Use VARCHAR for most text columns, CHAR only for genuinely fixed-width values like country codes, TEXT for free-form blobs like product descriptions or article bodies.

Step-by-step thought process: 1. Open with the fixed-vs-variable distinction — that is the core difference. 2. Mention storage: CHAR(10) always uses 10 characters of storage even if you store "OK"; VARCHAR uses what you write plus 1–2 bytes of length prefix. 3. Call out TEXT's off-row storage: in MySQL InnoDB and Postgres TOAST, large TEXT values live in a separate area and require an extra fetch — fine if you read rarely, painful for filters. 4. Indexing limit: many engines cap the indexed length of TEXT columns (MySQL needs a prefix length), VARCHAR is fully indexable up to its declared length.

Edge cases to mention: - VARCHAR(255) is a historical convention from MySQL where 255 was the largest value that fit in a single byte of length prefix — not a magic number any more. - CHAR comparisons sometimes ignore trailing spaces depending on collation — gotcha when joining CHAR to VARCHAR. - In Postgres, VARCHAR and TEXT are essentially the same storage-wise; the length cap is just a CHECK constraint.

What NOT to say: - Do not say "always use TEXT to avoid running out of space." TEXT has real indexing and performance costs. - Do not say "CHAR is faster because it's fixed-length" — modern engines handle VARCHAR efficiently and CHAR wastes storage.

Common follow-up: *"Why is VARCHAR(255) so common in legacy schemas?"* — Historical MySQL artefact: 255 was the largest length that fit in 1 byte of length prefix in older row formats. No longer a constraint in modern formats.

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  country_code CHAR(2) NOT NULL,      -- always 2 chars: 'IN', 'US'  
  email VARCHAR(254) NOT NULL,       -- RFC max length  
  bio TEXT                           -- free-form, off-row  
);
```

Q6 · DATE vs DATETIME vs TIMESTAMP vs TIME — Which One When

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, JP Morgan · Topic: Data Types

The question (interviewer's verbatim phrasing):

What's the difference between DATETIME and TIMESTAMP in MySQL? Which one would you use to store the time a payment was captured?

The 30-second answer: DATE stores just the calendar date. DATETIME stores a date + time with no timezone awareness. TIMESTAMP also stores date + time but in MySQL is stored internally as UTC and converted to the session timezone on read; in Postgres, TIMESTAMP WITH TIME ZONE is the equivalent. For a payment captured-at, use TIMESTAMP (or TIMESTAMPTZ in Postgres) so that a payment made at 3 PM IST is read correctly by your London-based reconciliation team.

Step-by-step thought process: 1. Start with the calendar-vs-instant distinction. DATE = day. DATETIME = wall-clock time with no timezone. TIMESTAMP = a fixed instant in time. 2. For anything event-related (paid_at, created_at, last_login) you want TIMESTAMP — multi-region teams demand timezone correctness. 3. For birthdays and holidays, DATE is right — those are calendar concepts, not instants. 4. Mention the MySQL TIMESTAMP 2038 problem — 4-byte signed int seconds since epoch, wraps in January 2038. DATETIME does not have this issue and is the modern default for distant-future or distant-past values.

Edge cases to mention: - TIMESTAMP in MySQL is automatically converted to UTC on write and back on read; if your application sets a wrong session timezone, you get silent corruption. - Postgres has both TIMESTAMP and TIMESTAMPTZ — always default to TIMESTAMPTZ unless you have a reason not to. -

TIME stores a duration or time-of-day without a date — rarely useful in transactional schemas, occasionally in scheduling.

What NOT to say: - Do not say "TIMESTAMP and DATETIME are the same in MySQL." They differ in timezone handling and storage range. - Do not store dates as VARCHAR — every junior schema has this mistake; sorting, filtering and date arithmetic break.

Common follow-up: "How do you query all payments from yesterday in Asia/Kolkata?" — Convert the boundary explicitly: `WHERE paid_at >= TIMESTAMP '2026-05-25 00:00:00+05:30' AND paid_at < TIMESTAMP '2026-05-26 00:00:00+05:30'`. Never trust implicit timezone of the session.

```
CREATE TABLE payments (  
  id BIGINT PRIMARY KEY,  
  paid_at TIMESTAMP NOT NULL,           -- instant; UTC-stored  
  refund_eligible_until DATE            -- calendar date, no time  
);
```

Q7 · BOOLEAN, BIT, TINYINT(1) — How Engines Actually Store Truth

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Wipro, Accenture · Topic: Data Types

The question (interviewer's verbatim phrasing):

Does MySQL have a real BOOLEAN type? How do you store a true/false flag like "is_verified" portably across MySQL, Postgres and SQL Server?

The 30-second answer: MySQL does not have a true boolean — BOOLEAN and BOOL are synonyms for TINYINT(1), and TRUE/FALSE are aliases for 1 and 0. Postgres has a real BOOLEAN type that stores t/f. SQL Server uses BIT. The portable schema convention is to declare the column as BOOLEAN, accept the engine-specific physical type, and never compare flags to literal integers in application code.

Step-by-step thought process: 1. Lead with the MySQL gotcha — Indian campus students often think MySQL has a real boolean and are surprised that `SELECT TRUE` returns 1. 2. Postgres is the cleanest — `WHERE is_verified` works directly; in MySQL the same query also works because non-zero TINYINT is truthy. 3. Mention NULL — a boolean column allows three states unless declared NOT NULL: TRUE, FALSE, UNKNOWN. Design your flag columns NOT NULL DEFAULT FALSE to avoid the third state. 4. Anticipate the follow-up about indexing low-cardinality columns: a boolean alone is a poor index; combine it with a more selective column.

Edge cases to mention: - In MySQL, you can insert 2 into a TINYINT(1) boolean column — it stores 2, and `WHERE flag = TRUE` will not match because `TRUE = 1` exactly. Use `WHERE flag != 0` if you mean

"truthy". - In Postgres, the literals are TRUE, FALSE, 't', 'f', 'yes', 'no' — be careful when comparing string literals. - Some ORMs (Hibernate, Sequelize) map Java/JS booleans to '1'/'0' strings — silent type mismatch with the database column.

What NOT to say: - Do not claim MySQL's BOOLEAN is a real boolean type — it is a TINYINT(1) alias. - Do not say "I'll use 'Y' / 'N' chars for clarity" — every linter, every analyst tool, and every JSON serializer expects true booleans.

Common follow-up: *"What's the cardinality issue with indexing a boolean column?"* — Two values means about half the rows match either way; the optimizer often prefers a full scan over the index. Boolean columns are useful as the second column in a composite index, not as a standalone one.

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  email VARCHAR(254) NOT NULL,  
  is_verified BOOLEAN NOT NULL DEFAULT FALSE  
);
```

Q8 · Column Constraints — PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL, DEFAULT

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Wells Fargo · Topic: Constraints

The question (interviewer's verbatim phrasing):

List the column-level constraints in SQL and tell me when you'd use a CHECK constraint instead of enforcing it in application code.

The 30-second answer: Six common ones — PRIMARY KEY (uniquely identifies a row, implies NOT NULL and UNIQUE), FOREIGN KEY (references another table's PK or unique column), UNIQUE (no duplicates but allows NULL), CHECK (arbitrary boolean condition per row), NOT NULL (rejects NULL), DEFAULT (assigns a value when none is supplied). Use CHECK for invariants that must hold regardless of which application is writing — e.g., amount > 0 on a payments table.

Step-by-step thought process: 1. Name all six clearly — interviewers want to hear PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL, DEFAULT in one breath. 2. Distinguish PRIMARY KEY from UNIQUE: PRIMARY KEY implies NOT NULL, UNIQUE allows one NULL (or many, depending on engine). 3. Explain CHECK with a real example: `CHECK (amount > 0)` on payments, `CHECK (status IN ('pending', 'captured', 'failed', 'refunded'))` on a status column. 4. Stress the database-as-last-line-of-defence principle: app code can have bugs, multiple services may write to the same table, CHECK constraints catch everything that slips through.

Edge cases to mention: - UNIQUE allows multiple NULLs in most engines because NULL != NULL — Postgres, MySQL, Oracle all behave this way; SQL Server allows only one NULL by default. - A FOREIGN KEY without an index on the child column can make DELETES on the parent very slow — always index FK columns. - CHECK constraints were not enforced in MySQL until 8.0.16; older versions silently parsed and ignored them.

What NOT to say: - Do not say "PRIMARY KEY and UNIQUE are the same." They differ on NULL handling and you can have many UNIQUE columns but only one PRIMARY KEY. - Do not skip DEFAULT — Indian campus panels often ask "name all constraints" and DEFAULT is the one most people forget.

Common follow-up: "What happens if I insert a row that violates a FOREIGN KEY constraint?" — The INSERT is rejected with an integrity error; the transaction continues unless you have configured it to roll back on error.

```
CREATE TABLE payments (  
  id BIGINT PRIMARY KEY,  
  order_id BIGINT NOT NULL REFERENCES orders(id),  
  amount_inr DECIMAL(12,2) NOT NULL CHECK (amount_inr > 0),  
  status VARCHAR(20) NOT NULL DEFAULT 'pending',  
  idempotency_key VARCHAR(64) UNIQUE  
);
```

Q9 · Identity, Sequence, AUTO_INCREMENT, SERIAL — Auto-Generating PKs

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Walmart Labs, Cognizant · Topic: Data Types

The question (interviewer's verbatim phrasing):

How do you auto-generate a primary key in MySQL, Postgres and SQL Server? What is the difference between SERIAL and GENERATED BY DEFAULT AS IDENTITY?

The 30-second answer: MySQL — AUTO_INCREMENT attribute on the column. SQL Server — IDENTITY(seed, increment). Postgres — historically SERIAL (which is sugar for a sequence + default + ownership), but the modern ANSI-standard form is GENERATED BY DEFAULT AS IDENTITY (or GENERATED ALWAYS AS IDENTITY for stricter control). All four ultimately use a counter; the differences are syntax, restart behaviour, and whether the user can override the value.

Step-by-step thought process: 1. Map each dialect to its keyword first — that's table stakes. 2. Walk through SERIAL vs IDENTITY in Postgres: SERIAL is an older convenience that creates a sequence object behind the scenes; IDENTITY is the SQL standard and gives cleaner ownership and permission semantics. New Postgres projects should prefer IDENTITY. 3. ALWAYS vs DEFAULT — GENERATED

ALWAYS rejects user-supplied values for the column; GENERATED BY DEFAULT allows them. Use ALWAYS for surrogate keys you never want apps to override. 4. Mention sequence holes: AUTO_INCREMENT and sequences never reuse a number, so a rolled-back INSERT leaves a gap. Do not promise gap-free numbering to a business stakeholder.

Edge cases to mention: - AUTO_INCREMENT in MySQL is stored in memory in older versions; after a server restart, the next value could be reset to MAX(id)+1, which can collide with rows in replication queues — fixed in MySQL 8 with persistent counters. - IDENTITY columns in SQL Server cannot be updated directly; you need SET IDENTITY_INSERT ON. - Sequences in Postgres are independent objects — you can share one across multiple tables, restart it, set its value, or grant on it separately.

What NOT to say: - Do not say "AUTO_INCREMENT guarantees consecutive numbers" — it does not; rollbacks leave gaps. - Do not say "SERIAL is the same as IDENTITY" — they are similar in effect but differ in ownership and standards compliance.

Common follow-up: "How do you get the last inserted id?" — MySQL: `LAST_INSERT_ID()`. Postgres: `RETURNING id` on the INSERT, or `currval('seq_name')`. SQL Server: `SCOPE_IDENTITY()`. JDBC and most drivers also expose a `getGeneratedKeys` API.

```
-- MySQL
CREATE TABLE orders (id BIGINT AUTO_INCREMENT PRIMARY KEY, ...);
-- Postgres (modern)
CREATE TABLE orders (id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, ...);
-- SQL Server
CREATE TABLE orders (id BIGINT IDENTITY(1,1) PRIMARY KEY, ...);
```

Q10 · CAST vs CONVERT — Explicit vs Implicit, When Implicit Is Dangerous

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, Razorpay · Topic: Data Types

The question (interviewer's verbatim phrasing):

What is the difference between CAST and CONVERT? Give me an example where implicit type conversion silently produced the wrong answer.

The 30-second answer: CAST is the ANSI-standard form — `CAST(expr AS type)` — works in every engine. CONVERT is engine-specific: in SQL Server it takes a style code for date formatting; in MySQL it is similar to CAST with charset support. Implicit conversion is when the engine auto-coerces — usually fine for INT-to-BIGINT, dangerous for VARCHAR-to-NUMBER because a non-numeric string can throw at runtime, and worst for VARCHAR-to-DATE because two engines may pick different parse formats and silently produce different rows.

Step-by-step thought process: 1. Lead with the standard answer — CAST is portable, CONVERT is engine-specific. 2. Give the classic implicit-conversion trap: `WHERE user_id = '123'` where `user_id` is BIGINT — works, but loses index usage in some engines because the index entries are numbers and the comparison forces a per-row cast. 3. Date strings are the worst: `WHERE order_date > '01-02-2026'` is ambiguous — is that 1 Feb or 2 Jan? Different parse settings give silently different rows. 4. The safe rule: always cast explicitly and always use ISO 8601 date literals (YYYY-MM-DD).

Edge cases to mention: - Implicit conversion can prevent index use: comparing an indexed VARCHAR column to an integer literal sometimes forces the engine to cast every row. - TRY_CAST (SQL Server) and TRY_CONVERT return NULL on failure instead of throwing — useful when cleaning messy data. - Postgres has the cast operator `::` — `'42'::int` — which is shorter but non-standard.

What NOT to say: - Do not say "implicit conversion is harmless." It can break index use and silently change result correctness. - Do not say "CAST and CONVERT are interchangeable" without qualifying — they are syntactically similar but CONVERT has engine-specific extras.

Common follow-up: "How does implicit conversion break index use?" — If the engine has to cast every indexed value to match the literal's type, it can't binary-search the index — falls back to a sequential scan. Always cast the literal to the column's type, not the other way around.

```
SELECT CAST('2026-05-26' AS DATE) AS d;           -- portable
SELECT CONVERT(VARCHAR, GETDATE(), 23);          -- SQL Server: ISO date
SELECT '42'::int + 1;                             -- Postgres shorthand
```

Section C — NULL, GROUP BY, HAVING, ORDER BY

Q11 · NULL Semantics and Three-Valued Logic

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Goldman, JP Morgan, Swiggy · Topic: NULL

The question (interviewer's verbatim phrasing):

What does NULL mean in SQL, and why does the expression `WHERE column = NULL` never return any rows?

The 30-second answer: NULL means "unknown" — not zero, not empty string, not missing. SQL uses three-valued logic: TRUE, FALSE, UNKNOWN. Any comparison with NULL — including `NULL = NULL` — evaluates to UNKNOWN, and WHERE only keeps rows where the predicate is TRUE. That is why `WHERE column = NULL` returns nothing — you must use `WHERE column IS NULL` instead.

Step-by-step thought process: 1. Open with the "unknown, not missing" framing — that single mental model fixes every NULL question. 2. Walk through three-valued logic: AND/OR/NOT have nine-row truth tables instead of four. TRUE AND UNKNOWN = UNKNOWN. FALSE AND UNKNOWN = FALSE (because no matter what UNKNOWN turns out to be, the AND is false). 3. Give the canonical traps: `NULL = NULL` is UNKNOWN; `NULL <> 1` is UNKNOWN; `NOT (NULL = 1)` is UNKNOWN. 4. The two correct ways to test for NULL: `IS NULL` and `IS NOT NULL`. Some dialects support `IS DISTINCT FROM` which treats two NULLs as equal — handy for change detection.

Edge cases to mention: - Aggregate functions skip NULLs — `AVG(amount)` averages only the non-null rows, which can be surprising; `COUNT(*)` counts all rows, `COUNT(amount)` counts only non-null amounts. - UNIQUE constraints allow multiple NULLs in most engines because `NULL != NULL`. - `COALESCE(col, default)` is the standard way to substitute a value for NULL; `NVL` is the Oracle dialect; `IFNULL` is MySQL.

What NOT to say: - Do not say "NULL is the same as zero" or "NULL is the same as empty string." They are different — `'' = ''` is TRUE; `NULL = NULL` is UNKNOWN. - Do not write `WHERE col = NULL` in production code and expect it to find NULL rows.

Common follow-up: "What does `COUNT(DISTINCT col)` do when col contains NULLs?" — It counts only the distinct non-NULL values; NULL is excluded. To include NULL as one of the distinct values, you have to count it separately.

```
SELECT * FROM users WHERE last_login_at IS NULL;      -- correct
SELECT * FROM users WHERE last_login_at = NULL;      -- always empty
SELECT COALESCE(promo_code, 'none') FROM bookings;  -- substitute
```

Q12 · GROUP BY — The Every-Non-Aggregated-Column Rule

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Cognizant · Topic: Aggregation

The question (interviewer's verbatim phrasing):

What rules govern which columns can appear in the SELECT list when you use GROUP BY? Why does this query fail in Postgres but pass in older MySQL?

The 30-second answer: Every column in the SELECT list must either appear inside an aggregate function (SUM, AVG, COUNT, MIN, MAX) or appear in the GROUP BY clause. Otherwise the value would be ambiguous — which row's value do you want? Postgres and SQL Server enforce this strictly. MySQL before 5.7 allowed it (returning an arbitrary value) — modern MySQL with `ONLY_FULL_GROUP_BY` enforces the rule too.

Step-by-step thought process: 1. State the rule precisely — "every non-aggregated column must appear in GROUP BY" — that one sentence is the answer for 80% of follow-ups. 2. Explain why: a GROUP BY collapses many rows into one; a column not in GROUP BY and not aggregated has no single defined value to return. 3. Mention the MySQL exception: pre-5.7 it returned an arbitrary row's value from the group, silently — a notorious source of "works on my machine" bugs. Modern MySQL with ONLY_FULL_GROUP_BY mode rejects it like Postgres does. 4. Functional dependency exception: if you GROUP BY the primary key, the engine knows every other column is uniquely determined by it, and modern Postgres allows them in SELECT — neat trick that confuses interviewers.

Edge cases to mention: - Expressions like `EXTRACT(MONTH FROM order_date)` must either be in GROUP BY exactly or wrapped in an aggregate — you cannot group by the column and select a derived expression of it without restating it. - `SELECT *` with GROUP BY is almost always wrong — you are asking for every column to be either grouped or aggregated, and `*` does neither. - GROUP BY with no rows in the input returns no rows; GROUP BY () with no input still returns one row of aggregates (e.g., `SELECT COUNT(*) FROM empty_table` returns 0).

What NOT to say: - Do not say "MySQL is more flexible than Postgres" as a positive — that flexibility silently returns wrong data. - Do not put a non-aggregated, non-grouped column in SELECT and hope the optimizer picks a sensible value.

Common follow-up: "For each merchant, show the merchant name and the total amount of orders in May 2026." — Group by merchant id, select merchant name (functionally dependent on id), and SUM(amount). Join merchants table for the name if it lives in another table.

```
SELECT merchant_id, COUNT(*) AS order_count, SUM(amount_inr) AS total
FROM orders
WHERE order_date >= '2026-05-01'
GROUP BY merchant_id;
```

Q13 · HAVING vs WHERE — Where Each One Belongs in the Pipeline

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, Wells Fargo · Topic: Aggregation

The question (interviewer's verbatim phrasing):

When do you use HAVING and when do you use WHERE? Can you give me an example where putting the same condition in HAVING versus WHERE produces a different answer?

The 30-second answer: WHERE filters individual rows before they are grouped; HAVING filters groups after aggregation. If you want only paid orders in your sum, that is a WHERE filter. If you want only merchants whose total exceeds 1 lakh, that is a HAVING filter — because "total exceeds 1 lakh" can only be evaluated after the SUM.

Step-by-step thought process: 1. Frame the logical order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. That ordering explains every difference. 2. Give the rule: aggregates can only appear in HAVING (and SELECT and ORDER BY); never in WHERE. 3. Show the divergent example: `WHERE amount > 1000` keeps only large individual orders before summing; `HAVING SUM(amount) > 1000` keeps merchants whose total crossed 1000. 4. Performance: filter as early as possible — push every non-aggregate condition into WHERE so the GROUP BY processes fewer rows.

Edge cases to mention: - You can use HAVING without GROUP BY — the whole table becomes one group, and HAVING filters that one summary row. - You cannot reference a SELECT alias inside WHERE (in most engines), but HAVING can sometimes reference aggregated aliases. - Window functions are evaluated after HAVING but before ORDER BY — that's why you cannot filter on a window function result in WHERE or HAVING; you need an outer query or CTE.

What NOT to say: - Do not say "HAVING and WHERE are interchangeable." They sit at different stages of the pipeline and produce different results. - Do not put `SUM(amount) > 1000` in a WHERE clause and expect it to work — most engines reject it; some give cryptic errors.

Common follow-up: "Show merchants whose total May revenue exceeded 1 lakh from captured payments only." — WHERE filters payments to captured + May; GROUP BY merchant; HAVING filters to SUM > 100000.

```
SELECT merchant_id, SUM(amount_inr) AS total
FROM payments
WHERE status = 'captured'
      AND paid_at >= '2026-05-01' AND paid_at < '2026-06-01'
GROUP BY merchant_id
HAVING SUM(amount_inr) > 100000;
```

Q14 · ORDER BY — Multiple Columns, NULL Placement, Expressions and Aliases

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Swiggy · Topic: Sorting

The question (interviewer's verbatim phrasing):

Sort orders by date descending, then by amount descending. Where will NULLs appear by default, and how do you control that?

The 30-second answer: `ORDER BY order_date DESC, amount_inr DESC` — comma-separated, each with its own direction. NULL placement is engine-dependent: Postgres puts NULLs last on ASC and first on DESC by default; MySQL is the opposite. Use the ANSI clause `NULLS FIRST` or `NULLS LAST` to control it portably (Postgres, Oracle); in MySQL emulate with `ORDER BY col IS NULL, col`.

Step-by-step thought process: 1. Lead with the multi-column syntax — each column gets its own ASC/DESC. Default is ASC. 2. Walk through NULL placement defaults — Postgres ASC NULLs last, DESC NULLs first; MySQL ASC NULLs first; Oracle ASC NULLs last; SQL Server ASC NULLs first. Memorising the matrix is less valuable than knowing how to override it. 3. Mention that you can ORDER BY an expression, a SELECT alias (in most engines), or a column position number — but position numbers are brittle and discouraged in modern code. 4. ORDER BY happens after GROUP BY and HAVING — so you can sort by an aggregate result like `SUM(amount) DESC`.

Edge cases to mention: - ORDER BY with LIMIT but no tie-breaker can return different rows on each run if there are ties — always add a deterministic tie-breaker like the primary key. - ORDER BY a column not in SELECT is legal in most engines; ORDER BY a column not visible to the query (e.g., after a SELECT DISTINCT) may be rejected. - Sorting on a TEXT or large VARCHAR column without an index forces a sort buffer or temporary file — can be expensive on large result sets.

What NOT to say: - Do not assume the default sort order is the same across engines — it is not for NULLs. - Do not use ORDER BY 1, 2, 3 (position-based) in code review — it's a maintenance landmine when the SELECT list changes.

Common follow-up: "Show the top 10 highest-value orders this month, deterministically, with NULL amounts pushed to the bottom." — `ORDER BY amount_inr DESC NULLS LAST, id ASC LIMIT 10`.

```
SELECT id, order_date, amount_inr
FROM orders
WHERE order_date >= '2026-05-01'
ORDER BY order_date DESC, amount_inr DESC NULLS LAST, id ASC
LIMIT 10;
```

Q15 · DISTINCT vs GROUP BY — When They Are Identical, When They Diverge

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Walmart Labs · Topic: Aggregation

The question (interviewer's verbatim phrasing):

Is `SELECT DISTINCT user_id FROM orders` the same as `SELECT user_id FROM orders GROUP BY user_id`? When would you choose one over the other?

The 30-second answer: For a single column with no aggregates, they are logically and (usually) physically identical — most optimizers generate the same plan. They diverge the moment you want an aggregate alongside the distinct values — for that, you need GROUP BY. DISTINCT is shorter and clearer for "give me unique rows"; GROUP BY is the right choice when you want unique groups plus a metric per group.

Step-by-step thought process: 1. Begin with the equivalence claim — interviewer is checking whether you mechanically prefer one or actually understand them. 2. Show the divergence: `SELECT user_id, COUNT(*) FROM orders GROUP BY user_id` has no DISTINCT equivalent — DISTINCT cannot compute per-group aggregates. 3. DISTINCT applies to the entire SELECT row, not a single column — `SELECT DISTINCT user_id, order_date` gives unique (user_id, order_date) pairs, not unique user_ids. 4. Performance: both usually trigger the same hash-aggregate or sort-distinct operator. DISTINCT on a non-indexed wide row is just as expensive as the equivalent GROUP BY.

Edge cases to mention: - `SELECT DISTINCT *` is a code smell — almost always indicates a flawed join that duplicated rows; fix the join, not the symptom. - `COUNT(DISTINCT col)` is its own beast — it requires the engine to hold all distinct values in memory or spill to disk; on a wide column at scale this can be costly. - DISTINCT does not allow ORDER BY on a column not in the SELECT list (in strict ANSI) — sometimes a surprise.

What NOT to say: - Do not say "DISTINCT is faster than GROUP BY" or vice versa as a blanket rule — they normally plan the same way. - Do not use DISTINCT to paper over a JOIN that's producing duplicates — debug the JOIN.

Common follow-up: "How would you count unique users per merchant in May 2026?" — `SELECT merchant_id, COUNT(DISTINCT user_id) FROM orders WHERE order_date >= '2026-05-01' AND order_date < '2026-06-01' GROUP BY merchant_id; .`

```
SELECT DISTINCT user_id FROM orders;
-- equivalent
SELECT user_id FROM orders GROUP BY user_id;
-- divergent (DISTINCT cannot do this)
SELECT user_id, COUNT(*) FROM orders GROUP BY user_id;
```

Section D — Pagination, CASE, Type Behaviour

Q16 · LIMIT, OFFSET, FETCH, TOP — Pagination Across Dialects

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Razorpay · Topic: Pagination

The question (interviewer's verbatim phrasing):

Page through orders 10 at a time. Show me the syntax in MySQL, Postgres and SQL Server, and tell me the performance issue with deep OFFSETs.

The 30-second answer: MySQL and Postgres — `LIMIT 10 OFFSET 20` (or `LIMIT 20, 10` in MySQL's older form). SQL Server — `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY` (ANSI

standard, also supported by Postgres 12+). The performance issue with deep OFFSET is that the engine still scans and discards every row before the OFFSET — page 1000 of 10 rows means scanning 10,000 rows then throwing away 9,990. Use keyset pagination (`WHERE id < last_seen_id ORDER BY id DESC LIMIT 10`) for deep paging.

Step-by-step thought process: 1. Map the three dialects — that's the surface answer. 2. Always pair LIMIT with ORDER BY — without a deterministic sort, pages can repeat or skip rows. 3. Explain the OFFSET cost: the engine cannot skip rows for free; it must produce them and discard them. At page 5,000 on a 10-per-page list, that's 50,000 wasted rows per request. 4. Introduce keyset (a.k.a. cursor) pagination as the production-grade alternative — uses the last row's id from the previous page as a starting point.

Edge cases to mention: - Without ORDER BY, LIMIT picks whichever rows the engine happens to produce first — the result is not deterministic and can change run to run. - Postgres OFFSET evaluates the entire underlying query before discarding — index range scans can still help, but the discarded work is real. - SQL Server before 2012 used ROW_NUMBER() OVER + a CTE to paginate — old-school but still in legacy systems.

What NOT to say: - Do not say "OFFSET is free." Deep OFFSETs are a real performance problem and a common Indian product company interview trap. - Do not omit ORDER BY — the page order will be non-deterministic.

Common follow-up: "Show me a keyset pagination query for orders sorted by id descending." — `WHERE id < :last_seen_id ORDER BY id DESC LIMIT 10;` — no OFFSET, scales to billions of rows.

```
-- MySQL / Postgres
SELECT id, amount_inr FROM orders ORDER BY id DESC LIMIT 10 OFFSET 20;
-- SQL Server / Postgres (ANSI)
SELECT id, amount_inr FROM orders ORDER BY id DESC OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
-- Keyset (preferred for deep pages)
SELECT id, amount_inr FROM orders WHERE id < 12345 ORDER BY id DESC LIMIT 10;
```

Q17 · CASE WHEN — Simple vs Searched, in SELECT vs WHERE

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture, JP Morgan, Walmart Labs · Topic: Expressions

The question (interviewer's verbatim phrasing):

Classify each payment as 'small' (under 500), 'medium' (500-5000) or 'large' (above 5000). Write it as a CASE expression and explain the two forms of CASE.

The 30-second answer: Searched CASE — `CASE WHEN cond1 THEN x WHEN cond2 THEN y ELSE z END` — most general; conditions can be anything. Simple CASE — `CASE col WHEN val1 THEN x WHEN val2 THEN y END` — shorter when matching one column against constants. CASE can sit anywhere an expression can: in SELECT, WHERE, ORDER BY, GROUP BY. ELSE is optional; without it, unmatched rows get NULL.

Step-by-step thought process: 1. Distinguish the two forms — that is the academic part of the answer. 2. Show the worked classification: `CASE WHEN amount < 500 THEN 'small' WHEN amount <= 5000 THEN 'medium' ELSE 'large' END`. 3. Mention that conditions are evaluated top-down and the first match wins — order matters when ranges overlap. 4. CASE in WHERE is legal but often less readable than rewriting as OR conditions; CASE in ORDER BY is a clean way to do custom sort orders ('captured' first, then 'pending', then everything else).

Edge cases to mention: - Without ELSE, an unmatched row produces NULL — easy to overlook when downstream code assumes a non-null bucket. - Mixing data types in the THEN branches forces an implicit cast — all branches must be type-compatible. - CASE expressions don't short-circuit boolean evaluation in every engine — be careful with division by zero; some engines evaluate all branches before picking one.

What NOT to say: - Do not say "CASE is only for SELECT" — it works in WHERE, GROUP BY, ORDER BY, HAVING too. - Do not say "ELSE is required" — it is optional; the default for an unmatched row is NULL.

Common follow-up: "How would you pivot a status column into three columns showing the count of each status per merchant?" — Use CASE inside SUM: `SUM(CASE WHEN status = 'captured' THEN 1 ELSE 0 END) AS captured_count`.

```
SELECT id,
       CASE
         WHEN amount_inr < 500   THEN 'small'
         WHEN amount_inr <= 5000 THEN 'medium'
         ELSE                     'large'
       END AS size_bucket
FROM payments;
```

Q18 · NULL in Aggregate Functions, COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Swiggy, JP Morgan, Goldman · Topic: Aggregation

The question (interviewer's verbatim phrasing):

What is the difference between `COUNT(*)`, `COUNT(column_name)` and `COUNT(DISTINCT column_name)`? Which one ignores NULLs?

The 30-second answer: `COUNT()` counts all rows including those with NULL values in any column. `COUNT(col)` counts only rows where `col` is NOT NULL. `COUNT(DISTINCT col)` counts distinct non-NULL values of `col`. The rule across all aggregates: `SUM`, `AVG`, `MIN`, `MAX`, `COUNT(col)` all skip NULLs silently; only `COUNT()` counts NULL rows.

Step-by-step thought process: 1. State the three behaviours crisply — interviewer is timing this answer. 2. Highlight the silent-skip behaviour of aggregates: `AVG(amount)` of a column with NULLs averages only the non-null rows, which can give a misleadingly high average if many rows are NULL. 3. Show the canonical use case: `COUNT(*)` for "how many rows", `COUNT(col)` for "how many had this attribute set", `COUNT(DISTINCT col)` for "how many unique values". 4. Combine with conditional aggregation: `COUNT(CASE WHEN status = 'captured' THEN 1 END)` — only TRUE rows contribute; the implicit ELSE NULL is skipped by COUNT.

Edge cases to mention: - `COUNT()` does not look at any specific column, so the engine can sometimes serve it from the table's row count metadata — very fast in MyISAM, slower in InnoDB and Postgres. - `COUNT(1)` is identical to `COUNT()` in every modern engine — the old myth that `COUNT(1)` is faster is false. - `AVG` over an all-NULL column returns NULL, not 0 — a frequent source of NaN propagation in dashboards.

What NOT to say: - Do not say "`COUNT()` is slower because it counts NULLs." NULL handling is not what makes it slow; what makes it slow is reading every row. - Do not say "`COUNT(col)` and `COUNT()` are the same" — they differ exactly when `col` has NULLs.

Common follow-up: "How would you count payments per merchant, including merchants with zero payments?" — LEFT JOIN merchants to payments, then `COUNT(payments.id)` (which excludes the unmatched-row NULLs) grouped by merchant; `COUNT(*)` would wrongly return 1 for merchants with no payment.

```
SELECT
  COUNT(*)                AS total_rows,
  COUNT(promo_code)       AS rows_with_promo,
  COUNT(DISTINCT promo_code) AS unique_promos
FROM bookings;
```

Q19 · Implicit Conversion Pitfalls and Index Sabotage

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Wells Fargo, Walmart Labs · Topic: Data Types

The question (interviewer's verbatim phrasing):

The query `SELECT * FROM users WHERE phone = 9876543210` is slow on a 50 million row table, even though there's an index on phone. The phone column is VARCHAR(15). Diagnose what is happening.

The 30-second answer: Implicit type conversion is killing the index. The literal `9876543210` is a numeric type; the column is VARCHAR. To compare them, the engine has to cast the VARCHAR column to a number for every single row — which means the index on the original string values is useless, and the engine falls back to a full table scan. Fix: quote the literal as a string — `WHERE phone = '9876543210'`.

Step-by-step thought process: 1. Diagnose the implicit conversion first — that's the headline. 2. Explain why the index becomes useless: indexes store the column's values in their native type. A B-tree on VARCHAR phone numbers cannot be searched by a numeric comparison, because the conversion happens after the index would be consulted. 3. The fix is always to cast the literal, not the column. Cast the literal to the column's type: `WHERE phone = '9876543210'`. Never the other way around. 4. Add the diagnostic step: `EXPLAIN` (or `EXPLAIN ANALYZE`) reveals "type conversion" or "function on column" notes that flag exactly this issue.

Edge cases to mention: - Functions on the indexed column (e.g., `WHERE LOWER(email) = 'x@y.com'`) have the same effect — index unusable unless you have a functional index on `LOWER(email)`. - Collation mismatches between columns being compared can trigger the same silent cast — joining a utf8mb4 column to a latin1 column may scan instead of using the index. - Postgres is stricter about implicit numeric-to-string conversions; MySQL is more permissive but pays the price in lost index use.

What NOT to say: - Do not say "the index is broken; rebuild it." Rebuilding does not help when the query forces a per-row cast. - Do not propose a CAST on the column — that makes the problem worse and explicit.

Common follow-up: "What other patterns silently kill index use?" — `WHERE function(col) = value`, `WHERE col + 1 = value`, `WHERE col LIKE '%foo%'` (leading wildcard), and OR conditions across columns without a compound index covering both. All of these defeat the index until you rewrite the query.

```
-- Slow: implicit cast on every row, full scan
SELECT * FROM users WHERE phone = 9876543210;
-- Fast: matches the indexed VARCHAR value directly
SELECT * FROM users WHERE phone = '9876543210';
```

Q20 · FOREIGN KEY Behaviour — ON DELETE / ON UPDATE Options

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cognizant, Wells Fargo · Topic: Constraints

The question (interviewer's verbatim phrasing):

You have an orders table with a FOREIGN KEY referencing users. The PM asks you to delete a user. What options does the FOREIGN KEY give you, and which one is the most dangerous in production?

The 30-second answer: Five options — RESTRICT (default in most engines, blocks the delete if any child row exists), NO ACTION (similar but deferred until end of statement), CASCADE (deletes all child rows along with the parent), SET NULL (sets the child FK column to NULL, requires the column to be nullable), SET DEFAULT (sets it to the column's default value). CASCADE is the most dangerous in production — one accidental DELETE on a parent can wipe out tens of millions of child rows silently.

Step-by-step thought process: 1. Name all five options crisply — that's the breadth check. 2. Explain RESTRICT vs NO ACTION: practically identical in most engines; technically NO ACTION is deferred to the end of the statement so multi-row updates that resolve before commit are allowed. 3. Walk through the CASCADE risk with a realistic example: deleting a merchant cascades to orders, payments, refunds, audit logs — gone, irreversibly, no warning. 4. Production-grade pattern: prefer RESTRICT, do soft deletes at the application layer (set deleted_at), and explicitly delete child rows in a controlled, audited migration when truly needed.

Edge cases to mention: - ON UPDATE CASCADE is rarely useful because primary keys should be immutable; if you're updating PKs, your data model has a deeper problem. - SET NULL requires the FK column to be nullable — if you have NOT NULL on it, the engine rejects the constraint definition. - Some engines (older MySQL with MyISAM) silently ignore FK constraints — always confirm your storage engine actually enforces them.

What NOT to say: - Do not say "ON DELETE CASCADE is convenient." It's convenient until it isn't, and there's no undo button. - Do not skip explaining the index on the child column — every FK should have an index on the referencing column, otherwise parent updates and deletes do a full scan.

Common follow-up: "A user requests deletion under data-privacy obligations. How do you handle it cleanly?" — Soft delete the user row (set deleted_at, scrub PII columns), keep the FK intact on orders for audit and tax records, document the retention exception. CASCADE would also delete the orders, which you usually need to retain for legal compliance.

```
CREATE TABLE orders (  
    id BIGINT PRIMARY KEY,  
    user_id BIGINT NOT NULL,  
    amount_inr DECIMAL(12,2) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT ON UPDATE RESTRICT  
);  
-- always index the child FK column  
CREATE INDEX idx_orders_user_id ON orders(user_id);
```

Chapter 2, Part A — Joins

Joins are the workhorse of every SQL interview round you'll ever sit in — campus drives, GCC panels, Razorpay-Swiggy-Flipkart product loops, even the surprise screen during the manager round. If you can't reason about joins at row-level cardinality, you don't make it past the first 20 minutes. This part of the chapter walks through 23 join questions in the order they actually get asked — starting from the four basic shapes, moving through self-joins, anti-joins, lateral joins and inequality joins, and finishing with the operator-level questions (hash vs merge vs nested-loop) and large-table patterns (partition-wise joins, star schema). Part B picks up where this ends and covers subqueries, CTEs and correlated logic — read them together if you can.

Q21 · INNER vs LEFT vs RIGHT vs FULL OUTER — Row-Level Differences

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture · Topic: Joins

The question (interviewer's verbatim phrasing):

Walk me through what INNER JOIN, LEFT JOIN, RIGHT JOIN and FULL OUTER JOIN return. Use a real example and tell me which rows would be different across the four.

The 30-second answer: INNER returns only rows that match on both sides; LEFT returns every left row plus matched right rows (NULLs where no match); RIGHT is the mirror of LEFT; FULL OUTER returns everything from both sides padded with NULLs on unmatched cells. Pick LEFT for "all of A, plus B if present" — that's 80% of real-world cases.

Step-by-step thought process: 1. Anchor with two tables — `orders` (every order placed) and `payments` (only orders that triggered a payment intent). This is a realistic Indian fintech setup where some orders never reach checkout. 2. INNER between these gives you orders that actually have a payment row — useful for revenue reports. 3. LEFT gives you all orders, with NULLs in payment columns for the ones that abandoned — that's how you build a funnel. 4. RIGHT gives you all payment rows,

including orphans where the order row was deleted — that's a data-integrity check. 5. FULL OUTER returns both sets of orphans simultaneously — rare in production, useful in reconciliation. 6. Mention that RIGHT JOIN is almost never written by hand — most engineers flip table order and use LEFT for readability.

Edge cases to mention: - If the right side has duplicates (one order, three payment attempts), even INNER and LEFT will multiply the left row — junior candidates miss this. - FULL OUTER is unsupported in MySQL until 8.0.31 — you simulate with UNION of LEFT and RIGHT JOIN. - NULL never equals NULL in a join condition — that's standard three-valued logic.

What NOT to say: - Do not say "LEFT JOIN returns all rows from both tables" — that's wrong, only the left table is preserved. - Do not draw a Venn diagram as your only explanation; the interviewer wants row-level reasoning.

Common follow-up: "Show me orders that were placed but never paid." — Classic anti-join: LEFT JOIN payments and filter WHERE p.order_id IS NULL .

```
-- Funnel: every order, plus payment status if any
SELECT o.id, o.user_id, o.amount, p.status AS payment_status
FROM orders o
LEFT JOIN payments p ON o.id = p.order_id;
```

Q22 · Self-Join — Manager and Employee Hierarchy

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, TCS, Wells Fargo · Topic: Joins

The question (interviewer's verbatim phrasing):

You've got an employees table with id , name , manager_id where manager_id is a self-reference. Write a query that lists every employee with their manager's name. Don't drop the CEO.

The 30-second answer: Alias the table twice — e for employee, m for manager — and write LEFT JOIN employees m ON e.manager_id = m.id . LEFT keeps the CEO row (whose manager_id is NULL) in the result. Use COALESCE(m.name, 'No manager') for clean display.

Step-by-step thought process: 1. Open with the core idea — a self-join is just a join between two aliases of the same table. 2. Justify LEFT over INNER: the CEO has no manager, so INNER would silently drop the topmost row. 3. Mention that the same pattern handles "find pairs of employees sharing a manager" — just swap the join condition to e1.manager_id = e2.manager_id AND e1.id < e2.id . 4. Flag that multi-level hierarchy ("manager's manager's manager") is not a self-join problem — it needs a recursive CTE, covered in Part B.

Edge cases to mention: - Cycles: if A reports to B and B reports to A (data corruption), a recursive walk loops forever — always cap depth. - Matrix orgs where one employee has two managers will produce two rows per employee — that's the right cardinality, but be ready to explain it. - If `manager_id` references a soft-deleted employee, the LEFT JOIN gives you NULL for manager name — handle with `COALESCE`.

What NOT to say: - Do not use the same alias on both sides — the query won't parse and you'll look careless. - Do not say "self-join is a special kind of join" — it's just a regular join with the same physical table referenced twice.

Common follow-up: "Now show me three levels — employee, manager, manager's manager." — Either chain self-joins explicitly or, cleaner, use a recursive CTE.

```
SELECT e.name AS employee,  
       COALESCE(m.name, 'No manager') AS manager  
FROM employees e  
LEFT JOIN employees m ON e.manager_id = m.id;
```

Q23 · Cross Join — Intentional vs Accidental Cardinality Blow-up

Tags: Difficulty: Easy · Asked at: TCS, Capgemini, HCL, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

What is a cross join and when have you used it intentionally? And how does an accidental cross join happen?

The 30-second answer: A cross join (Cartesian product) returns every row of A paired with every row of B — 1000×1000 gives you a million rows. Intentional use: generating date spines, calendar tables, or all combinations for a pricing matrix. Accidental: forgetting the ON clause or writing `FROM a, b` without a WHERE join condition.

Step-by-step thought process: 1. Define the operator first — explicit `CROSS JOIN` or the older comma-separated `FROM a, b` syntax with no WHERE. 2. Give one genuinely useful example: building a date-range $\times$ user-id grid before a LEFT JOIN to fact data, so empty days show up as zero instead of vanishing. 3. Then the disaster case: a junior dev writes `FROM orders, payments WHERE status = 'paid'` and forgets `o.id = p.order_id` — the query runs, returns 50 million rows, and OOMs the box. 4. Mention that most modern databases will fire a warning or refuse the implicit cross join in strict mode.

Edge cases to mention: - Cross join with a tiny tally table (1 to 100) is a common trick for splitting comma-separated values into rows. - `CROSS JOIN LATERAL` (covered later) is technically a cross join

with per-row context, not a flat Cartesian product. - If one side is empty, the result is empty — even if the other side has a million rows.

What NOT to say: - Do not say "cross join is bad and never used" — that's wrong, it's deliberate for date spines and combinatorics. - Do not say "it's the same as INNER JOIN with no condition" — true mechanically, but interviewers want the operator name.

Common follow-up: "How do you spot an accidental cross join in a slow query?" — Look at the planner output for "Nested Loop" with no join condition, or estimated rows that look suspiciously close to `rows_a × rows_b`.

```
-- Intentional: every day in May × every active merchant, for daily revenue grid
SELECT d.day, m.id AS merchant_id
FROM generate_series('2026-05-01'::date, '2026-05-31'::date, '1 day') AS d(day)
CROSS JOIN merchants m
WHERE m.status = 'active';
```

Q24 · Anti-Join — Three Ways to Find "Not in B", and the NOT IN Trap

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Goldman, JP Morgan India · Topic: Joins

The question (interviewer's verbatim phrasing):

Find users who have never placed an order. Show me three ways to write it and tell me which is safest.

The 30-second answer: Three forms — `LEFT JOIN orders ... WHERE order_id IS NULL`, `NOT EXISTS (...)`, and `NOT IN (...)`. The first two are safe. `NOT IN` is dangerous: if the subquery returns even one NULL, the entire result becomes empty because `x != NULL` evaluates to UNKNOWN.

Step-by-step thought process: 1. Open with the three forms — interviewer wants to see you know all of them. 2. Explain the NOT IN trap precisely: `user_id NOT IN (1, 2, NULL)` desugars to `user_id != 1 AND user_id != 2 AND user_id != NULL`. The last conjunct is UNKNOWN, so the whole row is UNKNOWN, not TRUE, so it's filtered out. 3. State your preference clearly — `NOT EXISTS`. It handles NULLs correctly, and Postgres, SQL Server and modern MySQL all rewrite it as an anti-join operator. 4. Mention when LEFT JOIN ... IS NULL wins: when you also need columns from the left table and want a single pass.

Edge cases to mention: - If the joined column is declared `NOT NULL`, `NOT IN` is technically safe — but don't trust schema you can't audit. - `NOT EXISTS` short-circuits on the first match per outer row

— usually faster on huge right tables. - Adding `DISTINCT` to fix NOT IN doesn't help — the NULL still poisons the comparison.

What NOT to say: - Do not claim "all three are equivalent" — they are not, under NULLs. - Do not say one is always fastest — depends on cardinality and indexes on the join column.

Common follow-up: "What does the planner do with NOT EXISTS?" — Postgres and modern SQL Server convert it to an "Anti Join" operator, which is a join that emits the left row only when no right match exists.

```
SELECT u.id, u.name
FROM users u
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

Q25 · Semi-Join — Why EXISTS Is the Right Expression

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Cred, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

What is a semi-join? Why is EXISTS the most natural way to express it, and when would a JOIN+DISTINCT be wrong?

The 30-second answer: A semi-join returns one row per outer match — "does at least one matching row exist on the right?" — without multiplying rows. `EXISTS` expresses this directly. `JOIN + DISTINCT` simulates it but is fragile: if you later add a column from the right table, the DISTINCT silently changes meaning.

Step-by-step thought process: 1. Lead with the definition — semi-join is "filter outer by existence on inner", never duplicate outer rows. 2. Concrete example: users who have made at least one successful payment.

`EXISTS (SELECT 1 FROM payments p WHERE p.user_id = u.id AND p.status = 'success')` returns each user exactly once. 3. Contrast with JOIN:

`JOIN payments ON ... WHERE status = 'success'` returns one row per successful payment per user — a user with 5 payments shows up 5 times. Adding DISTINCT fixes the count but loses ordering guarantees and adds a sort. 4. Note the operator-level point: Postgres and SQL Server show this as "Semi Join" in EXPLAIN — it's a first-class operator, not a hack.

Edge cases to mention: - Correlated EXISTS with a covering index on the right table's join column is one of the fastest patterns in SQL. - `IN (subquery)` is rewritten as a semi-join by modern optimizers

— usually equivalent to EXISTS in plan. - If the inner query has aggregation, semi-join semantics still hold — `EXISTS (SELECT SUM(...) FROM ...)` is always true if any row exists, because aggregates over empty sets return one row of NULLs.

What NOT to say: - Do not use `SELECT *` inside EXISTS — it works but `SELECT 1` reads as "I only care that a row exists". - Do not write `JOIN ... GROUP BY` to dedupe when EXISTS is the intent — it's slower and harder to read.

Common follow-up: "Why does `SELECT 1` inside EXISTS not matter for performance?" — Because the optimizer only checks for row existence, never materializes the projection.

```
-- Users with at least one successful payment, no duplicates
SELECT u.id, u.name
FROM users u
WHERE EXISTS (
  SELECT 1 FROM payments p
  WHERE p.user_id = u.id AND p.status = 'success'
);
```

Q26 · LATERAL Join — Per-Row Subquery (CROSS APPLY in SQL Server)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, Goldman · Topic: Joins

The question (interviewer's verbatim phrasing):

Tell me what a LATERAL join does and give an example where you couldn't get the same result with a plain JOIN.

The 30-second answer: LATERAL (or `CROSS APPLY` in SQL Server) lets the right side of a join reference columns from the left side — effectively a correlated subquery in the FROM clause. Use it for "top-N per group" queries — e.g., the 3 most recent orders per user — which are clumsy or impossible with plain joins.

Step-by-step thought process: 1. Anchor the syntax: `FROM users u, LATERAL (SELECT ... FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3) o` in Postgres; `OUTER APPLY` / `CROSS APPLY` in SQL Server. 2. Explain why a plain join can't do this: in a plain JOIN, you can't say `LIMIT 3 per left row` — you'd need a window function with `ROW_NUMBER() ... PARTITION BY user_id` and an outer filter. 3. The LATERAL form is often clearer and sometimes faster because it leverages indexed scans on the inner table per outer row. 4. Use `LEFT JOIN LATERAL` (or `OUTER APPLY`) to preserve outer rows that have no inner match.

Edge cases to mention: - LATERAL is in standard SQL since SQL:1999 but only became widely supported around 2014 (Postgres 9.3, SQL Server has had APPLY since 2005). - MySQL added

LATERAL in 8.0.14 — older versions can't do this directly. - Performance depends on having an index on `(user_id, created_at DESC)` on the inner table; without it, you scan the whole table per outer row.

What NOT to say: - Do not call it "a fancy subquery" — interviewers want the term "correlated derived table" or "per-row join". - Do not say "you can always rewrite LATERAL as a window function" — sometimes true, but window functions can't return arbitrary derived columns from a per-row computation.

Common follow-up: "What's the difference between *CROSS APPLY* and *OUTER APPLY* in SQL Server?" — *CROSS APPLY* is *INNER* (drops left rows with no inner match), *OUTER APPLY* is *LEFT* (keeps them with NULLs).

```
-- 3 most recent orders per user, Postgres syntax
SELECT u.id, u.name, o.order_id, o.created_at
FROM users u
LEFT JOIN LATERAL (
  SELECT order_id, created_at
  FROM orders
  WHERE user_id = u.id
  ORDER BY created_at DESC
  LIMIT 3
) o ON true;
```

Q27 · Does Multi-Table Join Order Matter?

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, Target India, AmEx · Topic: Joins

The question (interviewer's verbatim phrasing):

If I write `A JOIN B JOIN C`, does the order I write them in matter for performance? What does the optimizer actually do?

The 30-second answer: For *INNER* joins, written order does not matter — the optimizer reorders freely based on statistics. For *OUTER* joins, order matters because *LEFT JOIN* is not commutative. Modern optimizers consider many join shapes (left-deep, right-deep, bushy) and pick the cheapest based on cardinality estimates.

Step-by-step thought process: 1. Start with the inner-vs-outer distinction — that's the heart of the question. 2. For inner joins: the optimizer treats `A JOIN B JOIN C` as a set of join predicates and chooses the cheapest order — often starting with the most selective filter. 3. For outer joins: `(A LEFT JOIN B) LEFT JOIN C` is not the same as `A LEFT JOIN (B LEFT JOIN C)` because *LEFT JOIN* preserves the outer table, and the inner of the second join behaves differently. 4. Mention the

practical limit: Postgres explores all permutations up to `join_collapse_limit` (default 8); beyond that, it follows the written order. Same idea in other engines with different knob names.

Edge cases to mention: - Bushy plans (join two pairs in parallel) are sometimes optimal but most engines lean toward left-deep trees for simplicity. - Hints like `STRAIGHT_JOIN` in MySQL or `LEADING(...)` in Oracle force a specific order — last resort when stats are bad. - A missing or out-of-date `ANALYZE` can lead the optimizer to pick a terrible order even when the right one is obvious.

What NOT to say: - Do not say "order always matters" — that's true only for outer joins. - Do not say "the planner always picks the best plan" — it picks the best plan according to its cost model and stats, which can be wrong.

Common follow-up: "How would you debug a query where you suspect the join order is wrong?" — Run `EXPLAIN ANALYZE`, compare estimated vs actual rows at each join, and update stats with `ANALYZE` if estimates are off by an order of magnitude.

```
-- Optimizer picks the cheapest order; written order is just a starting point
SELECT u.name, o.id, p.status
FROM users u
JOIN orders o ON o.user_id = u.id
JOIN payments p ON p.order_id = o.id
WHERE u.region = 'south' AND o.created_at > CURRENT_DATE - INTERVAL '7 day';
```

Q28 · Join on Inequality — Range Joins and BETWEEN Joins

Tags: Difficulty: Medium · Asked at: Swiggy, Zomato, Cred, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

When would you join two tables on an inequality, like `BETWEEN` or `<` ? What's the performance impact?

The 30-second answer: Inequality joins are used for range matching — slabs (income tax brackets), effective-dated dimension lookups, "events that happened during a session". They're slower than equality joins because hash join doesn't apply; the planner falls back to nested loop or merge join with range scans. Indexing the range bounds helps a lot.

Step-by-step thought process: 1. Anchor with a concrete example — joining a `transactions` table to a `gst_slabs` table on `txn_amount BETWEEN slab_min AND slab_max` to determine the GST rate applicable. 2. Explain the operator-level cost: hash join needs equality, so it can't be used. The planner picks nested loop (cheap if the right table is small and indexed) or merge join with a range condition. 3. Mention covering indexes on `(slab_min, slab_max)` — they help, but range overlap is still $O(N)$ per probe in the worst case. 4. Effective-dated joins: `JOIN dim_product p ON`

`f.product_id = p.product_id AND f.event_date BETWEEN p.valid_from AND p.valid_to` is the standard Slowly Changing Dimension Type 2 lookup pattern.

Edge cases to mention: - Postgres GiST/SP-GiST indexes on `range` types make inequality joins much faster than B-tree on min/max. - If ranges overlap, you can get more than one match per outer row — verify cardinality with a quick COUNT. - Joining `created_at BETWEEN start AND end` without an index is a classic $O(N \times M)$ disaster on big tables.

What NOT to say: - Do not say "inequality joins are always slow" — with the right index they can be perfectly fine for modest sizes. - Do not say "use a subquery instead" — for range lookups, the inequality join is the cleanest expression.

Common follow-up: "Show me an effective-dated lookup." — Standard SCD2 pattern; verify each fact row matches exactly one dimension row.

```
-- Tag each transaction with the GST slab it falls into
SELECT t.id, t.amount, s.gst_rate
FROM transactions t
JOIN gst_slabs s
  ON t.amount >= s.slabs_min AND t.amount < s.slabs_max;
```

Q29 · USING vs ON — Semantic Differences

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wells Fargo · Topic: Joins

The question (interviewer's verbatim phrasing):

What's the difference between `JOIN ... USING (col)` and `JOIN ... ON a.col = b.col`? When would you prefer USING?

The 30-second answer: USING is shorthand when both tables have a column with the same name — it merges them into a single output column. ON is general — it can join on different column names or expressions. USING is cleaner for simple equality joins on identically-named keys; ON is necessary everywhere else.

Step-by-step thought process: 1. Start with the syntax — `JOIN orders o USING (user_id)` vs `JOIN orders o ON u.user_id = o.user_id`. 2. The big semantic difference: USING produces one merged column in the output (you write `SELECT user_id`, not `u.user_id`); ON keeps both columns visible. 3. With `SELECT *`, USING shows the joined column once; ON shows it twice (once per table) — that's the giveaway in interview screens. 4. Mention that USING only works when column names match exactly — different names (e.g., `id` vs `user_id`) force you to use ON.

Edge cases to mention: - USING with NATURAL JOIN sounds similar but NATURAL auto-joins on all same-name columns — covered next, and almost always a footgun. - USING is supported across Postgres, MySQL, Oracle, SQLite — but is not supported in SQL Server (which forces you to use ON). - You can chain USING in a multi-table join: `a JOIN b USING (k) JOIN c USING (k)` is valid if all three have `k`.

What NOT to say: - Do not say "they are identical" — output column shape differs. - Do not use USING on a column whose semantics differ between tables (e.g., `id` meaning `user_id` in one, `order_id` in another) — even if names match, the join is wrong.

Common follow-up: *"In an ORM-generated query, why do you see ON everywhere and rarely USING?"* — ORMs generate explicit qualified column names because they don't assume schema symmetry — defensive code.

```
-- USING merges the user_id column into one
SELECT user_id, u.name, o.id
FROM users u
JOIN orders o USING (user_id);
```

Q30 · NATURAL JOIN — Why You Almost Never Use It

Tags: Difficulty: Easy · Asked at: TCS, Wipro, HCL, Capgemini · Topic: Joins

The question (interviewer's verbatim phrasing):

What is a NATURAL JOIN and why do most senior engineers tell you not to use it?

The 30-second answer: NATURAL JOIN auto-joins on every column with the same name across both tables. It's a footgun because adding a new same-named column to either table (audit columns like `created_at`, `updated_by`) silently changes the join condition and breaks the query.

Step-by-step thought process: 1. Define it precisely — `a NATURAL JOIN b` finds all columns with identical names in both tables and joins on equality across all of them. 2. The disaster scenario: you have `users(id, name)` and `orders(id, user_id, ...)`. A NATURAL JOIN matches on `id` — but `id` in `users` is the user id, and `id` in `orders` is the order id. You get an empty result that looks intentional. 3. Even when it works initially, a schema migration adds `created_at` to both tables, and now NATURAL JOIN suddenly joins on `(id, created_at)` — completely silent breakage. 4. Recommend USING or ON every time — explicit beats implicit.

Edge cases to mention: - Some engines (Oracle) used to encourage NATURAL JOIN in tutorials; production code rarely uses it. - If you must use it, always pair with `SELECT` of explicit columns so you

can sanity-check the output shape. - NATURAL JOIN does not appear in any well-known production codebase you'd respect — that itself is the answer.

What NOT to say: - Do not say "NATURAL JOIN is faster" — it isn't, it's just shorter syntax. - Do not defend it on grounds of brevity — the silent-breakage cost is far higher than the keystroke savings.

Common follow-up: "Can you think of any case where NATURAL JOIN is genuinely useful?" — Almost no production case. Maybe one-off REPL exploration on a tiny well-known schema.

```
-- The cautionary tale — DO NOT do this in production
SELECT *
FROM users
NATURAL JOIN orders;
-- Implicitly joins on every same-named column. Add an audit column later, query silently breaks.
```

Q31 · Self-Join with a Composite Key — Anti-Pattern Detection

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, Goldman, JP Morgan · Topic: Joins

The question (interviewer's verbatim phrasing):

Suppose you self-join a `transactions` table on a composite key like `(merchant_id, txn_date)` to find consecutive transactions per merchant. What's the anti-pattern here and how do you avoid it?

The 30-second answer: Self-joining on `(merchant_id, txn_date)` with a `t1.txn_date = t2.txn_date - INTERVAL '1 day'` style condition works but explodes if multiple transactions exist per merchant per day. Use a window function (`LAG` over `PARTITION BY merchant_id ORDER BY txn_date`) — one pass, deterministic, no cardinality surprise.

Step-by-step thought process: 1. Explain why people reach for self-join: they want "current row vs previous row" and self-join feels intuitive. 2. The cardinality trap: if merchant M has 3 transactions on May 10 and 5 transactions on May 11, the self-join produces $3 \times 5 = 15$ pair rows for that merchant-day pair — usually not what you want. 3. The correct tool is `LAG` / `LEAD` window functions — they compute "previous row" per partition with no row multiplication. 4. Self-join is still the right tool for "find pairs where condition X" — e.g., finding overlapping bookings — where you genuinely need pairs.

Edge cases to mention: - If you must self-join, add an inequality like `t1.txn_id < t2.txn_id` to avoid duplicate `(A,B)` and `(B,A)` pairs. - Window functions need a deterministic `ORDER BY` to define "previous" — if `txn_date` is not unique within partition, add a tiebreaker like `txn_id`. - On giant tables, `LAG` is usually cheaper than self-join because it's one sort, not a join.

What NOT to say: - Do not say "self-join is always wrong" — it's the right tool for pair-finding problems.
- Do not skip the dedup condition on self-join pairs — duplicates will silently inflate any aggregation downstream.

Common follow-up: "Now rewrite this as a window function and explain why it's faster." — One sort instead of join + sort; no row multiplication.

```
-- Anti-pattern: self-join, risks row multiplication per merchant-date
SELECT t1.merchant_id, t1.amount AS today_amt, t2.amount AS yesterday_amt
FROM transactions t1
JOIN transactions t2
  ON t1.merchant_id = t2.merchant_id
 AND t1.txn_date = t2.txn_date + INTERVAL '1 day';

-- Correct: window function, single pass
SELECT merchant_id, txn_date, amount,
       LAG(amount) OVER (PARTITION BY merchant_id ORDER BY txn_date) AS prev_amount
FROM transactions;
```

Q32 · Joining on Functions — Index Implications

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Swiggy, Cred · Topic: Joins

The question (interviewer's verbatim phrasing):

If I join two tables on `UPPER(a.email) = UPPER(b.email)` or `COALESCE(a.merchant_id, b.merchant_id)`, what happens to my indexes?

The 30-second answer: Plain B-tree indexes on `a.email` are not used when the query applies a function to the column — the index is on raw values, not on `UPPER(email)`. Either store the canonical form in a generated column, or create a functional/expression index on `UPPER(email)`. `COALESCE` in a join condition is even worse — it usually defeats any index entirely.

Step-by-step thought process: 1. Start with the index mechanics — B-tree indexes order rows by the raw column value, so `UPPER(email) = UPPER(?)` can't binary-search them. 2. Two fixes: (a) functional index — `CREATE INDEX idx_email_upper ON users (UPPER(email))` — Postgres, Oracle, MySQL 8 all support this; (b) generated column — `email_normalized AS UPPER(email)` plus a regular index on it. 3. `COALESCE` in a join is harder — the optimizer can't push the predicate through, so it usually full-scans. Restructure with `UNION ALL` of two joins, one per branch, often cleaner. 4. Indian-context callout: case-insensitive email matching is the most common real reason to do this — and the most common cause of slow login queries.

Edge cases to mention: - `CITEXT` in Postgres is a case-insensitive text type — store emails as `CITEXT` and you avoid the function entirely. - Collation-based case-insensitive matching (`CI` collations in SQL Server / MySQL) avoids the function call but has its own gotchas. - Functional indexes have to match the function exactly — `UPPER(email)` index won't help `LOWER(email)` queries.

What NOT to say: - Do not say "indexes always work" — they don't, when functions are applied. - Do not silently apply functions in joins and assume the planner figures it out — it usually can't.

Common follow-up: *"How would you handle case-insensitive joins on email at scale?"* — `CITEXT` in Postgres, or a stored normalized column with a regular index.

```
-- Slow: index on email is unused
SELECT *
FROM users u
JOIN merchants m ON UPPER(u.email) = UPPER(m.contact_email);

-- Fast: functional index makes the join sargable
CREATE INDEX idx_users_email_upper ON users (UPPER(email));
CREATE INDEX idx_merchants_email_upper ON merchants (UPPER(contact_email));
```

Q33 · 3-Way and 4-Way Joins — Readability and Planner Behaviour

Tags: Difficulty: Medium · Asked at: TCS, Cognizant, Walmart Labs, Goldman · Topic: Joins

The question (interviewer's verbatim phrasing):

Write a 4-way join across users, orders, payments and refunds. What does the planner do, and how do you keep this readable?

The 30-second answer: Write the joins in logical-flow order (users → orders → payments → refunds), alias every table with a short consistent name (`u` , `o` , `p` , `r`), put filters in `WHERE` for clarity, and let the optimizer reorder. For readability, one join per line with the `ON` condition on the same line. The planner picks join order independently of how you wrote them (for inner joins).

Step-by-step thought process: 1. Show the query structure — every join on its own line, short aliases. 2. Anchor on the planner: for inner joins, written order is just a hint; the optimizer reorders based on stats. For outer joins, order is semantic — don't rely on the optimizer to rescue you. 3. Mention the readability convention: filters that reduce a single table go in `WHERE`; filters that preserve outer-join semantics go in `ON`. 4. Watch the cardinality — a 4-way join where any link is one-to-many can produce surprising row counts. Sanity-check with `COUNT(*)` against expected.

Edge cases to mention: - If you cross `join_collapse_limit` in Postgres (default 8), the planner stops exploring and uses written order — for big joins, written order suddenly matters. -

INFORMATION_SCHEMA queries that join 5+ system views can be terribly slow because stats are missing — sometimes you have to hand-order them. - A 4-way join with **SELECT *** returns a wide result — be explicit about columns to avoid surprises.

What NOT to say: - Do not write **FROM a, b, c, d WHERE ...** — old-style implicit joins; reviewers will reject it. - Do not omit aliases — **users.id** repeated 5 times kills readability.

Common follow-up: "What if you have a filter that applies after the 3rd join — should it go in ON or WHERE?" — In WHERE if it's an inner join; in ON if you must preserve outer-join NULLs.

```
SELECT u.id, u.name, o.id AS order_id, p.amount AS paid_amt, r.amount AS refund_amt
FROM users u
JOIN orders o ON o.user_id = u.id
LEFT JOIN payments p ON p.order_id = o.id
LEFT JOIN refunds r ON r.payment_id = p.id
WHERE u.region = 'west'
      AND o.created_at > CURRENT_DATE - INTERVAL '30 day';
```

Q34 · Outer Join with WHERE on the Outer Table — The Classic Trap

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

Look at this query — **users u LEFT JOIN orders o ON o.user_id = u.id WHERE o.status = 'paid'**. The candidate says it returns all users plus their paid orders. Are they right?

The 30-second answer: No. Putting **o.status = 'paid'** in WHERE silently converts the LEFT JOIN into an INNER JOIN, because NULL fails the equality and gets filtered out. The fix is to move the predicate into the ON clause: **ON o.user_id = u.id AND o.status = 'paid'**. Now LEFT semantics are preserved.

Step-by-step thought process: 1. State the rule crisply — any predicate on the outer table in WHERE silently kills outer-join semantics. 2. Walk through what happens row by row: LEFT JOIN produces NULL **status** for users with no orders. WHERE **status = 'paid'** returns UNKNOWN for NULL, so those rows are dropped. 3. The fix: move the filter to ON so it's evaluated during the join, not after. Now unmatched rows still get NULL on the right and are preserved by LEFT semantics. 4. Mention this is the single most common interview trap in the join section — interviewers ask it precisely because juniors fall for it constantly.

Edge cases to mention: - **WHERE o.status = 'paid' OR o.status IS NULL** is another fix — explicit, but less elegant than moving to ON. - The same trap applies to RIGHT JOIN with a predicate on

the left table. - If you genuinely want INNER semantics, leave the WHERE — don't use LEFT JOIN — be explicit about intent.

What NOT to say: - Do not say "this is a bug in SQL" — it's standard semantics; the candidate just doesn't know it. - Do not say "always put predicates in ON" — predicates on the *outer* (preserved) table belong in WHERE; only predicates on the *inner* table need to be in ON.

Common follow-up: "Now show me the correct query that returns all users plus their paid orders, with NULL for users with no paid order." — Just move the predicate to ON.

```
-- Wrong: silently becomes INNER JOIN
SELECT u.id, o.id, o.status
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid';

-- Right: filter in ON preserves LEFT semantics
SELECT u.id, o.id, o.status
FROM users u
LEFT JOIN orders o ON o.user_id = u.id AND o.status = 'paid';
```

Q35 · Hash Join vs Nested-Loop vs Merge Join — What and When

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan India, Target · Topic: Joins

The question (interviewer's verbatim phrasing):

Tell me what hash join, nested-loop join and merge join are. When does the optimizer pick each one?

The 30-second answer: Nested-loop scans the outer, probing the inner per row — best when outer is small and inner has an index. Hash join builds an in-memory hash on the smaller side and probes from the larger side — best when neither has a useful index but both fit in memory. Merge join sorts both sides on the join key and walks them in lock-step — best when both sides are already sorted (clustered index or pre-sorted output).

Step-by-step thought process: 1. Describe each algorithm in one sentence — the operator names are interview vocabulary. 2. Nested-loop's cost is $O(N \times M)$ without index, $O(N \times \log M)$ with an index on the inner — picked when outer is tiny or when inner has a sargable index. 3. Hash join is $O(N+M)$ but needs memory to hold the build side — picked for big equi-joins. Falls back to disk if memory is exhausted (Postgres "Hash Batches > 1"). 4. Merge join needs both sides sorted — picked when the planner sees pre-sorted input (clustered index, ORDER BY in subquery) or estimates the sort cost is cheaper than building a hash table.

Edge cases to mention: - Hash join only works for equality predicates — inequality joins fall back to nested-loop or merge. - MySQL only added hash join in 8.0.18 — older versions used nested-loop everywhere, which is why joins on big tables were so slow. - "Hash join with batch spill" means your hash didn't fit in `work_mem` ; raise it for that query or shrink the build side.

What NOT to say: - Do not say "hash join is always fastest" — for small outer with indexed inner, nested-loop wins easily. - Do not say "merge join needs sorted input always" — it can also work on indexed scans that produce sorted output for free.

Common follow-up: "How do you check which join the planner picked?" — `EXPLAIN ANALYZE` in Postgres, `EXPLAIN FORMAT=TREE` in MySQL 8, the graphical plan in SQL Server.

```
-- Inspect the chosen join algorithm
EXPLAIN ANALYZE
SELECT u.name, o.id
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.created_at > CURRENT_DATE - INTERVAL '1 day';
```

Q36 · Joining Huge Tables — Partition-Wise Joins

Tags: Difficulty: Hard · Asked at: Walmart Labs, Target India, Goldman, JP Morgan India · Topic: Joins

The question (interviewer's verbatim phrasing):

I have a `transactions` table with 5 billion rows partitioned by month and a `merchants` table with 50 million rows partitioned by region. How do I make joins fast?

The 30-second answer: Use partition-wise joins — when both tables are partitioned on the join key (or a related key), the planner joins partition-to-partition independently and in parallel, instead of building one giant hash table. In Postgres, enable `enable_partitionwise_join = on` . Make sure partition keys align and stats are fresh on every partition.

Step-by-step thought process: 1. Frame the problem — 5 billion × 50 million is hopeless as a single hash join; you need to split the work. 2. Postgres declarative partitioning (since 10.x): partition both tables on the same key (say `merchant_id` hash partitions) so partition `i` of transactions joins only with partition `i` of merchants. 3. Enable `enable_partitionwise_join` and `enable_partitionwise_aggregate` — both are off by default in older versions. 4. With Postgres parallel query, each partition pair can be joined by a worker in parallel — near-linear scale-out on a multi-core box.

Edge cases to mention: - Partition pruning happens first — if your WHERE clause filters to one month, only that partition pair joins. - Misaligned partition schemes defeat partition-wise joins — both tables

must share the partitioning strategy on the join key. - Stats are per-partition — **ANALYZE** the partitioned table to refresh all child partition stats.

What NOT to say: - Do not say "just add an index" — for billions of rows, an index alone won't save you; partitioning is the structural fix. - Do not claim partitioning is always a win — it adds complexity to schema migrations, backups and inserts.

Common follow-up: "What if the merchants table isn't partitioned? Can we still get any benefit?" — Postgres can still parallel-scan the transactions table and broadcast merchants to each worker — slower than partition-wise but better than serial.

```
-- Postgres declarative partitioning (range on month for transactions)
CREATE TABLE transactions (id BIGINT, merchant_id INT, txn_date DATE, amount NUMERIC)
PARTITION BY RANGE (txn_date);

CREATE TABLE transactions_2026_05
PARTITION OF transactions FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

-- Enable per-session
SET enable_partitionwise_join = on;
```

Q37 · Joining with Composite Indexes

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

I have a composite index on **(merchant_id, txn_date)**. When does the planner use it for a join, and when does it skip the index?

The 30-second answer: The composite index is used when the join condition references the leading columns in order — **merchant_id = ?** alone uses it, **merchant_id = ? AND txn_date BETWEEN ...** uses both. A predicate on **txn_date** alone, without **merchant_id**, can't seek the index — most engines won't use it at all (Postgres might choose an index-only scan if it covers the query).

Step-by-step thought process: 1. Start with the leftmost-prefix rule — composite B-tree indexes are like a phone book sorted by (last name, first name); you can search by last name alone, but not by first name alone. 2. Apply to joins: **JOIN ... ON t.merchant_id = m.id** uses the index for the seek. Add **AND t.txn_date > ...** and the index narrows the range further. 3. Index-only scans (covering index): if all columns the query needs are in the index, Postgres can skip the heap fetch — massive speed-up for read-heavy queries. 4. Order matters: build the index with the high-selectivity column first if your queries always filter by it.

Edge cases to mention: - `WHERE txn_date > ... AND merchant_id = ?` is still seekable — the engine reorders predicates. - Skip-scan (Oracle, MySQL 8.0, partial in Postgres 17) can sometimes use the leading column even when the query doesn't filter on it — but don't rely on it. - Composite index on `(a, b)` does NOT serve queries that join on `b` alone — that needs its own index on `b`.

What NOT to say: - Do not say "composite index speeds up everything that touches those columns" — only with leftmost-prefix. - Do not over-index — every extra index slows writes, and unused indexes are dead weight.

Common follow-up: "What order should the columns be in?" — Leftmost column should be the one used in equality predicates by most queries; range columns later.

```
-- Composite index supports the join + date range
CREATE INDEX idx_txn_merchant_date ON transactions (merchant_id, txn_date);

SELECT t.id, t.amount
FROM merchants m
JOIN transactions t ON t.merchant_id = m.id
WHERE t.txn_date >= '2026-05-01';
```

Q38 · Joining Temporal/Range Data — Effective Dating

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, AmEx, Target · Topic: Joins

The question (interviewer's verbatim phrasing):

I have a fact table `payments` with `paid_at` timestamp and a dimension `merchant_dim` with `valid_from`, `valid_to` (slowly-changing dimension type 2). Join them so each payment gets the merchant attributes that were in effect at `paid_at`.

The 30-second answer: Join on `merchant_id` and the temporal range — `JOIN merchant_dim m ON p.merchant_id = m.merchant_id AND p.paid_at >= m.valid_from AND p.paid_at < m.valid_to`. Build an index on `(merchant_id, valid_from, valid_to)` and ensure dim records are non-overlapping per merchant. This is the standard SCD2 join.

Step-by-step thought process: 1. Explain SCD2: each row in the dimension represents one version of merchant attributes, with valid-from / valid-to defining its window. The "current" version has `valid_to = '9999-12-31'` or NULL. 2. The join uses inequality on the time bounds — falls back to nested-loop or merge unless you have a smart range index. 3. Composite index on `(merchant_id, valid_from, valid_to)` lets the planner seek per merchant and then range-scan the time bounds — usually fast enough. 4. Postgres-specific: use a `tstzrange` column with a GiST index for genuinely large dimension tables — order-of-magnitude faster than B-tree on bounds.

Edge cases to mention: - Make sure dim windows don't overlap per merchant — overlapping windows give multiple matches and silently inflate fact rows. - Open-ended current rows with `valid_to = NULL` need `COALESCE(valid_to, 'infinity')` in the join condition. - Use `valid_to` as half-open (exclusive) to avoid boundary double-counting — that's the data warehousing convention.

What NOT to say: - Do not just join on `merchant_id` and assume "latest version" — you'll attribute payments to the wrong merchant attributes when history changes. - Do not write `valid_from <= paid_at AND paid_at <= valid_to` with closed bounds — boundary records will match twice.

Common follow-up: "How do you ensure dim windows don't overlap?" — Exclusion constraint in Postgres on `(merchant_id, tstzrange(valid_from, valid_to))`; trigger-based check elsewhere.

```
-- SCD2 join: each payment sees the merchant attributes in effect at paid_at
SELECT p.id, p.amount, m.name AS merchant_name, m.tier AS merchant_tier
FROM payments p
JOIN merchant_dim m
  ON p.merchant_id = m.merchant_id
 AND p.paid_at >= m.valid_from
 AND p.paid_at < COALESCE(m.valid_to, 'infinity'::timestampz);
```

Q39 · Star Join Optimization — Data Warehouse Pattern

Tags: Difficulty: Hard · Asked at: Walmart Labs, Target India, Goldman, AmEx · Topic: Joins

The question (interviewer's verbatim phrasing):

In a star schema, one fact table joins to many small dimensions. How does the optimizer handle this differently from a transactional 4-way join?

The 30-second answer: The optimizer recognises the star pattern — one big fact in the centre, many small dimensions around it — and uses star join optimization. Each dimension is filtered first, then a single hash join (or bitmap intersection) probes the fact table once with the combined dimension keys. Avoids walking the giant fact multiple times.

Step-by-step thought process: 1. Frame the schema — `fact_sales` (billions of rows) joined to `dim_product`, `dim_store`, `dim_date`, `dim_customer` (each thousands or millions, small relative to fact). 2. Naive plan: 4 sequential hash joins, each rescanning the fact in some shape. Star join optimization: filter dimensions first to small key sets, then one bitmap-or-hash scan of the fact. 3. Oracle has `STAR_TRANSFORMATION` hint; SQL Server detects star schemas heuristically; Postgres relies on partition-wise joins and parallel scans, but no formal "star transformation". 4. Practical wins: bitmap

indexes on each fact foreign key (Oracle/Postgres BRIN), or columnar formats (DuckDB, ClickHouse, Snowflake) where star joins are the bread and butter.

Edge cases to mention: - Snowflake (the warehouse, not the schema variant) handles star joins natively with micro-partition pruning. - Dimension filtering selectivity matters — if no dimension reduces the fact much, you're back to a full fact scan. - ClickHouse and DuckDB recommend denormalising small dimensions into the fact table — sometimes faster than star join for analytical queries.

What NOT to say: - Do not assume OLTP join intuition transfers — OLAP planners optimise differently. - Do not use star schema for OLTP — too many joins per transaction; use a normalised model.

Common follow-up: *"When would you denormalize the dimensions into the fact?"* — When the dimensions are tiny and read-mostly, or when you're on a columnar store where wide tables compress well.

```
-- Star-schema query — optimizer filters dims first, then probes fact once
SELECT d.day_of_week, p.category, SUM(f.amount) AS revenue
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_store s ON f.store_key = s.store_key
WHERE d.year = 2026
      AND p.category IN ('electronics', 'apparel')
      AND s.region = 'south'
GROUP BY d.day_of_week, p.category;
```

Q40 · Joining on NULL — Does It Match?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, TCS, Wells Fargo · Topic: Joins

The question (interviewer's verbatim phrasing):

I have two tables and the join column is nullable on both sides. Will NULL on one side match NULL on the other side?

The 30-second answer: No. In standard SQL, `NULL = NULL` evaluates to UNKNOWN, not TRUE, so the join condition fails and the rows don't match. If you genuinely want NULLs to join to each other, use `IS NOT DISTINCT FROM` (Postgres) or `a.col = b.col OR (a.col IS NULL AND b.col IS NULL)`.

Step-by-step thought process: 1. State the three-valued-logic rule — `NULL = NULL` is UNKNOWN, never TRUE. 2. Consequence: any row with NULL in the join column is silently dropped from INNER and LEFT/RIGHT match sets — only kept on the preserved side of outer joins as orphans. 3. The fix when

you actually want NULLs to match: `IS NOT DISTINCT FROM` (Postgres, SQL standard); MySQL has `<=>` (null-safe equals); SQL Server forces you to write the `OR ... IS NULL` expansion. 4. Real-world example: optional `referrer_id` on signups joining to a `referrers` dim — joining on NULL would treat every direct-traffic user as referred by every other direct-traffic user, which is nonsense — so plain equality is what you want, and NULL handling has to be explicit.

Edge cases to mention: - `UNION` deduplicates NULLs as if they were equal — inconsistent with join semantics; another classic interview trap. - `IS NOT DISTINCT FROM` blocks index usage in some engines — check the plan. - Group BY treats NULLs as a single group — also inconsistent with join equality.

What NOT to say: - Do not say "NULL matches NULL in a join" — straight wrong in standard SQL. - Do not paper over the issue with `COALESCE(col, -1)` unless you're sure no real value is `-1` — magic numbers backfire.

Common follow-up: "How does Oracle differ here?" — Oracle treats empty string as NULL, which compounds the trap; otherwise the standard rule applies.

```
-- Default: NULL does NOT match NULL
SELECT a.id, b.id
FROM a JOIN b ON a.key = b.key;
-- Rows with NULL in either a.key or b.key are dropped

-- Postgres: explicit NULL-safe equality
SELECT a.id, b.id
FROM a JOIN b ON a.key IS NOT DISTINCT FROM b.key;
```

Q41 · Joining Bookings on Overlapping Time Ranges

Tags: Difficulty: Hard · Asked at: Swiggy, Zomato, Ola, Razorpay · Topic: Joins

The question (interviewer's verbatim phrasing):

I have a `bookings` table with `(rider_id, start_time, end_time)`. Find all overlapping bookings for the same rider — these indicate a scheduling conflict.

The 30-second answer: Self-join the table on `rider_id` and the overlap condition `a.start_time < b.end_time AND a.end_time > b.start_time`, with `a.booking_id < b.booking_id` to avoid duplicate `(A,B)` and `(B,A)` pairs. Index on `(rider_id, start_time)` is essential; for very large tables, use a range-typed column with a GiST index in Postgres.

Step-by-step thought process: 1. Start with the overlap condition — two intervals `[a_start, a_end]` and `[b_start, b_end]` overlap iff `a_start < b_end AND a_end > b_start`.

Memorise this. 2. Self-join with `a.booking_id < b.booking_id` so each conflicting pair appears exactly once and a booking doesn't overlap with itself. 3. Performance: B-tree on `(rider_id, start_time)` reduces it to one rider's bookings at a time; for large data, Postgres `tstzrange` column with `GIST (range_col)` and the `&&` overlap operator is faster. 4. Mention that this is the same query shape used to detect double-booking in ride-hailing, hotel rooms, conference rooms, calendar invites.

Edge cases to mention: - Decide whether touching ranges (`a.end_time = b.start_time`) count as overlapping — usually no, so use strict `<` and `>`. - Open-ended bookings with `end_time IS NULL` (ongoing rides) need `COALESCE(end_time, 'infinity')` or special handling. - For a single rider with thousands of bookings, the self-join is $O(N^2)$ per rider — only practical with the index seek pattern.

What NOT to say: - Do not write `BETWEEN` for overlap detection — it's awkward and easy to get wrong (off-by-one on closed/open intervals). - Do not forget the `<` on `booking_id` — without it, you'll get every pair twice and the booking matched against itself.

Common follow-up: "How would you find the maximum number of concurrent bookings per rider at any time?" — Sweep-line algorithm: expand each booking to `(start, +1)` and `(end, -1)`, sum running totals per rider — that's a window function problem, covered in the windows chapter.

```
-- Overlapping bookings per rider (each pair appears once)
SELECT a.rider_id, a.booking_id AS booking_a, b.booking_id AS booking_b,
       a.start_time, a.end_time, b.start_time, b.end_time
FROM bookings a
JOIN bookings b
  ON a.rider_id = b.rider_id
 AND a.booking_id < b.booking_id
 AND a.start_time < b.end_time
 AND a.end_time  > b.start_time;
```

Q42 · Outer Join with Aggregation — Preserving Zeros

Tags: Difficulty: Medium · Asked at: Flipkart, Cred, Walmart Labs, Target · Topic: Joins

The question (interviewer's verbatim phrasing):

Show the total order amount per user, including users who placed zero orders (they should show as 0, not be missing).

The 30-second answer: LEFT JOIN orders to users, then `COALESCE(SUM(o.amount), 0)`. Without COALESCE, users with no orders get NULL from the SUM (SUM of zero rows is NULL, not zero), and the report looks broken. Group by user.

Step-by-step thought process: 1. Start with the LEFT JOIN — it preserves users who have no orders, with NULL on the order columns. 2. The aggregate trap: `SUM(NULL)` over an empty group is NULL. `COUNT(*)` over the LEFT JOIN counts at least 1 per user (because the user row is preserved) — use `COUNT(o.id)` instead, which doesn't count NULLs. 3. Wrap the SUM in COALESCE to surface zeros: `COALESCE(SUM(o.amount), 0)`. 4. Same trap with AVG and other aggregates — be intentional about NULL semantics.

Edge cases to mention: - `COUNT(*)` vs `COUNT(o.id)` is the most common bug in LEFT JOIN aggregations — pick the right one based on whether you want to count outer rows or matched inner rows. - Filtering on aggregate post-join: use HAVING, not WHERE — WHERE runs before grouping. - If you also LEFT JOIN payments and aggregate both, the two LEFT JOINs can multiply each other — sub-aggregate one in a CTE first.

What NOT to say: - Do not omit COALESCE and let NULLs leak into reports — finance teams will reject the dashboard. - Do not switch to INNER JOIN to avoid NULLs — that drops the users you want to preserve.

Common follow-up: "What if you also need refund totals per user from a third table?" — Don't chain LEFT JOINs and aggregate at the end — pre-aggregate refunds in a CTE, then LEFT JOIN the two aggregates to the user list.

```
-- All users, with zero for those who never ordered
SELECT u.id, u.name,
       COUNT(o.id)           AS order_count,
       COALESCE(SUM(o.amount), 0) AS total_amount
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

Q43 · Cardinality Explosion — Two LEFT JOINs Multiplying

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Walmart Labs · Topic: Joins

The question (interviewer's verbatim phrasing):

I LEFT JOIN orders to users, then LEFT JOIN refunds to the same query. My SUMs are suddenly double what they should be. What's going on?

The 30-second answer: Two one-to-many LEFT JOINs from the same parent multiply each other — a user with 3 orders and 2 refunds gives you $3 \times 2 = 6$ result rows, and any SUM over those rows inflates by 2x or 3x. Fix by pre-aggregating each branch in a CTE before joining to the parent.

Step-by-step thought process: 1. Explain the multiplication geometrically — joining users to orders gives one row per (user, order). Joining that result to refunds gives one row per (user, order, refund). If refunds are per-user (not per-order), every order row pairs with every refund row. 2. Numerical example: user U has 3 orders and 2 standalone refunds. The result has $3 \times 2 = 6$ rows. `SUM(order_amount)` sums each order amount twice; `SUM(refund_amount)` sums each refund 3 times. 3. The fix: aggregate each branch independently in a CTE — `orders_agg` and `refunds_agg` — then LEFT JOIN both back to users on `user_id`. Each user gets exactly one row from each CTE. 4. The faster alternative: separate aggregating subqueries with `SUM(...) FILTER (WHERE ...)` or scalar correlated subqueries — depends on what reads cleaner.

Edge cases to mention: - The bug is hidden when refunds happen to be linked to orders (`refunds.order_id`) — then refunds-per-user is naturally bounded by orders-per-user, and the multiplication is less visible but still real. - `COUNT(DISTINCT)` is sometimes used as a band-aid — it works for counts but not for SUMs. - Window functions can dedupe in some patterns but are usually clumsier than CTE pre-aggregation.

What NOT to say: - Do not say "use DISTINCT to dedupe" — DISTINCT runs on the whole row, not per branch, and can't unmultiply SUMs. - Do not blame the planner — the result is mathematically correct given the join shape; it's the query shape that's wrong.

Common follow-up: "How would you verify the bug before fixing it?" —

`SELECT user_id, COUNT(*) FROM result GROUP BY user_id` for one user — if the count exceeds the expected joined cardinality, you've caught it.

```
-- Bug: orders x refunds multiplication
SELECT u.id, SUM(o.amount), SUM(r.amount)
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
LEFT JOIN refunds r ON r.user_id = u.id
GROUP BY u.id;

-- Fix: pre-aggregate each branch
WITH orders_agg AS (
  SELECT user_id, SUM(amount) AS total_orders
  FROM orders GROUP BY user_id
),
refunds_agg AS (
  SELECT user_id, SUM(amount) AS total_refunds
  FROM refunds GROUP BY user_id
)
SELECT u.id, u.name,
       COALESCE(o.total_orders, 0) AS total_orders,
       COALESCE(r.total_refunds, 0) AS total_refunds
FROM users u
LEFT JOIN orders_agg o ON o.user_id = u.id
LEFT JOIN refunds_agg r ON r.user_id = u.id;
```

Chapter 2, Part B — Subqueries

Part B picks up where the joins left off and walks into the territory interviewers love most for the "thinking out loud" signal — subqueries. EXISTS, IN, correlated forms, derived tables, the NOT IN NULL trap, the four classic ways to find the nth highest salary, the planner's view of materialised CTEs — these are the questions that separate someone who has only written SELECT statements from someone who has actually debugged a slow query at 11pm on a Friday. Twenty-three questions, all field-tested in Indian panels — read them slowly the first time, then quickly the second time, then teach them to a friend on the third pass.

Q44 · Subquery in WHERE vs FROM vs SELECT vs HAVING

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Subqueries

The question (interviewer's verbatim phrasing):

You can put a subquery in WHERE, in FROM, in SELECT and in HAVING. Walk me through when you'd pick each one with a concrete example.

The 30-second answer: WHERE for filtering rows against a list or check; FROM for treating a subquery as a virtual table (derived table); SELECT for a per-row scalar lookup (one value); HAVING for filtering grouped rows by a comparison against a subquery result.

Step-by-step thought process: 1. Lead with the placement-equals-purpose framing — each spot serves a different job in the query pipeline. 2. WHERE example: `WHERE user_id IN (SELECT id FROM users WHERE city = 'Mumbai')` — filtering the outer row set. 3. FROM example: `FROM (SELECT user_id, SUM(amount) AS total FROM orders GROUP BY user_id) t` — a pre-aggregated derived table you then join or filter. 4. SELECT example: `SELECT o.id, (SELECT name FROM users WHERE id = o.user_id) AS user_name FROM orders o` — a scalar lookup per row. 5. HAVING example: `HAVING SUM(amount) > (SELECT AVG(total) FROM order_totals)` — comparing a group's aggregate against a single value the subquery returns. 6. Be ready to say which you avoid in production: SELECT subqueries are usually rewritten as JOINS for performance.

Edge cases to mention: - A SELECT-list subquery must return exactly one row and one column or you get a runtime error. - HAVING runs after GROUP BY, so its subquery can reference aggregates the WHERE clause cannot. - FROM subqueries must have an alias in standard SQL — Postgres enforces it strictly, MySQL forgives it sometimes.

What NOT to say: - Do not claim "they are all equivalent, just style" — they run at different stages and have different cardinality rules. - Do not put a multi-row subquery in SELECT and shrug when it errors.

Common follow-up: "Which one would the optimiser most likely rewrite, and why?" — A SELECT-list correlated subquery; many planners convert it to a LEFT JOIN with aggregation when it's safe.

```
SELECT o.id,  
       (SELECT name FROM users u WHERE u.id = o.user_id) AS user_name  
FROM orders o  
WHERE o.amount > (SELECT AVG(amount) FROM orders);
```

Q45 · Scalar Subquery and the Multi-Row Gotcha

Tags: Difficulty: Easy · Asked at: Infosys, Wipro, Accenture, Capgemini · Topic: Subqueries

The question (interviewer's verbatim phrasing):

What is a scalar subquery and what's the most common runtime error you see with them?

The 30-second answer: A scalar subquery returns exactly one row and one column. The classic error is when it accidentally returns more than one row — the database throws `subquery returned more than one row` in Postgres or `Subquery returns more than 1 row` (Error 1242) in MySQL.

Step-by-step thought process: 1. Define scalar — one row, one column — and where it can legally appear: anywhere an expression can. 2. The trap: a junior writes `WHERE salary > (SELECT salary FROM employees WHERE city = 'Bengaluru')` thinking Bengaluru has one employee, but it has fifty. 3. The fix is to pick an aggregate: `MAX`, `MIN`, `AVG`, or to add a filter that guarantees one row, or to switch the operator to `IN / ANY / ALL`. 4. Defensive habit: wrap with `LIMIT 1` only if you genuinely don't care which row — otherwise use an aggregate.

Edge cases to mention: - A scalar subquery returning zero rows evaluates to NULL — comparisons with NULL silently give UNKNOWN and filter out rows. - In SELECT list, a scalar subquery runs once per outer row when correlated, once total when not — performance footgun. - Oracle has `SINGLE_VALUE` operator that enforces the one-row contract at plan time.

What NOT to say: - Do not say "just add `LIMIT 1`" — that hides bugs and gives non-deterministic results. - Do not confuse scalar with derived table; scalar is one value, derived is a virtual table.

Common follow-up: "What if the scalar subquery returns NULL — does the outer comparison fail?" — It returns UNKNOWN, so the row is filtered out silently. Use `COALESCE` to give a safe default.

```
-- Safe: aggregate guarantees one row  
SELECT name, salary  
FROM employees  
WHERE salary > (SELECT AVG(salary) FROM employees WHERE city = 'Bengaluru');
```

Q46 · What Makes a Subquery "Correlated"

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, JP Morgan, Walmart Labs · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Explain what a correlated subquery is and why it can be slow.

The 30-second answer: A correlated subquery references a column from the outer query, which means it has to be evaluated once per outer row instead of once overall — that's an $O(n*m)$ pattern that can wreck large tables.

Step-by-step thought process: 1. The defining trait: the inner query has a column reference from the outer query in scope. 2. Contrast with a non-correlated subquery, which is independent of the outer row and runs once. 3. Performance cost: for every outer row, the inner subquery is re-executed — unless the optimiser rewrites it as a semi-join, which most modern planners do for EXISTS but not always for SELECT-list subqueries. 4. When correlated subqueries are actually fine: when the outer set is small, when the inner is index-supported, or when readability matters more than micro-optimisation.

Edge cases to mention: - A correlated subquery with **EXISTS** short-circuits — the inner query stops as soon as it finds the first matching row. - A correlated subquery in SELECT list cannot use LIMIT in some dialects (older MySQL) — workaround is a JOIN with aggregation. - Postgres's EXPLAIN ANALYZE shows correlated subqueries as a SubPlan node — read the **loops** counter to spot the per-row cost.

What NOT to say: - Do not say "correlated subqueries are always slow" — modern planners often unnest them into joins. - Do not paste a correlated subquery in an interview and say "I'd never write this" — show you know when it reads more clearly than a join.

Common follow-up: "Rewrite this correlated subquery as a JOIN." — Practice the EXISTS → semi-join and the SELECT-list scalar → LEFT JOIN + GROUP BY rewrites.

```
-- Correlated: o.user_id from the outer query is used inside
SELECT u.name
FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.id AND o.amount > 5000
);
```

Q47 · EXISTS vs IN — Semantic and Performance Differences

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Cred, Meesho · Topic: Subqueries

The question (interviewer's verbatim phrasing):

When should I reach for EXISTS and when for IN? Are they really the same thing?

The 30-second answer: Semantically they answer the same question for non-NULL data, but EXISTS short-circuits on first match and is the natural fit for correlated checks, while IN is cleaner with a static list or a one-shot independent subquery. They diverge sharply when NULLs are involved.

Step-by-step thought process: 1. Start with the boolean framing — both return TRUE/FALSE per outer row. 2. EXISTS: stops the inner scan on the first matching row, ideal when the inner table is large. 3. IN: materialises the candidate set (logically), good when the candidate set is small and reusable. 4. NULL behaviour: `value IN (1, 2, NULL)` evaluates to UNKNOWN when value is not 1 or 2 — the trap shows up with NOT IN (Q5). 5. Performance reality: modern optimisers (Postgres, SQL Server, MySQL 8) often produce the same plan for both, so pick whichever reads better.

Edge cases to mention: - EXISTS with `SELECT *` vs `SELECT 1` is identical in every modern optimiser — the column list is ignored. - IN with a subquery that returns millions of rows can be heavier than EXISTS because some planners materialise. - Oracle historically treated NOT IN poorly with NULLs — that has improved but legacy code still has scars.

What NOT to say: - Do not say `SELECT 1` is faster than `SELECT *` inside EXISTS — they parse identically. - Do not claim one is always faster — measure with EXPLAIN ANALYZE on your data.

Common follow-up: "What if I use `IN (SELECT id FROM huge_table)` — is the subquery executed once or per row?" — Once, if uncorrelated; the optimiser typically builds a hash set.

```
-- EXISTS reads cleanly when correlated
SELECT u.id, u.name
FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);
```

Q48 · The NOT IN NULL Trap (and the NOT EXISTS Fix)

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, AmEx · Topic: Subqueries

The question (interviewer's verbatim phrasing):

I ran `WHERE id NOT IN (SELECT manager_id FROM employees)` and got zero rows even though I expected hundreds. What happened?

The 30-second answer: The subquery returned at least one NULL. `NOT IN` with a NULL evaluates to UNKNOWN for every outer row, which filters everything out. Fix is to add `WHERE manager_id IS NOT NULL` to the subquery, or switch to `NOT EXISTS`, which handles NULLs correctly.

Step-by-step thought process: 1. Walk through the logic: `x NOT IN (1, NULL)` becomes `x != 1 AND x != NULL`. The second condition is UNKNOWN, the AND is UNKNOWN, the row is dropped. 2. The bug is silent — no error, just empty results — which is why it eats interview time. 3. NOT EXISTS does not have this problem because it's a row-existence check, not a value comparison. 4. Defensive coding habit: always filter NULLs from a NOT IN subquery, or use NOT EXISTS by default.

Edge cases to mention: - If the column inside the subquery is declared `NOT NULL`, NOT IN is safe — but you cannot always rely on schema you didn't write. - LEFT JOIN with `WHERE right.col IS NULL` (anti-join pattern) is also safe and often the fastest. - The same NULL trap affects `<>` and `!=` comparisons in WHERE generally — not unique to NOT IN.

What NOT to say: - Do not say "NOT IN and NOT EXISTS are equivalent" — they are not in the presence of NULLs. - Do not blame the database — it is following the SQL standard's three-valued logic correctly.

Common follow-up: "Rewrite the query three ways — NOT IN with filter, NOT EXISTS, LEFT JOIN." — Be able to switch between all three on the whiteboard.

```
-- Safe rewrite using NOT EXISTS
SELECT id, name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM employees m WHERE m.manager_id = e.id
);
```

Q49 · ANY, SOME, ALL — What They Mean and When They Appear

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, HCL · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Have you used ANY, SOME or ALL? What do they do and can you rewrite them without those keywords?

The 30-second answer: ANY and SOME are synonyms — `x > ANY(subquery)` is TRUE when x beats at least one row, equivalent to comparing against the MIN. `x > ALL(subquery)` is TRUE only when x beats every row, equivalent to comparing against the MAX. Both can be rewritten with aggregate subqueries or with EXISTS.

Step-by-step thought process: 1. State the semantic mapping clearly: `> ANY` = `> MIN`, `> ALL` = `> MAX`, `< ANY` = `< MAX`, `< ALL` = `< MIN`, `= ANY` = `IN`, `<> ALL` = `NOT IN`. 2. Why interviewers ask: the syntax is rare in day-to-day code but shows up in legacy reports and certifications. 3. Practical example: "employees who earn more than at least one manager" → `salary > ANY (SELECT salary FROM managers)`. 4. The cleaner rewrite is usually aggregate-based, which is what most teams prefer because the intent is obvious.

Edge cases to mention: - Empty subquery: `> ANY ()` is FALSE, `> ALL ()` is TRUE — counterintuitive, worth memorising. - NULL in subquery: `> ALL` becomes UNKNOWN if any compared row is NULL — same NULL trap as NOT IN. - MySQL supports ANY/SOME/ALL; SQLite supports them too; some lightweight engines do not.

What NOT to say: - Do not confuse `= ANY` with `IN` semantically without mentioning the NULL caveat. - Do not say "ALL means all rows" — it means the condition holds against every returned row.

Common follow-up: "Rewrite `salary > ALL (subquery)` using MAX." — `salary > (SELECT MAX(salary) FROM subquery_source)`.

```
-- Equivalent forms
SELECT name FROM employees WHERE salary > ALL (SELECT salary FROM managers);
SELECT name FROM employees WHERE salary > (SELECT MAX(salary) FROM managers);
```

Q50 · Derived Table vs CTE vs Temp Table

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Target · Topic: Subqueries

The question (interviewer's verbatim phrasing):

When would you use a derived table in FROM, a CTE, or a temp table? They look similar — what's the trade-off?

The 30-second answer: Derived table is a subquery in FROM scoped to one statement. CTE (`WITH ...`) is named, reusable within the same statement, and supports recursion. Temp table persists across statements in a session, can be indexed, and is right when you reuse the same result set several times or need stats for the planner.

Step-by-step thought process: 1. Derived table: cheap, scoped to one query, no extra round trip — fine for a one-shot aggregation. 2. CTE: same scope as derived table but named and readable; supports recursion; in Postgres 12+ the optimiser may inline it (non-materialised) for better plans. 3. Temp table: written to tempdb (SQL Server) or temp schema (Postgres) — survives across statements, can be indexed, stats are collected. Worth it when you reuse the set multiple times. 4. Decision rule: one-shot = derived or CTE; multi-statement reuse or need indexes = temp table.

Edge cases to mention: - In older MySQL, derived tables were always materialised — that changed in 5.7+. - Postgres 11 and earlier always materialised CTEs (optimisation fence) — 12+ inlines by default; use `MATERIALIZED` keyword to opt back in. - Temp tables in MySQL cannot be referenced more than once in the same query — surprise gotcha.

What NOT to say: - Do not claim "CTEs are always faster than subqueries" — depends on engine and version. - Do not use a temp table for a one-shot select; that's overkill and adds I/O.

Common follow-up: "Same query — CTE or derived table — does the plan change?" — In Postgres 12+ usually no, in older Postgres yes (CTE was a fence).

```
WITH user_totals AS (  
  SELECT user_id, SUM(amount) AS total FROM orders GROUP BY user_id  
)  
SELECT u.name, ut.total FROM users u JOIN user_totals ut ON ut.user_id = u.id;
```

Q51 · Correlated Subquery vs JOIN Rewrite — When Each Wins

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, JP Morgan, Goldman · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Here's a correlated subquery — when would you rewrite it as a JOIN and when would you leave it?

The 30-second answer: Rewrite as JOIN when you need columns from the inner table or when the optimiser fails to unnest. Leave as correlated when it reads more clearly, when the outer set is small, or when EXISTS gives you cheaper semi-join semantics than a JOIN-with-DISTINCT.

Step-by-step thought process: 1. The correlated form is just a different surface syntax for what is often a JOIN under the hood — modern planners frequently rewrite. 2. JOIN wins when you need columns from both tables: `SELECT u.name, o.amount FROM users u JOIN orders o ON u.id = o.user_id`. 3. Correlated EXISTS wins for "does any match exist" — cheaper than `JOIN ... GROUP BY u.id`. 4. SELECT-list scalar subquery often loses to `LEFT JOIN ... GROUP BY` when you need multiple aggregates per outer row.

Edge cases to mention: - A JOIN where the right side has duplicates multiplies the left rows — correlated EXISTS does not. Different cardinality, different result. - Some planners (older MySQL, older Oracle) struggle to unnest correlated subqueries; rewriting manually helps. - EXPLAIN ANALYZE is the truth-teller — never argue from style alone.

What NOT to say: - Do not say "always rewrite correlated subqueries as JOINS" — sometimes they are semantically different. - Do not write `JOIN ... DISTINCT` to dedupe and call it equivalent to EXISTS — it does extra work.

Common follow-up: "Show me a case where the JOIN gives more rows than the correlated subquery."
— Outer user joined to orders where user has 5 orders: JOIN gives 5 rows, EXISTS gives 1.

```
-- Correlated form
SELECT u.name FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

-- Equivalent semi-join (deduped JOIN)
SELECT DISTINCT u.name FROM users u JOIN orders o ON o.user_id = u.id;
```

Q52 · "Subquery Returned More Than One Row" — Diagnosing the Error

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture · Topic: Subqueries

The question (interviewer's verbatim phrasing):

You run a query and you see "Subquery returns more than 1 row" — what does it mean and how do you fix it?

The 30-second answer: A subquery used in a scalar context — like after `=`, `<`, `>` — returned multiple rows. Fix is to either switch the operator to `IN` / `ANY` / `ALL`, or to aggregate the subquery (MAX/MIN/AVG), or to add a filter that narrows it to one row.

Step-by-step thought process: 1. Recognise the context: this error appears when the subquery is used where a single value is expected. 2. Identify the four fixes: (a) `IN` if you want set membership, (b) aggregate if you want a single derived value, (c) `LIMIT 1` if you genuinely don't care which row (rare), (d) add a stronger predicate. 3. The bug usually lives in a join-condition assumption: candidate assumed "one product per category" but the schema allows many. 4. Defensive habit: if you write `WHERE col = (subquery)`, mentally check "is this guaranteed to be one row?"

Edge cases to mention: - `ORDER BY ... LIMIT 1` is non-deterministic without a unique tiebreaker — add a tiebreaker like `id ASC`. - UPDATE/DELETE with `WHERE col = (subquery)` will throw the same error. - In Postgres the message is "more than one row returned by a subquery used as an expression" — same root cause.

What NOT to say: - Do not silently slap on `LIMIT 1` without telling the interviewer it can hide the underlying schema issue. - Do not blame the data — the query is the bug.

Common follow-up: "What if the subquery returns zero rows?" — Returns NULL, comparison becomes UNKNOWN, row is filtered out — also silent.

```
-- Buggy: returns many rows
SELECT name FROM products WHERE price = (SELECT price FROM products WHERE category='Phones');

-- Fixed with aggregate
SELECT name FROM products WHERE price = (SELECT MAX(price) FROM products WHERE category='Phones');
```

Q53 · UPDATE with a Subquery — Multi-Row Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Target · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Update every order's `user_city` column with the city from the users table. Show me the Postgres and MySQL versions.

The 30-second answer: Postgres uses `UPDATE ... FROM` with the join in a FROM clause. MySQL uses `UPDATE table1 JOIN table2 SET ...` syntax. Both achieve the same multi-row correlated update; pure ANSI SQL would use a correlated subquery in the SET clause.

Step-by-step thought process: 1. Postgres pattern:

`UPDATE orders o SET user_city = u.city FROM users u WHERE u.id = o.user_id` — clean and readable. 2. MySQL pattern: `UPDATE orders o JOIN users u ON u.id = o.user_id SET o.user_city = u.city` — JOIN clause inside UPDATE. 3. ANSI fallback: `UPDATE orders SET user_city = (SELECT u.city FROM users u WHERE u.id = orders.user_id)` — works everywhere but slower and risks the multi-row error if the subquery is ambiguous. 4. Always wrap in a transaction in production — and test with SELECT first using the same WHERE clause.

Edge cases to mention: - If the correlated subquery returns no match, the column is set to NULL — guard with `WHERE EXISTS` or `COALESCE`. - Multi-row update in MySQL with a JOIN does not error on multiple matches — it just updates with the last one matched (driver-dependent), which is dangerous. - Postgres's `UPDATE ... FROM` does not handle multiple matches gracefully either — first match wins by physical order, which is non-deterministic.

What NOT to say: - Do not run UPDATE without a SELECT preview — the number-one cause of production incidents. - Do not assume `UPDATE ... JOIN` is portable — it is MySQL/MariaDB-specific.

Common follow-up: "What if the join produces multiple matches per outer row?" — Unpredictable behaviour. Add `DISTINCT ON` (Postgres) or aggregate the subquery first into a derived table.

```
-- Postgres
UPDATE orders o SET user_city = u.city FROM users u WHERE u.id = o.user_id;
-- MySQL
UPDATE orders o JOIN users u ON u.id = o.user_id SET o.user_city = u.city;
```

Q54 · DELETE with a Subquery

Tags: Difficulty: Medium · Asked at: Cognizant, HCL, Wipro, Capgemini · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Delete every order whose user has been inactive for more than two years. Write it three ways.

The 30-second answer: Three ways — `DELETE ... WHERE user_id IN (subquery)`, `DELETE ... WHERE EXISTS (correlated subquery)`, or vendor-specific `DELETE ... USING` (Postgres) / `DELETE FROM t1 USING t1 JOIN t2 ...` (MySQL). EXISTS is the safest with NULLs.

Step-by-step thought process: 1. IN form: `DELETE FROM orders WHERE user_id IN (SELECT id FROM users WHERE last_login < NOW() - INTERVAL '2 years')`. Watch for NULLs. 2. EXISTS form: `DELETE FROM orders o WHERE EXISTS (SELECT 1 FROM users u WHERE u.id = o.user_id AND u.last_login < NOW() - INTERVAL '2 years')` — NULL-safe. 3. Postgres USING: `DELETE FROM orders o USING users u WHERE u.id = o.user_id AND u.last_login < NOW() - INTERVAL '2 years'` — fastest when you need columns from another table. 4. Always start with the equivalent SELECT, run a COUNT(*), eyeball the result, then convert SELECT to DELETE in a transaction.

Edge cases to mention: - DELETE in MySQL with a subquery on the same table needs a wrapper derived table — MySQL forbids referencing the deleted-from table directly in the subquery. - Foreign keys with `ON DELETE CASCADE` mean one DELETE can cascade across many tables — review the schema first. - Soft delete is often the right pattern in production — flag a `deleted_at` instead of removing rows.

What NOT to say: - Do not run DELETE without a `LIMIT` or transaction in production unless you have a backup. - Do not assume DELETE with subquery is always slower than join-based — depends on the planner.

Common follow-up: "How do you delete in batches to avoid lock contention?" — Loop with `LIMIT 1000` per batch and commit between batches.

```
DELETE FROM orders o
WHERE EXISTS (
  SELECT 1 FROM users u WHERE u.id = o.user_id AND u.last_login < NOW() - INTERVAL '2 years'
);
```

Q55 · SELECT * with a Derived Table — Duplicate Column Name Trap

Tags: Difficulty: Easy · Asked at: Infosys, TCS, Wipro · Topic: Subqueries

The question (interviewer's verbatim phrasing):

I wrote `SELECT * FROM (SELECT a.*, b.* FROM a JOIN b ON a.id = b.a_id) t` and it errors. Why?

The 30-second answer: The derived table has two columns named `id` (one from each table) and the database refuses to expose the duplicate when materialised. Fix is to alias each column explicitly inside the derived table or to enumerate columns in the outer SELECT.

Step-by-step thought process: 1. Derived tables enforce unique column names — once the subquery is finalised, the outer query sees a virtual table with a fixed column list. 2. Postgres throws `column reference "id" is ambiguous` (or duplicate-column error depending on context). MySQL gives `Duplicate column name 'id'`. 3. Fix one: explicit aliases inside — `SELECT a.id AS a_id, b.id AS b_id, ... FROM a JOIN b ...`. 4. Fix two: never use `SELECT *` from a JOIN inside a derived table; enumerate columns. It's also better for index-only scans and schema evolution.

Edge cases to mention: - A plain JOIN without the derived table wrapper does allow duplicate column names in the result set — they only break when you wrap and re-select. - Some clients (psql) will display both columns; some ORMs blow up on duplicate column keys silently. - CTEs have the same rule — the column list must be unique.

What NOT to say: - Do not assume `SELECT *` is harmless; it's the root of half the schema-evolution bugs you'll ever see. - Do not blame the database when it's just enforcing relational closure.

Common follow-up: "What's the production lesson?" — Never `SELECT *` in views, CTEs, or derived tables. Always enumerate.

```
SELECT * FROM (
  SELECT a.id AS a_id, a.name, b.id AS b_id, b.amount
  FROM a JOIN b ON a.id = b.a_id
) t;
```

Q56 · Semi-Join and Anti-Join via Subqueries

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Cred, Razorpay · Topic: Subqueries

The question (interviewer's verbatim phrasing):

What is a semi-join and what is an anti-join? Give me the SQL surface syntax for each.

The 30-second answer: A semi-join returns left rows where at least one right match exists — expressed in SQL as `WHERE EXISTS (...)` or `WHERE col IN (...)`. An anti-join returns left rows where no right match exists — `WHERE NOT EXISTS (...)` or `LEFT JOIN ... WHERE right.col IS NULL`.

Step-by-step thought process: 1. Define the relational algebra: semi-join $\ltimes$ keeps left rows with a match; anti-join $\bowtie$ keeps left rows without. 2. Surface syntax for semi-join: `EXISTS` or `IN` — never `JOIN`, because `JOIN` multiplies rows when right has duplicates. 3. Surface syntax for anti-join: `NOT EXISTS` or `LEFT JOIN ... IS NULL`; avoid `NOT IN` because of the NULL trap. 4. Why interviewers ask: this is the vocabulary the planner uses in `EXPLAIN` — when you say "semi-join hash" the conversation accelerates.

Edge cases to mention: - Postgres `EXPLAIN` labels these explicitly: `Hash Semi Join` and `Hash Anti Join`. - MySQL 8 added semi-join optimisation that lets the planner unnest `IN/EXISTS` subqueries into hash semi-joins. - An anti-join can be more efficient than `LEFT JOIN ... IS NULL` because the planner can stop scanning the right side as soon as it finds one match.

What NOT to say: - Do not confuse a `JOIN` with semi-join — they have different cardinalities. - Do not say "EXISTS is just IN with extra syntax" — that ignores the correlated case where EXISTS is the natural form.

Common follow-up: "What does the planner do with `NOT EXISTS` in Postgres?" — Converts it to a Hash Anti Join, which is essentially a join that emits left rows only when no right row matches.

```
-- Semi-join: users with at least one order
SELECT u.id, u.name FROM users u WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);
-- Anti-join: users with no orders
SELECT u.id, u.name FROM users u WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);
```

Q57 · Top N Per Group — Subquery vs Window Function

Tags: Difficulty: Hard · Asked at: Swiggy, Razorpay, Walmart Labs, Goldman · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Give me the top three orders by amount for each customer. Show me the subquery way and the window-function way.

The 30-second answer: Window-function way: `ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY amount DESC)` then filter where the rank is ≤ 3 in a derived table. Subquery way: correlated

`WHERE 3 > (SELECT COUNT(*) FROM orders o2 WHERE o2.user_id = o.user_id AND o2.amount > o.amount)` — works on engines without window functions but is $O(n^2)$.

Step-by-step thought process: 1. Window approach is the modern default — clean, $O(n \log n)$, works on Postgres, SQL Server, MySQL 8, Oracle. 2. Subquery approach is the legacy fallback — correlated counter, runs once per row, expensive on large tables. 3. Tie handling: ROW_NUMBER gives a strict order, RANK ties give "1,1,3", DENSE_RANK ties give "1,1,2" — pick based on what the interviewer asks. 4. Engine note: pre-MySQL 8, window functions did not exist — the subquery form was the only choice and shows up in legacy code.

Edge cases to mention: - The correlated subquery form needs a tiebreaker column or it counts ties incorrectly — add `OR (o2.amount = o.amount AND o2.id < o.id)`. - If the partition has fewer than N rows, both methods return whatever exists — usually fine. - LATERAL joins (Postgres) are a third elegant approach for top-N — worth mentioning if asked.

What NOT to say: - Do not say "window functions are always better" — on MySQL 5.7 you simply do not have them. - Do not write the correlated form without a tiebreaker and then claim it handles ties correctly.

Common follow-up: "What changes if you switch from ROW_NUMBER to DENSE_RANK?" — Ties get the same rank; you may end up with more than three rows per group if ties exist.

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY amount DESC) AS rn
  FROM orders
) t
WHERE rn <= 3;
```

Q58 · Median Calculation via Subquery

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman, Wells Fargo, AmEx · Topic: Subqueries

The question (interviewer's verbatim phrasing):

There is no portable MEDIAN function in standard SQL. Compute the median salary using a subquery.

The 30-second answer: Use `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)` on Postgres, SQL Server, Oracle. On MySQL where it doesn't exist, use a self-join / subquery approach: find the value(s) where the count of smaller-or-equal equals the count of larger-or-equal, then average for even-count cases.

Step-by-step thought process: 1. State the modern answer first — `PERCENTILE_CONT(0.5)` is the cleanest and most portable across enterprise engines. 2. The subquery approach: `WHERE (SELECT COUNT(*) FROM t WHERE salary <= s.salary) >= total/2 AND (SELECT COUNT(*) FROM t WHERE salary >= s.salary) >= total/2` then `AVG` the result. 3. Even vs odd row counts: even counts need the average of the two middle values; odd counts give a single value. Both fall out of the `>= total/2` condition. 4. MySQL 8 alternative: assign `ROW_NUMBER()`, find the middle row(s), average.

Edge cases to mention: - Performance: the correlated `COUNT(*)` approach is $O(n^2)$ — fine for thousands, painful for millions. - NULL handling: most engines ignore NULLs in `PERCENTILE_CONT` — verify with your dataset. - Some engines have `MEDIAN()` as an extension (Snowflake, Redshift) — flag if it's available.

What NOT to say: - Do not give only the `PERCENTILE` answer if asked specifically about "subquery" approach — the interviewer wants both. - Do not forget the even-vs-odd case — that's the trap.

Common follow-up: "What's the time complexity of your subquery solution?" — $O(n^2)$ for the correlated count, or $O(n \log n)$ with `ROW_NUMBER`.

```
-- Modern, portable
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) FROM employees;
-- Subquery fallback on MySQL 8 with ROW_NUMBER
SELECT AVG(salary) FROM (
  SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) AS rn, COUNT(*) OVER () AS c
  FROM employees
) t WHERE rn IN ((c+1)/2, (c+2)/2);
```

Q59 · nth Highest Salary — Four Solutions

Tags: Difficulty: Hard · Asked at: TCS, Infosys, Wipro, Cognizant, Flipkart, Razorpay · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Find the second highest salary. Then generalise to nth. Show me four different ways.

The 30-second answer: Four ways — (1)

`SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees)` for n=2; (2) `DENSE_RANK() OVER (ORDER BY salary DESC)` then filter for `rank = n`; (3) `ORDER BY salary DESC LIMIT 1 OFFSET n-1`; (4) correlated `WHERE n-1 = (SELECT COUNT(DISTINCT salary) FROM e2 WHERE e2.salary > e.salary)`.

Step-by-step thought process: 1. Subquery (n=2 special case): max of all salaries less than the max — clear and easy but does not generalise to nth without nested layers. 2. DENSE_RANK: the cleanest general solution — `WHERE rnk = :n`. Use DENSE_RANK not RANK so ties don't skip ranks. 3. LIMIT/OFFSET: cleanest in MySQL/Postgres for one-shot use — `ORDER BY salary DESC LIMIT 1 OFFSET :n - 1` — but does not handle ties (treats duplicates as separate ranks). 4. Correlated COUNT(DISTINCT): the legacy subquery approach — works on any engine, handles ties correctly via DISTINCT, but $O(n^2)$. 5. Always ask: "do ties count as one rank or separate ranks?" That decides DENSE_RANK vs ROW_NUMBER vs LIMIT/OFFSET.

Edge cases to mention: - If fewer than n distinct salaries exist, return NULL or empty — handle gracefully with COALESCE. - LIMIT/OFFSET on a non-deterministic ORDER BY (with ties) is non-portable — add a tiebreaker. - The SUBQUERY-of-MAX approach can be nested for n=3 but becomes unreadable beyond n=2.

What NOT to say: - Do not say "all four are equivalent" — they differ in tie handling. - Do not write `LIMIT n, 1` and call it portable — that's MySQL syntax; standard is `LIMIT 1 OFFSET n-1`.

Common follow-up: "What if there are ties at the nth position — should you return all of them?" — Yes if the interviewer says "every employee earning the nth highest salary"; use DENSE_RANK and return all rows with that rank.

```
-- DENSE_RANK approach (cleanest general)
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 3;
```

Q60 · Subquery in SELECT (Projection) — Performance Cost

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Cred · Topic: Subqueries

The question (interviewer's verbatim phrasing):

What's the performance cost of putting a subquery in the SELECT list?

The 30-second answer: If correlated, the subquery runs once per outer row — $O(n*m)$ — which is fine on small outer sets but devastating on large ones. Most planners try to unnest into a LEFT JOIN with aggregation when safe, but they don't always succeed. Rewrite as a JOIN when the outer set is large or when you have multiple correlated projections.

Step-by-step thought process: 1. Lead with the cardinality: a SELECT-list correlated subquery is "for each row in the outer, run the inner". 2. Modern planners (Postgres, SQL Server) unnest single-aggregate correlated subqueries into LEFT JOINs — check EXPLAIN to confirm. 3. Multiple correlated subqueries in one SELECT list multiply the per-row cost — one JOIN-with-aggregation usually beats five scalar subqueries. 4. Non-correlated subqueries in SELECT list are evaluated once and treated as constants — cheap and fine.

Edge cases to mention: - A non-correlated `(SELECT MAX(salary) FROM employees)` in SELECT list runs once total — completely fine. - A correlated subquery hidden inside a CASE expression has the same cost — count CASE branches when estimating. - Some engines cache scalar subquery results per session — flag if the inner data is constant.

What NOT to say: - Do not say "subqueries in SELECT are always slow" — non-correlated ones are free. - Do not assume the planner unnests every time — verify with EXPLAIN.

Common follow-up: "How would you rewrite five correlated SELECT subqueries into one query?" — One LEFT JOIN to a pre-aggregated derived table.

```
-- Slow: correlated subquery per row
SELECT u.id, (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) AS order_count FROM users u;
-- Fast: aggregated derived table
SELECT u.id, COALESCE(t.cnt, 0) FROM users u
LEFT JOIN (SELECT user_id, COUNT(*) AS cnt FROM orders GROUP BY user_id) t ON t.user_id = u.id;
```

Q61 · IN with a List vs IN with a Subquery — Planner Differences

Tags: Difficulty: Medium · Asked at: Flipkart, Walmart Labs, JP Morgan · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Is there any difference between `IN (1, 2, 3)` and `IN (SELECT id FROM small_table)` ?

The 30-second answer: Yes — `IN (1, 2, 3)` is a constant list the planner can use to drive an index range scan or to expand into OR-equality checks. `IN (subquery)` is treated as a semi-join — the optimiser may rewrite it as a hash semi-join, a nested-loop with EXISTS-style short-circuit, or materialise the subquery into a temp set.

Step-by-step thought process: 1. Constant list: the planner sees all values at compile time, can prune partitions, can pick an index range scan. 2. Subquery: the planner does not know cardinality at compile time — uses estimates from stats. Plan can swing between hash semi-join (large subquery) and nested-loop EXISTS (small subquery). 3. Common bug: an IN list with thousands of literals slows down query parsing and can hit per-query length limits. Switch to a subquery or temp table. 4. ANALYZE / EXPLAIN reveals what the planner actually did — never argue from intuition.

Edge cases to mention: - Postgres has a limit (varies by build) on the number of IN-list parameters before performance degrades — somewhere around a thousand items, switch to ANY(array) or a VALUES clause. - Oracle has a hard limit of 1000 expressions in an IN list — a classic gotcha for migrating apps. - An IN subquery with DISTINCT is redundant — the semi-join semantics already deduplicate.

What NOT to say: - Do not say "IN (subquery) is just a syntactic sugar over IN list" — the planner handles them differently. - Do not paste 5000 IDs in an IN list and ask why your query is slow.

Common follow-up: "What's the limit of IN list items before it becomes a bad pattern?" — A few hundred to a thousand; beyond that use VALUES, temp table, or ANY(array).

```
-- Constant list
SELECT * FROM orders WHERE status IN ('paid', 'refunded', 'partial_refund');
-- Subquery
SELECT * FROM orders WHERE user_id IN (SELECT id FROM users WHERE city = 'Mumbai');
```

Q62 · Correlated Subquery for Running Condition Checks

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Wells Fargo, Razorpay · Topic: Subqueries

The question (interviewer's verbatim phrasing):

For each order, tell me whether the same user placed any earlier order on the same day. Use a correlated subquery.

The 30-second answer: Use `EXISTS (SELECT 1 FROM orders o2 WHERE o2.user_id = o.user_id AND o2.created_at < o.created_at AND DATE(o2.created_at) = DATE(o.created_at))` — the inner query references the outer row, making it a correlated check that returns TRUE per outer row when a same-user same-day earlier order exists.

Step-by-step thought process: 1. Identify the per-row check pattern — for each outer row, run a check against the rest of the data. 2. EXISTS is the natural shape because you just need a boolean, not a value. 3. The correlation is on two columns — `user_id` and the date part of `created_at` — both must match. 4. Index recommendation: composite index on `(user_id, created_at)` makes the inner query an index range scan and the whole pattern fast. 5. Modern alternative: `LAG()` window function — but interviewers asking about correlated subqueries often want to see the EXISTS form.

Edge cases to mention: - Timezone matters when comparing dates — be explicit about session timezone or use `created_at::date` consistently. - If you want a count, not just a check, replace EXISTS with `(SELECT COUNT(*) FROM ...)` — but be aware of the performance trade-off. - Window function `COUNT(*) OVER (PARTITION BY user_id, DATE(created_at) ORDER BY created_at ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)` is a one-pass alternative.

What NOT to say: - Do not write the correlated subquery and then claim it's $O(n)$ — it's potentially $O(n*m)$ without indexes. - Do not skip the index discussion — that's what makes the difference between 200ms and 200s.

Common follow-up: "How would you rewrite this without a correlated subquery?" — Use `LAG()` or `ROW_NUMBER()` with a PARTITION BY user_id and date.

```
SELECT o.id, o.user_id, o.created_at,
       EXISTS (
         SELECT 1 FROM orders o2
         WHERE o2.user_id = o.user_id
               AND o2.created_at < o.created_at
               AND DATE(o2.created_at) = DATE(o.created_at)
       ) AS has_earlier_same_day
FROM orders o;
```

Q63 · Materialized vs Not Materialized CTE (Postgres 12+ Change)

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Walmart Labs · Topic: Subqueries

The question (interviewer's verbatim phrasing):

What changed about CTE optimisation in Postgres 12? Should I still treat CTEs as optimisation fences?

The 30-second answer: Before Postgres 12, every CTE was materialised — an optimisation fence the planner could not see through, often hurting performance. From 12 onwards, simple CTEs are inlined by default (the planner can push predicates in), unless they are recursive, side-effecting, or referenced more than once. You can opt back into materialisation with `WITH cte AS MATERIALIZED (...)`.

Step-by-step thought process: 1. Pre-12 behaviour: CTE was materialised into a temp result, then the outer query ran against it. Predicates from outside could not be pushed in — slow. 2. Post-12 default: CTE is inlined like a subquery when used once and side-effect-free. Predicates push through; plans improve dramatically. 3. Opt-out: `WITH cte AS NOT MATERIALIZED (...)` — explicit inline; `WITH cte AS MATERIALIZED (...)` — explicit fence (useful when the CTE is expensive and reused). 4. Recursive CTEs are always materialised — no change there.

Edge cases to mention: - If your CTE is referenced multiple times, Postgres materialises it by default (since 12) — re-evaluation would be worse. - A CTE with a side effect (INSERT/UPDATE/DELETE inside `WITH`) is always materialised — it must run exactly once. - MySQL CTEs (8.0+) and SQL Server CTEs are inlined by default — Postgres 12 brought it in line with the rest of the world.

What NOT to say: - Do not say "CTEs are always optimisation fences" — that's outdated for Postgres 12+ and never true for SQL Server. - Do not say "use CTEs instead of subqueries for performance" — they are essentially identical in modern planners.

Common follow-up: "When would you explicitly use `MATERIALIZED`?" — When the CTE is expensive (heavy aggregation) and referenced multiple times, or when you want to deliberately stop predicate push-down for plan stability.

```
-- Explicit materialisation in Postgres 12+
WITH heavy AS MATERIALIZED (
  SELECT user_id, SUM(amount) total FROM orders GROUP BY user_id
)
SELECT u.name, h.total FROM users u JOIN heavy h ON h.user_id = u.id WHERE h.total > 10000;
```

Q64 · Subquery Returning Multiple Columns — Row Constructor

Tags: Difficulty: Medium · Asked at: PhonePe, Flipkart, Cred · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Can a subquery return multiple columns? How would you compare two columns at once?

The 30-second answer: Yes — use a row constructor: `WHERE (col_a, col_b) IN (SELECT a, b FROM ...)`. The outer row matches when both columns match the same row in the subquery, not just the columns independently.

Step-by-step thought process: 1. Single-column IN is the common case; row-constructor IN is the multi-column extension. 2. Use case: matching a composite key against a candidate set — `WHERE (city, pincode) IN (SELECT city, pincode FROM serviceable_areas)`. 3. The match is tuple-wise: `(Mumbai, 400001)` matches only when both city and pincode appear together in the

subquery, not when city matches one row and pincode matches another. 4. Supported in Postgres, MySQL, SQL Server (with some syntax variations), Oracle.

Edge cases to mention: - Indexes on `(city, pincode)` make tuple-IN fast; without them, the planner falls back to a hash semi-join. - NULL behaviour is the same trap as scalar IN — a NULL in either side gives UNKNOWN. - Some engines optimise row-constructor IN as a join under the covers — check EXPLAIN.

What NOT to say: - Do not say "you can't compare multiple columns in IN" — you can, with row constructors. - Do not split into two ANDed IN clauses; that returns a different result set (cross-match).

Common follow-up: "What's the difference between `(a, b) IN (subquery)` and `a IN (subquery_a) AND b IN (subquery_b)`?" — The first matches tuples; the second matches independently and over-returns.

```
SELECT order_id FROM orders
WHERE (city, pincode) IN (SELECT city, pincode FROM serviceable_areas);
```

Q65 · LATERAL Subquery — Correlated Derived Tables

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman · Topic: Subqueries

The question (interviewer's verbatim phrasing):

What does the LATERAL keyword do and when would you use it?

The 30-second answer: LATERAL lets a subquery in the FROM clause reference columns from preceding FROM items — essentially a correlated derived table. Common use is "for each user, give me the latest three orders" without resorting to window functions.

Step-by-step thought process: 1. Normal derived table in FROM cannot see columns from earlier FROM items — it's standalone. 2. LATERAL flips that: `FROM users u, LATERAL (SELECT * FROM orders o WHERE o.user_id = u.id ORDER BY created_at DESC LIMIT 3) o3` — the inner can see `u.id`. 3. Replaces the awkward window-function approach for top-N-per-group when you want full row data, not just rank. 4. Available in Postgres (since 9.3), Oracle 12c+, SQL Server (via CROSS APPLY which is the same thing). MySQL got it in 8.0.14.

Edge cases to mention: - LATERAL implicitly behaves like a JOIN — pair with `LEFT JOIN LATERAL` to keep outer rows when the inner returns nothing. - SQL Server's equivalent is `CROSS APPLY` (inner) and `OUTER APPLY` (left) — same semantics, different keyword. - Performance: LATERAL with a LIMIT inside is often the fastest way to get top-N-per-group when the outer set is small and you have an index.

What NOT to say: - Do not assume LATERAL is everywhere — MySQL added it late, SQLite still doesn't have it. - Do not confuse LATERAL with CROSS JOIN — CROSS JOIN has no correlation.

Common follow-up: "*When does LATERAL win over a window function?*" — When you want a fixed limit per group with full row data, and the outer set is small relative to the inner.

```
SELECT u.id, o3.*
FROM users u,
LATERAL (SELECT * FROM orders o WHERE o.user_id = u.id ORDER BY o.created_at DESC LIMIT 3) o3;
```

Q66 · Subquery in HAVING — Filtering Groups Against an Aggregate

Tags: Difficulty: Medium · Asked at: TCS, Infosys, JP Morgan, Wells Fargo · Topic: Subqueries

The question (interviewer's verbatim phrasing):

Find every city whose total order revenue is above the all-India average. Use a subquery in HAVING.

The 30-second answer: Group orders by city, then in HAVING compare the group's `SUM(amount)` against a subquery returning the all-India average revenue per city:

```
HAVING SUM(amount) > (SELECT AVG(s) FROM (SELECT SUM(amount) s FROM orders
GROUP BY city) t) .
```

Step-by-step thought process: 1. WHERE cannot reference aggregates — that's the job of HAVING, which runs after GROUP BY. 2. The subquery in HAVING must return a scalar (one row, one column) — guarantee that with an aggregate. 3. The classic pattern: city-level aggregate compared to the cross-city average — the inner subquery first aggregates per city, then averages those aggregates. 4. Watch the difference between "average revenue per city" (group then average) and "average revenue overall" (`AVG(amount)`) — interviewer usually wants the first.

Edge cases to mention: - A correlated HAVING subquery is legal — it can reference grouped columns from the outer query. - HAVING can reference aggregates that are not in SELECT — Postgres allows this freely; some engines require the aggregate in SELECT. - The inner aggregation can usually be hoisted into a CTE for readability without affecting performance.

What NOT to say: - Do not put the same filter in WHERE; aggregates are not available there. - Do not assume HAVING runs before GROUP BY — its whole purpose is post-grouping filtering.

Common follow-up: "*Could you rewrite this without HAVING using a derived table?*" — Yes — aggregate to a derived table, then filter in an outer WHERE.

```
SELECT city, SUM(amount) AS revenue
FROM orders
GROUP BY city
HAVING SUM(amount) > (
    SELECT AVG(s) FROM (SELECT SUM(amount) s FROM orders GROUP BY city) t
);
```

Chapter 3, Part A — Window Functions

Window functions used to be the "advanced topic" you could skip as a fresher; that is no longer true. Across 2024-2026 hiring cycles — TCS Digital, Infosys Power Programmer, Wipro Elite, every GCC panel (JP Morgan India, Walmart Labs, Wells Fargo, Goldman, Target, AmEx) and every product company (Razorpay, Swiggy, Cred, Meesho, Zerodha, Flipkart, PhonePe) — window functions are baseline expectation even for zero-experience candidates. If you cannot explain **PARTITION BY** versus **GROUP BY**, write a top-N-per-group query, or spot the **LAST_VALUE** default-frame trap, you will lose the round. This chapter (Part A) covers Window Functions end to end across thirty questions; Part B of the same chapter covers Common Table Expressions, both vanilla and recursive. Read both before any interview where the JD mentions SQL.

Q67 · Window Functions Introduction — What They Do and When to Use

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

What is a window function and how is it different from a regular aggregate? Give me an example where you'd reach for one.

The 30-second answer: A window function computes a value across a set of rows related to the current row — the "window" — without collapsing those rows into one. Unlike **GROUP BY** which reduces rows, a window function keeps every input row and adds a new column with the computed value. Reach for one whenever you need a per-row metric that depends on neighbouring rows: running totals, rank within a group, previous row's value, top-N per category.

Step-by-step thought process: 1. Open with the row-preservation point — that is the single biggest semantic difference from **GROUP BY** and the thing freshers miss. 2. Give the shape: **function() OVER (PARTITION BY ... ORDER BY ... frame)**. Every window function has this same skeleton. 3. Give a concrete example: "rank each customer's orders by amount, keeping all order rows." With **GROUP BY customer_id** you lose order-level rows; with **RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC)** you keep them and add a rank column. 4. Mention the

three families: ranking (`ROW_NUMBER` , `RANK` , `DENSE_RANK` , `NTILE` , `PERCENT_RANK` , `CUME_DIST`), aggregate-as-window (`SUM` , `AVG` , `COUNT` , `MIN` , `MAX` with `OVER`), and value access (`LAG` , `LEAD` , `FIRST_VALUE` , `LAST_VALUE` , `NTH_VALUE`). 5. Anticipate the follow-up: "When would you not use one?" — when you genuinely need to collapse rows (true grouping) or when the same result is cheaper via a self-join with an index, which is rare in 2026.

Edge cases to mention: - Window functions are evaluated after `WHERE` , `GROUP BY` and `HAVING` but before `ORDER BY` and `LIMIT` . That ordering causes the Filter-Window Trap (Q13). - You cannot reference a window function in `WHERE` — wrap the query in a CTE or subquery, or use `QUALIFY` on Snowflake / BigQuery / Teradata. - Window functions never reduce the row count of the input; if you need both row-level and aggregated output in one shot, that is exactly the use case.

What NOT to say: - Do not say "window function is just a fancy `GROUP BY` ." It is the opposite — it preserves rows rather than collapsing them. - Do not claim window functions are slow as a blanket rule; modern planners parallelise them and they often beat the equivalent self-join.

Common follow-up: "Show me a query that uses both `GROUP BY` and a window function in the same statement." — Easy: aggregate first in a subquery, then apply the window on the aggregated rows (see Q21).

```
SELECT
  order_id, customer_id, amount,
  RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS amount_rank,
  SUM(amount) OVER (PARTITION BY customer_id) AS customer_total
FROM orders;
```

Q68 · ROW_NUMBER vs RANK vs DENSE_RANK — When Each is the Wrong Choice

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Cred, Flipkart · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Explain the difference between `ROW_NUMBER` , `RANK` and `DENSE_RANK` . Give me a scenario where picking the wrong one quietly breaks the report.

The 30-second answer: `ROW_NUMBER` always gives unique sequential numbers even when sort keys tie. `RANK` gives the same rank to ties but leaves gaps (1, 2, 2, 4). `DENSE_RANK` gives the same rank to ties without gaps (1, 2, 2, 3). Wrong choice example: using `ROW_NUMBER` for a leaderboard breaks fairness because two players with identical scores get different ranks based on physical row order.

Step-by-step thought process: 1. Lead with the tie behaviour — that is the entire test. 2. Walk through one input set: scores `[100, 90, 90, 80]` → `ROW_NUMBER` gives `1,2,3,4` ; `RANK` gives `1,2,2,4` ; `DENSE_RANK` gives `1,2,2,3` . 3. Pick scenarios: leaderboards → `DENSE_RANK` (fair ranks, no gaps); deduplication ("keep the latest row per user") → `ROW_NUMBER` with an explicit tie-breaker; top-N where you want N positions including ties → `RANK` . 4. Stress that `ROW_NUMBER` without a deterministic `ORDER BY` is non-deterministic — two runs can give different numbers, which is a data-quality bug waiting to happen.

Edge cases to mention: - For dedup queries (`SELECT ... WHERE rn = 1`), always add a tie-breaker to the `ORDER BY` (for example `ORDER BY updated_at DESC, id DESC`) so two rows with identical timestamps still pick the same winner each run. - `RANK` and `DENSE_RANK` are stable across runs because tied rows share a rank; `ROW_NUMBER` is not. - All three require `ORDER BY` inside `OVER` — without it the result is meaningless / not allowed.

What NOT to say: - Do not say "they all do the same thing with minor differences." The tie behaviour can silently corrupt leaderboards and dedup logic. - Do not skip the tie-breaker in dedup `ROW_NUMBER` queries; reviewers in panels will hammer on that.

Common follow-up: "Write a dedup query that keeps the latest row per `user_id` from `user_events` ." — Classic warm-up question; answer below.

```
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY event_ts DESC, event_id DESC
    ) AS rn
  FROM user_events
)
SELECT * FROM ranked WHERE rn = 1;
```

Q69 · PARTITION BY Semantics vs GROUP BY

Tags: Difficulty: Easy · Asked at: TCS, Wipro, Goldman, AmEx · Topic: Window Functions

The question (interviewer's verbatim phrasing):

What is the difference between `PARTITION BY` inside a window and `GROUP BY` in the outer query? People mix these up — make sure I do not.

The 30-second answer: `GROUP BY` collapses the input into one row per group. `PARTITION BY` divides the input into groups for the window function to operate over but keeps every original row. Same grouping keys, opposite outcomes on row count.

Step-by-step thought process: 1. Anchor the answer on row count — `GROUP BY` reduces, `PARTITION BY` preserves. 2. Give a parallel example: `SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id` returns one row per customer. `SELECT order_id, customer_id, amount, SUM(amount) OVER (PARTITION BY customer_id) FROM orders` returns one row per order with the customer total alongside. 3. Mention scope: `GROUP BY` applies to the entire query and forces every selected column to be either grouped or aggregated. `PARTITION BY` is local to one `OVER` clause; the rest of the `SELECT` is unaffected. 4. Point out you can use both in the same query — group first, then apply a window on the grouped output.

Edge cases to mention: - Without `PARTITION BY`, the window covers the entire result set (a single partition) — useful for grand totals. - Multiple window functions in the same `SELECT` can each have their own partition; they do not have to agree. - `GROUP BY` removes duplicates implicitly; `PARTITION BY` does not — duplicates in the input remain in the output.

What NOT to say: - Do not say "they are interchangeable in some cases" — they are not, because the row count is different. - Do not claim `PARTITION BY` is slower than `GROUP BY` as a rule; it depends on whether you actually need every input row.

Common follow-up: "If I write `SUM(amount) OVER (PARTITION BY customer_id)` without an `ORDER BY`, what frame does it use?" — Whole partition (see Q7).

```
-- Per-order amount alongside the customer's grand total
SELECT
  order_id, customer_id, amount,
  SUM(amount) OVER (PARTITION BY customer_id) AS customer_total,
  amount * 1.0 / SUM(amount) OVER (PARTITION BY customer_id) AS share_of_customer
FROM orders;
```

Q70 · ORDER BY Inside OVER — When Required, When Optional

Tags: Difficulty: Easy · Asked at: Infosys, Accenture, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Is `ORDER BY` inside the `OVER` clause always required? Walk me through when it is mandatory and when you can leave it out.

The 30-second answer: Required for any function that depends on row order: all ranking functions (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `PERCENT_RANK`, `CUME_DIST`), value-access functions (`LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`), and any aggregate where you want a running / moving result. Optional when you want an unordered aggregate over the whole partition (for example, customer total alongside each row).

Step-by-step thought process: 1. Group window functions into "needs order" versus "does not need order" and recite the lists. 2. Give the running-total example to make the difference concrete:

`SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_ts)` gives a running total; remove the `ORDER BY` and you get the customer grand total on every row. 3. Warn that `ORDER BY` inside `OVER` is independent of the outer `ORDER BY` — they do different jobs. 4. Anticipate: "What happens if I add an `ORDER BY` to a plain `SUM() OVER ()`?" — you accidentally turn it into a running total and the numbers change. This is a common interview trap.

Edge cases to mention: - Adding `ORDER BY` without an explicit frame `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which is exactly what changes a grand total into a running total (Q7). - Ranking functions require `ORDER BY` even if the partition has a single row, otherwise the result is undefined / a syntax error depending on engine. - `ROW_NUMBER() OVER ()` without partition or order is legal on some engines but the row order is the planner's choice — never rely on it.

What NOT to say: - Do not say "`ORDER BY` is always optional in `OVER`" — false for ranking and value-access functions. - Do not skip the tie-breaker in `ORDER BY` for ranking queries that drive dedup logic; you will get non-deterministic output.

Common follow-up: "Show me a query where adding `ORDER BY` inside `OVER` gives the wrong number." — Running total when you meant grand total; demo below.

```
-- Grand total per customer (no ORDER BY)
SELECT order_id, customer_id, amount,
       SUM(amount) OVER (PARTITION BY customer_id) AS customer_total
FROM orders;

-- Running total per customer (with ORDER BY)
SELECT order_id, customer_id, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_ts) AS running_total
FROM orders;
```

Q71 · Frame Clauses — ROWS BETWEEN vs RANGE BETWEEN vs GROUPS

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Goldman, Cred · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Walk me through `ROWS BETWEEN`, `RANGE BETWEEN` and `GROUPS BETWEEN`. When does each behave differently?

The 30-second answer: `ROWS` counts physical rows from the current row. `RANGE` counts logical value distance — rows with the same `ORDER BY` value are treated as one peer group. `GROUPS`

(Postgres 11+) counts peer groups themselves. They diverge whenever the `ORDER BY` column has ties: `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` looks at exactly two physical rows, `RANGE` includes every peer of the current row, `GROUPS` walks one peer-group back.

Step-by-step thought process: 1. Pick a small dataset: scores `[100, 90, 90, 80]`. For the third row (score 90), `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW` covers rows 2 and 3 (sum = 180). `RANGE BETWEEN 1 PRECEDING AND CURRENT ROW` includes all rows whose `score` is within 1 of the current 90 — both 90s are peers, so the frame is rows 2 and 3 with the same value. `GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW` covers the peer group of 100 plus the peer group of both 90s. 2. State the practical rule: use `ROWS` when you mean "the last N rows" (moving averages, lag windows). Use `RANGE` for time-based windows (`RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW` on Postgres). Use `GROUPS` rarely — mostly for ranking-aware running aggregates. 3. Call out engine support: `ROWS` everywhere; `RANGE BETWEEN value PRECEDING` on Postgres, Snowflake, BigQuery, Oracle; `GROUPS` on Postgres 11+ only.

Edge cases to mention: - The default frame when you specify `ORDER BY` but no explicit frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, not `ROWS`. - `RANGE` with a numeric or interval offset requires the `ORDER BY` to be a single numeric or date/time column; multi-column `ORDER BY` fails. - For moving averages over event streams with duplicate timestamps, `ROWS` and `RANGE` give different answers — choose deliberately.

What NOT to say: - Do not say " `ROWS` and `RANGE` are the same when there are no ties" — that is technically true but glosses over the fact that production data always has ties. - Do not use `GROUPS` without checking that the engine supports it; on MySQL and SQL Server it does not exist.

Common follow-up: "Write a 7-day moving sum using `RANGE` ." — Below.

```
-- 7-day moving sum, peers included via RANGE
SELECT order_ts, amount,
       SUM(amount) OVER (
         ORDER BY order_ts
         RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW
       ) AS sum_last_7d
FROM orders;
```

Q72 · UNBOUNDED PRECEDING, UNBOUNDED FOLLOWING, CURRENT ROW, N PRECEDING / FOLLOWING

Tags: Difficulty: Medium · Asked at: TCS, Walmart Labs, AmEx · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Describe each frame boundary keyword and when you would use them.

The 30-second answer: **UNBOUNDED PRECEDING** = start of the partition. **UNBOUNDED FOLLOWING** = end of the partition. **CURRENT ROW** = the row being processed (or its peer group, depending on **ROWS** vs **RANGE**). **N PRECEDING** and **N FOLLOWING** = N rows back or forward. Combine them around **BETWEEN ... AND ...** to define any window shape from "everything so far" to "the next 5 rows" to "the entire partition."

Step-by-step thought process: 1. List each keyword with a one-line meaning so the interviewer hears coverage. 2. Map each common analytic to a frame: running total = **UNBOUNDED PRECEDING TO CURRENT ROW**; reverse running total = **CURRENT ROW TO UNBOUNDED FOLLOWING**; centered moving average = **2 PRECEDING TO 2 FOLLOWING**; full-partition aggregate = **UNBOUNDED PRECEDING TO UNBOUNDED FOLLOWING**. 3. Stress the difference between **CURRENT ROW** under **ROWS** (the literal current row) and under **RANGE** (the current row plus all peers with the same **ORDER BY** value). 4. Anticipate "What is the smallest frame?" — **BETWEEN CURRENT ROW AND CURRENT ROW**, which is one row (or its peer group).

Edge cases to mention: - **BETWEEN CURRENT ROW AND CURRENT ROW** under **RANGE** is not a single row when there are ties. - **N PRECEDING** / **N FOLLOWING** with **RANGE** requires the offset to be the same data type as the **ORDER BY** column (numeric for numbers, interval for dates). - Frame boundaries cannot cross: **BETWEEN 5 FOLLOWING AND 2 PRECEDING** is a syntax error.

What NOT to say: - Do not omit the frame when you intend a running total and assume the default does the right thing — it does on **ROWS** mental model, but the default is actually **RANGE** (Q7). - Do not use **UNBOUNDED FOLLOWING** casually with **LAST_VALUE** and assume you get the partition's last value — see Q9 for the trap.

Common follow-up: "Write a centered 5-row moving average." — **AVG(x) OVER (ORDER BY ts ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING)**.

```
SELECT order_ts, amount,
       AVG(amount) OVER (
         ORDER BY order_ts
         ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING
       ) AS centered_avg_5
FROM orders;
```

Q73 · The Default Frame Trap — ORDER BY Without Explicit Frame

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, PhonePe · Topic: Window Functions

The question (interviewer's verbatim phrasing):

If I write `SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_ts)` with no explicit frame, what frame does the engine use?

The 30-second answer: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. That is the SQL standard default, applied by every major engine. So the result is a running total — but with `RANGE` semantics, meaning rows that share the same `order_ts` all get the same running total, not staggered values.

Step-by-step thought process: 1. State the default verbatim so the interviewer knows you have read the spec, not guessed. 2. Explain the consequence: the moment you add `ORDER BY` to an aggregate's `OVER`, you turn a grand total into a running total. People who think `ORDER BY` is "just for sorting the window" get the wrong number. 3. Highlight the `RANGE` vs `ROWS` nuance: with duplicate `ORDER BY` values, `RANGE` lumps peers together, so two events at the same timestamp produce identical running totals. Use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` if you want strict per-row stepping. 4. Recommend the production habit: always specify the frame explicitly when using aggregates with `ORDER BY`. It is cheap insurance against this exact bug.

Edge cases to mention: - Without `ORDER BY`, the default frame is the entire partition — that is why `SUM() OVER (PARTITION BY ...)` gives a grand total. - Ranking functions ignore the frame entirely — `RANK`, `ROW_NUMBER`, `DENSE_RANK` use only `PARTITION BY` and `ORDER BY`. - `LAG` and `LEAD` also ignore the frame.

What NOT to say: - Do not say "the default is the whole partition when you have `ORDER BY`" — that is wrong; the default is up-to-current-row. - Do not assume `RANGE` and `ROWS` agree under the default; they only do when the `ORDER BY` column has no duplicates.

Common follow-up: "Rewrite this so it gives a per-row running total even when timestamps tie." — Use `ROWS` instead of the default `RANGE`.

```
-- Default frame: RANGE, so tied timestamps share a running total
SELECT order_ts, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_ts) AS running_total
FROM orders;

-- Strict per-row stepping
SELECT order_ts, amount,
       SUM(amount) OVER (
         PARTITION BY customer_id ORDER BY order_ts, order_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total_strict
FROM orders;
```

Q74 · LAG and LEAD — Period-Over-Period, Default Value, Offset

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Cred, Meesho · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Explain `LAG` and `LEAD`. Write a query that shows each month's revenue alongside the previous month's revenue and the month-over-month growth percentage.

The 30-second answer: `LAG(col, offset, default)` returns the value `offset` rows before the current row in the ordered window; `LEAD` does the same forward. Default offset is 1; default "default value" is NULL. For month-over-month, aggregate revenue per month in a subquery, then `LAG(revenue) OVER (ORDER BY month)` to get the previous month, and divide.

Step-by-step thought process: 1. State the signature precisely: `LAG(expression, offset, default)`. Most candidates forget the `default` argument. 2. Explain why `default` matters: the first row's `LAG` is NULL by default, which makes the MoM growth NULL too — sometimes you want `default = 0` or `default = expression` instead. 3. Walk the query: aggregate monthly revenue in a CTE, then in the outer query compute `prev_rev = LAG(rev) OVER (ORDER BY month)` and `growth_pct = (rev - prev_rev) * 100.0 / NULLIF(prev_rev, 0)`. Always wrap the divisor in `NULLIF` to avoid divide-by-zero. 4. Mention `LEAD` use cases: "what is the next event's timestamp" for inter-event gaps and churn metrics.

Edge cases to mention: - `LAG` / `LEAD` ignore the frame clause; they only need `PARTITION BY` and `ORDER BY`. - Offset must be a non-negative integer; negative values are not allowed (use `LEAD` instead). - For MoM growth across missing months, you must first generate a contiguous month series — `LAG` does not skip gaps in your data.

What NOT to say: - Do not say "`LAG` always returns the previous row's value" — it returns the value `offset` rows back; without specifying offset you trust the default 1. - Do not divide by `prev_rev` directly — divide by `NULLIF(prev_rev, 0)` to handle the first row and any zero-revenue months.

Common follow-up: "What if the previous month is missing entirely?" — Generate a month spine via `generate_series` (Postgres) or a calendar table, left join your data, then apply `LAG` on the spine.

```
WITH monthly AS (  
  SELECT DATE_TRUNC('month', order_ts) AS month, SUM(amount) AS rev  
  FROM orders GROUP BY 1  
)  
SELECT month, rev,  
  LAG(rev) OVER (ORDER BY month) AS prev_rev,  
  (rev - LAG(rev) OVER (ORDER BY month)) * 100.0  
  / NULLIF(LAG(rev) OVER (ORDER BY month), 0) AS mom_growth_pct  
FROM monthly  
ORDER BY month;
```

Q75 · FIRST_VALUE and LAST_VALUE — The Default-Frame Trap

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Walmart Labs, Goldman · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Write a query to show each order alongside the first order amount and the last order amount for that customer. Be careful with `LAST_VALUE`.

The 30-second answer:

`FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_ts)` works as expected. `LAST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_ts)` does not — under the default frame (`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`), `LAST_VALUE` returns the current row's value, not the partition's last value. Fix: extend the frame to `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Step-by-step thought process: 1. Set up the trap exactly as the interviewer would: write the naive query, predict it returns the wrong number, explain why. 2. The "why": `LAST_VALUE` returns the last value in the frame, and the default frame ends at the current row. So at row 3 of a 10-row partition, the last value in the frame is row 3's amount. 3. The fix: add

`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to extend the frame to the entire partition. 4. Alternative fix: use

`FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_ts DESC)` — reverse the order, take the first. Often cleaner.

Edge cases to mention: - The trap does not happen with `FIRST_VALUE` because under the default frame `UNBOUNDED PRECEDING` is the actual partition start. - `NTH_VALUE(col, n)` has the same trap as `LAST_VALUE` — extend the frame explicitly. - Some engines (Snowflake) interpret `IGNORE NULLS` modifier on these functions; check before relying on it.

What NOT to say: - Do not claim "`LAST_VALUE` is broken in SQL" — it is following the spec exactly; the default frame just is not what most people expect. - Do not write `LAST_VALUE` with just `PARTITION BY ... ORDER BY ...` in production code — always pin the frame, or reverse the order and use `FIRST_VALUE`.

Common follow-up: "Why is this an interview favourite?" — Because it tests whether you understand frames at all. If you do not know about default frames, you cannot read this query.

```
-- WRONG: returns current row's amount as "last"
SELECT order_id, customer_id, amount,
       FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_ts) AS first_amt,
       LAST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_ts) AS last_amt_wrong
FROM orders;

-- RIGHT: extend the frame
SELECT order_id, customer_id, amount,
       FIRST_VALUE(amount) OVER (PARTITION BY customer_id ORDER BY order_ts) AS first_amt,
       LAST_VALUE(amount) OVER (
         PARTITION BY customer_id ORDER BY order_ts
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS last_amt
FROM orders;
```

Q76 · Running Totals — SUM() OVER (ORDER BY ...) Pattern

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Flipkart · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Show me a running total of daily revenue for the last 30 days.

The 30-second answer: Aggregate daily revenue first, then

`SUM(daily_rev) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. The explicit `ROWS` frame guarantees per-row stepping even if two rows somehow share a day.

Step-by-step thought process: 1. Two layers: aggregate to one row per day, then window. Trying to do both in one shot leads to confused queries. 2. Inside the window, specify the frame explicitly. Default `RANGE` works here because the `ORDER BY` column is unique per row after aggregation, but `ROWS` is more defensive. 3. To restrict to "last 30 days", filter `WHERE day >= CURRENT_DATE - INTERVAL '30 days'` in the outer query — but be aware this is the Filter-Window Trap territory (Q13). For a running total that includes the warm-up history, filter inside the CTE on a wider range. 4. Anticipate: "What if revenue is sparse — some days have zero orders?" Either left-join a calendar / `generate_series`, or accept that gaps in the data mean gaps in the running total's day labels.

Edge cases to mention: - Without `PARTITION BY`, the running total is across the entire result. Add `PARTITION BY` to reset per group (per customer, per region, per month). - `SUM` of a NULL amount contributes nothing — but the row still exists. To exclude entirely, filter in `WHERE`. - For huge tables, a running total over the full table can be expensive — index on the `ORDER BY` column and consider materialising daily aggregates first.

What NOT to say: - Do not use a correlated subquery (`SELECT ... WHERE day <= outer.day`) — it is the pre-2003 pattern and orders of magnitude slower. - Do not aggregate and window in one shot without thinking about what you want to group on; usually two layers is clearer.

Common follow-up: *"How would you reset the running total at the start of each month?"* — `PARTITION BY DATE_TRUNC('month', day)` .

```
WITH daily AS (  
  SELECT DATE(order_ts) AS day, SUM(amount) AS daily_rev  
  FROM orders  
  WHERE order_ts >= CURRENT_DATE - INTERVAL '60 days'  
  GROUP BY DATE(order_ts)  
)  
SELECT day, daily_rev,  
  SUM(daily_rev) OVER (  
    ORDER BY day  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
  ) AS running_total  
FROM daily  
WHERE day >= CURRENT_DATE - INTERVAL '30 days'  
ORDER BY day;
```

Q77 · Moving Averages

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, AmEx, Goldman · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Write a query that gives the 7-day moving average of daily revenue. What does "7-day" mean exactly?

The 30-second answer: Two valid interpretations: a 7-row trailing average (`ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`), or a 7-day calendar-time trailing average (`RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW`). They differ when days are missing — `ROWS` blindly walks 6 rows back; `RANGE` walks 6 days back regardless of whether intermediate days exist in the data.

Step-by-step thought process: 1. Force the interviewer to clarify "7-day": rows or calendar days? In dashboards it almost always means calendar days, especially for sparse data. 2. Aggregate to per-day revenue first (one row per day). 3. Apply `AVG(daily_rev) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` for the trailing 7 rows. If days can be missing, swap to `RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW` (Postgres / Snowflake / BigQuery) — or left-join a calendar spine first

and stay on **ROWS** . 4. Be ready for "give me a centered moving average" — **ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING** . Centered is better for visualisations, trailing is better for live ops dashboards.

Edge cases to mention: - For the first 6 days the trailing window has fewer than 7 rows; **AVG** divides by the actual count, not by 7. If you want NULL for "not enough data yet", wrap in **CASE WHEN COUNT(*) OVER (...) < 7 THEN NULL ELSE AVG(...) END** . - Centered MA leaks future data into past rows — fine for retrospective charts, wrong for any predictive use. - On MySQL 8, **RANGE** with an interval offset works only for numeric columns; for date columns convert to epoch first.

What NOT to say: - Do not call a 7-row average a "7-day average" when your data has missing days — it is wrong by definition. - Do not use a correlated subquery for moving averages; performance is unacceptable on real data.

Common follow-up: "How would you compute a 28-day moving average that only emits when 28 days of data exist?" — Wrap in **CASE WHEN COUNT(*) OVER (...) = 28 THEN AVG(...) END** .

```
WITH daily AS (  
  SELECT DATE(order_ts) AS day, SUM(amount) AS daily_rev  
  FROM orders GROUP BY 1  
)  
SELECT day, daily_rev,  
  AVG(daily_rev) OVER (  
    ORDER BY day  
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
  ) AS ma_7d  
FROM daily  
ORDER BY day;
```

Q78 · NTILE — Bucketing and Unequal-Sized Buckets

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Use **NTILE** to split customers into 4 spend quartiles. What happens if the row count is not divisible by 4?

The 30-second answer: **NTILE(4) OVER (ORDER BY total_spend DESC)** assigns each row a bucket number 1 through 4. When row count is not divisible, the earlier buckets get one extra row each. With 10 rows split into 4 buckets you get sizes (3, 3, 2, 2), not (3, 3, 3, 1) and not (2, 2, 3, 3).

Step-by-step thought process: 1. Lead with the bucket-size rule — that is the trick question. Most candidates know the syntax but not the distribution. 2. Walk through the math: with N rows and B

buckets, the first `N mod B` buckets get `ceil(N/B)` rows; the rest get `floor(N/B)` rows. 3. Mention the alternative: `WIDTH_BUCKET` (Postgres / Oracle) which splits by value range rather than row count. Use `WIDTH_BUCKET` when you want equal value ranges; use `NTILE` when you want roughly equal row counts. 4. For deciles, use `NTILE(10)`. For percentiles in segmentation reports, `NTILE(100)` is standard.

Edge cases to mention: - `NTILE` requires `ORDER BY` and is sensitive to ties — tied rows can land in different buckets depending on the tie-breaker. - For exact percentile values use `PERCENTILE_CONT` or `PERCENT_RANK`; `NTILE` is for bucketing, not for the actual percentile value. - With fewer rows than buckets (`NTILE(10)` on 5 rows), each row gets its own bucket and buckets 6-10 stay empty.

What NOT to say: - Do not say "buckets are always equal size" — they are not when divisibility fails. - Do not use `NTILE` for marketing percentiles where the underlying values matter; you will misclassify outliers.

Common follow-up: "Customers in the top decile of spend?" — `WHERE bucket = 1` after `NTILE(10) OVER (ORDER BY total_spend DESC)`.

```
WITH spend AS (  
  SELECT customer_id, SUM(amount) AS total_spend  
  FROM orders GROUP BY customer_id  
)  
SELECT customer_id, total_spend,  
  NTILE(4) OVER (ORDER BY total_spend DESC) AS spend_quartile  
FROM spend;
```

Q79 · The Filter-Window Trap

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Goldman, Walmart Labs, Cred · Topic: Window Functions

The question (interviewer's verbatim phrasing):

I wrote `SELECT ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn FROM events WHERE event_ts >= '2026-04-01'`. Why are some users missing rows that I expected to see?

The 30-second answer: Because `WHERE` is evaluated before window functions. The filter throws away every event before April 1; the window then computes `ROW_NUMBER` over only the surviving rows, so a user whose latest event was in March is gone entirely. Fix: filter inside a CTE or subquery so the window sees the full history, or use `QUALIFY` on Snowflake / BigQuery / Teradata to filter on the window result.

Step-by-step thought process: 1. State the SQL evaluation order so the interviewer knows you are not guessing: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → window functions → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Window functions sit between `HAVING` and `SELECT`. 2. Show what that means for the query: the April filter culls the dataset; the window sees only April-onward rows; users with only pre-April events vanish; users with both pre-April and April events get an `rn = 1` that points to their first April event, not their lifetime first event. 3. Two fixes. (a) Apply the window inside a CTE on the full table, then filter in the outer query — but only filter on plain columns, not on the window column. (b) On Snowflake / BigQuery / Teradata, use `QUALIFY` which filters on the window result without needing a subquery. 4. Anticipate the deeper version: "I want the latest event per user but only consider April events" — that is genuinely a window-over-filtered-rows scenario and the original query was correct for that intent. Always clarify the intent before fixing.

Edge cases to mention: - `HAVING` is also before window functions — you cannot reference a window in `HAVING` either. - `ORDER BY` and `LIMIT` are after windows, so you can `ORDER BY` a window column and `LIMIT` on the result. - Putting a window function in `WHERE` is a syntax error on every major engine — the error message points you straight at this evaluation-order rule.

What NOT to say: - Do not say "just add a `WHERE rn = 1`" without restructuring — that is the second half of the bug, see Q15. - Do not assume the user wanted lifetime-first or April-first; ask. Different fixes for different intents.

Common follow-up: "Show me the CTE rewrite and the `QUALIFY` rewrite." — Both below.

```
-- Bug: window evaluated AFTER WHERE strips March data
SELECT user_id, event_ts,
       ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn
FROM events
WHERE event_ts >= '2026-04-01';

-- Fix 1: CTE, then filter on the window result in the outer query
WITH ranked AS (
  SELECT user_id, event_ts,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn
  FROM events
)
SELECT * FROM ranked WHERE rn = 1 AND event_ts >= '2026-04-01';

-- Fix 2: QUALIFY (Snowflake / BigQuery / Teradata)
SELECT user_id, event_ts,
       ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn
FROM events
QUALIFY rn = 1;
```

Q80 · The QUALIFY Clause — Filter on Window Result

Tags: Difficulty: Medium · Asked at: Razorpay (data-eng panel), Walmart Labs, Meesho · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Have you used `QUALIFY` ? Where is it supported and what does it save you?

The 30-second answer: `QUALIFY` is to window functions what `HAVING` is to aggregates — it filters on the window result without a CTE or subquery. Supported on Snowflake, BigQuery, Teradata, and recent Databricks. Not on Postgres, MySQL, SQL Server, or Oracle.

Step-by-step thought process: 1. Frame it by analogy: `WHERE` → row filter; `HAVING` → group filter; `QUALIFY` → window filter. The mental model lands immediately. 2. Show the common dedup pattern: `SELECT * FROM events QUALIFY ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) = 1`. No CTE, no subquery. 3. Call out engine support precisely so you do not look ignorant: Snowflake, BigQuery, Teradata, Databricks. Everywhere else, fall back to a CTE wrapper. 4. Mention you can reference the alias: `QUALIFY rn = 1` works if you defined `rn` in the `SELECT`. Cleaner than restating the whole window expression.

Edge cases to mention: - You can chain `QUALIFY` with `WHERE` and `HAVING`; they apply in their normal order, `QUALIFY` last. - `QUALIFY` is evaluated after window functions, so it can reference any window result. Do not try to use it on plain columns where `WHERE` belongs. - On engines without `QUALIFY`, the CTE wrapper is the idiomatic replacement — not a correlated subquery.

What NOT to say: - Do not claim `QUALIFY` is standard SQL — it is not; it is a vendor extension that has spread. - Do not use `QUALIFY` in queries that need to run on Postgres or MySQL; you will burn a deployment cycle.

Common follow-up: "Rewrite the dedup query for Postgres." — CTE with `ROW_NUMBER`, see Q2.

```
-- Snowflake / BigQuery / Teradata
SELECT user_id, event_ts, event_type,
       ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn
FROM events
QUALIFY rn = 1;
```

Q81 · Why You Cannot Put a Window Function in WHERE

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Goldman, PhonePe · Topic: Window Functions

The question (interviewer's verbatim phrasing):

I tried `WHERE ROW_NUMBER() OVER (...) = 1` and got an error. Why doesn't this work, and what's the alternative on Postgres?

The 30-second answer: Because `WHERE` runs before window functions are evaluated — there is nothing to filter on yet. The alternatives are a CTE or subquery wrapping the window, then filtering on the window column in the outer query; or `QUALIFY` on engines that support it.

Step-by-step thought process: 1. Restate the evaluation order rule from Q13 — `WHERE` is before windows, so windows do not exist yet at `WHERE` time. 2. Give the canonical CTE rewrite for Postgres / MySQL / SQL Server / Oracle: define `rn` in a CTE, filter on `rn = 1` outside. 3. Mention `LATERAL / CROSS APPLY` as alternative on engines that support them, useful when you want top-N per group with extra computed columns. 4. Anticipate "Why not just nest the window?" — you literally cannot; same error from the same rule.

Edge cases to mention: - The same rule applies in `HAVING` — `HAVING` is also before windows. - Some engines give a clearer error message than others; Postgres says "window functions are not allowed in WHERE". - You can reference window aliases in `ORDER BY` and `LIMIT` because those run after windows.

What NOT to say: - Do not try to bypass with a CASE expression in `WHERE` — same error, same rule. - Do not write a correlated subquery to "filter by rank" — use the CTE pattern.

Common follow-up: "Top 3 highest-paid employees per department on Postgres?" — `WITH ranked AS (SELECT ..., RANK() OVER (PARTITION BY dept ORDER BY salary DESC) r FROM emp) SELECT * FROM ranked WHERE r <= 3`. See Q17.

```
-- Wrong
SELECT * FROM events
WHERE ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) = 1;
-- ERROR: window functions are not allowed in WHERE

-- Right (works on every engine)
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn
  FROM events
)
SELECT * FROM ranked WHERE rn = 1;
```

Q82 · PERCENT_RANK and CUME_DIST

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, AmEx · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Explain `PERCENT_RANK` and `CUME_DIST`. When would you use each?

The 30-second answer: $\text{PERCENT_RANK}(x) = (\text{RANK}(x) - 1) / (N - 1)$ — relative rank in $[0, 1]$, with the top row at 0. $\text{CUME_DIST}(x) = (\text{number of rows} \leq x) / N$ — cumulative distribution in $(0, 1]$. Use `PERCENT_RANK` when you want "this employee is in the top 5%"; use `CUME_DIST` when you want "this employee earns more than 90% of peers."

Step-by-step thought process: 1. Walk the formulas with a 5-row example so the difference is concrete. Scores `[100, 90, 90, 80, 70]`. `RANK` is `[1, 2, 2, 4, 5]`. `PERCENT_RANK` is `[0, 0.25, 0.25, 0.75, 1]`. `CUME_DIST` is `[0.2, 0.6, 0.6, 0.8, 1]`. 2. Highlight the boundary behaviour: `PERCENT_RANK` includes 0 and 1; `CUME_DIST` excludes 0 but includes 1. 3. Practical use: percentile-based filtering (`WHERE pr <= 0.05` for top 5%) typically uses `PERCENT_RANK`; "what fraction of population is at or below me" uses `CUME_DIST`. 4. Both ignore the frame clause. Both require `ORDER BY` inside `OVER`.

Edge cases to mention: - With a single-row partition, `PERCENT_RANK` is 0 (denominator collapses to 0 by formula but the spec says 0); `CUME_DIST` is 1. - Ties: both return the same value for tied rows. - For exact percentile values (50th, 95th), use `PERCENTILE_CONT` / `PERCENTILE_DISC` (Q30), not these ranking functions.

What NOT to say: - Do not say `PERCENT_RANK` and `CUME_DIST` are the same — the formulas and the boundary inclusion differ. - Do not use these for "give me the 90th percentile value" — that is what `PERCENTILE_CONT` does.

Common follow-up: "Top 5% of customers by spend?" — `WHERE pr <= 0.05` over `PERCENT_RANK() OVER (ORDER BY spend DESC)`.

```
SELECT customer_id, total_spend,
       PERCENT_RANK() OVER (ORDER BY total_spend DESC) AS pr,
       CUME_DIST()    OVER (ORDER BY total_spend DESC) AS cd
FROM customer_spend;
```

Q83 · Top-N Per Group

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Cred, Flipkart, Meesho · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Top 3 highest-paid employees per department. Write it.

The 30-second answer: Use `ROW_NUMBER`, `RANK`, or `DENSE_RANK` partitioned by department and ordered by salary descending, wrap in a CTE, filter on the rank column outside. Choice depends on tie behaviour: `ROW_NUMBER` gives exactly 3 rows per department even with ties; `RANK` and `DENSE_RANK` may return more than 3 if salaries tie at the boundary.

Step-by-step thought process: 1. Lead with the function choice. Interviewers love asking "what if two people are tied at salary number 3?". `ROW_NUMBER` picks one arbitrarily (or by tie-breaker); `RANK` / `DENSE_RANK` keeps both — 4 rows for that department. 2. Write the CTE form (works on every engine): `WITH ranked AS (SELECT ..., DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) r FROM emp) SELECT * FROM ranked WHERE r <= 3;` 3. On Snowflake / BigQuery, use `QUALIFY` to skip the CTE. 4. Anticipate "What if I just need fast top-N per group without ranking semantics?" — `LATERAL` join with `LIMIT 3` on Postgres, or `CROSS APPLY` on SQL Server. Sometimes faster than the window approach on huge tables with deep partitions.

Edge cases to mention: - Always add a tie-breaker in `ORDER BY` (e.g. `ORDER BY salary DESC, emp_id`) to keep `ROW_NUMBER` deterministic. - "Top 3" with `DENSE_RANK` can return many more than 3 rows if 3 distinct salary values each have multiple people. - For "top 3 by department, but only departments with at least 5 employees", add `HAVING COUNT(*) >= 5` in a grouping step or filter on a window `COUNT(*) OVER (PARTITION BY dept)` in the outer query.

What NOT to say: - Do not say "limit 3 per group is a one-liner" — it is not in standard SQL without windows or lateral joins. - Do not use a self-join (the pre-window pattern) — quadratic in partition size and embarrassing in 2026.

Common follow-up: "Now give me only the single top earner per department." — Same shape, `WHERE r = 1`. Same dedup pattern as Q2.

```
WITH ranked AS (  
  SELECT emp_id, dept, salary,  
         DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS r  
  FROM employees  
)  
SELECT * FROM ranked WHERE r <= 3  
ORDER BY dept, r, emp_id;
```

Q84 · Gaps and Islands

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Goldman, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Given a table of daily user logins, find the longest continuous login streak per user.

The 30-second answer: Compute `login_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) * INTERVAL '1 day'` — consecutive dates produce the same anchor, so group by `(user_id, anchor)` to find each streak, then take the longest per user.

Step-by-step thought process: 1. Explain the trick: if logins are consecutive, then `login_date - row_number * 1 day` is constant within a streak. Different streaks get different anchors. This is the classic "gaps and islands" technique. 2. Step 1 — assign row numbers per user ordered by date. Step 2 — compute the anchor by subtracting `row_number` days from the date. Step 3 — group by `(user_id, anchor)` and count rows to get streak length. Step 4 — `MAX(streak_length) OVER (PARTITION BY user_id)` or `ORDER BY + LIMIT 1` per user for the longest. 3. Variation: detect islands of "active" status in time-series — same pattern with `DENSE_RANK` over status changes. 4. Anticipate "what about gaps of 1 day still counting"? — adjust the granularity (subtract 2 days, etc.) or use `LAG` to detect actual gap size.

Edge cases to mention: - Duplicate logins on the same date corrupt the trick; dedup first with `SELECT DISTINCT user_id, login_date`. - For non-date sequences (gap = integer ID), the same idea works: `id - ROW_NUMBER()` is constant within a run. - For complex islands (status changes), use the "tabibitosan" / `LAG` + cumulative sum variant where you mark each row as "new island = 1" and `SUM() OVER (...)` to assign island IDs.

What NOT to say: - Do not write nested correlated subqueries — they will time out on real data. - Do not assume dates are gap-free in the first place; check for duplicates and missing days upfront.

Common follow-up: "What if I want the current ongoing streak only?" — Filter to the island that contains today's date.

```
WITH numbered AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)) * INTERVAL '1 day' AS anchor
  FROM (SELECT DISTINCT user_id, login_date FROM logins) d
),
streaks AS (
  SELECT user_id, anchor, COUNT(*) AS streak_len,
         MIN(login_date) AS streak_start, MAX(login_date) AS streak_end
  FROM numbered
  GROUP BY user_id, anchor
)
SELECT user_id, streak_start, streak_end, streak_len
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY streak_len DESC) AS rn
  FROM streaks
) ranked
WHERE rn = 1;
```


Q85 · Sessionization — Assigning Session IDs to Event Streams

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, PhonePe, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

You have a user-events table. Assign session IDs where a new session starts whenever the gap between consecutive events for a user exceeds 30 minutes.

The 30-second answer: For each event, compute

`LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)` to get the previous event. Mark a new session whenever `event_ts - prev_ts > INTERVAL '30 minutes'` or `prev_ts IS NULL`. Then `SUM(new_session_flag) OVER (PARTITION BY user_id ORDER BY event_ts)` is the session number within the user.

Step-by-step thought process: 1. Step 1 — `LAG` gives the previous event timestamp per user. Step 2 — flag = 1 when the gap is too big (or first event), else 0. Step 3 — running sum of the flag gives sequential session IDs per user. Step 4 — concat with `user_id` if you want globally unique session IDs. 2. Specify the frame on the running sum implicitly (`ORDER BY` + default `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`) — correct because the flag is per row and we want cumulative. 3. Variations: 30-minute gap → product-level rule; "new session per day" → flag on date change; "new session per visit" → combine multiple flags with OR. 4. For performance on huge event streams, partition by user is necessary; without it the engine processes the entire table in one window.

Edge cases to mention: - Two events at the exact same timestamp produce gap = 0 — they belong to the same session. - A user's first event always has `prev_ts IS NULL`, which the flag treats as a new session — correct. - If you sessionise across `user_id` partitions and there are users with millions of events, watch sort cost — index `(user_id, event_ts)` helps.

What NOT to say: - Do not use a window with `RANGE` interval frame here — `LAG` + cumulative-sum is the standard pattern, more readable and engine-agnostic. - Do not forget the `PARTITION BY user_id` — without it sessions bleed across users.

Common follow-up: "Now show me the duration of each session." — Group by `(user_id, session_id)` and take `MAX(event_ts) - MIN(event_ts)`.

```
WITH flagged AS (  
  SELECT user_id, event_ts, event_type,  
    CASE  
      WHEN event_ts - LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)  
        > INTERVAL '30 minutes'  
      OR LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts) IS NULL  
      THEN 1 ELSE 0  
    END AS new_session  
  FROM events  
)  
SELECT user_id, event_ts, event_type,  
  SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_ts) AS session_id  
FROM flagged;
```

Q86 · Window Functions on Huge Tables — Sort Cost and Parallel Window

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Window functions get expensive on large tables. What drives the cost and what can you do about it?

The 30-second answer: Cost is dominated by the sort needed to satisfy `PARTITION BY ... ORDER BY ...`. If a covering index matches the partition+order keys, the planner skips the sort. Otherwise, expect an in-memory or on-disk sort proportional to $N \log N$. Modern engines (Postgres 14+, Snowflake, BigQuery, SQL Server) can parallelise window evaluation across partitions but not within a single partition.

Step-by-step thought process: 1. Identify the sort as the cost driver. Explain that each `(PARTITION BY, ORDER BY)` combination corresponds to a sort key; the planner does one sort per distinct window spec, sharing where possible. 2. Suggest using the same window spec across multiple expressions in the query so the planner sorts once — that is one of the biggest wins in real-world tuning. 3. Indexing: a btree index on `(partition_keys, order_keys)` can let the planner stream rows in order without sorting. Verify with `EXPLAIN ANALYZE`. 4. Parallel: Postgres 14+ parallelises across partitions; one giant partition gets one worker. Re-partition by adding a secondary partition key if the data is skewed.

Edge cases to mention: - Window functions cannot be pushed down past `GROUP BY` — if you can pre-aggregate, do it first; aggregating then windowing is almost always cheaper than the reverse. - `RANGE` with date-interval offsets can defeat parallelisation on some engines; `ROWS` is friendlier. - Materialise large window results into a temp table or CTAS if reused multiple times — windows are not free to recompute.

What NOT to say: - Do not say "indexes do not help windows" — they help when they match the sort order. - Do not blame the window for poor performance without checking **EXPLAIN** first — sometimes the cost is in the underlying scan or join.

Common follow-up: "How would you know if the planner skipped the sort?" — **EXPLAIN ANALYZE** shows **Sort** or **Index Scan** above the **WindowAgg** node.

```
-- Use one window spec for two expressions; planner sorts once
SELECT order_id, customer_id, amount,
       SUM(amount) OVER w AS running_total,
       ROW_NUMBER() OVER w AS rn
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts);
```

Q87 · Combining Window with GROUP BY — Running Totals Alongside Group Totals

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, AmEx · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Show monthly revenue for the year alongside the running year-to-date revenue, in one query.

The 30-second answer: Aggregate to monthly revenue with **GROUP BY** first, then apply a running **SUM() OVER (ORDER BY month)** on the aggregated rows. Window functions evaluate after **GROUP BY**, so this pipeline is natural — group first, window second, all in one query.

Step-by-step thought process: 1. Lay out the evaluation order again: **GROUP BY** happens before window functions, so the window sees one row per group. 2. Write the query as a single **SELECT**: **SELECT month, SUM(amount) AS monthly_rev, SUM(SUM(amount)) OVER (ORDER BY month) AS ytd FROM ... GROUP BY month**. Yes, **SUM(SUM(amount))** is legal — outer **SUM** is the window aggregate, inner **SUM** is the group aggregate. 3. If the **SUM(SUM(...))** syntax bothers reviewers, use a CTE: aggregate in CTE 1, window in the outer query. Same result, more readable. 4. Anticipate "what if I want year-over-year too?" — add **LAG(monthly_rev, 12) OVER (ORDER BY month)** for same-month-last-year.

Edge cases to mention: - The window's **ORDER BY** is independent of the outer **ORDER BY** — sort the final output separately if needed. - For YTD across multiple years, partition by year: **SUM(SUM(amount)) OVER (PARTITION BY EXTRACT(year FROM month) ORDER BY month)**. - Sparse months: if there are months with zero revenue, they will not appear unless you join a month spine first.

What NOT to say: - Do not say "you cannot mix `GROUP BY` and window functions" — you absolutely can; they compose cleanly. - Do not put the window inside the `GROUP BY` clause — it has to be in `SELECT`.

Common follow-up: "Same query but reset YTD at the start of each year." — `PARTITION BY EXTRACT(year FROM month)`.

```
SELECT DATE_TRUNC('month', order_ts) AS month,
       SUM(amount) AS monthly_rev,
       SUM(SUM(amount)) OVER (
         PARTITION BY EXTRACT(year FROM order_ts)
         ORDER BY DATE_TRUNC('month', order_ts)
       ) AS ytd_rev
FROM orders
GROUP BY 1, EXTRACT(year FROM order_ts)
ORDER BY month;
```

Q88 · Cumulative Distinct Count

Tags: Difficulty: Hard · Asked at: Razorpay, JP Morgan, Goldman, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Show the cumulative count of distinct users seen by each day. There is no `COUNT(DISTINCT) OVER ()` — how do you do it?

The 30-second answer: Two options. (a) Find each user's first-seen date, then group by that date to get new users per day, then `SUM(new_users) OVER (ORDER BY day)` gives cumulative distinct. (b) On Postgres / Snowflake / BigQuery use approximate sketches like `HLL` for streaming distinct counts. Vanilla `COUNT(DISTINCT col) OVER (ORDER BY ...)` is not supported.

Step-by-step thought process: 1. State the limitation upfront so the interviewer knows you have hit this wall before: most engines do not allow `COUNT(DISTINCT) OVER` with `ORDER BY` (Postgres rejects it; BigQuery has limited support; Snowflake supports it but with caveats). 2. The first-seen trick: `MIN(event_ts) AS first_seen GROUP BY user_id`, then aggregate by `DATE(first_seen)` to get new-user counts, then cumulate. This is exact, fast, and works everywhere. 3. For approximate at scale, `HLL_COUNT.MERGE_PARTIAL` on BigQuery or `APPROX_COUNT_DISTINCT` with HLL state on Snowflake lets you maintain running distinct estimates with bounded error. 4. Anticipate "What about distinct per rolling window, not cumulative?" — same approach with first-seen-within-window or HLL sketches per window.

Edge cases to mention: - Snowflake supports `COUNT(DISTINCT) OVER ()` without `ORDER BY` (partition-level distinct) but not as a running count. - For "distinct active users in the last 7 days each day", the first-seen trick does not work — you need either a self-join, a calendar+exists pattern, or HLL. - On data warehouses the HLL approach is dominant because exact cumulative distinct does not scale.

What NOT to say: - Do not write `COUNT(DISTINCT user_id) OVER (ORDER BY day)` and assume it works — Postgres throws an error, others give surprising answers. - Do not use a correlated subquery for cumulative distinct; on a billion-row table it never returns.

Common follow-up: "What's the error class of HLL?" — Typically ~1-2% relative error, configurable on most engines.

```
-- Exact cumulative distinct via first-seen trick
WITH first_seen AS (
  SELECT user_id, MIN(DATE(event_ts)) AS first_day
  FROM events
  GROUP BY user_id
),
daily_new AS (
  SELECT first_day, COUNT(*) AS new_users
  FROM first_seen
  GROUP BY first_day
)
SELECT first_day AS day, new_users,
       SUM(new_users) OVER (ORDER BY first_day) AS cumulative_distinct_users
FROM daily_new
ORDER BY first_day;
```

Q89 · Window Functions in DML — UPDATE Using Subquery With Window

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Set a `is_latest = TRUE` flag on the latest event per user in the `events` table. Use a window function.

The 30-second answer: Window functions are not allowed directly in `UPDATE SET` expressions — compute them in a subquery / CTE and join back. On Postgres: `UPDATE events e SET is_latest = TRUE FROM (SELECT event_id FROM (SELECT event_id, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY event_ts DESC) AS rn FROM events) t WHERE rn = 1) latest WHERE e.event_id = latest.event_id;`

Step-by-step thought process: 1. State the rule: most engines reject `UPDATE ... SET col = ROW_NUMBER() OVER (...)` directly. Wrap in a subquery / CTE. 2. Pattern: subquery / CTE returns the row identifiers (or values) you want to apply, the outer `UPDATE` joins on the primary key and applies the change. 3. On Postgres use `UPDATE ... FROM` ; on SQL Server use `UPDATE ... FROM ... JOIN` ; on MySQL 8 use `UPDATE ... JOIN` ; on Oracle use a merge or a correlated subquery. 4. Reset first if you are recomputing: `UPDATE events SET is_latest = FALSE;` then set TRUE on the new latest, otherwise stale TRUE flags hang around.

Edge cases to mention: - Wrap in a transaction so the reset and set are atomic — otherwise concurrent reads can see no row as latest. - For very large tables, consider an indexed boolean column and batch updates by user; full-table updates can be a long-running write. - If multiple events tie on `event_ts` , `ROW_NUMBER` picks one arbitrarily — add a tie-breaker (`ORDER BY event_ts DESC, event_id DESC`) to make this deterministic.

What NOT to say: - Do not write

`UPDATE events SET is_latest = ROW_NUMBER() OVER (...) = 1` and hope — this is rejected on every engine. - Do not skip the reset-to-FALSE step in a recompute scenario; you will end up with multiple "latest" rows per user over time.

Common follow-up: "How would you do this on MySQL 8?" —

`UPDATE events e JOIN (subquery with row_number) latest ON e.event_id = latest.event_id SET e.is_latest = TRUE .`

```
-- Postgres
BEGIN;
UPDATE events SET is_latest = FALSE;
UPDATE events e
SET is_latest = TRUE
FROM (
    SELECT event_id FROM (
        SELECT event_id,
            ROW_NUMBER() OVER (
                PARTITION BY user_id ORDER BY event_ts DESC, event_id DESC
            ) AS rn
        FROM events
    ) ranked WHERE rn = 1
    ) latest
WHERE e.event_id = latest.event_id;
COMMIT;
```

Q90 · Multiple Windows in One Query — Named WINDOW Clause for Readability

Tags: Difficulty: Easy · Asked at: Infosys, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

You have three window functions in one query that all use the same partition and order. How do you avoid repeating the `OVER` clause?

The 30-second answer: Use the named `WINDOW` clause: `... OVER w ... OVER w ... OVER w ... FROM table WINDOW w AS (PARTITION BY x ORDER BY y)`. Defines the window spec once and references it by name. Supported on Postgres, MySQL 8+, SQL Server (via syntax differences), Oracle, BigQuery, Snowflake.

Step-by-step thought process: 1. Show the verbose version first — three `OVER (PARTITION BY ... ORDER BY ...)` repetitions — so the interviewer sees the pain. 2. Refactor with the `WINDOW` clause. The clause goes between `HAVING` and `ORDER BY` in the query. 3. Beyond readability, this can hint the planner that the three windows share a sort — though most modern planners detect this anyway. 4. Show a variant: `WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts), w_unordered AS (PARTITION BY customer_id)` for two related but distinct windows.

Edge cases to mention: - The `WINDOW` clause name must be a simple identifier; reserved keywords blow up. - Named windows can inherit from each other: `WINDOW w1 AS (PARTITION BY x), w2 AS (w1 ORDER BY y)` — useful for hierarchical window specs. - On engines that do not support named windows in older versions (very old MySQL), fall back to repeated `OVER` clauses.

What NOT to say: - Do not call it "the WINDOW keyword in `OVER`" — the `WINDOW` clause is its own thing, separate from `OVER`. - Do not assume named windows change semantics — they are purely a syntactic shortcut.

Common follow-up: "Can you reference the same window in `OVER` and override the frame?" — Yes: `OVER (w ROWS BETWEEN ... )` inherits partition and order, overrides frame.

```
SELECT order_id, customer_id, amount,
       SUM(amount) OVER w AS running_total,
       AVG(amount) OVER w AS running_avg,
       ROW_NUMBER() OVER w AS rn
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_ts);
```

Q91 · Window vs Aggregate Within the Same Query — Conceptual Difference

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Accenture · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Conceptually, how is `SUM(amount)` in `SELECT ... GROUP BY` different from `SUM(amount) OVER (PARTITION BY ...)` in the same `SELECT` ?

The 30-second answer: The first reduces the input to one row per group and emits the group sum. The second keeps every input row and emits the group sum on each row. Same numeric answer per group, different row count. Output cardinality is the entire difference.

Step-by-step thought process: 1. Map the difference to row count, again. This is the recurring theme for windows: rows preserved, not collapsed. 2. Show two queries side by side:

`SELECT customer_id, SUM(amount) FROM orders GROUP BY customer_id` (one row per customer) versus `SELECT order_id, customer_id, amount, SUM(amount) OVER (PARTITION BY customer_id) FROM orders` (one row per order). 3. Practical hook: window aggregates are how you compute ratios — `amount * 1.0 / SUM(amount) OVER (PARTITION BY customer_id) AS share_of_customer` is one row per order with the share filled in. 4. Mention that you can run both in one query — group at the top level for summary totals, and the window inside a subquery for row-level metrics — but typically you choose one or the other per use case.

Edge cases to mention: - Window aggregates respect frame; group aggregates do not have a frame. - `DISTINCT` inside group aggregate (`COUNT(DISTINCT x)`) works; inside window aggregate it is restricted or unsupported on most engines (Q22). - `FILTER (WHERE ...)` clause works on both group and window aggregates on Postgres / SQL standard, useful for conditional sums.

What NOT to say: - Do not say "they always give the same answer" — same per-group total, completely different row count, which can mean completely different downstream behaviour. - Do not pick a window when `GROUP BY` is what you need — windows on full tables when you just want a summary are wasteful.

Common follow-up: "Customer share of total revenue across all customers?" — `amount * 1.0 / SUM(amount) OVER ()` (no partition = full table).

```
-- Group aggregate: one row per customer
SELECT customer_id, SUM(amount) AS cust_total
FROM orders GROUP BY customer_id;

-- Window aggregate: one row per order
SELECT order_id, customer_id, amount,
       SUM(amount) OVER (PARTITION BY customer_id) AS cust_total,
       amount * 1.0 / SUM(amount) OVER (PARTITION BY customer_id) AS share_of_customer
FROM orders;
```

Q92 · Median via PERCENTILE_CONT — Ordered-Set Aggregate vs Window

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan, AmEx · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Compute the median order amount per customer. Then compute the running median over the last 100 orders. Walk me through both.

The 30-second answer: Median per group: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) GROUP BY customer_id` — ordered-set aggregate, returns one row per customer. Running median: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) OVER (PARTITION BY customer_id ORDER BY order_ts ROWS BETWEEN 99 PRECEDING AND CURRENT ROW)` — same function used as a window aggregate on engines that support it (Oracle, Snowflake, Redshift). Postgres supports the grouped form but not the windowed form natively.

Step-by-step thought process: 1. Distinguish the two forms: ordered-set aggregate (`WITHIN GROUP ...` with `GROUP BY`) and ordered-set window aggregate (`WITHIN GROUP ...` with `OVER`). 2. For per-customer median, every engine supports the grouped form. Show it. 3. For running median, vendor support diverges. Oracle, Snowflake, Redshift, SQL Server 2012+ support it as a window function (in SQL Server, `PERCENTILE_CONT` is window-only — it requires `OVER`). Postgres has it as a grouped aggregate only — for running median you need a CTE that windows the data then aggregates, or a third-party extension. 4. Alternative: `PERCENTILE_DISC` returns an actual data value (the row whose cumulative distribution crosses 0.5); `PERCENTILE_CONT` interpolates between values. For symmetric numeric data, both are usually close.

Edge cases to mention: - Median of NULLs is undefined; `PERCENTILE_CONT` ignores NULLs. - On a window with a small frame, both `PERCENTILE_CONT` and `PERCENTILE_DISC` work but the result can swing widely with frame size — pick a frame that matches the analytic intent. - For median across a huge table, approximate medians via `APPROX_PERCENTILE` are often required for latency.

What NOT to say: - Do not write `MEDIAN(amount)` and hope it works everywhere — only some engines have a `MEDIAN` function; standard is `PERCENTILE_CONT(0.5)`. - Do not use `PERCENTILE_CONT` as a window on Postgres in production — check support first.

Common follow-up: "What if you only have Postgres and want a running median?" — Pre-compute via CTE per window position, or use an extension like `pg_median` / `tdigest`.

```
-- Median per customer (Postgres, Oracle, Snowflake – NOT SQL Server, which is window-only)
SELECT customer_id,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median_amount
FROM orders
GROUP BY customer_id;

-- Running median over last 100 orders (Snowflake / Oracle / Redshift)
SELECT order_id, customer_id, amount,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)
       OVER (PARTITION BY customer_id ORDER BY order_ts
            ROWS BETWEEN 99 PRECEDING AND CURRENT ROW) AS running_median_100
FROM orders;
```

Q93 · NTH_VALUE and the IGNORE NULLS Modifier

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, Walmart Labs · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Use `NTH_VALUE` to get each customer's third order amount. What is the `IGNORE NULLS` modifier and when does it matter?

The 30-second answer: `NTH_VALUE(amount, 3) OVER (PARTITION BY customer_id ORDER BY order_ts ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)` returns the third-ordered amount for each row in the customer's partition. The frame must span the whole partition (same trap as `LAST_VALUE`). `IGNORE NULLS` skips NULL values when counting positions — supported on Oracle, Snowflake, Postgres 16+, SQL Server 2022+, Redshift.

Step-by-step thought process: 1. Lead with the frame caveat: `NTH_VALUE` shares the `LAST_VALUE` trap. Under the default frame, you may not see what you expect because the frame ends at the current row. 2. Show the corrected query with explicit `UNBOUNDED PRECEDING TO UNBOUNDED FOLLOWING`. 3. Explain `IGNORE NULLS`: if your `amount` column can be NULL, the default `RESPECT NULLS` counts NULLs in the position; `IGNORE NULLS` skips them. 4. Anticipate "What about FROM LAST?" — `NTH_VALUE(amount, 3) FROM LAST OVER (...)` returns the third-to-last; supported on Oracle and some others.

Edge cases to mention: - If the partition has fewer than `n` rows, `NTH_VALUE` returns NULL. - `FROM FIRST` (the default) versus `FROM LAST` (Oracle and friends) — counts from start or from end. - `IGNORE NULLS` is also useful on `LAG` / `LEAD` / `FIRST_VALUE` / `LAST_VALUE` — for filling forward, `LAG(col IGNORE NULLS) OVER (ORDER BY ts)` is the canonical pattern on engines that support it.

What NOT to say: - Do not use `NTH_VALUE` without pinning the frame; you will get NULL or the wrong row. - Do not assume `IGNORE NULLS` is universal — older Postgres (pre-16) does not have it; you need a workaround with a CTE that filters NULLs first.

Common follow-up: *"Forward-fill missing values on Postgres."* — Use the "last seen non-null" trick with `COUNT(value) OVER (ORDER BY ts) AS grp` then `FIRST_VALUE(value) OVER (PARTITION BY grp ORDER BY ts)`. Or upgrade to 16+ for `IGNORE NULLS`.

```
SELECT customer_id, order_ts, amount,  
       NTH_VALUE(amount, 3) OVER (  
         PARTITION BY customer_id ORDER BY order_ts  
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
       ) AS third_order_amount  
FROM orders;
```

Q94 · Cumulative MIN / MAX — Running Extremes

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Cred, PhonePe · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Show each day's revenue with the running maximum revenue seen so far that month.

The 30-second answer: `MAX(daily_rev) OVER (PARTITION BY DATE_TRUNC('month', day) ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. Partition resets at each month; the frame says "everything from the month's start up to today."

Step-by-step thought process: 1. Recognise running max / min as a direct use of `MAX` / `MIN` with the running-total frame. 2. Pre-aggregate to one row per day if you have raw events. 3. Partition by month to reset the running max at each month boundary; without partition the running max is across the whole input. 4. Use explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` to be defensive, though the default frame with `ORDER BY` gives the same result here.

Edge cases to mention: - The first day of the month always equals the running max for that day. - For "running min so far this year" use `PARTITION BY EXTRACT(year FROM day)`. - Add `LAG(running_max) OVER (...)` to detect "today is a new month high" — useful for alerting.

What NOT to say: - Do not use a self-join (`WHERE other.day <= this.day`) — orders of magnitude slower than the window. - Do not forget the partition if you want monthly resets; without it you get an all-time running max.

Common follow-up: "Days where today's revenue equals the month-to-date max." — `WHERE daily_rev = running_max`.

```
WITH daily AS (  
    SELECT DATE(order_ts) AS day, SUM(amount) AS daily_rev  
    FROM orders GROUP BY 1  
)  
SELECT day, daily_rev,  
    MAX(daily_rev) OVER (  
    PARTITION BY DATE_TRUNC('month', day)  
    ORDER BY day  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS month_to_date_max  
FROM daily  
ORDER BY day;
```

Q95 · Rolling Conversion Rate — Window Aggregates With Numerator and Denominator

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Meesho · Topic: Window Functions

The question (interviewer's verbatim phrasing):

Compute the 7-day rolling conversion rate (orders / sessions) per day.

The 30-second answer: Two parallel rolling sums in one window, divided. Aggregate daily orders and sessions first, then `SUM(orders) OVER w * 1.0 / NULLIF(SUM(sessions) OVER w, 0) AS conv_rate_7d` with `w AS (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`.

Step-by-step thought process: 1. Aggregate to daily orders and daily sessions in a CTE — one row per day, two metric columns. 2. Apply the same window spec to both `SUM` s (use a named `WINDOW` clause for cleanliness). 3. Divide the two rolling sums. Always wrap the denominator in `NULLIF(..., 0)` to avoid divide-by-zero on zero-session days. 4. Anticipate "Why not `AVG(orders) / AVG(sessions)` ?" — it gives the same answer only if both windows have the same row count and no NULLs; rolling sums divided by each other are the canonical and defensive form.

Edge cases to mention: - For sparse data (missing days), join a calendar / `generate_series` so the rolling window does not skip days. - 7-row rolling vs 7-day rolling matters with missing days — pick `ROWS` or `RANGE BETWEEN INTERVAL '6 days' PRECEDING` accordingly. - For very low session counts, `conv_rate` swings wildly — apply a min-denominator filter (e.g. only emit if rolling sessions ≥ 100).

What NOT to say: - Do not compute conversion rate per day first and then average it — that gives the wrong answer because each day's rate is weighted equally regardless of session count. - Do not skip `NULLIF` on the denominator; the first day of low traffic will explode.

Common follow-up: "What if I want a 28-day rolling rate that only emits when 28 days of data exist?" — Wrap the result in `CASE WHEN COUNT(*) OVER w = 28 THEN ... END`.

```
WITH daily AS (
  SELECT DATE(event_ts) AS day,
         SUM(CASE WHEN event_type = 'order' THEN 1 ELSE 0 END) AS orders,
         SUM(CASE WHEN event_type = 'session_start' THEN 1 ELSE 0 END) AS sessions
  FROM events GROUP BY 1
)
SELECT day, orders, sessions,
       SUM(orders) OVER w * 1.0
       / NULLIF(SUM(sessions) OVER w, 0) AS conv_rate_7d
FROM daily
WINDOW w AS (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
ORDER BY day;
```

Q96 · Window Function Anti-Patterns to Avoid

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, JP Morgan, AmEx · Topic: Window Functions

The question (interviewer's verbatim phrasing):

What are the common window-function anti-patterns you have seen on production review?

The 30-second answer: Five repeat offenders. (1) `ROW_NUMBER` for dedup without a deterministic `ORDER BY` tie-breaker. (2) `LAST_VALUE` without an explicit `UNBOUNDED FOLLOWING` frame. (3) Filtering on a window column inside `WHERE` — the Filter-Window Trap from Q13. (4) Repeating the same partition+order across multiple windows instead of using the `WINDOW` clause. (5) Using a correlated subquery for running totals / moving averages instead of an `OVER` clause.

Step-by-step thought process: 1. Walk through each anti-pattern with the symptom and the fix. This is a great closing question because it lets you demonstrate breadth. 2. (1) Non-deterministic dedup — symptom: results change between runs; fix: add tie-breaker. 3. (2) `LAST_VALUE` returning current row — symptom: "last" matches current; fix: extend frame. 4. (3) Filter-Window Trap — symptom: missing rows; fix: CTE or `QUALIFY`. 5. (4) Repeated window spec — symptom: query bloat; fix: named `WINDOW` clause. 6. (5) Correlated subquery for running totals — symptom: query times out at scale; fix: rewrite as `SUM() OVER (ORDER BY ...)`. 7. Bonus mention: applying window after a `WHERE` that

strips needed history; aggregating-then-windowing when partition cardinality is huge; window over an unindexed sort.

Edge cases to mention: - `ROW_NUMBER` over a stable sort key (id + timestamp) is the safest dedup pattern; over a single ambiguous key it can return inconsistent results. - `LAST_VALUE` is so prone to misuse that many style guides ban it outright in favour of `FIRST_VALUE` with reversed ordering. - For audit trails, prefer windowed dedup over `DISTINCT ON` (Postgres-only) when you need to keep specific columns from the winning row.

What NOT to say: - Do not pretend these are obscure edge cases; they show up on every code review of a query with windows. - Do not say "the optimiser handles it" — the optimiser cannot rescue semantic bugs like the `LAST_VALUE` trap.

Common follow-up: *"Which of these would you put in a linter?"* — Filter-Window Trap and `LAST_VALUE` without frame are the easiest to flag automatically; non-deterministic `ORDER BY` is harder but possible.

```
-- Anti-pattern: running total via correlated subquery (slow)
SELECT o.order_id, o.amount,
       (SELECT SUM(amount) FROM orders o2 WHERE o2.order_ts <= o.order_ts) AS rt
FROM orders o;

-- Fix: window function
SELECT order_id, amount,
       SUM(amount) OVER (
         ORDER BY order_ts, order_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS rt
FROM orders;
```

Chapter 3, Part B — Common Table Expressions (CTEs)

CTEs are the single biggest readability upgrade SQL has shipped in the last twenty years. The moment your query crosses fifty lines, a well-named `WITH` block stops being a stylistic choice and starts being the difference between code your future self can debug and code that becomes a black box by next sprint. The basic form is a glorified named subquery, but the real power shows up in two places — first, when you chain five or six CTEs to decompose a brutal analytics query into named steps that read top-to-bottom like a recipe, and second, when you flip to `WITH RECURSIVE` and suddenly you can walk org charts, traverse friend graphs, generate date dimensions on the fly and unpack comma-separated strings without leaving SQL. This section runs thirty questions across the basic patterns, the recursive patterns, and the planner-level gotchas that interviewers love to dig into once they realise you actually know the syntax.

Q97 · Basic WITH Clause Syntax and When It Beats a Subquery

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture · Topic: CTEs

The question (interviewer's verbatim phrasing):

What is a CTE? Show me the syntax of a basic WITH clause and tell me when you would pick it over an inline subquery.

The 30-second answer: A CTE is a named, temporary result set defined with `WITH name AS (SELECT ...)` that the main query references like a regular table. Pick a CTE over a subquery the moment readability suffers, when the same derived set is needed in more than one place in the query, or when you want a self-documenting name on a derived step.

Step-by-step thought process: 1. Define the syntax cleanly — `WITH cte_name AS (SELECT ...)` `SELECT ... FROM cte_name`. Mention that the CTE lives only for the duration of that single statement. 2. State the readability case: a five-level nested subquery becomes five named CTEs that read top-to-bottom like English. 3. State the reuse case: if you need the same derived set joined twice in your main query, a CTE writes it once; a subquery writes it twice. 4. Be honest about the trade-off: a CTE that is referenced only once and is simple may not buy you anything over a subquery — use judgement.

Edge cases to mention: - A CTE is scoped to one statement only; the next statement cannot see it. If you need cross-statement reuse, use a temp table. - Some optimisers (older Postgres, for example) treat the CTE as an optimisation fence — covered in detail in a later question. - A CTE cannot be referenced in a different session or transaction — its lifetime ends with the statement.

What NOT to say: - Do not call a CTE a "temporary table" — it is not stored anywhere on disk and has no statistics of its own in most engines. - Do not say CTEs are always faster than subqueries; they are usually equivalent and sometimes slower.

Common follow-up: *"If a CTE is just syntactic sugar for a subquery, why does it matter?"* — Because code is read ten times more than it is written, and a named step that says `monthly_revenue` is worth a hundred lines of comments on an anonymous subquery.

```
WITH active_users AS (  
    SELECT user_id, MAX(last_login) AS last_seen  
    FROM sessions  
    WHERE last_login >= CURRENT_DATE - INTERVAL '30 days'  
    GROUP BY user_id  
)  
SELECT u.name, a.last_seen  
FROM users u  
JOIN active_users a ON u.id = a.user_id;
```

Q98 · Multiple CTEs in One Statement

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Wells Fargo · Topic: CTEs

The question (interviewer's verbatim phrasing):

Show me how to define two or three CTEs in the same query. What is the syntax and do I need to repeat the WITH keyword?

The 30-second answer: You declare WITH exactly once, then separate each CTE with a comma. The form is `WITH cte1 AS (...), cte2 AS (...), cte3 AS (...) SELECT ...`. The final main query then references any of them by name.

Step-by-step thought process: 1. Lead with the comma rule — beginners often repeat WITH for each block and get a parser error. 2. Each CTE has its own name and its own parenthesised SELECT. The order of declaration matters because a later CTE can reference an earlier one, but not the other way around. 3. Demonstrate with a two-step example: first CTE summarises orders per user, second CTE filters that down to high-value users, main query joins them to customer details. 4. Mention the readability win: three CTEs of ten lines each beat one subquery of thirty lines every time.

Edge cases to mention: - A CTE cannot reference a CTE declared after it — order matters in non-recursive CTEs. - All CTEs in the WITH block share the same scope; they are all visible to the final main statement. - Some engines (older MySQL versions before 8.0) did not support CTEs at all — always check the version.

What NOT to say: - Do not write `WITH cte1 AS (...) WITH cte2 AS (...)` — that is invalid syntax in every major dialect. - Do not assume CTEs run "in parallel" — they execute in whatever order the planner chooses, which is usually as needed.

Common follow-up: "*How many CTEs is too many?*" — When you stop being able to hold the dependency graph in your head. Five to seven is comfortable; fifteen probably means you should be writing views or a materialised pipeline.

```
WITH monthly_orders AS (  
    SELECT user_id, COUNT(*) AS order_count, SUM(amount) AS total  
    FROM orders  
    WHERE order_date >= DATE '2026-01-01'  
    GROUP BY user_id  
) ,  
high_value AS (  
    SELECT user_id, total  
    FROM monthly_orders  
    WHERE total > 50000  
)  
SELECT u.name, h.total  
FROM users u  
JOIN high_value h ON u.id = h.user_id;
```

Q99 · CTE Referencing Another CTE — Scoping Rules

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Walmart Labs · Topic: CTEs

The question (interviewer's verbatim phrasing):

Can one CTE reference another CTE in the same statement? Are there any ordering or scoping rules I should know?

The 30-second answer: Yes — any CTE can reference any CTE declared before it in the same WITH block. The scoping is strictly forward — `cte2` can see `cte1`, but `cte1` cannot see `cte2` unless we are in the recursive case, which is a special form.

Step-by-step thought process: 1. Establish the forward-reference rule clearly — this is the bit that confuses candidates who imagine CTEs as a flat namespace. 2. Walk through a three-CTE example: raw data, aggregated, then filtered. Each step builds on the previous. 3. Mention that the main SELECT at the end can reference any of the CTEs declared above, in any combination. 4. Contrast with recursive CTEs (covered in later questions), where a single CTE references itself.

Edge cases to mention: - Nesting CTEs visually (one inside another) is not allowed — they must all be declared at the same top level inside the WITH block. - A CTE can join multiple earlier CTEs — that is the whole point of decomposition. - If two CTEs have the same name, the parser will reject the query — names must be unique within a WITH block.

What NOT to say: - Do not say "CTEs are independent" — they are not, they form a dependency DAG. - Do not try to forward-reference by declaring a CTE later that the earlier one needs; that throws a parser error.

Common follow-up: "Can I have a CTE that depends on two earlier CTEs?" — Absolutely, that is the standard decomposition pattern. Each later CTE can join, union or filter against any combination of the earlier ones.

```
WITH raw_orders AS (  
    SELECT * FROM orders WHERE order_date >= '2026-04-01'  
),  
order_totals AS (  
    SELECT user_id, SUM(amount) AS total FROM raw_orders GROUP BY user_id  
),  
ranked_users AS (  
    SELECT user_id, total, RANK() OVER (ORDER BY total DESC) AS rnk  
    FROM order_totals  
)  
SELECT * FROM ranked_users WHERE rnk <= 10;
```

Q100 · Recursive CTE Basics — Anchor, Recursive Member, UNION

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Flipkart · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Explain the structure of a recursive CTE. What are the anchor and recursive parts, and why do we use UNION ALL between them?

The 30-second answer: A recursive CTE has two parts joined by UNION ALL — the anchor query that produces the seed rows, and the recursive query that references the CTE itself and runs repeatedly until it produces no new rows. UNION ALL is used because UNION (without ALL) silently deduplicates, which can hide rows you actually wanted and in pathological cases can also break the termination condition.

Step-by-step thought process: 1. Draw the structure on a whiteboard if you can — `WITH RECURSIVE cte AS (anchor UNION ALL recursive_part) SELECT * FROM cte`. 2. Explain the execution: the anchor runs once and seeds the result. Then the recursive part runs against the latest batch of new rows, producing more rows. This repeats until a step adds zero new rows. 3. Stress the WITH RECURSIVE keyword — Postgres and MySQL require it explicitly. SQL Server omits the RECURSIVE keyword but the structure is identical. 4. Hammer home why UNION ALL: UNION does an implicit DISTINCT against the entire result so far, which can both hurt performance massively and silently change which rows survive.

Edge cases to mention: - If your recursive part never produces zero new rows, you have an infinite loop — engines usually cap recursion depth (Postgres has no default cap, SQL Server defaults to 100 via MAXRECURSION). - The columns of the anchor and the recursive part must line up by position and be

type-compatible — common cause of "column type mismatch" errors. - The recursive reference can appear only once in the recursive member (standard SQL constraint).

What NOT to say: - Do not say "UNION and UNION ALL are interchangeable here" — they are not, this is a top-tier bug. - Do not omit the RECURSIVE keyword in Postgres or MySQL; the parser needs the hint.

Common follow-up: "*What happens if the anchor returns zero rows?*" — The recursion never starts, so the whole CTE produces zero rows. That is sometimes a real bug in your seed condition rather than a missing UNION.

```
WITH RECURSIVE numbers AS (  
  SELECT 1 AS n          -- anchor  
  UNION ALL  
  SELECT n + 1          -- recursive member  
  FROM numbers  
  WHERE n < 10  
)  
SELECT n FROM numbers;
```

Q101 · Recursive CTE for Organizational Hierarchy

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Wells Fargo, AmEx · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Given an employees table with `id`, `name` and `manager_id`, write a recursive CTE that returns the full reporting chain for every employee under the CEO.

The 30-second answer: Anchor on the CEO (the row where `manager_id IS NULL`), then recursively join the CTE to the employees table on `e.manager_id = cte.id` to walk down one level at a time. UNION ALL between the two and you get every employee with their depth in the hierarchy.

Step-by-step thought process: 1. Start by identifying the seed — usually the row with `manager_id IS NULL` is the top of the tree. If multiple roots exist, the anchor returns them all. 2. The recursive step joins the CTE (the "already visited" set, in practice just the latest layer) to the employees table on the manager link, adding one new layer per iteration. 3. Carry a `level` or `depth` column initialised to 1 in the anchor and incremented in the recursive step — extremely useful for indenting output or filtering. 4. Mention the inverse: if you want a single employee's chain of bosses, flip the join direction — anchor on the employee, recurse upward via `e.id = cte.manager_id`.

Edge cases to mention: - If the hierarchy contains a cycle (A reports to B, B reports to A by mistake), the recursion never terminates without a cycle check. - An employee with no manager and not the CEO

will not appear unless the anchor is written to include them — be explicit about what "root" means in your data. - Large hierarchies may need a recursion limit or an index on `manager_id` to be tolerable.

What NOT to say: - Do not use a self-join when asked specifically for a "full hierarchy" — a self-join only goes one level deep. - Do not forget the LEFT JOIN equivalent reasoning — the recursive CTE naturally handles all levels.

Common follow-up: "Show me how to find all direct and indirect reports of a specific manager." — Anchor on that manager's row and recurse downward; the result is the entire subtree.

```
WITH RECURSIVE org_chart AS (  
  SELECT id, name, manager_id, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id, oc.level + 1  
  FROM employees e  
  JOIN org_chart oc ON e.manager_id = oc.id  
)  
SELECT id, name, level FROM org_chart ORDER BY level, name;
```

Q102 · Recursive CTE for Graph Traversal — Friend of Friend

Tags: Difficulty: Hard · Asked at: Meesho, Flipkart, Cred, PhonePe · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Given a `friendships(user_a, user_b)` table, write a query that finds everyone within three degrees of a starting user.

The 30-second answer: Anchor on the starting user, then recursively join the friendships table to the CTE on either column to walk one edge at a time, carrying a depth counter and stopping when depth reaches three. Use a visited-set or path column to avoid revisiting users in cycles.

Step-by-step thought process: 1. Acknowledge that a friendship table is usually undirected — a pair (A, B) implies the connection in both directions. Either store both rows or handle both directions in the join. 2. Anchor seeds with the starting user at depth zero. Recursive part joins to friendships and increments depth. 3. Cycle prevention is essential — without it, A -> B -> A -> B loops forever. Track visited users in an array or a string path column. 4. Cap the depth in the WHERE clause of the recursive part (`WHERE depth < 3`) so you stop at the requested degree.

Edge cases to mention: - Undirected vs directed: clarify the schema before writing the join. If only one direction is stored, you need to handle both columns. - The result may contain the same friend reached

by multiple paths — apply DISTINCT or aggregate at the end. - For very dense graphs, even three degrees can return millions of rows — set expectations with the interviewer.

What NOT to say: - Do not claim "recursive CTEs cannot do graph traversal" — they can, with appropriate cycle handling. - Do not skip the cycle check on a graph just because hierarchies do not need it.

Common follow-up: "How would you find the shortest path between two users?" — Track the path, keep the minimum depth per target user, stop when the target is reached.

```
WITH RECURSIVE friend_network AS (  
  SELECT user_a, user_b, 1 AS depth, ARRAY[user_a, user_b] AS path  
  FROM friendships  
  WHERE user_a = 42  
  UNION ALL  
  SELECT fn.user_b, f.user_b, fn.depth + 1, fn.path || f.user_b  
  FROM friend_network fn  
  JOIN friendships f ON f.user_a = fn.user_b  
  WHERE fn.depth < 3  
    AND f.user_b <> ALL(fn.path)  -- cycle check  
)  
SELECT DISTINCT user_b, MIN(depth) AS min_depth  
FROM friend_network  
GROUP BY user_b;
```

Q103 · Generating a Number Series with Recursive CTE

Tags: Difficulty: Easy · Asked at: Wipro, Accenture, Walmart Labs · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Generate the numbers from 1 to 100 using a recursive CTE, without using any system functions like `generate_series` or a numbers table.

The 30-second answer: Anchor with `SELECT 1`, recurse with `SELECT n + 1 FROM cte WHERE n < 100`, UNION ALL between them. The recursive part keeps adding one until the WHERE clause fails.

Step-by-step thought process: 1. State the seed clearly — `SELECT 1 AS n`. 2. The recursive member adds one and references itself, with a termination predicate that stops it at the right boundary. 3. Mention that this is a foundational pattern — generating tally tables, date dimensions and string-splitting all reuse the same technique. 4. In Postgres you would normally just use `generate_series(1, 100)`, but the recursive form is what interviewers want to see — and what you need on dialects without that function.

Edge cases to mention: - In SQL Server, the default `MAXRECURSION` is 100, so generating more than 100 rows requires `OPTION (MAXRECURSION 0)` for unlimited. - Postgres has no default recursion cap, so you can blow up memory if your termination condition is wrong. - MySQL recursive depth is capped at 1000 by default via `cte_max_recursion_depth`.

What NOT to say: - Do not use UNION instead of UNION ALL — at depth 100, the implicit DISTINCT cost is wasteful and unnecessary. - Do not write the termination as `WHERE n <= 100` and then expect 100 rows; that gives you 101 because the row where `n = 100` still satisfies the predicate.

Common follow-up: "Generate every odd number from 1 to 99." — Same pattern, but seed with 1 and add 2 instead of 1, terminate at `n < 99`.

```
WITH RECURSIVE numbers(n) AS (  
  SELECT 1  
  UNION ALL  
  SELECT n + 1 FROM numbers WHERE n < 100  
)  
SELECT n FROM numbers;
```

Q104 · Recursive CTE for Date Dimension Generation

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Zerodha, Target · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Build a list of every date in the financial year from 1 April 2025 to 31 March 2026 using a recursive CTE.

The 30-second answer: Anchor with the start date, recurse by adding one day and stopping when the date crosses the end. UNION ALL gives you 365 (or 366) one-row results, one per date in the window.

Step-by-step thought process: 1. Anchor `SELECT DATE '2025-04-01' AS d`. 2. Recursive part `SELECT d + INTERVAL '1 day' FROM cte WHERE d < DATE '2026-03-31'` — exact date arithmetic syntax varies by dialect. 3. Useful in BI work — you join this CTE to your fact tables with a LEFT JOIN so days with no events still show up (instead of dropping out). 4. Mention that in production, most teams materialise a permanent date dimension table; the recursive CTE is the on-the-fly version when no such table exists.

Edge cases to mention: - Watch the time-zone behaviour of `INTERVAL '1 day'` versus `+ 1` integer — across DST boundaries some engines surprise you. - For a financial year you may also need week numbers, quarter numbers and fiscal period — add them as projected columns inside the CTE. - Generating decades of dates this way can exceed MySQL's default recursion limit; bump it or use a tally table.

What NOT to say: - Do not say "I would just use `generate_series`" without acknowledging that not every dialect has it. - Do not forget to handle the inclusive boundary — if you want 31 March included, the WHERE clause must allow `d = '2026-03-30'` to fire and produce 31 March.

Common follow-up: "Now also flag which dates are weekends and which are Indian public holidays." — Join the recursive CTE to a `holidays` table and use `EXTRACT(DOW FROM d)` for weekends.

```
WITH RECURSIVE fy_dates AS (  
  SELECT DATE '2025-04-01' AS d  
  UNION ALL  
  SELECT d + INTERVAL '1 day'  
  FROM fy_dates  
  WHERE d < DATE '2026-03-31'  
)  
SELECT d FROM fy_dates;
```

Q105 · Parsing a Comma-Separated String with Recursive CTE

Tags: Difficulty: Hard · Asked at: TCS, Infosys, Wipro · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Without using `STRING_SPLIT` or `UNNEST`, write a recursive CTE that splits a comma-separated string into one row per token.

The 30-second answer: Anchor extracts the substring before the first comma and the remaining string after it. Recursive part keeps doing the same on the remaining string until no comma is left, at which point the remaining string itself is the final token.

Step-by-step thought process: 1. Identify the position of the next comma with `POSITION` or `CHARINDEX` depending on dialect. 2. Anchor projects two columns — the first token, and the rest of the string after the first comma. 3. Recursive part takes the previous "rest" column, peels off the next token, and shrinks the rest further. 4. Termination is implicit — once `rest` has no comma, the recursive step does not produce another row.

Edge cases to mention: - Empty tokens between consecutive commas (`a,,b`) should still produce a row with an empty string. - Trailing commas leave a final empty token — decide whether to keep or filter it. - This is significantly slower than the built-in split functions; reach for the built-in when available.

What NOT to say: - Do not claim it is impossible without the built-ins; recursive CTEs are exactly the tool for this. - Do not forget the terminator case — the final token has no comma after it and must still be emitted.

Common follow-up: "How would you keep the original ordinal position of each token?" — Carry an integer counter starting at 1 and increment it in the recursive step.

```
WITH RECURSIVE split_str AS (
  SELECT
    SUBSTRING('amit,sneha,rohan,priya' FROM 1 FOR POSITION(',') IN 'amit,sneha,rohan,priya') - 1)
    SUBSTRING('amit,sneha,rohan,priya' FROM POSITION(',') IN 'amit,sneha,rohan,priya') + 1) AS rest
    1 AS ord
  UNION ALL
  SELECT
    CASE WHEN POSITION(',') IN rest) > 0
      THEN SUBSTRING(rest FROM 1 FOR POSITION(',') IN rest) - 1)
      ELSE rest END,
    CASE WHEN POSITION(',') IN rest) > 0
      THEN SUBSTRING(rest FROM POSITION(',') IN rest) + 1)
      ELSE '' END,
    ord + 1
  FROM split_str
  WHERE rest <> ''
)
SELECT ord, token FROM split_str WHERE token <> '';
```

Q106 · Transitive Closure with Recursive CTE

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

What is a transitive closure and how would you compute one with a recursive CTE?

The 30-second answer: A transitive closure of a relation R contains every pair (x, z) such that there is a chain $x \rightarrow y_1 \rightarrow y_2 \rightarrow \dots \rightarrow z$ in R. The recursive CTE seeds with the original relation and joins to itself repeatedly, accumulating new derived pairs until no new pairs appear.

Step-by-step thought process: 1. Define transitive closure with a concrete example: if Amit reports to Bina and Bina reports to Chetan, then the closure says Amit transitively reports to Chetan. 2. Anchor with the raw relation (every existing edge). 3. Recursive part joins the CTE to the original relation to extend known reach by one hop. Carry a "from" and "to" column. 4. Avoid duplicates with DISTINCT in the final SELECT, or use a join that filters out already-known pairs in the recursive step.

Edge cases to mention: - Without cycle prevention, a cyclic graph causes infinite recursion. - The closure can explode in size — n nodes can produce up to n^2 pairs. - For huge graphs, an iterative materialised approach in application code can be more memory-friendly than a single SQL recursion.

What NOT to say: - Do not say "transitive closure is a graph algorithm, SQL cannot do it" — recursive CTEs are exactly how SQL does it. - Do not assume the closure is symmetric; for a directed edge set it usually is not.

Common follow-up: "What is the difference between transitive closure and reachability?" — Reachability is the same idea expressed as a yes/no question between two specific nodes; closure is the full set of all reachable pairs.

```
WITH RECURSIVE reach AS (  
  SELECT source, target FROM edges  
  UNION ALL  
  SELECT r.source, e.target  
  FROM reach r  
  JOIN edges e ON e.source = r.target  
)  
SELECT DISTINCT source, target FROM reach;
```

Q107 · Recursive CTE Termination — Preventing Infinite Loops

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, AmEx · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

What can go wrong with a recursive CTE and how do you guarantee termination?

The 30-second answer: The recursive part must eventually produce zero new rows, otherwise the query runs forever. Guarantee termination either by including a strict inequality on a counter (like `WHERE depth < 10`), by relying on the data shape (a tree without cycles), or by adding an explicit cycle-detection check on a path column.

Step-by-step thought process: 1. Identify the three causes of non-termination: a missing or wrong WHERE clause in the recursive part, a cycle in the underlying graph, or a recursive condition that keeps producing the same rows. 2. The cleanest defence is a depth counter — initialise it in the anchor and increment in the recursive step, then bound it in the WHERE clause. 3. For graph traversal, add a path column (an array or a delimited string) and skip nodes already in the path. 4. Most engines have a fallback hard limit — SQL Server defaults to 100 recursion levels, MySQL to 1000, Postgres to unlimited (set `statement_timeout` to defend against runaway queries).

Edge cases to mention: - A poorly chosen anchor can produce the same row again in the recursive step, looping in place — check that the join condition strictly advances state. - Using UNION instead of UNION ALL silently dedupes — sometimes hiding the bug, sometimes preventing termination by making the new-rows count zero too early. - Engine-level safeguards exist, but you should not rely on them; write explicit termination conditions.

What NOT to say: - Do not say "the engine will stop it automatically" — Postgres will not stop until memory runs out. - Do not write `WHERE 1=1` and hope; that is exactly how you build an infinite loop.

Common follow-up: "Show me a recursive CTE that loops forever and how to fix it." — Take any recursive query, remove the depth check, run on a cyclic graph; fix by reintroducing the depth bound or a cycle-detection clause.

```
WITH RECURSIVE chain AS (  
  SELECT id, parent_id, 1 AS depth  
  FROM nodes  
  WHERE id = 1  
  UNION ALL  
  SELECT n.id, n.parent_id, c.depth + 1  
  FROM nodes n  
  JOIN chain c ON n.parent_id = c.id  
  WHERE c.depth < 20          -- termination guard  
)  
SELECT * FROM chain;
```

Q108 · MAXRECURSION in SQL Server and Cycle Detection

Tags: Difficulty: Medium · Asked at: Wells Fargo, JP Morgan, AmEx · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

In SQL Server, what is MAXRECURSION and how does it differ from the cycle-detection clause in Postgres?

The 30-second answer: `MAXRECURSION` in SQL Server is a query hint that caps the number of recursive iterations — the default is 100, and setting it to 0 means unlimited. It is a safety belt, not a cycle detector. Postgres 14 added a proper `CYCLE` clause to recursive CTEs that detects loops by tracking the path and stopping when a node is revisited.

Step-by-step thought process: 1. Position MAXRECURSION as a blunt instrument — it stops runaway queries by counting iterations, but does not understand why they ran away. 2. Position the Postgres `CYCLE` clause as a structural solution — it tracks a chosen column's path and marks rows that complete a cycle so you can filter them out. 3. Mention that in dialects without `CYCLE`, you replicate it manually by carrying a path column (array or string) and excluding nodes already in the path. 4. Both approaches have a place — MAXRECURSION as a safety net, `CYCLE` as the precise semantic.

Edge cases to mention: - `MAXRECURSION 0` is unlimited — accidentally setting that on a buggy CTE on a cyclic graph hangs the session. - The `CYCLE` clause requires you to name the columns to track and the marker column to add — it is not magic. - Other dialects (MySQL, Oracle) have their own knobs:

MySQL has `cte_max_recursion_depth`, Oracle has its own CONNECT BY syntax outside the CTE family.

What NOT to say: - Do not say "MAXRECURSION detects cycles" — it does not, it counts iterations. - Do not say "every database has a CYCLE clause" — only Postgres 14+ and standard-conformant engines do.

Common follow-up: "Show me the Postgres CYCLE clause syntax." — Covered in Q29 of this section.

```
-- SQL Server style: cap recursion at 50
WITH org AS (
  SELECT id, manager_id, 1 AS lvl FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.manager_id, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org
OPTION (MAXRECURSION 50);
```

Q109 · WITH RECURSIVE Syntax Differences Across Dialects

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Walk me through how recursive CTE syntax differs between Postgres, MySQL and SQL Server.

The 30-second answer: Postgres and MySQL both require `WITH RECURSIVE`; the keyword is mandatory. SQL Server uses plain `WITH` with no RECURSIVE keyword — the recursion is implied by the self-reference. All three share the anchor + UNION ALL + recursive-member structure. SQL Server adds the `OPTION (MAXRECURSION N)` hint; Postgres 14+ adds the `CYCLE` and `SEARCH` clauses.

Step-by-step thought process: 1. Lead with the RECURSIVE keyword difference — that single word trips up candidates moving between dialects. 2. Note that Oracle historically used `CONNECT BY` instead of recursive CTEs, though it now supports the standard form too. 3. Highlight that the body of the CTE — anchor, UNION ALL, recursive member — is identical across all three. The dialect differences are only in keyword and tuning hints. 4. Be ready to explain why Postgres made RECURSIVE explicit — it makes the parser's job easier and signals intent loudly.

Edge cases to mention: - MySQL added recursive CTEs only in 8.0; pre-8.0 you simulated recursion with stored procedures or session variables. - SQL Server's missing RECURSIVE keyword is technically non-standard; pure ANSI SQL requires it. - Oracle accepts the standard form from 11g R2 onwards and still supports `CONNECT BY` for backwards compatibility.

What NOT to say: - Do not say "all SQL is the same" — even something this basic varies across dialects. - Do not assume your Postgres CTE will run unchanged on SQL Server — at minimum, drop the `RECURSIVE` keyword.

Common follow-up: *"Which one would you use in a portable script?"* — Standard-conformant `WITH RECURSIVE` works on Postgres, MySQL, Oracle and IBM Db2; SQL Server is the odd one out, and you usually handle that with conditional generation.

```
-- Postgres / MySQL form
WITH RECURSIVE cte AS (...) SELECT * FROM cte;

-- SQL Server form (no RECURSIVE keyword)
WITH cte AS (...) SELECT * FROM cte OPTION (MAXRECURSION 0);
```

Q110 · CTE versus Subquery — Readability and Reuse Trade-offs

Tags: Difficulty: Easy · Asked at: Wipro, Accenture, Razorpay · Topic: CTEs

The question (interviewer's verbatim phrasing):

When should you pick a CTE over an inline subquery? Are there cases where the subquery is better?

The 30-second answer: Pick a CTE when the derived set is non-trivial, needs a meaningful name, or is referenced more than once. Stick with a subquery when it is a one-line expression used in exactly one place and naming it adds noise rather than clarity.

Step-by-step thought process: 1. State the three triggers for a CTE — complexity, naming, reuse. 2. Acknowledge the counter-case: a subquery like `(SELECT MAX(id) FROM orders)` used once in a `WHERE` clause is clearer inline than as a separate CTE. 3. Performance is usually equivalent on modern engines, so the decision is almost always about readability and maintenance cost. 4. Note that in older Postgres (pre-12) and some other engines, a CTE acts as an optimisation fence, so a subquery could be faster — covered in detail in a later question.

Edge cases to mention: - A CTE referenced ten times in the main query is still computed once logically (and once physically on most modern planners); a subquery copy-pasted ten times may be computed ten times. - Correlated subqueries cannot be lifted out into a CTE without restructuring — they reference outer columns. - Excessive CTE use can be its own anti-pattern — every step gets a name, but readers have to bounce between definitions.

What NOT to say: - Do not claim CTEs are always more performant — that has been false since Postgres 12 made them inlineable by default. - Do not claim CTEs are always more readable — at small sizes the subquery is fine.

Common follow-up: *"Convert this nested subquery into CTEs."* — A common practical exercise; aim for one CTE per logical step.

```
-- Subquery form (acceptable for simple cases)
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders WHERE amount > 5000);

-- CTE form (preferable when reused or complex)
WITH big_spenders AS (SELECT user_id FROM orders WHERE amount > 5000)
SELECT * FROM users WHERE id IN (SELECT user_id FROM big_spenders);
```

Q111 · CTE versus Temp Table — When Temp Table Wins

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, Walmart Labs · Topic: CTEs

The question (interviewer's verbatim phrasing):

A CTE and a temp table both let you name an intermediate result. When would you reach for a temp table instead of a CTE?

The 30-second answer: Reach for a temp table when you need the intermediate result in multiple statements, when you want to add indexes to it for repeated lookups, or when the planner is mis-estimating row counts because the CTE is opaque. CTEs are scoped to one statement; temp tables persist across statements within a session or transaction.

Step-by-step thought process: 1. State the lifetime difference — CTE lives for one statement; temp table lives for the session or transaction. 2. State the index difference — you can create indexes on a temp table, you cannot on a CTE. 3. State the statistics difference — temp tables can have actual row counts gathered, leading to better plans for downstream queries. CTEs often use guesses. 4. Acknowledge the trade-off — temp tables have setup cost (DDL, statistics, eviction) and require explicit cleanup in some engines.

Edge cases to mention: - In SQL Server, table variables (`@var`) behave differently from temp tables (`#temp`); table variables have very limited statistics and often misplan. - In Postgres, temp tables vanish at end-of-session by default but can be set to vanish at end-of-transaction with `ON COMMIT DROP`. - For very large intermediate sets that are joined repeatedly, a temp table with an index can be orders of magnitude faster than a CTE.

What NOT to say: - Do not say "temp tables are always faster" — small intermediate sets do not benefit from materialisation. - Do not say "CTE is just a temp table" — they are structurally and behaviourally different.

Common follow-up: *"How would you decide in a real performance tuning session?"* — Profile both, look at the planner output, and choose based on actual numbers, not folklore.

```
-- Temp table allows indexing and reuse across statements
CREATE TEMP TABLE big_spenders AS
SELECT user_id, SUM(amount) AS total FROM orders GROUP BY user_id HAVING SUM(amount) > 50000;
CREATE INDEX idx_bs_user ON big_spenders(user_id);

SELECT u.* FROM users u JOIN big_spenders b ON u.id = b.user_id;
SELECT COUNT(*) FROM big_spenders;
```

Q112 · CTE Materialization in Postgres — The 11 vs 12 Change

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Zerodha · Topic: CTEs

The question (interviewer's verbatim phrasing):

How did CTE behaviour change in Postgres 12 versus earlier versions, and why does it matter for performance?

The 30-second answer: In Postgres 11 and earlier, every CTE was an optimisation fence — the planner materialised it and could not push predicates into it. From Postgres 12 onward, the default is to inline a CTE that is referenced exactly once and is side-effect-free, so the planner can optimise across the boundary. The pre-12 behaviour is still available via the **MATERIALIZED** keyword.

Step-by-step thought process: 1. Establish the pre-12 behaviour — every CTE acted as a fence, sometimes producing dramatically worse plans because filters could not be pushed in. 2. Explain the 12+ default — inline single-reference, side-effect-free CTEs and treat them like subqueries for planning purposes. 3. Note the new keyword **MATERIALIZED** (force the old fence behaviour) and **NOT MATERIALIZED** (force the new inline behaviour even when other heuristics would not). 4. Anticipate the question about why the change happened — to remove a footgun that punished readability with bad plans.

Edge cases to mention: - CTEs that are referenced more than once, or are recursive, or contain side-effect statements (data-modifying CTEs) are still materialised by default in 12+. - The **MATERIALIZED** hint is the way to deliberately compute a CTE once and reuse it, which can be useful when the CTE is expensive and used multiple times. - This change is Postgres-specific — other engines have their own materialisation rules.

What NOT to say: - Do not say "CTEs are always optimisation fences" — that is the pre-12 Postgres view and is wrong on modern Postgres and most other engines. - Do not assume your old performance tuning notes from Postgres 9.x still apply.

Common follow-up: "When would you force *MATERIALIZED*?" — Covered next in Q17.

```
-- Postgres 12+: this CTE is inlined by default
WITH recent_orders AS (SELECT * FROM orders WHERE order_date > '2026-01-01')
SELECT * FROM recent_orders WHERE user_id = 7;

-- Force the old fence behaviour
WITH recent_orders AS MATERIALIZED (SELECT * FROM orders WHERE order_date > '2026-01-01')
SELECT * FROM recent_orders WHERE user_id = 7;
```

Q113 · MATERIALIZED and NOT MATERIALIZED Hints in Postgres

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Walmart Labs · Topic: CTEs

The question (interviewer's verbatim phrasing):

Explain when you would force *MATERIALIZED* on a Postgres CTE and when *NOT MATERIALIZED* is the right call.

The 30-second answer: Use *MATERIALIZED* when the CTE is expensive to compute and you want it computed once even though it is referenced multiple times — or when you want to deliberately fence the planner because inlining produces a worse plan. Use *NOT MATERIALIZED* when you want inlining for a multi-reference CTE that the planner would otherwise materialise.

Step-by-step thought process: 1. State the defaults: single-reference inline, multi-reference materialise, recursive materialise, side-effect materialise. 2. *MATERIALIZED* is for "compute this once and reuse" or for "I tested both plans and the materialised one is faster." 3. *NOT MATERIALIZED* is for "even though this is referenced twice, inlining is fine and gives a better plan." 4. Stress that you should always confirm with *EXPLAIN ANALYZE* — these hints are not folklore decisions, they are evidence-driven.

Edge cases to mention: - Data-modifying CTEs (*INSERT/UPDATE/DELETE* inside *WITH*) are always materialised — *NOT MATERIALIZED* cannot override that for safety reasons. - Forcing *MATERIALIZED* on a cheap CTE that the planner would have happily inlined just adds overhead. - The hints exist only on Postgres 12+; other dialects ignore the keyword or fail to parse it.

What NOT to say: - Do not sprinkle *MATERIALIZED* everywhere "to be safe" — you are guessing, and you usually guess wrong. - Do not use these hints in code that has to run on non-Postgres engines without conditional generation.

Common follow-up: "How do you decide which to use?" — Run both versions with **EXPLAIN (ANALYZE, BUFFERS)**, compare cost and rows-removed-by-filter; pick the lower cost plan.

```
WITH heavy AS MATERIALIZED (  
  SELECT user_id, SUM(amount) AS total  
  FROM orders  
  WHERE order_date BETWEEN '2025-04-01' AND '2026-03-31'  
  GROUP BY user_id  
)  
SELECT u.name, h.total  
FROM users u JOIN heavy h ON u.id = h.user_id  
WHERE h.total > 100000;
```

Q114 · CTE in Data-Modifying Statements

Tags: Difficulty: Medium · Asked at: Walmart Labs, Flipkart, AmEx, JP Morgan · Topic: CTEs

The question (interviewer's verbatim phrasing):

Can a CTE be used with INSERT, UPDATE or DELETE? Show me an example.

The 30-second answer: Yes — the WITH clause goes in front of an INSERT, UPDATE or DELETE just as it goes in front of a SELECT. The CTE produces a row set that the DML statement reads from. In Postgres, the WITH can also contain INSERT/UPDATE/DELETE statements themselves, with results piped into the final DML.

Step-by-step thought process: 1. Show the simplest form — a CTE that computes a filter set, then **DELETE FROM ... WHERE id IN (SELECT id FROM cte)**. 2. Walk through an update where the CTE computes the new values per row and the UPDATE joins to it. 3. For insertion, the main statement is **INSERT INTO target SELECT * FROM cte**. 4. Save the data-modifying CTE form for the next question — it is the Postgres-specific power feature.

Edge cases to mention: - Some dialects do not allow CTEs in DML statements — older MySQL versions, for example. - Inside transactional DML, the CTE is still a single-statement scope; its result vanishes after the DML completes. - Carefully test that your join logic in the DML does not multiply rows — the same junior-level bug as in SELECT joins.

What NOT to say: - Do not say "CTEs are only for SELECT" — that has been wrong since standard SQL allowed WITH in front of any DML. - Do not assume your CTE inside an UPDATE will run only on the rows that match — it produces the row set, then the UPDATE applies it.

Common follow-up: "Show me a CTE inside an UPDATE that uses a window function for the new value." — A common pattern: rank rows in the CTE, then UPDATE to set status by rank.

```
WITH inactive AS (  
    SELECT user_id  
    FROM sessions  
    GROUP BY user_id  
    HAVING MAX(last_login) < CURRENT_DATE - INTERVAL '180 days'  
)  
UPDATE users SET status = 'dormant'  
WHERE id IN (SELECT user_id FROM inactive);
```

Q115 · Postgres Data-Modifying CTE with RETURNING

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred · Topic: CTEs

The question (interviewer's verbatim phrasing):

In Postgres, what is a data-modifying CTE and how does the RETURNING clause make it powerful?

The 30-second answer: Postgres lets you put INSERT, UPDATE or DELETE statements inside the WITH block. Each DML can use a RETURNING clause to expose the affected rows as the CTE's result set, which subsequent CTEs (or the final SELECT) can consume. This lets you express "do A, then do B using A's outcome, then return C" in a single atomic statement.

Step-by-step thought process: 1. Open with the unique-to-Postgres part — INSERT/UPDATE/DELETE inside WITH, with RETURNING. 2. Demonstrate the classic move-rows pattern — DELETE returning rows, INSERT those rows into an archive in one statement. 3. Stress atomicity — all the DML inside one statement is in the same snapshot, so no race conditions between the steps. 4. Mention that other dialects do not support this — SQL Server and MySQL require multiple statements wrapped in a transaction.

Edge cases to mention: - The CTE referencing a data-modifying CTE sees the rows returned by RETURNING, not the post-DML state of the table. - The order of execution of multiple data-modifying CTEs in the same WITH is not guaranteed unless one references the other. - A data-modifying CTE is always materialised — you cannot inline it.

What NOT to say: - Do not say "Postgres data-modifying CTEs run in a defined order" — only the data-flow order is guaranteed. - Do not assume the rest of the statement sees the post-DML table state; it does not, it sees the snapshot from before the statement started.

Common follow-up: "What is the atomic move-rows-to-archive pattern?" — Shown below.

```
WITH deleted_orders AS (  
  DELETE FROM orders  
  WHERE order_date < CURRENT_DATE - INTERVAL '5 years'  
  RETURNING *  
)  
INSERT INTO orders_archive  
SELECT * FROM deleted_orders;
```

Q116 · Query Decomposition with CTEs

Tags: Difficulty: Medium · Asked at: TCS, Wipro, Walmart Labs, JP Morgan · Topic: CTEs

The question (interviewer's verbatim phrasing):

You inherit a 200-line query with five levels of nested subqueries. How would you rewrite it with CTEs?

The 30-second answer: Identify the logical steps — usually filtering, aggregation, ranking, joining — and lift each step into a named CTE. Order the CTEs so each one references only earlier ones. The final SELECT becomes a short query that joins the CTEs together.

Step-by-step thought process: 1. Start by listing the steps the original query performs in plain English. Each step usually maps to one CTE. 2. Name each CTE after the business concept — `active_users`, `monthly_revenue`, `top_customers` — not after technical structure. 3. Replace the innermost subquery first, then work outward, testing after each rewrite to confirm the result set is unchanged. 4. End with a short main SELECT that reads like a summary of the whole pipeline.

Edge cases to mention: - Sometimes the original query relied on correlated subqueries — those do not map cleanly to CTEs and may need restructuring with window functions. - Test the rewrite carefully: a CTE that uses different filter ordering can produce different results if the original relied on short-circuit evaluation. - Performance can shift either way — measure the rewrite, do not assume readability is free.

What NOT to say: - Do not say "I would just leave it as-is" — long nested subqueries are exactly the maintenance debt CTEs are designed to clear. - Do not split every micro-step into a CTE — five to seven steps is the sweet spot.

Common follow-up: "What if the rewrite is slower?" — Investigate with EXPLAIN; consider `MATERIALIZED` hints in Postgres or temp tables for the heaviest steps.

```
WITH eligible_orders AS (  
    SELECT * FROM orders WHERE order_status = 'completed' AND order_date >= '2026-01-01'  
),  
per_user AS (  
    SELECT user_id, COUNT(*) AS orders, SUM(amount) AS revenue  
    FROM eligible_orders GROUP BY user_id  
),  
ranked AS (  
    SELECT user_id, revenue, RANK() OVER (ORDER BY revenue DESC) AS rnk  
    FROM per_user  
)  
SELECT u.name, r.revenue  
FROM ranked r JOIN users u ON r.user_id = u.id  
WHERE r.rnk <= 100;
```

Q117 · Performance of CTEs on Large Data — Planner Barriers

Tags: Difficulty: Hard · Asked at: Goldman, Walmart Labs, JP Morgan, Cred · Topic: CTEs

The question (interviewer's verbatim phrasing):

A colleague says CTEs always hurt performance. What is the kernel of truth in that and when is it wrong?

The 30-second answer: The kernel of truth is that on older Postgres (pre-12) and a few other engines, a CTE acts as an optimisation fence — filters and predicates cannot be pushed through it, so the planner may compute a large intermediate that gets thrown away. The claim is wrong on modern Postgres 12+, on SQL Server (which has never treated CTEs as fences), and on MySQL 8+.

Step-by-step thought process: 1. Acknowledge the historical truth — anyone who learned SQL on Postgres 9 or 10 may have war stories. 2. Walk through what changed: Postgres 12 made single-reference, side-effect-free CTEs inlineable by default. 3. Note that other engines have always inlined CTEs in their planners, so the "always hurts" claim was never a universal truth. 4. The right framing today is: CTEs are a planner suggestion, and the planner is free to inline or materialise based on heuristics — measure if in doubt.

Edge cases to mention: - Recursive CTEs are always materialised — they have to be, by the nature of the algorithm. - Data-modifying CTEs in Postgres are always materialised — for safety. - Even with inlining, the planner may make different choices than for a hand-written subquery — measure both.

What NOT to say: - Do not parrot "CTEs are slow" — that is a pre-2019 view. - Do not assume your team's database is on the latest version; check first.

Common follow-up: *"How would you prove or disprove this on your stack?"* — Run EXPLAIN ANALYZE on the CTE form and the subquery form, compare cost and runtime.

```
-- On Postgres 12+, this is typically inlined and the WHERE filter is pushed in  
WITH big_set AS (SELECT * FROM events)  
SELECT * FROM big_set WHERE event_type = 'login';
```

Q118 · Recursive CTE with Multiple Anchor Members

Tags: Difficulty: Hard · Asked at: Walmart Labs, AmEx · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Can a recursive CTE have more than one anchor query? Why would you want that?

The 30-second answer: Yes — the anchor portion can be a UNION ALL of multiple non-recursive queries, all combined with the recursive member by a final UNION ALL. You would want it when the seed of your recursion comes from multiple sources, like multiple roots of a forest of trees, or two different starting conditions for a graph walk.

Step-by-step thought process: 1. Show the structure — `WITH RECURSIVE cte AS (anchor1 UNION ALL anchor2 UNION ALL recursive_part)`. 2. Demonstrate with a real case — walking a forest of organisational hierarchies where there are multiple CEOs (one per business unit) all seeded together. 3. Mention that some engines are strict about the placement — the recursive reference can appear only in the final member, not in the anchors. 4. The result combines all seeds and walks them in parallel, returning every node reachable from any seed.

Edge cases to mention: - Multiple anchors must have matching column lists with the recursive part — same number, same types. - Different engines vary in how strictly they enforce the anchor-then-recursive structure. - Combining anchors with UNION (without ALL) is technically allowed, but again you risk dedupe surprises.

What NOT to say: - Do not say "a recursive CTE has exactly one anchor" — it can have several. - Do not put the recursive reference inside an anchor — the parser will reject it.

Common follow-up: *"Show me a recursive CTE that walks two separate hierarchies in one go."* — That is exactly the multiple-anchor pattern.

```
WITH RECURSIVE org_forest AS (  
  SELECT id, name, manager_id, 1 AS lvl FROM employees WHERE manager_id IS NULL AND bu = 'India'  
  UNION ALL  
  SELECT id, name, manager_id, 1 FROM employees WHERE manager_id IS NULL AND bu = 'APAC'  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id, o.lvl + 1  
  FROM employees e JOIN org_forest o ON e.manager_id = o.id  
)  
SELECT * FROM org_forest;
```

Q119 · Recursive CTE Returning a Hierarchy Depth Column

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wells Fargo · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Modify a recursive CTE on the employees table to return each employee's depth in the org chart.

The 30-second answer: Initialise a `depth` column to 1 in the anchor and increment it by one in the recursive part. Every output row then carries the number of management levels above it, which is useful for indenting display, capping recursion, or analysing org structure flatness.

Step-by-step thought process: 1. Show the anchor projecting `1 AS depth`. 2. The recursive part references the CTE's current depth and adds one to it for the next layer. 3. Filter on depth in the final SELECT to limit how deep you walk — or in the recursive WHERE to stop the recursion early. 4. Bonus pattern — add a path column at the same time, often as an array, so you can show the full chain from CEO to employee.

Edge cases to mention: - If your tree has multiple roots, all roots get depth 1, which may or may not be what you want; you can offset based on a separate logic. - Depth is integer; large hierarchies could exceed even int4 limits in theory, though never in practice. - The depth column is computed inside the recursion, so it can be used in the WHERE for early termination.

What NOT to say: - Do not compute depth in application code after fetching; it is trivially cheap inside the CTE. - Do not assume "level 0 is root" — choose your convention and stick with it.

Common follow-up: "Show me the max depth of the entire org." — `SELECT MAX(depth) FROM org_chart`.


```
WITH RECURSIVE org_chart AS (  
  SELECT id, name, manager_id, 1 AS depth FROM employees WHERE manager_id IS NULL  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id, o.depth + 1  
  FROM employees e JOIN org_chart o ON e.manager_id = o.id  
)  
SELECT id, name, depth FROM org_chart ORDER BY depth, name;
```

Q120 · Cycle Detection with a Path Column

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Show me how to detect and avoid cycles in a recursive CTE by tracking a path column.

The 30-second answer: Carry an array (Postgres) or a delimited string (other dialects) that accumulates the IDs of every node already visited along the current chain. In the recursive step, only join to nodes whose ID is not already in the path. When the path would close back on itself, that row is rejected, breaking the cycle.

Step-by-step thought process: 1. Initialise the path in the anchor with the seed ID — `ARRAY[id] AS path` in Postgres. 2. In the recursive step, the new path is the old path with the new node appended. 3. Filter the join with `WHERE new_node_id <> ALL(path)` — this is the cycle check. 4. The final SELECT can either expose the path column for debugging or simply omit it for cleaner output.

Edge cases to mention: - Postgres arrays are convenient; other engines use string concatenation or a temporary support table. - The path can grow large in deep hierarchies; for narrow trees this is fine, for wide graphs less so. - Make sure the path comparison is on a stable, unique key like the primary key.

What NOT to say: - Do not rely on engine recursion limits as your cycle defence — they fire on every long path, cyclic or not. - Do not skip the check on the assumption that "our data has no cycles" — production data tends to surprise.

Common follow-up: "How would you do this in a dialect without arrays?" — Concatenate IDs into a delimited string and search for the new ID inside it before joining.

```
WITH RECURSIVE walk AS (  
  SELECT id, parent_id, ARRAY[id] AS path FROM nodes WHERE id = 1  
  UNION ALL  
  SELECT n.id, n.parent_id, w.path || n.id  
  FROM nodes n JOIN walk w ON n.parent_id = w.id  
  WHERE n.id <> ALL(w.path)  
)  
SELECT * FROM walk;
```

Q121 · Using CTEs Inside Views

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant · Topic: CTEs

The question (interviewer's verbatim phrasing):

Can a view contain a CTE in its definition? Are there any restrictions?

The 30-second answer: Yes — a view's defining query can include one or more CTEs, including recursive CTEs, in every major engine that supports both features. The restrictions are the usual view restrictions — no parameters, no statement-level side effects, and updatability depends on whether the view qualifies as a simple updatable view.

Step-by-step thought process: 1. State the simple yes — `CREATE VIEW v AS WITH cte AS (... ) SELECT ... FROM cte` is valid. 2. Note that the CTE inside the view runs every time the view is queried (subject to the engine's optimisation rules). 3. Materialised views are a different story — they can also contain CTEs but the result is precomputed. 4. Acknowledge that very complex view definitions with deep CTE stacks can be hard to debug; sometimes a chain of simpler views is clearer.

Edge cases to mention: - A view with a recursive CTE is read-only — you cannot INSERT/UPDATE/DELETE through it. - Some engines optimise CTE-bearing views differently from inline CTEs — measure on your stack. - Permissions on the underlying tables are still required for the user querying the view, unless the view is defined with elevated privileges.

What NOT to say: - Do not say "views cannot have CTEs" — they absolutely can. - Do not nest the CTE outside the view definition — it goes inside.

Common follow-up: "Would you prefer a chain of views or one view with multiple CTEs?" — Depends on reuse. If each intermediate is reused elsewhere, separate views. If only the final result is consumed, one view with internal CTEs is cleaner.

```
CREATE VIEW top_customers AS
WITH spending AS (
    SELECT user_id, SUM(amount) AS total FROM orders GROUP BY user_id
)
SELECT u.id, u.name, s.total
FROM users u JOIN spending s ON u.id = s.user_id
WHERE s.total > 100000;
```

Q122 · Nested Recursive CTEs

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Can you have a recursive CTE that uses another recursive CTE in its definition?

The 30-second answer: You cannot literally nest one recursive CTE inside the definition of another — the syntax does not allow it. You can, however, have multiple recursive CTEs in the same WITH block where a later recursive CTE uses the result of an earlier recursive CTE as part of its anchor or recursive member.

Step-by-step thought process: 1. Clarify the terminology — "nested" usually means defined inside; in WITH-block terms, all CTEs are at the same level. 2. Show that with `WITH RECURSIVE a AS (...), RECURSIVE b AS (... FROM a ...)`, b can reference a (including a recursive a). 3. Demonstrate a realistic use — first recursive CTE walks the org chart, second recursive CTE walks a project dependency tree using employees from the first. 4. Mention that the second recursive CTE still self-references for its own recursion; a CTE cannot recurse through another CTE.

Edge cases to mention: - Performance can degrade quickly when two recursive walks compound; profile carefully. - Some engines require the RECURSIVE keyword only once for the whole WITH block; others allow it per CTE — check your dialect. - Cycle handling has to be done independently for each recursive CTE.

What NOT to say: - Do not try to literally embed a WITH RECURSIVE inside another CTE body — that is not legal SQL. - Do not assume the planner will optimise across the two recursions; usually each is treated independently.

Common follow-up: "Show me a case where this is actually useful." — Walking both a category hierarchy and a tag hierarchy in one statement, then joining at the leaves.

```
WITH RECURSIVE categories_tree AS (  
  SELECT id, name, parent_id FROM categories WHERE parent_id IS NULL  
  UNION ALL  
  SELECT c.id, c.name, c.parent_id FROM categories c JOIN categories_tree ct ON c.parent_id = ct.id  
)  
,  
RECURSIVE tag_tree AS (  
  SELECT id, name, parent_id FROM tags WHERE parent_id IS NULL  
  UNION ALL  
  SELECT t.id, t.name, t.parent_id FROM tags t JOIN tag_tree tt ON t.parent_id = tt.id  
)  
SELECT * FROM categories_tree ct JOIN tag_tree tt ON ct.name = tt.name;
```

Q123 · CTE with an Inline WINDOW Clause

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Swiggy · Topic: CTEs

The question (interviewer's verbatim phrasing):

Show me a CTE that defines a named WINDOW clause for use inside the CTE's own SELECT.

The 30-second answer: The WINDOW clause goes inside the CTE's SELECT body, after the FROM/ WHERE/GROUP BY but before ORDER BY. You name a window once and reference it from multiple OVER clauses in the same SELECT, which keeps long window-function lists readable.

Step-by-step thought process: 1. Show the structure — the named window lives after WHERE and is referenced by name in each OVER clause. 2. The advantage is DRY-ness — if three columns all use the same PARTITION BY and ORDER BY, you write it once. 3. The named window is scoped to that single SELECT, so it does not leak across CTEs. 4. Mention that this combines naturally with the window functions from Part A of this chapter.

Edge cases to mention: - Named windows are part of standard SQL but support varies — older MySQL versions do not have it. - You can layer named windows — one window inherits from another with **INHERITS** in some dialects. - The named window must be defined exactly once per SELECT; you cannot redefine it inside.

What NOT to say: - Do not duplicate the same OVER clause across multiple columns when a named window will do. - Do not assume the named window persists outside the SELECT — it does not.

Common follow-up: "How does this interact with PARTITION BY and ORDER BY in a window function?" — The named window holds the partition and order specification; each OVER reference uses them as if written inline.

```
WITH monthly_summary AS (  
  SELECT  
    user_id,  
    DATE_TRUNC('month', order_date) AS mo,  
    SUM(amount) AS revenue,  
    RANK() OVER w AS rnk,  
    SUM(amount) OVER w AS running_total  
  FROM orders  
  WHERE order_date >= '2026-01-01'  
  GROUP BY user_id, DATE_TRUNC('month', order_date)  
  WINDOW w AS (PARTITION BY user_id ORDER BY DATE_TRUNC('month', order_date))  
)  
SELECT * FROM monthly_summary;
```

Q124 · The UNION vs UNION ALL Bug in Recursive CTEs

Tags: Difficulty: Medium · Asked at: TCS, Wipro, Razorpay, Cred · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

A recursive CTE that uses plain UNION instead of UNION ALL — what bugs can that cause?

The 30-second answer: UNION applies an implicit DISTINCT against the entire accumulated result, which can silently drop rows you actually wanted, hide cycles instead of detecting them, and in some cases stop the recursion early by zeroing out the new-rows count. UNION ALL is the correct default; only use UNION if you specifically need deduplication and understand the implications.

Step-by-step thought process: 1. Explain the mechanical difference — UNION dedupes against everything; UNION ALL just appends. 2. Walk through the silent-drop scenario — a recursive walk that visits the same node twice along legitimately different paths loses one of them under UNION. 3. Walk through the termination scenario — if the recursive step keeps producing rows that already exist (because of a poorly chosen seed or join), UNION reports "zero new rows" and the recursion stops prematurely. 4. Recommend a default — UNION ALL plus an explicit dedup step at the end if you need uniqueness.

Edge cases to mention: - The DISTINCT cost of UNION grows with the result size — performance suffers as the CTE accumulates more rows. - UNION's dedupe is positional — column-by-column equality, including NULL semantics. - Some dialects warn about UNION inside recursive CTEs; others do not.

What NOT to say: - Do not say "UNION and UNION ALL are equivalent" — they are not, particularly in recursive contexts. - Do not use UNION "to be safe" in recursive CTEs; it is the unsafe choice.

Common follow-up: "How would you dedupe the final output of a recursive CTE if you really need it?" — Use SELECT DISTINCT in the final SELECT, or aggregate with GROUP BY — both run once, after the recursion is complete.

```
-- Wrong: UNION can silently dedupe paths and stop early
WITH RECURSIVE bad AS (
  SELECT id, parent_id FROM nodes WHERE id = 1
  UNION
  SELECT n.id, n.parent_id FROM nodes n JOIN bad b ON n.parent_id = b.id
);

-- Right: UNION ALL keeps every new row; dedupe at the end if needed
WITH RECURSIVE good AS (
  SELECT id, parent_id FROM nodes WHERE id = 1
  UNION ALL
  SELECT n.id, n.parent_id FROM nodes n JOIN good g ON n.parent_id = g.id
)
SELECT DISTINCT * FROM good;
```

Q125 · Postgres CYCLE Clause (14+)

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Postgres 14 added a CYCLE clause to recursive CTEs. Show me how it works and why it is better than manual path tracking.

The 30-second answer: The CYCLE clause tells Postgres to track a chosen set of columns across the recursion and add a boolean flag that becomes true when the row completes a cycle, plus a path column for inspection. You filter on the cycle flag in the final SELECT (or the recursive WHERE) to stop loops cleanly, without writing your own array logic.

Step-by-step thought process: 1. Show the syntax — `CYCLE id SET is_cycle USING path` placed after the recursive CTE's body. 2. The engine adds two new columns to the CTE: `is_cycle` (boolean) and `path` (array of the tracked columns). 3. In the recursive part, the engine itself prevents adding rows that would extend a cycle, or marks them as cycles depending on the variant used — check the exact semantics in the Postgres docs. 4. The win over manual tracking is correctness and brevity — fewer chances to mess up the cycle check, and the engine optimises the path tracking internally.

Edge cases to mention: - The CYCLE clause exists only on Postgres 14+ — earlier versions need manual array tracking. - Other ANSI-standard databases (Oracle, Db2) have similar built-in cycle

handling; SQL Server does not currently. - For very deep paths, the array column can become memory-heavy; you can drop it from the final SELECT to save bytes.

What NOT to say: - Do not say "CYCLE replaces UNION ALL" — it does not, it augments cycle handling on top of the existing structure. - Do not assume this works on every database; it is Postgres 14+ specific.

Common follow-up: "What if I need to detect cycles in MySQL?" — Manual path tracking with a JSON array or concatenated string; no built-in equivalent yet.

```
WITH RECURSIVE walk AS (  
  SELECT id, parent_id FROM nodes WHERE id = 1  
  UNION ALL  
  SELECT n.id, n.parent_id FROM nodes n JOIN walk w ON n.parent_id = w.id  
) CYCLE id SET is_cycle USING path  
SELECT * FROM walk WHERE NOT is_cycle;
```

Q126 · Postgres SEARCH BREADTH/DEPTH FIRST Clause (14+)

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Walmart Labs · Topic: Recursive CTEs

The question (interviewer's verbatim phrasing):

Explain the SEARCH BREADTH FIRST and SEARCH DEPTH FIRST clauses in Postgres recursive CTEs and when each is useful.

The 30-second answer: The SEARCH clause tells Postgres how to order the rows of the recursion's output — **BREADTH FIRST BY col SET ord** returns all rows at depth 1 before any row at depth 2, while **DEPTH FIRST BY col SET ord** walks one branch all the way down before returning to siblings. The engine adds an ordering column that you can use in the final ORDER BY.

Step-by-step thought process: 1. Define BFS vs DFS in plain terms — BFS visits siblings first, DFS visits children first. 2. Show the syntax — **SEARCH BREADTH FIRST BY id SET order_col** placed after the CTE body. 3. BFS is useful when you want shortest-distance-from-root output, or when you want to process levels uniformly. DFS is useful when you want one chain at a time, like printing an org tree with indentation. 4. The added ordering column is exposed to the outer query, so you ORDER BY it for the desired traversal.

Edge cases to mention: - Like CYCLE, the SEARCH clause is Postgres 14+ — earlier versions emulate the ordering manually with depth columns and array sort keys. - BFS can be more memory-intensive for wide trees; DFS is more memory-intensive for deep ones. - Combining SEARCH with CYCLE is allowed and frequently the right move on real graph data.

What NOT to say: - Do not say "BFS and DFS are computer-science concepts, SQL does not care" — Postgres explicitly cares as of 14. - Do not skip the ORDER BY in the outer query — without it the SEARCH-set column is computed but ignored.

Common follow-up: "*Why not just ORDER BY the depth column?*" — That gives you levels, but the SEARCH clause gives you the proper traversal sequence within each level too, matching the BFS or DFS order exactly.

```
WITH RECURSIVE org AS (  
  SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id FROM employees e JOIN org o ON e.manager_id = o.id  
) SEARCH BREADTH FIRST BY id SET seq  
SELECT id, name FROM org ORDER BY seq;
```

Chapter 4, Part A — Index Types & Design

Indexes are the single biggest performance lever in SQL — a missing index turns a 5 ms query into a 5 second one, and a wrongly designed index can silently double your write cost without helping a single read. This section walks through the index types you'll actually use in production (B-tree, hash, GIN, GiST, BRIN, covering, partial, expression), the rules for designing composite indexes that the planner will actually pick, and the workload-based thinking that separates a candidate who memorised "add an index on the WHERE column" from one who can sit with a DBA and defend their choices. Part B picks up from here and covers query plans, EXPLAIN reading, statistics, and optimizer hints.

Q127 · B-tree Index Internals — What's Actually Stored

Tags: Difficulty: Medium · Asked at: Razorpay, Walmart Labs, Goldman, Flipkart · Topic: Index Internals

The question (interviewer's verbatim phrasing):

Explain how a B-tree index actually works under the hood. What does the database store, and why is lookup $O(\log n)$?

The 30-second answer: A B-tree index is a balanced multi-way tree where each node holds sorted key values and pointers to child nodes; the leaves hold the actual key plus a row pointer (a heap TID in Postgres, the clustered key in InnoDB). Lookup walks from root to leaf, doing one disk page read per level, and because each node fans out to hundreds of children, even a billion-row table is only 4-5 levels deep — hence the logarithmic cost.

Step-by-step thought process: 1. Start with the shape — sorted, balanced, fan-out of 100-500 per node depending on page size and key width. 2. Explain that internal nodes are just routing — they hold separator keys and child pointers, not row data. 3. Leaves hold either the row pointer (Postgres heap) or the row itself if the index is the clustered index (InnoDB primary key). 4. The "log n" is base-fanout, not base-2 — that's why a 1 billion row table needs only ~5 page reads. 5. Mention sibling pointers at the leaf level — that's what makes range scans cheap (`WHERE created_at BETWEEN ...`).

Edge cases to mention: - Index page splits happen when a leaf fills up — random inserts cause more splits than sequential ones, which is why UUID primary keys fragment InnoDB badly. - Wide keys (long varchar) reduce fan-out and push tree height up — prefer narrow keys where you can. - Deleted entries leave tombstones until VACUUM (Postgres) or purge (InnoDB) reclaims them — that's part of why index bloat happens.

What NOT to say: - Do not say "indexes are sorted arrays" — they aren't, and the difference matters for inserts. - Do not confuse B-tree with B+tree blindly — most production engines (Postgres, InnoDB, SQL Server) use B+tree variants where only leaves hold data pointers.

Common follow-up: *"Why does the planner sometimes skip the index and do a sequential scan?"* — When the estimated rows returned exceed roughly 5-10% of the table, random I/O from index lookups costs more than a sequential scan, so the planner picks the seq scan.

```
-- A simple B-tree index — the workhorse
CREATE INDEX idx_orders_user_id ON orders(user_id);

-- Range scans use the leaf-level sibling pointers
SELECT * FROM orders WHERE created_at BETWEEN '2026-05-01' AND '2026-05-26';
```

Q128 · Hash Index vs B-tree — When Each Wins

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, JP Morgan, Zerodha · Topic: Index Types

The question (interviewer's verbatim phrasing):

What's the difference between a hash index and a B-tree index? When would you pick a hash index?

The 30-second answer: Hash indexes support only equality lookups (`=`) and are $O(1)$ on average, while B-trees support equality, range, ordering and LIKE-prefix in $O(\log n)$. You pick hash when the column is only ever filtered by exact match and you want the tiny speed edge — but in practice B-tree is the default for almost everything.

Step-by-step thought process: 1. State the core trade-off — hash trades versatility for a constant-time lookup. 2. List what hash cannot do: range scans, ORDER BY, LIKE 'prefix%', anything other than `=`. 3. Mention engine support: Postgres has hash indexes (WAL-logged since 10), MySQL InnoDB has adaptive hash indexes built on top of B-trees automatically, MySQL MEMORY engine uses hash by default. 4. Real-world verdict: 95% of the time you should reach for B-tree first, hash only when profiling shows a hot equality lookup on a high-cardinality column.

Edge cases to mention: - Hash collisions degrade performance — a poorly distributed key (low cardinality) makes hash indexes worse than B-tree. - Postgres hash indexes were not crash-safe before version 10 — older blog posts will tell you to avoid them; that advice is outdated. - InnoDB's adaptive hash index is automatic and invisible — you cannot manually create one, but it kicks in for hot equality lookups.

What NOT to say: - Do not claim hash indexes are "always faster" — they lose immediately on any non-equality predicate. - Do not recommend hash indexes for primary keys — you lose ordered scans, which kill pagination performance.

Common follow-up: "Can a hash index help with `WHERE user_id IN (1, 2, 3)`?" — Yes for each individual value, but the planner has to do three separate hash lookups; a B-tree might do one range scan if the values are clustered.

```
-- Postgres hash index — only for = lookups
CREATE INDEX idx_sessions_token_hash ON sessions USING hash(session_token);

-- This works
SELECT * FROM sessions WHERE session_token = 'abc123';

-- This does NOT use the hash index — falls back to seq scan
SELECT * FROM sessions WHERE session_token LIKE 'abc%';
```

Q129 · Composite Index Column Order — The Leftmost-Prefix Rule

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Meesho, Razorpay · Topic: Composite Indexes

The question (interviewer's verbatim phrasing):

You create an index on `(user_id, status, created_at)`. Which of these queries can use the index efficiently?

The 30-second answer: The leftmost-prefix rule says a composite index on `(a, b, c)` can serve queries that filter on `a`, on `(a, b)`, or on `(a, b, c)` — but not on `b` alone, `c` alone, or `(b, c)`. The columns must be used in the index's left-to-right order, without skipping.

Step-by-step thought process: 1. Picture the index as one long sorted key — first by `a`, then by `b` within each `a`, then by `c` within each `(a, b)`. 2. A query filtering only on `b` cannot binary-search that structure, because rows with the same `b` are scattered across every value of `a`. 3. The skip-scan optimisation (Oracle, MySQL 8 partially) lets the planner iterate through distinct `a` values and seek within each — only viable when `a` has low cardinality. 4. Range predicates "stop" the index — after the first range column, the rest of the index is useful only for ordering, not for filtering. So `WHERE user_id = 5 AND status = 'paid' AND created_at > '2026-01-01'` uses all three; `WHERE user_id = 5 AND created_at > '2026-01-01' AND status = 'paid'` effectively only uses `user_id` and `created_at`.

Edge cases to mention: - ORDER BY can use the trailing columns of the index even if you haven't filtered on them, as long as you've equality-filtered the prefix. - A composite index on `(a, b)` can serve `WHERE a = 5 ORDER BY b` with no separate sort step. - Column order should put the most selective equality column first, but only if it's actually used in queries.

What NOT to say: - Do not say "the index can be used in any order" — that's only true with covering index-only scans on small tables. - Do not blindly put the highest-cardinality column first — put the column that appears in the WHERE clause of every query first.

Common follow-up: "Why don't we just create three separate single-column indexes?" — Because composite indexes can satisfy a multi-column WHERE in one B-tree traversal; with three single-column indexes, the planner has to do a bitmap AND across three index scans, which is slower for selective combined predicates.

```
-- Composite index – order matters
CREATE INDEX idx_orders_user_status_date ON orders(user_id, status, created_at);

-- Uses full index
SELECT * FROM orders WHERE user_id = 42 AND status = 'paid' AND created_at > '2026-05-01';

-- Cannot use index efficiently – no leftmost prefix
SELECT * FROM orders WHERE status = 'paid' AND created_at > '2026-05-01';
```

Q130 · Covering Index — INCLUDE Columns and Index-Only Scans

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, JP Morgan, AmEx · Topic: Covering Indexes

The question (interviewer's verbatim phrasing):

What is a covering index, and how is `INCLUDE` different from just adding columns to the index key?

The 30-second answer: A covering index contains every column the query needs — both for filtering and for output — so the planner can satisfy the query from the index alone without touching the heap.

INCLUDE (Postgres 11+, SQL Server) appends columns to the leaf entries as payload, so they're available for SELECT but don't participate in the B-tree key ordering, sort order, or uniqueness.

Step-by-step thought process: 1. Define the goal — eliminate the heap fetch step, which is the expensive random I/O. 2. Explain why **INCLUDE** is preferred over extending the key: included columns don't bloat the internal B-tree pages, keep fan-out high, and don't break the leftmost-prefix rule for other queries. 3. Walk through a concrete win: a payments table with millions of rows, query **SELECT amount, currency FROM payments WHERE order_id = ?**. Index on **(order_id) INCLUDE (amount, currency)** lets Postgres return both columns from the index leaf directly. 4. Mention the cost — index size grows, write amplification rises slightly because every UPDATE to an included column must update the index too.

Edge cases to mention: - MySQL doesn't have **INCLUDE** syntax; you get covering by extending the composite index, accepting the key-size cost. - Postgres requires VACUUM to mark the visibility map for index-only scans to actually skip the heap — on a heavily updated table, you may still see heap fetches in EXPLAIN. - Don't include wide TEXT or JSON columns — they bloat the index without giving you the speed you'd hope for.

What NOT to say: - Do not call every multi-column index a "covering index" — it's only covering for a specific query. - Do not include columns that change frequently — every UPDATE on them triggers an index update.

Common follow-up: "How do you verify the planner is actually doing an index-only scan?" — **EXPLAIN ANALYZE** shows **Index Only Scan** as the node type, plus a **Heap Fetches: 0** line in Postgres if the visibility map is up to date.

```
-- Postgres - INCLUDE keeps key narrow, payload available
CREATE INDEX idx_payments_order_covering
ON payments(order_id) INCLUDE (amount, currency);

-- Index-only scan
SELECT amount, currency FROM payments WHERE order_id = 12345;
```

Q131 · Partial Index — Indexing Only the Rows That Matter

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, PhonePe · Topic: Partial Indexes

The question (interviewer's verbatim phrasing):

You have a payments table with 10 million rows but only 50,000 are in 'pending' status. How would you index the pending ones efficiently?

The 30-second answer: Create a partial index with a `WHERE status = 'pending'` clause — Postgres builds the B-tree over only the matching rows, so the index is 200x smaller, fits in memory, and is dramatically faster for queries that filter the same predicate. SQL Server calls this a filtered index; MySQL has no native equivalent.

Step-by-step thought process: 1. Frame the problem — most rows are dead weight for this query; indexing them is wasteful. 2. Show the syntax: `CREATE INDEX idx ON payments(created_at) WHERE status = 'pending'`. 3. Explain the matching rule — the planner uses the partial index only when the query's WHERE clause logically implies the index's WHERE clause. So `WHERE status = 'pending' AND created_at > '2026-05-01'` matches; `WHERE created_at > '2026-05-01'` alone does not. 4. Real-world hit list: soft-delete patterns (`WHERE deleted_at IS NULL`), state machines (`WHERE status IN ('pending', 'processing')`), event tables (`WHERE event_type = 'payment_failed'`).

Edge cases to mention: - The predicate must be immutable — you cannot use `WHERE created_at > NOW() - INTERVAL '7 days'` because `NOW()` is volatile. - The planner has to prove logical implication — sometimes adding a redundant filter to your query is needed to nudge it. - Partial unique indexes are powerful: `CREATE UNIQUE INDEX ON users(email) WHERE deleted_at IS NULL` lets you enforce uniqueness only among active rows.

What NOT to say: - Do not say "MySQL supports partial indexes via index prefix" — those are different things; prefix indexes index the first N characters of a string column. - Do not create a partial index on a predicate that doesn't appear in your queries — it'll never be used.

Common follow-up: *"What happens when a row's status changes from 'pending' to 'paid'?"* — Postgres removes the row's entry from the partial index automatically as part of the UPDATE, similar to how it handles any indexed column change.

```
-- Partial index - Postgres
CREATE INDEX idx_payments_pending_created
ON payments(created_at)
WHERE status = 'pending';

-- Uses the partial index
SELECT * FROM payments WHERE status = 'pending' AND created_at > '2026-05-20';
```

Q132 · Expression Index — Indexing the Result of a Function

Tags: Difficulty: Medium · Asked at: Flipkart, JP Morgan, Wells Fargo, Cred · Topic: Functional Indexes

The question (interviewer's verbatim phrasing):

Your query is `WHERE LOWER(email) = ?`. The index on `email` isn't being used. How do you fix it?

The 30-second answer: Create an expression index on the same expression you're filtering by — `CREATE INDEX idx_users_email_lower ON users(LOWER(email))`. The planner can only use an index when the query's expression matches the indexed expression exactly, so wrapping the column in `LOWER()` in your WHERE clause invalidates a plain column index.

Step-by-step thought process: 1. Diagnose: a function on the column kills index usage unless an index exists on that function's output. 2. Show the fix with Postgres and Oracle syntax — both support expression indexes natively. 3. Common cases: case-insensitive search (`LOWER` , `UPPER`), date truncation (`DATE(timestamp)` , `DATE_TRUNC('day', ts)`), JSON path extraction (`(payload->>'user_id')`), trimmed strings (`TRIM(name)`). 4. Mention MySQL — functional indexes were added in 8.0.13; before that you had to add a generated column and index that.

Edge cases to mention: - The function must be IMMUTABLE in Postgres — depending on session settings, `LOWER` is technically marked immutable only because Postgres assumes the locale doesn't change at runtime; pay attention to collation-aware functions. - Inserts and updates pay the cost of evaluating the function each time — a heavy regex or JSON parse on every write hurts. - Expression indexes don't show up in `\d table_name` cleanly — they appear as `(expression)` rather than a column name.

What NOT to say: - Do not rewrite the query to avoid the function if the function is semantically required — the index is the correct fix, not contorting the SQL. - Do not create an expression index for a function that's used in only one query — measure first.

Common follow-up: "What if I want a case-insensitive comparison without an expression index?" — Use the `citext` extension in Postgres or a case-insensitive collation in MySQL/SQL Server — those keep the column itself case-insensitive and let plain indexes work.

```
-- Postgres expression index
CREATE INDEX idx_users_email_lower ON users(LOWER(email));

-- Now this uses the index
SELECT * FROM users WHERE LOWER(email) = 'rohan@example.in';

-- Date truncation index
CREATE INDEX idx_orders_created_date ON orders(DATE(created_at));
SELECT COUNT(*) FROM orders WHERE DATE(created_at) = '2026-05-26';
```

Q133 · Unique Index vs UNIQUE Constraint

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Constraints & Indexes

The question (interviewer's verbatim phrasing):

What's the difference between a unique index and a UNIQUE constraint?

The 30-second answer: Functionally they're the same — both enforce uniqueness and both are implemented under the hood by a unique B-tree index. The difference is metadata: a UNIQUE constraint is a logical declaration that other constraints (like foreign keys) can reference; a unique index is just an index that happens to be unique. In Postgres and MySQL, declaring a constraint automatically creates the index.

Step-by-step thought process: 1. Lead with "they share the implementation" — that disarms the trick framing. 2. Explain the metadata angle: a foreign key can reference a column that has a UNIQUE constraint, but not always a column with just a unique index (depends on engine). 3. Practical impact: a partial unique index is possible (`WHERE deleted_at IS NULL`), but a UNIQUE constraint cannot have a WHERE clause — so partial uniqueness must use the index syntax. 4. Tooling — `pg_dump` and ORM migrations may treat them differently when generating schema diffs.

Edge cases to mention: - Postgres treats them almost identically; `\d` shows the unique index, and a constraint creates one of the same name. - Composite UNIQUE constraints are common (`UNIQUE(user_id, product_id)` in a cart table) — same as a composite unique index. - Adding a UNIQUE constraint on a table with duplicate data fails — you must clean the data first.

What NOT to say: - Do not say "constraints are slower than indexes" — they aren't, because they ARE indexes underneath. - Do not skip the partial uniqueness trick — that's the practical scenario where the choice matters.

Common follow-up: "Can I drop the unique index that backs a UNIQUE constraint?" — No — you have to drop the constraint first, which then drops its backing index.

```
-- Constraint form — declarative, FK-referenceable
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE(email);

-- Unique index form — supports partial uniqueness
CREATE UNIQUE INDEX uq_users_email_active
ON users(email) WHERE deleted_at IS NULL;
```

Q134 · Primary Key Index — Clustered vs Heap-Organised

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, AmEx · Topic: Storage Internals

The question (interviewer's verbatim phrasing):

Explain the difference between how MySQL InnoDB and Postgres physically store a table relative to its primary key.

The 30-second answer: InnoDB and SQL Server use clustered indexes — the primary key IS the table; rows live in the leaf pages of the PK B-tree, sorted by the PK. Postgres uses a heap — rows are stored in insertion order in a separate heap file, and even the primary key is just a secondary index pointing to heap TIDs. This explains why InnoDB primary key lookups are one I/O and secondary index lookups are two, while Postgres treats them all equally.

Step-by-step thought process: 1. Open with the architectural contrast — clustered vs heap is the headline. 2. Show the lookup cost: InnoDB secondary index points to the PK value, which then traverses the PK tree (two B-tree walks). Postgres secondary index points to a heap TID directly (one B-tree walk + one heap page read). 3. Practical implication for InnoDB — random UUID primary keys cause leaf page splits everywhere; sequential keys (BIGINT AUTO_INCREMENT) keep inserts at the rightmost leaf. 4. Implication for Postgres — CLUSTER command can physically reorder a heap to match an index, but it's a one-time operation; new inserts go back to wherever there's free space.

Edge cases to mention: - SQL Server lets you choose — clustered index can be the PK or a different column, and you can have a heap table with no clustered index at all. - InnoDB hides a 6-byte `DB_ROW_ID` if you don't declare a PK — but you'd be silly not to. - Postgres has TOAST tables for large columns — those live separately from the main heap, with their own access path.

What NOT to say: - Do not claim Postgres "doesn't have clustered indexes at all" — CLUSTER command exists and physically reorders the heap; it's just not maintained continuously. - Do not say InnoDB secondary indexes "point to row IDs" — they point to primary key values.

Common follow-up: "Why does InnoDB recommend an integer surrogate primary key over a natural key?" — Because every secondary index stores the full PK as the row pointer. A natural PK like `email varchar(255)` makes every secondary index huge; a 4-byte INT keeps them lean.

```
-- InnoDB — sequential PK keeps leaf splits localised
CREATE TABLE orders (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  amount DECIMAL(10,2)
) ENGINE=InnoDB;

-- Postgres CLUSTER — one-time physical reorder
CLUSTER orders USING idx_orders_created_at;
```


Q135 · Indexing Foreign Key Columns — The Most Forgotten Win

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Flipkart · Topic: FK Indexes

The question (interviewer's verbatim phrasing):

You have an `orders` table with a foreign key to `users`. Do you need a separate index on the FK column, or does the FK constraint create one automatically?

The 30-second answer: MySQL InnoDB automatically creates an index for every foreign key, but Postgres and SQL Server do not — you must add it manually. Without that index, every join from the parent side and every CASCADE delete on the parent triggers a full sequential scan of the child table. This is one of the most common overlooked indexes in production.

Step-by-step thought process: 1. Lead with the engine difference — interviewers love this gotcha. 2. Explain the two pain points: joins (`SELECT * FROM users u JOIN orders o ON o.user_id = u.id`) and parent-side deletes (Postgres has to scan the child table to enforce `ON DELETE RESTRICT` or to actually cascade). 3. Diagnose: a slow DELETE on the parent that's "doing the right thing" almost always means the child's FK column isn't indexed. 4. Standard rule: index every FK column unless you can prove the child is small and the FK is rarely joined on.

Edge cases to mention: - The FK column might be the leading column of an existing composite index — check before adding a duplicate. - For high-write child tables, the FK index pays a write cost — measure if the FK is genuinely needed. - If the FK is `ON DELETE NO ACTION DEFERRABLE`, the locking penalty without the index is brutal during bulk deletes.

What NOT to say: - Do not assume your ORM created the index — most ORMs don't, including Django and Rails until recently. - Do not skip the index because "we never DELETE the parent" — joins also benefit.

Common follow-up: "How do you find unindexed FKs across your whole schema?" — Postgres has a well-known query joining `pg_constraint` and `pg_index`; MySQL Workbench and pt-duplicate-key-checker can flag them.

```
-- Postgres — must add manually
CREATE INDEX idx_orders_user_id ON orders(user_id);

-- Without this index, DELETE FROM users WHERE id = 42
-- triggers a seq scan on orders to check the FK
```

Q136 · Indexing Low-Cardinality Columns — When the Planner Skips

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay · Topic: Index Selectivity

The question (interviewer's verbatim phrasing):

You added an index on the `status` column which has only three values — and the planner still does a sequential scan. Why, and how do you make it useful?

The 30-second answer: Low-cardinality columns produce indexes where each value matches a huge chunk of the table — the planner correctly judges that random heap I/O across that many rows costs more than a sequential scan. Make the index useful by combining it with a more selective column in a composite index, by making it partial (`WHERE status = 'pending'`), or by accepting that you don't need an index at all.

Step-by-step thought process: 1. Explain the planner's reasoning — when `status = 'paid'` matches 80% of rows, scanning the index then jumping back to the heap 80 million times beats nothing. 2. Three real fixes: a. Composite index (`status, created_at`) — the leading status narrows, the trailing column adds selectivity. b. Partial index `WHERE status = 'pending'` — useful when one value is rare (the "pending" state in a state machine often is). c. BRIN or bitmap index if the table is huge and queries scan ranges of rows. 3. Acknowledge that for a column like a boolean `is_active` where 99% is true, a partial index on `WHERE is_active = false` is often the winning pattern.

Edge cases to mention: - The 5-10% rule is rough — it depends on the row width, page fill factor, and whether the index is covering. - Skewed distributions matter — if 'pending' is 1% and 'paid' is 99%, the same index helps for 'pending' queries and is skipped for 'paid' queries. - ANALYZE feeds the planner's statistics — stale stats cause wrong choices on low-cardinality columns more than anywhere else.

What NOT to say: - Do not say "low-cardinality columns should never be indexed" — partial and composite usage break that rule routinely. - Do not force the index with hints just because you can see it — the planner is usually right.

Common follow-up: *"What's the threshold where cardinality becomes 'high enough'?"* — There's no fixed number; what matters is selectivity per query. A column with 10 values where the typical query filter matches 100 rows out of 10 million is plenty selective.

```
-- Bad: indexing a low-cardinality column alone
CREATE INDEX idx_payments_status ON payments(status); -- often skipped

-- Better: composite with a selective second column
CREATE INDEX idx_payments_status_created ON payments(status, created_at);

-- Best for skewed: partial index on the rare value
CREATE INDEX idx_payments_pending ON payments(created_at) WHERE status = 'pending';
```

Q137 · GIN Index — JSON, Arrays and Full-Text in Postgres

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Meesho · Topic: Postgres Index Types

The question (interviewer's verbatim phrasing):

You're storing event payloads as JSONB and queries filter on `payload->>'merchant_id'`.
What index type should you use?

The 30-second answer: Use a GIN (Generalised Inverted Index) — it's designed for composite values where you want to look up rows containing specific elements. For JSONB, GIN can index every key-value pair (`USING GIN (payload jsonb_path_ops)`) so that `WHERE payload @> '{"merchant_id": "M123"}'` is fast. GIN is also the right choice for array containment (`WHERE tags @> ARRAY['fraud']`) and `tsvector` full-text search.

Step-by-step thought process: 1. Explain GIN's structure conceptually — it's an inverted index, like a search engine: for each indexed term, it stores the list of rows containing that term. 2. JSONB-specific advice: default operator class indexes all keys and values (bigger index, supports more operators); `jsonb_path_ops` indexes only paths (smaller, faster, supports only `@>`). 3. For expression queries (`payload->>'merchant_id' = 'M123'`), an expression B-tree index on that specific path is often faster than GIN if you only care about one key. 4. GIN writes are expensive — every key in the JSONB value becomes an index entry; the `fastupdate` setting batches them but trades query speed.

Edge cases to mention: - GIN on arrays supports `&&` (overlap), `@>` (contains), `<@` (contained by) — pick the right operator class. - Full-text search needs `to_tsvector` either at query time (slow) or as a generated/expression-indexed column. - GIN doesn't support ORDER BY — for ranked full-text, you index with GIN and sort using `ts_rank` separately.

What NOT to say: - Do not use GIN for plain scalar columns — B-tree wins every time. - Do not skip the operator class choice — `jsonb_path_ops` vs default is a real trade-off, not a footnote.

Common follow-up: "How do I full-text search a `description` column?" — `CREATE INDEX idx_products_desc_fts ON products USING GIN (to_tsvector('english', description))` then `WHERE to_tsvector('english', description) @@ to_tsquery('hindi & cotton')`.

```
-- JSONB containment with GIN
CREATE INDEX idx_events_payload_gin ON events USING GIN (payload jsonb_path_ops);
SELECT * FROM events WHERE payload @> '{"merchant_id": "M123"}';

-- Array overlap
CREATE INDEX idx_orders_tags_gin ON orders USING GIN (tags);
SELECT * FROM orders WHERE tags && ARRAY['fraud', 'review'];
```

Q138 · GiST Index — Geometric and Range Types

Tags: Difficulty: Hard · Asked at: Swiggy, Flipkart, Meesho, Zomato · Topic: Postgres Index Types

The question (interviewer's verbatim phrasing):

Your delivery app needs to find all riders within 5 km of a pickup point. What index supports this?

The 30-second answer: GiST (Generalised Search Tree) — a balanced tree where each node holds a bounding box (or analogous "key" for the data type) and supports operators like "contains," "overlaps," "nearest neighbour." For geospatial work you use PostGIS, which adds the `geometry` type and a GiST operator class so `ORDER BY location <-> point('77.5946 12.9716')` returns nearest riders in true index-sorted order.

Step-by-step thought process: 1. Frame GiST as a generalised structure — not just for geometry; also for range types (`tstzrange` , `int4range`) and exclusion constraints. 2. Walk through the geospatial query — `<->` is the distance operator, GiST powers k-nearest-neighbour without scanning every row. 3. Range type use case — `EXCLUDE USING GIST (room WITH =, during WITH &&)` enforces that no two bookings overlap for the same room. 4. Compare with GIN — GIN is better for full containment lookups on composite values; GiST is better for overlap, distance, and range operations.

Edge cases to mention: - GiST is lossy by default — the index gives a candidate set and the engine rechecks each row against the real predicate. - SP-GiST is a related index type for non-balanced trees (quadrees, suffix trees) — niche but useful for specific data shapes. - PostGIS extension is required for serious geospatial work; the built-in `point` type is limited.

What NOT to say: - Do not say "GiST is just for geometry" — range types and exclusion constraints are huge use cases. - Do not use GiST for plain text or integer columns — you're paying a complexity cost for no benefit.

Common follow-up: "How would you enforce 'no two events for the same room can overlap in time'?" — `ALTER TABLE bookings ADD CONSTRAINT no_overlap EXCLUDE USING GIST (room_id WITH =, during WITH &&)` — exclusion constraints are GiST's killer feature.

```
-- PostGIS nearest-neighbour
CREATE INDEX idx_riders_location ON riders USING GIST(location);
SELECT id, name FROM riders
ORDER BY location <-> ST_MakePoint(77.5946, 12.9716)::geography
LIMIT 10;

-- Range exclusion
ALTER TABLE bookings ADD CONSTRAINT no_overlap
EXCLUDE USING GIST (room_id WITH =, during WITH &&);
```

Q139 · BRIN Index — Append-Only Large Tables

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Wells Fargo, Goldman · Topic: BRIN Indexes

The question (interviewer's verbatim phrasing):

You have a transactions log table with 5 billion rows, append-only, queries always filter on `created_at`. A B-tree index would be 100 GB. What's the alternative?

The 30-second answer: A BRIN (Block Range Index) — Postgres divides the heap into ranges (128 pages by default) and stores only the min/max of the indexed column per range. The index is microscopic (a few MB for billions of rows) and works brilliantly when data is physically clustered by the indexed column, which is naturally true for append-only time-series tables.

Step-by-step thought process: 1. Contrast the size — B-tree stores one entry per row (~30-40 bytes each), BRIN stores one summary per N pages. 2. Explain the trade-off: BRIN narrows the search to candidate page ranges, then the engine scans those ranges. For `created_at`-correlated data, that's a tight range; for random columns, BRIN devolves to a full scan. 3. Tunable: `pages_per_range` parameter trades index size vs scan precision — smaller ranges give tighter pruning at higher index size. 4. Use cases: log tables, sensor data, financial transactions, anything that grows by appending and is queried by the same time column.

Edge cases to mention: - Correlation matters enormously — check `pg_stats.correlation` for the column; values near 1 or -1 mean BRIN will work well. - VACUUM updates BRIN summaries — long-running transactions can leave summaries stale. - Multi-column BRIN exists but is rarely useful — the correlation argument usually only holds for one column at a time.

What NOT to say: - Do not use BRIN on a column with random distribution — it'll be useless and you'll blame the engine. - Do not expect BRIN to replace B-tree for OLTP point lookups — it's a range scan tool.

Common follow-up: "How does BRIN compare to a partitioned table?" — They're complementary. Partitioning is a coarser-grained version of the same idea (skip whole partitions). BRIN works within a single table; partitioning is structural and helps with maintenance too.

```
-- BRIN on append-only timestamp column
CREATE INDEX idx_transactions_created_brin
ON transactions USING BRIN (created_at) WITH (pages_per_range = 64);

-- Tight range query — BRIN narrows to a few page ranges
SELECT * FROM transactions
WHERE created_at BETWEEN '2026-05-25' AND '2026-05-26';
```

Q140 · Hash Index in Postgres — Old Reputation vs Modern Reality

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman, AmEx · Topic: Postgres Index Types

The question (interviewer's verbatim phrasing):

Why did Postgres engineers historically tell people to avoid hash indexes, and is that advice still valid?

The 30-second answer: Before Postgres 10, hash indexes were not WAL-logged — they couldn't be replicated and weren't crash-safe; you had to REINDEX after a crash. Postgres 10 fixed this completely. Modern hash indexes are crash-safe, replicated, and slightly smaller than B-tree for equality-only workloads on long strings. They're still niche, but the old "never use them" advice is outdated.

Step-by-step thought process: 1. Explain the historical reason — the avoidance wasn't because hash was inherently bad, it was because the implementation was incomplete. 2. State what changed in version 10: full WAL logging, replication support, crash recovery. 3. When to actually pick hash today: high-cardinality columns with frequent equality lookups, where the key is long (so hash buckets save space vs B-tree leaf entries), and where you never need range scans or ORDER BY. 4. Acknowledge the practical truth — B-tree's versatility means most teams stick with it; hash usage in Postgres remains rare.

Edge cases to mention: - Hash indexes still don't support multicolumn indexes — only single-column. - They don't support uniqueness enforcement — for unique constraints, you need B-tree. - `pg_stat_user_indexes` shows scan counts — measure before committing to hash.

What NOT to say: - Do not parrot "hash indexes are deprecated in Postgres" — they're actively maintained and improved. - Do not assume hash is faster than B-tree by default — modern B-trees are extremely fast for equality lookups too.

Common follow-up: "Why doesn't hash support multi-column?" — Because the hash of `(a, b)` together is unrelated to hashes of `a` and `b` separately — you'd lose the leftmost-prefix flexibility entirely, and there's no commonly useful operator that benefits.

```
-- Postgres 10+ hash index – safe and replicated
CREATE INDEX idx_sessions_token_hash
ON sessions USING HASH (session_token);

-- Equality only
SELECT user_id FROM sessions WHERE session_token = 'eyJhbGc...';
```

Q141 · Composite vs Multiple Single-Column Indexes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, Flipkart · Topic: Index Strategy

The question (interviewer's verbatim phrasing):

Should I create one index on `(user_id, status)` or two separate indexes on `user_id` and `status` ?

The 30-second answer: For a query that filters on both columns together, the composite index is faster — one B-tree traversal finds the exact rows. Two single-column indexes force the planner to do a bitmap AND across two index scans, which costs more. But two single-column indexes are more flexible — they can serve queries that filter on either column alone. Pick composite when the queries always use both; pick singles when each column is independently filtered.

Step-by-step thought process: 1. Frame the trade-off clearly — speed for the combined case vs flexibility for individual cases. 2. Explain bitmap AND: each single-column index returns a bitmap of matching row pointers, the engine ANDs them, then fetches the surviving heap pages. Two random I/O paths instead of one. 3. The leftmost-prefix rule means the composite `(user_id, status)` already serves `WHERE user_id = ?` alone — so you might only need the composite + one separate index on `status` if `status`-only queries exist. 4. Write cost reminder — every additional index adds INSERT/UPDATE cost. Three indexes is three writes.

Edge cases to mention: - For OLAP workloads with diverse predicates, bitmap-AND on single-column indexes scales surprisingly well — that's why Postgres invests in it. - For OLTP with hot, predictable queries, composite always wins. - If the second column is low-cardinality, the composite index isn't much bigger than the single-column one — go composite without thinking.

What NOT to say: - Do not say "more indexes is always better" — write amplification is real. - Do not say "always use composite" — leftmost-prefix means a composite on `(a, b)` doesn't help a `WHERE b = ?` query at all.

Common follow-up: "How do I know what queries my app actually runs?" — Postgres `pg_stat_statements` extension, MySQL `performance_schema.events_statements_summary_by_digest`, or query logging — never guess, measure.

```
-- Composite — best for combined filter
CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- Two singles — better when each is independently filtered
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status);
```


Q142 · Index Design Decisions — Given This Workload, What Indexes?

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, JP Morgan · Topic: Index Design Strategy

The question (interviewer's verbatim phrasing):

Here's the workload for a `payments` table: 10 million rows, 95% reads / 5% writes, top queries are (a) `WHERE user_id = ? ORDER BY created_at DESC LIMIT 20`, (b) `WHERE status = 'pending' AND created_at < NOW() - INTERVAL '1 hour'`, (c) `WHERE order_id = ?`, (d) `SELECT SUM(amount) FROM payments WHERE merchant_id = ? AND created_at BETWEEN ? AND ?`. The table has columns `id PK`, `user_id`, `order_id`, `merchant_id`, `status`, `amount`, `currency`, `created_at`. Walk me through which indexes you'd create and why.

The 30-second answer: I'd add four indexes: a composite `(user_id, created_at DESC)` for query (a) — it serves the filter and the sort with no extra step; a partial `(created_at) WHERE status = 'pending'` for query (b) — tiny index since pending is rare; a single `(order_id)` for query (c) — high cardinality, frequent point lookup; and a covering composite `(merchant_id, created_at) INCLUDE (amount)` for query (d) so the SUM can be served from the index alone. With reads at 95%, the write-amplification cost of four indexes is acceptable.

Step-by-step thought process: 1. **Start by listing query patterns, not columns** — you index for queries, not for columns "that look important." 2. **Query (a) — `user_id` + `ORDER BY created_at LIMIT 20`:** A composite on `(user_id, created_at DESC)` lets the planner seek to the user, then walk the leaf in reverse order, stopping after 20 rows. Critical for pagination performance. If I created just `(user_id)`, the planner would fetch all of that user's payments and then sort them — fine for a user with 50 payments, catastrophic for a power user with 50,000. 3. **Query (b) — `pending payments older than 1 hour`:** Status is low-cardinality (3-4 values), but 'pending' is a tiny minority (maybe 0.1% of rows because most payments resolve in seconds). A partial index `(created_at) WHERE status = 'pending'` is ~10 MB instead of 2 GB and the planner uses it whenever the WHERE clause includes `status = 'pending'`. This is the classic state-machine indexing pattern. 4. **Query (c) — `by order_id`:** Point lookup on a high-cardinality column. Single-column B-tree index is the obvious answer. Also doubles as the implicit FK-side index if `order_id` references `orders.id`. 5. **Query (d) — `SUM aggregation by merchant + date range`:** This is the trickiest. Composite `(merchant_id, created_at)` serves filter + range. Adding `INCLUDE (amount)` makes it a covering index, so Postgres can do an index-only scan — no heap fetches for the SUM. Big win because aggregation queries scan many rows. 6. **Cardinality analysis I did mentally:** - `user_id`: high cardinality, used in (a) - `status`: low cardinality, but skewed — perfect for partial - `order_id`: very high cardinality, point lookup - `merchant_id`: medium-high cardinality, used with date range for analytics - `created_at`: monotonic; serves ordering and range 7. **Write-amplification math:** 5% writes = 500k writes/day on 10M rows. Each write updates 4 indexes (plus the heap). At ~30 µs per index update, that's ~60 ms of write cost per second of throughput — well within capacity for any modern Postgres

instance. If writes were 50%, I'd merge (a) and (d) into a single broader composite and drop one index. 8.

What I deliberately didn't add: an index on `currency` (rarely filtered), an index on `status` alone (covered by the partial for the only common status query), an index on `amount` (only used in aggregations, never in WHERE).

Edge cases to mention: - For query (a), I used `created_at DESC` in the index definition — modern Postgres supports it and the planner can use either direction, but explicit DESC avoids any chance of a sort step on older versions. - The partial index on 'pending' becomes less useful as pending grows — for a long-tail backlog, switch to a regular composite. - If query (d) sometimes filters by other status values too, the covering index needs to include `status` in the key as well — design for the actual query shape. - Always create new indexes with `CREATE INDEX CONCURRENTLY` on production tables — covered in detail in Q18.

What NOT to say: - Do not say "I'd just index every column" — write cost would balloon. - Do not propose indexes without naming the query they serve — interviewers want the mapping, not a list. - Do not skip the write-cost calculation — that's how you prove you've thought about it as a system, not just a read optimisation.

Common follow-up: "What if the write ratio changed to 50/50?" — I'd reduce to two indexes: the composite `(user_id, created_at DESC)` (most frequent query) and the partial pending index (cheap to maintain because most rows skip it). For (c) order_id lookups I'd accept a seq scan or denormalise the lookup into the application cache. Query (d) would move to an OLAP replica or a materialised view refreshed hourly.

```
-- (a) User's recent payments – composite with sort order
CREATE INDEX idx_payments_user_created
ON payments(user_id, created_at DESC);

-- (b) Stale pending payments – tiny partial index
CREATE INDEX idx_payments_pending_old
ON payments(created_at) WHERE status = 'pending';

-- (c) Order lookup – single column point access
CREATE INDEX idx_payments_order_id ON payments(order_id);

-- (d) Merchant aggregation – covering index for index-only scan
CREATE INDEX idx_payments_merchant_created
ON payments(merchant_id, created_at) INCLUDE (amount);
```

Q143 · Index-Only Scan — When the Heap Becomes Optional

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay · Topic: Query Execution

The question (interviewer's verbatim phrasing):

What is an index-only scan, and what does it take for the planner to actually do one?

The 30-second answer: An index-only scan returns query results entirely from the index leaf pages without ever reading the heap — possible only when the index contains every column the query needs. In Postgres it additionally requires the visibility map to confirm all matching rows in the relevant heap pages are visible to the snapshot; if not, the engine falls back to fetching the heap for those rows.

Step-by-step thought process: 1. Define the win — heap fetches are the most expensive part of an index lookup; eliminating them is often the difference between 0.1 ms and 10 ms. 2. Two requirements: the index must cover (key columns + INCLUDE columns must include every SELECT and WHERE column), and the engine must know the row is visible without reading it. 3. Postgres specifically — MVCC stores visibility in the heap row, not the index entry; the visibility map is a bitmap per heap page that marks "all rows here are visible to everyone." VACUUM updates it. 4. In InnoDB and SQL Server, index-only scans don't need a visibility map because the clustered index has visibility built in for the PK index, and secondary indexes track it differently — index-only is easier to achieve.

Edge cases to mention: - A high-write Postgres table will show many "Heap Fetches" in EXPLAIN even with a covering index — schedule more frequent VACUUM or use autovacuum tuning. - A wide INCLUDE on a heavily-updated row bloats the index for marginal gain. - ORDER BY clauses don't break index-only scans as long as the order matches the index order.

What NOT to say: - Do not say "covering index = index-only scan" — covering is a prerequisite, not a guarantee in Postgres. - Do not ignore the maintenance cost of frequent VACUUMs needed to keep index-only scans hot.

Common follow-up: "What does *Heap Fetches: 0* actually mean in EXPLAIN ANALYZE?" — Every candidate row was confirmed visible via the visibility map alone — no heap I/O happened. Higher numbers mean partial fallback.

```
CREATE INDEX idx_payments_user_amount
ON payments(user_id) INCLUDE (amount, currency);

-- Postgres can satisfy entirely from the index
EXPLAIN ANALYZE
SELECT amount, currency FROM payments WHERE user_id = 42;
-- Look for: "Index Only Scan" and "Heap Fetches: 0"
```

Q144 · Adding Indexes Online — Without Locking the Table

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Flipkart, PhonePe · Topic: Online DDL

The question (interviewer's verbatim phrasing):

Your production payments table has 500 million rows and you need to add an index. A plain `CREATE INDEX` would lock the table for 20 minutes. What do you do?

The 30-second answer: Use `CREATE INDEX CONCURRENTLY` in Postgres — it takes only a brief lock at the start and end, builds the index in the background while reads and writes continue, and takes longer overall but doesn't block traffic. In MySQL, use `pt-online-schema-change` (Percona Toolkit) or native `ALGORITHM=INPLACE, LOCK=NONE`. In SQL Server, `WITH (ONLINE=ON)` (Enterprise edition).

Step-by-step thought process: 1. Open with the cost of NOT going online — a long lock blocks writes; the app's writes pile up; you cascade-fail the connection pool. 2. Walk through `CREATE INDEX CONCURRENTLY` mechanics: scans the table once, builds the index, then scans again to pick up any rows changed during the first pass — that's why it's slower wall-clock but lock-free for writes. 3. Failure mode for Postgres concurrent index — if the build fails, you're left with an INVALID index that won't be used by the planner; you must `DROP INDEX` and retry. 4. MySQL with `pt-osc` — creates a shadow table, copies rows in batches, applies a trigger to capture ongoing changes, then atomically renames. Different model, same outcome.

Edge cases to mention: - `CREATE INDEX CONCURRENTLY` cannot be run inside a transaction in Postgres. - For UNIQUE indexes, concurrent build is slower because it must validate uniqueness at the end. - MySQL `INSTANT ADD COLUMN` (since 8.0.12) is for some DDL operations; index creation still uses `INPLACE`.

What NOT to say: - Do not run plain `CREATE INDEX` on a high-traffic table during business hours. - Do not assume `pt-osc` is risk-free — triggers add overhead and unique constraint changes are tricky.

Common follow-up: "How do you safely roll back if the concurrent build leaves an INVALID index?" — `SELECT * FROM pg_index WHERE indisvalid = false` to find them; `DROP INDEX CONCURRENTLY` to remove without locking.

```
-- Postgres - non-blocking indexbuild
CREATE INDEX CONCURRENTLY idx_payments_user_created
ON payments(user_id, created_at DESC);

-- Check for INVALID indexes after a failed build
SELECT n.nspname, c.relname FROM pg_index i
JOIN pg_class c ON c.oid = i.indexrelid
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE NOT i.indisvalid;
```

Q145 · Index Bloat — When and How to REINDEX

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Wells Fargo, Razorpay · Topic: Index Maintenance

The question (interviewer's verbatim phrasing):

An index that used to be 2 GB is now 8 GB even though the table size hasn't grown. What's happening, and how do you fix it?

The 30-second answer: That's index bloat — over time, deleted and updated rows leave dead entries in the index pages that VACUUM marks reusable but doesn't physically reclaim. New entries can't always fit into the gaps because of B-tree ordering, so the index grows on disk. Fix it with `REINDEX INDEX CONCURRENTLY` (Postgres 12+) or `pt-osc` in MySQL — both rebuild the index without long locks.

Step-by-step thought process: 1. Distinguish "table bloat" (heap) from "index bloat" — they have different causes and fixes. 2. Explain the mechanism: when a row's indexed column is updated, the old index entry is marked dead and a new one is inserted; VACUUM marks the dead one reusable but cannot compact the B-tree structure. 3. Detection: Postgres `pgstattuple` extension reports bloat percentage; the `pg_stat_user_indexes` view shows scan counts vs size. 4. Fix: `REINDEX CONCURRENTLY` rebuilds in the background. For MySQL InnoDB, `OPTIMIZE TABLE` rebuilds the clustered + all secondary indexes — heavy but effective.

Edge cases to mention: - HOT updates (Heap-Only Tuples) in Postgres avoid index bloat when no indexed column changes — design schemas to favour HOT. - Long-running transactions prevent VACUUM from cleaning up — kill that idle-in-transaction connection. - BRIN indexes don't really "bloat" the same way because their entries are summaries, not per-row.

What NOT to say: - Do not run blocking `REINDEX` (without `CONCURRENTLY`) on a production table — it locks reads and writes. - Do not assume autovacuum handles bloat completely — it reclaims space within pages, not the overall index size.

Common follow-up: "What's the difference between `VACUUM` and `VACUUM FULL`?" — `VACUUM` reclaims space within existing pages (non-blocking); `VACUUM FULL` rewrites the entire table to a new file and reclaims to OS (blocks everything for the duration).

```
-- Postgres 12+ - online reindex
REINDEX INDEX CONCURRENTLY idx_payments_user_created;

-- Or all indexes of a table
REINDEX TABLE CONCURRENTLY payments;

-- MySQL InnoDB
ALTER TABLE payments ENGINE=InnoDB; -- same as OPTIMIZE TABLE
```

Q146 · Invisible Indexes — Test-Dropping Without Risk

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, AmEx, JP Morgan · Topic: Index Management

The question (interviewer's verbatim phrasing):

You suspect an index isn't being used and want to remove it, but you're nervous a rare query depends on it. What's the safe way to test?

The 30-second answer: Mark it invisible — MySQL 8 has `ALTER TABLE ... ALTER INDEX ... INVISIBLE` and Oracle has `INVISIBLE` indexes. The index is still maintained on writes but the planner pretends it doesn't exist, so any query that gets slow points to a dependency. After a confidence period, drop it. Postgres has no native invisible index but achieves the equivalent with `UPDATE pg_index SET indisvalid = false`.

Step-by-step thought process: 1. Frame the risk — dropping an index and immediately finding a quarterly report grinds to a halt is a classic outage. 2. Show the MySQL syntax and the rollback step — `INVISIBLE` to test, `VISIBLE` to instantly restore. 3. Mention monitoring during the test period — slow query log, `pg_stat_statements` equivalent, application timing dashboards. 4. Postgres workaround — toggling `indisvalid` is hackier and not officially supported; an extension or scheduled drop-and-restore can also work.

Edge cases to mention: - Invisible indexes still cost write amplification — you're testing read impact only. - An index marked invisible is visible to integrity constraints — uniqueness still enforced. - Some optimisers use index statistics even when the index is invisible; behaviour varies by version.

What NOT to say: - Do not propose dropping the index and "watching for issues" — by the time you find out, the report has failed. - Do not skip telling the team — invisible indexes confuse the next person to do an audit.

Common follow-up: "How do you know which indexes are unused in the first place?" — Postgres `pg_stat_user_indexes.idx_scan` count; MySQL `sys.schema_unused_indexes` view; SQL Server `sys.dm_db_index_usage_stats`.

```
-- MySQL 8 - invisible test
ALTER TABLE payments ALTER INDEX idx_payments_status INVISIBLE;
-- Wait, monitor, confirm no queries got slow
ALTER TABLE payments DROP INDEX idx_payments_status;

-- Rollback if needed
ALTER TABLE payments ALTER INDEX idx_payments_status VISIBLE;
```

Q147 · Descending Indexes and ORDER BY Direction

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Razorpay, PhonePe · Topic: Index Sort Order

The question (interviewer's verbatim phrasing):

Does `ORDER BY created_at DESC LIMIT 20` need a separate index, or can it use an ASC index?

The 30-second answer: Modern Postgres can walk a B-tree in reverse, so an ASC index serves both `ORDER BY col ASC` and `ORDER BY col DESC` for single-column sorts. But for multi-column sorts like `ORDER BY a ASC, b DESC`, you need an index whose declaration matches that exact order — `CREATE INDEX ON t(a ASC, b DESC)`. MySQL didn't support DESC in index definitions before 8.0; older versions silently stored ASC and used a sort step.

Step-by-step thought process: 1. Explain the leaf-walking mechanism — B-trees have sibling pointers in both directions in modern engines. 2. Show the multi-column trap: `ORDER BY status ASC, created_at DESC` cannot be served by `(status, created_at)` (both ASC) in a single index scan; the planner must sort. 3. Postgres has always supported DESC in index definitions; MySQL added it in 8.0 (before that, the syntax was accepted but ignored). 4. Performance impact — a missing index direction triggers a sort node, which is fine for small result sets and painful at scale.

Edge cases to mention: - NULLS FIRST vs NULLS LAST matters too — Postgres lets you specify in both index and query; mismatches force a sort. - Composite indexes with mixed directions don't compose well — designing one per query is sometimes inevitable. - In MySQL 8, `EXPLAIN` shows whether the descending index is being used in the "key" output.

What NOT to say: - Do not assume ASC and DESC are interchangeable for composite indexes — that's the source of many surprise sort steps. - Do not skip the NULLS positioning when designing indexes for nullable sorted columns.

Common follow-up: "How do I check if a sort step is happening?" — Postgres `EXPLAIN ANALYZE` shows a `Sort` node above the `Index Scan` if so; MySQL shows `Using filesort` in EXPLAIN.

```
-- Postgres — supports DESC natively, ASC walks backward
CREATE INDEX idx_orders_created_desc ON orders(created_at DESC);

-- Multi-column with mixed direction
CREATE INDEX idx_orders_status_created
ON orders(status ASC, created_at DESC);

SELECT * FROM orders
WHERE status = 'paid'
ORDER BY created_at DESC LIMIT 20;
```

Q148 · Sorted Indexes for ORDER BY — Eliminating the Sort Step

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Sort Optimisation

The question (interviewer's verbatim phrasing):

How can an index eliminate an ORDER BY's sort step, and when can it not?

The 30-second answer: A B-tree stores data in sorted order, so if your **ORDER BY** columns are the leading columns of an index (or follow equality-filtered prefix columns), the planner can return rows in index order without an explicit sort. It cannot eliminate the sort when the ORDER BY columns aren't an index prefix, when the directions don't match, or when the query mixes columns from multiple indexes.

Step-by-step thought process: 1. Restate the mechanism — the index's leaf-level traversal IS a sorted scan; the planner just streams it out. 2. Equality-filter-then-sort case: index **(user_id, created_at)** serves **WHERE user_id = 5 ORDER BY created_at** with zero sort cost. 3. Range-filter-then-sort breaks this: **WHERE created_at > '...' ORDER BY user_id** cannot use the same index for both; either you sort or you create a different index. 4. ORDER BY + LIMIT is where the index win is most dramatic — without an index, you sort millions of rows just to throw away all but 20.

Edge cases to mention: - Adding **INCLUDE** columns doesn't break the sort property — only key columns determine order. - For multi-table queries, the optimiser sometimes does a sort even if each table contributes ordered rows — merging streams isn't always trivial. - A **Memory: ...** line in Postgres EXPLAIN signals an in-memory sort; if you see disk spill, the cost is far higher.

What NOT to say: - Do not assume every ORDER BY needs an index — small result sets sort just fine in memory. - Do not over-design composite indexes purely for sort elimination if the queries are rare.

Common follow-up: "How does ORDER BY + LIMIT 1 work with an index?" — The planner walks the leaf, returns the first matching row, and stops — **$O(\log n)$** for the seek plus one row. That's the canonical "top-1 per group" pattern.

```
-- Index supports filter + sort with no sort step
CREATE INDEX idx_payments_user_created ON payments(user_id, created_at);

EXPLAIN ANALYZE
SELECT * FROM payments
WHERE user_id = 42
ORDER BY created_at DESC LIMIT 20;
-- Look for: no "Sort" node, just Index Scan + Limit
```


Q149 · Filtered Index in SQL Server — Postgres Partial Equivalent

Tags: Difficulty: Medium · Asked at: AmEx, Goldman, Wells Fargo, JP Morgan · Topic: SQL Server Indexes

The question (interviewer's verbatim phrasing):

What's a filtered index in SQL Server, and how is it different from a partial index in Postgres?

The 30-second answer: Filtered indexes in SQL Server and partial indexes in Postgres are functionally the same — you attach a WHERE clause to the index definition and only matching rows get indexed. The syntax differs slightly (**WHERE** clause in both, but SQL Server has stricter predicate restrictions — no LIKE, no BETWEEN, simple comparisons only). The performance and use cases — soft-deletes, state machines, sparse columns — translate directly.

Step-by-step thought process: 1. Frame as "same feature, two vendors" — interviewer wants to see cross-engine fluency. 2. Show both syntaxes side by side. 3. Highlight SQL Server's stricter predicate restrictions — only simple comparison operators are allowed in the filter, no functions, no LIKE. 4. Highlight a SQL Server-specific gotcha: the connection's SET options (ANSI_NULLS, QUOTED_IDENTIFIER, etc.) must match the index's compiled options or the planner skips the index — common source of "why isn't my filtered index used" tickets.

Edge cases to mention: - Both engines require the query's WHERE clause to imply the index's filter for the index to be used. - SQL Server's filtered index can have INCLUDE columns just like a regular index — combine for covering filtered indexes. - MySQL has no native equivalent; you simulate with a generated column + regular index, or live with the full index.

What NOT to say: - Do not confuse filtered indexes with sparse columns in SQL Server — sparse columns are a storage optimisation, unrelated to indexing. - Do not assume the index is "always cheaper to maintain" — it only saves cost on writes that don't match the filter.

Common follow-up: "What SET options can break a filtered index?" — **SET ANSI_NULLS OFF** (default OFF for ODBC connections sometimes) silently invalidates filtered indexes — keep ANSI_NULLS ON consistently.

```
-- SQL Server filtered index
CREATE NONCLUSTERED INDEX idx_orders_pending
ON orders(created_at)
WHERE status = 'pending';

-- Postgres partial index — same idea
CREATE INDEX idx_orders_pending
ON orders(created_at) WHERE status = 'pending';
```


Q150 · LIKE 'prefix%' vs LIKE '%suffix' — and Trigram Indexes

Tags: Difficulty: Hard · Asked at: Swiggy, Cred, Razorpay, Flipkart · Topic: Text Search Indexes

The question (interviewer's verbatim phrasing):

Why does an index help `WHERE name LIKE 'Rohan%'` but not `WHERE name LIKE '%kumar'`, and how do you make the second case fast?

The 30-second answer: A B-tree is sorted left-to-right, so `LIKE 'Rohan%'` can be served as a range scan (`name >= 'Rohan' AND name < 'Rohao'`). A leading wildcard `LIKE '%kumar'` has no fixed prefix to seek — every row could match — so the index is useless. The fix is `pg_trgm` in Postgres: a trigram GIN/GiST index breaks every string into three-character chunks and indexes those, so substring matches become tractable.

Step-by-step thought process: 1. Explain the alphabetical-ordering reason for B-tree prefix wins. 2. Trigram concept — "rohan" becomes {" r", " ro", "roh", "oha", "han", "an "} indexed individually; a search for "%kumar%" looks up trigrams of "kumar" and intersects. 3. Show the index syntax and the matching operator — `%` for similarity, `LIKE` and `ILIKE` both supported via the right operator class. 4. MySQL alternative — full-text indexes via MyISAM or InnoDB FTS for word-boundary search; no native trigram support without third-party plugins.

Edge cases to mention: - Trigram indexes are large — three-character keys per character of input data; budget disk accordingly. - For case-insensitive trigram search, use `gin_trgm_ops` with `ILIKE` — Postgres normalises case for the comparison. - A leading wildcard on a B-tree expression index of `REVERSE(name)` turns `'%kumar'` into `'rakum%'` after reversing — a classic trick when trigram isn't available.

What NOT to say: - Do not say "LIKE always uses an index if it exists" — leading wildcards bust that completely. - Do not propose full-text search for prefix matching — overkill; B-tree is the right tool.

Common follow-up: "Can trigram speed up fuzzy match for typos like 'Rohaana' vs 'Rohan'?" — Yes — `pg_trgm` exposes a similarity operator `%` and a distance operator `<->`; you can `ORDER BY similarity LIMIT 5` for fuzzy autocomplete.

```
-- B-tree handles prefix
CREATE INDEX idx_users_name ON users(name);
SELECT * FROM users WHERE name LIKE 'Rohan%';

-- Trigram handles substring
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_users_name_trgm ON users USING GIN (name gin_trgm_ops);
SELECT * FROM users WHERE name ILIKE '%kumar%';
```

Q151 · Indexing for ORDER BY + LIMIT — The Pagination Killer

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Cred · Topic: Pagination

The question (interviewer's verbatim phrasing):

Your dashboard query is `SELECT * FROM orders WHERE user_id = ? ORDER BY created_at DESC LIMIT 50 OFFSET 1000`. Even with an index, it gets slow for power users. What's happening?

The 30-second answer: The index handles the WHERE and ORDER BY efficiently, but `OFFSET 1000` forces the engine to fetch and discard the first 1000 rows before returning the next 50 — so cost grows linearly with offset, not with limit. Fix it with keyset pagination: instead of OFFSET, pass the last seen `created_at` value and add `WHERE created_at < ?` to skip rows via the index seek itself, in constant time.

Step-by-step thought process: 1. Diagnose the OFFSET cost — the planner can't skip rows via the index seek; it must materialise them. 2. Demonstrate keyset pagination: `WHERE user_id = ? AND (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 50`. The composite comparison handles tie-breaking when timestamps repeat. 3. Index requirement: a composite on `(user_id, created_at, id)` makes the seek $O(\log n)$ regardless of how deep in the result set you are. 4. UX trade-off — keyset gives "next page" cleanly; jumping to page 50 directly is impossible. Most modern apps accept this for infinite-scroll patterns.

Edge cases to mention: - If `created_at` isn't unique, you must include a tiebreaker (the PK is the natural choice) — otherwise paging boundaries miss or repeat rows. - For total count display, you need a separate `COUNT(*)` query — keyset doesn't give you "page X of Y" cheaply. - MySQL syntax for tuple comparison works `((a, b) < (?, ?))`; some engines need explicit AND expansion.

What NOT to say: - Do not recommend OFFSET pagination for deep pages — it does not scale, period. - Do not skip the tiebreaker column; it's the difference between correct and silently-wrong.

Common follow-up: "How do you build a 'previous page' button with keyset?" — Reverse the sort and invert the comparison; or keep a stack of seen cursors on the client. Most teams just disable "previous" or fall back to OFFSET only for shallow paging.

```
-- Slow: deep OFFSET
SELECT * FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 50 OFFSET 1000;

-- Fast: keyset pagination
SELECT * FROM orders
WHERE user_id = 42
  AND (created_at, id) < ('2026-05-25 14:00:00', 998877)
ORDER BY created_at DESC, id DESC
LIMIT 50;
```

Q152 · Indexing for Filtering + Sorting in the Same Query

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, Razorpay · Topic: Composite Index Design

The question (interviewer's verbatim phrasing):

You have `WHERE status = 'paid' AND amount > 1000 ORDER BY created_at DESC LIMIT 100`. What's the optimal index?

The 30-second answer: For a query that mixes equality, range, and sort, the optimal composite index order is **equality columns first, then sort column, then range column** — so `(status, created_at DESC, amount)`. This lets the index seek to `status = 'paid'`, walk in `created_at DESC` order for free, and apply the `amount > 1000` filter as a residual check. Putting `amount` (range) before `created_at` (sort) would force a sort step.

Step-by-step thought process: 1. State the canonical rule: **Equality, Sort, Range (ESR)** — equality columns first because they're a clean prefix; sort column next so the index walk gives ordered output; range column last because it breaks the leaf-walk's usefulness for further columns. 2. Walk through the alternative `(status, amount, created_at)`: the planner seeks to `status = 'paid' AND amount > 1000`, but the rows within that range are ordered by `amount`, not `created_at` — so it must sort. 3. Mention the trade-off — putting a range column last means the index can't filter on it as efficiently; rows have to be loaded and checked. Acceptable if the equality + sort already narrows enough. 4. If `amount > 1000` matches only 1% of paid orders, consider a partial index `WHERE status = 'paid' AND amount > 1000` ordered by `created_at` — best of both worlds.

Edge cases to mention: - If the LIMIT is small, the engine streams a few rows and stops — even a suboptimal index plan can be fast in practice. - ESR is a heuristic, not a law — verify with EXPLAIN ANALYZE; the planner may surprise you based on statistics. - For multi-table joins, the ESR rule applies per table, and join order interactions can complicate the picture.

What NOT to say: - Do not pick column order alphabetically or by "importance" — pick by the query's predicate shape. - Do not skip the EXPLAIN verification — index choice is a hypothesis; the plan tells you if you were right.

Common follow-up: "What if I also need `ORDER BY created_at ASC` for the same data?" —

Postgres can walk a single-column index backward, but for the composite

(`status ASC, created_at DESC`), a query asking

`ORDER BY status ASC, created_at ASC` won't get index ordering; you'd add a second index or accept the sort.

```
-- Optimal: Equality, Sort, Range
CREATE INDEX idx_orders_status_created_amount
ON orders(status, created_at DESC, amount);

-- Bad ordering: forces a sort
-- CREATE INDEX idx_orders_status_amount_created
-- ON orders(status, amount, created_at DESC);

EXPLAIN ANALYZE
SELECT * FROM orders
WHERE status = 'paid' AND amount > 1000
ORDER BY created_at DESC LIMIT 100;
```

Q153 · When NOT to Add an Index

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, JP Morgan · Topic: Index Trade-offs

The question (interviewer's verbatim phrasing):

Give me three cases where adding an index is the wrong move.

The 30-second answer: First, very small tables — under a few thousand rows, a seq scan beats index seek + heap fetch. Second, write-heavy tables where the column is rarely filtered — write amplification will dwarf the rare read benefit. Third, low-cardinality columns where queries match a large fraction of rows — the planner will skip the index anyway, and you've paid the maintenance cost for nothing.

Step-by-step thought process: 1. Open with the principle — every index costs write performance, disk space, and planner time; the benefit must justify the cost. 2. Small tables: under 1000-5000 rows, a sequential scan is one disk page read — beats any index lookup. 3. Write-heavy + rarely filtered: a logging table writes 100k rows/min and is queried once a week by an analyst — better to do a sweep scan once a week. 4. Low cardinality + non-skewed: a boolean `is_active` where 50% are true — neither value is selective enough; the index never gets used. 5. Bonus: do not duplicate indexes. `(a)` and `(a, b)` covers `(a)` already; an extra single-column index is wasted.

Edge cases to mention: - "Rarely filtered" must include ORDER BY and JOIN usage too — a column you sort or join on is being filtered effectively. - Auto-generated columns from ORMs sometimes get unnecessary indexes — audit periodically. - Small tables today can become huge tomorrow — review index decisions when row counts cross thresholds.

What NOT to say: - Do not say "indexes are free, just add them" — write amplification is real and compounds across many indexes. - Do not blindly remove unused indexes without checking if they back a UNIQUE constraint or FK.

Common follow-up: "How do I find duplicate or redundant indexes?" — Postgres has the `pg_dba` toolkit; `pt-duplicate-key-checker` for MySQL; SQL Server has `sys.dm_db_index_usage_stats`.

```
-- Detect unused indexes (Postgres)
SELECT schemaname, relname, indexrelname, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Q154 · Cardinality and Selectivity — What Makes a "Good" Index

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Razorpay · Topic: Index Theory

The question (interviewer's verbatim phrasing):

Explain cardinality and selectivity. What ratio makes a column worth indexing?

The 30-second answer: Cardinality is the number of distinct values in a column; selectivity is what fraction of rows a typical query matches. A column with high cardinality (many distinct values) and high selectivity per query (matching <5% of rows) is the textbook ideal — those are the indexes the planner picks every time. Low cardinality + low selectivity (each value matches a big chunk) is where indexes get skipped.

Step-by-step thought process: 1. Define both terms precisely — cardinality is column-level, selectivity is query-level. 2. Give the rough threshold: queries matching under 5-10% of rows benefit from an index; above that, sequential scan often wins. 3. Show the math intuition — random heap I/O is roughly 30-100x slower than sequential reads, so the index has to save you that ratio's worth of work to pay off. 4. Selectivity changes per query — the same column can be selective for one filter (`status = 'pending'` , 0.1%) and useless for another (`status = 'paid'` , 80%) — partial indexes solve exactly this.

Edge cases to mention: - The planner uses statistics (Postgres `pg_statistic`, MySQL `information_schema.statistics`) to estimate selectivity — stale stats mean wrong choices; run ANALYZE. - Multi-column statistics (`CREATE STATISTICS` in Postgres 10+) help when columns are correlated. - A column's cardinality is fixed; its query selectivity depends on filter values, distribution, and skew.

What NOT to say: - Do not conflate cardinality and selectivity — they're related but distinct. - Do not claim "high cardinality always means good index" — selectivity per actual query is what matters.

Common follow-up: "What if my planner picks the wrong index?" — Update statistics first (`ANALYZE`); if still wrong, check for missing extended statistics on correlated columns; last resort is an optimiser hint or query rewrite.

```
-- Check cardinality
SELECT COUNT(DISTINCT status), COUNT(*) FROM payments;

-- Postgres extended stats for correlated columns
CREATE STATISTICS stats_payments_status_currency
ON status, currency FROM payments;
ANALYZE payments;
```

Q155 · How Many Indexes per Table — The Diminishing Returns Curve

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Walmart Labs, AmEx · Topic: Index Strategy

The question (interviewer's verbatim phrasing):

Is there a rule of thumb for the maximum number of indexes per table?

The 30-second answer: No hard rule, but the practical sweet spot for OLTP tables is roughly 4-7 indexes — enough to cover the hot read patterns without crippling write throughput. Each additional index adds INSERT/UPDATE/DELETE cost linearly, and the planner's join-order search gets slower with more options to consider. OLAP tables can tolerate more indexes because writes are batched.

Step-by-step thought score: 1. Open with the math — every index is one extra B-tree write per row insert. 2. Walk through write cost: a table with 10 indexes incurs roughly 10x the index write work of a table with one, plus the heap write. 3. Note the planner cost — more indexes = more candidate plans = more planning time per query. Usually negligible, but for very high QPS systems it adds up. 4. Practical advice: audit indexes quarterly; drop ones with zero scans; consolidate composite indexes that overlap.

Edge cases to mention: - Write-amplification depends on which columns change — UPDATES that touch only non-indexed columns are cheap (HOT updates in Postgres avoid all index work). - Partitioned tables effectively have an index per partition for each table-level index — the count multiplies. - Hot-path

tables in a payment system should be tightly indexed; cold-path event logs can run on BRIN + partition + nothing else.

What NOT to say: - Do not quote a specific number as gospel — "max 5 indexes" is a heuristic, not a rule. - Do not skip the cost of UPDATES in your calculus — they're the dominant write workload for many tables.

Common follow-up: "What's the cost difference between adding a 5th index and a 15th?" — On INSERT, roughly linear. On UPDATE, only indexes whose key columns changed pay the full cost; HOT updates skip even those. So the practical curve depends on column volatility, not just count.

```
-- Audit query (Postgres) – find indexes by usage
SELECT relname, indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
ORDER BY idx_scan ASC, pg_relation_size(indexrelid) DESC;
```

Q156 · Index Selectivity in Practice — Designing for Real Skew

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, JP Morgan, Walmart Labs · Topic: Index Design

The question (interviewer's verbatim phrasing):

You have a `transactions` table where `merchant_id` distribution is heavily skewed — one merchant has 40% of all rows, while 10,000 others have <0.01% each. How does this affect index design?

The 30-second answer: A single B-tree index on `merchant_id` works perfectly for the small merchants (highly selective) and is correctly skipped by the planner for the big one (low selectivity). The asymmetric handling is exactly what you want. For analytic queries on the big merchant, design a separate path: composite index `(merchant_id, created_at)` for time-range slicing, or partition the table by merchant if it gets large enough.

Step-by-step thought process: 1. Recognise skew is normal — payment processors, e-commerce platforms, and ride-hailing all have power-law merchant distributions. 2. Per-query selectivity reasoning: the planner uses histograms in `pg_stats` to estimate row counts per value — for the big merchant, it expects 40% of the table and skips the index; for a small merchant, it expects 100 rows and uses the index seek. 3. For the big merchant's queries, add the time dimension to slice: `(merchant_id, created_at)` narrows by date range even within that one giant merchant. 4. If one merchant

dominates by an extreme margin (90%+), consider list partitioning by merchant — keeps the big one's data isolated and small merchants' queries blazing fast.

Edge cases to mention: - Histogram bucket counts in Postgres default to 100 — for very skewed data, increase `default_statistics_target` for that column. - A query for the big merchant + a narrow date range may still beat seq scan — index is per-row, not per-merchant. - Skew can change over time as new merchants grow — review every 6 months.

What NOT to say: - Do not panic about skew — it's the planner's job to handle it, and modern planners do this well. - Do not over-partition prematurely — partitioning has operational overhead; only adopt when scale justifies.

Common follow-up: *"How do you check if Postgres has good statistics for a skewed column?"* —

`SELECT most_common_vals, most_common_freqs FROM pg_stats WHERE tablename='transactions' AND attname='merchant_id'` — those show the high-frequency values the planner is aware of.

```
-- Composite for big merchants with time-range slicing
CREATE INDEX idx_transactions_merchant_created
ON transactions(merchant_id, created_at);

-- Increase histogram resolution for skewed column
ALTER TABLE transactions ALTER COLUMN merchant_id
SET STATISTICS 500;
ANALYZE transactions;

-- For extreme skew, consider list partitioning
CREATE TABLE transactions_big_merchant
PARTITION OF transactions FOR VALUES IN ('M0001');
```

Chapter 4, Part B — Query Plans & Optimization

Picking the right indexes (Part A) gets you halfway. The other half is being able to open a query plan, point at the line that is killing you, and rewrite the SQL so the optimizer makes a better decision. This section covers what EXPLAIN and EXPLAIN ANALYZE actually show, how to read a plan top-down versus bottom-up, when the planner picks nested-loop versus hash versus merge joins, why statistics drift and break plans, and the rewrites you will reach for again and again — subquery to JOIN, NOT IN to NOT EXISTS, OR to UNION ALL, function-on-column to expression index. Each question is sized for a 60-to-90 second answer in an interview, with the edge case the panel is waiting to hear.

Q157 · EXPLAIN vs EXPLAIN ANALYZE

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, JP Morgan India, Walmart Labs · Topic: Query Plans

The question (interviewer's verbatim phrasing):

What is the difference between EXPLAIN and EXPLAIN ANALYZE? When do you use each one in production?

The 30-second answer: EXPLAIN shows the planner's estimated plan without running the query — it is fast and safe. EXPLAIN ANALYZE actually executes the query and shows real row counts, actual timing per node and loop counts, so you can compare estimates against reality. You use EXPLAIN to sanity-check; you use EXPLAIN ANALYZE to debug a slow query.

Step-by-step thought process: 1. Open with the core distinction — estimation versus execution. EXPLAIN is what the optimizer thinks will happen; EXPLAIN ANALYZE is what actually happened. 2. Mention the cost: EXPLAIN ANALYZE runs the query end-to-end, so on a slow query that scans a billion rows it will be slow and may touch production data. For UPDATE or DELETE, wrap it in BEGIN and ROLLBACK to avoid persisting changes. 3. Highlight the most useful columns: estimated rows versus actual rows. If the estimate is 10 and the actual is 10 million, the planner has stale statistics or a correlation it cannot see, and that is your first lead. 4. Vendor call-out: Postgres has EXPLAIN ANALYZE; MySQL 8 has EXPLAIN ANALYZE too; SQL Server uses SET STATISTICS PROFILE ON or the actual execution plan in SSMS; Oracle uses DBMS_XPLAN.DISPLAY_CURSOR after running with /+ GATHER_PLAN_STATISTICS /.

Edge cases to mention: - EXPLAIN ANALYZE on a destructive statement (DELETE, UPDATE) actually performs it — always wrap in a transaction you can roll back. - The timing overhead of EXPLAIN ANALYZE itself can inflate per-row timing on very fast nodes (microsecond hot loops). - ANALYZE in Postgres EXPLAIN ANALYZE is unrelated to the standalone ANALYZE command that refreshes statistics — same word, different operation.

What NOT to say: - Do not say "EXPLAIN runs the query" — it does not in any major engine. - Do not skip EXPLAIN ANALYZE because "the EXPLAIN looked fine" — estimated plans lie when stats are stale.

Common follow-up: *"What extra information does EXPLAIN (ANALYZE, BUFFERS) give you?"* — Buffer hits, reads from disk, and dirty buffers per node, which lets you distinguish a CPU-bound plan from an IO-bound one.

```
EXPLAIN ANALYZE
SELECT order_id, total_amount
FROM orders
WHERE customer_id = 4827 AND order_date >= CURRENT_DATE - INTERVAL '30 days';
```

Q158 · Reading a Plan — Top-Down or Bottom-Up

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Query Plans

The question (interviewer's verbatim phrasing):

When you open a query plan in Postgres or SQL Server, do you read it top-down or bottom-up?
Walk me through how you actually scan it.

The 30-second answer: Execution flows bottom-up — the deepest, most indented nodes run first and feed their parents — but you read top-down to understand the shape of the query, then dive into the leaves to find the expensive step. The slowest single node is usually where you spend your time.

Step-by-step thought process: 1. Clarify the two directions. Reading top-down tells you what the query is trying to do; reading bottom-up traces actual execution order, because each operator pulls rows from its children. 2. The first thing I scan is the rows column — both estimated and actual — looking for the largest blow-up. A node that returns 50 million rows when its parent only needs 100 is the leak. 3. Next, I look at the operator types — Seq Scan on a large table when there is a usable index is a red flag; Nested Loop with high loop count is another. 4. Finally, I check the cost contribution of each subtree to confirm where the time is actually going. In Postgres EXPLAIN ANALYZE this is "actual time" in milliseconds per loop.

Edge cases to mention: - In Postgres text plans, indentation tells you parent-child — nodes at the same indent level are siblings (typically join inputs). - SQL Server graphical plans read right-to-left, top-to-bottom — same execution-order rule, different visual convention. - A Materialize or Memoize node hides repeated work — its parent loop count can be misleadingly high while the child runs only once.

What NOT to say: - Do not say "I read it left-to-right" — that does not map to either rendering. - Do not focus on the topmost cost number — that is the cumulative cost, not the cost of the root operator alone.

Common follow-up: *"In a plan, what is the difference between cost and actual time?"* — Cost is the planner's unitless estimate; actual time is real milliseconds from EXPLAIN ANALYZE. The two can diverge wildly when stats are stale.

```
Limit (cost=0.00..8.27 rows=10 width=64) (actual time=0.04..0.12 rows=10 loops=1)
  -> Index Scan using idx_orders_customer on orders (actual rows=10 loops=1)
        Index Cond: (customer_id = 4827)
```

Q159 · Cost Units in a Plan

Tags: Difficulty: Medium · Asked at: Cred, Zerodha, Goldman, JP Morgan India · Topic: Query Plans

The question (interviewer's verbatim phrasing):

What do the cost numbers in EXPLAIN actually represent? Is a cost of 1000 in milliseconds?

The 30-second answer: Cost is a unitless score the planner uses to compare plans — it is roughly calibrated so 1.0 equals one sequential page fetch on the configured hardware. It is not milliseconds, not rows, not bytes. Two costs are only comparable within the same plan on the same database; cross-database comparisons are meaningless.

Step-by-step thought process: 1. Lead with the unitless-score point. The planner only needs ordinal correctness — plan A cheaper than plan B — and the cost reflects that. 2. In Postgres, the constants `seq_page_cost`, `random_page_cost`, `cpu_tuple_cost` and `cpu_operator_cost` are knobs that scale the cost. The defaults assume spinning disk; SSD shops often lower `random_page_cost` from 4.0 to 1.1. 3. Cost has two parts in Postgres: startup cost (work before the first row is emitted) and total cost (work to emit all rows). For a LIMIT 1 query, startup cost is what matters; for a full export, total cost matters. 4. Vendor call-out: SQL Server uses subtree cost in the same flavour but tuned differently; Oracle uses CPU cost plus IO cost. The interpretation is similar — internal score, not wall-clock time.

Edge cases to mention: - A plan with low cost can still be slow if statistics underestimated row counts. Cost trusts stats blindly. - Adjusting `random_page_cost` without testing changes the planner's preference between index scan and sequential scan across the whole database. - Cost does not account for cache state — a cold buffer pool will make a "cheap" plan crawl.

What NOT to say: - Do not say "lower cost always means faster" — only on average and only when stats are accurate. - Do not quote a cost as milliseconds in a status report — your senior will correct you.

Common follow-up: *"Why does my Postgres plan show cost 12000 but the query runs in 50ms?"* — Probably hot cache, accurate small-row count, or planner overestimation due to default page-cost constants on SSD hardware.

```
Seq Scan on orders (cost=0.00..1934.00 rows=100000 width=64)
-- 0.00 = startup cost, 1934.00 = total cost (unitless)
```

Q160 · Index Scan vs Index Seek vs Sequential Scan

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Wells Fargo · Topic: Query Plans

The question (interviewer's verbatim phrasing):

Explain the difference between index scan, index seek and sequential scan. When does the planner pick each?

The 30-second answer: Index seek (SQL Server term) jumps directly to a specific key range using the B-tree — cheap and bounded. Index scan reads the entire index in order — used when nearly all rows qualify or when ordering is needed. Sequential scan reads the full table without using any index — picked when the table is small, no usable index exists, or the filter matches a large fraction of rows.

Step-by-step thought process: 1. Open with the seek versus scan distinction. A seek is logarithmic on key depth; a scan is linear on the index leaves. 2. Explain when the planner switches to sequential scan: when the estimated fraction of matching rows is high (typically above 10 to 25 percent in Postgres), random IO from index lookups beats sequential IO from a table scan. 3. Postgres terminology: there is no "seek" — there are Index Scan, Index Only Scan, Bitmap Index Scan and Seq Scan. SQL Server distinguishes Index Seek from Index Scan; MySQL EXPLAIN shows type ref, range, index, or ALL. 4. Be ready for the trick question: a Seq Scan is sometimes correct and faster than an Index Scan — especially on tables that fit in memory or queries that need 80 percent of rows.

Edge cases to mention: - A "Full Index Scan" (Postgres Index Scan with no Index Cond) is essentially scanning the index instead of the heap — useful only when the index is much smaller than the table or covers the query. - On small tables (a few hundred rows), the planner correctly ignores indexes — the overhead of B-tree descent costs more than reading the heap. - SQL Server Index Scan with a residual predicate looks like a seek but is actually a scan plus filter — check the Predicate property.

What NOT to say: - Do not say "sequential scan is always bad" — interviewers love correcting this. - Do not confuse Index Only Scan with Index Scan in Postgres; they have different IO profiles.

Common follow-up: *"If my query has a WHERE on an indexed column but the plan shows Seq Scan, what do I check?"* — Selectivity (too many matching rows), implicit cast on the column, function applied to the column, stale stats, or the index is on a different column order.

```
-- Good: bounded lookup
Index Scan using idx_users_email on users  (cost=0.42..8.44 rows=1)
  Index Cond: (email = 'priya@example.in')
```

Q161 · Index-Only Scan — When Achievable

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, AmEx · Topic: Query Plans

The question (interviewer's verbatim phrasing):

What is an index-only scan, and how do you write a query and design an index so the planner can use one?

The 30-second answer: An index-only scan answers a query entirely from the index, never touching the table heap — the index must contain every column the query references (covering), and the visibility map

(Postgres) or row visibility metadata must confirm the index entries are current. You design for it by including the projected columns in the index, either as key columns or as INCLUDE columns.

Step-by-step thought process: 1. Define the term — the planner can satisfy SELECT and WHERE entirely from the B-tree leaves, skipping the heap fetch. 2. Two requirements: the index must cover all referenced columns (in WHERE, SELECT, ORDER BY, GROUP BY), and the engine must know that index entries reflect live row state. 3. Postgres uses a visibility map per heap page — if the page is all-visible, the index-only scan skips it. After a heavy UPDATE, you may see "Heap Fetches" in the plan; running VACUUM updates the visibility map and reduces them. 4. SQL Server calls this a covering index; the INCLUDE clause adds payload columns without bloating the key. Same effect: no Key Lookup needed.

Edge cases to mention: - Index-only scan on a recently updated table can degrade to many heap fetches until VACUUM runs. - In Postgres, only B-tree and a few other AMs support index-only scan; GIN and BRIN typically do not. - Adding INCLUDE columns bloats the index — fine for narrow payloads, costly for wide ones.

What NOT to say: - Do not say "index-only scan never touches disk" — it touches the index pages on disk if they are not cached. - Do not assume MySQL has a separate "index-only scan" operator — InnoDB shows it as Using index in EXPLAIN Extra.

Common follow-up: *"My index covers the columns but the plan still shows heap fetches. Why?"* — The visibility map is not marking pages all-visible yet (recent writes). Run VACUUM or wait for autovacuum.

```
-- Covering index for a top-N dashboard query
CREATE INDEX idx_orders_cust_date_amt
  ON orders (customer_id, order_date DESC) INCLUDE (total_amount);
```

Q162 · Bitmap Index Scan and Bitmap Heap Scan

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, JP Morgan India, Walmart Labs · Topic: Query Plans

The question (interviewer's verbatim phrasing):

In Postgres I see Bitmap Index Scan feeding a Bitmap Heap Scan. What is happening, and why does the planner choose this over a regular Index Scan?

The 30-second answer: The planner builds an in-memory bitmap of matching tuple IDs from one or more indexes, ORs or ANDs the bitmaps together, then visits the heap in physical block order. It is chosen when many rows match (random index-then-heap would thrash IO) and especially when multiple indexes can be combined for a single query.

Step-by-step thought process: 1. Walk through the two-phase shape. Phase one: Bitmap Index Scan collects TIDs into a bitmap. Phase two: Bitmap Heap Scan visits heap pages in physical order, rechecking the predicate if needed. 2. Explain the win: random IO becomes sequential-ish because the heap pages are read in block order — much friendlier to spinning disk and to OS readahead. 3. Multi-index combination: WHERE status = 'PENDING' AND city = 'Mumbai' can use idx_status and idx_city, build two bitmaps, AND them, and visit the heap once. A regular Index Scan can only use one index per scan node. 4. Vendor call-out: this operator pair is distinctive to Postgres. SQL Server has Index Intersection and Index Union in similar spirit; Oracle has bitmap conversion. MySQL InnoDB does not do bitmap heap scans.

Edge cases to mention: - The bitmap can be "lossy" — if it overflows work_mem, it stores page-level granularity instead of tuple-level, and the heap scan must recheck the predicate. - For very few matching rows, a plain Index Scan is cheaper because the bitmap setup overhead is wasted. - For very many matching rows, a Seq Scan beats the bitmap because the heap is read end-to-end anyway.

What NOT to say: - Do not say "bitmap scan is always faster than index scan" — it is a middle-ground operator. - Do not assume the bitmap is persisted — it is rebuilt every query.

Common follow-up: "How do I encourage or discourage a bitmap scan?" — Tune work_mem, effective_io_concurrency, and random_page_cost. SET enable_bitmapscan = off temporarily to compare plans.

```
Bitmap Heap Scan on orders (cost=...) Recheck Cond: status = 'PENDING' AND city = 'Mumbai'
-> BitmapAnd
    -> Bitmap Index Scan on idx_orders_status
    -> Bitmap Index Scan on idx_orders_city
```

Q163 · Nested-Loop Join — When the Planner Picks It

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Cred, Walmart Labs · Topic: Join Algorithms

The question (interviewer's verbatim phrasing):

When does the optimizer pick a nested-loop join, and when is that a bad idea?

The 30-second answer: Nested loop is chosen when the outer side is small (or LIMIT cuts it short) and the inner side has a usable index on the join column — for each outer row, probe the inner index. It is a disaster when the outer side blows up to millions of rows, because that becomes millions of index probes.

Step-by-step thought process: 1. Describe the algorithm: for each row in the outer table, scan or probe the inner table for matches. Total work is outer_rows times inner_lookup_cost. 2. The planner picks it

when `outer_rows` is small — say, under a few thousand — and there is a B-tree index on the inner join column so each probe is logarithmic. 3. It is the default for OLTP queries: lookup-style joins on indexed foreign keys with selective filters. EXPLAIN shows Nested Loop with an Index Scan on the inner side. 4. The failure mode: planner underestimates `outer_rows`. You see Nested Loop with loops = 4 million in EXPLAIN ANALYZE — your query runs for hours. Fix: refresh stats, rewrite to force hash, or add a missing index.

Edge cases to mention: - A nested loop without an inner index is a Cartesian-style scan per outer row — $O(N \times M)$. Almost always wrong on large tables. - Postgres can wrap the inner side in a Memoize node (since 14) to cache repeated probes — helps when the outer has duplicates on the join key. - For correlated subqueries, the planner often materializes the outer or uses a semi-join variant.

What NOT to say: - Do not say "nested loop is the slowest join" — for small outer sides with indexed inner, it is the fastest. - Do not blame the join algorithm without checking cardinality estimates first.

Common follow-up: *"My nested loop is doing 5 million inner probes — what is the fix?"* — Most often, missing or wrong index on the inner side, or stale stats making the outer look small. Re-ANALYZE and re-check.

```
Nested Loop (cost=...) rows=10
-> Index Scan on customers c (rows=10) Index Cond: city = 'Pune'
-> Index Scan on orders o (rows=1) Index Cond: customer_id = c.id
```

Q164 · Hash Join — When the Planner Picks It

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, JP Morgan India, Target · Topic: Join Algorithms

The question (interviewer's verbatim phrasing):

When is a hash join the right algorithm? What does the planner need to commit to one?

The 30-second answer: Hash join is chosen when both sides are large and there is no usable index on the join column, or when the join is non-correlated (equi-join on hashable types). The planner builds a hash table on the smaller side in memory, then streams the larger side through it. It needs an equality predicate and enough `work_mem` to hold the build side.

Step-by-step thought process: 1. Describe the two phases: build (hash the smaller side into a memory hash table) and probe (stream the larger side, hash each row, look up matches). 2. Picked when: equi-join on a hashable column, both inputs are large, and no good index makes nested loop viable. 3. `work_mem` matters — if the build side does not fit, Postgres falls back to a batched hash join that spills to disk (multiple temp files). EXPLAIN ANALYZE shows Batches > 1 and "Disk Usage" lines. 4. SQL

Server uses Grace and Recursive variants for large builds; Oracle has hash join with one-pass and multi-pass variants. MySQL added hash join in 8.0.18 and made it the default for unindexed joins in 8.0.20.

Edge cases to mention: - Hash join requires equality — range or inequality joins ($a.x < b.y$) cannot hash. - A heavily skewed build side (one key with millions of rows) causes hash collisions and pathological probe cost. - Building on the larger side by mistake (planner picked wrong) blows memory — usually a stats problem.

What NOT to say: - Do not say "hash join is faster than nested loop" — depends on sizes. - Do not assume hash join works for non-equi predicates.

Common follow-up: *"My hash join spills to disk — how do I know and what do I do?"* — EXPLAIN ANALYZE shows Batches > 1 or Buckets x Batches; raise work_mem for the session or add an index that enables a different algorithm.

```
Hash Join (cost=...) Hash Cond: o.customer_id = c.id
-> Seq Scan on orders o
-> Hash Buckets: 16384 Batches: 1 Memory Usage: 1024kB
    -> Seq Scan on customers c
```

Q165 · Merge Join — When the Planner Picks It

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Walmart Labs, Wells Fargo · Topic: Join Algorithms

The question (interviewer's verbatim phrasing):

When does the optimizer pick a merge join? Is it common in OLTP?

The 30-second answer: Merge join requires both inputs sorted on the join key — it is chosen when both sides are already sorted (often via a B-tree index) or when one side is sorted and the other is cheap to sort. It is rare in OLTP but common in analytics or when joining two large pre-indexed tables.

Step-by-step thought process: 1. Describe the algorithm: two pointers, one on each sorted input, advance together — like the merge step of mergesort. 2. The win: linear in the size of both inputs, no hash table, no random IO. The cost is the sort, which is why the planner picks it only when sort is cheap or unnecessary. 3. When already sorted: range scans on an index, or an ORDER BY that produces sorted output, can feed directly into the merge. 4. SQL Server uses Merge Join for ordered indexed sources; Oracle's Sort Merge Join is the same family. Postgres calls it Merge Join. MySQL did not have one classically; recent versions do under specific conditions.

Edge cases to mention: - If the planner has to sort one side, it adds an explicit Sort node above the input — visible in EXPLAIN. - Merge join is sensitive to duplicates on the join key — heavy duplicates can cause rewind logic. - For very small inputs, the constant overhead of merge loses to nested loop.

What NOT to say: - Do not say "merge join is the best for big joins" — hash join wins more often in practice. - Do not assume merge join automatically uses indexes — it uses sortedness, which an index can provide.

Common follow-up: *"Why does merge join sometimes appear without a Sort node?"* — Because the input is already sorted by an Index Scan on the join key.

```
Merge Join (cost=...) Merge Cond: a.id = b.account_id
-> Index Scan using accounts_pkey on accounts a
-> Index Scan using idx_txn_account on transactions b
```

Q166 · Statistics and Why Stale Stats Break Plans

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What are statistics in a query optimizer, and why does the same query suddenly run 100x slower one morning?

The 30-second answer: Statistics are sampled summaries of column data — row counts, distinct values, most common values, histograms — that the optimizer uses to estimate cardinality at each step. When stats are stale (last sampled days ago, after a bulk insert or delete), estimates diverge from reality and the planner picks a bad plan. The fix is to ANALYZE the table.

Step-by-step thought process: 1. Define statistics: pg_statistic in Postgres, sys.stats in SQL Server, all_tab_col_statistics in Oracle. Stored per column, sometimes per index. 2. Common stats: n_distinct, null_frac, most_common_values, histogram. The planner uses these to estimate the row count of every filter and join. 3. Stale stats scenario: an overnight bulk load multiplies the table 10x but autovacuum has not yet sampled it. The planner thinks 100k rows, plans nested loop, hits 1 million rows, dies. 4. Fix: run ANALYZE (Postgres), UPDATE STATISTICS (SQL Server), DBMS_STATS.GATHER_TABLE_STATS (Oracle). Tune autovacuum thresholds or schedule manual ANALYZE after bulk loads.

Edge cases to mention: - default_statistics_target in Postgres controls histogram resolution — raise from 100 to 1000 for columns with skewed distributions. - Stats sample only a small fraction of rows by default — extremely skewed columns may need higher targets or extended statistics. - After a partitioning change or a large UPDATE that rewrites the table, manual ANALYZE is often needed even with autovacuum on.

What NOT to say: - Do not say "the database always knows the row counts" — it estimates from samples. - Do not blame the query when the symptom is sudden — check stats freshness first.

Common follow-up: *"How often should I run ANALYZE on a high-write table?"* — Tune `autovacuum_analyze_scale_factor` lower (default 0.1 = 10 percent change); or run nightly explicit ANALYZE during quiet hours.

```
ANALYZE VERBOSE orders;  
-- Or, in production after a bulk load:  
ANALYZE orders (customer_id, order_date, status);
```

Q167 · Extended Statistics for Column Correlations

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan India · Topic: Optimizer

The question (interviewer's verbatim phrasing):

The planner thinks WHERE city = 'Mumbai' AND state = 'Maharashtra' will match 1000 rows but the actual is 1 million. Why, and how do you fix it?

The 30-second answer: By default, the planner assumes column independence — selectivity of A AND B equals $\text{selectivity}(A) \times \text{selectivity}(B)$. When columns are correlated (city implies state), this underestimates by orders of magnitude. In Postgres, CREATE STATISTICS on the column group teaches the planner about the dependency.

Step-by-step thought process: 1. Explain the independence assumption — it is mathematically wrong whenever columns are functionally or partially correlated. 2. Real example: city = 'Mumbai' selects about 0.1 percent of rows; state = 'Maharashtra' selects about 9 percent; the planner multiplies and predicts 0.009 percent, but city already implies state, so the real selectivity is 0.1 percent. 3. Fix in Postgres: CREATE STATISTICS stat_city_state (dependencies, ndistinct) ON city, state FROM orders; then ANALYZE. The planner now models the correlation. 4. SQL Server has multi-column statistics (auto-created with composite indexes, manually with CREATE STATISTICS); Oracle has Extended Statistics with DBMS_STATS.CREATE_EXTENDED_STATS.

Edge cases to mention: - Three kinds of dependency stats in Postgres: ndistinct (combined distinct values), dependencies (functional dependency strength), mcv (most common combinations). - Extended stats are not free — they add to ANALYZE time and storage. - They do not help if the correlation is conditional on a third column — still a known optimizer limitation.

What NOT to say: - Do not say "stats are stats, there is nothing more to add" — extended stats are a real and underused feature. - Do not assume composite indexes give the optimizer correlation knowledge in every engine — Postgres requires explicit CREATE STATISTICS for predicate correlation.

Common follow-up: "How do I see what extended statistics exist?" — `\d+ table_name` in psql shows statistics objects; query `pg_statistic_ext` for details.

```
CREATE STATISTICS stat_orders_city_state (dependencies)
  ON city, state FROM orders;
ANALYZE orders;
```

Q168 · MySQL 8 Histograms

Tags: Difficulty: Medium · Asked at: Flipkart, Meesho, Swiggy, Walmart Labs · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What are histograms in MySQL 8, and when does the optimizer actually use them?

The 30-second answer: MySQL 8 added column histograms (singleton or equi-height) that the optimizer uses to estimate selectivity on non-indexed columns. They are most useful for skewed distributions where the default uniformity assumption is wrong — for example, a status column that is 95 percent COMPLETED.

Step-by-step thought process: 1. Pre-MySQL-8, the optimizer relied on index statistics and uniformity assumptions for non-indexed columns. Heavily skewed columns gave bad row estimates. 2. `ANALYZE TABLE orders UPDATE HISTOGRAM ON status, payment_method WITH 100 BUCKETS` builds histograms — singleton if distinct values $\leq$ buckets, equi-height otherwise. 3. The optimizer uses them for range, equality and IN predicates on the column to estimate row counts. Visible in `EXPLAIN FORMAT=JSON` in the "rows_examined_per_scan" and cost output. 4. Histograms are static — they do not auto-refresh after DML. Re-run `ANALYZE TABLE ... UPDATE HISTOGRAM` after large data changes.

Edge cases to mention: - Histograms apply per column, not multi-column — for correlated predicates, you still need an index or rewrite. - Adding an index on a column overrides the histogram in most planner decisions because index statistics are richer. - They are stored in `information_schema.column_statistics` — easy to inspect.

What NOT to say: - Do not say "histograms automatically refresh" — they do not. - Do not confuse MySQL histograms with Postgres histograms (built into `pg_statistic`) — same concept, different lifecycle.

Common follow-up: "Do histograms help with skewed timestamp columns?" — Yes, especially equi-height histograms on `order_date` — they let the optimizer see that the last 7 days holds 80 percent of rows.

```
ANALYZE TABLE orders UPDATE HISTOGRAM ON status, payment_method WITH 100 BUCKETS;  
SELECT * FROM information_schema.column_statistics  
WHERE table_name = 'orders';
```

Q169 · Cardinality Estimation — What Goes Wrong

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan India · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What is cardinality estimation, and what are the top three reasons it goes wrong in production?

The 30-second answer: Cardinality estimation is the optimizer's guess of how many rows each operator will emit. The top three failure modes are stale statistics, correlated predicates the planner assumes independent, and complex expressions (function calls, IN with subqueries, OR with skew) where the estimator falls back to defaults like 10 percent or 0.5 percent.

Step-by-step thought process: 1. Define cardinality — number of rows. Every node in a plan has an estimate and (with EXPLAIN ANALYZE) an actual. The ratio is the most diagnostic single number in plan reading. 2. Failure mode one: stale stats — covered in Q10. The planner is using yesterday's numbers. 3. Failure mode two: correlated predicates — covered in Q11. Independence assumption multiplies selectivities incorrectly. 4. Failure mode three: opaque expressions — WHERE my_udf(x) = 5 cannot be estimated, planner uses a magic default; WHERE x IN (SELECT ...) is estimated from the subquery output; complex OR with overlapping ranges defaults to a rough guess.

Edge cases to mention: - Skew on a single value (90 percent of orders have status = 'COMPLETED') — without histograms, the planner uses average selectivity and underestimates "COMPLETED" or overestimates "REFUNDED". - Filter on a computed column without an expression index — planner has no stats for the expression. - Cross-database queries (foreign tables, dblink) — stats are usually unavailable.

What NOT to say: - Do not say "the planner always gets it right" — it is a probabilistic guess based on samples. - Do not blame the planner without checking estimated vs actual rows first.

Common follow-up: "How do I tell the planner about a UDF's selectivity?" — In Postgres, mark it IMMUTABLE and add a CREATE STATISTICS or use a generated stored column. In Oracle, use the ASSOCIATE STATISTICS clause.

```
-- The diagnostic line you scan first in EXPLAIN ANALYZE:
Seq Scan on orders  (cost=...) rows=100 (actual rows=1234567)
                ^^^^^^                ^^^^^^^^^^^^^^^
                        4-order-of-magnitude miss = stats problem
```

Q170 · Plan Regression — Same Query, New Plan

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan India · Topic: Optimizer

The question (interviewer's verbatim phrasing):

A report that ran in 2 seconds yesterday now takes 4 minutes. The query has not changed. Walk me through your diagnosis.

The 30-second answer: The plan changed — the SQL is identical but the optimizer is now picking a different plan. Compare today's EXPLAIN to a known-good baseline; the usual culprits are stale or freshly updated statistics flipping a join order, autovacuum just ran, a parameter sniffing event, or a small data shift crossing a planner threshold.

Step-by-step thought process: 1. First, confirm it is a plan change, not a load issue. Capture EXPLAIN today; compare to a saved plan from yesterday if you have one, or check shared cache / Query Store. 2. Common triggers: ANALYZE just ran with a fresh sample that changed n_distinct; a new index was added on another query and the planner picked it inappropriately here; data growth crossed an index-vs-seqscan tipping point. 3. Verify with actual versus estimated row counts. Big mismatch on a node that used to be tight = stats drift. 4. Mitigation: refresh stats with a higher target, add or hint the index, force the prior plan via Query Store baseline (SQL Server), SQL Plan Baseline (Oracle), or pg_hint_plan (Postgres).

Edge cases to mention: - Parameter sniffing in SQL Server can cause the first-execution plan to be cached and reused for inputs it does not suit — fix with OPTION (RECOMPILE) or OPTIMIZE FOR. - Postgres does not have per-plan baselines natively; pg_hint_plan or pg_plan_filter are extensions. - Cron-driven nightly ANALYZE can be the unexpected cause when stats refresh changes one estimate.

What NOT to say: - Do not say "the query must have changed" without checking — confirm SQL text identity first. - Do not blame the database without comparing plans side by side.

Common follow-up: "How do you prevent this in the first place?" — Capture plans regularly (auto_explain in Postgres, Query Store in SQL Server, AWR in Oracle); set alerts on duration regressions per query hash.

```
-- Postgres: enable plan logging for slow queries
SET auto_explain.log_min_duration = '500ms';
SET auto_explain.log_analyze = on;
```

Q171 · Pinned Plans and Forced Plans

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Wells Fargo, AmEx · Topic: Optimizer

The question (interviewer's verbatim phrasing):

Your team wants to pin the plan for a critical reporting query so it never regresses. How would you do it in SQL Server, Oracle, and Postgres?

The 30-second answer: SQL Server: Query Store forced plans — pick the known-good plan in Query Store and force it for the query_hash. Oracle: SQL Plan Baselines (DBMS_SPM.LOAD_PLANS_FROM_CURSOR_CACHE) — captures and pins the accepted plan. Postgres has no native equivalent; teams use pg_hint_plan to embed hints, or maintain SQL with explicit join order and index hints via comments the extension reads.

Step-by-step thought process: 1. State the use case — a high-stakes report or batch job where occasional regression is unacceptable, more important than possible future improvements. 2. SQL Server: enable Query Store, identify the plan by plan_id in sys.query_store_plan, call sys.sp_query_store_force_plan. Plan is honored until you unforce. 3. Oracle: capture cursor with DBMS_SPM.LOAD_PLANS_FROM_CURSOR_CACHE to create an accepted baseline. Optimizer will only use baseline plans for that SQL text. 4. Postgres: install pg_hint_plan, add */+ HashJoin(a b) IndexScan(orders idx_orders_date)* / comment hints. Native plan stability is not provided.

Edge cases to mention: - Pinned plans go stale — a 1-billion-row table growth makes the forced nested loop disastrous. Schedule periodic review. - Forcing a plan requires the underlying objects (indexes) to still exist; dropping them invalidates the plan. - Avoid pinning as a substitute for fixing stats — it hides the root cause.

What NOT to say: - Do not say "Postgres has native plan baselines" — it does not. - Do not pin every query "just in case" — you lose the optimizer's improvements forever.

Common follow-up: *"How do you know when to unpin?"* — Periodic review: compare the pinned plan's actual runtime against the optimizer's current best on a representative sample of inputs.

```
-- SQL Server: force a known-good plan in Query Store
EXEC sys.sp_query_store_force_plan @query_id = 1234, @plan_id = 5678;
```

Q172 · Rewrite Subquery to JOIN

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Rewrites

The question (interviewer's verbatim phrasing):

Rewrite this subquery as a JOIN: `SELECT name FROM customers WHERE id IN (SELECT customer_id FROM orders WHERE total > 10000)`.

The 30-second answer: `SELECT DISTINCT c.name FROM customers c JOIN orders o ON c.id = o.customer_id WHERE o.total > 10000`. The `DISTINCT` is needed because a customer with multiple matching orders would otherwise appear several times. Modern optimizers usually generate identical plans for both forms, but the `JOIN` version is sometimes clearer and lets you pull in columns from orders if needed.

Step-by-step thought process: 1. Identify the semantic: `IN` with subquery is a semi-join — one output row per matching left row, regardless of how many right matches. 2. The `JOIN` equivalent multiplies rows when the right side has duplicates — `DISTINCT` or `GROUP BY` is required to restore semi-join semantics. 3. Talk about when the rewrite is worth it: when you also want columns from orders, `JOIN` avoids a second query; otherwise, `EXISTS` is the cleanest form because it does not need `DISTINCT`. 4. Be ready for "Which plan does Postgres pick?" — usually a Hash Semi Join for both `IN` and `EXISTS`, identical plan.

Edge cases to mention: - `DISTINCT` in a `JOIN` rewrite is a real cost when the join is wide — `EXISTS` avoids that. - If the subquery is uncorrelated and returns a small static list, the planner may evaluate it once and treat it as a constant set. - The rewrite changes plan only in older optimizers (MySQL pre-5.6, ancient SQL Server).

What NOT to say: - Do not say "JOIN is always faster than IN" — modern optimizers rewrite them to the same shape. - Do not omit `DISTINCT` — duplicate rows are a real bug.

Common follow-up: "*When would EXISTS beat both?*" — When you only need a boolean check and the right side has wide rows or many duplicates — `EXISTS` short-circuits and avoids deduplication.

```
SELECT DISTINCT c.name
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE o.total > 10000;
```

Q173 · NOT IN to NOT EXISTS (NULL Safety)

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, JP Morgan India · Topic: Rewrites

The question (interviewer's verbatim phrasing):

Why is NOT IN dangerous, and how does rewriting to NOT EXISTS fix it?

The 30-second answer: NOT IN against a subquery returns the empty set if the subquery yields even one NULL — because $x \neq \text{NULL}$ is UNKNOWN, the whole expression collapses to UNKNOWN, which filters out the row. NOT EXISTS does not have this problem because it tests for existence per outer row, treating NULLs as non-matches naturally.

Step-by-step thought process: 1. Walk through the three-valued logic: WHERE id NOT IN (1, 2, NULL) becomes $\text{id} \neq 1 \text{ AND } \text{id} \neq 2 \text{ AND } \text{id} \neq \text{NULL}$ — the last predicate is UNKNOWN — the conjunction is UNKNOWN — the row is excluded. 2. NOT EXISTS rewrite: WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id). For each outer row, the subquery checks for a match; NULL on either side simply does not match, so the row is correctly returned. 3. Planner behaviour: modern optimizers transform NOT EXISTS into an anti-semi-join — efficient and correct. 4. Also mention LEFT JOIN ... WHERE right_pk IS NULL as a third correct anti-join form, semantically equivalent to NOT EXISTS.

Edge cases to mention: - If the column is declared NOT NULL, NOT IN is safe — but schemas evolve, and a NOT NULL column today may become nullable tomorrow. - NOT IN with a hard-coded list (NOT IN (1, 2, 3)) is safe because you control the list. - NULL semantics are SQL standard, not a quirk — every compliant database behaves this way.

What NOT to say: - Do not say "NOT IN and NOT EXISTS are interchangeable" — they are not, and this is a classic interview trap. - Do not blame the database for "broken" NOT IN behaviour — it is standard SQL.

Common follow-up: "How would you write the safest version once and for all?" — NOT EXISTS, every time. It is correct, fast, and reads cleanly.

```
-- Unsafe: NULL in orders.customer_id makes this return zero rows
SELECT * FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);

-- Safe: NOT EXISTS handles NULLs correctly
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

Q174 · OR to UNION ALL

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs, Cred · Topic: Rewrites

The question (interviewer's verbatim phrasing):

When does rewriting OR conditions as UNION ALL make a query faster?

The 30-second answer: When each side of the OR can independently use a different index, but the combined OR predicate forces the planner into a sequential scan or a bitmap-OR scan that is suboptimal. Splitting into two SELECTs joined by UNION ALL lets each side use its own index — typically twice the index scans, half the runtime, much smaller IO.

Step-by-step thought process: 1. Diagnose the symptom: WHERE phone = '9876543210' OR email = 'a@b.com' on a users table with separate indexes on each column. The planner may Seq Scan or do a BitmapOr; both can be slow. 2. Rewrite: SELECT ... WHERE phone = '9876543210' UNION ALL SELECT ... WHERE email = 'a@b.com' AND phone <> '9876543210'. Each branch uses its own index; the AND in the second branch prevents duplicates when a row matches both. 3. If duplicates are impossible or acceptable, UNION ALL (no dedup) is the faster choice; UNION dedups via a sort or hash. 4. Postgres often handles this via BitmapOr automatically — check the plan before rewriting.

Edge cases to mention: - UNION ALL preserves duplicates; UNION removes them — pick deliberately. - The dedup branch (AND phone <> ...) is necessary only when the same row could match both — for distinct columns on the same key, it is often skippable. - For three or more OR branches, the rewrite gets verbose; consider a CTE or a temp table.

What NOT to say: - Do not blindly rewrite every OR as UNION — the planner handles many ORs well. - Do not use UNION (with dedup) when UNION ALL would do — the sort/hash is wasted work.

Common follow-up: "What does Postgres do natively for this?" — BitmapOr: builds bitmaps for each index, ORs them, then Bitmap Heap Scan. Often as fast as the manual UNION ALL.

```
SELECT id, name FROM users WHERE phone = '9876543210'
UNION ALL
SELECT id, name FROM users WHERE email = 'a@b.com' AND phone <> '9876543210';
```

Q175 · Function on Column — The Index Killer

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Rewrites

The question (interviewer's verbatim phrasing):

Why is WHERE LOWER(email) = 'a@b.com' slow even with an index on email? How do you fix it?

The 30-second answer: Wrapping the column in LOWER() means the predicate is no longer on email; it is on LOWER(email). The B-tree index on email cannot be used because the values are stored in their

original case. Two fixes: create an expression index on LOWER(email), or store emails in a normalized lower-case column.

Step-by-step thought process: 1. Explain the principle: an index stores raw column values. A predicate on f(column) is not a predicate the index can answer unless an index exists on f(column). 2. The fix in Postgres: CREATE INDEX idx_users_email_lower ON users (LOWER(email)). The planner uses it for WHERE LOWER(email) = 'a@b.com'. 3. Vendor call-out: Postgres and Oracle support expression indexes directly; SQL Server uses computed columns marked PERSISTED with an index on the computed column; MySQL 8 supports functional indexes on expressions. 4. Cleaner long-term: store a normalized column (email_lower) updated by trigger or generated column, and index that. Cheaper at query time and clearer in joins.

Edge cases to mention: - Expression indexes require the expression to be IMMUTABLE — functions that depend on session settings (timezone, locale) cannot be indexed. - Updating a row triggers reindexing of the expression — slight write overhead. - Citext (case-insensitive text) in Postgres avoids the LOWER wrap entirely.

What NOT to say: - Do not say "the database should be smart enough to use the index anyway" — it cannot, by design. - Do not skip the IMMUTABLE check — non-immutable expressions fail to index.

Common follow-up: "Same problem with DATE_TRUNC('day', created_at) — what do you do?" — Same answer: expression index on DATE_TRUNC('day', created_at), or rewrite as created_at >= date AND created_at < date + 1 day so the raw column index works.

```
-- Fix with expression index
CREATE INDEX idx_users_email_lower ON users (LOWER(email));
-- Or rewrite to use a date range:
WHERE created_at >= '2026-05-26' AND created_at < '2026-05-27';
```

Q176 · Implicit Cast — The Silent Index Killer

Tags: Difficulty: Medium · Asked at: Razorpay, Goldman, JP Morgan India, Walmart Labs · Topic: Rewrites

The question (interviewer's verbatim phrasing):

Why does WHERE customer_id = '4827' (with quotes) sometimes do a sequential scan even when customer_id is indexed?

The 30-second answer: If customer_id is an integer column and you pass a string, the database implicitly casts one side to match the other. The cast may happen on the column side, which (like a

function) defeats the index. Pass the value with the correct type — `customer_id = 4827` — and the index is usable.

Step-by-step thought process: 1. Walk through the cast resolution. Different engines have different rules; in some, the column gets cast to the parameter's type, turning `customer_id` into `CAST(customer_id AS varchar)`, which no integer B-tree can satisfy. 2. Classic ORM bug: parameter binding sends the value as a string when the column is bigint; the planner casts the column. 3. Diagnose with `EXPLAIN` — look for an Index Cond with a `CAST` around the column, or Seq Scan where you expected Index Scan. 4. Fix at three layers: cast the parameter explicitly (`CAST(? AS BIGINT)`), fix the ORM type mapping, or change the column type (rarely the right answer).

Edge cases to mention: - `VARCHAR` vs `CHAR` — implicit padding can break equality. - Numeric precision mismatches (`NUMERIC(10,2)` vs `DOUBLE`) — cast to the higher type. - Date vs timestamp — equality on a timestamp column with a date value can cast to timestamp at midnight, which still uses index, but range comparisons can confuse the planner.

What NOT to say: - Do not say "implicit casts are harmless" — they are a top-three index-killer in production code. - Do not blame the database for not magically guessing your intent.

Common follow-up: *"Where do you see this most in real apps?"* — Java apps passing String parameters to a BIGINT id column; PHP apps with all-string variables. Always type your parameters explicitly.

```
-- Index killed by implicit cast
WHERE customer_id = '4827'           -- column may be cast to text

-- Correct
WHERE customer_id = 4827             -- planner uses idx_customer_id
```

Q177 · Parameter Sniffing in SQL Server

Tags: Difficulty: Hard · Asked at: AmEx, Wells Fargo, Goldman, JP Morgan India · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What is parameter sniffing in SQL Server, and how does it cause a query to be fast for some inputs and terrible for others?

The 30-second answer: On first execution of a parameterised query, SQL Server uses the actual parameter value to estimate cardinality and caches the resulting plan. If that first value was atypical — say, a customer with 5 orders — the plan optimised for it (nested loop with bookmark lookup) is then reused for a customer with 5 million orders, where a different plan would be far better.

Step-by-step thought process: 1. Explain plan caching: SQL Server stores compiled plans keyed by SQL text. Parameterised queries get one plan per text, optimised for the parameter values seen during compilation. 2. The sniffing problem: the first set of parameter values "wins" — even if they are unrepresentative — and every subsequent execution uses that plan. 3. Mitigations: OPTION (RECOMPILE) recompiles every execution (CPU cost but correct plan); OPTION (OPTIMIZE FOR (@p = value)) hints a specific representative value; OPTION (OPTIMIZE FOR UNKNOWN) uses average density. 4. Newer: Query Store with multiple plans per query and automatic plan correction (SQL Server 2017+) can detect regressions and revert.

Edge cases to mention: - Same problem exists in Oracle as "bind variable peeking" — solved by adaptive cursor sharing. - Postgres has a similar effect with prepared statements: after five executions, it switches from custom plan to generic plan; you can override with `plan_cache_mode`. - ORM-generated queries are especially vulnerable because the first call wins for the lifetime of the connection pool entry.

What NOT to say: - Do not say "parameter sniffing is always bad" — it is usually correct; only the edge case is bad. - Do not blanket-add OPTION (RECOMPILE) — it taxes CPU on hot queries.

Common follow-up: *"How do you diagnose parameter sniffing?"* — Look for the same query_hash in Query Store with wildly varying durations across plan executions; compare cached_plan parameters with actual workload distribution.

```
-- Force recompile for a specific call
SELECT * FROM orders WHERE customer_id = @cust_id OPTION (RECOMPILE);
```

Q178 · Prepared Statements vs Ad-Hoc and Plan Caching

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, PhonePe · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What is the difference between prepared statements and ad-hoc queries from the planner's point of view? Why might prepared statements be slower for some queries?

The 30-second answer: Prepared statements parse and plan once, then execute many times with different parameter values — saves CPU on planning. The risk is that the cached plan is generic (no parameter values to use for cardinality) and may be worse than an ad-hoc plan that sees the actual values. Postgres handles this with a 5-execution heuristic that switches between custom and generic plans.

Step-by-step thought process: 1. Two costs: parse-and-plan time versus execution time. Hot OLTP queries amortize prepare cost; one-shot reports do not benefit. 2. Plan generality: with PREPARE foo(int) AS SELECT ..., the planner does not know the value of \$1 during planning — it uses average selectivity,

often worse than seeing the literal. 3. Postgres specifically: tracks five executions to decide if custom plans (re-plan with each value) beat the generic plan; switches automatically. You can force with `plan_cache_mode = 'force_custom_plan'` or `'force_generic_plan'`. 4. Server-side prepared statements are the default in JDBC, psycopg2 server-side cursors, and SQL Server `sp_executesql` — easy to forget the optimizer caveats.

Edge cases to mention: - Massive variation in parameter selectivity (an IN list of size 1 vs 10000) suggests custom plans. - Connection pools with prepared statement caches reuse across customers — beware of leaking plans. - pgbouncer in transaction pooling mode breaks server-side prepared statements unless you use protocol-level support carefully.

What NOT to say: - Do not say "prepared statements are always faster" — for one-shot queries, the prepare overhead is wasted. - Do not assume the cached plan is the optimal plan — it may be the average plan, not the best for any specific input.

Common follow-up: *"How do you know your app is using server-side prepared statements?"* — Check the driver config; in Postgres, `pg_stat_statements` shows queries with `$1` instead of literal values when prepared.

```
PREPARE stmt(int) AS SELECT * FROM orders WHERE customer_id = $1;
EXECUTE stmt(4827);
-- Plan is decided after 5 executions in Postgres default
```

Q179 · Plan Cache Pollution

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Wells Fargo, AmEx · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What is plan cache pollution, and how does an app accidentally cause it?

The 30-second answer: Plan cache pollution is when the cache fills with thousands of near-identical plans because the application emits SQL text that differs slightly per call — typically by inlining literals instead of using parameters. Each variant gets its own cache entry; useful plans get evicted; planning CPU spikes.

Step-by-step thought process: 1. Cache keyed by exact SQL text in most engines. `SELECT * FROM orders WHERE id = 1` and `SELECT * FROM orders WHERE id = 2` are two cache entries. 2. The cure is parameterisation: `WHERE id = ?` — one cache entry for every customer id. 3. SQL Server has "forced parameterisation" and "optimize for ad hoc workloads" to mitigate by caching only plan stubs for one-shot SQL. 4. Postgres has `pg_stat_statements` that normalizes literals for monitoring, but the planner still re-plans per text. Use server-side prepared statements via your driver.

Edge cases to mention: - IN lists of varying size pollute (IN (1,2,3) vs IN (1,2,3,4)) — solve with arrays (= ANY(\$1)) or temp tables. - Dynamic SQL stitched in app code is a frequent culprit. - Some ORMs deliberately inline literals for "constants" — check the generated SQL.

What NOT to say: - Do not say "the cache is unlimited" — it is bounded; eviction hurts hot queries. - Do not blame the database; it is almost always the app.

Common follow-up: *"How do you measure cache hit rate?"* — In Postgres, count pg_stat_statements rows; in SQL Server, sys.dm_exec_cached_plans and sys.dm_exec_query_stats; in Oracle, V\$LIBRARYCACHE.

```
-- Pollutes: each value is a separate cache entry
SELECT * FROM orders WHERE id = 1;
SELECT * FROM orders WHERE id = 2;

-- Cache-friendly: one entry, reused
SELECT * FROM orders WHERE id = $1;
```

Q180 · Parallel Query

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan India, AmEx · Topic: Query Plans

The question (interviewer's verbatim phrasing):

When does Postgres or SQL Server parallelize a query, and what makes a function "parallel safe"?

The 30-second answer: The planner parallelises when the table is large enough (parallel_setup_cost amortizes), the query touches enough data to benefit, and every operator and function in the plan is parallel-safe. A parallel-safe function has no side effects, does not depend on session state, and returns the same answer regardless of which worker executes it.

Step-by-step thought process: 1. Postgres parallel triggers: max_parallel_workers_per_gather > 0, min_parallel_table_scan_size and min_parallel_index_scan_size cleared, and a parallel-safe plan. 2. EXPLAIN shows Gather (or Gather Merge), with parallel-aware nodes underneath (Parallel Seq Scan, Parallel Hash Join). 3. Functions and operators are categorized: PARALLEL SAFE (default for built-ins like + and length), PARALLEL RESTRICTED (cannot be in parallel worker but can be in the leader), PARALLEL UNSAFE (forces serial). User-defined functions default to UNSAFE — mark them explicitly. 4. SQL Server has Cost Threshold for Parallelism (default 5, often raised to 50+ in OLTP) and Max Degree of Parallelism (MAXDOP). Oracle uses PARALLEL hints and the parallel query slave pool.

Edge cases to mention: - Locks held across workers — some operations (CREATE INDEX CONCURRENTLY) cannot parallelise. - Parallel hash join scales well but bumps work_mem usage per worker. - For OLTP, parallel query is usually disabled or limited — it steals CPU from other connections.

What NOT to say: - Do not say "more parallelism is always faster" — overhead and synchronization cost can dominate for short queries. - Do not mark all UDFs PARALLEL SAFE without auditing for side effects.

Common follow-up: "*What is parallel-restricted versus parallel-unsafe?*" — Restricted runs only in the leader; unsafe disables parallelism entirely for the query.

```
CREATE FUNCTION clean_phone(text) RETURNS text
LANGUAGE plpgsql IMMUTABLE PARALLEL SAFE
AS $$ BEGIN RETURN regexp_replace($1, '\\D', '', 'g'); END $$;
```

Q181 · Materialized Views

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Meesho · Topic: Optimizer

The question (interviewer's verbatim phrasing):

When is a materialized view worth the operational cost, and what refresh patterns do you use?

The 30-second answer: Worth it when the underlying query is expensive (multi-table aggregation, joins on large tables), is read far more often than the underlying data changes, and freshness up to a known lag is acceptable. Refresh patterns: full refresh on a schedule (cron, REFRESH MATERIALIZED VIEW), incremental refresh (Oracle fast refresh, manual delta merge), or trigger-driven (CDC or change tables feeding the view).

Step-by-step thought process: 1. Decision criteria: query cost > refresh cost, read-to-write ratio favors caching, staleness window is acceptable to the business. 2. Postgres: CREATE MATERIALIZED VIEW; REFRESH MATERIALIZED VIEW (locks readers) or REFRESH MATERIALIZED VIEW CONCURRENTLY (requires unique index, slower but non-blocking). 3. Oracle has incremental refresh via materialized view logs that record deltas; SQL Server has indexed views with strict requirements (SCHEMABINDING, deterministic functions). 4. Operational: track refresh duration and lag; alert if lag exceeds SLA; for very large views, partition the underlying data and refresh only changed partitions.

Edge cases to mention: - REFRESH CONCURRENTLY requires a unique index on the view, and uses much more IO than the non-concurrent version. - Postgres materialized views are not auto-refreshed — you must schedule it. - For dashboards that need near-real-time data, a materialized view may not be the right tool — consider rollup tables with incremental updates.

What NOT to say: - Do not say "materialized views are just cached queries" — they are persistent storage with their own indexes. - Do not enable concurrent refresh without considering the IO cost.

Common follow-up: "How do you handle stale results when a user is viewing the dashboard during a refresh?" — CONCURRENTLY refresh ensures readers always see a consistent (possibly slightly stale) view; non-concurrent locks readers.

```
CREATE MATERIALIZED VIEW mv_top_customers AS
  SELECT customer_id, SUM(total_amount) AS lifetime_value
  FROM orders WHERE status = 'COMPLETED'
  GROUP BY customer_id;

CREATE UNIQUE INDEX ON mv_top_customers (customer_id);
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_customers;
```

Q182 · Query Hints — When Justified, When Smelly

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Walmart Labs, AmEx · Topic: Optimizer

The question (interviewer's verbatim phrasing):

When is it justified to use a query hint, and when is it a code smell?

The 30-second answer: Justified when you have measured the optimizer choosing badly for a stable workload despite correct stats, and you need a predictable plan now — typically for high-stakes batch jobs. Smelly when used as a default tool, when stats have not been refreshed, or when the workload is volatile and a fixed plan will eventually be wrong.

Step-by-step thought process: 1. Hint vocabulary: Oracle has the richest set (`/+ INDEX(t idx)` `USE_HASH(a b) /`); SQL Server has `OPTION` clauses (`OPTION (HASH JOIN, LOOP JOIN)`); MySQL has `/+ HASH_JOIN /` and `ignore/use index`; Postgres has none natively (use `pg_hint_plan` extension). 2. Justification: regulator-mandated nightly report cannot regress; a forced join order or join method gives predictability the optimizer cannot guarantee. 3. Smell: every other developer adds `/+ INDEX /` because someone once needed it. Hints freeze the plan against future schema changes, data growth, and optimizer improvements. 4. Always document why a hint exists. Add a comment with the date, the plan it replaced, and the trigger for re-evaluation.

Edge cases to mention: - A hint can be silently invalid (index dropped) and the planner ignores it — no error in some engines, just a regression. - Some hints are advisory; the optimizer can override them in newer versions. - Forcing a join order with Oracle's `LEADING` hint can be brittle to query rewrites.

What NOT to say: - Do not say "Postgres has hints" without the extension caveat — it does not natively. - Do not paste hints into queries to "speed them up" without diagnosing.

Common follow-up: *"What is the modern alternative to query hints?"* — SQL Server Query Store forced plans, Oracle SQL Plan Baselines, well-targeted CREATE STATISTICS. They are auditable and survive code reviews.

```
-- Oracle: hint a specific index
SELECT /*+ INDEX(orders idx_orders_cust_date) */
       customer_id, COUNT(*) FROM orders GROUP BY customer_id;
```

Q183 · EXPLAIN ANALYZE BUFFERS — IO vs Compute

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan India · Topic: Query Plans

The question (interviewer's verbatim phrasing):

In Postgres, what does EXPLAIN (ANALYZE, BUFFERS) tell you that EXPLAIN ANALYZE alone does not?

The 30-second answer: BUFFERS shows per-node buffer activity — shared hit (in cache), shared read (from disk), local and temp buffer usage. It tells you whether a slow query is IO-bound (lots of shared reads) or CPU-bound (lots of hits but still slow). It also reveals when work_mem spills cause temp file IO.

Step-by-step thought process: 1. Without BUFFERS, you know wall-clock time per node; with BUFFERS, you know the source of that time. 2. shared hit = cache hit, fast. shared read = disk read, slow on cold cache. temp written/read = sort or hash spill — raise work_mem. 3. A node with 10ms actual time and 100k shared hits is CPU-bound (lots of in-memory work); a node with 200ms and 5000 shared reads is IO-bound. 4. Use this to decide whether to add indexes (reduce shared reads), raise work_mem (eliminate temp), or restructure the query (reduce CPU).

Edge cases to mention: - track_io_timing must be enabled to see read/write times; otherwise you only get counts. - Buffer counts include both heap and index pages; the breakdown is not separated in the text plan. - Cold versus hot runs differ dramatically — run the query twice and look at the second run for a steady-state picture.

What NOT to say: - Do not say "BUFFERS slows the query down" — it adds negligible overhead. - Do not ignore temp buffer numbers; spilling is a top cause of slow analytics queries.

Common follow-up: *"How do you reduce shared reads?"* — Better indexes (so fewer pages are scanned), partitioning (so unrelated partitions are skipped), or larger shared_buffers if the working set fits.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, SUM(total_amount) FROM orders
WHERE order_date >= '2026-01-01' GROUP BY customer_id;
-- Buffers: shared hit=12345 read=678 written=0
```

Q184 · work_mem and Sort Spills

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan India, AmEx · Topic: Optimizer

The question (interviewer's verbatim phrasing):

What is work_mem in Postgres or sort_buffer_size in MySQL, and what happens when a sort exceeds it?

The 30-second answer: work_mem is the per-operation memory budget for sorts, hash tables, and other internal buffers. When a sort or hash exceeds it, Postgres switches to an external (on-disk) merge sort or batched hash join — spilling to temp files. Spill IO can multiply query time tenfold or more.

Step-by-step thought process: 1. Define: work_mem is per-sort, per-hash, per-query — not per-connection. A query with 5 sorts and 3 hashes can use 8 * work_mem. 2. Detect a spill: EXPLAIN ANALYZE shows "Sort Method: external merge" or "Batches: 16" instead of "Batches: 1". With BUFFERS, you see temp written/read. 3. Fix options: raise work_mem at the session level (SET work_mem = '256MB') for the slow query, add an index that avoids the sort, or rewrite to reduce the data being sorted. 4. Global work_mem is risky — each connection can use it many times; a high default plus 200 connections can OOM the host.

Edge cases to mention: - Parallel workers each get their own work_mem allocation — multiply by max_parallel_workers_per_gather. - Some operations (DISTINCT, GROUP BY, hash aggregate) use work_mem and spill similarly. - Spill files live in pg_default_temp_tablespace by default — moving temp to a fast SSD can soften the pain.

What NOT to say: - Do not say "work_mem is the total memory for the query" — it is per-operation. - Do not raise work_mem globally without considering concurrency.

Common follow-up: "When would you raise work_mem at the session level instead of globally?" — For a one-off batch job that legitimately needs 1GB; you set it inside the transaction so other connections are unaffected.

```
SET LOCAL work_mem = '256MB';
SELECT customer_id, ARRAY_AGG(order_id ORDER BY order_date)
FROM orders GROUP BY customer_id;
-- Sort Method: quicksort Memory: 218MB (no spill)
```

Q185 · Lateral Subquery for Top-N-Per-Group

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart, Walmart Labs · Topic: Rewrites

The question (interviewer's verbatim phrasing):

Show me how to get the three most recent orders per customer efficiently using LATERAL.

The 30-second answer: LATERAL lets a subquery reference columns from earlier items in the FROM list. For top-N-per-group, you iterate over customers and laterally take the top N orders per customer — with an index on (customer_id, order_date DESC), each lateral subquery is a cheap index scan limited to 3 rows.

Step-by-step thought process: 1. Without LATERAL, the standard solution is ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) — works but sorts the entire orders table. 2. With LATERAL and an index on (customer_id, order_date DESC), the planner does a Nested Loop: outer customers, inner index scan with LIMIT 3 — orders of magnitude faster when there are many customers and many orders. 3. Syntax: SELECT c.id, o.order_id, o.order_date FROM customers c, LATERAL (SELECT order_id, order_date FROM orders WHERE customer_id = c.id ORDER BY order_date DESC LIMIT 3) o. 4. SQL Server uses CROSS APPLY (and OUTER APPLY) for the same pattern. Oracle has LATERAL. MySQL 8.0.14+ supports LATERAL.

Edge cases to mention: - The index on (customer_id, order_date DESC) is the load-bearing part — without it, LATERAL is no better than ROW_NUMBER. - LATERAL excludes customers with zero orders; LEFT JOIN LATERAL keeps them. - For top-1-per-group, DISTINCT ON (Postgres) is even cleaner.

What NOT to say: - Do not say "window functions are always best for top-N" — for top-N-per-group on indexed columns, LATERAL is often dramatically faster. - Do not omit the supporting index — the rewrite alone does not save you.

Common follow-up: "What plan would you expect to see?" — Nested Loop with outer Seq Scan on customers and inner Index Scan + Limit on orders.

```
SELECT c.id, o.order_id, o.order_date, o.total_amount
FROM customers c,
LATERAL (
  SELECT order_id, order_date, total_amount
  FROM orders WHERE customer_id = c.id
  ORDER BY order_date DESC LIMIT 3
) o;
```

Q186 · MAX with Rare Filter — Index-Only Scan Opportunity

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Walmart Labs, AmEx · Topic: Query Plans

The question (interviewer's verbatim phrasing):

SELECT MAX(amount) FROM payments WHERE status = 'FAILED' is slow on a 500-million-row table. What is the right index and rewrite?

The 30-second answer: Build a partial index on (amount DESC) WHERE status = 'FAILED' — small, hot, and a Bitmap Index Scan or Index Scan plus Limit 1 reads only the rarely-failing rows. Alternatively, ORDER BY amount DESC LIMIT 1 with the same index gives the planner a direct lookup. Pair with index-only scan if no other columns are needed.

Step-by-step thought process: 1. Anti-pattern: full table scan filtering for FAILED rows, then computing MAX. On 500M rows, this is minutes even with high parallelism. 2. The right index: a partial index that holds only the failed rows, sorted by amount. CREATE INDEX idx_payments_failed_amt ON payments (amount DESC) WHERE status = 'FAILED'. The index is tiny (failed rows are rare), fits in cache, and answers the query with Limit 1. 3. Rewrite to enable the optimization: SELECT amount FROM payments WHERE status = 'FAILED' ORDER BY amount DESC LIMIT 1. Equivalent to MAX for non-NULL amounts, more index-friendly. 4. Confirm with EXPLAIN — you want Index Only Scan or Index Scan with Limit 1, not a Seq Scan with Aggregate above.

Edge cases to mention: - Postgres supports partial indexes natively; SQL Server has filtered indexes; Oracle has function-based indexes; MySQL 8 supports functional indexes but lacks partial. - If amount can be NULL, MAX ignores NULLs but ORDER BY DESC puts them first (with NULLS LAST not the default in some engines) — use ORDER BY amount DESC NULLS LAST. - Maintenance cost of the partial index is proportional to write rate of FAILED rows — usually very low for FAILED.

What NOT to say: - Do not say "MAX needs no special handling" — at 500M rows it absolutely does. - Do not build a full index on amount alone — large, unused for everything but this one query.

Common follow-up: "What if I need MAX(amount) per merchant_id where status = 'FAILED'?" — Composite partial index (merchant_id, amount DESC) WHERE status = 'FAILED', then DISTINCT ON or LATERAL with LIMIT 1 per merchant.

```
CREATE INDEX idx_payments_failed_amt
ON payments (amount DESC) WHERE status = 'FAILED';

SELECT amount FROM payments
WHERE status = 'FAILED'
ORDER BY amount DESC LIMIT 1;
```

Chapter 5, Part A — ACID & Isolation Levels

Every senior backend interview in India — Razorpay, PhonePe, Cred, Swiggy, Zerodha, Flipkart — and every GCC round (JP Morgan, Goldman, Wells Fargo, Walmart Labs, Target, AmEx) eventually walks into transactions. The interviewer wants to know whether you understand what ACID actually buys you, whether you can spot the difference between a dirty read and a non-repeatable read in a two-transaction trace, and whether you know which isolation level your engine ships with by default. This is the chapter where junior candidates recite textbook definitions and lose, and where the right candidate explains trade-offs and wins. The chapter has three parts — Part A (this file) covers ACID semantics, the four isolation levels and the anomalies they prevent; Part B covers locking, MVCC and how engines actually implement isolation under the hood; Part C covers deadlocks, conflict resolution and production patterns. Read them in order — the locking discussion in B only makes sense once you've internalised the anomaly catalogue here.

Section A — ACID & Transaction Control

Q187 · What Does ACID Actually Mean

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, JP Morgan · Topic: ACID

The question (interviewer's verbatim phrasing):

Walk me through ACID. Don't recite the textbook — tell me what each letter buys me in a real payment system.

The 30-second answer: Atomicity means the transaction is all-or-nothing — either every write commits or none do. Consistency means committed data never violates the constraints you've declared. Isolation means concurrent transactions don't see each other's half-finished work. Durability means once you get a COMMIT acknowledgement, the data survives a crash.

Step-by-step thought process: 1. Frame it around a transfer of two thousand rupees from account A to B. You debit A, credit B, write a row to the **transfers** ledger. ACID is what guarantees you can never end up with A debited but B not credited. 2. Atomicity is the rollback machinery — if the credit fails for any reason, the debit is undone. 3. Consistency is the constraint enforcement layer — primary keys, foreign keys, CHECK constraints, NOT NULL all survive the commit. Note: "consistency" here does not mean "users see the same data everywhere" (that is the distributed-systems definition, a different thing). 4. Isolation is the lever you tune — the cost of perfect isolation is serial execution, so engines give you four named levels with documented trade-offs. 5. Durability is the fsync at commit time — the redo log is flushed to disk before the engine returns OK.

Edge cases to mention: - Group commit batches multiple transactions into one fsync to amortise the disk cost — that's why throughput jumps when concurrency rises. - Durability is only as good as your storage stack — write caches on consumer SSDs without battery backing have lost committed data in real outages. - ACID is per-database; once you cross a service boundary the engine cannot help you, which is why distributed transactions and the saga pattern exist.

What NOT to say: - Do not say "Consistency means the data is the same everywhere" — that's the CAP-theorem C, not the ACID C. - Do not say "ACID makes the database slow" without qualifying — Atomicity, Consistency and Durability are cheap; Isolation is where the cost lives.

Common follow-up: *"Which letter is the engine actually working hardest on?"* — Isolation, by a wide margin. The other three are essentially bookkeeping; isolation is where locks, MVCC snapshots and conflict detection burn CPU.

```
-- The classic atomicity demo
BEGIN;
  UPDATE accounts SET balance = balance - 2000 WHERE id = 101;
  UPDATE accounts SET balance = balance + 2000 WHERE id = 202;
  INSERT INTO transfers (from_id, to_id, amount) VALUES (101, 202, 2000);
COMMIT;
```

Q188 · Atomicity in Practice — Rollback and Savepoints

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Wells Fargo, Razorpay · Topic: ACID

The question (interviewer's verbatim phrasing):

What does atomicity really give you, and how do savepoints fit in?

The 30-second answer: Atomicity means the entire transaction commits or none of it does — there is no in-between state visible to other sessions. Savepoints let you mark internal checkpoints inside a transaction so you can roll back part of the work without aborting the whole thing.

Step-by-step thought process: 1. Start with the binary view — BEGIN opens a unit of work, COMMIT publishes it, ROLLBACK throws it away. 2. The engine implements this with undo logs (or MVCC versions) — until COMMIT, every change is reversible. 3. Savepoints add a tree of partial commits — `SAVEPOINT s1` marks a point, `ROLLBACK TO s1` undoes work after that point but keeps the rest, `RELEASE s1` discards the savepoint marker. 4. Real use case: a bulk import where each row should fail independently — wrap each row in a savepoint, rollback the savepoint on error, continue with the next row, commit the batch at the end.

Edge cases to mention: - A ROLLBACK to a savepoint does not release locks acquired after the savepoint in every engine — read your engine's docs. - If the transaction is aborted by the engine

(deadlock, serialization failure), all savepoints are gone — you must restart from BEGIN. - Nested transactions in most engines are simulated using savepoints — there is no true nested transaction in MySQL or Postgres.

What NOT to say: - Do not claim "atomicity prevents partial failures" — it allows partial work to happen, it just hides it from other sessions and undoes it on rollback. - Do not use savepoints inside long-running OLTP transactions casually — they keep the transaction open longer, which hurts MVCC bloat in Postgres.

Common follow-up: *"What happens to the savepoint if the network drops mid-transaction?"* — The engine eventually times out the session and rolls back everything; the savepoints die with it.

```
BEGIN;
  INSERT INTO payments (order_id, amount) VALUES (1001, 499);
  SAVEPOINT after_payment;
  UPDATE inventory SET qty = qty - 1 WHERE sku = 'BOOK-42';
  -- inventory update failed (out of stock)
  ROLLBACK TO SAVEPOINT after_payment;
  -- payment row is preserved, inventory change is gone
COMMIT;
```

Q189 · Consistency — What It Really Means

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Cred, Walmart Labs · Topic: ACID

The question (interviewer's verbatim phrasing):

The C in ACID confuses everyone. What does Consistency actually mean inside a single-node database?

The 30-second answer: Consistency in ACID means the transaction moves the database from one valid state to another, where "valid" is defined by the constraints you've declared — primary keys, foreign keys, NOT NULL, CHECK, unique indexes and any triggers. If a transaction would violate any of these, the engine aborts the COMMIT.

Step-by-step thought process: 1. The trick in this question is to distinguish ACID consistency from CAP consistency — they're unrelated concepts that share a word. 2. ACID consistency is mostly the application's responsibility — the engine enforces the constraints you declared, but it cannot know your business invariants (for example, "loyalty points should equal lifetime spend divided by ten") unless you encode them as constraints. 3. The engine's job: at COMMIT, re-check deferred constraints, validate foreign keys, run AFTER triggers. If anything fails, the COMMIT becomes a ROLLBACK. 4. The application's job: write transactions that maintain higher-level invariants the engine cannot see — for example, "the sum of all transfer rows for an account equals the account balance."

Edge cases to mention: - Deferrable constraints (Postgres) let you violate foreign keys mid-transaction and only check at COMMIT — useful for circular insert orders. - A CHECK constraint with a subquery referencing another table is not portable and is usually rejected — use a trigger instead. - Cascading deletes happen inside the transaction's atomic unit — if any cascaded delete fails, the whole transaction rolls back.

What NOT to say: - Do not say "Consistency means readers see the latest data." That is read-your-writes or strong consistency in distributed systems. - Do not say "the database enforces all business rules" — it only enforces what you've declared as constraints.

Common follow-up: *"What's the difference between an immediate and a deferred constraint?"* — Immediate is checked after every statement; deferred is checked at COMMIT. Foreign keys are commonly deferred for batch loads.

```
-- Deferred constraint pattern
BEGIN;
  SET CONSTRAINTS ALL DEFERRED;
  INSERT INTO orders (id, customer_id) VALUES (5001, 999);
  INSERT INTO customers (id, name) VALUES (999, 'Priya Sharma');
COMMIT; -- FK check happens here, both rows now exist
```

Q190 · Isolation — The Controlled Relaxation Lever

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Flipkart, Goldman · Topic: ACID

The question (interviewer's verbatim phrasing):

Why is Isolation a knob with four settings instead of just being "on"?

The 30-second answer: Perfect isolation — Serializable — requires either heavy locking or expensive conflict detection, and on a busy OLTP system that's the difference between a thousand and ten thousand transactions per second. The four named isolation levels are graduated trade-offs: each weaker level forbids fewer anomalies but allows more concurrency.

Step-by-step thought process: 1. Frame it as a market: the engine sells you isolation in exchange for throughput, and you pick how much you can afford. 2. Anomalies are the unit of trade — dirty reads, non-repeatable reads, phantom reads, lost updates, write skew. Each level outlaws a specific subset. 3. Stronger levels need more bookkeeping: Read Committed needs only short read locks (or a per-statement snapshot); Repeatable Read needs a snapshot for the whole transaction; Serializable needs full conflict detection. 4. Most production systems run at Read Committed (the Postgres default) for OLTP and step up to Repeatable Read or Serializable only for specific transactions that need it.

Edge cases to mention: - The SQL standard defines four levels but leaves implementation open — Postgres "Read Uncommitted" is silently treated as Read Committed because MVCC makes dirty reads impossible. - MySQL InnoDB defaults to Repeatable Read with gap locks, which also prevents phantoms — stricter than the SQL standard requires. - Oracle and SQL Server (with READ_COMMITTED_SNAPSHOT on) use snapshot isolation under the Read Committed label.

What NOT to say: - Do not say "higher isolation is always safer, use Serializable" — on a busy system you will deadlock or hit serialization failures constantly. - Do not confuse the SQL standard isolation level with what your engine actually delivers — read your engine's documentation.

Common follow-up: *"If Serializable is correct, why is anything else allowed?"* — Throughput. A bank ledger that does ten thousand transfers per second cannot afford Serializable's conflict-detection overhead on every read.

```
-- Setting isolation for a single transaction
BEGIN;
  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  SELECT balance FROM accounts WHERE id = 101;
  -- ... rest of the transaction
COMMIT;
```

Q191 · Durability — fsync, Redo Logs and Group Commit

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, JP Morgan, Zerodha · Topic: ACID

The question (interviewer's verbatim phrasing):

Walk me through what actually happens between my COMMIT and the engine returning OK.

The 30-second answer: The engine writes the change to an in-memory log buffer, flushes the log to disk with fsync (or O_DIRECT), waits for the storage acknowledgement, then returns OK to your client. The actual data pages may not be written yet — only the redo log entry has to be durable, because crash recovery replays the log to rebuild the pages.

Step-by-step thought process: 1. Start with WAL — Write-Ahead Logging is the universal pattern. Every change is first appended to a sequential log file. 2. COMMIT writes a COMMIT record to the log and forces fsync on the log file. That's the durability boundary. 3. Data pages get written later by the background writer or checkpoint. If the engine crashes between COMMIT and data-page flush, recovery reads the WAL and reapplies the changes. 4. Group commit batches many concurrent COMMITs into one fsync — instead of one fsync per transaction, you get one fsync per batch, which dramatically improves throughput on busy systems.

Edge cases to mention: - `synchronous_commit = off` in Postgres trades a tiny window of durability loss for huge throughput gains — useful for analytics inserts where losing the last few seconds is acceptable. - Consumer SSDs with write caches that ignore fsync have lost committed data in real outages — datacentre-grade drives with power-loss protection are essential. - Replication adds a second durability layer — synchronous replication waits for a replica to acknowledge before returning OK, so a single-machine failure does not lose data.

What NOT to say: - Do not say "COMMIT writes to the data file" — it writes to the log; the data file is written asynchronously. - Do not assume fsync always actually flushes — the kernel, the controller and the drive can all lie. Test your stack.

Common follow-up: *"What is the trade-off of synchronous_commit off?"* — You lose up to `wal_writer_delay` worth of committed transactions on a crash. The database stays consistent, but recent commits vanish.

```
-- Postgres tuning knob for durability vs throughput
SET LOCAL synchronous_commit = off; -- for this transaction only
INSERT INTO event_log (event_type, payload) VALUES ('clicked', '{}');
COMMIT;
```

Q192 · BEGIN, COMMIT, ROLLBACK — The Basics Done Right

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Transactions

The question (interviewer's verbatim phrasing):

Walk me through BEGIN, COMMIT and ROLLBACK and tell me what happens if I forget to write COMMIT.

The 30-second answer: BEGIN (or START TRANSACTION) opens a unit of work, COMMIT makes the changes visible and durable, ROLLBACK throws away the changes. If you forget COMMIT and the session closes, the engine rolls back the transaction automatically — your changes are lost.

Step-by-step thought process: 1. State the three-step lifecycle clearly. Most engines treat BEGIN, START TRANSACTION and BEGIN TRANSACTION as synonyms. 2. Explain implicit vs explicit transactions: in autocommit mode (the default for most clients), every statement is its own transaction. BEGIN turns autocommit off until the next COMMIT or ROLLBACK. 3. Mention the connection-pool gotcha: if a transaction is left open when the connection returns to the pool, the next user inherits it — explicitly ROLLBACK in your `finally` block. 4. Be ready for the language follow-up: ANSI SQL says START TRANSACTION; MySQL, Postgres and SQL Server all accept BEGIN as a shortcut.

Edge cases to mention: - A DDL statement in MySQL (CREATE TABLE, ALTER TABLE) implicitly commits the current transaction — your previous DML is now durable whether you wanted it or not. - Postgres allows DDL inside transactions and rolls it back cleanly — useful for migrations. - Some clients (especially psql in pure interactive mode) will leave a failed transaction in an "aborted" state where every subsequent statement errors until you ROLLBACK.

What NOT to say: - Do not say "every query is a transaction" — autocommit makes it look that way, but explicit BEGIN changes the semantics. - Do not mix BEGIN with COMMIT WORK and SET TRANSACTION carelessly — read your engine's docs on which combinations actually work together.

Common follow-up: *"What happens if the connection drops mid-transaction?"* — The engine eventually times out the session and rolls back. Any locks held are released.

```
BEGIN;  
  UPDATE accounts SET balance = balance - 500 WHERE id = 1;  
  UPDATE accounts SET balance = balance + 500 WHERE id = 2;  
  -- error in application code here  
ROLLBACK; -- both updates undone
```

Q193 · Implicit vs Explicit Transactions

Tags: Difficulty: Easy · Asked at: Accenture, Cognizant, TCS, Wells Fargo · Topic: Transactions

The question (interviewer's verbatim phrasing):

What's the difference between an implicit and an explicit transaction, and which one does your JDBC driver use by default?

The 30-second answer: Implicit (autocommit) transactions wrap every single statement in its own transaction that commits automatically when the statement succeeds. Explicit transactions are bracketed by BEGIN and COMMIT and let you group multiple statements as one atomic unit. JDBC defaults to autocommit on; you must call `setAutoCommit(false)` to use explicit transactions.

Step-by-step thought process: 1. Explain autocommit first — it's the simpler model. Every INSERT, UPDATE, DELETE is committed before control returns to the application. 2. The cost of autocommit: each statement does its own fsync, so doing a thousand single-row inserts in a tight loop is hundreds of times slower than batching them inside one explicit transaction. 3. Explicit transactions are mandatory whenever you need atomicity across more than one statement — transfers, multi-row updates with consistency requirements, anything with savepoints. 4. The JDBC/ORM trap: connection pools usually reset autocommit between checkouts, but if your ORM (Hibernate, JPA, SQLAlchemy) starts a transaction lazily, you may be inside an implicit transaction without realising it.

Edge cases to mention: - MySQL's `autocommit` is a session variable that can be set per-connection or globally — check `SHOW VARIABLES LIKE 'autocommit'` if behaviour surprises you. - Postgres has no global autocommit toggle — it's a client-side concept; psql defaults to autocommit on, JDBC the same. - Spring's `@Transactional` annotation opens an explicit transaction; if you forget the annotation, every JPA save is a separate autocommit transaction.

What NOT to say: - Do not say "autocommit is bad, always use explicit transactions" — for single-statement OLTP work, autocommit is correct and simpler. - Do not assume your ORM has wrapped everything in a transaction — verify.

Common follow-up: *"What happens to autocommit when you call BEGIN?"* — It's suspended until COMMIT or ROLLBACK, then resumes.

```
-- MySQL autocommit demonstration
SET autocommit = 0;
INSERT INTO orders (id, amount) VALUES (1, 100);
INSERT INTO orders (id, amount) VALUES (2, 200);
COMMIT; -- both inserts published together
SET autocommit = 1;
```

Section B — The Four Isolation Levels

Q194 · Overview of the Four Isolation Levels

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, JP Morgan, Goldman · Topic: Isolation

The question (interviewer's verbatim phrasing):

Give me the four standard isolation levels and tell me which anomaly each one prevents.

The 30-second answer: Read Uncommitted allows dirty reads, non-repeatable reads and phantom reads. Read Committed forbids dirty reads but allows non-repeatable and phantom. Repeatable Read also forbids non-repeatable reads, allowing only phantoms. Serializable forbids all three plus write skew.

Step-by-step thought process: 1. Draw the standard SQL grid mentally — four levels, three classic anomalies, one cell per combination. 2. Read Uncommitted: no protection — a transaction can read uncommitted changes from another. Rarely useful. 3. Read Committed: a query sees only committed data, but two queries in the same transaction can see different snapshots — that's a non-repeatable read. 4. Repeatable Read: every query in the transaction sees the same snapshot of existing rows, but newly inserted rows from other transactions might appear — that's a phantom. 5. Serializable: the result must be equivalent to some serial ordering of all concurrent transactions — no phantoms, no write skew.

Edge cases to mention: - The SQL standard's grid is conservative — actual engines often deliver more protection than required (Postgres's Repeatable Read prevents phantoms because of MVCC, MySQL InnoDB's Repeatable Read prevents phantoms via gap locks). - Write skew is not in the SQL standard's anomaly list — it's an additional anomaly that only Serializable prevents, recognised after the standard was published. - The "Read Uncommitted" level in Postgres is treated as Read Committed because MVCC has no way to expose dirty data.

What NOT to say: - Do not memorise the grid without understanding what each anomaly looks like in concrete code — interviewers will ask you to construct one. - Do not assume the level names mean the same thing across engines — Oracle's Serializable is actually snapshot isolation, not true serializable.

Common follow-up: "Draw me the grid on the whiteboard." — Be ready to do this from memory.

Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible*
Serializable	Prevented	Prevented	Prevented

* Prevented in MySQL InnoDB and Postgres due to implementation details.

Q195 · Read Uncommitted — When (Almost Never) to Use It

Tags: Difficulty: Easy · Asked at: Cognizant, Accenture, Walmart Labs · Topic: Isolation

The question (interviewer's verbatim phrasing):

What does Read Uncommitted allow, and is there any sane reason to choose it?

The 30-second answer: Read Uncommitted lets one transaction see changes made by another transaction before that other transaction has committed — a dirty read. You almost never want this; the only honest use case is approximate reporting on a hot table where you don't care if you read data that will be rolled back a millisecond later.

Step-by-step thought process: 1. Explain the mechanism: at this level, the engine does no read locking at all and ignores the visibility rules. 2. Walk through the failure mode: transaction A debits an account by ten thousand rupees and then rolls back. Transaction B, running at Read Uncommitted, saw the debited balance and made a decision on it — that decision is now based on data that does not exist. 3. The

almost-acceptable use case: a dashboard counting approximate row counts on a high-write table, where being off by a few hundred rows is fine and you want to avoid any lock contention. 4. Reality check: in Postgres this level is silently upgraded to Read Committed because MVCC cannot return uncommitted data. In SQL Server, the `WITH (NOLOCK)` hint is the per-table equivalent and is widely (mis)used for the same reason.

Edge cases to mention: - `WITH (NOLOCK)` in SQL Server can return rows twice, skip rows entirely, or return rows with internally inconsistent column values during page splits — it's not a free performance win. - Read Uncommitted does not eliminate all overhead — internal latches still apply, just not transaction-level read locks. - In MySQL, Read Uncommitted is supported and does what you'd expect, but the InnoDB engine still has consistency checks that may produce surprising results.

What NOT to say: - Do not call Read Uncommitted "fast" — it's marginally faster only in lock-based engines, and in MVCC engines it's irrelevant. - Do not recommend it for any transaction that makes a decision based on the data — you can act on data that never existed.

Common follow-up: *"Why is NOLOCK so popular if it's so dangerous?"* — Habit, cargo-culted advice from old DBAs, and the fact that the failures are silent until they're not.

```
-- The dashboard pattern – approximate count, no lock contention
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT COUNT(*) FROM event_log WHERE created_at > NOW() - INTERVAL '1 hour';
```

Q196 · Read Committed — What It Prevents and What It Doesn't

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Goldman, JP Morgan · Topic: Isolation

The question (interviewer's verbatim phrasing):

Read Committed is the Postgres default. What does it actually guarantee, and where does it fall short?

The 30-second answer: Read Committed guarantees that every query sees only data that was committed at the time the query began executing. It prevents dirty reads but allows non-repeatable reads — two `SELECT` statements in the same transaction can see different values for the same row if another transaction commits a change between them.

Step-by-step thought process: 1. Frame the visibility rule: each statement gets its own fresh snapshot. The transaction does not have a single consistent view of the database. 2. Walk through the implication: if you `SELECT` a balance, then later in the same transaction `SELECT` it again, you might get different values. For reporting queries that read multiple tables, sums can be incorrect. 3. State why it's still the default: it's cheap (no long-held locks, no per-transaction snapshot), it covers the most common failure

mode (dirty reads), and most OLTP transactions are short enough that anomalies are unlikely. 4. The fix: if you need a consistent view across multiple statements, escalate to Repeatable Read for that transaction.

Edge cases to mention: - Even at Read Committed, an UPDATE on a row that's being modified by another transaction will block until that transaction commits or rolls back — write locks are still respected. - Postgres Read Committed uses a fresh MVCC snapshot per statement; SQL Server (without READ_COMMITTED_SNAPSHOT) uses short read locks released after each statement. - A SELECT FOR UPDATE inside Read Committed locks the rows you read — useful when you intend to update them next.

What NOT to say: - Do not say "Read Committed means I see consistent data" — between statements within the same transaction, you do not. - Do not assume Read Committed prevents lost updates — it does not. A read-modify-write loop at Read Committed can lose updates.

Common follow-up: "Give me a concrete example of a non-repeatable read at this level." — See Q14.

```
-- Read Committed: snapshot per statement
BEGIN; -- default isolation in Postgres
  SELECT balance FROM accounts WHERE id = 101; -- returns 5000
  -- another transaction commits an update to balance
  SELECT balance FROM accounts WHERE id = 101; -- returns 4000
COMMIT;
```

Q197 · Repeatable Read — Snapshots and Phantom Limits

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, Walmart Labs, Goldman · Topic: Isolation

The question (interviewer's verbatim phrasing):

What does Repeatable Read guarantee, and what's the difference between Postgres and MySQL InnoDB at this level?

The 30-second answer: Repeatable Read freezes a transaction-level snapshot of all rows that exist when the transaction starts — any subsequent SELECT in the same transaction sees the same values for those rows. The SQL standard says phantoms (newly inserted rows appearing in a re-run query) are still allowed, but Postgres prevents them via MVCC and MySQL InnoDB prevents them via gap locks.

Step-by-step thought process: 1. Explain the snapshot mechanism: at the first read, the engine pins a "logical clock" value, and every subsequent read filters out rows committed after that point. 2. Result: non-repeatable reads disappear — the same SELECT always returns the same data within the transaction. 3. The phantom hole in the standard: the standard only protects rows that already exist; new INSERTs from other transactions can show up in a range scan re-run. Real engines often patch this. 4. Postgres at Repeatable Read uses MVCC for everything — phantoms are naturally prevented. MySQL

InnoDB uses gap locks on index ranges to block other transactions from inserting into the scanned range.

Edge cases to mention: - Postgres Repeatable Read can fail at COMMIT with a serialization error if a concurrent transaction modified a row you wrote — be ready to retry. - MySQL InnoDB gap locks can cause surprising deadlocks because they lock ranges that don't contain any rows yet — a common production issue. - Repeatable Read does not protect against write skew — that requires Serializable.

What NOT to say: - Do not say "Repeatable Read is the same in every engine" — Postgres and MySQL InnoDB behave very differently underneath. - Do not assume Repeatable Read solves all read consistency problems — write skew is the famous exception.

Common follow-up: *"Why does MySQL InnoDB default to Repeatable Read while Postgres defaults to Read Committed?"* — Historical reasons; MySQL InnoDB's gap-lock implementation makes Repeatable Read cheap enough to be the default, while Postgres prioritises maximum concurrency at Read Committed.

```
-- Repeatable Read: stable view across the whole transaction
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id = 101; -- 5000
-- another transaction commits an update
SELECT balance FROM accounts WHERE id = 101; -- still 5000
COMMIT;
```

Q198 · Serializable — The Strongest Level and Its Implementations

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Cred, Razorpay, Zerodha · Topic: Isolation

The question (interviewer's verbatim phrasing):

What does Serializable guarantee, and how do Postgres and MySQL InnoDB actually implement it differently?

The 30-second answer: Serializable guarantees that the result of running concurrent transactions is equivalent to some serial ordering of them — no anomaly is possible. Postgres uses Serializable Snapshot Isolation (SSI), which extends snapshot isolation with conflict detection. MySQL InnoDB uses strict two-phase locking, taking shared locks on every read and exclusive locks on writes.

Step-by-step thought process: 1. State the guarantee in plain terms: if your transaction commits, the database looks as if all concurrent transactions ran one after the other in some valid order. 2. Lock-based implementation (SQL Server, MySQL InnoDB, DB2): take shared read locks and exclusive write locks, hold them until commit. Prevents anomalies but reduces concurrency and causes deadlocks. 3. SSI

implementation (Postgres 9.1+): use snapshot isolation for reads (no locks), but track read-write dependencies between transactions. If a dangerous cycle is detected, abort one transaction with a serialization error. 4. Trade-off: lock-based blocks more often but completes when it doesn't deadlock; SSI runs full speed but throws more retryable errors at COMMIT time.

Edge cases to mention: - Serializable always requires the application to be ready to retry — both lock-based and SSI implementations can abort transactions for correctness. - Postgres SSI has lower throughput than Repeatable Read because of the dependency tracking overhead, but higher than lock-based Serializable on read-heavy workloads. - Oracle and MySQL marketing sometimes calls snapshot isolation "Serializable" when it's actually weaker — read the docs.

What NOT to say: - Do not say "Serializable is the safest, use it always" — the abort rate on busy workloads will hurt; pick it transaction-by-transaction. - Do not confuse Serializable with snapshot isolation — write skew distinguishes them.

Common follow-up: "What's the retry pattern for a Serializable transaction?" — Catch the serialization failure error (SQLSTATE 40001), wait a short randomised time, restart the transaction from BEGIN, repeat with a retry limit.

```
-- Postgres Serializable with retry-on-error in pseudocode
BEGIN;
  SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
  -- read balances, validate invariant, update
  COMMIT; -- may raise 40001 serialization_failure - retry from BEGIN
```

Section C — Anomalies in Detail

Q199 · Dirty Read — Concrete Two-Transaction Example

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture, Cognizant · Topic: Anomalies

The question (interviewer's verbatim phrasing):

Show me a two-transaction trace that produces a dirty read.

The 30-second answer: Transaction A updates a row but has not committed. Transaction B, running at Read Uncommitted, reads the updated value. A then rolls back. B has now read a value that never existed in any committed state — the dirty read.

Step-by-step thought process: 1. Set up the scene: account 101 has balance 5000. 2. Transaction A starts and runs `UPDATE accounts SET balance = balance - 2000 WHERE id = 101`. The change is in A's private workspace, not yet committed. 3. Transaction B (at Read Uncommitted) runs

`SELECT balance FROM accounts WHERE id = 101`. It sees 3000 because Read Uncommitted ignores visibility rules. 4. Transaction A hits an error and runs ROLLBACK. The balance is back to 5000. 5. Transaction B has made a decision based on a value (3000) that was never durably true. If B sent the user an "insufficient funds" rejection or wrote a derived value to another table, the system is now in an incorrect state.

Edge cases to mention: - In Postgres, dirty reads are impossible at any isolation level because MVCC only exposes committed versions — even `SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED` gets you Read Committed semantics. - SQL Server's `WITH (NOLOCK)` hint is the per-query equivalent and allows dirty reads even when the session isolation is higher. - The "phantom of a dirty read" — reading a row that exists in a transaction's workspace but never gets committed — is the worst kind because the decision is irreversible.

What NOT to say: - Do not say "dirty reads are fine for analytics" — even analytics can produce wrong totals if a million-row insert rolls back. - Do not confuse dirty reads with non-repeatable reads — dirty reads see uncommitted data, non-repeatable reads see two different committed values.

Common follow-up: *"Does Postgres allow dirty reads under any setting?"* — No. The architecture makes them impossible.

```
-- Session A
BEGIN;
UPDATE accounts SET balance = balance - 2000 WHERE id = 101;
-- (do not commit yet)

-- Session B at Read Uncommitted (e.g. MySQL or SQL Server)
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT balance FROM accounts WHERE id = 101; -- sees 3000

-- Session A
ROLLBACK; -- balance is back to 5000, but B already acted on 3000
```

Q200 · Non-Repeatable Read — Concrete Example

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, JP Morgan, Walmart Labs · Topic: Anomalies

The question (interviewer's verbatim phrasing):

Show me a non-repeatable read at the Read Committed level.

The 30-second answer: Transaction A reads a row, transaction B updates and commits that row, transaction A reads the same row again and sees the new value. Two reads of the same row in the same transaction returned different values — that's the non-repeatable read.

Step-by-step thought process: 1. Setup: account 101 has balance 5000. Transaction A is a reporting transaction at Read Committed that needs to fetch the balance twice (once for display, once for calculation). 2. A's first read: 5000. 3. Transaction B opens, updates balance to 4000, commits. 4. A's second read: 4000. 5. A now has two inconsistent numbers in its internal state — if it was computing "balance minus pending withdrawals" and used the first balance with the latest withdrawal list, the math is wrong.

Edge cases to mention: - The fix is Repeatable Read — every read in A sees the same snapshot. - The pure read case is annoying but rarely catastrophic; the dangerous case is when A reads, makes a decision, and writes back, because that's the lost-update scenario. - An equivalent in MySQL with autocommit and no explicit BEGIN is impossible to demonstrate — autocommit ends each statement's transaction immediately, so each SELECT is a new transaction with its own snapshot semantics.

What NOT to say: - Do not say "non-repeatable reads are a Postgres bug" — they're explicitly allowed at Read Committed in the SQL standard. - Do not confuse with phantom reads — non-repeatable is about existing rows changing value; phantoms are about new rows appearing.

Common follow-up: *"How do I prevent this in Postgres?"* — Set the transaction to REPEATABLE READ or SERIALIZABLE.

```
-- Session A at Read Committed
BEGIN;
SELECT balance FROM accounts WHERE id = 101; -- 5000

-- Session B (separate connection)
BEGIN;
UPDATE accounts SET balance = 4000 WHERE id = 101;
COMMIT;

-- Session A continues
SELECT balance FROM accounts WHERE id = 101; -- 4000 - non-repeatable read
COMMIT;
```

Q201 · Phantom Read — Concrete Example

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Razorpay, Flipkart · Topic: Anomalies

The question (interviewer's verbatim phrasing):

Show me a phantom read. What's the difference from a non-repeatable read?

The 30-second answer: Transaction A runs a range query like `SELECT * FROM transfers WHERE amount > 10000`, transaction B inserts a new row matching that predicate and commits, transaction A

runs the same range query and sees the new row. The new row is the phantom. Non-repeatable reads change existing rows; phantoms add or remove rows from a result set.

Step-by-step thought process: 1. Setup: a treasury reconciliation transaction wants to find all transfers above ten thousand rupees today. 2. A's first range query: returns three rows. 3. B inserts a new transfer for fifteen thousand rupees and commits. 4. A's second range query: returns four rows. 5. The new fourth row is a phantom — A's view of the predicate has changed because new rows materialised under it.

Edge cases to mention: - The SQL standard says Repeatable Read does not prevent phantoms; this is the most famous gap in the standard. - Postgres Repeatable Read prevents phantoms because the snapshot includes the entire visibility state at the start of the transaction — new rows from other transactions are invisible. - MySQL InnoDB at Repeatable Read prevents phantoms via gap locks: when A's range query runs, it takes locks on the gaps in the index range, blocking B's INSERT until A commits. - Serializable always prevents phantoms by definition.

What NOT to say: - Do not say "phantoms only happen at Read Committed" — the standard allows them at Repeatable Read as well. - Do not assume your engine follows the standard literally — test it.

Common follow-up: *"What's the difference between a phantom read and a lost update?"* — Phantoms are about reads seeing new rows; lost updates are about two writers overwriting each other.

```
-- Session A
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT * FROM transfers WHERE amount > 10000 AND created_on = CURRENT_DATE;
-- 3 rows

-- Session B inserts a new high-value transfer and commits

-- Session A in MySQL InnoDB or Postgres at Repeatable Read
SELECT * FROM transfers WHERE amount > 10000 AND created_on = CURRENT_DATE;
-- Postgres: still 3 rows. SQL-standard-only engine: 4 rows (phantom).
COMMIT;
```

Q202 · Lost Update — How Isolation and Locking Prevent It

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Goldman, Walmart Labs, Cred · Topic: Anomalies

The question (interviewer's verbatim phrasing):

Two transactions both want to increment a counter. Walk me through how a lost update happens and three ways to prevent it.

The 30-second answer: A reads the counter, B reads the counter, A increments and writes, B increments and writes — B overwrites A's update with a value computed from the old read, and A's increment is lost. Prevent it with (1) atomic updates that don't require reading first, (2) SELECT FOR UPDATE to lock the row before reading, or (3) optimistic concurrency with a version column.

Step-by-step thought process: 1. Walk the failure: counter starts at 100. A reads 100, B reads 100, A writes 101, B writes 101. Final value is 101, but both transactions intended to increment, so it should be 102. 2. Fix one: replace the read-modify-write with `UPDATE counters SET value = value + 1 WHERE id = X` — the engine does the read and write under one lock, so concurrent updates are serialised. 3. Fix two: `SELECT value FROM counters WHERE id = X FOR UPDATE` takes a row lock that blocks the second reader until the first transaction commits. 4. Fix three: add a `version` column, read `(value, version)`, write `UPDATE counters SET value = ?, version = version + 1 WHERE id = X AND version = ?`. If the row count is zero, someone else updated first — retry.

Edge cases to mention: - At Repeatable Read in Postgres, a lost-update attempt raises a serialization error at the second write — Postgres detects the conflict and aborts one transaction. At Read Committed, it does not. - At Repeatable Read in MySQL InnoDB, the second UPDATE blocks until the first commits, then proceeds — no abort, but the read value is stale. You must re-read or use SELECT FOR UPDATE. - Optimistic concurrency works well when conflicts are rare; pessimistic locking is better when they're common.

What NOT to say: - Do not assume that "running at Repeatable Read prevents lost updates everywhere" — Postgres and MySQL InnoDB behave differently. - Do not solve a lost update by raising isolation level alone in MySQL InnoDB — you also need explicit locking or retry logic.

Common follow-up: *"Which approach do payment systems use?"* — Atomic UPDATES for counters; SELECT FOR UPDATE for the account balance row during a transfer; optimistic concurrency in user-facing screens where the row is held while the user types.

```
-- Atomic, no lost update possible
UPDATE counters SET value = value + 1 WHERE id = 'orders_placed';

-- Pessimistic, with read-then-write pattern
BEGIN;
SELECT balance FROM accounts WHERE id = 101 FOR UPDATE;
-- compute new balance in application
UPDATE accounts SET balance = ? WHERE id = 101;
COMMIT;

-- Optimistic
SELECT balance, version FROM accounts WHERE id = 101;
UPDATE accounts SET balance = ?, version = version + 1
  WHERE id = 101 AND version = ?; -- rowcount = 0 means retry
```

Q203 · Write Skew — The Serializable-Only Anomaly

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Cred, Razorpay · Topic: Anomalies

The question (interviewer's verbatim phrasing):

What is write skew, and why does Snapshot Isolation allow it while Serializable does not?

The 30-second answer: Write skew is an anomaly where two transactions read overlapping data, each makes an update based on what they read, and the combined effect violates an invariant that each transaction individually preserved. Snapshot Isolation can't see this because each transaction's snapshot looks correct in isolation; Serializable detects the read-write conflict across transactions and aborts one.

Step-by-step thought process: 1. Classic example: an on-call rota. Rule: at least one doctor must be on call. Two doctors, Asha and Vikram, are both on call. Both try to mark themselves off-call simultaneously. 2. Asha's transaction reads "two doctors on call, I can step off" and updates her row to off-call. 3. Vikram's transaction reads "two doctors on call, I can step off" (same snapshot) and updates his row to off-call. 4. Both commit. Now zero doctors are on call — the invariant is violated even though each transaction's snapshot showed two doctors when it made its decision. 5. Serializable catches this by tracking that both transactions read the row they didn't write — a read-write dependency cycle — and aborts one.

Edge cases to mention: - Snapshot Isolation (Oracle, MySQL InnoDB at Repeatable Read with appropriate locking, Postgres at Repeatable Read) allows write skew. - Postgres SSI at Serializable detects write skew via dependency tracking and raises a serialization error. - The cheap fix without Serializable: explicit locking with `SELECT FOR UPDATE` on the rows being checked, or a constraint that encodes the invariant directly.

What NOT to say: - Do not say "write skew is impossible if both transactions read the same data" — that's exactly when it happens. - Do not confuse with lost updates — lost updates overwrite the same cell; write skew involves disjoint cells with a shared invariant.

Common follow-up: "How would you fix the doctor-on-call problem without Serializable?" — `SELECT count(*) FROM doctors WHERE on_call = true FOR UPDATE` before the `UPDATE` — the lock serialises the two transactions.

```
-- Both transactions run at Repeatable Read
-- T1
BEGIN;
SELECT count(*) FROM doctors WHERE on_call = TRUE; -- 2
UPDATE doctors SET on_call = FALSE WHERE id = 1;
COMMIT;
-- T2 (concurrent)
BEGIN;
SELECT count(*) FROM doctors WHERE on_call = TRUE; -- 2 (snapshot)
UPDATE doctors SET on_call = FALSE WHERE id = 2;
COMMIT;
-- Result: count = 0, invariant violated
```

Q204 · Read Skew — Same Family, Different Symptom

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, JP Morgan · Topic: Anomalies

The question (interviewer's verbatim phrasing):

What is read skew, and how does it differ from a non-repeatable read?

The 30-second answer: Read skew is when a single transaction reads two related rows and sees an inconsistent state because another transaction modified one of them between the reads. It's effectively a non-repeatable read across two different rows that should sum or balance together — the inconsistency is in the relationship, not in any single row.

Step-by-step thought process: 1. Setup: a customer has two accounts, savings and current, with balances 5000 and 3000, total 8000. The system invariant is that the total is preserved when money is moved between them. 2. Reporting transaction R at Read Committed reads savings: 5000. 3. Transfer transaction T moves 2000 from savings to current and commits. 4. R reads current: 5000 (2000 plus 3000). 5. R now reports total 10000 instead of 8000 — the relationship between the two reads is wrong even though each individual read returned a value that was committed at some point.

Edge cases to mention: - Read skew is prevented at Repeatable Read in Postgres and at Repeatable Read in MySQL InnoDB because both reads see the same snapshot. - This is the classic argument for running reporting and consistency-check transactions at Repeatable Read or Serializable even when the rest of the system runs at Read Committed. - Read skew is sometimes called "inconsistent read" in older textbooks.

What NOT to say: - Do not confuse with non-repeatable read on the same row — read skew is about two different rows whose values are related. - Do not assume reporting tools are immune — long-running aggregations are particularly vulnerable.

Common follow-up: "What's the cheapest fix for read skew in a Postgres reporting query?" — Wrap the query in a Repeatable Read transaction; the snapshot is consistent.

```
-- Repeatable Read fix for read skew
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE customer_id = 999 AND type = 'savings';
-- another transaction commits a transfer between accounts
SELECT balance FROM accounts WHERE customer_id = 999 AND type = 'current';
-- both reads come from the same snapshot, totals are consistent
COMMIT;
```

Q205 · Snapshot Isolation — Vs Serializable

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Cred, PhonePe · Topic: Isolation

The question (interviewer's verbatim phrasing):

What is Snapshot Isolation, and how is it different from Serializable?

The 30-second answer: Snapshot Isolation gives every transaction a consistent point-in-time view of the database — reads see committed data as of the transaction's start, writes only conflict with concurrent writes to the same rows. It prevents dirty, non-repeatable and phantom reads, but it allows write skew. Serializable also prevents write skew by tracking read-write dependencies across transactions.

Step-by-step thought process: 1. Snapshot Isolation is the implementation Postgres uses for Repeatable Read, MySQL InnoDB uses with MVCC, Oracle uses for both Read Committed and Serializable, and SQL Server uses when READ_COMMITTED_SNAPSHOT is on. 2. Mechanism: when a transaction starts, the engine records the current commit sequence number. Reads filter out any row version committed after that number. 3. Write conflicts: if your transaction tries to update a row that another transaction has already updated since your snapshot, the engine raises a write conflict — you abort and retry. 4. The famous gap: Snapshot Isolation does not detect read-only conflicts that lead to write skew. Serializable Snapshot Isolation (SSI) extends snapshot isolation with read tracking.

Edge cases to mention: - Oracle markets its Snapshot Isolation as "Serializable" — it's not the same as true Serializable; write skew is possible. - Postgres calls its level "Repeatable Read" but it's exactly Snapshot Isolation behaviour, stricter than the SQL standard requires. - Snapshot Isolation throughput is close to Read Committed because reads are not blocked; the cost is the MVCC version chain maintenance.

What NOT to say: - Do not say "Snapshot Isolation is the same as Serializable" — write skew is the counter-example. - Do not say "Oracle's Serializable is broken" — it's documented as Snapshot Isolation; the naming is misleading but not incorrect by Oracle's definition.

Common follow-up: "*When is Snapshot Isolation good enough?*" — When your invariants are local to single rows or are enforced by constraints. When you have multi-row invariants (the on-call rota, account totals), step up to Serializable.

```
-- Snapshot Isolation in Postgres (called Repeatable Read)
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
-- All reads see the database as of this point
-- Writes that conflict with concurrent updates raise serialization_failure
COMMIT;
```

Q206 · Serializable Snapshot Isolation (SSI) — Postgres

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Cred, Razorpay · Topic: Isolation

The question (interviewer's verbatim phrasing):

Walk me through how Postgres implements Serializable. Why is it called Serializable Snapshot Isolation?

The 30-second answer: Postgres at Serializable runs every transaction at Snapshot Isolation and additionally tracks the read and write sets of each transaction. When it detects a pattern of dependencies that could produce a non-serializable outcome — specifically a read-write cycle — it aborts one of the transactions with a serialization failure. The application is expected to retry.

Step-by-step thought process: 1. Background: lock-based Serializable (used by SQL Server and MySQL InnoDB at this level) takes shared read locks on every row read and holds them to commit. It works but blocks readers behind writers. 2. SSI's idea: don't take read locks. Instead, record what each transaction read. At commit time, check whether the read-write dependency graph contains a "dangerous structure" — two consecutive read-write edges forming a cycle. 3. If a dangerous structure is found, abort one of the participating transactions. The other commits successfully. 4. Trade-off: SSI does not block readers, so throughput is much higher than lock-based Serializable on read-heavy workloads. The cost is more retries when conflicts happen.

Edge cases to mention: - The application must catch SQLSTATE 40001 (serialization_failure) and retry — without retry logic, your Serializable code fails under load. - SSI has higher overhead than Repeatable Read because of the dependency tracking; on a workload that doesn't need Serializable, prefer

Repeatable Read. - DECLARE CURSOR queries and certain index access patterns can cause more false-positive aborts; Postgres documentation lists known cases.

What NOT to say: - Do not say "SSI is just snapshot isolation with extra steps" — the extra steps are the difference between correct and incorrect for write-skew-prone workloads. - Do not assume SSI prevents all retries — it can detect more false positives than lock-based Serializable because dependency analysis is approximate.

Common follow-up: "What's the typical retry pattern?" — `BEGIN; ... COMMIT;` wrapped in a loop, catching SQLSTATE 40001, exponential backoff with jitter, max five retries before surfacing the error to the user.

```
-- Pseudocode for SSI retry
for attempt in 1..5:
  try:
    BEGIN;
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    -- do the work
    COMMIT;
    break;
  except SerializationFailure (SQLSTATE 40001):
    ROLLBACK;
    sleep(random(0, 0.1 * 2^attempt));
```

Section D — Engine Defaults and Configuration

Q207 · Engine Defaults — The Cheat Sheet

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, JP Morgan, Goldman · Topic: Isolation

The question (interviewer's verbatim phrasing):

What's the default isolation level in Postgres, MySQL InnoDB, SQL Server and Oracle?

The 30-second answer: Postgres defaults to Read Committed. MySQL InnoDB defaults to Repeatable Read (and prevents phantoms via gap locks). SQL Server defaults to Read Committed. Oracle defaults to Read Committed (and offers Serializable, which is actually Snapshot Isolation).

Step-by-step thought process: 1. Postgres Read Committed: every statement sees a fresh MVCC snapshot. Dirty reads are impossible (MVCC), but non-repeatable and phantom reads are allowed. 2. MySQL InnoDB Repeatable Read: every transaction sees a transaction-level snapshot. Phantoms are blocked by gap locks on index ranges scanned by SELECT. 3. SQL Server Read Committed: by default

uses short read locks. With `READ_COMMITTED_SNAPSHOT = ON` (recommended in modern setups), it switches to snapshot-based reads and behaves like Postgres Read Committed. 4. Oracle Read Committed: snapshot-based, behaves like Postgres Read Committed. Oracle's "Serializable" is actually Snapshot Isolation.

Edge cases to mention: - Cloud-managed engines may override defaults — Amazon RDS for SQL Server enables `READ_COMMITTED_SNAPSHOT` by default; Aurora Postgres matches stock Postgres. - Migrating from MySQL to Postgres without changing application code is a common source of surprise — the same code runs at Repeatable Read in MySQL but Read Committed in Postgres, exposing anomalies that were hidden. - ORMs often have a per-session or per-transaction isolation setting that overrides the engine default — check your Hibernate or SQLAlchemy configuration.

What NOT to say: - Do not say "all major engines default to Read Committed" — MySQL InnoDB is the exception. - Do not assume engine names mean the same behaviour — read the docs for the specific version.

Common follow-up: "*Which one is right?*" — Read Committed is the better default for general OLTP because it maximises concurrency; Repeatable Read is safer for transactions that read related rows. Pick per-transaction, not per-engine.

```
-- Check your current default
-- Postgres
SHOW default_transaction_isolation;
-- MySQL
SELECT @@transaction_isolation;
-- SQL Server
DBCC USEROPTIONS;
```

Q208 · SET TRANSACTION ISOLATION LEVEL — Syntax and Scope

Tags: Difficulty: Medium · Asked at: Infosys, Cognizant, Wells Fargo, Walmart Labs · Topic: Isolation

The question (interviewer's verbatim phrasing):

How do you change the isolation level, and what's the difference between session-level and transaction-level scope?

The 30-second answer: `SET TRANSACTION ISOLATION LEVEL <level>` applies only to the next transaction in the session. `SET SESSION TRANSACTION ISOLATION LEVEL <level>` applies to all future transactions in the session until the session ends. The engine default applies to all sessions that don't override.

Step-by-step thought process: 1. Three scopes: global default (engine config or `SET GLOBAL` in MySQL), session-level (until the connection closes), transaction-level (just this `BEGIN...COMMIT` block). 2. Most production code uses transaction-level — set the level inside the `BEGIN` block for the specific transaction that needs it, leave the rest at the default. 3. The MySQL syntax: `SET [GLOBAL|SESSION] TRANSACTION ISOLATION LEVEL ...`. The Postgres syntax: `SET TRANSACTION ISOLATION LEVEL ...` (transaction scope) or `SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL ...` (session scope). 4. ORM gotcha: connection pools reuse connections; if you change session-level isolation, the next user inherits it. Always set transaction-level scope in pooled environments.

Edge cases to mention: - In Postgres, `SET TRANSACTION ISOLATION LEVEL` must be the first statement in the transaction; placing it later raises an error. - In SQL Server, `SET TRANSACTION ISOLATION LEVEL` is session-scoped; to scope it per-transaction, you reset it after the `COMMIT`. - JDBC's `Connection.setTransactionIsolation()` is session-scoped — set it explicitly per checkout in a pool, or set it inside each transaction via SQL.

What NOT to say: - Do not change the global default to chase performance — it affects every session. - Do not assume session-level survives a connection reset — pools may reset the session state to a clean baseline.

Common follow-up: *"How does HikariCP handle isolation level changes?"* — It tracks the isolation level set when the connection was created and resets to the configured default on return-to-pool.

```
-- Postgres, transaction scope
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- ... work ...
COMMIT;

-- Postgres, session scope
SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- MySQL, session scope
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Q209 · Choosing Isolation Level for OLTP Workloads

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Flipkart, Cred · Topic: Isolation

The question (interviewer's verbatim phrasing):

How do you decide which isolation level to use for an OLTP system like a payments service?

The 30-second answer: Start with the engine default — Read Committed in Postgres, Repeatable Read in MySQL InnoDB — for the bulk of OLTP work. Step up to Repeatable Read for transactions that read related rows and need a consistent view. Use Serializable only for transactions with multi-row invariants where retry-on-failure is acceptable. Use explicit locking (SELECT FOR UPDATE) where you must read-then-write a single row safely.

Step-by-step thought process: 1. OLTP characteristics: short transactions (sub-second), high concurrency, small number of rows per transaction, latency-sensitive. 2. Most transactions don't have cross-row invariants — a single account debit-credit can be done atomically with `UPDATE balance = balance - X`. Read Committed is fine. 3. Read-then-write patterns (read balance, validate, then update) need either SELECT FOR UPDATE at Read Committed, or Repeatable Read with retry. 4. Multi-row invariants (Q17's on-call rota, inventory totals) need Serializable or explicit locks across all rows in the invariant.

Edge cases to mention: - Higher isolation increases the abort/retry rate — your application must have retry logic for any code path that uses Repeatable Read or Serializable in Postgres. - Holding higher-isolation transactions open for long causes MVCC bloat in Postgres (the engine cannot vacuum tuples older than the oldest open snapshot). - For payment systems, idempotency keys are often a cheaper way to handle retries than escalating isolation level — design the contract so duplicate submissions are safe.

What NOT to say: - Do not say "always use Serializable, it's safest" — the throughput penalty and abort rate are real. - Do not assume one isolation level fits the whole application — pick per-transaction based on what it does.

Common follow-up: *"How does the payments team at Razorpay or Cred actually choose?"* — Default to Read Committed with SELECT FOR UPDATE on the account row during transfer; idempotency keys to handle retries; Serializable reserved for periodic reconciliation jobs.

```
-- OLTP transfer pattern: Read Committed + row lock
BEGIN;
SELECT balance FROM accounts WHERE id = 101 FOR UPDATE;
SELECT balance FROM accounts WHERE id = 202 FOR UPDATE;
-- validate, then update
UPDATE accounts SET balance = balance - 2000 WHERE id = 101;
UPDATE accounts SET balance = balance + 2000 WHERE id = 202;
INSERT INTO transfers (from_id, to_id, amount) VALUES (101, 202, 2000);
COMMIT;
```

Q210 · Choosing Isolation Level for Reporting Workloads

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, JP Morgan, Swiggy · Topic: Isolation

The question (interviewer's verbatim phrasing):

A nightly reporting query needs consistent totals across many tables. How do you set up isolation for it?

The 30-second answer: Use Repeatable Read (or Serializable Snapshot Isolation in Postgres) so the entire report reads from one consistent snapshot. Even better, run reporting on a read replica where the workload is isolated from OLTP traffic. If the report is read-only and very long, use Postgres's `READ ONLY DEFERRABLE` mode to avoid serialization aborts.

Step-by-step thought process: 1. Reporting characteristics: long-running queries, multi-table joins and aggregates, read-only, sensitivity to read skew (inconsistent totals across related tables). 2. Read Committed is wrong because between table reads, other transactions can commit and change the data — totals will be inconsistent. 3. Repeatable Read pins the snapshot at the start of the report and every subsequent read sees the same view. 4. In Postgres, `BEGIN TRANSACTION READ ONLY DEFERRABLE ISOLATION LEVEL SERIALIZABLE` waits until it can get a safe snapshot, then runs without ever raising a serialization failure — perfect for long batch reports.

Edge cases to mention: - Long-running Repeatable Read transactions block VACUUM in Postgres — don't leave a reporting connection open for hours. - A reporting query on the primary OLTP database competes for I/O with transactional work — read replicas are usually a better answer. - Replication lag means the replica's snapshot is slightly behind the primary; for legally precise reports (audit, regulatory), you may need a freeze point on the primary.

What NOT to say: - Do not run an hour-long reporting transaction at Read Committed and trust the totals — they're not consistent. - Do not raise reporting isolation to Serializable expecting performance gains — Repeatable Read on a replica is almost always better.

Common follow-up: *"What if the report needs millisecond-fresh data?"* — Then run it on the primary at Repeatable Read, accept the impact on OLTP, and keep the report short.

```
-- Postgres long-running reporting transaction
BEGIN TRANSACTION READ ONLY DEFERRABLE ISOLATION LEVEL SERIALIZABLE;
-- multiple queries — all see the same snapshot
SELECT SUM(amount) FROM transfers WHERE created_on = CURRENT_DATE;
SELECT SUM(balance) FROM accounts;
-- ... more aggregations
COMMIT;
```

Q211 · Read Replicas and Isolation

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Walmart Labs, JP Morgan · Topic: Isolation

The question (interviewer's verbatim phrasing):

If I send my read traffic to a Postgres read replica, what isolation guarantees do I have?

The 30-second answer: Reads on a streaming replica are always snapshot-isolated against the replica's own state — within a transaction, you get a consistent view. But the replica is asynchronously behind the primary by some amount of replication lag, so reads can return slightly stale data. You cannot maintain isolation guarantees across primary and replica without explicit coordination.

Step-by-step thought process: 1. Replica semantics: the replica replays the WAL from the primary, so its state is the primary's state minus replication lag (usually milliseconds, occasionally seconds). 2. Inside a transaction on the replica, you get the standard isolation guarantees against the replica's data — Read Committed or Repeatable Read works as expected. 3. Cross-node consistency is the problem: write on the primary, immediately read on the replica — you might read stale data because the replica hasn't replayed your write yet. This breaks read-your-writes. 4. Fixes: route critical reads back to the primary, use synchronous replication for the specific operations that need it, or use a session-pinned router that tracks the primary's LSN and routes reads to the replica only after it has caught up.

Edge cases to mention: - Postgres hot standby has a `hot_standby_feedback` setting that lets the replica tell the primary about its long-running queries, preventing VACUUM from removing tuples the replica still needs — important for long reports. - Replication lag can grow unbounded under primary write pressure; monitor it. - Aurora and other cloud Postgres variants have very low replication lag (single-digit milliseconds) but it's still not zero.

What NOT to say: - Do not say "read replicas have the same isolation as the primary" — they have the same isolation but against a stale snapshot. - Do not assume read-your-writes works on a replica.

Common follow-up: "How do you handle the read-your-writes problem in practice?" — Either route the request to the primary if it follows a write in the same user session, or stick to eventually-consistent semantics in the UI ("your changes will appear shortly").

```
-- On a Postgres replica
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id = 101; -- consistent within the transaction
-- but might be lag-stale relative to the primary
COMMIT;
```

Q212 · Implicit Isolation in JDBC and ORM Connection Pools

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan, Cred · Topic: Isolation

The question (interviewer's verbatim phrasing):

In a typical Spring + HikariCP + Hibernate setup, how is the transaction isolation level set, and what bugs can you hit?

The 30-second answer: The isolation level can be set at the HikariCP pool level (default per connection), in JDBC via `Connection.setTransactionIsolation()`, in Spring via `@Transactional(isolation = ...)`, or in SQL inside the transaction. The bugs come from these layers disagreeing — a Spring annotation that's silently ignored, a pool default that's reset between checkouts, or an ORM that lazy-starts a transaction without inheriting your setting.

Step-by-step thought process: 1. The layers, top down: application code (Spring `@Transactional`) → ORM (Hibernate session) → JDBC connection (HikariCP) → engine session. 2. Spring's `@Transactional(isolation = Isolation.SERIALIZABLE)` calls `Connection.setTransactionIsolation()` on the underlying JDBC connection when the transaction starts. Without the annotation, you get the connection's default. 3. HikariCP has a `transactionIsolation` property that sets the default for every connection it hands out. If the application changes it during a transaction and forgets to reset, the next user inherits the change. 4. The classic bug: `@Transactional` on a method that's called via `this.method()` instead of an injected proxy — Spring's AOP doesn't intercept, the annotation is ignored, you silently run at the connection default.

Edge cases to mention: - HikariCP resets connection state on return-to-pool only if `resetConnectionOnClose` is enabled — read the version's docs. - Hibernate's `FlushMode.AUTO` can cause writes to happen before the transaction is explicitly committed — surprising for code reading the database between statements. - The `propagation` attribute on `@Transactional` interacts with the isolation level — nested propagation creates savepoints, not new transactions.

What NOT to say: - Do not trust that `@Transactional` is honoured without verifying with a test that inspects the actual isolation level. - Do not set the pool default to Serializable and assume the app is safe — every read query now pays the cost.

Common follow-up: "How do you verify the isolation level is actually being applied?" — Run `SELECT current_setting('transaction_isolation')` (Postgres) or `SELECT @@transaction_isolation` (MySQL) inside the transaction and log the result; better, write a test that asserts the value.

```
// Spring + JDBC, explicit isolation
@Transactional(isolation = Isolation.SERIALIZABLE)
public void transferFunds(long fromId, long toId, BigDecimal amount) {
    // Inside this method, the JDBC connection runs at Serializable
    // until the method returns and Spring commits/rolls back
}
```


Q213 · Putting It All Together — Picking the Right Level for a Real Scenario

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred, Goldman, JP Morgan · Topic: Isolation

The question (interviewer's verbatim phrasing):

You're designing a peer-to-peer payments service in India. Walk me through the isolation level choices for each major transaction type.

The 30-second answer: Transfer between two accounts: Read Committed with

`SELECT FOR UPDATE` on both account rows. User-balance display: Read Committed, single query. Periodic settlement reconciliation: Repeatable Read on a read replica. Promo-code redemption with limited-quantity inventory: Serializable with retry, because the invariant is multi-row. Idempotency-key checks: Read Committed with a unique constraint as the real guard.

Step-by-step thought process: 1. List the transaction types in the system: transfer, balance read, statement generation, reconciliation, promo-code redemption, idempotency-key lookup. 2. For each, identify the invariants and the read-write pattern. 3. Transfer: read two balances, validate sufficient funds, debit, credit, log. Multi-row write but no multi-row invariant beyond conservation; `SELECT FOR UPDATE` on both rows at Read Committed is enough. 4. Balance display: read one row; no isolation concern, Read Committed. 5. Statement generation: range query over many rows; needs a consistent snapshot, so Repeatable Read. 6. Reconciliation: hours-long aggregate over the whole ledger; Repeatable Read on a read replica to avoid affecting OLTP, ideally Postgres's `READ ONLY DEFERRABLE` Serializable. 7. Promo-code redemption: invariant "no more than N redemptions"; Serializable with retry, or a database-side counter with atomic decrement and a `CHECK` constraint. 8. Idempotency-key check: insert into `idempotency_keys` table with the key as a unique constraint; the constraint enforces uniqueness, isolation level is irrelevant.

Edge cases to mention: - Indian regulatory requirements (RBI guidelines for PPI and PA-PG operators) often dictate audit trail integrity — append-only ledger tables with strict isolation for the audit transaction. - Idempotency keys solve duplicate-submission problems more elegantly than escalating isolation — design the API contract to accept them. - Always benchmark — a theoretical isolation choice that doubles your latency is the wrong choice.

What NOT to say: - Do not use one isolation level for the whole service — the levels exist to be picked per transaction. - Do not skip the retry logic when using Repeatable Read or Serializable in Postgres — both can raise serialization failures.

Common follow-up: "What does Razorpay or PhonePe actually do?" — They use a combination: idempotency keys at the API layer, single-row atomic updates wherever possible (balance counters), `SELECT FOR UPDATE` on the account row during transfer, and Repeatable Read for batch reconciliation. The fanciest isolation level is rare in hot paths.

```
-- Transfer with SELECT FOR UPDATE at Read Committed
BEGIN;
-- lock both rows in a deterministic order to avoid deadlock
SELECT id, balance FROM accounts
  WHERE id IN (101, 202)
  ORDER BY id
  FOR UPDATE;
-- validate, write
UPDATE accounts SET balance = balance - 2000 WHERE id = 101;
UPDATE accounts SET balance = balance + 2000 WHERE id = 202;
INSERT INTO transfers (from_id, to_id, amount, idempotency_key)
  VALUES (101, 202, 2000, 'idem-abc-123');
COMMIT;

-- Promo redemption with retry-friendly Serializable
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT used_count, max_count FROM promos WHERE code = 'WELCOME50' FOR UPDATE;
-- validate used_count < max_count
UPDATE promos SET used_count = used_count + 1 WHERE code = 'WELCOME50';
INSERT INTO redemptions (user_id, promo_code) VALUES (5001, 'WELCOME50');
COMMIT; -- may raise 40001 - retry
```

Chapter 5, Part B — Locking & MVCC

This is where the rubber actually meets the road on isolation. Part A talked about what the isolation levels promise; Part B is about how engines deliver on those promises — through lock managers, lock modes, intent hierarchies, and the elegant trick called MVCC that lets readers and writers stop fighting each other. The questions below come up almost exclusively in product company rounds (Razorpay, Cred, Flipkart, PhonePe, Swiggy, Zerodha) and GCC senior rounds (Walmart Labs, Goldman, JP Morgan India, AmEx, Wells Fargo) where the panel wants to know if you can debug a production lock-wait incident at 2 AM. Spend extra time on the MVCC, VACUUM, and SELECT FOR UPDATE blocks — those three topics together account for roughly 60% of the lock-related follow-ups I've seen.

Q214 · Shared (S) Lock vs Exclusive (X) Lock — The Basics

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Wells Fargo · Topic: Locking

The question (interviewer's verbatim phrasing):

What is the difference between a shared lock and an exclusive lock, and which operations acquire which?

The 30-second answer: A shared lock (S) lets multiple transactions read the same row concurrently but blocks anyone from writing it. An exclusive lock (X) is grabbed for writes and blocks everyone else — readers and writers both. S is compatible with S; X is compatible with nothing.

Step-by-step thought process: 1. Open with the compatibility line — that's the actual rule. S/S = yes, S/X = no, X/X = no. 2. Map to operations: plain `SELECT` historically takes S (in pure 2PL engines like SQL Server under Read Committed without snapshot, or MySQL with `LOCK IN SHARE MODE`). `INSERT` / `UPDATE` / `DELETE` always takes X on the affected rows. 3. Crucial caveat: in modern MVCC engines (Postgres, Oracle, MySQL InnoDB with default settings), plain `SELECT` does not take an S lock at all — it reads from a snapshot. S locks are only acquired with explicit `FOR SHARE` / `LOCK IN SHARE MODE`. 4. Be ready for: "So when do S locks actually appear in Postgres?" — mainly `SELECT FOR SHARE` and DDL operations like `ALTER TABLE` taking lower-mode table locks.

Edge cases to mention: - SQL Server with Read Committed (default) does take short-lived S locks during reads — that's why writers can block readers there unless you enable Read Committed Snapshot Isolation. - The lock is on the row in row-level engines, but logically the S/X distinction also applies at page and table grain. - A transaction holding an S lock cannot be granted an X lock on the same row until other S holders release — this is the classic lock-upgrade deadlock that the U lock was invented to prevent.

What NOT to say: - Do not say "SELECT always takes a shared lock." In modern MVCC engines it usually takes none. - Do not claim X locks block only writers; they block readers too in non-MVCC engines.

Common follow-up: "If two transactions both hold an S lock on the same row and both want to upgrade to X, what happens?" — Classic upgrade deadlock; both wait for the other to release S. Engines either detect it and abort one, or use a separate U lock mode to prevent it (covered in Q3).

```
-- Postgres: explicit shared lock
BEGIN;
SELECT balance FROM accounts WHERE id = 7001 FOR SHARE;
-- another txn can also FOR SHARE this row, but cannot UPDATE it
COMMIT;
```

Q215 · Update (U) Lock — Why It Exists

Tags: Difficulty: Medium · Asked at: SQL Server shops, Walmart Labs, Goldman · Topic: Locking

The question (interviewer's verbatim phrasing):

What is an Update lock and what problem does it solve that plain S and X locks do not?

The 30-second answer: A U lock is an intent-to-update marker — it is compatible with existing S locks but incompatible with other U locks and with X locks. It prevents the lock-upgrade deadlock where two transactions both hold S, both try to upgrade to X, and deadlock waiting for each other.

Step-by-step thought process: 1. Frame the scenario first: txn A and txn B both read row R with an S lock (legal — S/S). Both then decide to update it. Both request X. Both wait for the other to drop S. Deadlock. 2. Solution: when a transaction reads a row that it might later update (e.g., `UPDATE ... WHERE` cursor positioning), it takes a U lock instead of S. U/S = yes (readers still allowed), but U/U = no (only one would-be updater at a time). 3. When the actual write happens, U is upgraded to X cleanly — no other U holder is in the way. 4. This is heavily a SQL Server / Sybase concept. Oracle and Postgres avoid the problem differently via MVCC and explicit `FOR UPDATE` / `FOR NO KEY UPDATE` modes (Postgres takes a row-exclusive marker directly).

Edge cases to mention: - U locks appear in SQL Server query plans when you see `WITH (UPDLOCK)` hints — used in read-then-write patterns inside the same transaction. - The U lock model assumes a 2PL engine; in MVCC engines it does not strictly apply, but the analogous problem still exists and is solved by `SELECT FOR UPDATE`. - The classic SQL Server pattern: `SELECT ... WITH (UPDLOCK, HOLDLOCK) WHERE id = ?` to grab a U lock that survives until commit.

What NOT to say: - Do not say Postgres has U locks — it does not in the SQL Server sense. - Do not call it an "upgrade lock that lets two transactions write" — only one U at a time on a row.

Common follow-up: "How does Postgres avoid this same deadlock without a U lock?" — Through `SELECT FOR UPDATE` which directly takes a row-exclusive lock at read time, and through MVCC which avoids the S-lock-during-read situation entirely.

```
-- SQL Server: take U lock during read so subsequent UPDATE doesn't deadlock
BEGIN TRAN;
SELECT balance FROM accounts WITH (UPDLOCK, HOLDLOCK) WHERE id = 7001;
UPDATE accounts SET balance = balance - 500 WHERE id = 7001;
COMMIT;
```

Q216 · Intent Locks — IS, IX, SIX and the Hierarchy

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs · Topic: Locking

The question (interviewer's verbatim phrasing):

What are intent locks (IS, IX, SIX) and why does the lock manager need them?

The 30-second answer: Intent locks are coarse-grained markers placed on a parent object (table or page) saying "some child rows are locked in S or X mode." They let the engine check table-level

compatibility in O(1) instead of scanning every row, which is what makes lock escalation and table-level operations like `ALTER TABLE` performant.

Step-by-step thought process: 1. Without intent locks, when someone wants to lock the whole table in X mode (say, an ALTER), they'd have to walk every row to confirm nobody holds a conflicting lock. That's catastrophically slow on a billion-row table. 2. With intent locks, every row-level S sets an IS on the table; every row-level X sets an IX. To take a table X, you check the table itself — if any IS or IX exists, you wait. 3. SIX is a compound mode: S on the table (you are reading the whole thing) plus IX (you are also writing some specific rows). It exists so that bulk-read-mostly-with-occasional-writes scans don't need to constantly transition between modes. 4. Compatibility matrix matters: IS//IS = yes, IS//IX = yes, IX//IX = yes (both intent only — actual row conflicts checked at row level), IS//X = no, IX//S = no, SIX//SIX = no.

Edge cases to mention: - All major engines (SQL Server, Oracle, MySQL InnoDB, DB2) use intent locks; only some expose them in monitoring views. - In Postgres these are expressed as table-level lock modes like `ROW SHARE`, `ROW EXCLUSIVE`, `SHARE ROW EXCLUSIVE` — same idea, different vocabulary. - The hierarchy can be three or four levels deep: database → table → page → row. Each level needs its own intent.

What NOT to say: - Do not say intent locks "lock the table." They do not block compatible work — they only block conflicting coarse-grained requests. - Do not skip SIX — interviewers love testing whether you know the compound mode exists.

Common follow-up: "Translate ROW SHARE in Postgres to SQL Server vocabulary." — That is approximately IS — taken by `SELECT FOR SHARE` / `FOR UPDATE` to allow row work while signalling intent at table level.

```
-- Postgres: SELECT FOR UPDATE takes ROW SHARE on the table (analogous to IS/IX)
BEGIN;
SELECT id, balance FROM accounts WHERE id = 7001 FOR UPDATE;
-- table-level ROW SHARE blocks competing ALTER TABLE but not other FOR UPDATES on other rows
COMMIT;
```

Q217 · Row-level vs Page-level vs Table-level Locks

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Walmart Labs · Topic: Locking

The question (interviewer's verbatim phrasing):

Walk me through row, page and table-level locks — what are the tradeoffs and which engine does what by default?

The 30-second answer: Finer granularity gives higher concurrency but higher overhead per lock (memory, lock manager CPU); coarser granularity is cheap but blocks more transactions. Postgres, Oracle, and MySQL InnoDB lock at the row level by default. SQL Server starts at the row level and can escalate to page or table when row counts cross a threshold (around 5000 locks per statement).

Step-by-step thought process: 1. Cost model: each lock takes around 100-200 bytes of memory in the lock manager. A million-row UPDATE means a million locks — gigabytes of lock metadata, slow lookups. 2. Page-level (8 KB in Postgres/SQL Server, 16 KB in InnoDB): one lock covers all rows on the page. Cheap but causes false conflicts when two transactions touch different rows that happen to live on the same page. 3. Table-level: one lock, maximum blocking. Used for DDL and explicit `LOCK TABLE`. 4. Engine defaults: Postgres only locks at row and table grain — no page locks. Oracle row only. InnoDB row only. SQL Server row by default but escalates.

Edge cases to mention: - Postgres uses a separate predicate-lock mechanism for SSI Serializable instead of page locks — predicate locks are tracked in a special structure and can sit at index-page grain internally. - InnoDB's row lock is logically a lock on an index record, not on a heap tuple — important when explaining gap locks (Q12). - SQL Server's lock escalation can be tuned per table (`ALTER TABLE ... SET (LOCK_ESCALATION = DISABLE | TABLE | AUTO)`).

What NOT to say: - Do not say "Postgres uses page locks by default" — it doesn't. - Do not claim row-level is always best — there's a memory cliff at scale.

Common follow-up: "What happens when SQL Server hits the escalation threshold?" — It tries to take a table-level X (or S, matching the work), and if that's not immediately available, it can fail with lock-escalation deadlock errors. Usually resolved by raising threshold or batching.

```
-- SQL Server: disable lock escalation on a hot table
ALTER TABLE orders SET (LOCK_ESCALATION = DISABLE);
```

Q218 · Lock Compatibility Matrix

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Cred · Topic: Locking

The question (interviewer's verbatim phrasing):

Draw the lock compatibility matrix for S, X, U, IS, IX and SIX. Which combinations are compatible?

The 30-second answer: Anything purely intent-based (IS, IX) is compatible with anything that's also intent-based or shared at the parent level. S conflicts with X, U, IX, SIX. X conflicts with everything except — well, nothing. U is compatible with S only. SIX is compatible only with IS.

Step-by-step thought process: 1. Memorise the principle, not the table: stronger modes block more. The ordering roughly is $IS < IX < S < SIX < U < X$. 2. Two questions to ask of any pair: do they both want to read (then mostly compatible), or does at least one want to write (then almost always conflict at the right grain). 3. Intent locks at the table level don't conflict among themselves because the real conflict check happens at the row level. 4. SIX is the trickiest — remember it means "I'm reading the whole thing AND writing some rows" so it must block anyone who'd write the whole thing or read with intent to write.

Edge cases to mention: - The matrix is engine-specific in details — Oracle has additional modes like SRX (Share Row eXclusive) that approximate SIX. - Schema locks (Sch-S, Sch-M in SQL Server) are a separate hierarchy used for DDL — Sch-M blocks everything including reads. - Postgres has eight table-level modes (ACCESS SHARE through ACCESS EXCLUSIVE) — same compatibility logic, different names.

What NOT to say: - Do not recite the matrix without explaining the underlying principle — interviewers will probe with edge pairs you haven't memorised. - Do not conflate row-grain and table-grain in the same matrix.

Common follow-up: *"If a transaction holds IX on a table and you want to take S on the table, can you?"* — No. IX means someone is writing rows; S means you want to read the whole consistent table. They conflict.

```
-- Postgres: ACCESS EXCLUSIVE (taken by DROP TABLE) blocks every other mode
-- BEGIN; DROP TABLE accounts; -- waits behind any open SELECT
```

Q219 · SELECT FOR UPDATE — What It Locks and When To Use It

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Zerodha, Flipkart · Topic: Locking

The question (interviewer's verbatim phrasing):

What does SELECT FOR UPDATE actually lock, and give me a real scenario where you'd use it.

The 30-second answer: It places a row-level exclusive intent on every row the query returns, blocking other transactions from updating, deleting, or locking those rows until you commit. Use it for the classic read-check-then-write pattern, e.g., reading a wallet balance, validating it's enough, then debiting.

Step-by-step thought process: 1. Mechanics: in Postgres it takes a **FOR UPDATE** row lock (strongest row-lock mode), in InnoDB it takes an X record lock plus relevant gap locks under Repeatable Read. 2. Concrete payments scenario: wallet debit. Without FOR UPDATE, two concurrent debit requests can both read the same balance, both pass the "enough money" check, and both write — overdrafting the wallet. With FOR UPDATE, the second one waits. 3. Crucial: it must be inside a transaction. Outside a transaction the lock is released immediately and you've gained nothing. 4. Be ready for **FOR UPDATE**

NOWAIT and **FOR UPDATE SKIP LOCKED** variants — **SKIP LOCKED** is the killer feature for job queues built on a SQL table.

Edge cases to mention: - In Postgres, **FOR UPDATE** also locks rows referenced by foreign keys from the selected rows — sometimes more than you expect. Use **FOR NO KEY UPDATE** if you don't need that. - It does not prevent INSERT of new rows that would also match the WHERE clause — that's the phantom problem; gap locks (Q12) handle that in InnoDB Repeatable Read. - In auto-commit mode (default for many drivers), each statement is its own transaction — FOR UPDATE effectively does nothing.

What NOT to say: - Do not say it locks the table — it locks only the returned rows (plus, in InnoDB, gaps). - Do not skip the "must be in a transaction" point.

Common follow-up: "What is *FOR UPDATE SKIP LOCKED* used for?" — Worker queue pulling. Worker A picks row 1, locks it; worker B's **SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1** skips row 1 and grabs row 2. No queue server needed.

```
-- Wallet debit, safe against concurrent debits
BEGIN;
SELECT balance FROM wallets WHERE user_id = 91234 FOR UPDATE;
-- application checks balance >= 500
UPDATE wallets SET balance = balance - 500 WHERE user_id = 91234;
COMMIT;
```

Q220 · SELECT FOR NO KEY UPDATE and FOR KEY SHARE (Postgres)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Postgres-heavy shops · Topic: Locking · Dialect: Postgres

The question (interviewer's verbatim phrasing):

Postgres has four row-lock modes for SELECT — FOR UPDATE, FOR NO KEY UPDATE, FOR SHARE, FOR KEY SHARE. When would you pick the weaker ones?

The 30-second answer: The weaker modes exist so that you don't accidentally block foreign-key checks. **FOR KEY SHARE** is the lightest — it only blocks transactions that would change the key columns. **FOR NO KEY UPDATE** is the write equivalent — you're updating, but not the key, so FK checks from other tables can still proceed.

Step-by-step thought process: 1. Hierarchy from weakest to strongest: FOR KEY SHARE < FOR SHARE < FOR NO KEY UPDATE < FOR UPDATE. 2. Postgres takes FOR KEY SHARE automatically on rows referenced by an FK during INSERT/UPDATE of child rows. That means if you grab FOR UPDATE on a parent row, you block child INSERTs — often unintended. 3. Use FOR NO KEY UPDATE in the

common case (updating a non-key column like `balance` or `status`) — it doesn't block child-table FK validations. 4. Use FOR KEY SHARE if you want to read a row and ensure its primary key won't change (e.g., long-running computation referencing it) but allow updates to other columns.

Edge cases to mention: - This is largely a Postgres concern; MySQL InnoDB's analogue is the difference between record locks and gap locks. - Forgetting to use FOR NO KEY UPDATE in high-throughput payments where parent rows are constantly updated can cause cascading slowdowns on child INSERTs. - The four modes were added in Postgres 9.3; pre-9.3 code that uses FOR UPDATE may now over-lock unnecessarily — worth auditing.

What NOT to say: - Do not say all four are interchangeable. - Do not claim FOR KEY SHARE is "read-only" — it does take a lock and is visible in `pg_locks`.

Common follow-up: "I have a `users` table referenced by `orders`. I do FOR UPDATE on a user row. Why are my order INSERTs now slow?" — Because INSERT into orders takes FOR KEY SHARE on the parent user row, which conflicts with your FOR UPDATE. Switch to FOR NO KEY UPDATE.

```
-- Update non-key column without blocking child FK inserts
BEGIN;
SELECT id, balance FROM users WHERE id = 91234 FOR NO KEY UPDATE;
UPDATE users SET balance = balance - 500 WHERE id = 91234;
COMMIT;
```

Q221 · Lock Escalation — SQL Server and Oracle

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan India · Topic: Locking · Dialect: SQL Server, Oracle

The question (interviewer's verbatim phrasing):

What is lock escalation, when does it trigger, and why is it sometimes a problem?

The 30-second answer: Lock escalation is the engine deciding that holding too many fine-grained locks is wasteful and converting them to one coarse lock. SQL Server does this when a single statement holds approximately 5000 row/page locks; Oracle effectively does not escalate (it claims to keep row-level forever). Escalation can suddenly block unrelated transactions because what was hundreds of row locks becomes one table lock.

Step-by-step thought process: 1. The motive: lock manager memory. At several gigabytes per million-row UPDATE, escalation reclaims memory by collapsing many row locks into one table lock. 2. SQL Server's threshold is around 5000 locks on a single object in one statement, or when total lock memory exceeds a percentage of buffer pool. It tries to escalate; if blocked, retries periodically. 3. The risk: a

long-running UPDATE that escalates suddenly blocks all concurrent SELECTs on the same table. The application sees a stall with no clear cause. 4. Mitigation: batch large updates into chunks of a few hundred rows each, or `ALTER TABLE ... SET (LOCK_ESCALATION = DISABLE)`.

Edge cases to mention: - Postgres does not escalate row locks to table locks — but it has a different problem: it stores row locks inside the tuple header (using XID + multixact), so they're free in terms of lock manager memory. - InnoDB also doesn't escalate — it has a fixed-size lock-table that can pressure memory if you UPDATE a million rows in one transaction. - DB2 LUW escalates similarly to SQL Server with a `LOCKLIST` parameter.

What NOT to say: - Do not say Postgres escalates — common myth. - Do not claim escalation always helps performance — it usually hurts concurrency.

Common follow-up: *"How would you safely UPDATE 10 million rows in SQL Server?"* — Loop in batches of 1000-5000 rows with explicit transactions, ideally driven by the primary key, with brief pauses for the log to checkpoint.

```
-- SQL Server: batched delete to avoid escalation
WHILE 1 = 1
BEGIN
    DELETE TOP (1000) FROM audit_log WHERE created_at < '2024-01-01';
    IF @@ROWCOUNT = 0 BREAK;
    WAITFOR DELAY '00:00:00.100';
END
```

Q222 · Lock Granularity Tradeoffs

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Walmart Labs · Topic: Locking

The question (interviewer's verbatim phrasing):

If I asked you to design a lock manager, how would you decide whether to lock at row, page, or table grain?

The 30-second answer: Optimise for the dominant access pattern. OLTP with single-row reads and writes → row level. Bulk DML and DDL → table level. Anything in between either picks the smallest grain that still gives reasonable lock-manager memory, or starts small and escalates.

Step-by-step thought process: 1. Cost dimensions: lock memory (per-lock overhead), lock manager CPU (hash-table lookups), false conflicts (two unrelated rows on same page), application latency under contention. 2. Tradeoff curve: row level minimises false conflicts but inflates memory; table level minimises memory but creates massive contention. 3. Mixed workloads benefit from escalation strategies (SQL Server) or skipping page-level entirely with cheap row locks stored in tuple headers

(Postgres MVCC). 4. Design rule: pick row-level for OLTP, accept escalation or batch large jobs, use explicit **LOCK TABLE** only for migrations or one-time backfills.

Edge cases to mention: - For analytics / append-only fact tables, table-level locks are often fine because there's only one writer (the ETL). - For job-queue tables with SKIP LOCKED, row-level is mandatory — otherwise workers serialize. - Page-level locks made sense in the 1990s when memory was scarce; modern systems mostly avoid them.

What NOT to say: - Do not say "always use the finest grain." Memory matters. - Do not say table-level locks are always bad — they're correct for DDL.

Common follow-up: "What if you have a hot row that 10K txns/sec want to write?" — Row-level locking won't save you; you need application-level sharding (split the counter, advisory locks, queue serialization, or in-memory aggregation).

```
-- Postgres ROW EXCLUSIVE (row-level X-intent) blocks DDL but not other row writes on other rows
BEGIN;
UPDATE inventory SET stock = stock - 1 WHERE sku = 'IPH-15-BLK-128';
COMMIT;
```

Q223 · MVCC — What It Is and Why Readers Don't Block Writers

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Postgres-heavy shops, Goldman · Topic: MVCC

The question (interviewer's verbatim phrasing):

Explain MVCC in your own words. Why does it mean readers don't block writers, and writers don't block readers?

The 30-second answer: MVCC stores multiple versions of each row, tagged with the transaction IDs that created and deleted them. A reader sees the version visible to its snapshot — it never needs to lock anything, because the version it wants is already there. Writers create new versions without disturbing the old ones, so readers see consistent data while writes happen in parallel.

Step-by-step thought process: 1. Frame against the alternative: pure 2PL. There, readers grab S, writers grab X, and they collide. That's where the famous "writers block readers" pain comes from in legacy SQL Server or DB2. 2. MVCC's insight: keep old versions around. When txn A reads at time T, it sees the version that was current at T. When txn B writes concurrently, it creates version T+1 without touching version T. 3. Snapshot is the key abstraction: each reading transaction has a snapshot of which other transactions were committed when it started. It only sees rows from those committed txns. 4. Cost:

storage (old versions must persist until no snapshot needs them) and a cleanup mechanism (VACUUM in Postgres, undo segments in Oracle, purge thread in InnoDB).

Edge cases to mention: - "Readers don't block writers" is specifically about plain SELECT. **SELECT FOR UPDATE** does take a lock. - Two writers updating the same row still block each other in MVCC — MVCC solves the read-write conflict, not the write-write one. - Long-running readers can prevent cleanup of versions they might still need — operational pain (Q15).

What NOT to say: - Do not say MVCC eliminates all locking — it doesn't, only read locks (mostly). - Do not say MVCC is free — the storage and cleanup cost is real.

Common follow-up: *"What happens to memory/disk when a row is updated 10 times in MVCC?"* — Ten versions exist (briefly). Old ones cleaned up by VACUUM (Postgres) or undo expiration (Oracle/InnoDB). Bloat happens when cleanup can't keep up.

```
-- Postgres: long SELECT does not block concurrent UPDATES
-- Session A: BEGIN; SELECT * FROM large_report; -- runs for 10 min
-- Session B: UPDATE accounts SET balance = balance + 1 WHERE id = 7001; -- not blocked
```

Q224 · MVCC Implementations — Postgres vs Oracle vs MySQL InnoDB

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, JP Morgan, Database-platform teams · Topic: MVCC

The question (interviewer's verbatim phrasing):

How do Postgres, Oracle, and MySQL InnoDB each implement MVCC differently?

The 30-second answer: Postgres stores new versions inline in the heap (every UPDATE writes a new tuple, old becomes dead, VACUUM cleans up). Oracle stores the current version in place and pushes the old version into undo segments (UNDO tablespace). InnoDB does the same as Oracle — current row in place, old versions in rollback segments — with the wrinkle that secondary indexes don't carry versions, so they trigger extra lookups.

Step-by-step thought process: 1. Postgres approach (heap-versioned): pros are simple recovery, easy hot-standby replay; cons are heap bloat, index bloat, and the VACUUM tax. 2. Oracle approach (undo segments): the table always has the latest committed value; older snapshots reconstruct from undo. Pros: tables stay compact, no VACUUM. Cons: long readers can hit **ORA-01555 snapshot too old** if undo retention is too low. 3. InnoDB approach: similar to Oracle — clustered index has latest version, rollback segments hold older. Secondary indexes are not versioned, so to honour snapshots, InnoDB also tracks "is this row visible at my snapshot" via a back-pointer to the

clustered index — extra IO for index reads. 4. SQL Server (when RCSI/Snapshot Isolation is enabled) stores old versions in tempdb, similar in spirit to Oracle's undo but in a separate storage.

Edge cases to mention: - Postgres's HOT (Heap-Only Tuple) optimisation: if the update doesn't change any indexed column, the new version chains to the old via a pointer and the index doesn't need updating — huge win for hot updates. - Oracle's UNDO_RETENTION parameter trades off snapshot longevity against tablespace size. - InnoDB's history list length (visible in `SHOW ENGINE INNODB STATUS`) is the lag indicator — high values mean long readers blocking purge.

What NOT to say: - Do not say all MVCC implementations are the same — they have very different operational profiles. - Do not claim Oracle/InnoDB never have bloat — undo can grow unbounded under pressure.

Common follow-up: "Why does Postgres need VACUUM but Oracle doesn't?" — Because Postgres versions live in the same heap as live rows; Oracle keeps them in a separate UNDO area that has its own lifecycle and expires automatically.

```
-- Postgres: see dead tuple count
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables
WHERE relname = 'accounts';
```

Q225 · Postgres Snapshot — xmin, xmax, txid Mechanics

Tags: Difficulty: Hard · Asked at: Postgres-heavy shops, Cred, Razorpay, Database teams · Topic: MVCC · Dialect: Postgres

The question (interviewer's verbatim phrasing):

At a high level, how does Postgres decide which version of a row your transaction can see? What are xmin and xmax?

The 30-second answer: Every row carries `xmin` (the transaction ID that inserted it) and `xmax` (the transaction ID that deleted or updated it, zero if still live). When your txn starts, it captures a snapshot — the set of currently active transactions. A row is visible if its `xmin` is committed-and-before-snapshot, and its `xmax` is either zero, in-the-future, or aborted.

Step-by-step thought process: 1. Two hidden system columns: `xmin`, `xmax`. Plus `cmin`, `cmax` for sub-transaction (command) ordering. 2. Snapshot at txn start: an `xmin` cutoff (oldest still-running txn), an `xmax` cutoff (next txid to be assigned), and a list of active xids in between. 3. Visibility rules: row is visible if its xmin is committed and not in the active list, AND its xmax is null or invisible (uncommitted or in active list). 4. UPDATE doesn't modify a row in place — it writes a new row with new

xmin and marks the old row's xmax. The old row stays around until VACUUM determines no snapshot can still see it.

Edge cases to mention: - Transaction ID wraparound: xids are 32-bit, so after 2 billion they wrap. Postgres `VACUUM FREEZE` prevents catastrophic data loss by marking rows as visible to all current and future txns. - Subtransactions (savepoints) use a separate xid that combines with the parent's. - `txid_current()` returns the current transaction's ID; `pg_current_snapshot()` returns the snapshot for debugging.

What NOT to say: - Do not say xmin/xmax are visible columns by default — they're hidden but accessible via `SELECT xmin, xmax FROM table`. - Do not confuse txid (a global counter) with the visibility rules (which use the snapshot relative to that counter).

Common follow-up: "What happens if I never VACUUM and run out of txids?" — Wraparound failure: Postgres halts writes to protect data. The dreaded "must be vacuumed within X transactions" warning leads to forced shutdown if ignored.

```
-- Inspect MVCC metadata on rows
SELECT xmin, xmax, * FROM accounts WHERE id = 7001;
```

Q226 · VACUUM in Postgres — Why It Exists

Tags: Difficulty: Medium · Asked at: Postgres shops, Razorpay, Cred, Database-platform teams · Topic: MVCC · Dialect: Postgres

The question (interviewer's verbatim phrasing):

Why does Postgres need VACUUM? What exactly does it do?

The 30-second answer: VACUUM reclaims space from dead tuples (the old row versions left behind by UPDATE/DELETE) so the heap doesn't grow forever, and it advances the "frozen xid" pointer to prevent transaction ID wraparound. Without VACUUM, both bloat and wraparound shutdown would eventually kill the database.

Step-by-step thought process: 1. UPDATE in Postgres = INSERT new + mark old dead. The dead tuple still occupies its slot on the page; new inserts can't reuse it until VACUUM frees it. 2. Plain `VACUUM` marks dead tuple space as reusable (visible in the free-space map) but does not return space to the OS — the file stays the same size. New rows fill the holes. 3. `VACUUM FULL` rewrites the entire table to compact it and returns space to the OS, but takes an `ACCESS EXCLUSIVE` lock — i.e., the table is unreadable for the duration. Almost never run in production. 4. VACUUM also updates the visibility map (helps index-only scans), advances xid horizon (FREEZE), and updates statistics if `VACUUM ANALYZE`.

Edge cases to mention: - VACUUM cannot remove a dead tuple if any active transaction's snapshot might still need it — long readers block cleanup (Q15). - `pg_visibility` extension lets you inspect which pages are all-visible / all-frozen. - Toast tables (out-of-line storage for large columns) have their own VACUUM cycles.

What NOT to say: - Do not say VACUUM "frees disk space" by default — it doesn't, only VACUUM FULL does. - Do not say it's optional — autovacuum runs automatically for a reason.

Common follow-up: "*Why is VACUUM FULL dangerous on a busy table?*" — It takes an exclusive lock for the entire rewrite. On a 100 GB table, that's potentially hours of blocked traffic. Use `pg_repack` for online equivalent.

```
-- Manual vacuum with stats update
VACUUM ANALYZE orders;
-- See bloat estimate per table
SELECT relname, pg_size_pretty(pg_relation_size(relid)) AS size,
       n_dead_tup, n_live_tup
FROM   pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 10;
```

Q227 · Autovacuum — Triggers and Tuning

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postgres-heavy shops, Database teams · Topic: MVCC · Dialect: Postgres

The question (interviewer's verbatim phrasing):

When does autovacuum kick in, and what would you tune on a high-write table?

The 30-second answer: Autovacuum triggers on a table when dead tuples exceed

`autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * n_live_tup` (default $50 + 0.2 \times \text{live rows}$). On a million-row hot table, that means waiting for ~200K dead tuples before any cleanup — usually too lazy. Tune by lowering the scale factor or per-table threshold.

Step-by-step thought process: 1. Default triggers: 20% dead-tuple fraction, plus statistics-based analyze trigger at 10%. Both are way too high for tables with millions of rows and high churn. 2. Per-table override: `ALTER TABLE orders SET (autovacuum_vacuum_scale_factor = 0.02, autovacuum_analyze_scale_factor = 0.01);` — runs at 2% dead instead of 20%. 3. Concurrency: by default, only a few autovacuum workers run at once (`autovacuum_max_workers`, default 3). On a multi-TB DB with many hot tables, this is a bottleneck — raise it. 4. Cost-based throttling: autovacuum sleeps based on `autovacuum_vacuum_cost_limit / cost_delay` to limit IO impact. On SSDs you almost always want to raise the limit so vacuum finishes faster.

Edge cases to mention: - Anti-wraparound vacuum (`for prevention of wraparound`) is non-skippable — even if you've disabled autovacuum on a table, this kind runs. It's also more expensive (full scan). - Autovacuum can be starved by long open transactions — VACUUM can't remove tuples newer than the oldest snapshot. - Postgres 13+ added per-index vacuum parallelism (`max_parallel_maintenance_workers`).

What NOT to say: - Do not say "disable autovacuum and run nightly manual VACUUM" — invites wraparound disasters. - Do not claim defaults are fine for everyone — they're tuned for tiny conservative installs.

Common follow-up: "How do you know autovacuum is falling behind?" — Check `pg_stat_user_tables.last_autovacuum` and `n_dead_tup` . Also `pg_stat_activity` for active autovacuum workers and `pg_stat_progress_vacuum` .

```
-- Tighter autovacuum for a hot table
ALTER TABLE transactions SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.01
);
```

Q228 · Bloat — When VACUUM Isn't Enough, `pg_repack`

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postgres-heavy shops · Topic: MVCC · Dialect: Postgres

The question (interviewer's verbatim phrasing):

What is table bloat, and what do you do when VACUUM alone can't recover it?

The 30-second answer: Bloat is dead tuple space that ordinary VACUUM has marked reusable but never returned to the OS — over time the table file grows much larger than the live data inside it. If new writes don't reuse the space fast enough (sparse delete patterns, long-stalled writers), you need a full table rewrite. `pg_repack` does it online without the `ACCESS EXCLUSIVE` lock that VACUUM FULL takes.

Step-by-step thought process: 1. Detect bloat: compare `pg_relation_size(table)` against live row count × average row size. Or use `pgstattuple` / `pg_repack --dry-run` for a precise estimate. 2. Why VACUUM alone fails: if you DELETE the head of a table (oldest rows) and only INSERT to the end, the leading pages become permanently empty. VACUUM frees them but the file can only truncate the trailing empty pages. 3. `pg_repack` solution: it creates a shadow table, copies live rows in batches, replays changes via triggers, then swaps the relfilenode atomically. The lock window is

seconds, not hours. 4. Prevention: use partitioning so you can drop entire partitions instead of DELETE; tune autovacuum aggressively on churn-heavy tables.

Edge cases to mention: - Index bloat is often worse than table bloat — every UPDATE writes a new index entry. `REINDEX CONCURRENTLY` (Postgres 12+) rebuilds without locking. - `pg_repack` needs a primary key or unique index; it cannot repack tables without one. - Toast tables also bloat independently — sometimes the parent is fine but the toast is huge.

What NOT to say: - Do not recommend VACUUM FULL casually — production teams hate the lock window. - Do not assume bloat = forgot to VACUUM — long-running readers and rare delete patterns cause it even with perfect autovacuum.

Common follow-up: "What's the safest way to remove old data from a billion-row table?" — Partition by date and drop partitions. DELETE doesn't truncate; partition drop does.

```
-- Detect bloat-prone tables (rough)
SELECT relname, n_dead_tup, n_live_tup,
       n_dead_tup::float / NULLIF(n_live_tup, 0) AS dead_ratio
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_ratio DESC NULLS LAST;
```

Q229 · Long-Running Transactions + Bloat — The Operational Risk

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postgres shops, On-call interviews · Topic: MVCC

The question (interviewer's verbatim phrasing):

One of your developers leaves a transaction open for two hours. What can go wrong in Postgres?

The 30-second answer: The long transaction holds back the global xmin horizon — VACUUM cannot remove any dead tuple newer than that transaction's snapshot. So across every table in the database, bloat accumulates. After enough hours, hot tables grow several times their normal size, queries slow down, and disk fills.

Step-by-step thought process: 1. Mechanism: VACUUM's correctness relies on never removing a tuple any active snapshot might still need. The oldest snapshot in the system gates everything. 2. Symptoms: monitoring shows `n_dead_tup` climbing, query latency creeping up, disk usage growing despite no new data. `pg_stat_activity` shows one txn with `xact_start` hours ago. 3. Defense: `idle_in_transaction_session_timeout` (kills sessions sitting idle inside a transaction), `statement_timeout` (kills long single queries), monitoring on `now() - xact_start > '10 minutes'`. 4. Same issue exists in Oracle (snapshot too old at the opposite end — the long reader fails) and InnoDB (history list grows, purge falls behind).

Edge cases to mention: - Replication slots can also pin xmin — an unconsumed logical replication slot has the same effect as a forgotten transaction. - A 2-hour analytical SELECT on a read replica with `hot_standby_feedback=on` propagates the xmin pin back to the primary, causing primary bloat. - Application connection-pool middleware that swallows commits can silently leave txns open.

What NOT to say: - Do not say "long transactions are only bad in Postgres." They're problematic everywhere, just in different ways. - Do not suggest disabling autovacuum as a fix — that makes it worse.

Common follow-up: "How would you find and kill a long-running idle transaction?" — Query `pg_stat_activity` for state = 'idle in transaction' and old `state_change`, then `pg_terminate_backend(pid)`.

```
SELECT pid, username, state, age(now(), xact_start) AS txn_age, query
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'active')
      AND xact_start < now() - INTERVAL '5 minutes'
ORDER BY xact_start;
```

Q230 · Gap Locks (MySQL InnoDB)

Tags: Difficulty: Hard · Asked at: MySQL-heavy shops, Flipkart, PhonePe, Swiggy, Razorpay · Topic: Locking · Dialect: MySQL InnoDB

The question (interviewer's verbatim phrasing):

What is a gap lock in InnoDB, and what problem does it solve?

The 30-second answer: A gap lock locks the empty space between index records, preventing other transactions from INSERTing rows that would land in that gap. InnoDB uses gap locks under Repeatable Read to prevent phantom reads — without them, a re-executed range query could see new rows that didn't exist before.

Step-by-step thought process: 1. The phantom problem: `SELECT * FROM orders WHERE amount > 1000` returns 10 rows. Another txn INSERTs an order with amount 5000. You re-execute. Now 11 rows. That's a phantom. 2. Pure row locks don't help — you can only lock rows that exist. To prevent the INSERT, you must lock the gap into which the new row would be inserted. 3. Gap locks are taken on index records — InnoDB locks the gap between consecutive index entries. Under Repeatable Read, range scans automatically take gap locks on the searched range. 4. Tradeoff: gap locks reduce concurrency significantly. Two concurrent INSERTs into the same range can block even if they're on totally different rows. That's why MySQL also offers Read Committed which disables gap locks.

Edge cases to mention: - Gap locks only exist on unique-index range scans and non-unique index lookups; primary-key point lookups don't take them. - `SELECT ... FOR UPDATE` on a range under Repeatable Read takes gap + record locks across the entire range. Heavy contention risk. - `READ COMMITTED` + `innodb_locks_unsafe_for_binlog` (now deprecated naming) disables gap locks but sacrifices phantom prevention.

What NOT to say: - Do not say "gap locks prevent updates" — they prevent INSERTs into the gap, updates of existing rows are governed by row locks. - Do not claim Postgres has gap locks — Postgres uses MVCC + predicate locks (Q19) for the same problem.

Common follow-up: "Why did my INSERT block another INSERT on a totally different value?" — Likely a gap lock from a range SELECT FOR UPDATE in the first txn. Switch to Read Committed or narrow the SELECT.

```
-- MySQL InnoDB under default Repeatable Read
START TRANSACTION;
SELECT * FROM orders WHERE amount BETWEEN 1000 AND 5000 FOR UPDATE;
-- Other sessions cannot INSERT orders with amount in [1000, 5000] until commit
COMMIT;
```

Q231 · Next-Key Locks (MySQL InnoDB)

Tags: Difficulty: Hard · Asked at: MySQL-heavy shops, Flipkart, PhonePe, Swiggy · Topic: Locking · Dialect: MySQL InnoDB

The question (interviewer's verbatim phrasing):

What is a next-key lock in InnoDB and how does it relate to gap locks?

The 30-second answer: A next-key lock is the combination of a record lock on a row plus a gap lock on the gap immediately before it. It's the default lock mode for InnoDB index scans under Repeatable Read, providing phantom-read prevention plus row exclusion in one unit.

Step-by-step thought process: 1. Mental model: imagine the index entries as `[5, 10, 20, 30]`. A next-key lock on the entry `20` covers the row `20` AND the gap `(10, 20)` — i.e., everything from just after 10 up to and including 20. 2. When you `SELECT ... WHERE x > 15 FOR UPDATE`, InnoDB takes next-key locks on `20` and `30` — that's row 20, gap (10,20), row 30, gap (20,30), plus the supremum gap (30, +∞) so no INSERT > 30 either. 3. Compared to plain gap locks alone, next-key locks let one operation atomically lock both the row and its predecessor gap, which is what you actually need for range scans. 4. Disabled in Read Committed (no gap component, just record locks remain).

Edge cases to mention: - Next-key locking on unique indexes with exact match degenerates to just the record lock (no gap needed — uniqueness already prevents new identical row). - In replication-heavy setups, next-key locks were once required for statement-based binlog correctness; row-based binlog relaxes this. - Visible in `SHOW ENGINE INNODB STATUS` as `lock_mode X locks gap before rec` and similar.

What NOT to say: - Do not say next-key = next row lock — it specifically includes the preceding gap. - Do not claim it's unique to MySQL — the concept exists in other engines under different names but is most explicit in InnoDB.

Common follow-up: *"Why does a SELECT FOR UPDATE on a non-existent row block other writers?"* — Even though the row doesn't exist, InnoDB takes a gap lock on where it would be, preventing concurrent INSERT of that key. This is what makes `INSERT ... ON DUPLICATE KEY UPDATE` correct against concurrent inserts.

```
-- InnoDB Repeatable Read; next-key locks across the range
START TRANSACTION;
SELECT * FROM orders WHERE id > 5000 FOR UPDATE;
-- Locks rows AND gaps; no new row with id > 5000 can be inserted
COMMIT;
```

Q232 · Predicate Locks (Postgres SSI)

Tags: Difficulty: Hard · Asked at: Postgres shops, Razorpay, Cred, Database teams · Topic: Locking · Dialect: Postgres

The question (interviewer's verbatim phrasing):

Postgres claims Serializable without taking strict locks like InnoDB does. How does that work?

The 30-second answer: Postgres uses Serializable Snapshot Isolation (SSI), which builds on MVCC by tracking lightweight predicate locks — records of which ranges a transaction read. At commit time, if the system detects a dangerous read-write dependency cycle among active txns, one is aborted with a serialization failure. No range gap locks at write time.

Step-by-step thought process: 1. MVCC alone gives Repeatable Read snapshot consistency but allows write skew (Q from Part A). SSI adds detection. 2. Predicate locks (not real exclusive locks): for each SELECT in a Serializable txn, Postgres records the predicate (often as page-level read marks). They don't block anyone — they're metadata. 3. When two transactions form a "dangerous structure" — A reads what B wrote and B reads what A wrote — SSI aborts one with SQLSTATE 40001 (`serialization_failure`). 4. The cost is occasional spurious aborts, which the application must retry. The benefit is no blocking at write time — high concurrency for read-heavy Serializable workloads.

Edge cases to mention: - SSI's predicate-lock memory is bounded; very long transactions can cause Postgres to escalate to page-level or relation-level conflict tracking, increasing false aborts. - Read-only optimisation: Postgres detects truly read-only Serializable txns and skips overhead entirely. - Not all dialects with "Serializable" do SSI — Oracle's Serializable is closer to Snapshot Isolation, InnoDB's adds explicit range locks.

What NOT to say: - Do not say Postgres Serializable is "lock-free" — it tracks reads via predicate-lock structures, just not blocking ones. - Do not promise zero aborts — applications must retry on 40001.

Common follow-up: *"How would my application code handle a Serializable abort?"* — Catch SQLSTATE 40001, sleep briefly with jitter, and re-run the entire transaction. Idempotency at the application boundary is critical.

```
-- Serializable txn; application must be ready for 40001 retry
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(amount) FROM payments WHERE status = 'pending';
INSERT INTO ledger_entries (...) VALUES (...);
COMMIT;
```

Q233 · Lock-Free Reads vs Locking Reads — MVCC Default

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, PhonePe, Walmart Labs · Topic: MVCC

The question (interviewer's verbatim phrasing):

In an MVCC database, when does a SELECT take a lock versus when is it lock-free?

The 30-second answer: Plain SELECT (no FOR clause) is lock-free under MVCC — it reads the snapshot, takes no row locks, blocks no writers. Locking reads happen only when you explicitly ask: `FOR UPDATE`, `FOR SHARE`, `FOR NO KEY UPDATE`, `FOR KEY SHARE` (Postgres), or `LOCK IN SHARE MODE` (MySQL).

Step-by-step thought process: 1. The lock-free default is the whole point of MVCC — reads never block writes. 2. Locking reads exist for the read-check-then-write idiom where you need to make sure the row doesn't change between SELECT and UPDATE. 3. Important caveat: at the table level, even plain SELECT takes a weak `ACCESS SHARE` (Postgres) / IS (InnoDB) intent lock — it blocks DROP/ALTER but nothing else. 4. SQL Server's situation differs by isolation: with Read Committed (legacy default) reads take short S locks; with Read Committed Snapshot (RCSI) reads are MVCC and lock-free.

Edge cases to mention: - `SELECT ... FOR UPDATE NOWAIT` returns immediately with an error instead of blocking — useful for "try to lock" UX. - `SELECT ... FOR UPDATE SKIP LOCKED` skips

already-locked rows — ideal for worker queues. - Even lock-free reads can be slow if they have to chase long version chains (high history-list / many dead tuples).

What NOT to say: - Do not say all SELECTs are lock-free — explicit locking modes exist. - Do not forget the weak table-level intent.

Common follow-up: *"What stops a worker from picking the same job twice in a SKIP LOCKED queue?"*
— The row-level lock from the first worker is incompatible; second worker's SKIP LOCKED simply moves past it. Atomic, no race.

```
-- Postgres worker queue picking one job, skipping locked ones
BEGIN;
SELECT id, payload FROM job_queue
WHERE status = 'pending'
ORDER BY priority, id
FOR UPDATE SKIP LOCKED LIMIT 1;
-- process the job, UPDATE status = 'done', COMMIT
```

Q234 · Pessimistic vs Optimistic Concurrency

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Swiggy, Goldman · Topic: Concurrency

The question (interviewer's verbatim phrasing):

What's the difference between pessimistic and optimistic concurrency control, and when would you pick each?

The 30-second answer: Pessimistic locks the row up front (e.g., `SELECT FOR UPDATE`), assuming a conflict is likely; optimistic doesn't lock but checks a version number at commit time, assuming conflicts are rare. Pessimistic is safer under high contention but reduces throughput; optimistic is faster under low contention but forces the application to handle retries.

Step-by-step thought process: 1. Pessimistic pattern: open txn, `SELECT ... FOR UPDATE`, perform business logic, `UPDATE`, commit. Lock held for the duration of business logic — bad if logic is slow. 2. Optimistic pattern: `SELECT` with current version, do logic, then `UPDATE ... WHERE id = ? AND version = ?` with `SET version = version + 1`. If row count is zero, someone else updated — abort and retry. 3. Picking: high-contention hot rows (wallet of a celebrity) → pessimistic to avoid retry storms. Low-contention many-row updates (user profile edits) → optimistic to avoid lock-manager overhead. 4. Hybrid: optimistic first, fall back to pessimistic when retry rate exceeds a threshold.

Edge cases to mention: - Optimistic doesn't prevent lost updates in the database — the WHERE clause does. If the WHERE matches zero rows, the application must detect and retry. - Pessimistic over a slow

network is brutal — round-trip latency multiplied by lock-hold time means low throughput. - Optimistic with a too-coarse version (whole row hash) gives false conflicts on independent column updates.

What NOT to say: - Do not say one is universally better. - Do not forget the application-level retry loop for optimistic.

Common follow-up: "How do you implement optimistic locking in SQL?" — Version column or last_updated timestamp; UPDATE WHERE id AND version, check affected-rows. Covered in Q22.

```
-- Optimistic update: only succeeds if version matches
UPDATE accounts
SET balance = balance - 500, version = version + 1
WHERE id = 7001 AND version = 12;
-- if affected_rows = 0, the row changed under us - retry
```

Q235 · Optimistic Locking — Version Column Pattern (OCC)

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Flipkart, GCC mid · Topic: Concurrency

The question (interviewer's verbatim phrasing):

Implement optimistic concurrency control with a version column. Walk me through both the schema and the update statement.

The 30-second answer: Add an integer `version` column to the row. On every UPDATE, increment it and include the old version in the WHERE. If the affected row count is zero, someone else won the race — abort and retry the whole read-modify-write cycle.

Step-by-step thought process: 1. Schema: `ALTER TABLE accounts ADD COLUMN version INT NOT NULL DEFAULT 0;` 2. Read: `SELECT id, balance, version FROM accounts WHERE id = 7001;` — capture version (say, 12). 3. Update: `UPDATE accounts SET balance = balance - 500, version = version + 1 WHERE id = 7001 AND version = 12;` — driver returns affected-row count. 4. Branch on affected: 1 → success, 0 → conflict, re-read and retry up to N times with jittered backoff.

Edge cases to mention: - Use INT BIGINT or a UUID for the version if you fear overflow over decades of frequent updates. - A timestamp column is convenient but loses to clock skew on multi-master setups; integer counter is safer. - Frameworks like Hibernate / JPA do this automatically with `@Version` — but the application still must handle `OptimisticLockException`.

What NOT to say: - Do not skip the version-increment in the SET clause — without it the next conflict can't be detected. - Do not retry infinitely — cap attempts and surface error to user.

Common follow-up: "What if the user's edit and someone else's edit don't actually conflict — different columns?" — With a single row-level version, you still get a false conflict. For finer detection, partition fields into independent versioned groups or use per-column timestamps.

```
CREATE TABLE accounts (  
  id BIGINT PRIMARY KEY,  
  balance NUMERIC(18,2) NOT NULL,  
  version INT NOT NULL DEFAULT 0  
);  
  
UPDATE accounts  
SET balance = balance - 500, version = version + 1  
WHERE id = 7001 AND version = 12;  
-- returns 1 if won, 0 if someone else updated since we read
```

Q236 · Pessimistic Locking — SELECT FOR UPDATE Pattern

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, TCS, Infosys · Topic: Concurrency

The question (interviewer's verbatim phrasing):

Show me the canonical pessimistic locking pattern in SQL — a wallet debit with concurrency safety.

The 30-second answer: Inside a transaction, `SELECT ... FOR UPDATE` the wallet row to take a row-exclusive lock, perform the balance check in application code, then `UPDATE` and commit. Other concurrent debits wait for the lock to release.

Step-by-step thought process: 1. Open transaction. Outside a txn the FOR UPDATE lock is released immediately. 2. SELECT FOR UPDATE on the wallet row. Blocks competing FOR UPDATES on the same row but lets unrelated rows proceed. 3. Application checks balance $\geq$ debit amount. If not, ROLLBACK and surface insufficient-funds to user. 4. UPDATE the row, COMMIT — releases the lock for the next waiter.

Edge cases to mention: - Always include a `lock_timeout` (Postgres) or `innodb_lock_wait_timeout` (MySQL) — never wait forever. - For multi-row debits in a fixed order to avoid deadlocks (e.g., always lock lower-id wallet first when transferring). - Pair with `SELECT FOR NO KEY UPDATE` if foreign-key blocking from child tables is causing surprise stalls (Q7).

What NOT to say: - Do not forget the BEGIN — the most common bug. - Do not perform slow network calls between SELECT FOR UPDATE and COMMIT — you'll block everyone.

Common follow-up: *"What if the debit logic takes 500 ms because of an external fraud-check API call?"*
 — Move the fraud check before the FOR UPDATE, or take the lock briefly only around the balance-and-update, accepting a small risk window mitigated by idempotency.

```
BEGIN;
SELECT balance FROM wallets WHERE user_id = 91234 FOR UPDATE;
-- application: check balance >= 500, abort otherwise
UPDATE wallets SET balance = balance - 500 WHERE user_id = 91234;
INSERT INTO ledger (user_id, amount, type) VALUES (91234, -500, 'DEBIT');
COMMIT;
```

Q237 · Advisory Locks (Postgres pg_advisory_lock)

Tags: Difficulty: Hard · Asked at: Postgres shops, Razorpay, Cred, Cron-job heavy backends · Topic: Locking · Dialect: Postgres

The question (interviewer's verbatim phrasing):

What is a Postgres advisory lock, and when would you use one instead of a row lock?

The 30-second answer: Advisory locks are application-defined locks identified by an arbitrary integer key, with no row or table tied to them. Useful for mutual exclusion outside the data model — e.g., "only one process runs the nightly billing job," "only one consumer can process this user's events," "claim distributed leadership without an extra service."

Step-by-step thought process: 1. Two flavours: session-scoped (`pg_advisory_lock(key)` — held until explicit unlock or session end) and transaction-scoped (`pg_advisory_xact_lock(key)` — released at txn end). 2. Two variants per flavour: blocking (`pg_advisory_lock`) and try (`pg_try_advisory_lock` — returns immediately with boolean). 3. Keys can be one bigint or two ints (good for namespacing — `pg_advisory_lock(MY_APP_NS, user_id)`). 4. Visible in `pg_locks` with locktype = 'advisory' — easy to monitor.

Edge cases to mention: - Session locks survive transaction boundaries — easy to leak. Always unlock or rely on session end. - The key is just an integer; collisions between unrelated app features are possible if you don't namespace. - For distributed leadership use cases, advisory locks beat Zookeeper/etcd if Postgres is already on the critical path.

What NOT to say: - Do not confuse advisory locks with row locks — they have no FK / row relationship. - Do not use them as a substitute for FOR UPDATE on data rows.

Common follow-up: *"How would you make sure a cron job runs on exactly one machine?"* — Every box tries `pg_try_advisory_lock(unique_job_key)` . Only one succeeds, runs the job, releases. Others retry next tick.

```
-- Distributed mutual exclusion for a cron
SELECT pg_try_advisory_lock(42); -- returns true on first caller
-- ... run the job ...
SELECT pg_advisory_unlock(42);
```

Q238 · LOCK TABLE — When (Almost Never) to Use Explicitly

Tags: Difficulty: Medium · Asked at: GCC senior rounds, Migration-heavy teams · Topic: Locking

The question (interviewer's verbatim phrasing):

When would you use an explicit LOCK TABLE statement?

The 30-second answer: Almost never in OLTP. Legitimate uses are one-time migrations that need a brief moment of exclusive access, fixing data corruption, or workflows where you genuinely need the whole table held still (rare). For normal operations, row-level locks via FOR UPDATE are the right tool.

Step-by-step thought process: 1. Postgres syntax: `LOCK TABLE accounts IN ACCESS EXCLUSIVE MODE;` — blocks every operation including SELECT. Eight modes available, picking the weakest needed is critical. 2. MySQL: `LOCK TABLES accounts WRITE;` — but in InnoDB this is usually unnecessary; row locks handle everything. 3. Use case: dual-write migration window — briefly LOCK TABLE in EXCLUSIVE while you flip a feature flag, then release. 4. Risk: any LOCK TABLE in production with no timeout becomes a stop-the-world incident if a previously-acquired weaker lock holder is slow to release.

Edge cases to mention: - `LOCK TABLE ... NOWAIT` exists — returns error instead of waiting. Always use it. - Postgres `LOCK TABLE` automatically takes the named mode plus all weaker modes — picking ACCESS EXCLUSIVE blocks every read. - DDL like `ALTER TABLE` takes ACCESS EXCLUSIVE implicitly; that's the actual common case of "locking the whole table" without writing LOCK TABLE.

What NOT to say: - Do not say it's a normal performance tuning tool. - Do not skip the NOWAIT — production stalls are how you learn this.

Common follow-up: *"Why is online schema change tooling (gh-ost, pt-online-schema-change, pg_repack) a thing if we have LOCK TABLE?"* — Because LOCK TABLE for an hour-long migration is unacceptable. The tools rewrite the table in shadow form and swap atomically with a sub-second lock window.

```
-- Postgres: explicit lock with timeout to avoid runaway
BEGIN;
SET lock_timeout = '5s';
LOCK TABLE accounts IN ACCESS EXCLUSIVE MODE;
-- brief migration step
COMMIT;
```

Q239 · Waiting on Lock — lock_timeout, deadlock_timeout, innodb_lock_wait_timeout

Tags: Difficulty: Medium · Asked at: GCC senior rounds, On-call interviews, Razorpay, Cred · Topic: Locking

The question (interviewer's verbatim phrasing):

What knobs control how long a transaction waits for a lock, and what should the defaults be for an OLTP application?

The 30-second answer: Postgres has `lock_timeout` (per-statement wait limit) and `deadlock_timeout` (how long to wait before running deadlock detection). MySQL InnoDB has `innodb_lock_wait_timeout` (default 50 seconds — way too high for OLTP). SQL Server has `LOCK_TIMEOUT` (per-session, default infinite). For OLTP, set lock waits to a few seconds at most.

Step-by-step thought process: 1. Postgres `lock_timeout` (per-session): hard cap on how long any one statement waits for any lock. Reasonable default for OLTP: 5-10 seconds. 2. Postgres `deadlock_timeout` (server-wide default 1s): only triggers the deadlock-detection algorithm after this delay — the wait itself can continue indefinitely if no deadlock found. Lowering it costs CPU; raising it delays deadlock victim selection. 3. InnoDB `innodb_lock_wait_timeout` (default 50s, way too long for web apps): set to 5-10 seconds. Per-session override available. 4. SQL Server `SET LOCK_TIMEOUT 5000;` (5 seconds). Default `-1` waits forever — avoid in OLTP.

Edge cases to mention: - Lock timeout returns an error to the application; the txn must decide to retry or abort. - Statement timeout (`statement_timeout` in Postgres) is a stronger umbrella — kills the whole statement regardless of cause. - Deadlock detection is automatic in every major engine — one victim is rolled back with a specific error code. Application must retry.

What NOT to say: - Do not say "raise the timeout to fix the problem" — almost always wrong; fix the contention instead. - Do not confuse lock timeout (waiting for lock) with statement timeout (overall runtime cap).

Common follow-up: "What error code does Postgres return on lock timeout vs deadlock?" — 55P03 (lock_not_available) for lock timeout, 40P01 (deadlock_detected) for deadlock. Catch and retry separately.

```
-- Postgres: per-session lock and statement timeouts
SET lock_timeout = '5s';
SET statement_timeout = '30s';
```

Q240 · Diagnosing Lock Waits + Connection Pool Pitfalls

Tags: Difficulty: Hard · Asked at: GCC senior rounds, On-call, Razorpay, Cred, Flipkart, PhonePe · Topic: Operations

The question (interviewer's verbatim phrasing):

A user reports the app feels stuck. You suspect lock waits. How do you investigate in Postgres, MySQL, and SQL Server — and what connection-pool gotcha do you watch for?

The 30-second answer: Postgres: query `pg_stat_activity` joined with `pg_locks` to find the blocking chain. MySQL: query `INFORMATION_SCHEMA.INNODB_TRX` and `performance_schema.data_locks` / `data_lock_waits`. SQL Server: `sys.dm_tran_locks` joined with `sys.dm_exec_requests`. Connection-pool pitfall: a transaction left open by one request because the pool returns the connection without rollback — the next request reuses it and the lock leaks across users.

Step-by-step thought process: 1. Postgres recipe: a query joining `pg_stat_activity` with `pg_locks` and `pg_blocking_pids` gives you the blocked-blocker pairs with txn ages and SQL text. 2. MySQL 8 recipe: `performance_schema.data_lock_waits` is the modern equivalent of the old `INNODB_LOCK_WAITS`; join with `INNODB_TRX` for txn metadata. 3. SQL Server recipe: `sys.dm_os_waiting_tasks` filtered by `wait_type` like 'LCK%', joined to `sys.dm_exec_requests` for SQL text. Or `sp_who2` for a quick look. 4. Connection-pool pitfall: many pools (HikariCP, pgbouncer in session mode) don't automatically rollback on connection return. If application code commits in some branches and not others, an open txn rides along to the next user. Symptoms: unrelated users start blocking each other, locks held by sessions whose `query` field shows the unrelated user's SQL.

Edge cases to mention: - `pg_terminate_backend(pid)` is the nuclear option — kills the session and rolls back its txn. - pgbouncer in transaction mode is safer for leaked txns since it returns the backend connection only between transactions, not within one. - Application frameworks often have a "begin transaction on request, commit/rollback at end" wrapper — the bug is usually code path that bypasses the wrapper.

What NOT to say: - Do not say "just restart the database" — masks the issue and is heavyweight. - Do not forget the pool-side defensive **RESET** / **DISCARD ALL** (Postgres) on connection return.

Common follow-up: "Write a Postgres query to show me the blocker chain right now." — Practise this. It's a five-table join but the snippet below is close to memorisable.

```
-- Postgres: who is blocking whom right now
SELECT blocked.pid AS blocked_pid,
       blocked.query AS blocked_query,
       blocking.pid AS blocking_pid,
       blocking.query AS blocking_query,
       age(now(), blocked.xact_start) AS blocked_age
FROM pg_stat_activity AS blocked
JOIN pg_stat_activity AS blocking
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.wait_event_type = 'Lock';
```

Chapter 5, Part C — Deadlocks & Transaction Conflicts

This is the closer chapter on transactions, isolation and locking — the bit where theory meets the production pager. Parts A and B taught you what isolation levels promise and how lock managers work; Part C is about what breaks when two transactions want the same rows at the same time. We will walk through deadlock formation, the wait-for graph that engines use to detect cycles, the lock-ordering discipline that prevents most of them, and the retry-with-backoff pattern that handles the rest. Then we will turn to real production conflict patterns I have actually debugged at Indian fintech and e-commerce shops: the two-direction wallet transfer, the hot invoice counter, the foreign-key insert storm, MERGE pitfalls on SQL Server, deferred constraints in Postgres, and the saga-versus-two-phase-commit choice when one database is no longer enough. Twenty-seven questions, and the last one is the comprehensive "talk me through deadlocks end-to-end" question that every senior backend panel eventually asks.

Q241 · What Exactly Is a Deadlock?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Walmart Labs · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Define a deadlock in your own words and draw me the classic two-resource cycle.

The 30-second answer: A deadlock is a state where two or more transactions are each waiting for a lock that the other holds, so none can make progress — a circular wait. The simplest case is Txn A holds

row X and wants row Y, while Txn B holds row Y and wants row X. Neither will release without committing, and neither can commit without the other releasing. The engine has to break the cycle by killing one transaction.

Step-by-step thought process: 1. Lead with the precise definition — "circular wait for resources" — because that is the textbook phrasing the interviewer is listening for. 2. Sketch the two-transaction cycle with concrete row IDs: Txn A locks accounts.id=1, Txn B locks accounts.id=2, then A asks for 2 and B asks for 1. Both block forever without intervention. 3. Make the consequence explicit: a deadlock is not slowness, it is a logical impossibility — waiting longer cannot resolve it. 4. Close with the engine's response: detect, pick a victim (usually the cheaper rollback), throw a deadlock error to the victim's session.

Edge cases to mention: - Deadlocks can involve three or more transactions in a longer cycle — A waits B, B waits C, C waits A. - Not every "stuck" query is a deadlock; a long lock wait that eventually times out is a different problem with a different fix. - Read-only transactions can still deadlock if they take share locks (SELECT FOR SHARE) or if they touch index entries that are being modified.

What NOT to say: - Do not say "the database deadlocks because it is overloaded." Load makes deadlocks more frequent but does not cause the cycle; bad lock ordering does. - Do not call a transaction that is simply slow a deadlock — vocabulary matters in an interview.

Common follow-up: *"How is a deadlock different from livelock?"* — Livelock is two transactions politely retrying and re-conflicting forever; both are running but neither finishes. Deadlock is a frozen cycle; livelock is a busy loop.

```
-- Classic two-row cycle on the accounts table
-- Session A
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1; -- holds row 1
UPDATE accounts SET balance = balance + 500 WHERE id = 2; -- waits for row 2

-- Session B (interleaved)
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 2; -- holds row 2
UPDATE accounts SET balance = balance + 500 WHERE id = 1; -- waits for row 1 -> deadlock
```

Q242 · How the Engine Detects a Deadlock

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Goldman · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Walk me through how a database actually figures out that two transactions are deadlocked.

The 30-second answer: The engine maintains a wait-for graph where each node is a transaction and a directed edge from A to B means "A is waiting for a lock held by B". A deadlock is, by definition, a cycle in this graph. Postgres scans for cycles when a wait has lasted longer than `deadlock_timeout` (default one second); MySQL InnoDB and SQL Server detect cycles immediately whenever a wait edge is added.

Step-by-step thought process: 1. Start with the data structure — wait-for graph — because that is the canonical algorithm and shows you understand the internals. 2. Describe the trigger differences: Postgres is lazy (waits the timeout first to avoid wasted CPU on short waits), InnoDB and SQL Server are eager (run the cycle check on every new wait). 3. Mention the victim choice heuristics: the engine picks whichever transaction is "cheaper" to roll back — fewer locks held, less log written, or in SQL Server the lowest deadlock priority. 4. Close with what the application sees: the victim's next statement (or commit) raises a deadlock error and the transaction is automatically rolled back.

Edge cases to mention: - Postgres's `deadlock_timeout` is a latency knob, not a correctness one — set it lower (say 200ms) for OLTP, higher for batch. - Distributed deadlocks (across two databases or microservices) are not detected by any single engine; you need timeouts or application-level coordination. - Some engines log the full deadlock graph; SQL Server can emit XML deadlock graphs to the `system_health` Extended Event session.

What NOT to say: - Do not say "the database polls for deadlocks." Eager detection runs only on lock-acquisition events. - Do not claim Postgres detects deadlocks instantly — the `deadlock_timeout` delay is the explicit trade-off.

Common follow-up: "What is the cost of lowering Postgres's `deadlock_timeout` to 100ms?" — More CPU spent running cycle checks for waits that would have resolved naturally; fine on contention-heavy systems, wasteful on quiet ones.

```
-- Inspect current value in Postgres
SHOW deadlock_timeout;      -- default '1s'

-- Tune for an OLTP workload that retries fast
SET deadlock_timeout = '200ms';  -- session-level
ALTER SYSTEM SET deadlock_timeout = '200ms';  -- cluster-wide (requires reload)
```

Q243 · Deadlock vs Lock Wait — Different Problems

Tags: Difficulty: Easy · Asked at: Infosys, Wipro, Flipkart, Swiggy · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

A query is hung for thirty seconds. How do you decide whether it is a deadlock or just a long lock wait, and why does that matter?

The 30-second answer: A deadlock is a circular wait that the engine resolves by killing one transaction with a specific deadlock error code. A long lock wait is a one-way wait that ends when the holder commits, rolls back, or hits the lock-wait timeout. Look at the error message and the wait state — different problems, different fixes.

Step-by-step thought process: 1. Frame it as two distinct failure modes: cycle (deadlock) versus chain (lock wait). The fixes have nothing in common. 2. Give the inspection commands: in Postgres, `pg_locks` plus `pg_stat_activity` shows who is waiting on whom; in MySQL, `SHOW ENGINE INNODB STATUS` plus `performance_schema.data_locks`; in SQL Server, `sys.dm_tran_locks` and `sys.dm_os_waiting_tasks`. 3. Explain the time signature: deadlocks resolve in roughly `deadlock_timeout` on Postgres (one second by default) or immediately on InnoDB; long lock waits can stretch for minutes until the holder commits. 4. Fix-time: deadlocks need lock ordering or retry logic; lock waits need shorter transactions, lower isolation, or a covering index that avoids the lock altogether.

Edge cases to mention: - A long-running `SELECT FOR UPDATE` followed by a slow API call is the textbook cause of lock waits — keep network I/O out of transactions. - A deadlock under serializable isolation can appear as a serialization failure (SQLSTATE 40001) rather than a deadlock error (40P01) — Postgres differentiates them. - Lock-wait timeout (MySQL: `innodb_lock_wait_timeout`, default 50s) trips before any deadlock check if there is no cycle.

What NOT to say: - Do not lump them together. "It's a locking issue" is junior phrasing; the senior answer names the mode. - Do not assume retry will fix a long lock wait — if the holder never releases, you will just retry into the same wait.

Common follow-up: "What is the SQLSTATE for a deadlock and what is it for a serialization failure?" — `40P01` is deadlock detected, `40001` is serialization failure; both are retryable but they signal different conditions.

```
-- Postgres: who is blocking whom right now?
SELECT blocked.pid      AS blocked_pid,
       blocked.query     AS blocked_query,
       blocking.pid      AS blocking_pid,
       blocking.query    AS blocking_query
FROM   pg_stat_activity AS blocked
JOIN   pg_stat_activity AS blocking
      ON blocking.pid = ANY(pg_blocking_pids(blocked.pid));
```


Q244 · Reading a Deadlock Error Message

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Wells Fargo, AmEx · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Postgres has just thrown `ERROR: deadlock detected` at you with a multi-line `DETAIL` block. What are you reading for?

The 30-second answer: You are reading for the two process IDs, the relations and rows involved, and the statements each transaction was running when the cycle closed. That tells you exactly which two code paths in your application interleaved badly so you can fix the lock order.

Step-by-step thought process: 1. Start by naming the four key fields: process IDs, lock mode, relation (table or index), and the offending tuple or page identifier. 2. Map each PID back to your application — log connection identifiers or `application_name` so you can correlate the database PID with the request that fired it. 3. Note which transaction was chosen as the victim (Postgres logs "canceling statement due to deadlock"); the other one survived and committed. 4. The fix is almost always upstream: the two surviving statements show you the inconsistent lock order — sort the rows you touch by primary key before updating them.

Edge cases to mention: - MySQL's `SHOW ENGINE INNODB STATUS` LATEST DETECTED DEADLOCK section dumps the same information but only keeps the most recent deadlock — capture it before the next one overwrites. - SQL Server XML deadlock graphs in Extended Events give you the wait-for cycle, lock resources, and the call stack of each session — far richer than the plain error. - Always log the full `DETAIL` block; the one-line `ERROR` alone tells you nothing.

What NOT to say: - Do not say "deadlocks are random" — the error message has the deterministic cause inside it. - Do not paper over the error with a blind retry without first reading the message; the deadlock will come back.

Common follow-up: *"You see the same two tables in every deadlock message for the last week. What does that tell you?"* — A consistent ordering bug between two code paths; fix the lock order once and the recurring deadlock disappears.

```
-- Sample Postgres deadlock error
-- ERROR:  deadlock detected
-- DETAIL: Process 18421 waits for ShareLock on transaction 9921;
--          blocked by process 18422.
--          Process 18422 waits for ShareLock on transaction 9920;
--          blocked by process 18421.
-- HINT:   See server log for query details.
-- CONTEXT: while updating tuple (5,3) in relation "accounts"
```

Q245 · The Classic Two-Transfer Deadlock

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred, Zerodha · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Two users hit "send money" at the same instant — Anjali sends one thousand rupees to Rahul, and Rahul sends one thousand to Anjali. Walk me through how this deadlocks.

The 30-second answer: Transfer one locks Anjali's row first, then asks for Rahul's. Transfer two locks Rahul's row first, then asks for Anjali's. Each holds what the other needs — perfect cycle. The fix is to always lock the lower-numbered account ID first regardless of the direction of the transfer.

Step-by-step thought process: 1. Diagram the interleaving: T1 updates accounts.id=101 (Anjali), T2 updates accounts.id=247 (Rahul), then T1 wants 247 and T2 wants 101. 2. Explain why row-level locks make this so easy to hit — both transactions hold an exclusive lock on the row they debited and need an exclusive lock on the row they are crediting. 3. Show the fix in code: before any UPDATE, sort the two account IDs ascending and lock them in that order with `SELECT ... FOR UPDATE`. Now both transactions take row 101 first, the second waits, no cycle. 4. Mention the alternative — wrap the whole transfer in a SERIALIZABLE transaction and let the engine retry — but lock ordering is cheaper and more predictable.

Edge cases to mention: - The same problem appears in inventory transfers between warehouses, ledger postings between general-ledger accounts, and message-passing between conversation threads. - If you have three-leg transfers (A → B → C in one transaction), you still sort all three IDs and lock in ascending order. - A transfer from an account to itself is a real edge case — guard with `IF from_id = to_id THEN RAISE EXCEPTION`.

What NOT to say: - Do not say "we should use higher isolation level" as the first fix. SERIALIZABLE just turns the deadlock into a serialization failure — same retry needed. - Do not claim ORM magic will save you; Hibernate and ActiveRecord will deadlock just as cheerfully if you don't sort.

Common follow-up: "Show me the lock-order pattern in actual SQL." — Sort by primary key, SELECT FOR UPDATE in that order, then run the debits and credits.

```
-- Safe transfer with consistent lock order
BEGIN;
-- always lock the lower id first, regardless of direction
SELECT id FROM accounts
WHERE id IN (101, 247)
ORDER BY id
FOR UPDATE;

UPDATE accounts SET balance = balance - 1000 WHERE id = 101;
UPDATE accounts SET balance = balance + 1000 WHERE id = 247;

INSERT INTO transfers(from_id, to_id, amount) VALUES (101, 247, 1000);
COMMIT;
```

Q246 · Lock Ordering as the Universal Prevention

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Walmart Labs, Target · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

If you had to give one rule to a junior engineer to stop most deadlocks, what would it be?

The 30-second answer: Always acquire locks in the same global order — usually ascending by primary key, or alphabetical by table name when multiple tables are involved. If every transaction grabs resources in the same direction, no two transactions can form a cycle, so deadlocks are mathematically impossible.

Step-by-step thought process: 1. State the principle: cycles need at least two transactions going in different directions; force them all the same way and the cycle cannot form. 2. Concrete recipe — within a transaction touching multiple rows of one table, sort the IDs and lock with **FOR UPDATE** in that order. Across multiple tables, fix an alphabetical or dependency order (e.g. always users before orders, never the reverse). 3. Mention the code-review checklist: any code path that updates two rows of the same table without sorting them is a future deadlock. 4. Close with the limit: lock ordering prevents within-database deadlocks but not distributed ones across services.

Edge cases to mention: - If you cannot sort (e.g., you don't know the second ID until midway through the transaction), wrap in a try/retry block and accept occasional deadlocks. - For a graph of interdependent tables, draw the dependency DAG once and write the order into a coding standard. - Application-level lock managers (e.g., Redis Redlock) can enforce a global order across services where the database cannot.

What NOT to say: - Do not say "increase isolation to fix deadlocks" — higher isolation usually makes them more frequent, not less. - Do not rely on lock-free patterns alone; some operations truly need multi-row locking and ordering is the only safe option.

Common follow-up: "What if the two rows you need to lock are in different tables — how do you sort?"

— Establish a fixed table order in the team's coding standard (e.g., always lock `accounts` before `transfers`) and sort by primary key within each table.

```
-- Touching two tables: fixed order accounts -> transfers
BEGIN;
SELECT id FROM accounts WHERE id IN (:src, :dst) ORDER BY id FOR UPDATE;
SELECT id FROM transfers WHERE id = :reversal_id FOR UPDATE;
-- now safe to mutate either table in any order
COMMIT;
```

Q247 · The Retry-with-Backoff Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Cred · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Your code deadlocks one out of every thousand transactions. How do you handle it cleanly?

The 30-second answer: Wrap the whole transaction in a retry loop that catches the deadlock error (SQLSTATE 40P01 on Postgres, 1213 on MySQL, 1205 on SQL Server), waits with exponential backoff plus jitter, and re-runs the entire transaction. Cap retries at three to five; if it still fails, bubble the error up as a real failure. The retried transaction must be idempotent so a partial earlier attempt doesn't corrupt state.

Step-by-step thought process: 1. Open with the contract: deadlocks are recoverable errors, and the engine has already rolled back the victim — your job is to start a new transaction and try again. 2. Show the loop structure: catch the specific error code, sleep `base * 2^attempt + jitter` milliseconds, retry the whole transaction (not just the failing statement). 3. Emphasise idempotency: external side effects (emails, API calls) must not be inside the transaction, or use an outbox table so the side effect happens only after the eventual commit. 4. Cap attempts and alert on the cap — if you retry forever, you hide a real lock-ordering bug behind a busy loop.

Edge cases to mention: - Jitter (random 0-100ms added to each delay) prevents the two retrying transactions from colliding again on the exact same schedule. - Some ORMs (Sequel, ActiveRecord) ship a built-in `transaction(retry_on: DeadlockError)` helper — prefer the framework helper over hand-rolled loops. - A retry loop in a long-running batch job should commit more often, not retry the entire batch — minimise the rollback cost.

What NOT to say: - Do not retry on every error blindly — primary-key violations and check constraints will not get better with another try. - Do not retry without backoff — two stubborn retriers can livelock indefinitely.

Common follow-up: *"What is the right base delay and cap?"* — A base of 25-50ms and a cap of five retries works for OLTP; tune up if you see frequent collisions, alert if retries spike.

```
-- Pseudocode pattern; adapt to your language
-- for attempt in 1..5:
--   try:
--     BEGIN; ...; COMMIT; return success
--   except DeadlockDetected:
--     sleep(25ms * 2^attempt + random(0,100ms))
--   raise PermanentDeadlockError(after 5 attempts)
```

Q248 · Page-Level Deadlocks

Tags: Difficulty: Hard · Asked at: JP Morgan, Goldman, Walmart Labs, AmEx · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

You insist your code locks different rows, but the deadlock log shows a conflict. What is going on?

The 30-second answer: The engine takes locks at multiple granularities — row, page, index entry, sometimes table. Two transactions touching different rows can still collide on the same B-tree page or index leaf, and the deadlock surfaces as a page-level lock cycle. Look at the lock resource in the deadlock graph; if it says **page** or **key** rather than **row**, you have a granularity issue, not a row-collision bug.

Step-by-step thought process: 1. Define the granularity hierarchy: row, page, extent, table — engines may escalate up the hierarchy to save memory in the lock manager. 2. SQL Server is particularly known for escalating row locks to page or table locks once a transaction touches more than five thousand rows; that escalation is a deadlock magnet. 3. Postgres rarely takes page locks for DML but does take them on index pages during splits; bulk loaders into the same index can collide. 4. Diagnose by reading the deadlock graph's resource type; fix by lowering granularity (e.g., **WITH (ROWLOCK)** hint in SQL Server, smaller batch sizes in Postgres) or by reordering the order rows are touched.

Edge cases to mention: - Heap tables in SQL Server escalate locks aggressively; adding a clustered index often reduces page-level deadlocks. - MySQL InnoDB uses gap locks at REPEATABLE READ, which lock ranges between index entries — two inserters near the same gap can deadlock. - Lock escalation thresholds are tunable on SQL Server (**ALTER TABLE ... SET (LOCK_ESCALATION = DISABLE)**) but disabling escalation can blow up memory.

What NOT to say: - Do not insist "the rows are different so it cannot deadlock." Granularity changes everything. - Do not blanket-disable lock escalation in production without understanding the memory cost.

Common follow-up: *"How would you reproduce a page-level deadlock for testing?"* — Bulk-insert into the same index leaf from two sessions with identical batch sizes; the gap-lock collision shows up reliably under REPEATABLE READ.

```
-- SQL Server: read the deadlock resource type
SELECT XEvent.value('(data/value)[1]', 'varchar(max)') AS deadlock_xml
FROM (SELECT CAST(target_data AS XML) AS TargetData
      FROM sys.dm_xe_session_targets st
      JOIN sys.dm_xe_sessions s ON s.address = st.event_session_address
      WHERE s.name = 'system_health' AND st.target_name = 'ring_buffer') AS Data
CROSS APPLY TargetData.nodes('//RingBufferTarget/event[@name="xml_deadlock_report"]') AS XEventData
```

Q249 · Foreign-Key Insert Deadlocks

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Goldman · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Two transactions are inserting child rows that reference different parent rows. Why do they sometimes deadlock?

The 30-second answer: When you insert a child row, the engine takes a share-lock on the referenced parent row to make sure it cannot be deleted mid-transaction. If two transactions each insert children referencing two parents in opposite order — Txn A child of P1 then P2, Txn B child of P2 then P1 — the parent share locks form the exact same cycle as a row-update deadlock.

Step-by-step thought process: 1. Explain the implicit lock: FK enforcement takes a share lock on the parent to prevent concurrent delete; this lock is invisible in application code but very real. 2. Show the cycle: inserts that reference multiple parents in different orders deadlock just like updates. Common in order-line tables (each line references a product), in junction tables, and in audit-log children. 3. The fix mirrors the transfer case: sort the parent IDs and either touch them in order or take an explicit **SELECT FOR KEY SHARE** (Postgres) before the inserts. 4. Alternative fix: defer constraint checks until commit (deferrable FKs in Postgres), which moves the lock to commit time and reduces contention.

Edge cases to mention: - Postgres uses a weaker lock mode called **FOR KEY SHARE** for FK enforcement since version 9.3 — older patterns assumed **FOR SHARE** and over-locked. - MySQL InnoDB takes a shared lock on the parent and historically deadlocked very easily; modern versions

added the `innodb_lock_wait_timeout` short-circuit. - If the parent table is updated frequently (e.g., a counter on each parent), FK checks plus those updates collide constantly.

What NOT to say: - Do not say "we should drop the foreign keys" as a deadlock fix — you trade a deadlock for orphan rows, which is far worse in finance. - Do not assume FK locks are free; they are real locks held until commit.

Common follow-up: "*How do deferrable constraints help?*" — They postpone the FK check (and the lock) until COMMIT, so two transactions can both insert children, both check parents at commit, and only collide if the parents are actually conflicting.

```
-- Postgres deferrable FK
ALTER TABLE order_lines
  ADD CONSTRAINT fk_product
  FOREIGN KEY (product_id) REFERENCES products(id)
  DEFERRABLE INITIALLY DEFERRED;

BEGIN;
SET CONSTRAINTS fk_product DEFERRED; -- already deferred, shown for clarity
INSERT INTO order_lines(order_id, product_id, qty) VALUES (1, 100, 2);
INSERT INTO order_lines(order_id, product_id, qty) VALUES (1, 101, 1);
COMMIT; -- FK checked here, lock held only briefly
```

Q250 · Index-Level Deadlocks

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Target · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

A team complains their UPDATE deadlocks even though they only touch one row. How can a single-row update deadlock?

The 30-second answer: The row is one resource, but the indexes that point to it are separate resources. Updating a row that participates in two indexes takes locks on both index entries, and another transaction updating a different row that happens to live in the same index leaf can collide. The cycle forms across indexes, not across rows.

Step-by-step thought process: 1. Reinforce the multi-resource view: a row update is "update the heap tuple plus every index that contains the updated columns." Each index lock is independent. 2. Walk through a sample cycle: Txn A updates accounts(id=10, status=...) — locks heap row plus status index entry. Txn B updates accounts(id=11, status=...) — locks heap row plus an adjacent index entry on the same page. Page-level lock manager hands them in opposite order: cycle. 3. Symptoms in the deadlock graph: the resource is `index` or `key` rather than `row`. Look for it explicitly. 4. Mitigations: drop

unused indexes, use `INCLUDE` columns in the index to avoid heap visits, batch updates within a single transaction in primary-key order so index leaves are touched left-to-right.

Edge cases to mention: - Wide secondary indexes with hot-spot key prefixes (e.g., `(status, created_at)` where status is mostly the same value) cluster updates onto the same leaves — high deadlock risk. - Postgres updates rewrite the entire row even if you change one column, but HOT updates skip index maintenance when no indexed column changed — keeps locks narrower. - Covering indexes can reduce deadlocks by avoiding extra heap access locks under serialisable.

What NOT to say: - Do not say "you cannot deadlock a single-row update" — you absolutely can, through indexes. - Do not blindly drop indexes to fix deadlocks; you may break the read path that justified them.

Common follow-up: "How do you tell if a Postgres update went HOT?" — Inspect `pg_stat_user_tables.n_tup_hot_upd`; a high ratio of HOT to total updates means the optimiser is avoiding index maintenance.

```
-- Postgres: check HOT update ratio
SELECT relname,
       n_tup_upd,
       n_tup_hot_upd,
       ROUND(100.0 * n_tup_hot_upd / NULLIF(n_tup_upd, 0), 1) AS hot_pct
FROM pg_stat_user_tables
WHERE relname = 'accounts';
```

Q251 · Distributed Deadlocks Across Services

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, JP Morgan, Goldman · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

You have two services, each owning its own database. Can they deadlock, and who would detect it?

The 30-second answer: Yes, they can — Service A holds a lock in DB1 while waiting for a response from Service B, which holds a lock in DB2 waiting for Service A. Neither database can see the cross-service wait, so the deadlock is invisible to both engines. Detection has to happen at the application level via timeouts; resolution is to fail one side and retry.

Step-by-step thought process: 1. Establish the visibility problem: each database knows only about waits inside itself; the wait-for graph cannot cross process or network boundaries. 2. The symptom is a request that hangs forever — both services have open transactions, both are blocked on a network call to the other. 3. Detection must be application-level: per-call timeouts, circuit breakers, distributed-tracing

alerts when transaction duration crosses a threshold. 4. Prevention: never hold a database transaction open across a synchronous call to another service. Use the outbox or saga pattern so each service commits independently and reconciliation is asynchronous.

Edge cases to mention: - Even within one database, two connections that route through a service mesh with retries can create deadlock-like patterns the engine cannot resolve cleanly. - Two-phase commit coordinators detect distributed deadlocks at commit time but introduce their own availability problems (see Q21). - Long-running gRPC calls inside a `BEGIN ... COMMIT` block are the single most common cause of cross-service stuck transactions I have debugged.

What NOT to say: - Do not claim the database will save you — engines explicitly disclaim cross-database deadlock detection. - Do not propose distributed locks (e.g., Zookeeper) as the default fix; they just relocate the deadlock problem.

Common follow-up: *"What's the cleanest pattern to avoid this?"* — Saga: each service commits its local transaction, publishes an event, and the next service consumes it. Compensating actions handle failures.

```
-- Anti-pattern: don't do this
BEGIN;
UPDATE wallet SET balance = balance - 100 WHERE user_id = 5;
-- BAD: synchronous external call inside the transaction
-- response = http_post('https://partner/transfer', amount=100)
UPDATE ledger SET ext_id = :response_id WHERE ...;
COMMIT;
```

Q252 · Deadlocks in Batch Jobs vs OLTP

Tags: Difficulty: Medium · Asked at: Wells Fargo, Target, AmEx, Walmart Labs · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Your nightly settlement batch deadlocks with the live OLTP traffic every other night. How do you reason about that?

The 30-second answer: Batch jobs lock many rows in big transactions; OLTP locks single rows in short transactions. When they overlap, the batch's wide lock footprint conflicts with rapid-fire OLTP updates and one side gets killed. Either run the batch in a low-traffic window, chunk it into smaller transactions, or use a read-only snapshot for the batch and only write back the deltas.

Step-by-step thought process: 1. Diagnose the asymmetry: a batch that locks 100k rows for two minutes blocks any OLTP transaction that needs even one of those rows. Probability of collision scales with batch size. 2. Chunking: break the batch into transactions of 1000-5000 rows each with `LIMIT` and primary-key cursoring; release locks between chunks. 3. Snapshot strategy: read everything you

need from a read replica or via **SET TRANSACTION SNAPSHOT** (Postgres) and only the final write hits the primary. The write itself is a single small transaction. 4. Schedule: run heavy batch at a time window with minimal OLTP — many fintechs run reconciliation between 02:00 and 04:00 IST for this exact reason.

Edge cases to mention: - A batch that uses **SERIALIZABLE** on a huge dataset is essentially a deadlock generator; **READ COMMITTED** with explicit **FOR UPDATE** on the rows actually written is usually safer. - Chunked batches need a stable ordering (primary key) so each chunk is deterministic and skipped rows can be picked up on rerun. - Long batches also bloat undo logs (Postgres MVCC, MySQL undo) — a separate problem worth mentioning if the interviewer asks.

What NOT to say: - Do not run a 4-hour batch as one transaction. Period. - Do not raise isolation to "fix" batch deadlocks — it makes the lock footprint worse.

Common follow-up: *"How do you make a chunked batch resumable?"* — Track the highest processed primary key in a checkpoint table; on rerun, start from that key.

```
-- Chunked nightly settlement loop (pseudo)
-- Pick rows in PK order, settle 5000 at a time, commit between chunks
DO $$
DECLARE last_id BIGINT := 0;
BEGIN
  LOOP
    WITH batch AS (
      SELECT id FROM pending_settlements
      WHERE id > last_id
      ORDER BY id
      LIMIT 5000
      FOR UPDATE SKIP LOCKED
    )
    UPDATE pending_settlements p
      SET status = 'SETTLED', settled_at = now()
      FROM batch WHERE p.id = batch.id
      RETURNING p.id INTO last_id;
    EXIT WHEN NOT FOUND;
    COMMIT;
  END LOOP;
END $$;
```

Q253 · High-Contention Rows and Read-Modify-Write

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, PhonePe, Swiggy · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Every order increments the same `invoice_counter` row. Why is this slow even though each transaction is tiny?

The 30-second answer: Because every transaction must take an exclusive lock on the same row, they serialize one-by-one regardless of how many CPUs your database has. Throughput collapses to one transaction per round-trip latency, and any other transaction that touches this row indirectly (e.g., through a FK) joins the queue. Hot single rows are the most common source of production "the database is slow" tickets.

Step-by-step thought process: 1. Frame the bottleneck: the lock manager forces strict serial execution on the hot row. With 5ms round-trip latency and 200 writers, you get at most 200 increments per second total — no matter the hardware. 2. The classic anti-pattern is

`SELECT counter FROM ... ; UPDATE counter SET counter = :value + 1` — two statements, race window, deadlock-prone, and slow. 3. Atomic increment via `UPDATE counter SET counter = counter + 1 RETURNING counter` collapses to one statement and one lock acquisition — drops the window but the serial bottleneck remains. 4. Real fix: shard the counter into N rows (`counter_0..counter_31`), increment a random shard, sum at read time. Throughput scales with N. Used by Stripe, Shopify, and every fintech ledger I have audited.

Edge cases to mention: - Sharded counters work for monotonic-ish totals (revenue, click count) but not for strict sequence numbers — you need a different pattern for gapless invoice IDs. - If you must have a gapless sequence, accept the bottleneck and pre-allocate ranges (HiLo pattern) so each app instance hands out 1000 IDs without touching the DB. - Postgres `SEQUENCE` objects use a special lock-free mechanism — much faster than rolling your own counter row.

What NOT to say: - Do not say "we will solve it with a faster server" — vertical scaling cannot break the serial dependency. - Do not implement counters as `SELECT ... FOR UPDATE; UPDATE ...` — atomic single-statement is always better.

Common follow-up: "How would you implement a sharded counter?" — N rows keyed `(counter_name, shard_id)`, app hashes the increment to a random shard, read aggregates all shards.

```
-- Sharded counter for revenue accumulator
CREATE TABLE counters (
  name      TEXT NOT NULL,
  shard_id  INT  NOT NULL,
  value     BIGINT NOT NULL DEFAULT 0,
  PRIMARY KEY (name, shard_id)
);

-- Increment a random shard
UPDATE counters
SET value = value + :amount
WHERE name = 'gross_revenue_paise'
  AND shard_id = FLOOR(RANDOM() * 32);

-- Read total
SELECT SUM(value) FROM counters WHERE name = 'gross_revenue_paise';
```

Q254 · The Hot Invoice Counter — A Real Production Story

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Zerodha · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Your invoicing service needs gap-less invoice numbers for GST compliance. How do you generate them without making the DB the bottleneck?

The 30-second answer: A truly gap-less invoice number forces serialisation — you cannot shard. Use a single counter row with atomic increment, but pre-allocate ranges via the HiLo pattern: each app instance fetches a block of, say, 100 numbers, hands them out in memory, and only revisits the DB every 100 invoices. Throughput scales with block size, and gaps appear only on app crashes (regulator usually permits documented gaps).

Step-by-step thought process: 1. Acknowledge the constraint: Indian GST law requires gap-less sequential invoice numbers per financial year per series. Sharded counters cannot do this. 2. HiLo pattern: a small counter table holds the next available "block start"; each app pulls a block, increments the block-start by `block_size`, commits, and serves invoice numbers from local memory. 3. Risk: if the app crashes mid-block, the unused tail is wasted — gaps. Document the policy and most regulators accept it; otherwise reduce block size to one and accept the throughput hit. 4. Alternative for less-strict numbering: append a per-instance suffix to a sharded counter (e.g., `INV-2026-{shard}-{seq}`) — fast but no longer a single gap-less sequence.

Edge cases to mention: - The counter table must use `SERIALIZABLE` or `SELECT FOR UPDATE` to avoid two app instances picking overlapping blocks. - A failover or rolling deploy must hand off the

unused tail or accept the gap — operational discipline matters. - For financial-year-bounded series (`INV-2025-NNN`), reset the counter at year boundary via a scheduled job, not lazily.

What NOT to say: - Do not pretend sharding works for gap-less sequences — the interviewer is testing whether you understand the constraint. - Do not store invoice numbers in Redis without a write-through to the database; a Redis flush destroys your audit trail.

Common follow-up: *"What block size do you pick?"* — Trade-off: larger blocks give higher throughput but more gaps on crash. 50-100 is a common compromise for typical SaaS invoicing volumes.

```
-- HiLo allocator
CREATE TABLE invoice_blocks (
  series      TEXT PRIMARY KEY,
  next_block  BIGINT NOT NULL,
  block_size  INT     NOT NULL DEFAULT 100
);

-- Fetch a block atomically
UPDATE invoice_blocks
SET next_block = next_block + block_size
WHERE series = 'INV-2025'
RETURNING next_block - block_size AS block_start, block_size;
```

Q255 · Lock-Free Patterns — Append-Only and Event Sourcing

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan, Zerodha · Topic: Conflicts

The question (interviewer's verbatim phrasing):

How do you design a write path that essentially cannot deadlock?

The 30-second answer: Make the write path append-only. Every business event becomes an immutable INSERT into a log table, and the current state is derived by aggregating the log. INSERTs never conflict on existing rows, so concurrent writers do not lock each other. Read paths use materialised views or projection tables refreshed asynchronously.

Step-by-step thought process: 1. State the principle: row updates create lock contention; row inserts on a table with no unique key contention create almost none. 2. Event-sourced wallet example: every credit and debit is an INSERT into `wallet_events`. To compute a balance, sum the events for that user; for performance, periodically snapshot the running balance. 3. Concurrent two-direction transfers from Q5 stop deadlocking because both transactions just append two events each — no shared row to lock. 4. Trade-offs: read complexity goes up, you need projection rebuilds, and unique business

invariants (e.g., "balance never goes negative") move out of the database into the application or into a periodic check.

Edge cases to mention: - "Balance must not go negative" cannot be enforced purely append-only without a synchronous check — either do an optimistic read-then-insert with retry, or accept eventual reconciliation. - Event tables grow forever — partition by month and archive cold partitions. - Snapshot tables prevent the read aggregation from getting slower as the log grows; rebuild snapshots in background jobs.

What NOT to say: - Do not claim "event sourcing eliminates all concurrency problems" — uniqueness invariants still need handling. - Do not implement append-only as `UPDATE ... last_modified = now()` — that is the opposite, you are still updating in place.

Common follow-up: "How do you enforce 'balance >= 0' in an event-sourced wallet?" — Run the debit as `INSERT ... WHERE (SELECT balance ...) >= amount` with a unique constraint on event_id; on conflict you retry with the new balance.

```
-- Event-sourced wallet writes
CREATE TABLE wallet_events (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT    NOT NULL,
  amount      BIGINT    NOT NULL, -- positive credit, negative debit
  event_type  TEXT      NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Both transfers from Q5 become four INSERTs total, no row conflict
INSERT INTO wallet_events(user_id, amount, event_type) VALUES (101, -1000, 'transfer_out');
INSERT INTO wallet_events(user_id, amount, event_type) VALUES (247, 1000, 'transfer_in');
```

Q256 · The Saga Pattern for Cross-Service Workflows

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred, Walmart Labs · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Distributed transactions across three microservices are off the table. How do you keep a multi-step workflow consistent?

The 30-second answer: Use the saga pattern — split the workflow into a sequence of local transactions, each in its own service and database, with a compensating action defined for each step. On failure, you run the compensations in reverse order to roll back the parts that already succeeded. No distributed locks, no two-phase commit.

Step-by-step thought process: 1. Define the saga: ordered list of (forward step, compensation) pairs. Each step commits in its own database; failures trigger compensations from the failure point backward. 2. Worked example: an order saga reserves inventory, charges the wallet, creates the shipment. If shipment creation fails, you refund the wallet and release the inventory. 3. Orchestration vs choreography: orchestration uses a central coordinator (state machine); choreography uses events between services. Orchestration is easier to reason about; choreography scales better. 4. Idempotency is non-negotiable — every step and every compensation must be safe to retry, because network failures will redeliver messages.

Edge cases to mention: - Compensations are not always perfectly reversible (you cannot un-send a notification email) — design "semantic" compensations like a follow-up apology email. - Long-running sagas must persist state in a saga log so a process restart can resume. - A saga is eventually consistent, not atomic — intermediate observers can see partial state. Document this with the business.

What NOT to say: - Do not propose two-phase commit as a casual alternative — its availability cost is high (see Q21). - Do not skip compensations on "expected to succeed" steps — Murphy's law applies in production.

Common follow-up: *"How do you make payment charging idempotent?"* — Send an idempotency key with the charge request; the payment service stores it and returns the same result on retry.

```
-- Saga state log (simplified)
CREATE TABLE saga_log (
  saga_id      UUID PRIMARY KEY,
  step         TEXT NOT NULL,
  status       TEXT NOT NULL,      -- pending | done | compensated | failed
  payload      JSONB,
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Q257 · Two-Phase Commit — What and Why It's Rare

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, AmEx · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Explain two-phase commit. Why do most modern systems avoid it?

The 30-second answer: 2PC is a protocol where a coordinator first asks every participant "can you commit?" (prepare phase) and, only if all say yes, sends "commit" (commit phase). It guarantees atomicity across multiple databases. The cost is that participants hold locks the entire time the coordinator is reachable, and a coordinator crash between phases leaves everyone blocked — high availability impact, very low operational tolerance.

Step-by-step thought process: 1. Sketch the two phases: PREPARE asks each participant to durably persist the transaction in a state from which it can either commit or rollback; COMMIT (or ROLLBACK) is the final decision. 2. The atomicity guarantee is real — either all participants commit or none do. 3. The cost: any in-doubt transaction (after PREPARE, before COMMIT) holds locks until the coordinator recovers and resolves it. If the coordinator crashes permanently, you have manual cleanup. 4. Most modern systems prefer eventual consistency via sagas, the outbox pattern, or distributed log streaming — gives up strict atomicity for huge availability gains.

Edge cases to mention: - Postgres supports 2PC via `PREPARE TRANSACTION 'gid' / COMMIT PREPARED 'gid'`, but it must be explicitly enabled with `max_prepared_transactions > 0`. - XA transactions in MySQL and SQL Server are 2PC variants and have a long history of bugs and operational pain. - 2PC is still useful inside one company that controls the coordinator (e.g., a payment hub between two internal databases) — just not across organisational boundaries.

What NOT to say: - Do not say "2PC is always wrong" — for some financial settlements it is the right call. - Do not confuse 2PC with eventual consistency; 2PC is strict atomic commit.

Common follow-up: *"What happens if the coordinator crashes after PREPARE on participant A but before sending PREPARE to participant B?"* — A is in-doubt and locked until the coordinator recovers and resolves; B never participated and is fine. Operators must manually commit or rollback A in the meantime.

```
-- Postgres 2PC syntax
BEGIN;
-- do work
PREPARE TRANSACTION 'order-42';    -- locks held, transaction in-doubt
-- coordinator decides
COMMIT PREPARED 'order-42';        -- or ROLLBACK PREPARED 'order-42';
```

Q258 · The Read-Modify-Write Race

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, TCS · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Your code reads a row, computes a new value in application memory, and writes it back. Why is this dangerous under concurrency?

The 30-second answer: Between your SELECT and UPDATE another transaction can modify the same row, and your UPDATE overwrites their change — the lost-update problem. Fix it with a single atomic statement (`UPDATE ... SET col = col + 1`), an explicit `SELECT FOR UPDATE` lock, an

optimistic version column (`WHERE version = :read_version`), or by raising isolation to `SERIALIZABLE`.

Step-by-step thought process: 1. Show the race timeline: T1 reads balance=1000, T2 reads balance=1000, T1 writes balance=900, T2 writes balance=800 (intended 1000-200). The first decrement is lost. 2. Atomic-statement fix is simplest: let the database compute the new value. `UPDATE wallet SET balance = balance - 200 WHERE user_id = 5` cannot lose updates. 3. When you cannot do it atomically (because the new value comes from external computation), use `SELECT ... FOR UPDATE` so the second reader blocks, or an optimistic CAS: `UPDATE ... WHERE version = :read_version` and retry on zero rows affected. 4. Discuss the trade-off: pessimistic (`FOR UPDATE`) is simple but reduces concurrency; optimistic is high-throughput but adds retry complexity.

Edge cases to mention: - Pure ORM `record.update(balance: record.balance - 200)` is a read-modify-write — almost always wrong for counters. - Optimistic locking only works if you have a version column or a timestamp that monotonically increases; bare `WHERE balance = :read_balance` is a weak version key. - Snapshot isolation (Postgres `REPEATABLE READ`, SQL Server SI) raises an error on conflicting writes — built-in optimistic protection.

What NOT to say: - Do not claim "the database handles concurrency" — it provides primitives; you still have to use them. - Do not use `SERIALIZABLE` as a default fix; the cost of retries can exceed the cost of getting the SQL right.

Common follow-up: "Show me an optimistic locking pattern with a version column." —

`UPDATE x SET balance = :new_value, version = version + 1 WHERE id = :id AND version = :read_version` — if zero rows affected, re-read and retry.

```
-- Optimistic locking with version column
SELECT balance, version FROM wallet WHERE user_id = 5;
-- compute new_balance in app
UPDATE wallet
SET balance = :new_balance,
    version = version + 1
WHERE user_id = 5
    AND version = :read_version;
-- if rowcount = 0, someone else updated; reread + retry
```

Q259 · INSERT ... ON CONFLICT — Atomic Upsert

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, Razorpay, Swiggy · Topic: Conflicts

The question (interviewer's verbatim phrasing):

You want to insert a row if it doesn't exist and update it if it does. How do you avoid the race?

The 30-second answer: Use the database's atomic upsert syntax — Postgres `INSERT ... ON CONFLICT (key) DO UPDATE`, MySQL `INSERT ... ON DUPLICATE KEY UPDATE`, SQLite `INSERT ... ON CONFLICT DO UPDATE`. These execute as a single statement under a single lock acquisition, so there is no window between "check exists" and "insert or update" for another transaction to slip in.

Step-by-step thought process: 1. Diagnose the race: the naive pattern `IF EXISTS THEN UPDATE ELSE INSERT` has a TOCTOU (time-of-check to time-of-use) gap that causes either duplicate-key errors or lost updates. 2. Show the Postgres syntax: `INSERT ... VALUES (...) ON CONFLICT (user_id) DO UPDATE SET visits = excluded.visits + 1` — clean, atomic, single round-trip. 3. MySQL equivalent: `INSERT ... ON DUPLICATE KEY UPDATE visits = visits + VALUES(visits)`. SQL Server uses `MERGE` (with caveats — see Q20). 4. Use case fit: counter pages, session tables, idempotent webhook processing. The pattern eliminates an entire class of race conditions.

Edge cases to mention: - `ON CONFLICT` requires a unique index or primary key on the conflict columns; otherwise the conflict cannot be detected. - `DO NOTHING` variant is useful for idempotent event processing — insert if new, silently skip if seen before. - Returning clause (`RETURNING *`) tells you whether the row was inserted or updated by checking `xmax` or the version column.

What NOT to say: - Do not implement upsert as `BEGIN; SELECT; IF NULL INSERT ELSE UPDATE; COMMIT` — that is exactly the race you are trying to avoid. - Do not assume `MERGE` is always atomic; in some engines it is not (see Q20).

Common follow-up: "How do you tell if `ON CONFLICT` inserted or updated?" — Postgres: `RETURNING (xmax = 0) AS inserted`; MySQL: `ROW_COUNT()` returns 1 for insert, 2 for update.

```
-- Atomic upsert for a page-view counter
INSERT INTO page_views(user_id, page_id, views, last_seen)
VALUES (5, 'p123', 1, now())
ON CONFLICT (user_id, page_id)
DO UPDATE SET views = page_views.views + 1,
              last_seen = excluded.last_seen
RETURNING views;
```

Q260 · MERGE INTO and Its SQL Server Deadlock Pitfalls

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Wells Fargo, Walmart Labs · Topic: Conflicts

The question (interviewer's verbatim phrasing):

MERGE looks elegant. Why do experienced SQL Server engineers avoid it for high-concurrency workloads?

The 30-second answer: SQL Server's MERGE has a well-documented set of bugs that cause it to occasionally violate uniqueness constraints, return wrong results in concurrent execution, and deadlock against itself under load. The MS docs themselves recommend explicit INSERT/UPDATE patterns or upsert-by-attempt for high-throughput scenarios. Postgres MERGE (added in v15) has been better behaved but is still newer than ON CONFLICT.

Step-by-step thought process: 1. Acknowledge MERGE's appeal — single statement that handles insert, update, delete in one go — and the disappointment that the implementation has been buggy. 2. The classic SQL Server bug: two concurrent MERGEs targeting the same key can both pass the WHEN NOT MATCHED check and both attempt insert, causing a primary key violation. Use `HOLDLOCK` hint to serialize, but that hurts throughput. 3. SQL Server-specific deadlock: MERGE may take locks in different orders depending on the matched/not-matched plan, leading to deadlocks under load that ON CONFLICT/IF EXISTS patterns avoid. 4. Practical guidance: for concurrent upserts on SQL Server, prefer `IF EXISTS UPDATE ELSE INSERT` wrapped in serializable, or the "try insert, catch dup, then update" idiom. Reserve MERGE for batch ETL where concurrency is low.

Edge cases to mention: - Postgres MERGE (v15+) does not have the SQL Server bugs but still does not have ON CONFLICT's `excluded` pseudo-table; use ON CONFLICT for upserts in Postgres. - Oracle MERGE is the original and is generally considered safe but uses different syntax (no `WHEN MATCHED THEN DELETE` in older versions). - BigQuery and Snowflake MERGE are designed for analytic workloads where one-row-at-a-time concurrency is not a use case.

What NOT to say: - Do not call MERGE "the modern way to upsert" in SQL Server — it has been controversial since 2008. - Do not blindly add HOLDLOCK without measuring the throughput impact.

Common follow-up: "Show me the safe SQL Server upsert pattern." —

`IF EXISTS (SELECT 1 FROM target WITH (UPDLOCK, HOLDLOCK) WHERE key = @k)`
`UPDATE ... ELSE INSERT ...` inside SERIALIZABLE, or the try/catch pattern.

```
-- SQL Server safe upsert (no MERGE)
BEGIN TRAN;
IF EXISTS (SELECT 1 FROM accounts WITH (UPDLOCK, HOLDLOCK) WHERE id = @id)
    UPDATE accounts SET balance = balance + @delta WHERE id = @id;
ELSE
    INSERT INTO accounts(id, balance) VALUES (@id, @delta);
COMMIT TRAN;
```

Q261 · Application-Level Locks vs Database Locks

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Walmart Labs, JP Morgan · Topic: Conflicts

The question (interviewer's verbatim phrasing):

When would you reach for a Redis lock or a Zookeeper mutex instead of a database-level lock?

The 30-second answer: Use a database lock when the resource being protected lives in the database and the lock can be held for the duration of one short transaction. Use an application-level lock (Redis, Zookeeper, etcd) when the protected work is longer than a database transaction should be — like running a periodic job once across a fleet of workers, or coordinating writes across multiple databases.

Step-by-step thought process: 1. Frame the choice: database locks are scoped to a transaction and freed at commit/rollback; application locks are scoped to a TTL or a lease and freed by explicit release. 2. Database locks are correct for: row updates, FK enforcement, anything where the lock and the data are co-located. 3. Application locks are correct for: leader election in a worker pool, ensuring a cron job runs exactly once across N replicas, serialising writes to an external system without a database transaction. 4. The big risk with application locks is fencing — if a process is paused (GC, network), its lock can expire and another process can hold it, but the original may resume and act on stale assumptions. Use a fencing token or a CAS check.

Edge cases to mention: - Postgres advisory locks (`pg_advisory_lock(key)`) are application-level locks living in the database — best of both worlds: distributed coordination plus crash-detection via session close. - Redis Redlock has well-documented edge cases under network partitions; for safety-critical work, use Zookeeper or etcd which provide stronger guarantees. - Holding any lock — DB or app — across a network call is asking for trouble; release before the call if possible.

What NOT to say: - Do not use a Redis lock to protect data that already lives in the database — you have two sources of truth and they will diverge. - Do not assume Redis SETNX is a safe distributed lock without TTL handling and fencing.

Common follow-up: "How would you ensure a nightly batch runs only once across three pods?" — Use a Postgres advisory lock at the start of the batch; if `pg_try_advisory_lock` returns false, the pod exits cleanly.

```
-- Postgres advisory lock to ensure single-leader batch
SELECT pg_try_advisory_lock(:job_id);
-- if true, this pod is the leader; do the work
-- when done or on crash, the session closing releases the lock
SELECT pg_advisory_unlock(:job_id);
```

Q262 · Lock Timeout vs Deadlock Timeout

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, AmEx, Target · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

What is the difference between `lock_timeout` and `deadlock_timeout` in Postgres?

The 30-second answer: `lock_timeout` is how long any one statement waits for a lock before giving up with an error — a guard against indefinite waits. `deadlock_timeout` is how long the engine waits before running a deadlock detection scan on a wait edge. They are entirely independent:

`lock_timeout` caps total wait, `deadlock_timeout` decides when to spend CPU looking for cycles.

Step-by-step thought process: 1. Define each with one sentence: `lock_timeout` = abort after this much wait, `deadlock_timeout` = start cycle-checking after this much wait. 2. Show interaction: with `deadlock_timeout = 1s` and `lock_timeout = 5s`, a wait of two seconds with no cycle gets a deadlock-check (negative result), continues waiting, and aborts at five seconds with a lock-timeout error. 3. Tuning intuition: lower `deadlock_timeout` for OLTP if you would rather detect deadlocks early; raise `lock_timeout` for batch jobs where waiting is expected. 4. MySQL InnoDB has a different model: `innodb_lock_wait_timeout` is both lock-wait abort and (since InnoDB detects deadlocks eagerly) the only timeout you need to set.

Edge cases to mention: - `statement_timeout` (Postgres) is a third, even broader timer — kills any statement exceeding the limit regardless of why it is slow. - Setting `lock_timeout` per-transaction is useful for DDL: `SET LOCAL lock_timeout = '100ms'; ALTER TABLE ...` so a stuck ALTER does not freeze the application. - SQL Server's `SET LOCK_TIMEOUT` is in milliseconds and only affects lock waits, not query execution time.

What NOT to say: - Do not confuse the two — interviewers test exactly this distinction. - Do not set `lock_timeout` very high in OLTP; one stuck transaction can chain-block the whole application.

Common follow-up: "How would you run an ALTER TABLE safely on a busy Postgres database?" — Wrap with low `lock_timeout` and retry on timeout: `SET lock_timeout = '200ms'; ALTER TABLE ...; RESET lock_timeout;` — back off if it cannot grab the lock quickly.

```
-- Set per-session lock_timeout
SET lock_timeout = '500ms';

-- Or per-transaction
BEGIN;
SET LOCAL lock_timeout = '100ms';
ALTER TABLE accounts ADD COLUMN kyc_status TEXT;
COMMIT;
```

Q263 · Deferred Constraints in Postgres

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Explain deferrable constraints in Postgres. When does the check actually run?

The 30-second answer: A deferrable constraint can have its check postponed from "after each statement" (the default IMMEDIATE) to "at commit time" (DEFERRED). This lets you temporarily violate the constraint in the middle of a transaction — for example, swapping two rows that both depend on each other — and only requires the final committed state to be consistent.

Step-by-step thought process: 1. Define the two modes: IMMEDIATE checks after every statement; DEFERRED checks at COMMIT. A constraint declared DEFERRABLE can switch between them per transaction via `SET CONSTRAINTS`. 2. The killer use case is cyclical FKs and self-referential swaps — `UPDATE A.next = B.id; UPDATE B.next = A.id` would fail IMMEDIATE on the first statement but works DEFERRED. 3. Performance angle from Q9: deferred FK checks happen at commit, so multiple insert statements within one transaction collide less with other transactions during the body of the work. 4. Limitation: errors at commit are harder to recover from than statement-level errors; the transaction is fully rolled back with no chance to retry just the failing piece.

Edge cases to mention: - Only FK, UNIQUE, and EXCLUDE constraints can be DEFERRABLE in Postgres; CHECK constraints and NOT NULL cannot. - `SET CONSTRAINTS ALL DEFERRED` switches every deferrable constraint in the transaction; useful for bulk loads. - MySQL does not support deferrable constraints at all; SQL Server has limited support via `WITH NOCHECK` on DDL but not per-transaction.

What NOT to say: - Do not say "deferred constraints disable checking" — they only postpone it. - Do not declare every constraint DEFERRABLE "just in case"; immediate checks catch bugs sooner.

Common follow-up: "How would you migrate data with circular FKs?" — Use DEFERRABLE INITIALLY DEFERRED on both FKs, bulk-load both tables in any order, COMMIT, and the FK check at commit verifies everything is consistent.

```
-- Cyclical FK with deferrable constraints
ALTER TABLE node
  ADD CONSTRAINT fk_parent FOREIGN KEY (parent_id) REFERENCES node(id)
  DEFERRABLE INITIALLY DEFERRED;

BEGIN;
INSERT INTO node(id, parent_id) VALUES (1, 2);
INSERT INTO node(id, parent_id) VALUES (2, 1);
COMMIT; -- both checked here, both pass
```

Q264 · Why Increment Counters Are a Bottleneck (Deeper)

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Razorpay, Cred · Topic: Conflicts

The question (interviewer's verbatim phrasing):

We talked about the hot counter. Walk me through what actually happens inside the database when 1000 transactions try to increment one row.

The 30-second answer: Each transaction's UPDATE waits for an exclusive row lock. The lock manager queues them in FIFO order, and each one only runs after the previous one has committed and released the lock. So 1000 transactions execute serially with one commit per round-trip, giving throughput of roughly $1 / \text{commit_latency}$ per second — typically a few hundred per second, regardless of CPU count.

Step-by-step thought process: 1. Step into the lock manager: an exclusive row lock cannot be shared, so the second writer blocks; the third blocks behind the second; you build a queue. 2. Each queued writer needs the previous one's COMMIT to fully durably persist (fsync) before it can proceed, because the new value depends on the previous transaction's outcome. 3. With 1ms commit latency, theoretical max is 1000/sec; in practice you see lower because of network round-trips and other overheads. 4. The fix patterns from Q13/Q14 all attack this serial dependency: shard the row, batch the increments in memory and flush periodically, or use a sequence object.

Edge cases to mention: - Group commit (Postgres `commit_delay`, MySQL `binlog_group_commit_sync_delay`) can amortise fsync across multiple transactions, raising counter throughput somewhat. - Postgres `SEQUENCE` uses an internal in-memory cache (`CACHE n`) so allocations don't go through the WAL on every call — much faster than a counter row. - In-memory engines (Redis INCR) hit hundreds of thousands per second by skipping disk persistence entirely — at the cost of durability guarantees.

What NOT to say: - Do not say "we'll add an index to speed it up" — the index doesn't help a write that is already direct lookup. - Do not blame "row lock latency" — the latency comes from waiting in the queue, not from the lock primitive itself.

Common follow-up: "Why doesn't read concurrency help with this?" — Counters are write-only; reads can use replicas, but the writes funnel through the primary's lock manager.


```
-- Verify counter contention in Postgres
SELECT relname, schemaname, n_tup_upd, n_tup_hot_upd
FROM pg_stat_user_tables
WHERE relname = 'counters'
ORDER BY n_tup_upd DESC;

-- Live blockers on the hot row
SELECT pid, wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE query ILIKE '%counters%';
```

Q265 · The Outbox Pattern for Reliable Side Effects

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, JP Morgan, Cred · Topic: Conflicts

The question (interviewer's verbatim phrasing):

You commit a payment in your DB and need to send a confirmation message. How do you ensure the message is sent exactly when (and only when) the transaction commits?

The 30-second answer: Use the transactional outbox pattern. In the same transaction that records the payment, INSERT a row into an `outbox` table describing the message to send. A separate background relay polls the outbox and publishes the messages, marking them as sent. The DB transaction guarantees both rows are present or neither; the relay guarantees at-least-once delivery; with idempotent consumers you get effectively exactly-once.

Step-by-step thought process: 1. Diagnose the naive approach: sending the message inside the transaction means a successful send + failed commit, or successful commit + failed send. Either way, your DB and downstream are inconsistent. 2. Outbox table is the bridge: the message lives in the database alongside the business data, so they share atomicity. The transaction commit publishes the intent to send. 3. The relay (a small worker) reads unsent rows, publishes to Kafka/SQS/RabbitMQ, and marks them sent. Crashes between publish and mark mean redelivery, which is why consumers must be idempotent. 4. Trade-offs: tiny additional latency (relay polling interval), an extra table to monitor for backlog, but you eliminate an entire class of integration bugs.

Edge cases to mention: - Relay throughput must exceed message-generation rate; size the relay properly or use change-data-capture (Debezium) to stream the outbox directly. - Outbox grows unbounded — schedule a job to delete sent rows older than a retention window (e.g., 7 days). - For ordering guarantees, partition the outbox by aggregate ID and consume per-partition serially.

What NOT to say: - Do not send messages inside `BEGIN ... COMMIT` blocks — that is the bug the outbox pattern exists to fix. - Do not use the outbox for synchronous request/response; it is for fire-and-forget events.

Common follow-up: "How does this compare to two-phase commit between DB and message broker?"

— Outbox is simpler, doesn't require the broker to support 2PC, and degrades gracefully (broker outage means messages queue up safely in the outbox).

```
-- Outbox table and atomic insert with business write
CREATE TABLE outbox (
  id          BIGSERIAL PRIMARY KEY,
  aggregate   TEXT NOT NULL,
  event_type  TEXT NOT NULL,
  payload     JSONB NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at     TIMESTAMPTZ
);

BEGIN;
INSERT INTO payments(id, user_id, amount, status) VALUES (...);
INSERT INTO outbox(aggregate, event_type, payload)
  VALUES ('payment', 'payment.succeeded', '{"id":"...", "amount":1000} '::jsonb);
COMMIT;
```

Q266 · SKIP LOCKED for Concurrent Job Queues

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: Conflicts

The question (interviewer's verbatim phrasing):

Five workers all want to grab the next job from a `jobs` table. How do they avoid stepping on each other without serialising?

The 30-second answer: Use `SELECT ... FOR UPDATE SKIP LOCKED` — each worker locks the next available row and the others skip rows already locked by peers, so all five workers grab different jobs in parallel without blocking. It is the canonical concurrent-queue pattern in modern Postgres, MySQL 8+, and Oracle.

Step-by-step thought process: 1. Without SKIP LOCKED, all five workers race on the same `SELECT FOR UPDATE`, four block on the lock, and only one progresses at a time — the queue becomes serial. 2. SKIP LOCKED tells the engine to ignore rows that are currently row-locked and return the next eligible one. Five workers running this concurrently each pick different rows. 3. Pattern: `SELECT id FROM jobs WHERE status = 'pending' ORDER BY created_at LIMIT 1 FOR UPDATE SKIP LOCKED`; then `UPDATE jobs SET status = 'in_progress', worker = :id WHERE id = :picked`; do work; commit. 4. Failure mode: if a worker crashes mid-job, the `in_progress` row stays locked until the transaction ends (connection close releases the lock); a janitor job resets stale `in_progress` rows older than a threshold back to `pending`.

Edge cases to mention: - SKIP LOCKED is not in standard SQL; was added to Postgres 9.5, MySQL 8.0, Oracle 11g. SQL Server uses `READPAST` hint for similar effect. - The SKIP LOCKED scan still has to read past locked rows, so ordering by an indexed column matters — `ORDER BY priority, created_at` works only if there is a matching index. - Combine with `LIMIT` to bound the scan per worker.

What NOT to say: - Do not implement queue workers by polling the same row with `FOR UPDATE` — you serialise the whole pool. - Do not assume SKIP LOCKED works on all databases — it doesn't on older MySQL or SQLite.

Common follow-up: *"How do you recover from a worker that crashed mid-job?"* — A scheduled task resets `in_progress` rows whose `picked_at` timestamp is older than the job SLA back to `pending`.

```
-- Worker picks one job, no contention with peers
BEGIN;
SELECT id
FROM jobs
WHERE status = 'pending'
ORDER BY priority DESC, created_at
LIMIT 1
FOR UPDATE SKIP LOCKED;

-- assume returned id = 42
UPDATE jobs
SET status = 'in_progress', worker_id = :me, picked_at = now()
WHERE id = 42;
COMMIT;
-- do the actual work, then mark done
```

Q267 · Transactions & Deadlocks

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, JP Morgan, Goldman, Cred, Walmart Labs · Topic: Deadlocks

The question (interviewer's verbatim phrasing):

Give me the complete picture: what is a deadlock, how does the engine detect it, how do you prevent it, what do you do when you cannot prevent it, and how does behaviour differ across Postgres, MySQL and SQL Server?

The 30-second answer: A deadlock is a circular wait between two or more transactions, each holding a lock the other needs. Engines detect it by finding a cycle in the wait-for graph — Postgres lazily after `deadlock_timeout` (default 1s), InnoDB and SQL Server eagerly on every wait. Prevention is

consistent lock ordering (sort row IDs ascending before locking). When prevention is impossible, wrap the transaction in a retry-with-exponential-backoff loop and make sure the work is idempotent. The classic Indian-fintech example is two simultaneous wallet transfers between the same pair of accounts in opposite directions — fix by always locking the lower account ID first.

Step-by-step thought process: 1. Define deadlock crisply: a cycle in the lock wait-for graph. Two transactions, each holding a row the other wants, neither can release until commit, neither can commit without the other releasing. The engine must break the cycle by aborting one transaction. 2. Walk through detection. The wait-for graph has transactions as nodes and a directed edge from A to B whenever A is waiting on a lock held by B. A deadlock is, by definition, a cycle. Postgres runs the cycle scan only after a wait has lasted `deadlock_timeout` to avoid spending CPU on short waits — set it to 200ms for OLTP, leave at 1s for batch. InnoDB and SQL Server check eagerly on every wait edge, so detection is essentially instant but with continuous overhead. The victim is whichever transaction the engine deems cheaper to roll back — SQL Server lets you bias this with `DEADLOCK_PRIORITY`. 3. The classic two-transfer scenario. Anjali and Rahul both hit "send money" at the same time, in opposite directions. Transfer one locks Anjali's row, asks for Rahul's; transfer two locks Rahul's row, asks for Anjali's. Perfect cycle. Same shape appears in inventory transfers between warehouses, ledger postings, and graph-walking deletes. 4. Prevention via lock ordering. Before any UPDATE, sort the row IDs ascending and lock them in that order with `SELECT FOR UPDATE`. Now both transfers take Anjali first, the second waits, no cycle ever forms. This is the single most effective deadlock prevention technique and works across all engines. Document it as a coding standard and enforce in code review. 5. Other prevention patterns. Atomic single-statement upserts via ON CONFLICT eliminate the read-modify-write race window. Append-only event-sourced writes avoid row-level contention entirely. Deferrable foreign keys in Postgres move FK checks to commit time, reducing the window for FK-induced deadlocks. Smaller transactions reduce the lock footprint and probability of overlap. 6. The retry pattern. Even with perfect lock ordering you will sometimes deadlock because of index-level or page-level conflicts you cannot foresee. Wrap the transaction in a retry loop, catch the specific error code (`40P01` Postgres, `1213` MySQL, `1205` SQL Server), sleep `base * 2^attempt + jitter`, retry up to 3-5 times. Bubble the error up after the cap and alert; persistent retries hide a real bug. 7. Idempotency is mandatory. The first attempt may have triggered side effects before the deadlock killed it — emails, external API calls. Pull all side effects out of the transaction (use the outbox pattern from Q25) and key external operations with idempotency keys. 8. Distributed deadlocks. When the cycle crosses two databases or two services, no single engine sees the whole graph. Use per-call timeouts, never hold a transaction open across a synchronous network call, and prefer saga-based eventual consistency (Q16) over distributed locks or two-phase commit (Q17).

Edge cases to mention: - Higher isolation makes deadlocks more frequent, not less. SERIALIZABLE turns many phantom-read scenarios into serialization failures (SQLSTATE 40001) that look very similar to deadlocks (40P01) and need the same retry handling. - Hot single-row contention (Q13/Q24) is not a deadlock but masquerades as one in slow-query reports — the fix is sharding or atomic increment, not lock ordering. - Foreign-key insert deadlocks (Q9) happen even without explicit UPDATE because FK enforcement takes implicit parent share locks. Deferrable FKs reduce the lock window. - Index-level deadlocks (Q10) can occur on single-row updates because the same UPDATE touches multiple index entries, each its own lock resource. - Lock-wait timeout (`lock_timeout` Postgres,

`innodb_lock_wait_timeout` MySQL) is unrelated to deadlock detection — it caps total wait, independent of whether a cycle exists.

What NOT to say: - Do not say "deadlocks mean the database is overloaded" — load increases frequency but bad lock ordering is the actual cause. - Do not retry on every error blindly. Deadlock (40P01) and serialization failure (40001) are retryable; primary-key violation, check constraint, and FK violation are not. Match on the specific SQLSTATE. - Do not raise isolation to fix deadlocks — usually makes them worse and doesn't change the underlying contention pattern. - Do not propose distributed locks (Redis, Zookeeper) as the default solution for in-database deadlocks. They are useful for cross-service coordination, not for resources that already live in the database. - Do not hold a database transaction open across a synchronous network call. This is the single most common production stuck-transaction pattern.

Common follow-up: *"Walk me through your last production deadlock and what you changed to fix it."* — A great answer names a specific table, the two code paths that interleaved, the lock ordering fix you applied, and the retry pattern you added as defence in depth. Bonus marks for showing the deadlock graph from the logs and explaining how you confirmed the fix via load testing.

```
-- Defensive pattern: lock ordering + retry, in the two-transfer case
-- Application pseudocode (Postgres-flavoured)
-- for attempt in 1..5:
--   try:
--     BEGIN;
--     ids := SORT([from_id, to_id]);           -- ascending
--     SELECT id FROM accounts WHERE id IN (ids) ORDER BY id FOR UPDATE;
--     UPDATE accounts SET balance = balance - amount WHERE id = from_id;
--     UPDATE accounts SET balance = balance + amount WHERE id = to_id;
--     INSERT INTO transfers(from_id, to_id, amount) VALUES (from_id, to_id, amount);
--     INSERT INTO outbox(event_type, payload) VALUES ('transfer.succeeded', payload);
--     COMMIT;
--     return success
--   except DeadlockDetected (SQLSTATE 40P01):
--     ROLLBACK;
--     sleep(50ms * 2^attempt + random(0,100ms))
--   raise PermanentDeadlockError
```

Chapter 6 — Normalization & Schema Design

Schema design is where classroom theory and production reality argue. Service MNCs love to ask the textbook stuff by rote — recite 1NF, 2NF, 3NF, BCNF in order, draw the dependency diagram, prove the decomposition is lossless. Product companies and GCC panels flip the script — they ask "tell me about a time you deliberately denormalized, and what bit you six months later." Both flavours show up in the same week of interviews, so you need to walk in carrying both vocabularies. This chapter mixes the academic ladder with the production trade-offs — normal forms first, then star schemas, surrogate keys, multi-tenant patterns, hierarchical data, sharding, and the hardest question of all: when to break the rules. Read it like you're prepping for two interviews at once, because you are.

Q268 · First Normal Form — Atomic Columns Only

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Normal Forms

The question (interviewer's verbatim phrasing):

What does it mean for a table to be in First Normal Form? Give me an example of a table that violates 1NF and how you would fix it.

The 30-second answer: 1NF says every column holds a single atomic value and every row is unique — no repeating groups, no comma-separated lists, no arrays stuffed into a varchar. Fix it by splitting the multi-valued column into its own child table with a foreign key back to the parent.

Step-by-step thought process: 1. Define atomic clearly — a value the database cannot meaningfully split further for query purposes. A phone number is atomic; a comma-separated list of phone numbers is not. 2. Give a concrete violation. A `students` table with a `subjects` column holding `'Maths,Physics,Chemistry'` breaks 1NF. You cannot index it, you cannot join on it, you cannot count subjects without string functions. 3. Show the fix — create a `student_subjects` junction table with `(student_id, subject)` rows. Now every cell is atomic and queries become first-class SQL. 4. Mention the modern wrinkle — JSON and array columns in Postgres or MySQL technically hold non-atomic values, but most engineers accept them because the database can index inside them. Be ready to defend that nuance if the interviewer pushes.

Edge cases to mention: - Repeating groups across columns (`phone1` , `phone2` , `phone3`) also violate 1NF in spirit even though each cell is atomic — same fix, move to a child table. - A primary key is required for 1NF — a table with duplicate rows and no key is not in 1NF at all. - ENUMs are atomic — a column holding `'pending'` or `'paid'` is fine, only a list of statuses would be a problem.

What NOT to say: - Do not say "1NF means no NULLs" — that is a common misconception, NULLs are perfectly allowed. - Do not claim a JSON column always violates 1NF without nuance — many production systems intentionally use JSONB for flexible attributes.

Common follow-up: *"What is the practical cost of leaving a comma-separated column in production?"* — You lose indexes, you lose joinability, you lose referential integrity, and every query needs `LIKE '%X%'` which is unindexable and wrong (`'Maths'` matches `'Mathsy'`).

```
-- Violates 1NF
CREATE TABLE students_bad (id INT, name TEXT, subjects TEXT);
INSERT INTO students_bad VALUES (1, 'Riya', 'Maths,Physics');

-- 1NF compliant
CREATE TABLE students (id INT PRIMARY KEY, name TEXT);
CREATE TABLE student_subjects (
    student_id INT REFERENCES students(id),
    subject TEXT,
    PRIMARY KEY (student_id, subject)
);
```

Q269 · Second Normal Form — Killing Partial Dependencies

Tags: Difficulty: Easy · Asked at: Infosys, TCS, Accenture, HCL · Topic: Normal Forms

The question (interviewer's verbatim phrasing):

Explain Second Normal Form with an example. Why does 2NF only matter when you have a composite primary key?

The 30-second answer: 2NF says no non-key column depends on only part of a composite key. It only matters when the key is composite — if your key is a single column, you cannot have a partial dependency, so 2NF is automatic. Fix violations by pulling the partially dependent columns into a separate table keyed on just the relevant part.

Step-by-step thought process: 1. Set up the classic example — an `order_items` table with composite key `(order_id, product_id)`. Columns `quantity` and `unit_price` depend on the full key, but `product_name` depends only on `product_id`. That is a partial dependency. 2. Explain why it is bad — `product_name` repeats for every order line, so changing the product name means updating thousands of rows, and the same name can drift to different spellings. 3. Show the fix — move `product_name` to a `products` table keyed on `product_id`. Now `order_items` only stores facts about the line, and product metadata lives in one place. 4. Hammer the single-column-key point — if your PK is just `order_id`, every other column depends on the whole key by definition. 2NF is only interesting with composite keys.

Edge cases to mention: - If you use a surrogate key like `id BIGSERIAL` and ignore the natural composite, you can hide 2NF violations behind the surrogate — the underlying dependency still exists. - 2NF assumes the table is already in 1NF — if you have repeating groups, fix that first. - Reporting tables and OLAP fact tables often violate 2NF deliberately to avoid runtime joins.

What NOT to say: - Do not say "2NF means no duplicates" — that is what a primary key already enforces, 2NF is about dependencies. - Do not skip 1NF as a prerequisite — normal forms are a ladder, you cannot jump rungs.

Common follow-up: *"How would you spot a 2NF violation in a code review?"* — Look for composite PKs where any non-key column repeats identically across rows that share part of the key — that repetition is the smoking gun.

```
-- 2NF violation: product_name depends only on product_id
CREATE TABLE order_items_bad (
  order_id INT, product_id INT,
  product_name TEXT, quantity INT, unit_price NUMERIC,
  PRIMARY KEY (order_id, product_id)
);

-- 2NF compliant
CREATE TABLE products (product_id INT PRIMARY KEY, product_name TEXT);
CREATE TABLE order_items (
  order_id INT, product_id INT REFERENCES products,
  quantity INT, unit_price NUMERIC,
  PRIMARY KEY (order_id, product_id)
);
```

Q270 · Third Normal Form — No Transitive Dependencies

Tags: Difficulty: Easy · Asked at: TCS, Wipro, Cognizant, Capgemini · Topic: Normal Forms

The question (interviewer's verbatim phrasing):

Define Third Normal Form. What is a transitive dependency and how do you remove it?

The 30-second answer: 3NF says every non-key column depends on the key, the whole key, and nothing but the key — a transitive dependency is when a non-key column depends on another non-key column instead of the primary key. Fix it by splitting the dependent group into its own table.

Step-by-step thought process: 1. Use a concrete example — an `employees` table with `(emp_id, name, dept_id, dept_name, dept_location)`. The key is `emp_id`, but `dept_name` and `dept_location` depend on `dept_id`, not directly on `emp_id`. That is transitive — `emp_id` determines `dept_id` which determines `dept_name`. 2. Explain why it bites — change a department's name and you update every employee row, with the usual drift and inconsistency risk. 3. Show the fix — create a `departments` table keyed on `dept_id` holding `dept_name` and `dept_location`, and leave only `dept_id` in `employees`. 4. Make the contrast with 2NF clear —

2NF is about partial dependencies on a composite key, 3NF is about chained dependencies through any non-key column.

Edge cases to mention: - Calculated columns like `total = quantity * price` look transitive but are usually treated as denormalization for performance, not a 3NF concern — the dependency is on a function, not another stored column. - Some 3NF violations are intentional — caching `country_name` next to `country_code` saves a join and rarely changes. - 3NF still allows redundancy across tables — only within a single table is the rule strict.

What NOT to say: - Do not say "3NF means foreign keys everywhere" — foreign keys are about referential integrity, not normalization. - Do not confuse 3NF with BCNF — they overlap heavily but differ on edge cases with overlapping candidate keys.

Common follow-up: *"Is 3NF enough for production?"* — For OLTP, yes, almost always. For OLAP and reporting, 3NF is usually too join-heavy and people denormalize into star schemas.

```
-- 3NF violation: dept_name depends on dept_id, not emp_id
CREATE TABLE employees_bad (
  emp_id INT PRIMARY KEY, name TEXT,
  dept_id INT, dept_name TEXT, dept_location TEXT
);

-- 3NF compliant
CREATE TABLE departments (
  dept_id INT PRIMARY KEY, dept_name TEXT, dept_location TEXT
);
CREATE TABLE employees (
  emp_id INT PRIMARY KEY, name TEXT,
  dept_id INT REFERENCES departments
);
```

Q271 · Boyce-Codd Normal Form — Stricter Than 3NF

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, Walmart Labs · Topic: Normal Forms

The question (interviewer's verbatim phrasing):

What is BCNF and how is it different from 3NF? Give me an example where a table is in 3NF but not in BCNF.

The 30-second answer: BCNF says every functional determinant must itself be a candidate key. 3NF allows a non-prime attribute to depend on a candidate key, but BCNF closes the loophole where a prime attribute depends on a non-key column. They differ only in rare cases with overlapping candidate keys.

Step-by-step thought process: 1. Establish the rule clearly — if $X \rightarrow Y$ holds, then X must be a superkey. No exceptions. 2. Give the canonical example — a course-instructor-room table where $(\text{course}, \text{day}) \rightarrow \text{room}$ and $\text{room} \rightarrow \text{course}$. The candidate keys are $(\text{course}, \text{day})$ and $(\text{room}, \text{day})$. The dependency $\text{room} \rightarrow \text{course}$ has room on the left, which is not a candidate key — that violates BCNF even though the table is in 3NF. 3. Show the fix — decompose into two tables, one mapping $\text{room} \rightarrow \text{course}$ and another mapping $(\text{course}, \text{day}) \rightarrow \text{room}$. The decomposition is lossless but loses one functional dependency. 4. Acknowledge the trade-off — BCNF decomposition can lose dependencies that 3NF preserves, which is why some teams stop at 3NF deliberately.

Edge cases to mention: - Most 3NF tables are already in BCNF — the violating cases need overlapping candidate keys, which are uncommon in well-designed schemas. - BCNF decomposition is always lossless join but not always dependency-preserving — a known limitation. - For tables with a single candidate key, 3NF and BCNF are identical — there is no way to violate one without the other.

What NOT to say: - Do not say "BCNF is just 3NF with stricter rules and you should always aim for it" — sometimes 3NF is the right stopping point because BCNF loses dependencies. - Do not invent BCNF violations from textbook examples without overlapping candidate keys.

Common follow-up: "When would you deliberately stay in 3NF and not go to BCNF?" — When the BCNF decomposition would force you to lose a functional dependency that you want the database to enforce via a single table constraint.

```
-- 3NF but not BCNF: room -> course, but room is not a candidate key
CREATE TABLE course_schedule (
  course TEXT, day TEXT, room TEXT,
  PRIMARY KEY (course, day),
  UNIQUE (room, day)
);
-- BCNF fix: two tables, lossless but loses the room->course dependency
```

Q272 · 4NF and 5NF — Multi-Valued and Join Dependencies

Tags: Difficulty: Hard · Asked at: Goldman Sachs, JP Morgan, Oracle India · Topic: Normal Forms

The question (interviewer's verbatim phrasing):

Briefly, what do 4NF and 5NF address? Have you ever encountered a real-world violation?

The 30-second answer: 4NF removes multi-valued dependencies — when a single key implies two independent sets of values. 5NF removes join dependencies that cannot be inferred from candidate

keys. In practice, most production schemas never explicitly worry about 4NF or 5NF because they would be caught earlier as obvious modelling mistakes.

Step-by-step thought process: 1. Define a multi-valued dependency — $X \twoheadrightarrow Y$ means X determines a set of Y values independent of any other column. If a table holds $(student, language, hobby)$ where languages and hobbies are independent, you have two MVDs and a 4NF violation. 2. Show the 4NF fix — split into $(student, language)$ and $(student, hobby)$. Joining them back gives the original, but without the spurious cross-product rows. 3. Touch 5NF lightly — it handles join dependencies that need three or more tables to reconstruct. The textbook example is $(supplier, part, project)$ where you can only reconstruct the truth by joining three binary projections. Real production almost never hits this honestly. 4. Be honest in the answer — admit these are rare and most engineers never explicitly check for them. Interviewers asking 4NF or 5NF are usually testing depth of knowledge, not expecting war stories.

Edge cases to mention: - A 4NF violation often hides as a single table with two independent multi-select fields — splitting them is almost always the right call. - 5NF is more theoretical than practical — many textbooks treat it as a curiosity. - If a 4NF or 5NF violation is profitable to keep (rare), document it clearly.

What NOT to say: - Do not pretend you fight 5NF battles every sprint — it is intellectually dishonest and interviewers see through it. - Do not confuse multi-valued dependencies with multi-valued columns — the former is structural, the latter breaks 1NF.

Common follow-up: *"If 4NF and 5NF are so rare, why do textbooks teach them?"* — They complete the theory and prove that 3NF and BCNF are not always enough. Knowing they exist is more important than using them daily.

```
-- 4NF violation: student determines languages AND hobbies independently
-- (student, language, hobby) with cartesian product rows

-- 4NF fix: two separate tables
CREATE TABLE student_languages (student_id INT, language TEXT, PRIMARY KEY (student_id, language))
CREATE TABLE student_hobbies (student_id INT, hobby TEXT, PRIMARY KEY (student_id, hobby));
```

Q273 · Denormalization — When and Why

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe, Cred · Topic: Schema Design

The question (interviewer's verbatim phrasing):

Tell me about a time you deliberately denormalized a schema. What did you gain and what did you give up?

The 30-second answer: You denormalize when read amplification is killing you — joins, aggregates, or cross-table lookups happen on every request and the latency budget will not stretch. You gain query speed and simpler reads, and you give up write consistency, more storage, and the need for application-level discipline to keep duplicated data in sync.

Step-by-step thought process: 1. Lead with a real scenario. An order listing page joins orders, users, addresses, and payment status — at scale, the join cost crushes p95 latency, so you cache the user name and shipping city directly on the `orders` row. 2. State the trade-offs explicitly — every user name change now needs to update orders, or you accept stale data, or you run a backfill job nightly. 3. Mention common patterns — counter columns (`total_orders` on `users`), denormalized search columns, materialized aggregates on dashboards. 4. Show the discipline — denormalization should be a deliberate decision logged in an ADR, not an accidental schema drift. And it usually needs invariant checks or reconciliation jobs to catch drift.

Edge cases to mention: - For write-heavy systems, denormalization can actually hurt because every write fans out to multiple rows. - Some denormalizations are safe because the source is immutable — copying `country_code` → `country_name` rarely needs sync. - Denormalization is easier to add than to remove — overcommitting is a common regret.

What NOT to say: - Do not say "denormalization is always for performance" — sometimes it is for simplification of downstream consumers. - Do not denormalize speculatively — wait for actual evidence of join cost, then measure the difference.

Common follow-up: *"How do you keep denormalized columns in sync?"* — Triggers, application-level writes in the same transaction, async event consumers, or periodic reconciliation jobs. Each has trade-offs; pick consciously.

```
-- Denormalized order table for fast listing
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT,
  user_name TEXT,          -- denormalized snapshot
  shipping_city TEXT,      -- denormalized snapshot
  total_amount NUMERIC,
  status TEXT
);
```

Q274 · OLTP vs OLAP — Two Different Schema Philosophies

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, AmEx, JP Morgan · Topic: Schema Design

The question (interviewer's verbatim phrasing):

What is the difference in schema philosophy between OLTP and OLAP systems? Can the same schema serve both?

The 30-second answer: OLTP is built for short, write-heavy transactions and prefers fully normalized schemas to avoid update anomalies. OLAP is built for read-heavy analytical scans and prefers denormalized star or snowflake schemas to avoid joins. Trying to serve both from one schema usually compromises both — most teams replicate OLTP data into a separate analytical store.

Step-by-step thought process: 1. Start with the workload contrast — OLTP does many small queries touching few rows each, OLAP does few queries touching millions of rows each. 2. Show how schema follows workload — OLTP normalizes to make writes cheap and consistent, OLAP denormalizes to make scans and aggregates cheap. 3. Explain the indexing difference — OLTP uses point-lookup B-tree indexes, OLAP prefers columnar storage with bitmap or zone indexes. 4. Address the "can one schema serve both" question — usually no. ETL pipelines or CDC streams ship OLTP rows into a warehouse where the schema is reshaped into facts and dimensions.

Edge cases to mention: - HTAP systems (TiDB, SingleStore) attempt to serve both workloads from one engine — they trade off some of each world's optimality. - Small startups often use the OLTP database for analytics for a long time and only move once it hurts. - OLAP queries on OLTP can block writes and trigger lock contention — a real operational risk.

What NOT to say: - Do not say "OLAP is just OLTP with more data" — the access patterns are fundamentally different. - Do not propose running analytical workloads against your primary database in an interview answer — interviewers win.

Common follow-up: "How do you keep the warehouse fresh?" — Batch ETL on a schedule, CDC streaming via tools like Debezium, or near-real-time pipelines. Pick based on freshness SLA and engineering cost.

```
-- OLTP normalized
SELECT u.name, p.amount
FROM users u JOIN payments p ON u.id = p.user_id
WHERE u.id = 123;

-- OLAP denormalized fact table
SELECT region, SUM(amount) FROM fct_payments
WHERE year = 2026 GROUP BY region;
```

Q275 · Star Schema — Fact and Dimension Tables

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target India, Flipkart, AmEx · Topic: Data Warehousing

The question (interviewer's verbatim phrasing):

Walk me through the star schema. What goes in a fact table, what goes in a dimension table, and why is it called a star?

The 30-second answer: A star schema has one central fact table holding numeric measures and foreign keys, surrounded by dimension tables holding descriptive attributes — drawing the relationships looks like a star. Facts are usually narrow and very long; dimensions are wide and short.

Step-by-step thought process: 1. Define the fact table — rows are events or transactions, columns are quantitative measures (amount, quantity, duration) plus foreign keys into dimensions. 2. Define dimension tables — `dim_customer`, `dim_product`, `dim_date`, each holding descriptive context for the facts. 3. Explain why it scans fast — analytical queries filter on dimension attributes and aggregate fact measures, which works well with columnar storage and bitmap indexes on the dimension keys. 4. Mention the date dimension specifically — it is the most common dimension table because slicing by week, quarter, fiscal year, or holiday flag is a standard analytical need.

Edge cases to mention: - A fact table can be billions of rows; dimensions are usually thousands to millions. - Conformed dimensions (the same `dim_customer` shared across multiple fact tables) make cross-domain analysis possible. - Some facts have zero measures and exist only to record an event — these are called factless fact tables.

What NOT to say: - Do not describe star schema as just denormalized 3NF — the modelling intent is fundamentally different. - Do not put descriptive attributes in the fact table — that bloats every row and breaks the model.

Common follow-up: "What is grain?" — The level of detail in each fact row. One fact row per order line is a different grain from one fact row per order. Pick the finest grain you might ever need.

```
CREATE TABLE fct_sales (  
    sale_id BIGINT, sale_date_key INT, customer_key INT, product_key INT,  
    store_key INT, quantity INT, amount NUMERIC  
);  
CREATE TABLE dim_customer (customer_key INT PRIMARY KEY, name TEXT, city TEXT, segment TEXT);  
CREATE TABLE dim_product (product_key INT PRIMARY KEY, name TEXT, category TEXT, brand TEXT);
```

Q276 · Snowflake Schema — When Normalized Dimensions Help

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Oracle India · Topic: Data Warehousing

The question (interviewer's verbatim phrasing):

What is a snowflake schema and when would you use it over a star schema?

The 30-second answer: A snowflake schema normalizes the dimension tables, so a `dim_product` might reference `dim_category` which references `dim_department`. You snowflake when dimensions are huge or change at different rates and storage savings or referential cleanliness outweigh the extra join cost.

Step-by-step thought process: 1. Describe the structure — dimensions decomposed into related tables, joined in a hierarchical pattern that looks like a snowflake. 2. Explain the gain — less duplicated text in wide dimensions, easier maintenance when a category name changes, cleaner separation when different teams own different levels of the hierarchy. 3. Explain the cost — analytical queries now require more joins, which hurts performance on wide aggregates and adds query authoring complexity. 4. State the practical rule — most modern warehouses prefer star schemas because storage is cheap and BI tools generate cleaner SQL against flat dimensions. Snowflake only when you have a strong reason.

Edge cases to mention: - Slowly changing dimensions (SCD type 2) make snowflaking harder because history needs to be tracked across multiple normalized tables. - Some BI tools require flattened dimensions and will materialize the joins anyway — defeats the snowflake benefit. - Columnar databases compress repeated dimension values so well that snowflaking saves less storage than you think.

What NOT to say: - Do not say "snowflake is always better because it is more normalized" — it is a trade-off, not an upgrade. - Do not confuse Snowflake the schema with Snowflake the data warehouse vendor — they are unrelated.

Common follow-up: *"How does this affect query performance?"* — More joins, longer plans, more memory for hash joins. On large dimensions the difference is measurable; on small ones it is negligible.

```
-- Snowflake variant: product references category, category references department
CREATE TABLE dim_department (department_key INT PRIMARY KEY, name TEXT);
CREATE TABLE dim_category (category_key INT PRIMARY KEY, name TEXT, department_key INT);
CREATE TABLE dim_product (product_key INT PRIMARY KEY, name TEXT, category_key INT);
```

Q277 · Slowly Changing Dimensions — Types 1, 2, 3, 6

Tags: Difficulty: Hard · Asked at: Walmart Labs, AmEx, Target India, JP Morgan · Topic: Data Warehousing

The question (interviewer's verbatim phrasing):

Explain the different types of slowly changing dimensions. When would you pick SCD type 2 over type 1?

The 30-second answer: Type 1 overwrites and loses history. Type 2 inserts a new row with effective dates so all history is preserved. Type 3 adds a column for the previous value, keeping only one step of history. Type 6 combines all three. Pick type 2 when historical accuracy matters for analysis — type 1 when you only ever care about current state.

Step-by-step thought process: 1. Use a customer dimension example. The customer moves from Pune to Bengaluru. Type 1 overwrites the city. Type 2 inserts a new row with new `effective_from` / `effective_to` and marks the old one expired. Type 3 adds `previous_city`. Type 6 keeps the type 2 history plus a current-value column for easy filtering. 2. Justify type 2 — if you ask "how many sales came from Pune in 2024", type 1 lies because the customer now shows Bengaluru, and historical sales appear to come from Bengaluru. Type 2 ties each sale to the dimension row that was current at the time. 3. Mention surrogate keys — type 2 absolutely requires surrogate keys because the natural customer ID stays the same across versions, but each version needs a distinct dimension key. 4. Discuss type 6 briefly — it satisfies both "current state" and "historical state" queries, at the cost of redundancy.

Edge cases to mention: - Type 2 dimensions grow without bound — partitioning by year or archiving expired rows becomes important. - Bi-temporal modelling (effective time + transaction time) extends SCD type 2 for late-arriving data. - Some teams use append-only event tables instead of SCDs and reconstruct state via window functions.

What NOT to say: - Do not say "type 2 is always best" — it is heavier and unnecessary for attributes that genuinely do not need history. - Do not skip the surrogate key requirement — it is what makes type 2 work mechanically.

Common follow-up: "How do you join a fact table to a type 2 dimension correctly?" — Join on the surrogate key that was active at the fact's event time, not the natural key. The fact captures the surrogate at insert time.

```
CREATE TABLE dim_customer (  
  customer_key BIGSERIAL PRIMARY KEY,  
  customer_id INT,           -- natural key, repeats across versions  
  name TEXT, city TEXT,  
  effective_from DATE, effective_to DATE,  
  is_current BOOLEAN  
);
```

Q278 · Surrogate Key vs Natural Key

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Flipkart · Topic: Keys

The question (interviewer's verbatim phrasing):

When would you use a surrogate key over a natural key? Give me a case where the natural key wins.

The 30-second answer: Use a surrogate key when the natural key is mutable, composite, externally controlled, or not guaranteed unique under all future scenarios. Use the natural key when it is small, immutable, meaningful to humans, and convenient for direct lookup — like a country code in a `dim_country` table.

Step-by-step thought process: 1. Define each — surrogate is a system-generated key with no business meaning (BIGSERIAL, UUID), natural is a key that exists in the real world (PAN, GSTIN, ISBN, email). 2. List surrogate advantages — immutable, narrow, fast to join, decoupled from business changes. If the natural key changes (rare, but happens — like a renamed product SKU), the surrogate stays put. 3. List natural advantages — no extra lookup needed, smaller schema, queries are self-documenting (`WHERE country_code = 'IN'` reads better than `WHERE country_key = 47`). 4. Practical rule — most production OLTP tables use a surrogate primary key and add a unique index on the natural key. You get the best of both.

Edge cases to mention: - PAN, Aadhaar, GSTIN feel natural but are mutable in edge cases — surrogate is safer. - Junction tables can use the composite of the two foreign keys as the natural primary key — no surrogate needed. - For very small reference tables (status codes, country lists), natural keys work well.

What NOT to say: - Do not say "always use surrogate keys" — that is dogmatic, and natural keys are correct in many lookup tables. - Do not use email as a primary key — emails change, and cascading updates across foreign keys is painful.

Common follow-up: *"What happens if a natural key changes after thousands of foreign key references exist?"* — Without ON UPDATE CASCADE you break referential integrity; with it, you trigger expensive cascading writes. A surrogate avoids both.

```
-- Surrogate key with unique natural key index
CREATE TABLE customers (
  customer_id BIGSERIAL PRIMARY KEY,    -- surrogate
  pan_number CHAR(10) UNIQUE NOT NULL, -- natural key
  name TEXT
);
```

Q279 · Composite Key vs Surrogate ID

Tags: Difficulty: Medium · Asked at: Infosys, TCS, Razorpay, PhonePe · Topic: Keys

The question (interviewer's verbatim phrasing):

A junction table like `order_items` — do you keep the composite `(order_id, product_id)` as the primary key, or add a surrogate `id` ?

The 30-second answer: Default to the composite — it is the natural identity and self-enforces uniqueness. Add a surrogate only when the row is referenced by other tables, when the composite is wide and ugly to propagate, or when you anticipate the composite needing to change.

Step-by-step thought process: 1. Lay out the composite case — `(order_id, product_id)` is the truth. It enforces that one product appears once per order without an extra constraint. 2. Lay out the surrogate case — if another table needs to reference a specific `order_item`, propagating two columns is awkward, and a single `order_item_id` is cleaner. 3. Mention the multiple-rows-same-product nuance — if the business allows the same product twice in an order with different unit prices (rare but possible for promotions), the composite no longer holds and a surrogate becomes necessary. 4. Acknowledge framework preferences — many ORMs default to surrogate IDs everywhere, so teams using Hibernate or Django often add a surrogate just for consistency, even on junction tables.

Edge cases to mention: - Composite keys hurt index size when wide — three or four columns of TEXT can make a 200-byte index entry. - If the junction stores additional columns (`quantity`, `unit_price`), some teams call it an associative entity and lean towards a surrogate. - Wide composite keys propagate into foreign-key tables and bloat them too — a hidden cost.

What NOT to say: - Do not say "always add a surrogate to junction tables" — it adds storage and weakens the natural uniqueness constraint. - Do not omit the unique index when you add a surrogate — without it, you can insert the same `(order_id, product_id)` twice.

Common follow-up: "If you add a surrogate, where do you put the uniqueness constraint?" — A UNIQUE index on the original composite columns — otherwise the surrogate masks duplicates.

```
-- Composite key as PK
CREATE TABLE order_items (
  order_id INT, product_id INT, quantity INT, unit_price NUMERIC,
  PRIMARY KEY (order_id, product_id)
);

-- Surrogate variant with uniqueness preserved
CREATE TABLE order_items (
  id BIGSERIAL PRIMARY KEY,
  order_id INT, product_id INT, quantity INT, unit_price NUMERIC,
  UNIQUE (order_id, product_id)
);
```

Q280 · UUID vs Auto-Increment ID

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Swiggy, PhonePe · Topic: Keys

The question (interviewer's verbatim phrasing):

UUID or auto-increment integer for primary keys? Walk me through the trade-offs.

The 30-second answer: Auto-increment is small, fast, and great for index locality, but it leaks information (row count, growth rate) and serializes writes on the sequence. UUIDs are globally unique, generatable client-side, and safe to expose, but they are wide, random, and destroy B-tree locality which slows inserts on large tables.

Step-by-step thought process: 1. Start with the size and locality angle — a BIGINT is 8 bytes, a UUID is 16 bytes plus index overhead. Random UUIDs cause B-tree page splits during inserts, hurting write throughput on large tables. 2. List integer drawbacks — sequential IDs let attackers enumerate (`/orders/1234` then `/orders/1235`), and a centralized sequence is a write hotspot in distributed systems. 3. List UUID drawbacks — bigger storage, bigger indexes, random insert order, harder to read in logs and URLs. 4. Mention the modern compromise — ULID, KSUID, UUIDv7, and Snowflake IDs are sortable and time-prefixed, giving you UUID-like uniqueness with much better locality.

Edge cases to mention: - MySQL InnoDB primary keys are physically clustered, so random UUIDs hurt much more than they do in Postgres heap tables. - UUID v1 leaks the MAC address; UUID v4 is fully random and safer for exposure. - Some teams use BIGINT internally and UUID externally — best of both worlds with a translation layer.

What NOT to say: - Do not say "UUIDs are always better because they are unique" — auto-increment is also unique within the table. - Do not ignore index size — on a billion-row table, the difference is measured in gigabytes.

Common follow-up: *"How do UUIDs affect replication?"* — They let you generate IDs offline and merge later without collisions, which is valuable in mobile-first or multi-master setups.

```
-- Auto-increment
CREATE TABLE orders (id BIGSERIAL PRIMARY KEY, ...);

-- UUID
CREATE TABLE orders (id UUID PRIMARY KEY DEFAULT gen_random_uuid(), ...);
```

Q281 · ULID, KSUID, Snowflake — Sortable Alternatives

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart, PhonePe · Topic: Keys

The question (interviewer's verbatim phrasing):

You want global uniqueness without sacrificing insert locality. What are your options beyond UUID?

The 30-second answer: ULID, KSUID, UUIDv7, and Twitter Snowflake IDs all prefix a timestamp to the random portion, so IDs generated later sort after IDs generated earlier. You get UUID-like uniqueness with index locality close to auto-increment.

Step-by-step thought process: 1. Explain why sortability matters — when IDs are inserted in order, the B-tree grows on the right edge and page splits stay minimal, preserving insert throughput. 2. Describe each — ULID is 26 chars base32 with 48-bit timestamp + 80-bit randomness; KSUID is 27 chars with 32-bit timestamp; UUIDv7 is RFC standard with millisecond timestamp; Snowflake is 64-bit with timestamp + worker + sequence. 3. State trade-offs — ULID and KSUID need a library (no native support yet in most DBs), UUIDv7 is becoming standard, Snowflake needs a coordinator for worker IDs. 4. Recommend UUIDv7 for new green-field work — it is RFC standard, sortable, generatable client-side, and is gaining first-class database support.

Edge cases to mention: - All these schemes leak the creation timestamp — if that is sensitive, stick with random UUIDs. - Clock skew across servers can produce out-of-order IDs — use NTP and accept the rare local reordering. - Snowflake IDs need careful worker ID assignment; collisions cause silent duplicates.

What NOT to say: - Do not say "they are all the same" — the encoding, length, and timestamp resolution differ in ways that matter. - Do not invent your own ID scheme — pick a known one and survive code review.

Common follow-up: *"How do you generate ULIDs from your application server?"* — A library generates them client-side; the database just stores them as a CHAR(26) or BYTEA — no central coordinator needed.

```
-- UUIDv7 (Postgres 18+) or use uuid_v7() extension
CREATE TABLE events (id UUID PRIMARY KEY DEFAULT uuid_v7(), payload JSONB);
```

Q282 · Junction Tables for Many-to-Many

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Schema Design

The question (interviewer's verbatim phrasing):

Students and courses — a student can enrol in many courses and a course has many students. How do you model that?

The 30-second answer: A junction table `enrolments` with two foreign keys, `(student_id, course_id)`, plus any attributes specific to the enrolment itself (grade, enrolment date). The composite is the primary key.

Step-by-step thought process: 1. State the rule — many-to-many always needs an intermediate table. There is no clean way to do it with just two tables. 2. Define the junction — minimal version is two FKs forming a composite PK. Richer version adds enrolment-specific attributes that belong neither to the student nor the course. 3. Discuss naming — call it after the relationship (`enrolments`, `subscriptions`) not after the two parents (`student_course`) when possible. Better readability and supports future attributes. 4. Mention the natural extension — if the junction grows attributes, it stops being a pure junction and becomes an associative entity, which often takes a surrogate PK.

Edge cases to mention: - Self-referencing many-to-many uses two FKs into the same parent table (`user_id`, `friend_id` in a friends table). - If the same pair must repeat (a student takes the same course twice), the composite PK fails — switch to a surrogate plus a timestamp. - Cascading deletes need careful thought — deleting a student should usually cascade to enrolments, but a course delete might be restricted.

What NOT to say: - Do not stuff course IDs into a comma-separated column on the student row — that violates 1NF and breaks everything. - Do not omit the foreign key constraints — referential integrity is free protection.

Common follow-up: "How do you efficiently query all students in a course?" — Index the junction on `(course_id, student_id)` for that direction, and on `(student_id, course_id)` for the reverse — covering indexes if you need only the IDs.

```
CREATE TABLE enrolments (  
  student_id INT REFERENCES students,  
  course_id INT REFERENCES courses,  
  enrolled_on DATE,  
  grade CHAR(1),  
  PRIMARY KEY (student_id, course_id)  
);
```

Q283 · Polymorphic Association — The SQL Anti-Pattern

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart, Swiggy · Topic: Schema Design

The question (interviewer's verbatim phrasing):

A `comments` table has `commentable_id` and `commentable_type` so it can comment on articles, photos, or videos. What is wrong with this design?

The 30-second answer: Polymorphic association breaks referential integrity — the database cannot enforce a foreign key when the target table varies by row. You lose constraint protection, join performance suffers, and queries need conditional joins or unions. Better alternatives are exclusive foreign keys, an inheritance pattern, or a separate junction table per parent type.

Step-by-step thought process: 1. Diagnose the problem — `commentable_id` references different tables depending on `commentable_type`, so no single FK constraint can guard it. Orphan rows can accumulate silently. 2. Describe the alternatives — option A is multiple nullable FKs with a CHECK constraint enforcing exactly one is set; option B is a separate `commentable` parent table with subtype tables; option C is dedicated `article_comments`, `photo_comments`, `video_comments` tables. 3. State the trade-off — polymorphic feels elegant in ORMs (Rails, Django) but the database cannot help you, so bugs surface in production rather than at schema time. 4. Recommend option A or C for most cases — they keep referential integrity and let the optimizer plan joins properly.

Edge cases to mention: - Some teams accept polymorphic association in low-stakes tables (notifications, audit logs) where occasional orphans are tolerable. - Postgres now supports inheritance and partitioning that can model polymorphic-like patterns cleanly. - Polymorphic FKs make analytics queries painful — every report needs the type conditional.

What NOT to say: - Do not call it elegant just because the ORM hides it — that is what the framework wants you to think. - Do not say "we can enforce it in application code" — application code drifts, the database does not forget.

Common follow-up: *"How do you migrate away from polymorphic association safely?"* — Add the proper FK columns, dual-write for a period, backfill, validate, then drop the polymorphic columns. Standard staged migration.

```
-- Polymorphic (anti-pattern)
CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  commentable_id BIGINT, commentable_type TEXT,
  body TEXT
);

-- Exclusive FK alternative
CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  article_id BIGINT REFERENCES articles,
  photo_id BIGINT REFERENCES photos,
  video_id BIGINT REFERENCES videos,
  body TEXT,
  CHECK (num_nonnulls(article_id, photo_id, video_id) = 1)
);
```

Q284 · Entity-Attribute-Value — When It Makes Sense

Tags: Difficulty: Hard · Asked at: Healthcare GCCs, Oracle India, Salesforce India · Topic: Schema Design

The question (interviewer's verbatim phrasing):

When is it acceptable to use the EAV pattern, and when is it definitely the wrong call?

The 30-second answer: EAV makes sense for sparse, user-defined attribute sets where the schema cannot be known in advance — clinical observations, custom form builders, product catalogues with thousands of categories. It is the wrong call for any attribute set you can enumerate, because EAV destroys type safety, query simplicity, and performance.

Step-by-step thought process: 1. Describe the structure — three columns: entity ID, attribute name, value (often stored as text and cast). You can also separate value tables by type (`value_text` , `value_int` , `value_date`). 2. Defend the rare use case — a clinical EHR where physicians record dozens of observation types per patient, most rows sparse, new attributes added without schema migrations. 3. List the harms — queries need self-joins per attribute, indexes are hard, types are lost, aggregations are painful, and the optimizer has no clue what is in the data. 4. Mention the modern alternative — JSONB columns. They preserve flexibility without abandoning the relational model and can be indexed via GIN or expression indexes.

Edge cases to mention: - EAV combined with metadata tables (a table describing valid attributes and their types) is the only sane way to keep some sanity. - Reporting on EAV data usually requires a denormalized pivot — built as a materialized view. - JSONB columns have made true EAV almost obsolete for new applications.

What NOT to say: - Do not call EAV elegant or flexible without listing the costs — interviewers are looking for nuance. - Do not use EAV when you know the attribute set — it is over-engineering for short-term flexibility.

Common follow-up: *"Why is JSONB usually a better choice today?"* — Indexable, queryable with JSON path operators, preserves types within the document, and you do not need self-joins per attribute.

```
-- EAV pattern
CREATE TABLE patient_observations (
  patient_id INT, attribute TEXT, value TEXT, recorded_at TIMESTAMP
);

-- Modern alternative using JSONB
CREATE TABLE patient_observations (
  patient_id INT, observations JSONB, recorded_at TIMESTAMP
);
CREATE INDEX ON patient_observations USING GIN (observations);
```

Q285 · Soft Delete — Pros, Cons, Alternatives

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Flipkart · Topic: Schema Design

The question (interviewer's verbatim phrasing):

A `deleted_at` column on every table — is that a good idea? What are the alternatives?

The 30-second answer: Soft delete is convenient for recovery and audit but pollutes every query with `WHERE deleted_at IS NULL`, breaks unique constraints, weakens foreign keys, and grows tables forever. Alternatives include moving deleted rows to an archive table, append-only event sourcing, or true hard deletes with an audit log.

Step-by-step thought process: 1. Lay out the convenience — undelete is a simple update, audit trail is built in, and historical queries can include past data. 2. Lay out the friction — every read query must filter, unique indexes need to be partial (`UNIQUE (email) WHERE deleted_at IS NULL`), foreign keys can point at soft-deleted rows leading to logical inconsistency, and table size grows without bound. 3. Discuss alternatives — archive tables move deletes elsewhere keeping the live table clean; event sourcing makes deletion a recorded event with state reconstruction; hard delete with separate audit captures the deletion fact without bloating the live table. 4. Recommend a hybrid — soft delete on user-facing entities for short-term recovery, hard delete after retention period, with audit log for forensics.

Edge cases to mention: - Compliance regulations (DPDP, RBI norms) sometimes require true deletion, not soft delete — flag this aggressively in interviews. - Cascading soft deletes are tricky — you have to

write the cascade logic yourself; SQL does not understand `deleted_at` . - Soft delete fails silently with most ORMs that do not auto-filter — every query becomes a bug waiting to happen.

What NOT to say: - Do not propose soft delete as the universal default — it has real costs. - Do not forget about indexes — query plans degrade as the deleted fraction grows.

Common follow-up: *"How do you handle unique constraints with soft delete?"* — Partial unique index that ignores deleted rows, so a deleted user can re-register the same email.

```
-- Soft delete with partial unique index
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY, email TEXT, deleted_at TIMESTAMP
);
CREATE UNIQUE INDEX ON users (email) WHERE deleted_at IS NULL;
```

Q286 · Audit Log — Separate Table or Same Table With History

Tags: Difficulty: Medium · Asked at: JP Morgan, Goldman Sachs, Wells Fargo, AmEx · Topic: Schema Design

The question (interviewer's verbatim phrasing):

You need to track every change to the `accounts` table. Do you keep history in the same table or a separate `accounts_history` table?

The 30-second answer: For OLTP, keep current state in `accounts` and history in `accounts_history` — the live table stays small and fast, and audit queries do not slow normal traffic. For analytical use, sometimes a single bi-temporal table with effective dates is preferable for reconstruction queries.

Step-by-step thought process: 1. Describe the separate-table approach — `accounts` has current rows, `accounts_history` has full snapshots with `valid_from` , `valid_to` , and a `changed_by` audit field. A trigger or application logic writes to history on every update. 2. Describe the same-table approach (SCD type 2 style) — `accounts` has all versions with `is_current` and date columns. Current-state queries need a filter; historical queries are natural. 3. State the trade-off — separate table keeps hot path fast and clean; same table simplifies reconstruction but bloats the working set. 4. Mention temporal tables — some databases (SQL Server, PostgreSQL extensions, Oracle Flashback) support system-versioned temporal tables natively.

Edge cases to mention: - High-write tables generate huge history volumes — partition the history table by month or use compressed columnar storage. - For PII fields, audit log retention may need redaction

rules to comply with privacy laws. - Reconstructing state at an arbitrary timestamp from event logs alone is expensive — snapshots help.

What NOT to say: - Do not say "just use triggers and forget it" — triggers can silently fail and bloat write times. - Do not store history forever without a retention policy — that is a future operations bill.

Common follow-up: "What audit fields do you always include?" — `changed_at` , `changed_by` , `change_type` (insert/update/delete), and the full row payload (often JSONB). Minimum viable audit.

```
CREATE TABLE accounts (id BIGSERIAL PRIMARY KEY, balance NUMERIC, updated_at TIMESTAMP);

CREATE TABLE accounts_history (
  history_id BIGSERIAL PRIMARY KEY,
  account_id BIGINT, balance NUMERIC,
  valid_from TIMESTAMP, valid_to TIMESTAMP,
  changed_by TEXT, change_type TEXT
);
```

Q287 · Append-Only Table Design

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Cred, Walmart Labs · Topic: Schema Design

The question (interviewer's verbatim phrasing):

When does an append-only table make sense and what changes about how you query it?

The 30-second answer: Append-only tables make sense for events, ledger entries, and audit trails where every state change is a fact that must never be edited or deleted. Queries shift to deriving current state via aggregation, window functions, or materialized views over the event stream.

Step-by-step thought process: 1. Define the pattern — inserts only, no updates, no deletes. Mutations become new rows with timestamps. The truth is the sequence, not the latest row. 2. Give examples — bank ledger entries (every transaction is a fact), inventory movements (each stock change recorded), payment events, user activity logs. 3. Discuss querying — current balance is `SUM(amount)` ; current inventory is `SUM(quantity_delta)` ; current status is a window function picking the latest event per entity. 4. Mention performance — append-only is great for write throughput (no row contention), easy to partition by time, easy to archive cold partitions, and naturally compatible with event sourcing.

Edge cases to mention: - Snapshot rows speed up derivation — compute current balance every N events and aggregate forward from there. - Compaction and retention policies are still needed even if individual rows are immutable. - Append-only data is easier to replicate and audit because the order of facts is preserved.

What NOT to say: - Do not claim append-only is always slower for reads — with snapshots and partitioning, it can be comparable to mutable schemas. - Do not forget indexes on `entity_id` + `event_time` — most queries need that combination.

Common follow-up: *"How do you correct a bad event in an append-only table?"* — Insert a compensating event that reverses the original, never modify the original. The audit trail stays clean.

```
CREATE TABLE ledger_entries (  
  entry_id BIGSERIAL PRIMARY KEY,  
  account_id BIGINT, amount NUMERIC,  
  event_time TIMESTAMP, event_type TEXT,  
  reference_id BIGINT  
);  
-- Current balance via aggregation  
SELECT account_id, SUM(amount) FROM ledger_entries GROUP BY account_id;
```

Q288 · Multi-Tenant Schema — Three Strategies

Tags: Difficulty: Hard · Asked at: Salesforce India, Zoho, Freshworks, Razorpay · Topic: Multi-Tenancy

The question (interviewer's verbatim phrasing):

You are designing a SaaS platform serving thousands of business customers. Shared schema, schema-per-tenant, or database-per-tenant — which do you pick and why?

The 30-second answer: Shared schema with `tenant_id` columns scales operationally but needs careful row-level security and noisy-neighbour management. Schema-per-tenant offers isolation but blows up at thousands of tenants. Database-per-tenant gives the strongest isolation and per-customer backups but is the most expensive. Pick by tenant count, isolation needs, and per-tenant scale.

Step-by-step thought process: 1. Explain shared schema — every table has `tenant_id`, every query filters on it, indexes start with `tenant_id`. Cheap, dense, but a missing filter leaks data. Row-level security (RLS) helps enforce. 2. Explain schema-per-tenant — one Postgres schema per customer. Isolation per tenant, separate migrations risk if not automated, connection pool fragmentation, hard at 10000+ tenants. 3. Explain database-per-tenant — strongest isolation, per-tenant backup and restore, per-tenant tuning. Most operationally expensive, only viable for small numbers of large enterprise tenants. 4. Recommend a tiered approach — shared schema for the long tail, dedicated database for the largest enterprise customers who need isolation guarantees.

Edge cases to mention: - Shared schema with RLS is industry standard for B2B SaaS at scale (Salesforce, Shopify largely use shared models). - Per-tenant encryption keys force a per-tenant database for some compliance regimes. - Migrations on shared schema apply once; on schema-per-tenant they iterate; on database-per-tenant they orchestrate.

What NOT to say: - Do not say "always isolate per customer for safety" — at scale that is operationally infeasible. - Do not forget about cross-tenant analytics — shared schema makes it trivial, isolated models make it painful.

Common follow-up: *"How do you prevent a tenant_id filter from being forgotten?"* — Row-level security policies, or wrapping every query through a tenant-aware data access layer that injects the filter.

```
-- Shared schema with RLS
CREATE TABLE invoices (
  id BIGSERIAL PRIMARY KEY,
  tenant_id BIGINT NOT NULL,
  amount NUMERIC
);
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON invoices USING (tenant_id = current_setting('app.tenant_id')::BI
```

Q289 · Sharding Strategies — Range, Hash, Directory

Tags: Difficulty: Hard · Asked at: Flipkart, Razorpay, PhonePe, Swiggy, Cred · Topic: Sharding

The question (interviewer's verbatim phrasing):

Walk me through range, hash, and directory-based sharding. When does each strategy fit?

The 30-second answer: Range sharding splits by key ranges (orders by date, users by ID range) — good for time-ordered queries but creates hotspots on the latest shard. Hash sharding spreads load uniformly via `hash(key) mod N` — good for write balance but bad for range queries. Directory sharding uses a lookup table for flexible mapping — most flexible but adds a dependency.

Step-by-step thought process: 1. Range — keys map to contiguous shards. Easy to reason about, easy to query a date range, but every new write hits the current shard (write hotspot) and resharding requires range splits. 2. Hash — `hash(user_id) mod 16` distributes uniformly. Eliminates hotspots but range queries scatter across all shards, and resharding when N changes is painful unless you use consistent hashing. 3. Directory — a metadata service maps each entity to a shard. Allows arbitrary placement, easy rebalancing, but introduces a lookup hop and the directory becomes a single point of contention. 4. State the practical pattern — hash for user-facing OLTP with even load, range for time-series and analytical workloads, directory for systems needing per-tenant placement control.

Edge cases to mention: - Cross-shard joins and transactions are hard regardless of strategy — design to avoid them. - Consistent hashing softens the resharding pain by minimizing keys that need to move when N changes. - Hot keys (a celebrity user) can crush hash sharding's load balancing — sometimes you split or replicate hot keys separately.

What NOT to say: - Do not say "hash is always best because it balances load" — range queries become impossible. - Do not introduce sharding before you have measured single-node limits — premature sharding is a massive complexity tax.

Common follow-up: *"What does a cross-shard transaction look like?"* — Two-phase commit if you must, but most teams refactor to avoid them entirely via saga patterns or per-shard data ownership.

```
-- Hash sharding example: pick shard via deterministic function
-- shard_id = hash(user_id) % 16
-- Range sharding example: orders by created_at quarter
```

Q290 · Time-Series Schema Design

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, IoT-heavy GCCs · Topic: Schema Design

The question (interviewer's verbatim phrasing):

Design a schema for storing one billion metric readings per day. What are the key choices?

The 30-second answer: Partition by time (usually daily or hourly), keep rows narrow (`entity_id`, `metric_name`, `timestamp`, `value`), use columnar or specialised time-series storage where possible, and aggregate into rollup tables for long-term retention. Avoid wide rows and per-metric tables.

Step-by-step thought process: 1. Time partitioning is non-negotiable — partition by day for queries that scan ranges, drop old partitions for cheap retention enforcement. 2. Schema shape — narrow `(entity_id, metric, timestamp, value)` rows scale better than wide rows with many metric columns, because cardinality varies wildly across metrics. 3. Storage — TimescaleDB, ClickHouse, InfluxDB, and similar specialised stores compress time-series 10x to 100x better than row-oriented OLTP. Use them when scale justifies. 4. Rollups — keep raw data for 7 to 30 days, aggregate to 1-minute then 1-hour buckets for longer history. Storage cost drops dramatically.

Edge cases to mention: - Out-of-order writes are common in IoT — your schema and indexes must accept them gracefully. - Cardinality explosion (millions of distinct metric labels) kills query performance — bound your dimensions. - Compression depends on data sort order — sort by `(entity_id, timestamp)` and watch ratios climb.

What NOT to say: - Do not store time-series in a vanilla normalized OLTP schema at scale — it does not last. - Do not promise unlimited retention without rollups — that is a future operations crisis.

Common follow-up: *"How do you support both real-time and historical queries from one schema?"* — Raw partition for recent data, aggregated rollup tables for older data, a unified view that picks the right source based on the time range.

```
-- Time-partitioned readings (Postgres declarative partitioning)
CREATE TABLE metric_readings (
  entity_id BIGINT, metric TEXT, ts TIMESTAMP, value DOUBLE PRECISION
) PARTITION BY RANGE (ts);

CREATE TABLE metric_readings_2026_05 PARTITION OF metric_readings
FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
```

Q291 · Hierarchical Data — Four Patterns

Tags: Difficulty: Hard · Asked at: Walmart Labs, Flipkart, Oracle India, Razorpay · Topic: Hierarchies

The question (interviewer's verbatim phrasing):

Compare adjacency list, nested set, materialized path, and closure table for storing a category tree.

The 30-second answer: Adjacency list is simple and write-friendly but slow for ancestor queries. Nested set is fast for subtree reads but slow on writes. Materialized path is fast for both with string operations. Closure table is the most expressive — stores every ancestor-descendant pair — at the cost of extra storage and trigger maintenance.

Step-by-step thought process: 1. Adjacency list — each row stores `parent_id`. Trivial to write, but finding all descendants needs a recursive CTE on every read. 2. Nested set — each row stores `left` and `right` numeric bounds. Reading any subtree is a single range query, but every insert or move shifts large numeric ranges, killing write performance on busy trees. 3. Materialized path — each row stores its full path as a string (`/electronics/phones/android`). Queries use `LIKE 'prefix/%'` for descendants and string parsing for ancestors. Fast both ways, harder to enforce integrity. 4. Closure table — separate table with one row per ancestor-descendant pair plus depth. Any tree query is a simple join. Storage is $O(\text{nodes} \times \text{depth})$ and inserts need cascade updates, but the queries are clean.

Edge cases to mention: - Postgres `ltree` extension is a polished materialized path implementation with operators and indexes. - For shallow trees (3-4 levels max), denormalizing `level1_id`, `level2_id`, `level3_id` is often the most pragmatic option. - Closure tables shine when subtree queries dominate and writes are infrequent (org charts, file systems).

What NOT to say: - Do not claim one pattern is universally best — each fits different read-write ratios. - Do not implement nested set without understanding rebalancing cost — it is the classic rookie mistake.

Common follow-up: "For a category tree on an e-commerce site, which would you choose?" — Materialized path with `ltree` for most cases — fast reads, decent writes, and built-in index support.

```
-- Adjacency list
CREATE TABLE categories (id INT PRIMARY KEY, name TEXT, parent_id INT REFERENCES categories);

-- Closure table
CREATE TABLE category_closure (
  ancestor INT, descendant INT, depth INT,
  PRIMARY KEY (ancestor, descendant)
);
```

Q292 · Denormalizing Computed Columns

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: Denormalization

The question (interviewer's verbatim phrasing):

When is it acceptable to store a computed value like `order_total` in the database instead of computing it on read?

The 30-second answer: Store the computed value when the read cost is high, the recomputation is deterministic from a fixed input set, and slight staleness is acceptable or the write path can keep it in sync transactionally. Keep it computed on read when accuracy matters more than latency or the inputs change frequently.

Step-by-step thought process: 1. Establish the trade-off — denormalizing a computed column saves read CPU and join cost but introduces sync risk on every input change. 2. Categorise inputs — if all inputs come from rows you control in the same transaction, atomic updates work. If inputs come from external sources or other services, async updates are inevitable and you accept brief staleness. 3. Mention generated columns — most modern databases support `GENERATED ALWAYS AS (...)` `STORED` for deterministic computations, which keeps the value in sync automatically. 4. Discuss reconciliation — periodic jobs that recompute and detect drift catch the silent bugs that always appear over months.

Edge cases to mention: - Aggregates like `total_orders_count` are notoriously hard to keep in sync under concurrent inserts and deletes — use atomic increments or recompute on a schedule. - Generated columns cannot reference other tables, so they only work for in-row computations. - For dashboards, materialized views often beat ad-hoc denormalized columns — they centralize the refresh.

What NOT to say: - Do not denormalize a value that depends on a frequently-updated input — every input change becomes a fan-out write. - Do not skip the reconciliation job — without it, drift is invisible until a customer complains.

Common follow-up: "What is the safest way to keep `order_total` in sync?" — Compute it in the same transaction that inserts or updates the line items, with a server-side trigger or an explicit update in the application's transactional write path.

```
-- Postgres generated column
CREATE TABLE order_lines (
  id BIGSERIAL PRIMARY KEY,
  quantity INT, unit_price NUMERIC,
  line_total NUMERIC GENERATED ALWAYS AS (quantity * unit_price) STORED
);
```

Q293 · Materialized View vs Denormalization

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, Flipkart, JP Morgan · Topic: Denormalization

The question (interviewer's verbatim phrasing):

What is the difference between using a materialized view and adding denormalized columns to the source table?

The 30-second answer: A materialized view is a separate object refreshed on a schedule or manually, decoupled from the source table's writes. Denormalized columns live in the source table itself and need to be kept in sync transactionally. Materialized views are better for analytical aggregates; denormalized columns are better for hot read paths needing transactional consistency.

Step-by-step thought process: 1. Define a materialized view — a stored result set of a query, refreshable but not auto-synced. Reads are fast; staleness is the cost. 2. Define denormalized columns — extra columns on the source table populated on every relevant write. Real-time accuracy; write complexity is the cost. 3. State when each fits — materialized view for dashboards, reports, and aggregates that tolerate freshness lag. Denormalized columns for per-request lookups where freshness must match the source. 4. Mention refresh modes — Postgres supports `REFRESH MATERIALIZED VIEW CONCURRENTLY` for non-blocking refresh; some databases support incremental refresh for huge views.

Edge cases to mention: - Materialized views can be indexed and queried like regular tables — that is half their value. - Concurrent refresh requires a unique index on the materialized view. - If your aggregate is per-row in the source, denormalized columns scale; if it spans rows, materialized views scale better.

What NOT to say: - Do not say "materialized views are always fresh" — they are not, refresh is on demand. - Do not store materialized aggregates as denormalized columns when they cross many rows — write amplification destroys throughput.

Common follow-up: *"How do you handle near-real-time analytics needs?"* — Either schedule refreshes every minute or use streaming aggregation (CDC + stream processor + materialized table) for true real-time.

```
CREATE MATERIALIZED VIEW mv_daily_sales AS
SELECT DATE(sale_at) AS day, region, SUM(amount) AS total
FROM sales GROUP BY DATE(sale_at), region;

CREATE UNIQUE INDEX ON mv_daily_sales (day, region);
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_sales;
```

Q294 · Schema Migrations — Additive vs Destructive

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Cred, Flipkart, PhonePe · Topic: Migrations

The question (interviewer's verbatim phrasing):

Walk me through deploying a schema change that renames a column on a busy production table. Why is rename considered destructive?

The 30-second answer: A rename is destructive because old application code expects the old name and breaks the moment the migration runs — you cannot deploy schema and code atomically. The safe pattern is additive — add the new column, dual-write from the application, backfill, switch reads, then drop the old column in a later release.

Step-by-step thought process: 1. Distinguish — additive changes (new column nullable, new table, new index concurrently) are safe to deploy any time. Destructive changes (drop, rename, type change) break either old code or old data and require coordination. 2. Lay out the staged rename — release 1 adds the new column and dual-writes to both old and new. Release 2 backfills historical rows. Release 3 switches reads to the new column. Release 4 stops writing the old column. Release 5 drops the old column. 3. Mention online tools — pt-online-schema-change, gh-ost, pg_repack for non-blocking schema changes on huge tables. 4. Note the rollback consideration at every step — if release 3 fails, you must be able to flip back to the old column, which means it must still hold valid data.

Edge cases to mention: - Adding a NOT NULL column with no default is destructive in many engines because it requires a full table rewrite — add nullable, backfill, then add the constraint. - Index creation should always be **CONCURRENTLY** in Postgres or **ONLINE** in MySQL on busy tables. - ENUMs are notoriously painful to migrate — many teams prefer lookup tables for that reason.

What NOT to say: - Do not propose **ALTER TABLE ... RENAME COLUMN** in a single deploy on a live system — it is a recipe for outage. - Do not skip the backfill step — partial data in the new column will silently corrupt downstream consumers.

Common follow-up: *"What happens if you drop the old column too early?"* — Any rolled-back deployment or surviving old process will throw column-not-found errors and the rollback path is broken. Always give a generous deprecation window.

```
-- Staged rename: phone_no -> contact_number
ALTER TABLE customers ADD COLUMN contact_number TEXT;
UPDATE customers SET contact_number = phone_no WHERE contact_number IS NULL;
-- Then in later releases: switch reads, stop writes to old, drop old.
```

Q295 · When to Break Normalization Rules

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart, Swiggy, Walmart Labs · Topic: Schema Philosophy

The question (interviewer's verbatim phrasing):

Give me a real production scenario where you knowingly broke normal form. What was the justification and what was the cost?

The 30-second answer: A common case is snapshotting prices on the `order_lines` row at purchase time — strictly a 3NF violation because the price already exists in the `products` table, but historical orders must reflect the price the customer actually paid, not the current price. The cost is duplicated data; the benefit is correctness over time.

Step-by-step thought process: 1. Lead with the snapshot example — orders must immortalize the deal as it was. If you store only `product_id` and look up the current price at read time, every price change retroactively changes order history. That is wrong. 2. Generalize the principle — normalize to eliminate redundancy of facts that are timeless, denormalize to capture facts that are bound to a point in time. 3. Add a second example — caching `user_country` on `orders` for analytics speed even though it could be joined. The join is cheap; the denormalization is for predictable query plans on a billion-row analytical table. 4. State the discipline — every deliberate denormalization should be documented in the migration, motivated by a measured need, and accompanied by a sync strategy (or a clear "this is a snapshot, do not sync" note).

Edge cases to mention: - Snapshotting includes more than price — product name, tax rate, currency, applied discounts. Anything the customer agreed to at that moment. - Counter caches (`comment_count` on `posts`) are a frequent 3NF violation; they need atomic increments or periodic recompute. - Reporting columns added to OLTP tables for product-team convenience are a slippery slope — push these into a warehouse instead.

What NOT to say: - Do not claim normalization is always best — interviewers want pragmatic judgement. - Do not claim denormalization is free — every duplicated value is a future bug surface.

Common follow-up: *"How do you know when denormalization has gone too far?"* — When you can no longer answer "what is the source of truth for this field" with one sentence, you have a problem.

```
CREATE TABLE order_lines (  
  id BIGSERIAL PRIMARY KEY,  
  order_id BIGINT, product_id BIGINT,  
  product_name_at_purchase TEXT, -- snapshot  
  unit_price_at_purchase NUMERIC, -- snapshot  
  quantity INT  
);
```

Q296 · Database-First vs Code-First Schema Design

Tags: Difficulty: Medium · Asked at: Microsoft IDC, Oracle India, Salesforce India, TCS · Topic: Schema Workflow

The question (interviewer's verbatim phrasing):

Database-first or code-first schema design — which do you prefer and why?

The 30-second answer: Database-first models the schema in DDL or a dedicated tool, then generates or hand-writes code to match. Code-first defines entities in application code and generates the schema from them. Database-first respects the database as the source of truth and is preferred for shared, long-lived systems; code-first is faster for solo or small-team apps where the schema closely mirrors the domain model.

Step-by-step thought process: 1. Define each clearly — database-first starts with DDL scripts, ER diagrams, or tools like Liquibase. Code-first starts with classes in Hibernate, Entity Framework, Django ORM, or SQLAlchemy, and the framework generates migrations. 2. List database-first advantages — full control over constraints, indexes, partitioning, sequences, RLS policies, and edge SQL features. Migrations are explicit and reviewable. 3. List code-first advantages — domain model stays close to code, DTOs map cleanly, less context switching, refactoring tools work end-to-end. 4. State the recommendation — code-first for monoliths owned by one team and one application; database-first when multiple services share the schema or DBAs need a seat at the table.

Edge cases to mention: - Code-first generated migrations are notoriously bad at non-trivial changes (rename, type change, index conditions) — review every generated migration before applying. - Database-first plays better with multi-language ecosystems — one schema, many service clients. - Hybrid approaches are common — code-first for the bulk, with hand-written SQL migrations for the tricky bits.

What NOT to say: - Do not say "ORMs solve everything" — they hide important schema details that bite at scale. - Do not write off code-first as amateur — top product companies ship serious systems with it.

Common follow-up: *"How do you handle conflicts between ORM-generated migrations and DBA-managed schemas?"* — Treat the database as the source of truth, regenerate the ORM model from the schema, and disable ORM auto-migrations in production environments.

```
-- Database-first migration (Liquibase changelog style)
CREATE TABLE invoices (
  id BIGSERIAL PRIMARY KEY,
  tenant_id BIGINT NOT NULL,
  invoice_number TEXT NOT NULL,
  amount NUMERIC(12,2) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE (tenant_id, invoice_number)
);
CREATE INDEX ON invoices (tenant_id, created_at DESC);
```

Chapter 7 — Aggregations & Grouping

Every reporting query you will ever write lives in this chapter — sales by month, top customers by GMV, refund rate by city, distinct payers per campaign — and almost every panel I have sat in spends at least one full question stress-testing whether you understand the rules properly. The mechanics look deceptively simple — SUM here, GROUP BY there — but interviewers know the gotchas: the ANSI rule about every non-aggregated column appearing in GROUP BY, MySQL's old loose mode versus ONLY_FULL_GROUP_BY in 5.7+, the difference between WHERE and HAVING, how COUNT(*) and COUNT(col) and COUNT(DISTINCT col) each behave differently with NULLs, conditional aggregation via SUM(CASE WHEN ...) versus the cleaner FILTER clause, GROUPING SETS and ROLLUP and CUBE for multi-level totals, the GROUPING() function to tell a rollup NULL apart from a data NULL, ordered-set aggregates like PERCENTILE_CONT for median, ARRAY_AGG and STRING_AGG and LISTAGG for collapsing groups into a single value, pivoting via CASE versus a native PIVOT operator, approximate count distinct via HyperLogLog, and how the planner uses (or refuses to use) an index when you GROUP BY an expression. This chapter walks through all of it in the same Q&A shape as Chapters 1–6 so you can drill straight into interview prep.

Q297 · GROUP BY Basics — What Can Appear in SELECT

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Accenture, Wipro · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Explain GROUP BY in your own words. What columns are allowed in the SELECT list of a grouped query, and what happens if you put one that is not allowed?

The 30-second answer: GROUP BY collapses rows that share the same values in the grouping columns into a single output row per group. The SELECT list may only contain the grouping columns themselves, aggregate functions (SUM, COUNT, AVG, MIN, MAX), constants, or expressions built purely from these — any other column raises an error under ANSI SQL.

Step-by-step thought process: 1. Open with the collapsing metaphor — one output row per distinct combination of GROUP BY columns. That sentence alone tells the interviewer you understand the semantics. 2. State the rule precisely: every non-aggregated column in SELECT must appear in the GROUP BY clause. This is the rule the interviewer wants to hear word-for-word. 3. Give a concrete example: `SELECT city, SUM(amount) FROM orders GROUP BY city` — legal. `SELECT city, customer_id, SUM(amount) FROM orders GROUP BY city` — illegal because `customer_id` is neither grouped nor aggregated. 4. Mention that some engines (older MySQL, SQLite) silently pick an arbitrary value for the ungrouped column, which is dangerous — covered properly in Q2 and Q3.

Edge cases to mention: - Postgres allows you to skip listing a column in GROUP BY if it is functionally dependent on the primary key — e.g. you can `SELECT customer_id, customer_name, COUNT(*)` and group only by `customer_id` if `customer_id` is the PK of `customers`. - Aliases from the SELECT list cannot be referenced in GROUP BY in most dialects (Postgres allows it; SQL Server does not). - Constants like `SELECT 'India' AS country, SUM(amount) FROM orders GROUP BY ...` are always fine.

What NOT to say: - Do not claim GROUP BY "sorts" the result — it groups, and ordering is only a historical side-effect of certain plans. - Do not say "you can put any column in SELECT" — that is the MySQL trap from 5.6 days.

Common follow-up: *"Why does ANSI require every non-aggregated column in GROUP BY?"* — Because otherwise the value for that column is ambiguous when the group contains more than one distinct value, and the engine has no principled way to choose.

```
SELECT city, COUNT(*) AS orders, SUM(amount) AS gmv
FROM orders
GROUP BY city;
```

Q298 · Every-Non-Aggregated-Column Rule — ANSI vs MySQL Traditional

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Wipro, TCS · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

If I run `SELECT city, customer_id, SUM(amount) FROM orders GROUP BY city`, what should happen? What actually happens in different databases?

The 30-second answer: Under ANSI SQL the query is invalid and should throw an error because `customer_id` is neither in GROUP BY nor wrapped in an aggregate. Postgres and SQL Server reject it. Old MySQL (and SQLite) accept it and return an arbitrary `customer_id` value picked from one of the rows in each group — useful for nothing and dangerous in production.

Step-by-step thought process: 1. Quote the ANSI rule first: every non-aggregated column must be listed in GROUP BY. Interviewers test for this exact phrasing. 2. Walk through what each engine does: Postgres throws `column "customer_id" must appear in the GROUP BY clause or be used in an aggregate function`. SQL Server throws a similar error. MySQL in the old default mode silently returned a random representative. 3. Explain why MySQL's behaviour was bad: the chosen value depended on storage order, indexes used and even the plan — so the same query could return different numbers on different days. 4. State that MySQL 5.7 changed the default to `ONLY_FULL_GROUP_BY`, aligning it with ANSI — covered in Q3.

Edge cases to mention: - Functional dependency: if `customer_id` is functionally determined by the grouping column (e.g. you group by `email` and `email` is unique per customer), Postgres recognises this and allows it. - `ANY_VALUE(customer_id)` is the standard escape hatch to say "I really do want an arbitrary value, don't yell at me" — supported in MySQL 5.7+ and BigQuery. - This rule applies to expressions too: `SELECT city, customer_id + 1 ...` still violates it.

What NOT to say: - Do not call the MySQL behaviour "a feature." It is a historical wart that MySQL itself fixed. - Do not say the engine "picks the first row" — there is no first row in a relational set.

Common follow-up: "How do you fix the query if you want the largest `customer_id` per city?" — Use `MAX(customer_id)`, or a window function with `ROW_NUMBER` and a filter.

```
-- Correct: aggregate or group
SELECT city, MAX(customer_id) AS top_customer_id, SUM(amount) AS gmv
FROM orders
GROUP BY city;
```

Q299 · ONLY_FULL_GROUP_BY Mode in MySQL 5.7+

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Cred · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

What is `ONLY_FULL_GROUP_BY` in MySQL and why was it added?

The 30-second answer: `ONLY_FULL_GROUP_BY` is an SQL mode flag that makes MySQL reject `GROUP BY` queries which violate the ANSI rule (every non-aggregated `SELECT` column must be in `GROUP BY` or be functionally dependent on it). It became the default in MySQL 5.7 to bring MySQL in line with Postgres, SQL Server, Oracle and the SQL standard.

Step-by-step thought process: 1. Define the mode: when ON, MySQL behaves like every other ANSI-compliant database. When OFF (legacy mode), it returns the random-representative-value behaviour that bit so many teams. 2. Explain why it matters in practice: when teams upgrade from MySQL 5.6 to 5.7 or 8.0, queries that "worked" suddenly throw errors. Senior engineers know to grep the codebase for grouped queries before the upgrade. 3. Mention how to inspect: `SELECT @@sql_mode;` shows current modes. To turn it off temporarily, you can `SET sql_mode = (SELECT REPLACE(@@sql_mode, 'ONLY_FULL_GROUP_BY', ''))` — though you should fix the queries instead. 4. Bring up `ANY_VALUE()` as the right escape hatch when you genuinely want an arbitrary representative.

Edge cases to mention: - MySQL's functional-dependency detection is weaker than Postgres — it covers primary keys but not all unique-constraint cases. - AWS RDS and Aurora MySQL ship with `ONLY_FULL_GROUP_BY` on by default; turning it off cluster-wide requires a parameter group change. - The mode applies per session, so libraries can set it differently from your interactive shell — leading to "works locally, breaks in CI" surprises.

What NOT to say: - Do not advise "just turn it off." That hides bugs and breaks portability. - Do not say it affects only `SELECT` — it also affects views and stored procedures that contain grouped queries.

Common follow-up: "Rewrite a query that fails `ONLY_FULL_GROUP_BY` without changing the data returned." — Wrap the offending column in `MAX()` or `ANY_VALUE()`, or add it to the `GROUP BY`.

```
-- Works under ONLY_FULL_GROUP_BY
SELECT city, ANY_VALUE(customer_id) AS sample_cust, SUM(amount) AS gmv
FROM orders
GROUP BY city;
```

Q300 · GROUP BY with HAVING vs WHERE

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wells Fargo, JP Morgan · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

What is the difference between WHERE and HAVING? Can you always replace one with the other?

The 30-second answer: WHERE filters rows before grouping; HAVING filters groups after the aggregate is computed. They are not interchangeable — you cannot reference an aggregate in WHERE, and you should not push row-level predicates into HAVING because that prevents index usage.

Step-by-step thought process: 1. State the order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. WHERE happens before GROUP BY, HAVING after. 2. Give a clean example: `WHERE status = 'paid'` filters orders before grouping (uses index on status); `HAVING SUM(amount) > 100000` filters groups whose total exceeds a lakh. 3. Show the common mistake: `HAVING status = 'paid'` works in some dialects but is semantically wrong — push it to WHERE for performance. 4. Mention that HAVING can reference both aggregate functions and the grouping columns themselves.

Edge cases to mention: - HAVING without GROUP BY treats the whole table as one group — `SELECT COUNT(*) FROM orders HAVING COUNT(*) > 0` is legal. - You cannot reference SELECT aliases in HAVING in SQL Server (must repeat the aggregate expression); Postgres and MySQL allow it. - Filtering on grouping columns in HAVING vs WHERE is logically equivalent but the planner usually pushes them to WHERE anyway — don't rely on this, write it correctly.

What NOT to say: - Do not say "HAVING is just WHERE for grouped queries." That is a common but wrong simplification. - Do not put `WHERE SUM(amount) > 100000` — it is illegal because the aggregate has not been computed yet.

Common follow-up: "Why does pushing predicates into WHERE matter for performance?" — WHERE can use indexes and reduce rows before sorting/hashing for GROUP BY; HAVING runs after, so the database may have already done a lot of unnecessary work.

```
SELECT city, SUM(amount) AS gmv
FROM orders
WHERE status = 'paid'
GROUP BY city
HAVING SUM(amount) > 100000;
```

Q301 · COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)

Tags: Difficulty: Easy · Asked at: Accenture, Cognizant, Walmart Labs, Goldman · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Tell me the difference between COUNT(*), COUNT(column) and COUNT(DISTINCT column). When would you reach for each?

The 30-second answer: COUNT(*) counts every row including NULLs. COUNT(col) counts non-NULL values of `col`. COUNT(DISTINCT col) counts the number of distinct non-NULL values. They give different numbers whenever NULLs or duplicates exist in `col`.

Step-by-step thought process: 1. Lead with NULL handling — that is the differentiator. `COUNT(*)` includes a row even if every column is NULL; `COUNT(col)` skips rows where `col` is NULL. 2. Give a tiny worked example: 10 orders, 3 with `coupon_code` NULL. `COUNT(*) = 10`, `COUNT(coupon_code) = 7`. If 4 of those 7 use the same coupon and 3 use distinct ones, `COUNT(DISTINCT coupon_code) = 4`. 3. State the common interview use-case: `SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END)` is functionally the same as `COUNT(*) FILTER (WHERE status='paid')` or `COUNT(CASE WHEN status='paid' THEN 1 END)`. 4. Mention performance: COUNT(DISTINCT) is the slowest because the engine must build a hash or sort to dedupe — covered in Q10.

Edge cases to mention: - COUNT(1) and COUNT(*) are identical in every modern engine — no, COUNT(1) is not faster; that is a myth from 1990s Oracle. - In Postgres, `COUNT(*) OVER ()` gives total row count alongside other columns without a separate query. - COUNT over an empty set returns 0 (not NULL) — unlike SUM/AVG/MIN/MAX which return NULL.

What NOT to say: - Do not claim COUNT(1) is faster than COUNT() — *modern planners treat them identically*. - Do not say COUNT(col) and COUNT() "always return the same" — only true if the column is NOT NULL.

Common follow-up: "In a left-joined query, why do COUNT() and COUNT(right_table.id) differ?" — Because LEFT JOIN creates NULL rows on the right when no match exists; COUNT() counts them, COUNT(right.id) does not.

```
SELECT
  COUNT(*)           AS total_orders,
  COUNT(coupon_code) AS orders_with_coupon,
  COUNT(DISTINCT coupon_code) AS unique_coupons_used
FROM orders;
```

Q302 · Performance of COUNT(*) on Large Tables — Fast Counts

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Target · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

SELECT COUNT(*) on a fifty-crore-row orders table takes forty seconds. How do you make it instant?

The 30-second answer: Exact COUNT(*) requires scanning the whole table or a covering index, which is unavoidable. For approximate counts, read from the system catalogs — `pg_class.reltuples` in Postgres, `INFORMATION_SCHEMA.TABLES.TABLE_ROWS` in MySQL — which return the planner's estimate in microseconds. For dashboards that need a near-real-time number, maintain a counter table updated by triggers or async aggregation jobs.

Step-by-step thought process: 1. Diagnose first: explain why COUNT(*) is slow — Postgres has MVCC, so each tuple visibility must be checked; MySQL InnoDB has no internal row count, so it must scan. 2. Offer the catalog trick: `SELECT reltuples::bigint FROM pg_class WHERE relname = 'orders';` returns an estimate that is usually accurate to within a percent if ANALYZE has run recently. 3. For accurate-enough counts in MySQL: `SELECT TABLE_ROWS FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME='orders';` — also an estimate. 4. Final layer: precomputed counters. A `counts` table with one row per entity, updated by triggers or batch jobs, gives O(1) reads at the cost of write overhead.

Edge cases to mention: - Postgres VACUUM or autovacuum keeps `reltuples` fresh — if stats are stale, the estimate drifts. - A partial index covering the predicate (`WHERE deleted_at IS NULL`) can make `SELECT COUNT(*)` faster because the index is smaller than the heap. - BigQuery, Snowflake and other columnar engines maintain exact row counts in metadata and COUNT(*) is already instant.

What NOT to say: - Do not say "add an index on a column" — COUNT(*) does not use a column-level index unless it is a covering one. - Do not propose `SELECT COUNT(1)` — same speed.

Common follow-up: "How accurate is `pg_class.reltuples` ?" — Within a few percent for tables that get autovacuumed regularly; can drift further on insert-heavy tables until the next ANALYZE.

```
-- Postgres fast estimate
SELECT reltuples::bigint AS estimated_rows
FROM pg_class
WHERE relname = 'orders';
```

Q303 · SUM / AVG / MIN / MAX — NULL Handling

Tags: Difficulty: Easy · Asked at: TCS, Wipro, Infosys, Cognizant · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

What happens when SUM, AVG, MIN and MAX encounter NULLs in the column? What if every row is NULL?

The 30-second answer: All four aggregates skip NULLs silently. AVG divides by the count of non-NULL values, not the total row count. If every input value is NULL — or if the grouped set is empty for that group — they return NULL, not zero (except COUNT, which returns 0).

Step-by-step thought process: 1. State the rule: aggregates ignore NULLs. This is consistent across every dialect. 2. Walk through AVG carefully: `AVG(refund_amount)` over 10 rows where 6 are NULL averages just the 4 non-NULL values. If you wanted to treat NULL as zero, use `AVG(COALESCE(refund_amount, 0))`. 3. Mention MIN and MAX: order-sensitive operations skip NULLs, so `MIN(birth_date)` returns the earliest non-NULL date. 4. Final point: when every value is NULL, the result is NULL — wrap in `COALESCE(SUM(x), 0)` for a numeric default.

Edge cases to mention: - SUM over an empty set returns NULL, not 0 — surprising the first time you hit it in a report. - `AVG(salary)` and `SUM(salary)/COUNT(*)` give different answers when some salaries are NULL. - Boolean aggregation: in Postgres, `bool_and` and `bool_or` ignore NULLs; in MySQL you use `MIN(flag::int)` and `MAX(flag::int)` style tricks.

What NOT to say: - Do not say "NULLs are treated as zero." They are not — they are skipped entirely. - Do not say AVG always returns a decimal — in some engines it returns the integer division if the input is integer (SQL Server is one).

Common follow-up: "Why is `SUM(x)/COUNT(*)` sometimes different from `AVG(x)`?" — Because COUNT(*) counts all rows including those where x is NULL, while AVG divides by the count of non-NULL x values.

```
SELECT
  customer_id,
  SUM(amount)           AS total_paid,
  AVG(amount)           AS avg_order_value,
  COALESCE(SUM(refund_amount), 0) AS total_refunded
FROM orders
GROUP BY customer_id;
```

Q304 · Conditional Aggregation — SUM(CASE WHEN ...)

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Wells Fargo, AmEx · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Without using FILTER or multiple subqueries, how do you compute the count of paid orders and the count of refunded orders in a single grouped query?

The 30-second answer: Use conditional aggregation — wrap a CASE expression inside SUM or COUNT. `SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END)` gives the paid count; `SUM(CASE WHEN status='refunded' THEN 1 ELSE 0 END)` the refunded count. Both run in a single pass over the data.

Step-by-step thought process: 1. Frame the problem: you want multiple counts/sums conditioned on different predicates in one query. The naive approach is multiple subqueries; the elegant approach is conditional aggregation. 2. Show the pattern: `SUM(CASE WHEN <predicate> THEN <value> ELSE 0 END)` works in every dialect. 3. Note the COUNT trick: `COUNT(CASE WHEN <predicate> THEN 1 END)` — the ELSE branch defaults to NULL, which COUNT skips. This is shorter than the SUM-CASE form. 4. State the performance win: one scan over the table instead of N scans for N subqueries — verified by EXPLAIN showing a single Seq Scan with multiple Aggregate nodes feeding off it.

Edge cases to mention: - For averages with conditions, use `AVG(CASE WHEN ... THEN x END)` — the NULL ELSE keeps the denominator honest. - The FILTER clause (Q9) is a cleaner, ANSI-standard alternative in Postgres and SQLite. - Be careful with `ELSE 0` vs `ELSE NULL` — for COUNT they differ, for SUM they don't.

What NOT to say: - Do not write five separate subqueries and JOIN them — interviewer is testing exactly this anti-pattern. - Do not say "use IF()" — it works in MySQL but is non-portable.

Common follow-up: "What if I want the percentage of paid orders per city?" — `100.0 * SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END) / COUNT(*)` — the `100.0` forces float division.

```
SELECT
  city,
  COUNT(*)                AS total_orders,
  SUM(CASE WHEN status='paid' THEN 1 ELSE 0 END) AS paid,
  SUM(CASE WHEN status='refunded' THEN 1 ELSE 0 END) AS refunded,
  SUM(CASE WHEN status='paid' THEN amount ELSE 0 END) AS paid_gmv
FROM orders
GROUP BY city;
```

Q305 · FILTER Clause (Postgres / SQL Standard) — Cleaner Conditional Aggregation

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, Goldman · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Show me the same paid-vs-refunded breakdown using the FILTER clause. Which databases support it?

The 30-second answer: The SQL:2003 FILTER clause attaches a WHERE-style predicate to an aggregate: `COUNT(*) FILTER (WHERE status='paid')`. It is supported in Postgres, SQLite 3.30+ and HSQLDB. SQL Server, MySQL and Oracle do not yet implement it, so you fall back to `SUM(CASE WHEN ...)`.

Step-by-step thought process: 1. Introduce FILTER as the ANSI-standard alternative to SUM-CASE. It reads as natural English: "count all orders filtered by status equals paid." 2. Write the example:

`COUNT(*) FILTER (WHERE status='paid') AS paid` — half the characters of the SUM-CASE equivalent. 3. Note dialect support: Postgres since 9.4, SQLite 3.30, plus standard mandates it. SQL Server / MySQL / Oracle still require the CASE workaround. 4. Mention that FILTER works with any aggregate — SUM, AVG, ARRAY_AGG, PERCENTILE_CONT — not just COUNT.

Edge cases to mention: - FILTER and window functions compose: `SUM(amount) FILTER (WHERE status='paid') OVER (PARTITION BY city)` is valid in Postgres. - The FILTER predicate is evaluated per row before aggregation — same semantics as CASE inside the aggregate. - For ORDER BY within the aggregate (e.g. ARRAY_AGG), FILTER and ORDER BY can coexist: `ARRAY_AGG(id ORDER BY id) FILTER (WHERE status='paid')`.

What NOT to say: - Do not claim it is faster than SUM-CASE — Postgres compiles both to the same plan in most cases. - Do not assume MySQL supports it — junior candidates often confuse Postgres extensions with universal SQL.

Common follow-up: "Rewrite the FILTER query for SQL Server." — Replace each `FILTER (WHERE p)` with `CASE WHEN p THEN x END` inside the aggregate.

```
-- Postgres / SQLite / standard SQL
SELECT
  city,
  COUNT(*)                               AS total,
  COUNT(*) FILTER (WHERE status='paid')   AS paid,
  COUNT(*) FILTER (WHERE status='refunded') AS refunded,
  SUM(amount) FILTER (WHERE status='paid') AS paid_gmv
FROM orders
GROUP BY city;
```

Q306 · COUNT(DISTINCT) — Common Performance Trap

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Swiggy, Flipkart · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Why is `COUNT(DISTINCT user_id)` slow on a billion-row events table, and what would you do to speed it up?

The 30-second answer: `COUNT(DISTINCT)` must dedupe every value, which forces either a hash table sized to the cardinality or a sort over the entire grouped subset — both of which spill to disk on large data. Speed it up by using approximate count distinct (HyperLogLog, BigQuery's `APPROX_COUNT_DISTINCT`, Snowflake's `APPROX_COUNT_DISTINCT`, Postgres's `hll` extension) when an exact number is not required.

Step-by-step thought process: 1. Explain the mechanics: deduplication needs memory proportional to the cardinality of the distinct values. With a billion unique users you cannot fit them in RAM. 2. Show the trap: `COUNT(DISTINCT)` in a query with multiple grouping columns multiplies the work — the planner must dedupe per group. 3. Offer the approximate alternative: HyperLogLog gives 99% accuracy with constant memory (~12 KB per sketch). BigQuery exposes it as `APPROX_COUNT_DISTINCT`; Snowflake the same; Redshift has `APPROX_COUNT_DISTINCT`; Postgres needs the `hll` extension. 4. Mention pre-aggregated daily tables as the production solution: materialise distinct user counts per day, then sum or merge sketches.

Edge cases to mention: - Multiple `COUNT(DISTINCT)` in one query is even worse — each needs its own dedup pipeline. Postgres handles them by sorting; some engines parallelise. - For exact counts, a covering index on the distinct column lets the engine do an Index Only Skip Scan (Postgres 13+, MySQL 8 with loose index scan). - HyperLogLog sketches are mergeable — you can store one sketch per day and combine them later for weekly/monthly distincts without re-scanning raw events.

What NOT to say: - Do not say "add an index" without specifying it must cover the distinct column. - Do not promise exact results from `APPROX_COUNT_DISTINCT` — it is approximate by design.

Common follow-up: "How accurate is HyperLogLog?" — Standard error around $1.04 / \sqrt{m}$ where m is the number of registers; with the default 2^{14} registers you get ~0.81% error.

```
-- BigQuery / Snowflake / Redshift
SELECT
  DATE(event_ts) AS day,
  APPROX_COUNT_DISTINCT(user_id) AS daily_users
FROM events
GROUP BY day;
```

Q307 · Approximate Count Distinct — HyperLogLog Mechanics

Tags: Difficulty: Hard · Asked at: Walmart Labs, Swiggy, PhonePe, Goldman · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Explain at a high level how HyperLogLog estimates cardinality without storing the actual values.

The 30-second answer: HyperLogLog hashes each input value, splits the hash into a bucket index and a remainder, and tracks the maximum number of leading zeros seen in the remainder per bucket. The cardinality estimate is derived from the harmonic mean of those max-leading-zero counts scaled by the number of buckets. Memory is constant (kilobytes) regardless of input size, and accuracy is around 1%.

Step-by-step thought process: 1. Set up the intuition: if you hash values uniformly, observing a hash with k leading zeros is roughly a $1\text{-in-}2^k$ event, suggesting you have seen on the order of 2^k distinct items. 2. Walk through the algorithm: take a uniform hash, use the first b bits to pick a bucket ($m = 2^b$ buckets), use the remaining bits to count leading zeros, and store only the max per bucket. 3. Cardinality estimate: $n \approx \alpha_m \cdot m^2 / (\text{sum over buckets of } 2^{(-\text{max_zeros})})$ — you do not need the formula in the interview, just the harmonic-mean intuition. 4. Mention sketch mergeability — two HLLs covering disjoint sets can be merged by taking bucket-wise max, giving the HLL of the union. This is why APPROX_COUNT_DISTINCT scales horizontally.

Edge cases to mention: - Small cardinalities have higher error — HyperLogLog++ from Google switches to LinearCounting below a threshold. - Hash quality matters — a bad hash function degrades accuracy. Production engines use MurmurHash3 or xxHash. - BigQuery exposes the sketches as bytes (`HLL_COUNT.INIT`, `HLL_COUNT.MERGE`, `HLL_COUNT.EXTRACT`), so you can pre-compute and merge later.

What NOT to say: - Do not say "it samples the data" — HyperLogLog observes every value, it just stores almost nothing. - Do not claim it works for joins or sums — it is strictly a cardinality estimator.

Common follow-up: "How would you estimate distinct users over a sliding 7-day window cheaply?" — Keep one HLL sketch per day, merge the last 7 sketches at query time, extract the cardinality. $O(7)$ merges.

```
-- BigQuery: precompute daily sketches, merge for weekly distinct users
WITH daily AS (
  SELECT DATE(event_ts) AS day, HLL_COUNT.INIT(user_id) AS sketch
  FROM events GROUP BY day
)
SELECT HLL_COUNT.MERGE(sketch) AS weekly_users
FROM daily
WHERE day BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 6 DAY) AND CURRENT_DATE();
```

Q308 · ARRAY_AGG / STRING_AGG / GROUP_CONCAT — Dialect Comparison

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Wells Fargo, AmEx · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

I want to collapse all product names a customer has ever ordered into a single comma-separated string per customer. Show me how in Postgres, MySQL and SQL Server.

The 30-second answer: Use the dialect's collection aggregate — `STRING_AGG(product_name, ',')` in Postgres and SQL Server 2017+, `GROUP_CONCAT(product_name SEPARATOR ',')` in MySQL, and `LISTAGG(product_name, ',')` in Oracle. `ARRAY_AGG` returns an array instead of a string in Postgres and BigQuery if you want to keep typed elements.

Step-by-step thought process: 1. Compare the three function names side by side — interviewers love this matrix. 2. Show the Postgres / SQL Server form: `STRING_AGG(product_name, ',') WITHIN GROUP (ORDER BY product_name)` — note the WITHIN GROUP form for ordered aggregation. 3.

Show the MySQL form:

`GROUP_CONCAT(product_name ORDER BY product_name SEPARATOR ',')`. Default separator is comma. 4. Mention size limits — MySQL's `group_concat_max_len` defaults to 1024 bytes and truncates silently. Production teams raise it to several MB.

Edge cases to mention: - NULL values are skipped by all of these — wrap in `COALESCE` if you want a placeholder. - `ARRAY_AGG` in Postgres returns NULL when the group is empty (from a left join with no matches); use `COALESCE(ARRAY_AGG(x), '{}')`. - `LISTAGG` in Oracle 12.2+ supports `ON OVERFLOW TRUNCATE` to avoid the dreaded ORA-01489 when the concatenated string exceeds 4000 bytes.

What NOT to say: - Do not call them "the same function in different databases" — they have subtle ordering and NULL differences. - Do not forget the separator in `GROUP_CONCAT` — the default is comma in MySQL but engines vary.

Common follow-up: "What if I need the result deduplicated?" — `GROUP_CONCAT(DISTINCT product_name)` in MySQL; `STRING_AGG(DISTINCT product_name, ',')` in Postgres; `LISTAGG(DISTINCT ...)` in Oracle 19c+.

```
-- Postgres
SELECT customer_id,
       STRING_AGG(product_name, ', ' ORDER BY product_name) AS products
FROM order_items
GROUP BY customer_id;
```


Q309 · ARRAY_AGG ORDER BY — Controlling Element Order

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Swiggy · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

When you collapse values into an array with ARRAY_AGG, the order looks random. How do you guarantee a specific order?

The 30-second answer: Use the ORDER BY clause inside the aggregate:

`ARRAY_AGG(product_name ORDER BY ordered_at)`. Without that clause, the order is whatever the planner decides — usually but not always heap order, and never something you should rely on.

Step-by-step thought process: 1. Establish the rule: aggregates can take an ORDER BY inside the parentheses, separate from the query's outer ORDER BY. This is ANSI standard. 2. Write the example: `ARRAY_AGG(product_name ORDER BY ordered_at DESC) AS recent_first` — gives the customer's products in reverse chronological order. 3. Mention that this matters for downstream consumers who treat the first element as "most recent" or "primary." 4. Bring up `FIRST_VALUE` / `LAST_VALUE` window functions as the alternative when you want just one element, not the whole sequence.

Edge cases to mention: - The ORDER BY can reference columns not in the GROUP BY — they apply per group during aggregation. - MySQL's GROUP_CONCAT, Postgres's STRING_AGG and Oracle's LISTAGG all support the same internal ORDER BY. - Combined with FILTER: `ARRAY_AGG(id ORDER BY id) FILTER (WHERE status='paid')` — keeps both ordering and predicate.

What NOT to say: - Do not put ORDER BY at the end of the SELECT and expect it to sort within each group — it sorts the outer result. - Do not say "the order is always insertion order" — it depends on the plan.

Common follow-up: "How do you get the top three recent orders per customer as an array?" — `ARRAY_AGG(order_id ORDER BY ordered_at DESC) [1:3]` in Postgres, or a subquery with ROW_NUMBER and a filter.

```
SELECT customer_id,  
       ARRAY_AGG(product_name ORDER BY ordered_at DESC) AS purchase_history  
FROM   order_items  
GROUP BY customer_id;
```


Q310 · LISTAGG (Oracle, SQL:2016)

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

You are working in Oracle. How do you collapse the city names for each state into a comma-separated string?

The 30-second answer: Use `LISTAGG(city, ', ' ) WITHIN GROUP (ORDER BY city)` grouped by state. LISTAGG was added in Oracle 11g Release 2 and standardised in SQL:2016. It returns VARCHAR2 up to 4000 bytes by default — use `ON OVERFLOW TRUNCATE` from 12.2 onwards to avoid the overflow error.

Step-by-step thought process: 1. State the syntax: `LISTAGG(expr, delimiter) WITHIN GROUP (ORDER BY ...)` — the WITHIN GROUP is mandatory because LISTAGG is an ordered-set aggregate. 2. Walk through the example: `SELECT state, LISTAGG(city, ', ' ) WITHIN GROUP (ORDER BY city) FROM cities GROUP BY state` . 3. Mention the 4000-byte VARCHAR2 limit in older Oracle versions and the `ON OVERFLOW TRUNCATE WITH COUNT` extension that returns a truncated string with a count of omitted elements. 4. Compare quickly: LISTAGG is to Oracle what STRING_AGG is to Postgres / SQL Server and GROUP_CONCAT is to MySQL — same job, different name and minor syntax.

Edge cases to mention: - DISTINCT support was added only in Oracle 19c — `LISTAGG(DISTINCT city, ', ' )` . - Without ORDER BY in WITHIN GROUP, Oracle treats it as unordered and the result is non-deterministic. - NULLs are skipped silently — wrap in NVL if you need a placeholder.

What NOT to say: - Do not omit the WITHIN GROUP clause — Oracle raises a syntax error. - Do not assume CLOB output — it is VARCHAR2 by default; use `LISTAGG(...) WITHIN GROUP (...) ON OVERFLOW TRUNCATE` for long lists.

Common follow-up: "How do you do this in Oracle 11g Release 1 (no LISTAGG)?" — Use `WM_CONCAT` (undocumented) or hierarchical query with `CONNECT_BY_ROOT` and `SYS_CONNECT_BY_PATH` . Modern code avoids both.

```
-- Oracle 12.2+
SELECT state,
       LISTAGG(city, ', ' ON OVERFLOW TRUNCATE WITH COUNT)
       WITHIN GROUP (ORDER BY city) AS cities
FROM cities
GROUP BY state;
```

Q311 · GROUPING SETS — Multiple Groupings in One Query

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Goldman, AmEx · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

I want total sales by city, by state and overall — all in a single query. How?

The 30-second answer: Use GROUPING SETS — a single query block that lets you specify multiple grouping combinations. `GROUP BY GROUPING SETS ((city), (state), ())` returns three logical result sets stacked together: per city, per state and the grand total (empty grouping).

Step-by-step thought process: 1. Frame the problem: without GROUPING SETS you would UNION ALL three separate GROUP BY queries — same data scanned three times, ugly to maintain. 2. Show the syntax: `GROUP BY GROUPING SETS ((city), (state), ())`. The empty parentheses denote the grand-total row. 3. Explain the result shape: every row has both `city` and `state` columns, but for state-level rows `city` is NULL and vice versa; for the grand-total row both are NULL. 4. Bring in the GROUPING() function (Q18) as the tool to tell rollup NULLs apart from data NULLs.

Edge cases to mention: - Supported in Postgres 9.5+, SQL Server, Oracle, BigQuery, Snowflake. MySQL 8.0 only added ROLLUP; GROUPING SETS proper is not yet in MySQL. - The set of grouping columns must still cover everything in SELECT that is not aggregated. - Performance: a smart planner scans the table once and computes all grouping levels in one pass — much faster than UNION ALL of three queries.

What NOT to say: - Do not write three subqueries and UNION ALL them — that's the anti-pattern the interviewer is looking for you to replace. - Do not say `()` means "no GROUP BY" — it means an explicit empty grouping, which produces one grand-total row.

Common follow-up: "Can I combine GROUPING SETS with a regular GROUP BY column?" — Yes: `GROUP BY country, GROUPING SETS ((city), (state), ())` produces those three groupings per country.

```
SELECT country, state, city,
       SUM(amount) AS gmv
FROM orders
GROUP BY GROUPING SETS ((country, state, city), (country, state), (country), ());
```

Q312 · ROLLUP — Hierarchical Totals

Tags: Difficulty: Medium · Asked at: Wells Fargo, Goldman, Walmart Labs, Target · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

I want sales by country, then sub-totalled per country, then a grand total — all in one query.
Which feature do I use?

The 30-second answer: ROLLUP. `GROUP BY ROLLUP(country, state, city)` produces sub-totals at each level of the hierarchy plus a grand total at the end — equivalent to GROUPING SETS of `(country, state, city)`, `(country, state)`, `(country)` and `()`.

Step-by-step thought process: 1. Define ROLLUP as a shorthand for hierarchical GROUPING SETS — it generates `n+1` grouping levels for `n` columns, removing one column from the right each time. 2. Walk through the rows: full detail rows have all three columns populated; the state-aggregate row has city as NULL; the country-aggregate row has both state and city NULL; the final grand-total row has all three NULL. 3. Mention MySQL's `WITH ROLLUP` syntax — `GROUP BY country, state, city WITH ROLLUP` — slightly different from the ANSI form but does the same job. 4. Tie back to GROUPING() to distinguish rollup NULLs from real NULLs in the data.

Edge cases to mention: - Order of columns in ROLLUP matters — `ROLLUP(country, state)` produces country sub-totals, while `ROLLUP(state, country)` would produce state sub-totals. - MySQL `WITH ROLLUP` does not allow ORDER BY in the same query block; you must wrap the result in a subquery. - BigQuery, Snowflake, Postgres and SQL Server all support the standard ROLLUP syntax.

What NOT to say: - Do not confuse ROLLUP with CUBE — ROLLUP is hierarchical ($n+1$ groupings), CUBE is combinatorial (2^n groupings). - Do not put ROLLUP inside the SELECT list — it belongs in the GROUP BY clause only.

Common follow-up: "How does ROLLUP differ from CUBE?" — ROLLUP gives only the hierarchical prefixes; CUBE gives every possible subset of grouping columns. For 3 columns, ROLLUP makes 4 levels and CUBE makes 8.

```
SELECT country, state, city, SUM(amount) AS gmv
FROM orders
GROUP BY ROLLUP(country, state, city)
ORDER BY country NULLS LAST, state NULLS LAST, city NULLS LAST;
```

Q313 · CUBE — All Combinations

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, AmEx · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

When would you use CUBE instead of ROLLUP?

The 30-second answer: Use CUBE when you need totals across every possible combination of grouping columns, not just the hierarchical ones. `CUBE(a, b, c)` produces $2^3 = 8$ grouping sets — `(a,b,c)`, `(a,b)`, `(a,c)`, `(b,c)`, `(a)`, `(b)`, `(c)` and `()`.

Step-by-step thought process: 1. Contrast with ROLLUP: ROLLUP gives prefixes only (hierarchical), CUBE gives the full power set. 2. Sketch the use case: a finance dashboard that needs gross margin sliced by every combination of product category, region and channel — CUBE gives the full cross-tab in one query. 3. Mention complexity: CUBE on 5 columns is 32 grouping sets — be deliberate, do not use it casually because the row count explodes. 4. Bring in the `GROUPING()` function (Q18) to label each output row with which dimensions are rollup totals vs raw values.

Edge cases to mention: - Supported in Postgres 9.5+, SQL Server, Oracle, BigQuery, Snowflake. Not yet in MySQL (only WITH ROLLUP). - Combining with regular GROUP BY: `GROUP BY year, CUBE(category, region)` cubes within each year. - Memory: a smart planner computes all 2^n groupings in a single pass using a multi-level hash aggregate — but on huge data it can spill.

What NOT to say: - Do not use CUBE as a "give me all combinations just in case" — pick the specific groupings you need with `GROUPING SETS`. - Do not say "CUBE is faster than ROLLUP" — it does strictly more work.

Common follow-up: "What's the output row count for CUBE on 3 columns over a 1000-row table?" — Up to 1000 detail rows plus aggregate rows for each of the 8 grouping sets — depends on how many distinct values each column has.

```
SELECT category, region, channel, SUM(amount) AS gmv
FROM sales
GROUP BY CUBE(category, region, channel);
```

Q314 · GROUPING() Function — Distinguishing NULLs

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, AmEx, Wells Fargo · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

In a ROLLUP query, how do you tell a NULL that means "grand total" apart from a NULL that means "no data"?

The 30-second answer: Use the GROUPING() function: `GROUPING(city)` returns 1 if the row's city column is a rollup-generated NULL (i.e. city is not part of the current grouping set), and 0 otherwise. You wrap it in a CASE to display 'Grand Total' instead of NULL, or to filter to a specific grouping level.

Step-by-step thought process: 1. State the problem clearly: ROLLUP and CUBE introduce NULLs in the grouping columns for sub-total and grand-total rows. Without GROUPING() you cannot distinguish those from data NULLs. 2. Show the basic use: `CASE WHEN GROUPING(city) = 1 THEN 'All Cities' ELSE city END AS city_label`. 3. Mention GROUPING_ID — a bitmask combining the GROUPING() output for multiple columns, useful for filtering to a specific grouping set: `WHERE GROUPING_ID(country, state, city) = 0` keeps only the most detailed level. 4. Tie back to reports: GROUPING() is what makes pivot-style summary reports clean.

Edge cases to mention: - GROUPING() works in Postgres, SQL Server, Oracle, BigQuery, Snowflake. MySQL has GROUPING() since 8.0 for use with WITH ROLLUP. - The function takes the column as argument — `GROUPING(city)`, not `GROUPING('city')` — and the column must appear in the GROUP BY clause. - Combined with HAVING: `HAVING GROUPING(city) = 0` filters out summary rows entirely.

What NOT to say: - Do not use `IS NULL` to detect rollup NULLs — that hits both data NULLs and rollup NULLs. - Do not call it GROUPING() without a column argument — it is not a count.

Common follow-up: "How do you keep only the grand-total row?" — `HAVING GROUPING(country) = 1 AND GROUPING(state) = 1 AND GROUPING(city) = 1` or `HAVING GROUPING_ID(country, state, city) = 7`.

```
SELECT
  CASE WHEN GROUPING(country) = 1 THEN 'All Countries' ELSE country END AS country,
  CASE WHEN GROUPING(state) = 1 THEN 'All States' ELSE state END AS state,
  SUM(amount) AS gmv
FROM orders
GROUP BY ROLLUP(country, state);
```

Q315 · Cumulative Aggregation — Window vs Subquery

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, PhonePe · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

I want a running total of daily sales — each row showing the cumulative sum up to that day. Show me the window-function way and the subquery way.

The 30-second answer: Use a window function: `SUM(amount) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` . The subquery alternative — `SELECT day, (SELECT SUM(x.amount) FROM daily x WHERE x.day <= d.day) FROM daily d` — works but is $O(n^2)$ and dramatically slower on anything beyond a few thousand rows.

Step-by-step thought process: 1. Introduce window functions as the canonical answer — supported in every modern database (Postgres 8.4+, MySQL 8, SQL Server 2012+, Oracle, BigQuery, Snowflake). 2. Write the window: `SUM(amount) OVER (ORDER BY day)` — default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` , which gives the running total. 3. Compare performance: window is a single pass over the sorted rows; correlated subquery rescans the relation for each row — quadratic. 4. Mention partitioning: `OVER (PARTITION BY customer_id ORDER BY day)` gives running totals per customer in the same query.

Edge cases to mention: - Default frame for ordered window aggregates is RANGE not ROWS — this matters when multiple rows tie on `day` ; ROWS gives a strict cumulative count, RANGE bunches ties. - MySQL 8 added window functions; older MySQL versions need the subquery or session-variable hack. - BigQuery and Snowflake support window aggregates with the same standard syntax.

What NOT to say: - Do not use a self-join with `LEFT JOIN ... ON x.day <= d.day` — that is the same quadratic problem in disguise. - Do not claim the correlated subquery is "more readable" — it is slower and harder to extend.

Common follow-up: "How do you get a moving 7-day window instead of cumulative?" — `SUM(amount) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` .

```
SELECT day,
       SUM(amount) OVER (ORDER BY day
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_gmv
FROM daily_sales
ORDER BY day;
```

Q316 · Pivoting via CASE

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Accenture, Wells Fargo · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

I have a long-format table with `(customer_id, month, amount)` and I want a wide report with one row per customer and one column per month for the last six months. Write the SQL.

The 30-second answer: Use conditional aggregation with one CASE expression per output column:

`SUM(CASE WHEN month = '2026-01' THEN amount END) AS jan_2026, SUM(CASE WHEN month = '2026-02' THEN amount END) AS feb_2026, ...`. Group by customer_id. This is the portable pivot pattern that works in every database.

Step-by-step thought process: 1. Define pivoting: turning row labels into column headers. Useful for reports the business wants in wide format. 2. Write the pattern: one `SUM(CASE WHEN month='YYYY-MM' THEN amount END)` per desired output column. Use SUM not just plain CASE to collapse multiple rows per (customer, month). 3. Explain why you should not use ELSE 0 here — leaving it as NULL means SUM ignores it; ELSE 0 distorts AVG if you ever change the aggregate. 4. Bring in PIVOT (Q21) as the SQL-Server / Oracle specific syntax that is cleaner but non-portable.

Edge cases to mention: - The output columns are fixed at query write time — you cannot pivot on a dynamic set of months without procedural code or dynamic SQL. - For percentage breakdowns (`CASE WHEN ... THEN 1.0 ELSE 0 END`), AVG gives the share directly. - Postgres has the `crosstab` function in the `tablefunc` extension as an alternative.

What NOT to say: - Do not write twelve subqueries and join them — that's the anti-pattern. - Do not say "use a pivot table" without showing SQL — interviewers want the query.

Common follow-up: "What if I want only the months that have any data?" — That's dynamic pivoting; you need to generate the SQL string at runtime in the app layer, or use a server-side procedure.

```
SELECT customer_id,
       SUM(CASE WHEN month = '2025-12' THEN amount END) AS dec_2025,
       SUM(CASE WHEN month = '2026-01' THEN amount END) AS jan_2026,
       SUM(CASE WHEN month = '2026-02' THEN amount END) AS feb_2026,
       SUM(CASE WHEN month = '2026-03' THEN amount END) AS mar_2026,
       SUM(CASE WHEN month = '2026-04' THEN amount END) AS apr_2026,
       SUM(CASE WHEN month = '2026-05' THEN amount END) AS may_2026
FROM monthly_sales
GROUP BY customer_id;
```

Q317 · Pivoting via PIVOT (SQL Server / Oracle)

Tags: Difficulty: Medium · Asked at: Wells Fargo, JP Morgan, Goldman, TCS · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Do you know the native PIVOT operator? Write the same monthly report using PIVOT in SQL Server.

The 30-second answer: SQL Server and Oracle have a native `PIVOT (aggregate FOR pivot_column IN (value1, value2, ...))` clause. It is syntactic sugar over the CASE pattern — readable when the column list is short, awkward when it is long, and not supported in Postgres or MySQL.

Step-by-step thought process: 1. Show the SQL Server syntax: `SELECT * FROM (subquery with the three columns) AS src PIVOT (SUM(amount) FOR month IN ([2026-01], [2026-02], ...)) AS p`. 2. Highlight what the PIVOT clause does: chooses an aggregate, a column whose values become headers, and a fixed list of those values. Everything else in the source becomes a grouping key implicitly. 3. Mention Oracle's slightly different syntax — `PIVOT (SUM(amount) FOR month IN ('2026-01' AS jan, '2026-02' AS feb, ...))` — and the column aliasing. 4. State the limitation: the IN list must be a constant set. For dynamic columns you need dynamic SQL.

Edge cases to mention: - Implicit grouping is by every column not in the PIVOT clause and not in the FOR list — pre-project your source to avoid surprises. - Postgres has no PIVOT operator; use `tablefunc.crosstab()` or the manual CASE pattern. - BigQuery added PIVOT in 2021 — same idea, slightly different syntax for the IN list.

What NOT to say: - Do not assume PIVOT is portable — it is not. Always check the dialect. - Do not skip the source subquery; PIVOT requires a clean projection, otherwise the implicit grouping picks up unwanted columns.

Common follow-up: *"How would you do this with dynamic months?"* — Build the SQL string in your application or use SQL Server's dynamic SQL via `sp_executesql`.

```
-- SQL Server
SELECT *
FROM (
    SELECT customer_id, [month], amount FROM monthly_sales
) src
PIVOT (
    SUM(amount) FOR [month] IN ([2025-12], [2026-01], [2026-02], [2026-03], [2026-04], [2026-05])
) AS pvt;
```

Q318 · Unpivot Pattern

Tags: Difficulty: Medium · Asked at: Cognizant, Wipro, Walmart Labs, AmEx · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

The opposite problem now — I have a wide table with one column per month, and I want to convert it back to long format. How?

The 30-second answer: Use the UNPIVOT operator in SQL Server / Oracle, the `unnest` plus VALUES trick in Postgres, or a UNION ALL of selects (one per column) in any dialect. Each option turns one wide row into N long rows where N is the number of unpivoted columns.

Step-by-step thought process: 1. Start with the universal solution: UNION ALL one SELECT per column. Verbose but works everywhere. 2. Show SQL Server's UNPIVOT:

`... UNPIVOT (amount FOR month IN (jan, feb, mar)) AS u` — clean and readable. 3. Postgres uses lateral with VALUES: `SELECT customer_id, m.month, m.amount FROM monthly_wide w, LATERAL (VALUES ('jan', w.jan), ('feb', w.feb), ('mar', w.mar)) AS m(month, amount)`. 4. Mention BigQuery's UNPIVOT (added 2021) and Snowflake's UNPIVOT operator.

Edge cases to mention: - UNPIVOT in SQL Server skips NULL values silently by default; if you want to keep them you have to convert to non-NULL placeholders first. - The UNION ALL approach is portable but does N scans of the table — fine for small tables, bad for large ones. - Postgres's `jsonb_each` is a neat alternative when the wide row is already JSON: it unpivots key-value pairs natively.

What NOT to say: - Do not say "unpivot is just pivot in reverse" — they are operators on the same data shape, but the semantics around NULL handling and column lists differ. - Do not write CROSS APPLY when LATERAL is the modern term in Postgres.

Common follow-up: *"Why would you store data in wide form if it's painful to query?"* — Sometimes you don't choose — legacy systems, CSV imports, spreadsheet exports. Knowing how to unpivot is what gets you back to a queryable shape.

```
-- Portable union approach
SELECT customer_id, '2026-01' AS month, jan_2026 AS amount FROM monthly_wide
UNION ALL
SELECT customer_id, '2026-02', feb_2026 FROM monthly_wide
UNION ALL
SELECT customer_id, '2026-03', mar_2026 FROM monthly_wide;
```

Q319 · Conditional COUNT — Buckets in One Query

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Flipkart · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

I want the count of orders in three buckets — below ₹500, ₹500 to ₹2000, and above ₹2000 — in a single query.

The 30-second answer: Conditional COUNT with CASE expressions: `COUNT(CASE WHEN amount < 500 THEN 1 END), COUNT(CASE WHEN amount BETWEEN 500 AND 2000 THEN 1 END),`

`COUNT(CASE WHEN amount > 2000 THEN 1 END)`. The ELSE branch is implicit NULL, which COUNT skips. One pass over the data.

Step-by-step thought process: 1. Restate the requirement: bucket the rows and count each bucket — a histogram with three bins. 2. Choose the pattern: `COUNT(CASE WHEN ... THEN 1 END)` is cleaner than `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` because there is no need for ELSE 0 — COUNT ignores NULL. 3. Suggest extending to dollars: replace `THEN 1` with `THEN amount` and `COUNT` with `SUM` to get total GMV per bucket. 4. Mention `width_bucket` (Q28) for many-bin histograms where writing each CASE manually becomes painful.

Edge cases to mention: - Boundary conditions: BETWEEN is inclusive on both ends; if you have rows exactly at the boundary they go in one bin only. - Add a sanity check: the three bucket counts should sum to COUNT() — *if not, you missed a range*. - For percentage shares, divide each conditional count by COUNT() over the same query (no need for a subquery).

What NOT to say: - Do not write three separate queries and union them — interviewer is testing whether you know conditional aggregation. - Do not forget the boundary rounding — `< 500` vs `<= 500` changes the answer.

Common follow-up: "What if buckets are dynamic — defined in another table?" — Cross join your data with the bucket table, JOIN ON `amount BETWEEN bucket_low AND bucket_high`, then GROUP BY bucket.

```
SELECT
  COUNT(CASE WHEN amount < 500           THEN 1 END) AS low,
  COUNT(CASE WHEN amount BETWEEN 500 AND 2000 THEN 1 END) AS mid,
  COUNT(CASE WHEN amount > 2000           THEN 1 END) AS high,
  COUNT(*)                                AS total
FROM orders;
```

Q320 · Median Calculation — PERCENTILE_CONT vs Workarounds

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan, Walmart Labs, Cred · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Compute the median order amount. Use the modern approach, then show me how you'd do it without PERCENTILE_CONT.

The 30-second answer: Modern: `SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) FROM orders` — ANSI SQL ordered-set aggregate, supported as a standalone aggregate in Postgres, Oracle, BigQuery, Snowflake. SQL Server implements PERCENTILE_CONT only as a window

function (requires `OVER`). Older / MySQL: use `ROW_NUMBER` and a self-aware filter — pick the middle row(s) of the sorted set and average them if the count is even.

Step-by-step thought process: 1. Lead with `PERCENTILE_CONT` — interviewers want to know you know the modern function. Mention that 0.5 = median, 0.95 = 95th percentile. 2. Differentiate from `PERCENTILE_DISC`, which returns an actual value from the data set; `PERCENTILE_CONT` interpolates between rows when the percentile rank is non-integer. 3. Show the MySQL workaround: `SELECT AVG(amount) FROM (SELECT amount, ROW_NUMBER() OVER (ORDER BY amount) AS rn, COUNT(*) OVER () AS n FROM orders) t WHERE rn IN (FLOOR((n+1)/2), CEIL((n+1)/2))` — handles both odd and even counts. 4. Mention `PERCENTILE_CONT() WITHIN GROUP (...) OVER (PARTITION BY ...)` for median per group.

Edge cases to mention: - NULLs are ignored by `PERCENTILE_CONT` — same as other aggregates. - For per-group median, use a window-style invocation in Postgres / Oracle: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) OVER (PARTITION BY city)`. - MySQL 8 added `PERCENT_RANK` and `CUME_DIST` but still not `PERCENTILE_CONT` — the workaround remains.

What NOT to say: - Do not say "median is just AVG of MIN and MAX" — that's the range midpoint, not the median. - Do not write a stored procedure when a single query suffices.

Common follow-up: "What is the difference between `PERCENTILE_CONT` and `PERCENTILE_DISC`?" — `CONT` interpolates between two adjacent values when the rank lands between rows; `DISC` picks the value at the lower of those two rows. `CONT` can return a value that does not appear in the data; `DISC` cannot.

```
-- Postgres / Oracle (SQL Server uses IDENTITY(1,1) instead) / BigQuery
SELECT city,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS median_amount
FROM orders
GROUP BY city;
```

Q321 · Mode — Most Frequent Value

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, AmEx, Wells Fargo · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Find the most common payment method per city. What's the cleanest way?

The 30-second answer: Postgres has a native `MODE() WITHIN GROUP (ORDER BY payment_method)` ordered-set aggregate. Elsewhere, use `ROW_NUMBER` over `COUNT(*)` per `(city, payment_method)` and pick the top row per city, or use a correlated subquery / lateral join.

Step-by-step thought process: 1. Start with the Postgres / Oracle native: `MODE() WITHIN GROUP (ORDER BY payment_method)` — returns the single most frequent value (alphabetically first on ties for Postgres). 2. Walk through the portable approach: aggregate to `(city, method, cnt)`, then use `ROW_NUMBER() OVER (PARTITION BY city ORDER BY cnt DESC, method)` and filter to `rn = 1`. 3. Mention tie-breaking: `MODE()` with no second tiebreaker is non-deterministic on ties; the `ROW_NUMBER` approach lets you specify which method wins on a tie. 4. Note this is the same pattern as top-N per group from Chapter 8 — mode is just top-1 by count.

Edge cases to mention: - For multiple modes when there is a tie, you need `RANK() OVER (PARTITION BY city ORDER BY cnt DESC)` and filter to `rk = 1` — returns all tied modes. - Empty group: `MODE` returns `NULL`; the `ROW_NUMBER` approach returns no row. - `NULLs` in the data: `MODE` skips `NULLs` in the input the same way other aggregates do.

What NOT to say: - Do not call mode "the average of the most common values" — that is not what mode means. - Do not write a subquery per city — terrible at scale.

Common follow-up: "How would you compute mode without a window function?" — Self-join the aggregated counts on `city, max(cnt) per city`, then pick the matching method. Possible but ugly.

```
-- Postgres native
SELECT city,
       MODE() WITHIN GROUP (ORDER BY payment_method) AS top_method
FROM orders
GROUP BY city;

-- Portable
SELECT city, payment_method, cnt
FROM (
  SELECT city, payment_method, COUNT(*) AS cnt,
         ROW_NUMBER() OVER (PARTITION BY city ORDER BY COUNT(*) DESC) AS rn
  FROM orders
  GROUP BY city, payment_method
) t
WHERE rn = 1;
```

Q322 · Standard Deviation — STDDEV vs STDDEV_POP vs STDDEV_SAMP

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, AmEx, Wells Fargo · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

What is the difference between STDDEV_POP and STDDEV_SAMP, and which does plain STDDEV map to?

The 30-second answer: STDDEV_POP divides by N (the full population size); STDDEV_SAMP divides by N - 1 (sample variance, Bessel's correction). Plain STDDEV is dialect-dependent — Postgres treats it as STDDEV_SAMP, Oracle treats it as STDDEV_SAMP, SQL Server's STDEV is also sample-based. Be explicit when it matters.

Step-by-step thought process: 1. Explain when each applies: STDDEV_POP if you have the whole population (e.g. every order ever placed); STDDEV_SAMP if your data is a sample of a larger population (most empirical statistics). 2. The N-1 correction matters for small samples; for large samples the two are nearly identical. 3. State the dialect mapping: Postgres / Oracle / MySQL / SQL Server all default to sample variance for plain `STDDEV` or `STDEV` — but write the explicit name to remove doubt. 4. Mention VAR_POP and VAR_SAMP — same pattern for variance, and the square root relationship: $STDDEV_SAMP = \sqrt{VAR_SAMP}$.

Edge cases to mention: - STDDEV_SAMP on a single row returns NULL (cannot divide by N-1 = 0); STDDEV_POP returns 0. - Both ignore NULLs and treat the remaining values as the dataset. - BigQuery uses STDDEV (sample) and STDDEV_POP; Snowflake uses STDDEV and STDDEV_POP (where STDDEV is sample by default).

What NOT to say: - Do not say "they are the same on large data" — they converge but they are not identical. - Do not use plain STDDEV in financial reports without checking the dialect's default.

Common follow-up: "How do you compute the coefficient of variation per city?" — `STDDEV_SAMP(amount) / NULLIF(AVG(amount), 0)` — guards against division by zero.

```
SELECT city,
       AVG(amount)           AS mean_amount,
       STDDEV_SAMP(amount) AS sample_sd,
       STDDEV_POP(amount)  AS population_sd
FROM orders
GROUP BY city;
```

Q323 · Histogram via width_bucket or CASE

Tags: Difficulty: Medium · Asked at: Walmart Labs, Swiggy, Target, PhonePe · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Build a 10-bucket histogram of order amounts between ₹0 and ₹10,000.

The 30-second answer: Use Postgres's `width_bucket(amount, 0, 10000, 10)` which returns the bucket number 1 to 10 (plus 0 for values below the low bound and 11 for values above). Group by the bucket number and count. For other dialects, derive the bucket with `FLOOR(amount / 1000) + 1` and clamp.

Step-by-step thought process: 1. Set up the function: `width_bucket(value, low, high, bucket_count)` is ANSI standard, supported in Postgres and Oracle, returning equal-width bucket numbers. 2. Build the query: `SELECT width_bucket(amount, 0, 10000, 10) AS bucket, COUNT(*) FROM orders GROUP BY bucket ORDER BY bucket`. 3. For dialects without `width_bucket` (MySQL, SQL Server), use `LEAST(GREATEST(FLOOR(amount/1000), 0), 9) + 1 AS bucket` — same effect. 4. Mention that for non-uniform bins (e.g. log scale), you need CASE expressions or `width_bucket` with an array of breakpoints (Postgres extension).

Edge cases to mention: - `width_bucket` returns 0 for values below `low` and `bucket_count + 1` for values at or above `high` — handle these explicitly if you do not want them. - For very skewed data, equal-width bins are uninformative — quantile-based bins via `NTILE` are usually better. - For visualisation, add a `bucket_low` and `bucket_high` label per row using the same arithmetic.

What NOT to say: - Do not write ten CASE WHENs by hand if `width_bucket` is available — it's exactly what the function is for. - Do not say it works on logarithmic bins by default — it does not; you'd take LOG of the input first.

Common follow-up: "How would you make the bins logarithmic instead?" —

`width_bucket(LOG(amount), LOG(1), LOG(10000), 10)` — assumes positive values.

```
-- Postgres
SELECT width_bucket(amount, 0, 10000, 10) AS bucket,
       COUNT(*) AS orders,
       MIN(amount) AS bucket_low,
       MAX(amount) AS bucket_high
FROM orders
GROUP BY bucket
ORDER BY bucket;
```

Q324 · TOP-N With Ties — RANK vs FETCH FIRST WITH TIES

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, Wells Fargo, AmEx · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

Give me the top 5 customers by GMV. What if there's a tie at the boundary — should the 6th row be included?

The 30-second answer: If you must include all ties at the boundary, use

`FETCH FIRST 5 ROWS WITH TIES` (SQL:2008) — supported in Postgres 13+, SQL Server, Oracle 12c+. Or use `RANK() OVER (ORDER BY gmv DESC)` in a CTE and filter `WHERE rk <= 5`. Plain `LIMIT 5` truncates arbitrarily and breaks ties non-deterministically.

Step-by-step thought process: 1. Frame the tie problem: with `LIMIT 5` and three customers tied for 4th place, you arbitrarily keep one and drop two — different runs may return different customers. 2. Introduce `FETCH FIRST N ROWS WITH TIES` — ANSI standard since SQL:2008. Requires ORDER BY to determine ties. 3. Compare with the RANK approach: `RANK() OVER (ORDER BY gmv DESC) AS rk WHERE rk <= 5` — same idea, more portable across older databases (works in MySQL 8 too). 4. Mention DENSE_RANK if you want "top 5 distinct GMV values regardless of how many customers share them."

Edge cases to mention: - LIMIT in Postgres / MySQL has no WITH TIES variant — you must use FETCH FIRST or the RANK CTE. - ROW_NUMBER ties are broken arbitrarily by the planner — use RANK or DENSE_RANK if you care about ties. - Combined with grouping: top 5 customers per city via `RANK() OVER (PARTITION BY city ORDER BY gmv DESC)`.

What NOT to say: - Do not say "the database will sort and pick consistently" — it won't, plans change. - Do not claim `LIMIT 5` is identical to `FETCH FIRST 5 ROWS ONLY` in terms of tie handling — they are the same, but neither handles ties.

Common follow-up: "What if I want the top 5% of customers by GMV?" —

`NTILE(20) OVER (ORDER BY gmv DESC)` and filter to tile = 1.

```
-- Postgres 13+ / SQL Server / Oracle 12c+
SELECT customer_id, SUM(amount) AS gmv
FROM orders
GROUP BY customer_id
ORDER BY gmv DESC
FETCH FIRST 5 ROWS WITH TIES;
```

Q325 · GROUP BY on Expression — When Planner Uses an Index

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Walmart Labs, Flipkart · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

`GROUP BY DATE(created_at)` is slow on a hundred-million-row table. Why, and how do you make it fast?

The 30-second answer: A regular B-tree on `created_at` cannot be used to group by `DATE(created_at)` because the function changes the value the engine would compare. Create a functional index on `DATE(created_at)` — Postgres, MySQL 8+, Oracle and SQL Server all support computed-column or expression indexes. Alternatively, store the truncated date as a generated column and index it.

Step-by-step thought process: 1. Explain why the index does not help: the planner compares the indexed value (a timestamp) to the grouping expression (a date). It cannot prove they correspond without trying every row. 2. Introduce expression indexes: `CREATE INDEX idx_orders_date ON orders (DATE(created_at))` — Postgres, MySQL 8+. The planner can then group via index range scan. 3. Show the alternative — a virtual / generated column `created_date GENERATED ALWAYS AS (DATE(created_at)) STORED` indexed normally. 4. Mention BRIN indexes for naturally ordered data (time series) — small and effective for date-grouped scans on huge tables.

Edge cases to mention: - The expression in GROUP BY must exactly match the indexed expression — `DATE(created_at)` and `DATE_TRUNC('day', created_at)` are not the same to the planner. - Functional indexes are immutable — the function must be deterministic. - BigQuery / Snowflake do not have user-defined indexes; you partition the table by date instead — same outcome, different mechanism.

What NOT to say: - Do not say "just add an index on created_at" — that does not help here. - Do not blame "the database" without diagnosing — run EXPLAIN, look for Seq Scan vs Index Scan.

Common follow-up: "What if the GROUP BY is on a CASE expression that depends on multiple columns?" — Same trick: index the expression directly, as long as it is deterministic.

```
-- Postgres functional index
CREATE INDEX idx_orders_day ON orders (DATE(created_at));

-- Query that can now use it
SELECT DATE(created_at) AS day, COUNT(*)
FROM orders
GROUP BY DATE(created_at);
```

Q326 · Aggregating JSON and Arrays

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Swiggy · Topic: Aggregations & Grouping

The question (interviewer's verbatim phrasing):

The events table stores a JSONB column `payload` with `{"category": "...", "amount": ...}`. Give me total amount per category in Postgres.

The 30-second answer: Extract the JSON fields with the `->>` operator and cast appropriately, then aggregate: `SELECT payload->>'category' AS category, SUM((payload->>'amount')::numeric) FROM events GROUP BY payload->>'category'`. For better performance on large JSON tables, create an expression index on the category key.

Step-by-step thought process: 1. Show the JSON extraction operators: `->` returns JSON; `->>` returns text; `#>` and `#>>` for nested paths. 2. Group by the extracted key and aggregate the numeric value cast from text. 3. Index hint: `CREATE INDEX ON events ((payload->>'category'))` — a Postgres functional index on the key makes the GROUP BY use an index scan. 4. Mention `jsonb_each_text` and `jsonb_array_elements` for cases where you need to expand the JSON into rows first.

Edge cases to mention: - Casting `(payload->>'amount')::numeric` throws if any value is non-numeric — wrap in a CASE or use `nullif` to skip. - MySQL has equivalent JSON functions (`JSON_EXTRACT`, `->>`); SQL Server uses `JSON_VALUE`; BigQuery uses dot notation on `STRUCT`. - For arrays inside JSON, unnest first: `LATERAL jsonb_array_elements(payload->'items') AS item` gives one row per item.

What NOT to say: - Do not say "JSON queries are always slow" — with the right expression index they are competitive with normal columns. - Do not use `->` when you want text — `->'category'` returns JSON, including the surrounding quotes; `->>'category'` returns the bare text.

Common follow-up: "How do you aggregate values inside a JSON array per row?" — Use `jsonb_array_elements` to unnest into a derived table, then aggregate normally.

```
-- Postgres
SELECT payload->>'category' AS category,
       SUM((payload->>'amount')::numeric) AS total_amount,
       COUNT(*) AS events
FROM events
GROUP BY payload->>'category'
ORDER BY total_amount DESC;
```

Chapter 8 — Set Operations & Advanced SQL

Set operations and MERGE-style patterns are where interviewers separate the candidates who only know SELECT-JOIN-GROUP-BY from the ones who can actually ship data pipelines. UNION, INTERSECT and EXCEPT are the algebra of result sets — once you internalise them, half of the "find rows in A but not in B" puzzles vanish. MERGE / UPSERT is the daily bread of ingestion code: every nightly batch, every event-stream sink, every refresh job needs to insert-or-update without race

conditions. PIVOT, UNPIVOT and crosstab are how raw narrow tables become reports your stakeholders can read. Then there are the quiet power tools — TABLESAMPLE for ad-hoc QA, LATERAL with set-returning functions for elegant denormalisation, RETURNING / OUTPUT for single-round-trip writes, and the perpetual TRUNCATE-vs-DELETE landmine. This chapter covers thirty questions that surface in mid-to-senior rounds at Razorpay, Swiggy, Cred, Zerodha, JP Morgan India, Walmart Labs and the GCC analytics teams — the ones where the panel cares as much about your dialect awareness as your SQL.

Q327 · UNION vs UNION ALL — Semantic and Performance Difference

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wells Fargo · Topic: Set Operations

The question (interviewer's verbatim phrasing):

What's the difference between UNION and UNION ALL? Which one should I default to and why?

The 30-second answer: UNION removes duplicate rows from the combined result; UNION ALL keeps every row from every input. UNION ALL is faster because it skips the sort/hash deduplication step — default to it unless you genuinely need distinct rows.

Step-by-step thought process: 1. Open with the semantic difference — UNION is a set union (duplicates removed), UNION ALL is a multiset union (everything kept). 2. Explain the cost: UNION needs to sort or hash every row across both inputs to detect duplicates, which is $O(N \log N)$ work plus memory. UNION ALL is a simple streaming concatenation. 3. Make the recommendation concrete: if you know the two inputs cannot overlap (e.g. last-month orders + this-month orders), UNION ALL is correct and cheap. If you are combining customers from two regions and the same email might appear in both, UNION is what you want. 4. Be ready for the follow-up: "How would you dedupe more efficiently than UNION?" — sometimes a `GROUP BY` on the natural key after UNION ALL is faster, especially if you also want aggregation.

Edge cases to mention: - UNION treats NULLs as equal for deduplication purposes — `(1, NULL)` and `(1, NULL)` collapse to one row, even though `NULL = NULL` is UNKNOWN in WHERE. - UNION DISTINCT is the SQL standard name; most engines accept it as a synonym. - If you have an index that already produces sorted, unique output, UNION may be nearly free — but you cannot rely on this; check the plan.

What NOT to say: - Do not say "UNION is always slower" — when the planner sees a unique index covering the union key, the dedup is essentially free. - Do not claim UNION ALL gives wrong results in production — it gives the result the query asked for; the bug is in choosing the wrong operator.

Common follow-up: "You have ten partition tables you want to combine before aggregating. UNION or UNION ALL?" — UNION ALL, every time. The aggregation will dedupe naturally, and you avoid an unnecessary sort.

```
-- Fast: known non-overlapping months
SELECT order_id, amount FROM orders_apr
UNION ALL
SELECT order_id, amount FROM orders_may;
```

Q328 · INTERSECT — What It Does and When to Reach for It

Tags: Difficulty: Easy · Asked at: Infosys, Accenture, JP Morgan, Goldman · Topic: Set Operations

The question (interviewer's verbatim phrasing):

Explain INTERSECT with an example. When would you use it instead of a JOIN?

The 30-second answer: INTERSECT returns rows that appear in both result sets, deduplicated. Reach for it when you want the conceptually-clean answer to "which keys appear in both these queries", without writing a self-join or an IN subquery.

Step-by-step thought process: 1. Concrete example: users who placed orders in both March and April. Two SELECTs returning `user_id`, with INTERSECT between them. 2. Compare to alternatives: `INNER JOIN` on `user_id` works but multiplies rows if either side has duplicates; `IN` with two subqueries is verbose. INTERSECT is the cleanest expression of "set membership in both." 3. Mention column compatibility: both inputs must have the same number of columns and compatible data types in order. The result column names come from the first SELECT. 4. Anticipate: "Is it standard SQL?" Yes — supported by Postgres, SQL Server, Oracle, SQLite and MySQL 8.0.31+.

Edge cases to mention: - INTERSECT removes duplicates by default; `INTERSECT ALL` (Postgres, Oracle) preserves multiplicity — useful when counts matter. - NULLs in matching positions are treated as equal, same as UNION's deduplication. - Performance is typically a sort-merge or hash; large inputs benefit hugely from indexes on the join columns.

What NOT to say: - Do not call INTERSECT a JOIN — it is a set operation that works on entire rows, not column matches. - Do not assume MySQL has always supported it; older versions need EXISTS workarounds.

Common follow-up: "Find products bought by every customer in your premium tier." — Classic "relational division" problem; INTERSECT chained across per-customer queries is one approach, but a `GROUP BY product_id HAVING COUNT(DISTINCT customer_id) = (SELECT COUNT(*) FROM premium_customers)` is usually cleaner.

```
-- Users active in both March and April
SELECT user_id FROM orders WHERE order_month = '2026-03'
INTERSECT
SELECT user_id FROM orders WHERE order_month = '2026-04';
```

Q329 · EXCEPT (MINUS in Oracle) — Set Subtraction

Tags: Difficulty: Easy · Asked at: Wipro, TCS, Walmart Labs, AmEx · Topic: Set Operations

The question (interviewer's verbatim phrasing):

What does EXCEPT do? And what's the Oracle equivalent? Show me a use case from a real reconciliation job.

The 30-second answer: EXCEPT returns rows from the first query that do not appear in the second, deduplicated. Oracle calls the same operator MINUS. It's the natural way to express "in source, not in target" — the bread-and-butter of reconciliation between systems.

Step-by-step thought process: 1. Lead with the standard-vs-Oracle naming so the interviewer knows you've worked across both. 2. Concrete reconciliation scenario: nightly job loads `orders_staging` from upstream; you run `SELECT order_id FROM orders_staging EXCEPT SELECT order_id FROM orders` to find rows that the staging table has but production hasn't ingested. The output is your missing-rows backlog. 3. Mention column compatibility: same number of columns, compatible types, deduplication of NULLs as equal. 4. Be ready for: "Why not just do a LEFT JOIN with IS NULL?" — Both work; EXCEPT is more declarative and dedupes for free, but LEFT JOIN lets you pull additional columns from the source without subquerying.

Edge cases to mention: - `EXCEPT ALL` (Postgres, Oracle 21+) preserves multiplicity — if the left has three copies and the right has one, you get two. - MySQL added EXCEPT in 8.0.31; for older versions, use `NOT EXISTS` or `LEFT JOIN ... IS NULL`. - Performance is typically hash-based; an index on the comparison columns matters a lot on large tables.

What NOT to say: - Do not confuse `EXCEPT` with `NOT IN` — `NOT IN` blows up on NULL in the inner set; `EXCEPT` does not. - Do not assert that MINUS works in Postgres — it doesn't; you'll have to switch keywords.

Common follow-up: "What if I want both 'in source not in target' AND 'in target not in source' in one query?" — Run `(A EXCEPT B) UNION ALL (B EXCEPT A)` — covered in Q8 as the symmetric-difference / diff pattern.

```
-- Orders in staging that haven't been ingested into production
SELECT order_id FROM orders_staging
EXCEPT
SELECT order_id FROM orders; -- Oracle: MINUS
```

Q330 · Column Compatibility in Set Operations

Tags: Difficulty: Easy · Asked at: Cognizant, Infosys, Wells Fargo, Target · Topic: Set Operations

The question (interviewer's verbatim phrasing):

What are the rules for column compatibility across UNION, INTERSECT and EXCEPT? Where do people get tripped up?

The 30-second answer: Every branch must have the same number of columns, and the corresponding columns must be of compatible types (the engine will coerce numeric and string families). Column names come from the first branch — the names in subsequent branches are ignored.

Step-by-step thought process: 1. Open with the three rules: same column count, position-based matching, type compatibility. 2. Explain the "type compatibility" nuance — INT and BIGINT will merge; INT and TEXT will not (Postgres throws, MySQL silently coerces — bug magnet). 3. Common trip-up: people assume column names match by name, not position. If branch 1 is `SELECT user_id, name` and branch 2 is `SELECT name, user_id`, the result is nonsense but executes. 4. Anticipate: "What if one branch has a NULL column and the other has a typed column?" — NULL is typeless until paired; the engine takes the type from the non-null side.

Edge cases to mention: - DATE and TIMESTAMP unify to TIMESTAMP in Postgres; the precision of the looser type wins. - VARCHAR(50) and VARCHAR(100) unify to the wider; CHAR-padded strings can introduce trailing-space surprises. - ENUM types do not unify cleanly in MySQL — cast both sides to a common string type first.

What NOT to say: - Do not insist that column names matter for matching — they do not, and you will look junior. - Do not assume implicit conversion is always safe — in MySQL, `'1'` vs `1` unify silently but with weird collation effects.

Common follow-up: "How would you align two SELECTs with different column sets for a UNION?" — Add `NULL AS missing_col` (with an explicit CAST if needed) to the side that lacks the column, in the right position. Be explicit about types when NULLs appear.

```
-- Pad the missing column with NULL of the correct type
SELECT id, name, email FROM users_v1
UNION ALL
SELECT id, name, CAST(NULL AS VARCHAR(255)) FROM users_v2;
```

Q331 · ORDER BY with UNION — Only at the End

Tags: Difficulty: Easy · Asked at: TCS, Wipro, Accenture, Razorpay · Topic: Set Operations

The question (interviewer's verbatim phrasing):

Where does ORDER BY go when you use UNION? What happens if you put it on each branch?

The 30-second answer: ORDER BY applies to the final combined result, so it goes once at the end of the last SELECT. Per-branch ORDER BY is either rejected (Postgres, SQL Server) or silently ignored (some MySQL setups) because the union must reshuffle anyway.

Step-by-step thought process: 1. Lead with the rule: one ORDER BY, at the very end, after the last branch. 2. Explain why: UNION is conceptually a single result-set producer; the engine is free to interleave or sort the combined output however it likes during deduplication. A per-branch ORDER BY would be meaningless. 3. Note the syntax workaround if you really need per-branch ordering — wrap each branch in a derived table with `(SELECT ... ORDER BY ... LIMIT N)` and union the wrappers. This is useful for "top 5 from each region" patterns. 4. Mention column reference: ORDER BY at the end references the first branch's column names (or positional `ORDER BY 1, 2`).

Edge cases to mention: - LIMIT and OFFSET also belong only at the end of the union, unless wrapped per-branch. - Postgres lets you write `ORDER BY` against any column expression from the first branch; SQL Server is stricter about needing the column to appear in the SELECT list. - For very large unions, sorting at the end can spill to disk — sometimes pre-sorting each branch with an index helps the engine do a merge-sort.

What NOT to say: - Do not say "ORDER BY each branch for performance" — it is at best ignored, at worst rejected. - Do not assume the union preserves the order of the branches — it does not; rows may interleave.

Common follow-up: "Give me the top 3 sellers from North and top 3 from South in one result, ordered by region then rank." — Wrap each as a derived table with its own ORDER BY + LIMIT, UNION ALL them, then ORDER BY at the outer level.

```
(SELECT 'North' AS region, seller_id, sales FROM north_sales ORDER BY sales DESC LIMIT 3)
UNION ALL
(SELECT 'South' AS region, seller_id, sales FROM south_sales ORDER BY sales DESC LIMIT 3)
ORDER BY region, sales DESC;
```

Q332 · Set Operations vs JOIN — When Each Is the Right Tool

Tags: Difficulty: Medium · Asked at: Flipkart, Swiggy, Cred, JP Morgan · Topic: Set Operations

The question (interviewer's verbatim phrasing):

When would you reach for a set operation versus a JOIN? They feel similar — when does the distinction actually matter?

The 30-second answer: JOIN combines columns from two tables row-by-row using a match condition; set operations combine entire rows of compatible result sets. Use JOIN when you need columns from both sides; use set operations when you are asking a yes/no question about row membership across queries.

Step-by-step thought process: 1. Frame the distinction: JOIN is horizontal (widening rows with more columns); UNION/INTERSECT/EXCEPT are vertical (stacking or filtering whole rows). 2. Give a concrete contrast: "Customers who appear in both the iOS and Android user tables" — INTERSECT on `user_id`. "Customers with their iOS device model and their Android device model side by side" — INNER JOIN. 3. Mention the duplication subtlety: a JOIN can multiply rows when the match isn't one-to-one; set operations dedupe (UNION, INTERSECT, EXCEPT) or preserve (UNION ALL). 4. Performance angle: set operations often need full materialisation and sort/hash for dedupe; JOIN with good indexes can stream. Choose by intent first, then optimise if needed.

Edge cases to mention: - Anti-join (`A LEFT JOIN B ... WHERE B.id IS NULL`) and EXCEPT solve overlapping problems; benchmark on your data. - Semi-join (`WHERE EXISTS`) and INTERSECT-on-key both express "in both" but EXISTS lets you keep extra columns from A without losing them to the set dedup. - Some optimizers rewrite EXCEPT as anti-join internally — check `EXPLAIN` .

What NOT to say: - Do not claim JOIN can always replace set operations — UNION across heterogeneous sources (e.g. multiple shards) has no JOIN equivalent. - Do not assume INTERSECT is always faster than a JOIN with DISTINCT — depends on the planner and your indexes.

Common follow-up: "Rewrite this EXCEPT as a LEFT JOIN." — Practice both directions; interviewers love checking that you can move between formulations.


```
-- EXCEPT vs LEFT JOIN equivalence
SELECT order_id FROM staging EXCEPT SELECT order_id FROM live;
-- Equivalent
SELECT s.order_id FROM staging s
LEFT JOIN live l ON s.order_id = l.order_id
WHERE l.order_id IS NULL;
```

Q333 · Preserving Duplicates with UNION ALL — Why It Matters

Tags: Difficulty: Medium · Asked at: Walmart Labs, Cognizant, Goldman, Target · Topic: Set Operations

The question (interviewer's verbatim phrasing):

Why would I deliberately want to keep duplicates across a UNION? Give me a realistic scenario.

The 30-second answer: You want duplicates whenever each row represents an independent fact — a payment, a click, a sensor reading — and aggregating across branches must count each one. Stripping duplicates with UNION would silently undercount whenever two branches genuinely contain identical rows.

Step-by-step thought process: 1. Concrete fact-table scenario: hourly clickstream from two regions. Same `(user_id, page, hour)` could legitimately appear in both. UNION ALL preserves both rows so a downstream `COUNT(*)` reflects total clicks. 2. Performance bonus: UNION ALL skips the dedup pass entirely — for partitioned tables stitched together, it is essentially free compared to UNION. 3. Highlight the safety angle: if you know your branches cannot overlap (different time periods, different shards), UNION ALL is both correct and faster. UNION there is just slower with no upside. 4. Mention the analytical pattern: pre-aggregate per branch, UNION ALL the aggregates, then aggregate again at the top level — classic for combining multi-region telemetry.

Edge cases to mention: - UNION ALL is stable in the sense that row count equals sum of branch row counts; downstream sanity checks rely on this. - Some optimisers turn `SELECT DISTINCT ... FROM (UNION ALL ...)` into a fused operator that is faster than `UNION` directly. - Beware of accidentally creating duplicates when migrating from UNION to UNION ALL — re-check downstream consumers.

What NOT to say: - Do not assume "duplicates are always bad" — fact tables are full of legitimate repeats. - Do not switch UNION to UNION ALL just for speed without checking whether the original logic relied on deduplication.

Common follow-up: "You changed UNION to UNION ALL and your numbers doubled. What happened?" — Your branches overlap; either filter one side or revert to UNION; or pre-aggregate to a granularity where overlap is impossible.


```
-- Sum clicks across two regions where overlap is meaningful
SELECT page, COUNT(*) AS clicks
FROM (
  SELECT page FROM clicks_north
  UNION ALL
  SELECT page FROM clicks_south
) t
GROUP BY page;
```

Q334 · Diff Between Two Tables — EXCEPT + UNION Pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, JP Morgan, AmEx · Topic: Set Operations

The question (interviewer's verbatim phrasing):

Give me a single query that returns rows present in only one of two tables — the symmetric difference. How do you make it tell you which side each row came from?

The 30-second answer: `(A EXCEPT B) UNION ALL (B EXCEPT A)` is the symmetric difference. To label the side, add a string literal column in each branch:

`SELECT 'only_in_A' AS source, ... FROM (A EXCEPT B)` and the equivalent for B.

Step-by-step thought process: 1. Write the symmetric-difference pattern first — interviewers want to see the EXCEPT-UNION ALL combo recognised on sight. 2. Add the source label so the diff is actionable: when you hand the result to ops, they need to know which side is missing the row. 3. Mention the inverse — rows present in both — is INTERSECT; the three pieces (`only A` , `only B` , `both`) make a full Venn breakdown. 4. Performance note: each EXCEPT does its own sort/hash dedup. For huge tables, a single FULL OUTER JOIN with a CASE expression is often faster and shows both sides plus matched rows in one pass.

Edge cases to mention: - Both inputs must have identical column projections; if schemas differ, project a common subset first. - NULLs collapse as equal in EXCEPT, just like UNION — useful for tables with nullable comparison columns. - For row-level diff including column-level differences (not just key presence), FULL OUTER JOIN with `CASE WHEN a.col IS DISTINCT FROM b.col` is the better tool.

What NOT to say: - Do not propose two separate queries and tell the user to eyeball them — give one labelled result. - Do not use `OR` to mix EXCEPT branches; UNION ALL is the only safe combiner.

Common follow-up: "Now show me rows where the key exists in both but a column value differs." — That's a FULL OUTER JOIN with a `WHERE a.col IS DISTINCT FROM b.col` filter; covered in JOIN chapter but worth knowing here.

```
-- Symmetric diff with source labels
SELECT 'only_in_staging' AS source, order_id FROM (
  SELECT order_id FROM staging EXCEPT SELECT order_id FROM live
) s
UNION ALL
SELECT 'only_in_live' AS source, order_id FROM (
  SELECT order_id FROM live EXCEPT SELECT order_id FROM staging
) l;
```

Q335 · UPSERT in Postgres — INSERT ... ON CONFLICT DO UPDATE

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Zerodha · Topic: MERGE / UPSERT

The question (interviewer's verbatim phrasing):

How do you do an UPSERT in Postgres? Walk me through the syntax and what **EXCLUDED** refers to.

The 30-second answer: Use **INSERT ... ON CONFLICT (key_cols) DO UPDATE SET col = EXCLUDED.col**. The **ON CONFLICT** clause needs a unique or primary-key constraint on the named columns; **EXCLUDED** is a virtual row containing the values that the failed insert was trying to set.

Step-by-step thought process: 1. Show the canonical form first — most candidates know "ON CONFLICT" but stumble on **EXCLUDED**. 2. Explain **EXCLUDED**: when a row conflicts, Postgres binds the proposed insert values to a virtual table named EXCLUDED, so **SET amount = EXCLUDED.amount** means "use the new amount we tried to insert." 3. Mention the existing-row reference: inside the DO UPDATE, the conflicting row's current values are accessible by the table name (or alias), so **SET amount = orders.amount + EXCLUDED.amount** does an additive upsert. 4. Note the conflict target options: by column list **(user_id, day)**, by constraint name **ON CONSTRAINT uq_orders_user_day**, or **DO NOTHING** if you just want to skip.

Edge cases to mention: - The conflict target must be an actual unique constraint or unique index; a regular index will not do. - Partial unique indexes work as conflict targets, but you must include their **WHERE** clause in the conflict spec for the planner to use them. - Triggers fire as expected: INSERT triggers on the insert path, UPDATE triggers on the conflict path.

What NOT to say: - Do not call this "MERGE" — Postgres also has a MERGE statement (added in 15), but ON CONFLICT is the idiomatic upsert and has different concurrency semantics. - Do not assume **DO UPDATE SET col = EXCLUDED.*** is valid — you must list each column.

Common follow-up: "What's the locking story?" — ON CONFLICT acquires a row lock on the conflicting row; it is safe under concurrent inserts without needing SERIALIZABLE isolation, which is why it's the recommended upsert path in Postgres.

```
INSERT INTO daily_totals (user_id, day, amount)
VALUES (42, '2026-05-26', 199)
ON CONFLICT (user_id, day)
DO UPDATE SET amount = daily_totals.amount + EXCLUDED.amount;
```

Q336 · INSERT ... ON CONFLICT DO NOTHING — When and Why

Tags: Difficulty: Easy · Asked at: Cred, Razorpay, Flipkart, Swiggy · Topic: MERGE / UPSERT

The question (interviewer's verbatim phrasing):

Why would you use `ON CONFLICT DO NOTHING` ? Isn't that just swallowing errors?

The 30-second answer: DO NOTHING gives you an idempotent insert — if the row already exists, the insert silently no-ops instead of raising a unique-violation error. It's perfect for at-least-once event ingestion where retries are expected and re-processing is harmless.

Step-by-step thought process: 1. Frame it as idempotency, not error-swallowing — that reframe matters in the interview. 2. Use the canonical scenario: a webhook handler receives `payment_confirmed` events, sometimes duplicated by the provider. The handler does `INSERT ... ON CONFLICT (event_id) DO NOTHING` and returns 200, knowing duplicates are absorbed cleanly. 3. Mention the RETURNING gotcha: with DO NOTHING, RETURNING gives you nothing for the conflicted rows, so you cannot tell from the result whether the row was actually inserted or skipped. Use `xmax = 0` trick or do an explicit lookup if you need that signal. 4. Contrast with DO UPDATE: DO NOTHING is the right choice when the existing row is authoritative; DO UPDATE is right when the new payload should win or merge.

Edge cases to mention: - DO NOTHING also suppresses other constraint violations targeted by the conflict spec — but unrelated constraint failures (e.g. CHECK) still raise. - Concurrent inserts with DO NOTHING are safe; Postgres serialises on the unique index. - Without specifying a conflict target, `ON CONFLICT DO NOTHING` suppresses any unique-violation — useful sometimes, dangerous mostly.

What NOT to say: - Do not say "it ignores all errors" — only unique-key conflicts on the targeted columns are absorbed. - Do not use it without a conflict target unless you genuinely want to suppress every unique violation in the table.

Common follow-up: "How do you tell whether a particular insert was actually inserted versus skipped?"

— Use `INSERT ... ON CONFLICT (id) DO UPDATE SET id = EXCLUDED.id RETURNING *, (xmax = 0) AS was_inserted` — the `xmax` trick is the well-known Postgres idiom.

```
INSERT INTO processed_events (event_id, payload)
VALUES ('evt_123', '{"amount":499}')
ON CONFLICT (event_id) DO NOTHING;
```

Q337 · MySQL UPSERT — INSERT ... ON DUPLICATE KEY UPDATE

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Flipkart · Topic: MERGE / UPSERT

The question (interviewer's verbatim phrasing):

What's MySQL's equivalent of Postgres ON CONFLICT? Are there any traps?

The 30-second answer: `INSERT ... ON DUPLICATE KEY UPDATE col = VALUES(col)` (or `col = NEW.col` in MySQL 8.0.20+). It triggers on any unique-key violation, not just a named one — and that breadth is the main trap.

Step-by-step thought process: 1. Show the syntax; note `VALUES(col)` references the would-be insert value (legacy syntax), and the newer alias-based `AS new ... SET col = new.col` is the preferred modern form. 2. Spell out the trap: `ON DUPLICATE KEY` reacts to a violation of *any* unique key on the table, not a specific one. If you have two unique indexes and a conflict happens on the "wrong" one, the UPDATE still runs against the existing matching row — which may not be the row you expected. 3. Mention auto-increment behaviour: a conflicting insert still increments the auto-increment counter on InnoDB by default, leading to gaps. Set `innodb_autoinc_lock_mode=0` if gaps are unacceptable (rarely worth the contention cost). 4. Contrast with REPLACE INTO — covered in Q28; ON DUPLICATE KEY UPDATE is almost always the safer choice.

Edge cases to mention: - The affected-rows count is 1 for insert, 2 for update (a MySQL quirk) — your client library may or may not expose that. - If the row exists but the UPDATE makes no actual change, MySQL reports 0 affected rows. - Multi-row INSERT with ON DUPLICATE KEY UPDATE is supported and atomic per row but not transactional unless wrapped explicitly.

What NOT to say: - Do not call it "MERGE" — MySQL has no MERGE statement; this is the only native upsert path. - Do not use the legacy `VALUES()` function in MySQL 8.0.20+ — it emits a deprecation warning.

Common follow-up: "How do you upsert multiple rows in one statement?" — Provide a multi-row VALUES list; ON DUPLICATE KEY UPDATE applies per row independently. Useful for batch ingestion.

```
INSERT INTO daily_totals (user_id, day, amount) VALUES
(42, '2026-05-26', 199),
(43, '2026-05-26', 299) AS new
ON DUPLICATE KEY UPDATE amount = daily_totals.amount + new.amount;
```

Q338 · SQL Server MERGE — Syntax and Concurrency Bugs

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan, Wells Fargo, AmEx · Topic: MERGE / UPSERT

The question (interviewer's verbatim phrasing):

How does SQL Server's MERGE work? And why do some senior engineers recommend never using it for upsert?

The 30-second answer: MERGE takes a source and target, declares match conditions, and branches into WHEN MATCHED / WHEN NOT MATCHED actions. It is powerful but has a long history of known concurrency bugs and edge cases — Microsoft MVPs widely recommend using `INSERT ... WHERE NOT EXISTS` + `UPDATE` under serialisable isolation instead for high-concurrency upserts.

Step-by-step thought process: 1. Walk the syntax: `MERGE INTO target USING source ON match_condition WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...`. 2. Acknowledge the power: in a single statement you can insert new rows, update existing ones and even delete unmatched-by-source rows — useful for slowly-changing-dimension Type 2 loads. 3. Surface the concurrency story: under concurrent execution, MERGE has been the source of duplicate-key violations and deadlocks because the read of source/target and the write are not protected as a single locked unit by default. Microsoft has fixed several of the worst issues but the community advice (Aaron Bertrand's blog, Paul White posts) remains cautious. 4. Recommend the practical workaround for concurrent upsert: explicit transaction at HOLDLOCK/SERIALIZABLE, attempt UPDATE, then INSERT WHERE NOT EXISTS, all within `BEGIN TRY/CATCH`.

Edge cases to mention: - MERGE requires a semicolon terminator in SQL Server — a notorious source of confusing parse errors. - Source duplicates against a single target row cause MERGE to fail with "the MERGE statement attempted to UPDATE or DELETE the same row more than once" — pre-aggregate the source. - Triggers and OUTPUT clauses interact with MERGE in non-obvious ways; test thoroughly.

What NOT to say: - Do not claim MERGE is the standard upsert — it's optional in the ISO spec, and dialects implement it differently or not at all. - Do not assume MERGE is atomic for concurrency without explicit isolation control — that's the bug.

Common follow-up: "What's your concrete recommendation for upsert in SQL Server today?" — For high-concurrency workloads, an explicit transaction with `UPDATE WITH (UPDLOCK, HOLDLOCK)`

followed by `INSERT WHERE NOT EXISTS` is the safe pattern; MERGE is acceptable for batch loads where you control the source.

```
MERGE INTO daily_totals AS t
USING (VALUES (42, '2026-05-26', 199)) AS s(user_id, day, amount)
ON t.user_id = s.user_id AND t.day = s.day
WHEN MATCHED THEN UPDATE SET amount = t.amount + s.amount
WHEN NOT MATCHED THEN INSERT (user_id, day, amount) VALUES (s.user_id, s.day, s.amount);
```

Q339 · MERGE WHEN MATCHED / WHEN NOT MATCHED Patterns

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, JP Morgan, Target · Topic: MERGE / UPSERT

The question (interviewer's verbatim phrasing):

Explain the various WHEN clauses in MERGE. Can you have multiple WHEN MATCHED branches?

The 30-second answer: MERGE supports `WHEN MATCHED [AND extra_condition] THEN UPDATE/DELETE`, `WHEN NOT MATCHED [BY TARGET] THEN INSERT`, and `WHEN NOT MATCHED BY SOURCE THEN UPDATE/DELETE`. Yes, multiple WHEN MATCHED branches are allowed if each has a distinguishing predicate.

Step-by-step thought process: 1. Lay out the three families: matched, not matched (new row in source), not matched by source (row vanished from source — used for soft deletes or SCD). 2. Show the conditional branching: `WHEN MATCHED AND s.is_active = 0 THEN DELETE` plus `WHEN MATCHED AND s.is_active = 1 THEN UPDATE SET ...` lets you encode a state machine in one statement. 3. Mention the "not matched by source" clause as the rarely-known third option — invaluable for full-refresh dimension tables. 4. Anticipate the dialect spread: SQL Server has all three; Postgres 15 MERGE supports MATCHED and NOT MATCHED but the "NOT MATCHED BY SOURCE" wording landed in Postgres 17.

Edge cases to mention: - Branches are evaluated in order; the first matching one wins. Put more specific predicates first. - Mixing INSERT and DELETE across branches is allowed; the engine plans both paths. - An OUTPUT (SQL Server) or RETURNING (Postgres) clause emits one row per affected row, with an `$action` column indicating INSERT/UPDATE/DELETE — invaluable for audit.

What NOT to say: - Do not assume Postgres MERGE has full SQL Server parity — they share the surface but not every clause is available pre-17. - Do not stack WHEN MATCHED branches without `AND` predicates — without distinguishing conditions, only the first ever fires.

Common follow-up: "How would you use *MERGE* for a Type 2 SCD load?" — Use `WHEN MATCHED AND s.hash <> t.hash` to expire the old row (`UPDATE valid_to = NOW()`), then a separate `INSERT` step (often outside `MERGE`) for the new version — pure-`MERGE` SCD2 is tricky because one source row needs both an `UPDATE` and an `INSERT`, which `MERGE` cannot do natively.

```
MERGE INTO products t
USING staging_products s ON t.sku = s.sku
WHEN MATCHED AND s.is_active = 0 THEN DELETE
WHEN MATCHED AND s.is_active = 1 AND s.price <> t.price THEN UPDATE SET price = s.price
WHEN NOT MATCHED THEN INSERT (sku, price, is_active) VALUES (s.sku, s.price, 1);
```

Q340 · Postgres 17 MERGE with RETURNING Clause

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan, Goldman · Topic: `MERGE` / `UPSERT`

The question (interviewer's verbatim phrasing):

Postgres 17 added a `RETURNING` clause to `MERGE`. What does it give you and why is `merge_action()` useful?

The 30-second answer: `RETURNING` on `MERGE` emits one row per affected row, just like `RETURNING` on `INSERT`/`UPDATE`/`DELETE`. The new `merge_action()` function tells you which branch fired — `'INSERT'`, `'UPDATE'` or `'DELETE'` — so a single round trip both performs the upsert and tells the caller what actually happened per row.

Step-by-step thought process: 1. Frame the value: before Postgres 17, you had to choose between `MERGE` (powerful but no `RETURNING`) and `ON CONFLICT` (`RETURNING` but only one action). Now `MERGE` gives you both. 2. Show `merge_action()` in `RETURNING` — combined with the row's columns, your application learns exactly which keys were created vs updated vs removed without a second query. 3. Mention the use case: audit logging, downstream change-stream emission, idempotency tracking — any pipeline that needs "what changed" feedback per row. 4. Be ready for: "Can you filter `RETURNING`?" Yes — wrap the `MERGE` in a CTE and filter the CTE output, just as you would with `INSERT`/`UPDATE`.

Edge cases to mention: - The `RETURNING` list can reference target columns (post-write state) and source columns; for `DELETE`s, source columns reflect the matched source row. - Available only in Postgres 17 and later — for earlier versions, an `INSERT ... ON CONFLICT ... RETURNING` handles the upsert case but not three-way actions. - Combining `RETURNING` with `WITH cte AS (MERGE ...)` lets you chain — e.g. log to an audit table in the same statement.

What NOT to say: - Do not assume MySQL's INSERT ... ON DUPLICATE KEY supports RETURNING — it does not, even in 8.x. - Do not promise this feature in older Postgres — it really is 17+; mistake = lost credibility.

Common follow-up: "How would you log every change to an audit table in one statement?" — Use `WITH changed AS (MERGE ... RETURNING merge_action() AS action, id, ...) INSERT INTO audit_log SELECT NOW(), action, id, ... FROM changed`.

```
-- Postgres 17+
WITH changed AS (
  MERGE INTO daily_totals t
  USING incoming s ON t.user_id = s.user_id AND t.day = s.day
  WHEN MATCHED THEN UPDATE SET amount = t.amount + s.amount
  WHEN NOT MATCHED THEN INSERT (user_id, day, amount) VALUES (s.user_id, s.day, s.amount)
  RETURNING merge_action() AS action, t.user_id, t.day, t.amount
)
SELECT * FROM changed;
```

Q341 · Bulk INSERT — COPY, LOAD DATA INFILE, BULK INSERT

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Cred, Target · Topic: Bulk Load

The question (interviewer's verbatim phrasing):

What's the fastest way to load a million rows into Postgres, MySQL and SQL Server respectively?

The 30-second answer: Postgres: `COPY ... FROM STDIN` (or `\copy` from psql). MySQL: `LOAD DATA INFILE` (or `LOAD DATA LOCAL INFILE` for client-side files). SQL Server: `BULK INSERT` or `bcp`. All three bypass per-row overhead and stream rows in chunks.

Step-by-step thought process: 1. Lead with the three commands so the interviewer knows you've used each. 2. Explain why row-by-row INSERT is slow: per-statement parsing, per-row WAL/redo, per-row constraint checks, per-row index updates. Bulk loaders batch these. 3. Production tips: drop or disable secondary indexes during bulk load and rebuild after; widen `work_mem` or `bulk_logged` recovery model temporarily; load into an unlogged or staging table first if you can tolerate the durability tradeoff. 4. Mention transactional behaviour: COPY runs in a single transaction in Postgres (atomic — all or nothing); LOAD DATA INFILE is transactional in InnoDB; BULK INSERT defaults to minimal logging in `BULK_LOGGED` recovery model.

Edge cases to mention: - LOAD DATA LOCAL INFILE is often disabled by default for security (`--local-infile=1` flag required client and server side). - COPY in Postgres expects either text (tab-separated) or CSV format with carefully configured escape characters; mis-quoting is the leading cause

of corrupt loads. - BULK INSERT requires the file to be visible from the SQL Server service account, not from your client machine — a frequent first-time stumble.

What NOT to say: - Do not propose row-by-row INSERTs in a loop for a million-row load — that's an instant downgrade signal. - Do not assume bulk loaders skip constraints — they enforce PK/UNIQUE/FK by default; you must explicitly disable if you want speed (and accept the risk).

Common follow-up: "How do you load a file that doesn't quite match the table schema?" — COPY supports column lists and CSV header skipping; LOAD DATA INFILE supports `(@col1, @col2) SET col_actual = TRIM(@col1)` for inline transformation.

```
-- Postgres COPY
COPY orders (id, user_id, amount) FROM '/data/orders.csv' WITH (FORMAT csv, HEADER true);
-- MySQL
LOAD DATA INFILE '/data/orders.csv' INTO TABLE orders
  FIELDS TERMINATED BY ',' IGNORE 1 LINES (id, user_id, amount);
-- SQL Server
BULK INSERT orders FROM 'C:\data\orders.csv' WITH (FORMAT='CSV', FIRSTROW=2);
```

Q342 · INSERT INTO ... SELECT — Copy with Transformation

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Bulk Load

The question (interviewer's verbatim phrasing):

Show me how to copy data from one table to another with transformation. What are the gotchas?

The 30-second answer: `INSERT INTO target (cols) SELECT expr_list FROM source WHERE ...`. The SELECT can transform, filter, aggregate, JOIN — anything. Gotchas: column order matters, types must align, and the whole thing runs as a single transaction unless you batch.

Step-by-step thought process: 1. Show the canonical form with explicit column lists on both sides — never rely on positional matching of unnamed `INSERT INTO target SELECT * FROM source` because schema drift breaks it silently. 2. Highlight the transformation power: rename, recast, compute, aggregate during the copy. Common use: snapshotting a normalised live table into a denormalised reporting table. 3. Transaction gotcha: a million-row INSERT INTO ... SELECT generates a million WAL records and locks. For huge migrations, chunk it (`INSERT INTO ... SELECT ... WHERE id BETWEEN x AND y LIMIT N`) in a loop with commits between. 4. Mention CTAS (CREATE TABLE AS SELECT) as the close cousin when the target doesn't yet exist — faster because the new table starts empty and can skip WAL via `CREATE UNLOGGED TABLE` in Postgres.

Edge cases to mention: - DEFAULT values on the target apply only to columns omitted from the column list; if you include the column in your list and pass NULL, the default does not fire. - IDENTITY / SERIAL

columns: in Postgres, omit them from the insert column list to let the sequence assign; in SQL Server, use `SET IDENTITY_INSERT target ON` if you want to preserve source values. - Locking: INSERT INTO ... SELECT takes a row-share lock on the source; large copies can interact with concurrent writers — schedule off-peak.

What NOT to say: - Do not use `SELECT *` in production copy queries — schema drift will silently misalign columns. - Do not assume the INSERT honours the SELECT's ORDER BY for storage order (only matters in clustered-index databases like SQL Server with `WITH (TABLOCK)`).

Common follow-up: "How would you copy 100M rows without taking the system down?" — Batch by primary-key ranges, commit per batch, throttle with `pg_sleep` or sleep equivalents, and monitor replication lag.

```
INSERT INTO orders_archive (id, user_id, amount, archived_at)
SELECT id, user_id, amount, NOW()
FROM orders
WHERE created_at < NOW() - INTERVAL '1 year';
```

Q343 · PIVOT — Turning Rows into Columns (SQL Server / Oracle)

Tags: Difficulty: Medium · Asked at: TCS, Wipro, Wells Fargo, AmEx · Topic: PIVOT / UNPIVOT

The question (interviewer's verbatim phrasing):

Show me how PIVOT works. Turn monthly sales rows into one row per region with twelve month columns.

The 30-second answer: `PIVOT (aggregate(value_col) FOR pivot_col IN (lit1, lit2, ...))` rotates rows into columns. SQL Server and Oracle have a dedicated PIVOT operator; in Postgres and MySQL, you emulate it with conditional aggregation (`SUM(CASE WHEN month = 'Jan' THEN amount END) AS jan`).

Step-by-step thought process: 1. Show the SQL Server PIVOT skeleton: a subquery projecting just the three relevant columns (region, month, amount), then `PIVOT (SUM(amount) FOR month IN ([Jan], [Feb], ...)) AS p`. 2. Stress that the pivot value list (`Jan` , `Feb` , ...) must be hard-coded — PIVOT cannot expand a list dynamically. For dynamic pivots, you build the statement string at the application layer or use dynamic SQL. 3. Show the equivalent conditional aggregation pattern that works in every dialect: `SELECT region, SUM(CASE WHEN month = 'Jan' THEN amount END) AS jan, ... GROUP BY region`. 4. Mention that PIVOT is syntactic sugar over conditional aggregation under the hood — same plan, different ergonomics.

Edge cases to mention: - PIVOT requires the input subquery to project exactly the columns involved; extra columns become part of the implicit `GROUP BY` and may explode row counts unexpectedly. - NULL handling: if no source row matches a pivot value for a group, the cell is NULL — coalesce to 0 in the outer SELECT if you need a numeric grid. - Oracle PIVOT supports multiple aggregates `(SUM(amount) sum, COUNT(*) cnt FOR ...)` producing columns like `Jan_sum`, `Jan_cnt`.

What NOT to say: - Do not promise PIVOT in MySQL or Postgres — there is no native operator; emulate it. - Do not write a PIVOT and forget to wrap the source in an explicit subquery — extra columns in the base table will silently change the grouping.

Common follow-up: "What if I don't know the month list at write time?" — Build the SQL as a string at the app layer, or in T-SQL use `STRING_AGG` to construct the pivot column list and `EXEC sp_executesql` to run the assembled statement.

```
-- SQL Server
SELECT region, [Jan], [Feb], [Mar]
FROM (SELECT region, month, amount FROM sales) src
PIVOT (SUM(amount) FOR month IN ([Jan], [Feb], [Mar])) AS p;
```

Q344 · UNPIVOT — Turning Columns into Rows

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Wells Fargo, Target · Topic: PIVOT / UNPIVOT

The question (interviewer's verbatim phrasing):

Now go the other way. I have one row per region with twelve month columns; give me three columns: region, month, amount.

The 30-second answer: `UNPIVOT (amount FOR month IN ([Jan], [Feb], ...))` flips columns into rows. In dialects without UNPIVOT (Postgres pre-15, MySQL), simulate with `CROSS JOIN LATERAL (VALUES ('Jan', jan), ('Feb', feb), ...) AS u(month, amount)` or a UNION ALL across single-column SELECTs.

Step-by-step thought process: 1. Walk the SQL Server UNPIVOT first: source has `region, jan, feb, mar`; output is one row per (region, month, amount). 2. Show the portable LATERAL VALUES trick — works in Postgres and any dialect with VALUES-in-FROM support, and is often more readable than UNPIVOT. 3. Highlight the NULL behaviour: UNPIVOT drops rows where the source column is NULL. If you need to keep them, the LATERAL VALUES form preserves NULLs naturally. 4. Anticipate: "Why would I want a wide table unpivoted?" — Almost any analytics workflow prefers long-format data because GROUP BY and pivot tools (Tableau, Looker) operate on long format more flexibly.

Edge cases to mention: - All unpivoted columns must share a compatible data type — you cannot UNPIVOT an INT and a DATE into the same output column. - UNPIVOT in SQL Server uses dataset-style names; quoting with square brackets handles reserved words and unusual identifiers. - For very wide tables (200+ columns), code-generate the LATERAL VALUES list from `information_schema.columns`.

What NOT to say: - Do not implement UNPIVOT by writing 12 separate UNION ALL queries when LATERAL VALUES does it in one block — the former is verbose and brittle. - Do not claim UNPIVOT preserves NULLs — by default it drops them; explicitly say "with INCLUDE NULLS" or use the LATERAL pattern.

Common follow-up: "Unpivot a wide table with 50 numeric columns." — Generate the VALUES tuple list from `information_schema.columns` in a templating step; keeps the SQL maintainable as new columns are added.

```
-- Portable LATERAL form (Postgres)
SELECT region, month, amount
FROM sales_wide s,
LATERAL (VALUES ('Jan', jan), ('Feb', feb), ('Mar', mar)) AS u(month, amount);
```

Q345 · PIVOT via Conditional Aggregation (Postgres / MySQL)

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Flipkart · Topic: PIVOT / UNPIVOT

The question (interviewer's verbatim phrasing):

Postgres and MySQL don't have a PIVOT keyword. How do you pivot anyway?

The 30-second answer: Conditional aggregation — wrap each pivoted value in a CASE inside an aggregate and GROUP BY the surviving dimension. `SUM(CASE WHEN status = 'paid' THEN amount END) AS paid_amount, SUM(CASE WHEN status = 'failed' THEN amount END) AS failed_amount, ...` with `GROUP BY user_id`.

Step-by-step thought process: 1. Show the pattern: pick the dimension that becomes the row key (GROUP BY column), the dimension that becomes the new column headers (CASE branches), and the value (the aggregate target). 2. Explain why this is universal: every SQL engine since 1992 has supported `CASE` inside aggregates, so this works without dialect features. 3. Talk about which aggregate to pick: SUM for additive, MAX for boolean (one-hot style), COUNT for counts, BOOL_OR for presence in Postgres. 4. Anticipate the dynamic-columns question: if the set of column headers is unknown at write time, you must build the SQL string in the app layer — there is no truly dynamic pivot in pure SQL.

Edge cases to mention: - NULLs in the CASE ELSE branch are skipped by aggregates like SUM and AVG (giving you the intended "no-value" cell); be explicit if you want 0 instead with

`COALESCE(SUM(...), 0)` . - FILTER clause (`SUM(amount) FILTER (WHERE status = 'paid')`) is cleaner Postgres/standard syntax than CASE inside SUM, and the planner handles it identically. - For high-cardinality pivots (hundreds of columns), conditional aggregation becomes unwieldy — consider keeping the data long and pivoting at the BI layer.

What NOT to say: - Do not pretend Postgres has PIVOT — it does not (the `tablefunc` extension's `crosstab()` is the closest thing, covered in Q20). - Do not write `CASE WHEN ... THEN 1 ELSE 0 END` summed for a presence flag when `MAX(CASE WHEN ... THEN 1 END)` or `BOOL_OR` is what you mean — they have different semantics for counts.

Common follow-up: "Rewrite this with FILTER syntax." —

`SUM(amount) FILTER (WHERE status = 'paid') AS paid` — same plan, cleaner read, ISO SQL standard.

```
SELECT
  user_id,
  SUM(amount) FILTER (WHERE status = 'paid') AS paid,
  SUM(amount) FILTER (WHERE status = 'failed') AS failed,
  SUM(amount) FILTER (WHERE status = 'refund') AS refund
FROM payments
GROUP BY user_id;
```

Q346 · crosstab() in Postgres tablefunc Extension

Tags: Difficulty: Hard · Asked at: Cred, Razorpay, JP Morgan, Goldman · Topic: PIVOT / UNPIVOT

The question (interviewer's verbatim phrasing):

What does Postgres's crosstab function do? When is it worth using over conditional aggregation?

The 30-second answer: `crosstab()` is a function from the `tablefunc` extension that takes a SQL string of `(row_key, category, value)` triples and returns a wider record with one column per category. It shines when you have many pivot columns and want server-side pivoting without writing out one CASE per column.

Step-by-step thought process: 1. Lead with the extension prereq: `CREATE EXTENSION tablefunc;` — not in core Postgres; some managed services need this enabled by the admin. 2. Show the two-argument form: first arg is the source query producing `(row_key, category, value)` ordered by row_key, second arg names the pivot categories so the returned record has predictable column types. 3. Stress the output-typing oddity: you must declare the column list in the

calling query (`AS ct(region TEXT, jan NUMERIC, feb NUMERIC, ...)`) because the function returns a generic record. 4. Compare to conditional aggregation: for 3-5 categories, CASE/FILTER is clearer. For 20-50 categories, crosstab() reduces boilerplate dramatically.

Edge cases to mention: - The source query must be ordered by row_key (and category, if multi-row) — otherwise crosstab silently emits wrong results because it relies on grouping consecutive rows. - Missing (row, category) combinations produce NULL cells; the result has no row for a row_key that had zero categories. - Performance: crosstab is essentially syntactic sugar over the same conditional-aggregation plan, plus the overhead of crossing a function boundary.

What NOT to say: - Do not use crosstab without ordering the input — the silent-wrong-answer trap. - Do not assume RDS/Aurora has tablefunc enabled by default — check before deploying.

Common follow-up: "What happens if I forget the ORDER BY?" — Grouping breaks; rows from different keys leak into each other's output rows. Always include `ORDER BY 1, 2` in the source query.

```
CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT * FROM crosstab(
  $$SELECT region, month, amount FROM sales ORDER BY 1, 2$$,
  $$VALUES ('Jan'),('Feb'),('Mar')$$
) AS ct(region TEXT, jan NUMERIC, feb NUMERIC, mar NUMERIC);
```

Q347 · TABLESAMPLE — Random Sample of a Large Table

Tags: Difficulty: Medium · Asked at: Walmart Labs, Cred, JP Morgan, Goldman · Topic: Advanced Single-Query

The question (interviewer's verbatim phrasing):

I need a random 1% sample of a 500M-row table for an ad-hoc analysis. What's the fastest way?

The 30-second answer: Use `TABLESAMPLE` — `SELECT * FROM orders TABLESAMPLE BERNOULLI (1)` (1%) or `TABLESAMPLE SYSTEM (1)`. BERNOULLI samples row-by-row (accurate fraction, slower); SYSTEM samples whole blocks (faster, less uniform). For most ad-hoc analyses, SYSTEM is the right speed/accuracy tradeoff.

Step-by-step thought process: 1. Compare the two methods: BERNOULLI scans every row and includes each with probability p; SYSTEM picks random blocks (8 KB pages in Postgres) and includes all rows on those blocks. 2. Explain why ORDER BY RANDOM() LIMIT N is terrible at scale — it sorts the entire table by a random column. TABLESAMPLE is orders of magnitude faster. 3. Mention `REPEATABLE (seed)` for reproducible samples — handy when collaborating on a sampled dataset across sessions. 4. Cover dialect notes: SQL Server has TABLESAMPLE (block-based only); MySQL has

no TABLESAMPLE — use `WHERE RAND() < 0.01` (still scans every row, but much cheaper than ORDER BY RAND()).

Edge cases to mention: - SYSTEM sampling correlates with physical layout; if data is clustered by date, your "random" sample may have skewed date distribution. - For very small percentages, BERNOULLI may return fewer rows than expected due to variance; sample slightly larger and LIMIT. - Postgres can combine TABLESAMPLE with `WHERE` filters, but the sample applies first so total result rows shrink twice.

What NOT to say: - Do not propose `ORDER BY RANDOM() LIMIT 100` for a large table — it's a junior-grade answer. - Do not assume TABLESAMPLE gives a statistically perfect SRS — block-based SYSTEM has cluster bias.

Common follow-up: "How do I get a repeatable sample for two teams to share?" — `TABLESAMPLE BERNOULLI (1) REPEATABLE (42)` — the seed makes the random choice deterministic across runs.

```
-- Postgres: ~1% by block
SELECT * FROM orders TABLESAMPLE SYSTEM (1) REPEATABLE (42);
-- ~1% by row (more uniform)
SELECT * FROM orders TABLESAMPLE BERNOULLI (1);
```

Q348 · WITH ORDINALITY — Row Numbers from Set-Returning Functions

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan, Goldman · Topic: Advanced Single-Query

The question (interviewer's verbatim phrasing):

What does WITH ORDINALITY do in Postgres? Give me a use case where it saves the day.

The 30-second answer: `WITH ORDINALITY` attaches an auto-incrementing position column to the output of a set-returning function, starting at 1. Without it, the function returns just the values; with it, you get `(value, ordinality)` pairs so you can join back to original positions.

Step-by-step thought process: 1. Walk the canonical example: `SELECT unnest(ARRAY['a','b','c']) WITH ORDINALITY AS u(element, position)`. You get `('a', 1), ('b', 2), ('c', 3)` — perfect for preserving order through an unnest. 2. Show the killer use case: JSON arrays. `SELECT j.value, j.ordinality FROM jsonb_array_elements(payload) WITH ORDINALITY AS j(value, ordinality)` — you can correlate each array element back to its position for stable updates or joins. 3. Combine with LATERAL to unnest per row while preserving the source row's identity AND the position within its array.

4. Mention that this is standard SQL (since SQL:2003) but rarely implemented — Postgres has it; many others do not.

Edge cases to mention: - Ordinality starts at 1 and increments by 1; there is no syntax for 0-based or custom-step. - Ordering of the set-returning function's output is preserved only within a single call; multiple invocations in the same SELECT may interleave unpredictably. - Works with any set-returning function: `unnest`, `jsonb_array_elements`, `generate_series`, `string_to_table`, even user-defined.

What NOT to say: - Do not use `row_number() OVER (ORDER BY ...)` to fake ordinality on an unnested array — the inner ordering is not preserved; WITH ORDINALITY is the only correct path. - Do not assume this works in MySQL or SQL Server — it does not; emulate with window functions over a derived numbering column.

Common follow-up: "Update the third element of a JSONB array in place." — Use `WITH ORDINALITY` to find it, then `jsonb_set` with the path; the ordinality value gives you the index for the path.

```
SELECT element, position
FROM unnest(ARRAY['onboarding','payment','done']) WITH ORDINALITY AS u(element, position);
-- (onboarding,1), (payment,2), (done,3)
```

Q349 · LATERAL with Set-Returning Functions

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, JP Morgan · Topic: Advanced Single-Query

The question (interviewer's verbatim phrasing):

Explain LATERAL with an example involving a set-returning function. Why is it different from a regular subquery in FROM?

The 30-second answer: LATERAL lets a derived table or function call in the FROM clause reference columns from earlier items in the same FROM clause. Without LATERAL, a subquery in FROM is evaluated once independently; with LATERAL, it's evaluated per left-side row, like a correlated subquery you can use as a table source.

Step-by-step thought process: 1. Set up the difference with a concrete example: you have `orders` and a function `get_top_lines(order_id) RETURNS TABLE`. Without LATERAL, you cannot pass `o.id` into the function from the FROM clause; with LATERAL you can. 2. Combine LATERAL with set-returning functions: `SELECT o.id, l.sku, l.qty FROM orders o, LATERAL get_top_lines(o.id, 3) l` — you get up to 3 line rows per order, all in one fluent expression. 3. Highlight that LATERAL is the "for each row in the left, evaluate the right" pattern — formally a `CROSS`

APPLY in SQL Server. 4. Mention practical patterns: top-N-per-group via LATERAL on a subquery, JSON array exploding per row, computing per-row scalars that you also reuse later in the SELECT.

Edge cases to mention: - LEFT JOIN LATERAL preserves the outer row even when the function returns zero rows — pair with **ON true** to make it explicit. - LATERAL implies an implicit comma-join order; the LATERAL side cannot be referenced from items to its left. - Postgres, Oracle and SQL Server (as CROSS/OUTER APPLY) all support this; MySQL added LATERAL in 8.0.14.

What NOT to say: - Do not call LATERAL "just a subquery" — the lateral reference is what makes it powerful; a regular FROM subquery cannot see other FROM items. - Do not assume **CROSS APPLY** and **LATERAL** are syntactically identical — semantically equivalent, syntactically different.

Common follow-up: "Get the 3 most recent orders per user in one query." — **SELECT u.id, recent.* FROM users u, LATERAL (SELECT * FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3) recent** — the prototypical LATERAL top-N pattern.

```
-- Top 3 most expensive line items per order
SELECT o.id, l.sku, l.amount
FROM orders o,
LATERAL (
  SELECT sku, amount FROM order_lines
  WHERE order_id = o.id
  ORDER BY amount DESC LIMIT 3
) l;
```

Q350 · generate_series() and Sequence Generation Patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Flipkart · Topic: Advanced Single-Query

The question (interviewer's verbatim phrasing):

How do you generate a series of dates for a calendar dimension, or fill in gaps in a time series, without a calendar table?

The 30-second answer: Postgres has **generate_series(start, stop, step)** for integers, dates and timestamps — perfect for producing every day in a range. In MySQL/SQL Server (where there is no native equivalent), use a recursive CTE bounded by date math or a numbers table.

Step-by-step thought process: 1. Show the Postgres date version: **SELECT d::date FROM generate_series('2026-01-01'::date, '2026-12-31'::date, '1 day') d** — instant calendar. 2. Use case for gap-filling: LEFT JOIN your daily-aggregated metric against the generated series so dates with zero events still appear (with COUNT 0 / SUM 0 after coalescing). 3. Show the MySQL recursive CTE equivalent:

WITH RECURSIVE cal AS (SELECT DATE '2026-01-01' AS d UNION ALL SELECT d + 1 FROM cal WHERE d < '2026-12-31') SELECT * FROM cal . 4. SQL Server can do the same recursive CTE with `OPTION (MAXRECURSION 0)` to override the default 100-step limit.

Edge cases to mention: - generate_series with timestamps is timezone-aware in Postgres — pass timestamptz inputs if you care about DST. - For very long ranges (decades), recursive CTEs in MySQL may hit `cte_max_recursion_depth` (default 1000); increase the session variable. - A persistent calendar table is often a better long-term choice than generating series on the fly — it can hold attributes like is_holiday, fiscal_quarter, etc.

What NOT to say: - Do not use a recursive CTE in Postgres for date series when generate_series exists — it's slower and more verbose. - Do not generate huge series unbounded; always bound by an end date or LIMIT to prevent runaway plans.

Common follow-up: "Fill in zero-revenue days in the past 90 days for each user." — Cross-join generate_series of dates with the user list, LEFT JOIN actual sales, COALESCE the amount to 0. Standard gap-fill pattern.

```
-- Postgres: every day in May 2026
SELECT d::date AS day
FROM generate_series('2026-05-01'::date, '2026-05-31'::date, '1 day') d;
```

Q351 · DELETE with USING / DELETE with JOIN

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Walmart Labs · Topic: DML

The question (interviewer's verbatim phrasing):

How do you delete from one table based on a condition that requires JOINing to another table? Each dialect does it slightly differently.

The 30-second answer: Postgres: `DELETE FROM target USING other WHERE join_cond AND filter` . MySQL: `DELETE t FROM target t JOIN other o ON ... WHERE ...` . SQL Server: `DELETE FROM target FROM target JOIN other ON ... WHERE ...` . All three say "delete rows from target that match a condition you can only express by referencing another table."

Step-by-step thought process: 1. Show all three dialect forms — interviewers love when you can write the same logical operation in each major engine. 2. Stress that the table being deleted from is the one named immediately after DELETE (or in the FROM list); the joined table contributes filter logic only. 3. Concrete use case: delete payments whose orders have been soft-deleted. `DELETE FROM payments USING orders WHERE payments.order_id = orders.id AND orders.deleted_at IS NOT NULL` . 4. Anticipate: "Why not DELETE with a subquery?" — Subquery works too (`DELETE FROM`

`payments WHERE order_id IN (SELECT id FROM orders WHERE deleted_at IS NOT NULL)`) and is more portable; JOIN-style is more readable when you need columns from multiple joined tables for filtering.

Edge cases to mention: - ON DELETE CASCADE on a FK will delete child rows automatically — sometimes a JOIN-DELETE is reinventing what the schema already does. - DELETE with JOIN can be slow on huge tables; batch with `WHERE id BETWEEN x AND y LIMIT N` to avoid long-running transactions and replication lag. - Some dialects allow deleting from multiple tables in one statement (MySQL: `DELETE t1, t2 FROM ...`); rarely a good idea — explicit is clearer.

What NOT to say: - Do not assume Postgres allows `DELETE FROM target JOIN other` — it does not; use `USING` instead. - Do not run a large unbatched JOIN-DELETE in production without monitoring replication and lock waits.

Common follow-up: "How would you safely delete 10M rows without blocking the table?" — Loop in batches of 10K-50K with explicit transactions and short sleeps, optionally with a covering index on the filter column for fast row identification.

```
-- Postgres
DELETE FROM payments USING orders
WHERE payments.order_id = orders.id AND orders.deleted_at IS NOT NULL;

-- MySQL
DELETE p FROM payments p JOIN orders o ON p.order_id = o.id WHERE o.deleted_at IS NOT NULL;
```

Q352 · UPDATE with FROM / JOIN

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Walmart Labs · Topic: DML

The question (interviewer's verbatim phrasing):

Same setup for UPDATE — how do you update one table using values from another?

The 30-second answer: Postgres: `UPDATE target SET col = other.col FROM other WHERE join_cond` . MySQL: `UPDATE target t JOIN other o ON ... SET t.col = o.col` . SQL Server: `UPDATE target SET col = o.col FROM target t JOIN other o ON ...` . The semantics are identical — pull values from the joined table into the target.

Step-by-step thought process: 1. Show each dialect; mention that the Postgres form is closest to SQL standard with the explicit FROM clause. 2. Concrete example: update `orders.payment_status` from a staging table after a reconciliation run. `UPDATE orders o SET payment_status = s.new_status FROM payment_recon s WHERE o.id = s.order_id` . 3. Stress the cardinality

trap: if the join condition matches multiple right-side rows for one left row, the behaviour is dialect-specific — Postgres picks an arbitrary one (and logs a warning in stricter modes); MySQL also picks arbitrarily. Pre-aggregate or use DISTINCT ON to avoid this. 4. Anticipate: "Why not a correlated subquery in SET?" — Works too (`SET col = (SELECT s.col FROM staging s WHERE s.id = orders.id)`) but it's harder for the planner to optimise as a join.

Edge cases to mention: - Updating the join column itself (e.g. `SET id = ...`) is risky; the row may move in the index and confuse the in-flight scan — separate the rename into its own statement. - WHEN multiple update candidates exist, you can rank with DISTINCT ON or window functions in a derived table, then JOIN to that. - Always confirm the affected-row count after a JOIN-UPDATE — if it's wildly higher than expected, you may have an accidental cartesian.

What NOT to say: - Do not put the join condition in the SET clause's subquery and then forget the outer WHERE — you'll update every row in the target with NULL where the subquery doesn't match. - Do not use UPDATE-JOIN to do bulk insert-or-update; that's what MERGE / ON CONFLICT is for.

Common follow-up: "What if multiple staging rows match the same target row?" — Pre-aggregate the staging side to one row per target key, or use `DISTINCT ON (target_id) ... ORDER BY priority` in Postgres to deterministically pick the winner.

```
-- Postgres
UPDATE orders o
SET payment_status = s.new_status
FROM payment_recon s
WHERE o.id = s.order_id;
```

Q353 · TRUNCATE vs DELETE — Performance and Transactional Differences

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant · Topic: DML

The question (interviewer's verbatim phrasing):

What's the difference between TRUNCATE and DELETE? Why is TRUNCATE faster, and what does it cost you?

The 30-second answer: DELETE removes rows one-by-one (logged, fires triggers, can be ROLLBACK-ed), respecting WHERE clauses. TRUNCATE drops and recreates the table's data pages — instant, minimally-logged, but no WHERE, no per-row triggers, may reset identity counters, and can't be ROLLBACK-ed in some dialects.

Step-by-step thought process: 1. Tabulate the differences: row-by-row vs page-drop; logged vs minimally logged; triggers vs no triggers; ROLLBACK-able everywhere in Postgres but auto-commits in

MySQL (DDL behaviour); identity reset in SQL Server (with **RESEED**) and Oracle (with **REUSE STORAGE**), depends on Postgres options. 2. Use case for TRUNCATE: staging tables that are wiped before each batch load — instant clear, no per-row WAL. 3. Use case for DELETE: anywhere you need a WHERE clause, want triggers to fire, or want to ROLLBACK if subsequent steps fail. 4. Mention foreign-key gotchas: TRUNCATE refuses (Postgres without CASCADE) when other tables reference the truncated rows; CASCADE truncates them too, which is rarely what you want.

Edge cases to mention: - Postgres TRUNCATE is transactional and can ROLLBACK; MySQL TRUNCATE on InnoDB is technically a DDL statement and implicitly commits before and after. - TRUNCATE in SQL Server requires ALTER permission, not just DELETE — privilege model differs. - DELETE leaves dead row versions (Postgres) requiring VACUUM; TRUNCATE reclaims space immediately.

What NOT to say: - Do not assume TRUNCATE always resets the auto-increment — in Postgres it depends on **RESTART IDENTITY**; default is **CONTINUE IDENTITY**. - Do not use TRUNCATE inside a routine app transaction in MySQL expecting ROLLBACK to undo it — it commits.

Common follow-up: "You truncate a table that has FKs from another table. What happens?" — Postgres refuses unless you **TRUNCATE ... CASCADE** (which truncates the referencing table too); MySQL/InnoDB also refuses. Generally a sign you should DELETE with conditions instead.

```
-- Postgres: instant wipe of staging
TRUNCATE TABLE staging_orders RESTART IDENTITY;

-- DELETE with condition (the only option when you need WHERE)
DELETE FROM staging_orders WHERE batch_id = 42;
```

Q354 · REPLACE INTO (MySQL) — Why It's Not an UPSERT

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Flipkart · Topic: MERGE / UPSERT

The question (interviewer's verbatim phrasing):

MySQL has REPLACE INTO. Isn't that an upsert? Why do experts warn against using it?

The 30-second answer: REPLACE INTO is **DELETE + INSERT** under the hood when a unique-key conflict is detected — not a true UPDATE. That means triggers fire as deletes-and-inserts, auto-increment values change, foreign keys with ON DELETE CASCADE wipe out child rows, and replication can record extra events. ON DUPLICATE KEY UPDATE is the correct upsert.

Step-by-step thought process: 1. Lead with the mechanism — DELETE + INSERT — because it explains every gotcha. 2. Walk the consequences one by one: trigger semantics differ (BEFORE/AFTER

DELETE then BEFORE/AFTER INSERT instead of BEFORE/AFTER UPDATE); identity columns advance; ON DELETE CASCADE wipes children silently; row-based replication records two events instead of one. 3. Give the scenario: a `users` table with REPLACE INTO and `addresses(user_id) REFERENCES users(id) ON DELETE CASCADE`. Every REPLACE INTO on `users` deletes all addresses. Real production outages have been caused exactly by this. 4. State the recommendation clearly: use `INSERT ... ON DUPLICATE KEY UPDATE` for upsert semantics; reserve REPLACE INTO for the rare case where DELETE+INSERT is genuinely what you want.

Edge cases to mention: - REPLACE INTO requires the same INSERT privilege plus DELETE privilege — different from ON DUPLICATE KEY's required INSERT + UPDATE. - For tables with multiple unique keys, REPLACE may delete multiple existing rows in a single REPLACE — silently — to clear the way for the new one. - Auto-increment value of the new row is fresh, not the deleted row's value — FK references to that ID become invalid.

What NOT to say: - Do not call REPLACE INTO "an upsert" without immediately flagging the DELETE semantics — that's the trap. - Do not assume application code dependent on the same primary-key value after a REPLACE will keep working — it may not, if the AI counter advances.

Common follow-up: "Why does REPLACE INTO still exist if it's dangerous?" — Historical and there are niche cases (e.g. wanting trigger semantics of DELETE+INSERT, or full row replacement when partial UPDATE is wrong). Mostly: legacy; for new code, prefer `INSERT ... ON DUPLICATE KEY UPDATE`.

```
-- DANGEROUS if children depend on this row via ON DELETE CASCADE
REPLACE INTO users (id, name, email) VALUES (42, 'Asha', 'asha@example.com');

-- SAFE upsert
INSERT INTO users (id, name, email) VALUES (42, 'Asha', 'asha@example.com')
ON DUPLICATE KEY UPDATE name = VALUES(name), email = VALUES(email);
```

Q355 · RETURNING Clause — Single-Round-Trip Writes in Postgres / Oracle

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, JP Morgan · Topic: DML

The question (interviewer's verbatim phrasing):

What does the RETURNING clause do? Show me how it saves a round trip in an INSERT, an UPDATE and a DELETE.

The 30-second answer: RETURNING (Postgres and Oracle) appends a SELECT-style projection to INSERT, UPDATE or DELETE, emitting one row per affected row right back from the DML. You avoid the classic INSERT-then-SELECT-curryval or UPDATE-then-SELECT pattern — one statement, one round trip, no race.

Step-by-step thought process: 1. Show INSERT RETURNING: `INSERT INTO orders (user_id, amount) VALUES (42, 199) RETURNING id, created_at` — the server-assigned ID and timestamp come back without a second query. 2. UPDATE RETURNING: useful for state-machine transitions where you want to know which rows actually changed and their new state. `UPDATE orders SET status = 'paid' WHERE id = 1 AND status = 'pending' RETURNING id, status`. 3. DELETE RETURNING: archive-then-delete pattern. `DELETE FROM sessions WHERE expires_at < NOW() RETURNING *` — capture the deleted rows for an audit log in the same statement. 4. Combine with CTEs: `WITH d AS (DELETE FROM sessions WHERE ... RETURNING *) INSERT INTO sessions_archive SELECT * FROM d` — atomic move in one statement.

Edge cases to mention: - RETURNING can reference any column of the affected row, including computed columns and generated identities. - For multi-row INSERT, RETURNING emits one row per inserted row; useful for bulk inserts with server-side ID assignment. - Oracle uses `RETURNING ... INTO` for PL/SQL variables; the pure-SQL form (`RETURNING ... FROM` style emitting a result set) is mostly Postgres territory.

What NOT to say: - Do not insist RETURNING is in every dialect — MySQL has nothing equivalent; SQL Server's analogue is OUTPUT (Q30). - Do not skip `RETURNING *` when you actually need just one column — be explicit; * is wasteful in wire-traffic terms.

Common follow-up: "How do you do this in MySQL?" — Insert and call `LAST_INSERT_ID()` for the new ID; for updates and deletes, there is no native equivalent — you must SELECT before or after, accepting a race window.

```
-- INSERT RETURNING
INSERT INTO orders (user_id, amount) VALUES (42, 199)
RETURNING id, created_at;

-- Archive-then-delete in one shot
WITH gone AS (
  DELETE FROM sessions WHERE expires_at < NOW() RETURNING *
)
INSERT INTO sessions_archive SELECT * FROM gone;
```

Q356 · OUTPUT Clause (SQL Server) — Equivalent of RETURNING

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan, Wells Fargo, AmEx · Topic: DML

The question (interviewer's verbatim phrasing):

SQL Server has OUTPUT instead of RETURNING. How does it compare? What can it do that RETURNING can't?

The 30-second answer: OUTPUT emits one row per affected row from INSERT/UPDATE/DELETE/MERGE, with access to two virtual tables — `inserted` (post-image) and `deleted` (pre-image). For UPDATE, you can return both the old and the new value in the same row — something RETURNING does not natively support.

Step-by-step thought process: 1. Show INSERT OUTPUT: `INSERT INTO orders (user_id, amount) OUTPUT inserted.id, inserted.created_at VALUES (42, 199)`. Note that `inserted` is the only useful pseudo-table for INSERT. 2. Show UPDATE OUTPUT: `UPDATE orders SET status = 'paid' OUTPUT deleted.status AS old_status, inserted.status AS new_status WHERE id = 1`. Captures the change delta atomically. 3. Show DELETE OUTPUT: `DELETE FROM sessions OUTPUT deleted.* WHERE expires_at < GETDATE()` — archive the deleted rows in one round trip. 4. Mention OUTPUT INTO: pipe the OUTPUT into another table directly: `OUTPUT inserted.id, inserted.created_at INTO audit_log(order_id, created_at)` — single-statement audit trail.

Edge cases to mention: - OUTPUT works on MERGE too, with an `$action` column ('INSERT' / 'UPDATE' / 'DELETE') — invaluable for tracking what each branch did. - Triggers and OUTPUT both fire on the same DML; if you have an AFTER trigger that modifies the row further, OUTPUT sees the pre-trigger state for that statement. - Some constraints on OUTPUT clauses with constraints or computed columns — rare in practice but worth checking docs for complex cases.

What NOT to say: - Do not assume `inserted` is available for DELETE or `deleted` for INSERT — each DML exposes only the relevant pseudo-tables. - Do not conflate OUTPUT with triggers — they coexist and have different lifecycles.

Common follow-up: "How do you log a before-and-after diff for every UPDATE?" — `OUTPUT deleted.col_a AS old_a, inserted.col_a AS new_a, ... INTO audit_log` — emits the diff into your audit table directly from the UPDATE statement.

```
-- Capture before/after on update
UPDATE orders
SET status = 'paid'
OUTPUT deleted.status AS old_status, inserted.status AS new_status
WHERE id = 1;

-- Archive deletes inline
DELETE FROM sessions
OUTPUT deleted.* INTO sessions_archive
WHERE expires_at < GETDATE();
```

Chapter 9 — Stored Procedures, Functions & Triggers

If you're targeting GCC roles at JP Morgan India, Wells Fargo, Goldman, AmEx, or any Service-MNC engagement at TCS, Infosys, Wipro, Cognizant, Accenture — especially the ones glued to a legacy Java/Spring or .NET/C# codebase, or to an Oracle/PostgreSQL/SQL Server estate that has been alive since

2008 — procedural SQL is not optional. You will be asked to write a stored procedure in PL/pgSQL, T-SQL or PL/SQL, defend why you'd choose a function over a procedure, debug a misbehaving trigger, and explain whether business logic belongs in the database at all. Modern stacks (Razorpay, Cred, Swiggy, anything Postgres-on-RDS with a Go or Node service in front) lean lighter on procs and heavier on application code — but even those teams ask "when would you reach for a stored procedure?" and they expect a thoughtful answer, not a reflex. This chapter walks through the thirty questions that come up most often, with the dialect call-outs you absolutely need because procedural SQL is the most vendor-divergent corner of the language.

Q357 · Stored Procedure vs Function — The Semantic Difference

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Wells Fargo · Topic: Procedures vs Functions

The question (interviewer's verbatim phrasing):

What is the difference between a stored procedure and a function in SQL? Where would you use one over the other?

The 30-second answer: A function is meant to compute and return a value (scalar or table) and is callable from inside a SELECT or WHERE clause; a procedure is meant to perform an action (DML, multiple statements, transaction control) and is invoked with `CALL` or `EXEC`. Functions typically cannot have side effects on data; procedures usually can. Procedures can manage transactions; functions usually run inside the caller's transaction.

Step-by-step thought process: 1. Lead with the returning-a-value vs performing-an-action framing — that is what the interviewer is testing. 2. Mention callability: functions can sit in a SELECT list (`SELECT calc_gst(amount) FROM invoices`); procedures cannot. 3. Talk about side effects. In Postgres a function can technically do DML, but the convention is "function = pure-ish, procedure = side-effects." In Oracle, functions historically were not allowed to modify data inside a query. 4. Mention transactions — only procedures can run `COMMIT` and `ROLLBACK` inside the body (Postgres 11+, SQL Server T-SQL, Oracle PL/SQL).

Edge cases to mention: - MySQL has both `FUNCTION` and `PROCEDURE`, and functions are restricted by `log_bin_trust_function_creators` for replication safety. - SQL Server scalar UDFs are notorious for forcing row-by-row execution before the 2019 inlining work — covered in a later question. - Postgres blurs the line — a `FUNCTION` can do DML, and pre-11 there was no `PROCEDURE` at all; people wrote everything as functions.

What NOT to say: - Do not say "functions cannot modify data" as an absolute — that depends on the dialect and the volatility tag. - Do not say "procedures are always faster" — that is folklore, not architecture.

Common follow-up: *"Then why does Postgres even have procedures if functions can do everything?"* — Because procedures can issue `COMMIT` / `ROLLBACK` mid-body, which functions cannot. That is the real reason `CREATE PROCEDURE` was added in Postgres 11.

```
-- Postgres: function returns a value, callable in SELECT
CREATE FUNCTION calc_gst(amount NUMERIC) RETURNS NUMERIC
LANGUAGE sql IMMUTABLE AS $$ SELECT amount * 0.18 $$;

-- Postgres: procedure performs an action, called with CALL
CREATE PROCEDURE settle_invoice(p_invoice_id BIGINT)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE invoices SET status = 'SETTLED' WHERE id = p_invoice_id;
    COMMIT;
END $$;

CALL settle_invoice(101);
```

Q358 · When Procedural SQL Is the Right Call (Modern Trade-offs)

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, JP Morgan, Walmart Labs · Topic: Architecture

The question (interviewer's verbatim phrasing):

Modern teams say "keep logic in the application." When would you still write a stored procedure in 2026?

The 30-second answer: When you need to minimise network round-trips for a chatty, multi-statement workflow; when you need a single transactional boundary that the application cannot easily provide; when multiple consumer apps must share the exact same logic and you cannot host a shared service; or when row-level performance matters and shipping the data to the app is the bottleneck.

Step-by-step thought process: 1. Acknowledge the modern bias — Razorpay, Swiggy, Cred-style stacks push logic into Go/Node/Java services because procs are hard to test, hard to version, and hard to scale horizontally. 2. List the legitimate use cases: complex bulk-update jobs (`SETTLE_PAYMENTS_FOR_DAY`), reconciliation, end-of-day batch, outbox writers, audit trails, anything where shipping 5 lakh rows to the app and back is wasteful. 3. Mention the GCC reality: banks at JP Morgan India, Wells Fargo, Goldman often have decades-old Oracle PL/SQL packages — you will be asked to extend them, not replace them. 4. Be ready to defend "I'd put this in the app, not in a proc" — interviewers respect a reasoned no.

Edge cases to mention: - Procedures defeat horizontal scaling — the DB becomes the bottleneck. A Go service can scale to 50 pods; the proc cannot. - ORM frameworks (JPA, Entity Framework, GORM) can call procs but cannot easily diff/version them — drift is a real risk. - Cloud-managed Postgres on RDS or Aurora has limits on `LANGUAGE plperl`, `plpython3u` etc — you may not get the extension you want.

What NOT to say: - Do not say "always use procs because they are faster" — that is 2005 thinking. - Do not say "never use procs" — that disqualifies you from any bank or GCC role.

Common follow-up: *"Have you ever moved logic out of a stored procedure into application code? Why?"* — Be ready with a real story: testability, observability, deploy independence, or freeing the DB CPU.

```
-- Reasonable proc use: bulk settlement that touches 1M rows per night
CREATE PROCEDURE settle_eod_payouts(p_run_date DATE)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE payouts SET status='SETTLED', settled_at=now()
    WHERE status='PENDING' AND run_date = p_run_date;
END $$;
```

Q359 · PL/pgSQL Basics — Block Structure

Tags: Difficulty: Easy · Asked at: Flipkart, Meesho, Cred, Zerodha · Topic: PL/pgSQL

The question (interviewer's verbatim phrasing):

Write a basic PL/pgSQL function that takes a customer id and returns their total order amount. Walk me through the block structure.

The 30-second answer: PL/pgSQL functions live inside `$$ ... $$` dollar-quoted blocks. The body has an optional `DECLARE` for local variables, a mandatory `BEGIN ... END` for the executable section, and an optional `EXCEPTION` block for error handling. You write `RETURN <value>` to send a scalar back.

Step-by-step thought process: 1. Show the skeleton — `CREATE FUNCTION ... RETURNS ... LANGUAGE plpgsql AS $$ DECLARE ... BEGIN ... END $$;` 2. Explain the use of `SELECT ... INTO` for assigning query results to variables — this is PL/pgSQL-specific syntax. 3. Mention `RAISE NOTICE` for debug output and `RAISE EXCEPTION` for errors. 4. Call out that `LANGUAGE sql` is a different beast — no `DECLARE`, no procedural control flow, just a single SQL expression.

Edge cases to mention: - `SELECT INTO STRICT` raises an exception if zero or more than one row is returned — useful for invariants. - Variables shadowing column names is a classic bug. Prefix locals with `v_` or `_` by convention. - Postgres re-parses the body on every fresh session for the first call — be aware on connection-poolers.

What NOT to say: - Do not call dollar-quoting "string escaping" — it is a literal delimiter that avoids quote-escaping headaches. - Do not assume PL/pgSQL is the only language available — Postgres supports `plpython3u`, `plperl`, `pltcl` too (often disabled on RDS).

Common follow-up: "What happens if the customer does not exist?" — Without `STRICT`, the variable becomes NULL and the function returns NULL. With `STRICT`, you get a `NO_DATA_FOUND` exception you can trap.

```
CREATE OR REPLACE FUNCTION customer_total(p_customer_id BIGINT)
RETURNS NUMERIC
LANGUAGE plpgsql AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COALESCE(SUM(amount), 0) INTO v_total
    FROM orders WHERE customer_id = p_customer_id;
    RETURN v_total;
END $$;
```

Q360 · T-SQL Basics — Stored Procedure on SQL Server

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Accenture, Wells Fargo · Topic: T-SQL

The question (interviewer's verbatim phrasing):

Write a T-SQL stored procedure that takes an order id, returns the order header and the line items as two result sets.

The 30-second answer: Use `CREATE OR ALTER PROCEDURE`, declare parameters with `@` prefix, write each SELECT as a separate statement — SQL Server happily returns multiple result sets to the client. Use `SET NOCOUNT ON` at the top to suppress row-count chatter.

Step-by-step thought process: 1. Show the skeleton: `CREATE OR ALTER PROCEDURE dbo.get_order @order_id INT AS BEGIN ... END`. 2. Mention `SET NOCOUNT ON` — without it, the `(1 row affected)` messages are sent to the client and can confuse ORMs like Entity Framework and Dapper. 3. Two SELECTs return two result sets — drivers like ADO.NET's `SqlDataReader.NextResult()` walk them. 4. Talk about parameter direction — defaulting to IN; OUTPUT params are declared `@name INT OUTPUT`.

Edge cases to mention: - `CREATE OR ALTER` requires SQL Server 2016 SP1 or newer; older versions force a `DROP IF EXISTS` + `CREATE` pattern. - `SET XACT_ABORT ON` is critical when you want a runtime error to roll back the whole transaction — by default, T-SQL is lenient. - Returning more than one result set breaks some ORMs that map to a single entity — coordinate with the application team.

What NOT to say: - Do not use `CREATE PROCEDURE` without `OR ALTER` on modern SQL Server — you'll annoy reviewers. - Do not forget `SET NOCOUNT ON` — that is a giveaway you have not written T-SQL in production.

Common follow-up: "How does the .NET application read both result sets?" — `using var rdr = cmd.ExecuteReader(); rdr.Read(); ...; rdr.NextResult(); rdr.Read(); ...`

```
CREATE OR ALTER PROCEDURE dbo.get_order
    @order_id INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT id, customer_id, total_amount, status FROM dbo.orders WHERE id = @order_id;
    SELECT id, order_id, sku, qty, unit_price FROM dbo.order_lines WHERE order_id = @order_id;
END
```

Q361 · PL/SQL Basics — Oracle Package Structure (Light Touch)

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, JP Morgan, Wells Fargo · Topic: PL/SQL

The question (interviewer's verbatim phrasing):

If you walked into an Oracle shop tomorrow, what is the first thing you'd notice about how procs and functions are organised compared to Postgres or SQL Server?

The 30-second answer: Oracle organises related procedures and functions into `PACKAGE` (the spec) and `PACKAGE BODY` (the implementation) — a namespace plus a public/private boundary in one. Postgres and SQL Server have no such concept; you get loose functions and procedures with at best a schema as the only grouping.

Step-by-step thought process: 1. Lead with the package concept — it is what an Oracle dev cares about most. 2. The spec declares what is callable from outside; the body holds the implementation plus private helpers. 3. Mention that compiling the spec invalidates dependants, but compiling only the body does not — this matters for deploys. 4. Acknowledge you only need a light touch unless the role is explicitly Oracle PL/SQL — but knowing packages exist is enough to pass a screening round.

Edge cases to mention: - `PRAGMA AUTONOMOUS_TRANSACTION` lets a sub-procedure commit independently of the caller — useful for audit logs that must persist even on rollback. - Package-level variables persist for the duration of the session — a footgun if you forget. - Oracle's compilation model is strict — a broken body marks the package `INVALID` and callers fail until it is recompiled.

What NOT to say: - Do not claim Postgres has packages — schemas are not the same thing. - Do not over-claim PL/SQL experience if you have only read about it. GCC interviewers spot this fast.

Common follow-up: "What is `PRAGMA AUTONOMOUS_TRANSACTION` and when have you used it?" — Audit logging that must survive a parent rollback, or rate-limit counters that must commit immediately.

```
-- Oracle package spec
CREATE OR REPLACE PACKAGE pkg_orders AS
  FUNCTION get_total(p_order_id NUMBER) RETURN NUMBER;
  PROCEDURE cancel_order(p_order_id NUMBER);
END pkg_orders;
/

-- Oracle package body
CREATE OR REPLACE PACKAGE BODY pkg_orders AS
  FUNCTION get_total(p_order_id NUMBER) RETURN NUMBER IS
    v_total NUMBER;
  BEGIN
    SELECT total_amount INTO v_total FROM orders WHERE id = p_order_id;
    RETURN v_total;
  END;
  PROCEDURE cancel_order(p_order_id NUMBER) IS
  BEGIN
    UPDATE orders SET status='CANCELLED' WHERE id = p_order_id;
  END;
END pkg_orders;
/
```

Q362 · MySQL Stored Procedure Syntax and DELIMITER

Tags: Difficulty: Easy · Asked at: Flipkart, PhonePe, Meesho, Zerodha · Topic: MySQL

The question (interviewer's verbatim phrasing):

Why do MySQL stored procedure scripts almost always start with `DELIMITER //` ? What does that actually do?

The 30-second answer: The MySQL client uses `;` to separate statements. A procedure body contains semicolons inside it, which would otherwise be interpreted as end-of-statement and break the parser.

`DELIMITER //` tells the client "use `//` as the statement terminator instead," so the entire `CREATE PROCEDURE ... END` is treated as one statement. Reset it to `;` afterwards.

Step-by-step thought process: 1. Frame it as a client-side parsing issue, not a server-side language feature. 2. Show the canonical pattern: `DELIMITER //` ... `CREATE PROCEDURE foo() BEGIN ... END //` ... `DELIMITER ;`. 3. Mention that some clients (MySQL Workbench, DBeaver) abstract this away; the CLI `mysql` and most CI pipelines do not. 4. Be ready for the question about parameter directions — MySQL supports `IN`, `OUT`, `INOUT`.

Edge cases to mention: - The delimiter is a per-client convention; the server has no idea what `//` is. It just sees a single string. - If you forget to reset to `;`, every subsequent statement in the script confuses the client. - Migration tools like Liquibase or Flyway handle this for you with `delimiter::END` markers — read their docs.

What NOT to say: - Do not say "DELIMITER is part of the SQL standard" — it is a MySQL CLI convention. - Do not paste a multi-line procedure into the `mysql` shell without delimiter handling — it will fail in spectacular ways.

Common follow-up: "What's the equivalent in Postgres?" — Dollar-quoting (`$$`) plays the same role; it lets you put semicolons inside the body without confusing the parser.

```
DELIMITER //
```

```
CREATE PROCEDURE refund_order(IN p_order_id INT, OUT p_refund_amount DECIMAL(10,2))
BEGIN
    SELECT total_amount INTO p_refund_amount FROM orders WHERE id = p_order_id;
    UPDATE orders SET status='REFUNDED' WHERE id = p_order_id;
END //
```

```
DELIMITER ;
```

Q363 · IN, OUT and INOUT Parameters

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, AmEx · Topic: Parameters

The question (interviewer's verbatim phrasing):

Explain IN, OUT and INOUT parameters with a real example. Which databases support each?

The 30-second answer: `IN` is the default — caller passes a value, callee reads it. `OUT` — callee writes a value, caller reads it after the call. `INOUT` — both directions. PL/SQL, MySQL and PL/pgSQL (in procedures since 14) all support the three modes; SQL Server uses `OUTPUT` keyword instead of `OUT` / `INOUT` and a single parameter declared `OUTPUT` can be read on both sides.

Step-by-step thought process: 1. Define the three modes plainly — interviewers test vocabulary first. 2. Give an example: a settlement procedure that takes the order id as **IN** and returns the refund amount as **OUT**. 3. Mention that returning data via OUT params is older school; returning a result set (a SELECT inside the procedure) is more modern and ORM-friendly. 4. Be ready for "Why not just RETURN it like a function?" — procedures may need to return multiple values plus a result set; OUT params let you do both.

Edge cases to mention: - In SQL Server T-SQL, the caller must include **OUTPUT** in the **EXEC** call too: **EXEC dbo.foo @x OUTPUT;** . - In Postgres functions (not procedures) before 14, OUT parameters were essentially a way to define a composite return type. - Java JDBC **CallableStatement** requires you to **registerOutParameter** before executing — easy to forget.

What NOT to say: - Do not say "OUT params are faster than returning a result set" — they are different tools, not a perf shootout. - Do not confuse **INOUT** with a global variable — it is still scoped to the call.

Common follow-up: "How does a JDBC client read an OUT parameter?" —

```
cs.registerOutParameter(2, Types.NUMERIC); cs.execute(); BigDecimal x = cs.getBigDecimal(2); .
```

```
-- Postgres procedure with INOUT
CREATE PROCEDURE adjust_balance(INOUT p_balance NUMERIC, p_delta NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    p_balance := p_balance + p_delta;
END $$;

DO $$ DECLARE v_bal NUMERIC := 1000;
BEGIN
    CALL adjust_balance(v_bal, 250);
    RAISE NOTICE 'New balance: %', v_bal; -- 1250
END $$;
```

Q364 · Returning a Result Set vs a Scalar Value

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Return Types

The question (interviewer's verbatim phrasing):

When would you have a stored procedure return a single scalar versus a full result set? How does the application read each?

The 30-second answer: Return a scalar (via RETURN in a function, or via OUT/RETURN code in a proc) when the caller needs a single computed value — a count, a status, a derived id. Return a result set when the caller needs rows — a list of orders, a paginated query, a dashboard panel. Result sets are read with cursors on the client (`ResultSet` / `SqlDataReader` / `pg_cursor`); scalars are read directly.

Step-by-step thought process: 1. Pose it as an API design question, not just a syntax question. 2. Scalars are simpler and cache-friendly — ORMs map them cleanly to a single field. 3. Result sets are the natural shape when the procedure is essentially a SELECT with parameters — common in report and dashboard scenarios. 4. Mention that some teams prefer to wrap everything in procs (banking, fintech back-office) for an audit boundary; others prefer plain SELECT calls from the app.

Edge cases to mention: - SQL Server procs returning multiple result sets confuse strict ORMs — you may need a multi-mapping layer or move to a function/TVF. - Postgres `RETURNS TABLE` is essentially "function returns a result set" — it streams rows back, the caller iterates. - Oracle uses `SYS_REFCURSOR` as the canonical way for a procedure to return a result set to a client.

What NOT to say: - Do not claim "you cannot return a result set from a stored procedure" — every major dialect supports it, just differently. - Do not return a giant result set when only a scalar is needed — that is a wasted round-trip.

Common follow-up: "Show me how Oracle returns a result set from a procedure." — `OUT p_cur SYS_REFCURSOR` + `OPEN p_cur FOR SELECT ...`.

```
-- SQL Server: returns one row
CREATE OR ALTER PROCEDURE dbo.get_order_count
    @customer_id INT, @cnt INT OUTPUT
AS BEGIN
    SET NOCOUNT ON;
    SELECT @cnt = COUNT(*) FROM dbo.orders WHERE customer_id = @customer_id;
END
```

Q365 · Table-Valued Functions — Inline vs Multi-Statement (SQL Server)

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, JP Morgan · Topic: T-SQL TVF

The question (interviewer's verbatim phrasing):

Explain the difference between an inline table-valued function and a multi-statement table-valued function in SQL Server. Which performs better and why?

The 30-second answer: An inline TVF is essentially a parameterised view — its body is a single SELECT, and the optimizer can inline (merge) it into the calling query for accurate cardinality estimates. A multi-statement TVF declares a return table variable and `INSERT` s into it, then returns it; the optimizer treats

it as a black box with a hard-coded row estimate (1 row, or 100, depending on version), so plans degrade badly.

Step-by-step thought process: 1. Lead with the cardinality estimate point — that is the real performance reason. 2. Inline TVF: `RETURNS TABLE AS RETURN (SELECT ...)` — no DECLARE, no BEGIN/END. 3. Multi-statement TVF: `RETURNS @t TABLE(...) AS BEGIN INSERT INTO @t ... RETURN END`. 4. Recommend the inline form whenever the logic fits a single SELECT — almost always faster.

Edge cases to mention: - SQL Server 2017 introduced interleaved execution that helps multi-statement TVF estimates somewhat, but it does not fully close the gap. - 2019's scalar-UDF inlining is a separate feature from TVF inlining — do not conflate them. - Inline TVFs cannot have control flow (IF/CASE outside SELECT, loops) — if you need that, you are stuck with multi-statement or a refactor.

What NOT to say: - Do not claim "both perform the same" — multi-statement TVFs are a well-known performance trap. - Do not use a multi-statement TVF when an inline one would do — that is a code-smell.

Common follow-up: "What is the row estimate for a multi-statement TVF in older SQL Server versions?" — 1 row before 2014; 100 rows from 2014 onwards (cardinality estimator change).

```
-- Inline TVF — parameterised view
CREATE OR ALTER FUNCTION dbo.orders_by_customer(@cust_id INT)
RETURNS TABLE AS RETURN (
    SELECT id, total_amount, created_at FROM dbo.orders WHERE customer_id = @cust_id
);

-- Usage
SELECT * FROM dbo.orders_by_customer(101);
```

Q366 · Set-Returning Functions in Postgres — RETURNS TABLE / SETOF

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Meesho · Topic: PL/pgSQL

The question (interviewer's verbatim phrasing):

Write a Postgres function that returns a list of orders for a given customer. What is the difference between RETURNS SETOF and RETURNS TABLE?

The 30-second answer: `RETURNS SETOF orders` returns a set of rows of the existing `orders` table type. `RETURNS TABLE (id BIGINT, amount NUMERIC, ...)` defines an ad-hoc row type inline. Functionally equivalent; the TABLE form is preferred when you want to project specific columns or compose them from multiple tables.

Step-by-step thought process: 1. Show both signatures side by side — interviewers want to see you have used both. 2. Use `RETURN QUERY` inside the body to stream rows back from a SELECT. 3.

Mention that the function is then callable like a table: `SELECT * FROM`

`orders_for_customer(101);` — that is the SRF idiom. 4. Note that `LANGUAGE sql` is often a better choice than `LANGUAGE plpgsql` for these (covered next).

Edge cases to mention: - `RETURN QUERY` does not actually return at that point — execution continues; `RETURN` (no QUERY) ends the function. - If you forget `RETURN QUERY` and just write a SELECT, the rows are discarded silently. - The output column names from `RETURNS TABLE(...)` are visible inside the function body as if they were local variables — collisions cause bugs.

What NOT to say: - Do not claim Postgres has TVFs in the SQL Server sense — Postgres calls them SRFs (set-returning functions), and the semantics are slightly different. - Do not return a giant result set when a streaming cursor would be better.

Common follow-up: "What if the function needs to compute totals, not just return rows?" — Use `RETURN QUERY SELECT ..., SUM(...) OVER (...) FROM ...` — window functions inside the SRF.

```
CREATE OR REPLACE FUNCTION orders_for_customer(p_cust_id BIGINT)
RETURNS TABLE (id BIGINT, amount NUMERIC, status TEXT)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT o.id, o.total_amount, o.status
    FROM orders o WHERE o.customer_id = p_cust_id
    ORDER BY o.created_at DESC;
END $$;

SELECT * FROM orders_for_customer(101);
```

Q367 · LANGUAGE sql vs LANGUAGE plpgsql in Postgres

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Goldman · Topic: PL/pgSQL

The question (interviewer's verbatim phrasing):

In Postgres, when would you use `LANGUAGE sql` and when would you use `LANGUAGE plpgsql` ? Is there a performance difference?

The 30-second answer: Use `LANGUAGE sql` when the function body is a single SQL expression or query — it is simpler, the planner can inline it into the calling query, and there is no PL/pgSQL interpreter

overhead. Use `LANGUAGE plpgsql` when you need procedural control flow — variables, IF, LOOP, EXCEPTION handlers. SQL functions are typically faster for simple cases because of inlining.

Step-by-step thought process: 1. Frame it as "SQL = declarative single expression, PL/pgSQL = procedural block with control flow." 2. Talk about inlining: a simple `LANGUAGE sql` function used in a WHERE clause can be replaced by its body during planning, so the planner can use indexes correctly. PL/pgSQL functions are usually treated as opaque. 3. Mention that PL/pgSQL has higher first-call overhead because it parses and plans the body lazily and caches it per session. 4. Be ready for: "If they are otherwise equivalent, default to SQL" — that is the conventional wisdom.

Edge cases to mention: - `LANGUAGE sql` cannot use `RAISE`, IF/THEN, or EXCEPTION — once you need any of those, you must move to PL/pgSQL. - Inlining requires the function to be marked appropriately (volatility, strictness) — see the next question. - Postgres 14+ supports SQL-standard `BEGIN ATOMIC` for SQL functions, which improves dependency tracking — older versions used the string-body form.

What NOT to say: - Do not default to PL/pgSQL for everything — you lose inlining and add overhead. - Do not claim SQL functions are always faster — for complex bodies with lots of control flow, PL/pgSQL is the only option.

Common follow-up: "Can a SQL function call another function?" — Yes, just like any SELECT can. And the inner one can also be inlined if it qualifies.

```
-- SQL function - inlinable
CREATE FUNCTION gst_amount(amount NUMERIC) RETURNS NUMERIC
LANGUAGE sql IMMUTABLE AS $$ SELECT amount * 0.18 $$;

-- PL/pgSQL function - has control flow
CREATE FUNCTION grade_score(s INT) RETURNS TEXT
LANGUAGE plpgsql IMMUTABLE AS $$
BEGIN
    IF s >= 90 THEN RETURN 'A';
    ELSIF s >= 75 THEN RETURN 'B';
    ELSE RETURN 'C';
    END IF;
END $$;
```

Q368 · IMMUTABLE, STABLE, VOLATILE — Postgres Function Volatility Tags

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, Goldman · Topic: PL/pgSQL

The question (interviewer's verbatim phrasing):

What do IMMUTABLE, STABLE and VOLATILE mean on a Postgres function? Why does it matter for indexing?

The 30-second answer: **IMMUTABLE** means the function always returns the same output for the same input and reads no database state — the planner can evaluate it once, cache results, and use it in expression indexes. **STABLE** means it returns the same value within a single statement but may depend on database state; usable in indexes for the duration of a scan. **VOLATILE** (default) means anything goes — the planner cannot optimise around it.

Step-by-step thought process: 1. Define each tag clearly — order matters: IMMUTABLE → STABLE → VOLATILE, weakest constraints last. 2. Give examples: `lower(text)` is IMMUTABLE; `now()` is STABLE; `random()` is VOLATILE. 3. The killer use case: only IMMUTABLE functions can appear in expression indexes (`CREATE INDEX ON users (lower(email))`). 4. Mention that mis-tagging a function as IMMUTABLE when it actually reads tables causes silent wrong results when indexed.

Edge cases to mention: - A function marked IMMUTABLE that does a SELECT from a table is a foot-gun: the planner may use cached results that no longer reflect the table. - **STABLE** is enough for partial-index predicates and for functions used in **WHERE** that the planner needs to push down. - Default is VOLATILE — be explicit; reviewers notice.

What NOT to say: - Do not mark a function IMMUTABLE just to "make it faster" — correctness first. - Do not confuse the SQL standard **DETERMINISTIC** (MySQL) with Postgres IMMUTABLE — the spirit is similar but the rules differ.

Common follow-up: "What happens if you index on a STABLE function?" — Postgres refuses; expression indexes require IMMUTABLE.

```
-- IMMUTABLE – eligible for expression indexes
CREATE FUNCTION normalise_email(t TEXT) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$ SELECT lower(trim(t)) $$;

CREATE INDEX idx_users_email_norm ON users (normalise_email(email));
```

Q369 · Functions in WHERE Clause — Index Implications

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Razorpay, Cred · Topic: Indexes

The question (interviewer's verbatim phrasing):

Why does wrapping a column in a function inside a WHERE clause often kill the index? What can you do about it?

The 30-second answer: The planner needs to compare the indexed column directly. Wrapping it in a function (`WHERE lower(email) = 'a@b.com'`) means the index on `email` no longer matches the predicate. The fix is to create a matching expression index (`CREATE INDEX ON t (lower(email))`) or rewrite the query so the column is bare on the left side.

Step-by-step thought process: 1. State the rule: "function on column = no index seek." 2. Give the classic example — `WHERE date_trunc('day', created_at) = '2026-05-26'` — and the fix `WHERE created_at >= '2026-05-26' AND created_at < '2026-05-27'`. 3. The other fix: expression index on `date_trunc('day', created_at)`. The function must be IMMUTABLE (or at least STABLE for some uses). 4. Mention SARGable predicates as the underlying concept — even if the interviewer does not use the term, you should.

Edge cases to mention: - Implicit conversion is the same trap: `WHERE numeric_col = '123'` may cast every row. - SQL Server is stricter than Postgres about needing the type to match exactly to use an index. - In Oracle, function-based indexes are the equivalent — same idea, same syntax around `CREATE INDEX ... ON t (func(col))`.

What NOT to say: - Do not say "the index just is not used, no big deal" — full table scans on a 10-crore-row table are a big deal. - Do not claim the database "decides" not to use it for fun — the predicate genuinely is not seekable.

Common follow-up: "How do you spot this in an EXPLAIN plan?" — Look for `Seq Scan` (Postgres) or `Table Scan` / `Index Scan with filter` (SQL Server) on the table where you expected an `Index Seek`.

```
-- Bad: function on column kills the index
SELECT * FROM users WHERE lower(email) = 'a@b.com';

-- Fix A: expression index
CREATE INDEX idx_users_email_lower ON users (lower(email));

-- Fix B: store normalised value at write time and index it
```

Q370 · Function Inlining — Postgres SQL and SQL Server Scalar UDF (2019+)

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, JP Morgan · Topic: Optimization

The question (interviewer's verbatim phrasing):

Explain function inlining in Postgres and the scalar UDF inlining work in SQL Server 2019. Why does it matter?

The 30-second answer: Inlining means the planner replaces a function call with its body inside the calling query, so it can be optimised as one unit — accurate cardinality, index use, parallelism. Postgres has done this for simple `LANGUAGE sql` functions for years (a feature called "SQL function inlining"). SQL Server 2019 introduced "Scalar UDF Inlining" (project Froid) that does the same for scalar T-SQL UDFs that were historically a perf disaster.

Step-by-step thought process: 1. Lead with the perf story — un-inlined scalar UDFs forced row-by-row execution in SQL Server and crippled queries. 2. Postgres inlining conditions: `LANGUAGE sql`, single SELECT body, no DML, volatility compatible with the call site. 3. SQL Server 2019 conditions: scalar UDF, no time-dependent intrinsic, no aggregate, set compatibility level 150+. 4. Mention that inlining can be disabled per-query (`OPTION (USE HINT('DISABLE_TSSQL_SCALAR_UDF_INLINING'))`) if a plan regresses.

Edge cases to mention: - SQL Server 2022 expanded the conditions further — fewer cases blocked. - Postgres SQL inlining is invisible in `EXPLAIN`: the body is folded in before planning, so the function name disappears. - Disabling inlining via `OPTION (RECOMPILE)` or hints is a real release-time tool when plans regress on upgrade.

What NOT to say: - Do not say "stored procedures are inlined" — they are not. Only functions, and only under specific conditions. - Do not assume your perf problem is solved by upgrading SQL Server — test the specific function.

Common follow-up: "What did Microsoft call the SQL Server 2019 inlining project?" — Project Froid. Bonus points for naming it.

```
-- SQL Server 2019+: scalar UDF that may be inlined
CREATE OR ALTER FUNCTION dbo.gst_amount(@amount DECIMAL(10,2))
RETURNS DECIMAL(10,2)
WITH INLINE = ON -- explicit opt-in/out
AS BEGIN RETURN @amount * 0.18 END
```

Q371 · Error Handling — TRY/CATCH, EXCEPTION, DECLARE HANDLER

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Wells Fargo, AmEx · Topic: Error Handling

The question (interviewer's verbatim phrasing):

Walk me through error handling inside a stored procedure in SQL Server, Postgres and MySQL.
What is the equivalent in each?

The 30-second answer: SQL Server uses `BEGIN TRY ... END TRY BEGIN CATCH ... END CATCH` with `ERROR_MESSAGE()` / `ERROR_NUMBER()` inside the CATCH. Postgres uses `BEGIN ... EXCEPTION WHEN <condition> THEN ... END` inside a PL/pgSQL block. MySQL uses `DECLARE ... HANDLER FOR <condition> <statement>` declared at the top of the block. Oracle is similar to Postgres with `EXCEPTION WHEN OTHERS THEN ...`.

Step-by-step thought process: 1. Show the three syntaxes side by side — that is the recall test. 2. Mention `RAISERROR` / `THROW` (SQL Server), `RAISE EXCEPTION` (Postgres), `SIGNAL SQLSTATE` (MySQL) for raising your own errors. 3. Always combine TRY/CATCH with transaction control — log the error, then ROLLBACK and rethrow. 4. Be ready for "What happens if you do not handle an error?" — the transaction state in SQL Server is determined by `XACT_ABORT`; in Postgres the whole transaction aborts and subsequent commands fail until ROLLBACK.

Edge cases to mention: - In Postgres, an unhandled exception inside a PL/pgSQL block aborts only the current sub-transaction (savepoint); the outer transaction can be salvaged with `EXCEPTION WHEN OTHERS`. - SQL Server's `TRY/CATCH` does not catch compile-time errors — only runtime. - MySQL handlers come in `CONTINUE`, `EXIT` and `UNDO` flavours; most production code uses `EXIT`.

What NOT to say: - Do not swallow errors with an empty CATCH/EXCEPTION block — that hides bugs. - Do not use `RAISERROR` (with E) in new SQL Server code — use `THROW`.

Common follow-up: "In Postgres, what is the difference between `RAISE NOTICE` and `RAISE EXCEPTION`?" — `NOTICE` writes a message and continues; `EXCEPTION` aborts the block and propagates.

```
-- SQL Server
BEGIN TRY
    UPDATE accounts SET balance = balance - 500 WHERE id = 1;
END TRY
BEGIN CATCH
    THROW;
END CATCH;

-- Postgres
BEGIN
    UPDATE accounts SET balance = balance - 500 WHERE id = 1;
EXCEPTION WHEN check_violation THEN
    RAISE NOTICE 'check failed: %', SQLERRM;
    ROLLBACK;
END;
```


Q372 · Transaction Management Inside Procedures

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, JP Morgan, Wells Fargo · Topic: Transactions

The question (interviewer's verbatim phrasing):

Can a stored procedure manage its own transactions? What about a function?

The 30-second answer: Procedures can manage transactions in T-SQL (`BEGIN TRAN/COMMIT/ROLLBACK`), in PL/SQL (`COMMIT / ROLLBACK` directly), in MySQL (with the usual caveats), and in Postgres 11+ procedures. Functions in most dialects cannot — they run inside the caller's transaction and cannot issue COMMIT/ROLLBACK. That is the single biggest reason Postgres added procedures in version 11.

Step-by-step thought process: 1. Lead with the rule: procedures yes, functions usually no. 2. Discuss the autonomous transaction pattern — when a sub-operation must commit even if the caller rolls back. Oracle has `PRAGMA AUTONOMOUS_TRANSACTION` ; Postgres uses `dblink` or background workers; SQL Server uses linked-server tricks. None are pretty. 3. Mention SAVEPOINTS — most dialects support them for nested partial rollback within a single transaction. 4. Be ready for "How do you implement an audit log that survives rollback?" — separate connection, autonomous transaction, or queue-based pattern.

Edge cases to mention: - In Postgres procedures, you can only `COMMIT` if there is no open cursor and no transaction-bound state — otherwise you get an error. - SQL Server has `@@TRANCOUNT` — nested `BEGIN TRAN` increments, but only the outermost `COMMIT` actually commits. - MySQL DDL inside a procedure implicitly commits — a common surprise.

What NOT to say: - Do not assume `BEGIN TRAN` in a procedure starts a fresh transaction if one already exists — it nests. - Do not assume `ROLLBACK` inside a procedure rolls back only the procedure's work — it rolls back to the outermost savepoint by default.

Common follow-up: "What is `@@TRANCOUNT` and why do batch jobs check it before COMMIT?" — Defensive coding: do not COMMIT if you are nested inside a larger caller transaction.

```
-- Postgres 11+ procedure with explicit transaction control
CREATE PROCEDURE settle_batch(p_run_date DATE)
LANGUAGE plpgsql AS $$
DECLARE r RECORD;
BEGIN
  FOR r IN SELECT id FROM payouts WHERE run_date = p_run_date AND status = 'PENDING'
  LOOP
    UPDATE payouts SET status='SETTLED' WHERE id = r.id;
    IF mod(r.id, 1000) = 0 THEN COMMIT; END IF;
  END LOOP;
  COMMIT;
END $$;
```

Q373 · Cursors — How, When, Why Mostly Avoid

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, AmEx, JP Morgan · Topic: Cursors

The question (interviewer's verbatim phrasing):

Explain cursors. When would you use one and why are they generally discouraged?

The 30-second answer: A cursor lets you walk through a result set row by row inside a stored procedure — `DECLARE cur CURSOR FOR ...`, `OPEN`, `FETCH NEXT`, `CLOSE`, `DEALLOCATE`. They are discouraged because they force row-at-a-time processing which is dramatically slower than a single set-based UPDATE or INSERT...SELECT. Use them only when the logic genuinely cannot be expressed as a single statement — typically when each row depends on a side effect of the previous one.

Step-by-step thought process: 1. Show the canonical SQL Server cursor: DECLARE, OPEN, FETCH NEXT INTO, WHILE @@FETCH_STATUS = 0. 2. The killer point: a single `UPDATE ... FROM` is usually 10-100x faster than a cursor doing the same work. 3. Legitimate cursor uses: complex per-row side effects (calling external systems, generating non-deterministic ids, audit trails with branching), or batched processing where you commit every N rows. 4. Mention that Postgres has `FOR ... IN ... LOOP` which is essentially a hidden cursor — same performance trade-off applies.

Edge cases to mention: - SQL Server cursors have many options (`FORWARD_ONLY`, `STATIC`, `DYNAMIC`, `LOCAL` / `GLOBAL`) — `LOCAL FAST_FORWARD` is the cheapest. - Forgetting `CLOSE` and `DEALLOCATE` leaks resources for the session. - Postgres `REFCURSOR` is a different thing — it is a way to pass a query handle back to a client, not a row-walker.

What NOT to say: - Do not write a cursor when a single set-based statement does the job — that is the most common interview red flag. - Do not claim cursors are "always bad" — there are real cases.

Common follow-up: "Show me how you would rewrite this cursor as a single *UPDATE*." — Practice this; it is a 2-minute litmus test for whether you actually think in sets.

```
-- SQL Server cursor (avoid when set-based works)
DECLARE @id INT, @amt DECIMAL(10,2);
DECLARE cur CURSOR LOCAL FAST_FORWARD FOR
    SELECT id, amount FROM payouts WHERE status='PENDING';
OPEN cur;
FETCH NEXT FROM cur INTO @id, @amt;
WHILE @@FETCH_STATUS = 0
BEGIN
    EXEC dbo.notify_external_system @id, @amt; -- legitimate side effect
    FETCH NEXT FROM cur INTO @id, @amt;
END
CLOSE cur; DEALLOCATE cur;
```

Q374 · Trigger Basics — BEFORE vs AFTER, ROW vs STATEMENT

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Accenture, Wells Fargo · Topic: Triggers

The question (interviewer's verbatim phrasing):

Explain BEFORE vs AFTER triggers, and FOR EACH ROW vs FOR EACH STATEMENT triggers.

The 30-second answer: A **BEFORE** trigger fires before the row is written and can modify the row being inserted/updated (**NEW.col := ...**). An **AFTER** trigger fires after the write and is used for side effects (audit, derived inserts). **FOR EACH ROW** fires once per affected row and exposes **NEW / OLD** ; **FOR EACH STATEMENT** fires once per statement regardless of row count and is cheaper for bulk operations.

Step-by-step thought process: 1. State the four-quadrant model: {BEFORE, AFTER} × {ROW, STATEMENT}. 2. BEFORE ROW is where you mutate NEW to enforce derived columns. AFTER ROW is where you log to an audit table. 3. STATEMENT triggers are best when you only need to react to "something changed" rather than per-row data — e.g., refresh a materialised view, invalidate a cache. 4. Note that SQL Server does not have BEFORE/AFTER in the standard sense — it has **INSTEAD OF** and **AFTER** (FOR/AFTER). Postgres and Oracle have BEFORE/AFTER/INSTEAD OF.

Edge cases to mention: - Postgres **AFTER ... STATEMENT** triggers can use transition tables (**REFERENCING NEW TABLE AS ...**) to see the full delta in one place — efficient for bulk audit. - BEFORE ROW triggers run for every row even on bulk inserts — expensive on large COPY/INSERT-SELECT. - Order of multiple triggers on the same table is alphabetical by name in Postgres — easy to forget.

What NOT to say: - Do not say "BEFORE triggers prevent inserts" without nuance — they prevent only by raising an exception or returning NULL. - Do not assume statement-level triggers do not see the data — they can, via transition tables.

Common follow-up: "What is the difference between OLD and NEW in a trigger?" — OLD is the row before the change (NULL on INSERT); NEW is after (NULL on DELETE).

```
-- Postgres: BEFORE ROW trigger that auto-fills a derived column
CREATE OR REPLACE FUNCTION set_full_name() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    NEW.full_name := NEW.first_name || ' ' || NEW.last_name;
    RETURN NEW;
END $$;

CREATE TRIGGER trg_set_full_name
BEFORE INSERT OR UPDATE ON users
FOR EACH ROW EXECUTE FUNCTION set_full_name();
```

Q375 · Trigger Use Cases — Audit Logs, Derived Columns, Validation

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro, Wells Fargo · Topic: Triggers

The question (interviewer's verbatim phrasing):

Give me three legitimate, defensible use cases for triggers.

The 30-second answer: Audit logs — capture who changed what and when into a history table; derived columns — keep a denormalised field in sync (full name from first/last, search vectors, computed totals); validation — enforce business invariants that cannot be expressed as a CHECK constraint or foreign key alone.

Step-by-step thought process: 1. Name the three and give an example for each. 2. Audit: AFTER INSERT/UPDATE/DELETE writing to `customers_audit(operation, old_row, new_row, changed_by, changed_at)`. Use `current_user` / `SESSION_USER` to capture the actor. 3. Derived: BEFORE INSERT/UPDATE setting `NEW.total = NEW.qty * NEW.price` — but flag the alternative, GENERATED columns, in the next question. 4. Validation: BEFORE INSERT/UPDATE that raises an exception if a cross-table invariant is violated (e.g., "a discount may not exceed the customer's tier max").

Edge cases to mention: - For audit, an `AFTER STATEMENT` trigger with transition tables is the modern, efficient pattern. - For derived columns, prefer generated/computed columns where the DB supports them — fewer moving parts. - For validation, prefer constraints (CHECK, FK, UNIQUE) where possible; triggers only when the rule spans tables.

What NOT to say: - Do not list "complex business logic" as a use case — that is exactly the anti-pattern, covered next. - Do not say "anything you can do in the app" — most teams want app logic in the app.

Common follow-up: "How do you capture the application user, not just the DB user, for audit?" — Pass an app-set session variable (`SET LOCAL app.user_id = '12345'`) and read it inside the trigger.

```
-- Postgres audit trigger
CREATE TABLE customers_audit (
  id BIGSERIAL PRIMARY KEY, op TEXT, old_row JSONB, new_row JSONB,
  changed_by TEXT DEFAULT current_user, changed_at TIMESTAMPTZ DEFAULT now()
);

CREATE OR REPLACE FUNCTION audit_customers() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO customers_audit(op, old_row, new_row)
  VALUES (TG_OP, to_jsonb(OLD), to_jsonb(NEW));
  RETURN NULL;
END $$;

CREATE TRIGGER trg_customers_audit
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION audit_customers();
```

Q376 · Trigger Anti-Patterns — Business Logic and Cascading Triggers

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Walmart Labs · Topic: Triggers

The question (interviewer's verbatim phrasing):

What are the classic anti-patterns with triggers? Why do senior engineers usually want them removed?

The 30-second answer: Business logic in triggers — it becomes invisible side effects that the application is unaware of, making debugging nightmarish. Cascading triggers — a trigger fires another trigger which fires another, forming chains nobody can trace. Performance traps — per-row triggers on bulk operations multiply cost. Hidden coupling — schema changes break trigger code in surprising ways.

Step-by-step thought process: 1. Lead with the observability point: a developer reads the app code, sees an INSERT, and has no idea that three triggers, two audit inserts and a derived update also happened. 2. Cascading example: INSERT on `orders` fires a trigger that updates `customer_summary`, which fires a trigger that recomputes `analytics_daily`. Now an EXPLAIN of a simple INSERT is incomprehensible. 3. Performance: a 1-lakh-row bulk INSERT with a row trigger that does even a single index lookup becomes a multi-minute job. 4. The cultural reason: triggers cannot be code-reviewed, traced, or version-controlled with the application code easily — they drift.

Edge cases to mention: - Postgres triggers can be RECURSIVE — easy to write an infinite loop accidentally if the trigger updates the same table. - SQL Server has a server-level setting `nested triggers` and `recursive triggers` — defaults differ. - Some teams adopt a rule: triggers may only write to audit tables, nothing else. Worth stating in interview.

What NOT to say: - Do not defend complex business triggers as "encapsulation" — that is folklore from the 1990s. - Do not say "we have always done it this way" — interviewers want reflection, not defence.

Common follow-up: *"Tell me about a bug you debugged where a trigger was the root cause."* — Have a real story or a believable hypothetical: e.g., a row count was higher than expected because an audit trigger double-inserted.

```
-- Anti-pattern: business logic in a trigger
CREATE TRIGGER trg_send_email AFTER INSERT ON orders
FOR EACH ROW EXECUTE FUNCTION send_order_confirmation(); -- DON'T

-- Better: write to an outbox table, app processes it
```

Q377 · Trigger Performance Impact

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Razorpay, Goldman · Topic: Triggers

The question (interviewer's verbatim phrasing):

How do triggers affect performance? How would you measure their cost?

The 30-second answer: Row-level triggers add per-row CPU and any I/O the trigger body does — on bulk inserts this multiplies the cost dramatically. Statement-level triggers add a fixed overhead per statement, regardless of row count, so they are cheaper. Measure with EXPLAIN ANALYZE in Postgres (trigger times appear at the bottom of the plan) or with `SET STATISTICS TIME ON` in SQL Server.

Step-by-step thought process: 1. Start with the row-vs-statement cost model. 2. EXPLAIN ANALYZE in Postgres shows a "Trigger ..." line per trigger with `time=` and `calls=` — read that. 3. SQL Server's actual execution plan shows trigger execution as a sub-plan; statistics time gives the per-trigger CPU. 4. The other hidden cost: triggers prevent some bulk operations from being minimally logged (SQL Server) and from using COPY fast-path (Postgres in some cases).

Edge cases to mention: - A trigger that does its own SELECT can lock-escalate or hold locks longer than expected. - Triggers can prevent partition-wise operations and parallel plans in some optimisers. - In Postgres, FOR EACH STATEMENT with transition tables is the preferred pattern for audit on bulk inserts.

What NOT to say: - Do not say "triggers are free" — they never are. - Do not optimise prematurely — measure before you remove or rewrite.

Common follow-up: "How would you migrate from a per-row audit trigger to a statement-level one without losing data?" — Use transition tables (Postgres 10+, SQL Server) — the statement trigger sees the full delta.

```
-- Postgres: EXPLAIN ANALYZE shows trigger cost
EXPLAIN (ANALYZE, BUFFERS)
INSERT INTO orders(customer_id, total_amount) VALUES (1, 100);
-- Trigger trg_set_full_name: time=0.123 calls=1
```

Q378 · INSTEAD OF Triggers — When You Need Them

Tags: Difficulty: Medium · Asked at: TCS, Infosys, JP Morgan, Wells Fargo · Topic: Triggers

The question (interviewer's verbatim phrasing):

What is an INSTEAD OF trigger? When would you use one?

The 30-second answer: An **INSTEAD OF** trigger replaces the original DML — when the trigger fires, the original INSERT/UPDATE/DELETE does not happen automatically; the trigger body decides what to do. The killer use case is making views updatable when they otherwise would not be (joins, aggregates, computed columns) — the trigger translates the view-level DML into the appropriate base-table writes.

Step-by-step thought process: 1. Define INSTEAD OF clearly — it is a replacement, not an addition. 2. Use case 1: updatable views. The view has a JOIN, so a direct UPDATE is ambiguous. INSTEAD OF UPDATE picks which base table(s) to update. 3. Use case 2: routing inserts to a partition or sharded child table. 4. Postgres and SQL Server both support INSTEAD OF; MySQL has very limited support (only on views with restrictions).

Edge cases to mention: - The trigger body must do the actual work — easy to forget and accidentally make a no-op. - INSTEAD OF triggers fire even for bulk DML, but they can be set as STATEMENT-level in some dialects to amortise cost. - Conflicting INSTEAD OF and AFTER triggers can create non-obvious ordering bugs.

What NOT to say: - Do not confuse INSTEAD OF with BEFORE — BEFORE happens, then the DML happens; INSTEAD OF replaces the DML. - Do not put INSTEAD OF triggers on base tables unless you really mean it — that hides the DML semantics.

Common follow-up: "What is an alternative to INSTEAD OF for updatable views in Postgres?" — Simple views (single table, no aggregates) are updatable automatically; the **updatable_views** extension and PostgreSQL 9.3+ auto-updatable views cover many cases.

```
-- Postgres: INSTEAD OF trigger on a view
CREATE VIEW order_summary AS
SELECT o.id, c.name AS customer_name, o.total_amount FROM orders o JOIN customers c ON o.customer_id = c.id;

CREATE OR REPLACE FUNCTION update_order_total() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    UPDATE orders SET total_amount = NEW.total_amount WHERE id = NEW.id;
    RETURN NEW;
END $$;

CREATE TRIGGER trg_view_update INSTEAD OF UPDATE ON order_summary
FOR EACH ROW EXECUTE FUNCTION update_order_total();
```

Q379 · Disabling Triggers Temporarily

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Wipro · Topic: Triggers

The question (interviewer's verbatim phrasing):

You are running a one-off bulk data load. The triggers on the target table would multiply the cost. How do you disable them temporarily and safely?

The 30-second answer: Postgres: `ALTER TABLE t DISABLE TRIGGER trg_name` (or `ALL`); SQL Server: `DISABLE TRIGGER trg_name ON t` ; MySQL has no per-trigger disable — you DROP and re-CREATE. Always wrap in a transaction or maintenance window, re-enable in the same script, and verify post-load that the derived data is consistent.

Step-by-step thought process: 1. Show the syntax per dialect. 2. Warn about the risk: while triggers are disabled, audit logs and derived columns will be wrong; plan a back-fill. 3. The defensive pattern: disable → load → re-enable → run a back-fill SELECT-INTO to populate audit and derived columns. 4. Note Postgres has `DISABLE TRIGGER USER` (skip user-defined triggers but keep system triggers like FK) — useful nuance.

Edge cases to mention: - In Postgres, `DISABLE TRIGGER ALL` also disables internal triggers used by foreign keys — usually a bad idea. Use `DISABLE TRIGGER USER` instead. - SQL Server's `DISABLE TRIGGER ... ON ALL SERVER` is a server-level scope — make sure you scope to a single table. - After re-enabling, the trigger does not fire retroactively — affected rows during the disable window are skipped permanently.

What NOT to say: - Do not disable FK enforcement triggers in production without a maintenance window — referential integrity gaps are painful to fix. - Do not forget to re-enable. Add it to a `FINALLY` style block (`BEGIN ... EXCEPTION WHEN OTHERS THEN ALTER TABLE ... ENABLE TRIGGER ...`).

Common follow-up: "Is there a way to skip a trigger only for a single statement?" — Postgres has `session_replication_role = 'replica'` for the session, which skips most user triggers. Use with extreme care.

```
-- Postgres
ALTER TABLE orders DISABLE TRIGGER trg_audit;
-- bulk load happens here
ALTER TABLE orders ENABLE TRIGGER trg_audit;
INSERT INTO orders_audit(op, new_row) SELECT 'INSERT', to_jsonb(o) FROM orders o WHERE created_at
```

Q380 · Sequence vs SERIAL vs IDENTITY — Generating IDs

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Razorpay, Cred · Topic: ID Generation

The question (interviewer's verbatim phrasing):

What is the difference between a sequence, SERIAL and IDENTITY? Which is the modern preferred choice?

The 30-second answer: A `SEQUENCE` is a standalone database object that hands out monotonically increasing numbers — `nextval('seq')`. `SERIAL` is a Postgres legacy shortcut that creates a sequence and wires it as the default on a column. `IDENTITY` is the SQL-standard form, supported by Postgres 10+, SQL Server (always), and Oracle 12c+. Modern preference: `GENERATED ALWAYS AS IDENTITY` — explicit, portable, and free of the ownership/permissions quirks of `SERIAL`.

Step-by-step thought process: 1. Define each: sequence (object), `SERIAL` (Postgres macro), `IDENTITY` (standard column property). 2. Show the syntax: `id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY` vs `id BIGSERIAL PRIMARY KEY`. 3. Mention the quirks of `SERIAL` — the underlying sequence is owned by the column but inserts can overflow if the app manually inserts an id (gaps in the sequence). 4. Note MySQL uses `AUTO_INCREMENT` which is column-bound; it is similar in spirit to `IDENTITY`.

Edge cases to mention: - Sequences are not transactional — `nextval` increments even if the transaction rolls back, so you can have gaps. This is intentional for concurrency. - `GENERATED ALWAYS AS IDENTITY` rejects manual inserts; `GENERATED BY DEFAULT AS IDENTITY` allows them. - SQL Server `IDENTITY_INSERT ON` lets you manually insert into an `IDENTITY` column for one statement.

What NOT to say: - Do not promise "gapless ids" from any of these — sequences and identities all have gaps under load. - Do not say `SERIAL` is deprecated — it still works in Postgres; it is just not recommended in new code.

Common follow-up: *"How would you guarantee gapless invoice numbers?"* — You cannot with a sequence; use a separate counter table with `SELECT ... FOR UPDATE` and accept the lock contention as the cost.

```
-- Modern Postgres preferred form
CREATE TABLE invoices (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  amount NUMERIC NOT NULL
);

-- SQL Server
CREATE TABLE invoices (
  id BIGINT IDENTITY(1,1) PRIMARY KEY,
  amount DECIMAL(10,2) NOT NULL
);
```

Q381 · AFTER INSERT Trigger for Outbox Pattern

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Walmart Labs, Goldman · Topic: Triggers

The question (interviewer's verbatim phrasing):

Walk me through implementing the transactional outbox pattern using an AFTER INSERT trigger. Why is this safer than the application publishing directly to Kafka?

The 30-second answer: On every INSERT into the business table, an `AFTER INSERT` trigger writes a row into an `outbox` table inside the same transaction. A separate poller process (or Debezium-style CDC) reads `outbox` and publishes to Kafka. Because the business write and the outbox write share one transaction, the message is published if and only if the business state changed — no dual-write inconsistency.

Step-by-step thought process: 1. Set up the problem: dual-write — write to DB then publish to Kafka — can leave one side committed and the other not. 2. Show the trigger that inserts a JSON payload into `outbox(id, aggregate_type, aggregate_id, payload, status, created_at)`. 3. The poller selects `WHERE status='PENDING' FOR UPDATE SKIP LOCKED`, publishes, marks 'SENT'. 4. Mention Debezium / CDC as an alternative: read the transaction log directly, no outbox table needed — but more infrastructure.

Edge cases to mention: - Outbox can grow large — archive or partition it. - `SKIP LOCKED` is critical to allow multiple poller workers without contention (Postgres 9.5+, MySQL 8+, SQL Server with `READPAST`). - Ordering: if you care about order per aggregate, the poller must group by `aggregate_id` and process sequentially.

What NOT to say: - Do not claim Kafka transactions solve this — Kafka transactions are between Kafka topics, not between Kafka and a database. - Do not put the Kafka publish inside the trigger — that breaks the transactional boundary and re-introduces dual-write.

Common follow-up: *"What is Change Data Capture and how does it compare to the outbox table?"* — CDC reads the WAL/redo log directly (Debezium), so no trigger needed; less app-side code but more infrastructure to run.

```
-- Postgres outbox trigger
CREATE TABLE outbox (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  aggregate_type TEXT, aggregate_id BIGINT,
  payload JSONB, status TEXT DEFAULT 'PENDING',
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE OR REPLACE FUNCTION outbox_order_created() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO outbox(aggregate_type, aggregate_id, payload)
  VALUES ('order', NEW.id, jsonb_build_object('event','order.created','order_id',NEW.id, 'amount',NEW.amount));
  RETURN NULL;
END $$;

CREATE TRIGGER trg_outbox_order AFTER INSERT ON orders
FOR EACH ROW EXECUTE FUNCTION outbox_order_created();
```

Q382 · Generated/Computed Columns — Alternative to Triggers

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Flipkart, Meesho · Topic: Schema

The question (interviewer's verbatim phrasing):

When can you use generated columns instead of a trigger to maintain a derived field?

The 30-second answer: Whenever the derived value is a deterministic expression of other columns in the same row, prefer a generated column — `GENERATED ALWAYS AS (qty * price) STORED`. The DB maintains it, you cannot accidentally write to it, and the optimiser knows it is deterministic. Triggers should only be used when the derived value depends on other tables or on non-row-local state.

Step-by-step thought process: 1. Lead with the rule: same-row deterministic → generated column; cross-row or cross-table → trigger. 2. Show Postgres syntax: `GENERATED ALWAYS AS (...)` `STORED`. MySQL has STORED and VIRTUAL. SQL Server has `AS (expr) PERSISTED`. 3. Generated columns can be indexed (STORED) — that is the killer feature for search and sort scenarios.

4. Note Postgres only supports STORED, not VIRTUAL — important when comparing with MySQL features.

Edge cases to mention: - The generation expression must be IMMUTABLE in Postgres — you cannot reference `now()` or another table. - Adding a STORED generated column rewrites the whole table — expensive on large tables. - MySQL VIRTUAL generated columns are computed on read — index them with a functional index if needed.

What NOT to say: - Do not use a trigger to maintain `full_name = first || ' ' || last` when a generated column does it cleaner. - Do not assume generated columns work across joins — they do not; same-row only.

Common follow-up: "How would you store a tsvector for full-text search?" — `tsv tsvector GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED`, then GIN index on `tsv`.

```
CREATE TABLE order_lines (
  id BIGSERIAL PRIMARY KEY,
  qty INT, unit_price NUMERIC(10,2),
  line_total NUMERIC(12,2) GENERATED ALWAYS AS (qty * unit_price) STORED
);
```

Q383 · Security — SECURITY DEFINER vs SECURITY INVOKER

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, JP Morgan, Goldman · Topic: Security

The question (interviewer's verbatim phrasing):

What is the difference between SECURITY DEFINER and SECURITY INVOKER on a stored procedure or function? What are the security risks?

The 30-second answer: `SECURITY INVOKER` (default) runs the body with the caller's privileges — they need direct grants on every object touched. `SECURITY DEFINER` runs it with the privileges of the function owner — useful when you want to give controlled access to data without granting it directly. The risk is privilege escalation: a poorly written SECURITY DEFINER function with dynamic SQL is an SQL-injection-into-superuser vector.

Step-by-step thought process: 1. Define both with examples. 2. The legit pattern: a function owned by `app_admin` that allows `app_user` to update one specific row in a sensitive table — the user has no direct grant. 3. The risk: search_path attacks in Postgres (caller manipulates schema search path so the function executes a different table). Always `SET search_path = pg_catalog, public` inside

DEFINER functions. 4. Mention SQL Server's `EXECUTE AS OWNER` / `EXECUTE AS USER` clauses — same concept.

Edge cases to mention: - Dynamic SQL inside DEFINER functions is the classic exploit — concatenated user input runs with elevated privileges. - Postgres has `SECURITY DEFINER ... SET search_path = ...` to lock the path at function definition time. - Granting EXECUTE on a DEFINER function silently grants access to the underlying tables — audit carefully.

What NOT to say: - Do not default to SECURITY DEFINER — that is a privilege-escalation magnet. - Do not use dynamic SQL in DEFINER functions without parameterising every input.

Common follow-up: "What is the search_path attack in Postgres?" — Attacker creates a table in their own schema with the same name, sets search_path, and the DEFINER function operates on the attacker's table. Always set search_path explicitly.

```
CREATE OR REPLACE FUNCTION refund_order(p_order_id BIGINT)
RETURNS VOID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog, public
AS $$
BEGIN
    UPDATE orders SET status='REFUNDED' WHERE id = p_order_id;
END $$;
```

Q384 · Stored Procedure and ORM Interaction

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Wells Fargo, AmEx · Topic: ORMs

The question (interviewer's verbatim phrasing):

Your Spring Boot or .NET application uses JPA / Hibernate / Entity Framework. How do you call a stored procedure from it? What goes wrong?

The 30-second answer: JPA has `@NamedStoredProcedureQuery` and `EntityManager.createStoredProcedureQuery`. Entity Framework Core has `FromSqlRaw` / `ExecuteSqlRaw` and explicit `DbCommand` for procs. The friction: ORMs map rows to entities and procs may return shapes that do not match, may have multiple result sets, may return refcursors (Oracle), or may need OUT parameter registration. Mapping conventions also differ — JPA column naming, EF's keyless entities.

Step-by-step thought process: 1. Show the canonical Spring Data / JPA pattern with `@Procedure` annotation. 2. Mention the EF Core approach with `FromSqlRaw` for result-set-returning procs. 3. The

pain points: OUT parameter registration, refcursors in Oracle (need `OracleTypes.CURSOR`), and Postgres procedures vs functions (procedures need `CallableStatement.execute()`, not `executeQuery`). 4. Note that many teams adopt the rule: ORMs for CRUD, raw JDBC/Dapper for procs — fewer surprises.

Edge cases to mention: - Hibernate's named procedure mapping does not support all dialect quirks — fall back to plain JDBC for edge cases. - Postgres procedures (since 11) cannot be called inside a JDBC `executeQuery` — must use `CallableStatement.execute()`. - EF Core 7+ has improved stored procedure support but TVPs (table-valued parameters) still need raw SQL.

What NOT to say: - Do not claim ORMs handle procs transparently — they all have rough edges. - Do not embed business logic in procs just because the ORM is annoying — fix the integration, not the architecture.

Common follow-up: "How would you map a refcursor returned by an Oracle proc to a list of entities?" — Use `CallableStatement.registerOutParameter(2, OracleTypes.CURSOR)`, then `ResultSet rs = (ResultSet) cs.getObject(2);` and walk it manually.

```
// Spring Data JPA
@Procedure(procedureName = "settle_invoice")
void settleInvoice(@Param("p_invoice_id") Long invoiceId);

// JDBC for trickier cases
try (CallableStatement cs = conn.prepareCall("{ call get_order_count(?, ?) }")) {
    cs.setInt(1, customerId);
    cs.registerOutParameter(2, Types.INTEGER);
    cs.execute();
    int count = cs.getInt(2);
}
```

Q385 · Versioning Stored Procedures — Migration Patterns

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant, Razorpay, Cred · Topic: Migrations

The question (interviewer's verbatim phrasing):

How do you version stored procedures so the team can deploy changes safely?

The 30-second answer: Keep the proc source in the same repo as the application, in a `db/migrations/` folder, with one timestamped file per change. Use a migration tool — Flyway, Liquibase, Sqitch, or EF Migrations — that tracks applied versions in a `schema_version` table. Always use `CREATE OR REPLACE` (Postgres, Oracle) or `CREATE OR ALTER` (SQL Server 2016+) so re-running the migration is idempotent.

Step-by-step thought process: 1. The principle: procs are code; they belong in version control with the app. 2. Flyway pattern: `V001__create_settle_proc.sql` , `V002__update_settle_proc.sql` — incremental, immutable history. 3. For breaking changes, deploy the new proc under a versioned name (`settle_invoice_v2`), switch callers gradually, drop the old one in a follow-up release. 4. Mention idempotent re-runs: `CREATE OR REPLACE FUNCTION ...` in Postgres, `CREATE OR ALTER PROCEDURE ...` in SQL Server 2016+, `DROP IF EXISTS; CREATE` for older versions.

Edge cases to mention: - Long-running deploys: a proc replace acquires a lock; on busy systems schedule during low traffic. - Some dialects mark dependants invalid on proc replace (Oracle); recompile them as part of the migration. - Rollback: keep a rollback SQL alongside each forward migration, or rely on a separate down-migration file (Liquibase).

What NOT to say: - Do not edit procs directly in production via SSMS or pgAdmin — that leaves no audit trail and breaks reproducibility. - Do not commit procs to a "stored_procedures" repo separate from the app — drift is guaranteed.

Common follow-up: *"How do you handle a proc that has a breaking signature change while old app pods are still running?"* — Side-by-side versions: deploy v2 alongside v1, migrate callers, drop v1 next release.

```
-- V015__update_settle_invoice.sql (Flyway)
CREATE OR REPLACE PROCEDURE settle_invoice(p_invoice_id BIGINT, p_settled_at TIMESTAMPTZ DEFAULT NULL)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE invoices SET status='SETTLED', settled_at = p_settled_at WHERE id = p_invoice_id;
END $$;
```

Q386 · Testing Stored Procedures

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Walmart Labs, JP Morgan · Topic: Testing

The question (interviewer's verbatim phrasing):

How do you write tests for stored procedures? What makes them harder to test than application code?

The 30-second answer: Three layers: unit tests with a framework like pgTAP (Postgres), tSQLt (SQL Server), or utPLSQL (Oracle) — these run inside the DB and assert on outputs and side effects. Integration tests from the app side — spin up a real DB in Docker, run migrations, invoke the proc, assert. Avoid mocking the DB entirely. The difficulty: procs depend on schema state, are slow to set up and tear down, and require careful fixture management.

Step-by-step thought process: 1. Show pgTAP / tSQLt as the in-DB option — they let you write `BEGIN; ... ; ROLLBACK;` style tests with proper assertions. 2. From the app: Testcontainers (JVM) or Docker Compose to spin up a real DB; run migrations against it; call the procs through the same client the app uses. 3. Fixtures: every test seeds its own data and rolls back at the end; isolated transactions per test. 4. Talk about CI integration — these tests are slower than pure unit tests, so they typically run on a separate CI stage.

Edge cases to mention: - Schema drift between dev/test/prod is the biggest practical risk — always migrate the test DB with the same tool as prod. - Some procs do non-transactional work (DDL, COMMIT inside body) — those need separate test cleanup logic. - For triggers, test by inserting/updating and asserting on the side effects (audit rows, derived columns).

What NOT to say: - Do not say "we test in QA manually" — that fails any quality bar above startup-level. - Do not mock the DB and pretend that covers the proc — you would only be testing your mock.

Common follow-up: "How would you set up a Testcontainers-based integration test for a Postgres procedure in Java?" — `@Testcontainers` class, `PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16")`, run Flyway against `pg.getJdbcUrl()`, then connect via `DataSource` and call the proc.

```
-- pgTAP example
BEGIN;
SELECT plan(2);
INSERT INTO invoices(id, amount, status) VALUES (1, 100, 'PENDING');
CALL settle_invoice(1);
SELECT is((SELECT status FROM invoices WHERE id = 1), 'SETTLED', 'invoice is settled');
SELECT isnt((SELECT settled_at FROM invoices WHERE id = 1), NULL, 'settled_at populated');
SELECT * FROM finish();
ROLLBACK;
```

Chapter 10, Part A — JSON in SQL

JSON columns went from "nice to have" to expected in product company rounds across 2024-2026 — every Razorpay, Swiggy, Cred, Meesho, PhonePe and Flipkart panel I've sat in front of now asks at least one JSON question, and GCC rounds at Walmart Labs, JP Morgan and Target are catching up fast. The trouble is that JSON is the most divergent area across engines — Postgres has `jsonb` and SQL/JSON path, MySQL has its own JSON type with `JSON_TABLE`, SQL Server has `OPENJSON` and `JSON_VALUE`, and the SQL:2016 standard added syntax that very few vendors actually implement. This section covers Postgres `jsonb` deeply because that's what most Indian product teams run on, plus MySQL JSON, SQL Server JSON and the standard functions you should call out when the interviewer asks "is this portable?". Read it once for breadth, then drill the four or five Qs that map to whatever stack the company you're interviewing at actually uses.

Q387 · JSON vs JSONB in Postgres

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, Meesho · Topic: JSON

The question (interviewer's verbatim phrasing):

Postgres has two JSON types — `json` and `jsonb`. What's the difference and which one would you pick for a `user_preferences` column?

The 30-second answer: `json` stores the raw text exactly as you sent it; `jsonb` parses it into a binary decomposed form. `jsonb` is slower to insert and slightly larger on disk but much faster to query, supports GIN indexes, and dedupes keys — pick `jsonb` for `user_preferences` and basically every other production case.

Step-by-step thought process: 1. Open with the storage difference: `json` is just text with a validity check; `jsonb` is parsed once and stored as a tree of keys and values. 2. State the consequences: `jsonb` strips whitespace, removes duplicate keys (keeping the last one), and does not preserve key order — `json` preserves all three. 3. Push the performance angle: every `->`, `->>`, `@>` operator on `json` has to re-parse the text; `jsonb` walks the binary tree directly. Over millions of rows the gap is enormous. 4. Close with indexing — only `jsonb` supports GIN indexes and the path-based operators, which is the reason almost every Indian product team I've seen defaults to `jsonb`.

Edge cases to mention: - If you need to preserve the exact incoming bytes for audit or signature verification, `json` is the right choice — `jsonb` will reorder keys. - `jsonb` insert is roughly 5-15% slower than `json` because of the parse step; for write-heavy logs people sometimes pick `json`. - `jsonb` has a soft limit around 268 MB per value but practical performance degrades long before that.

What NOT to say: - Do not say "they're basically the same." The query performance gap is one to two orders of magnitude on indexed lookups. - Do not claim `json` supports GIN indexes — it does not.

Common follow-up: "If I already have a `json` column in production, how would you migrate to `jsonb`?" — `ALTER TABLE ... ALTER COLUMN col TYPE jsonb USING col::jsonb` — but it rewrites the whole table, so do it during a maintenance window or use a shadow column with a backfill.

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  email TEXT NOT NULL,  
  preferences JSONB NOT NULL DEFAULT '{} '::jsonb  
);
```

Q388 · When to Use a JSON Column vs Separate Relational Schema

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Cred, Walmart Labs · Topic: JSON

The question (interviewer's verbatim phrasing):

A product manager says "let's just dump everything into a JSON column, it's flexible." When is that the right call and when is it a trap?

The 30-second answer: Use JSON for sparse, schema-light, rarely-queried payloads — feature flags, user preferences, third-party API responses. Use a relational schema for anything you filter, join, or aggregate on regularly. The trap is querying *inside* JSON on the hot path — that's slow and hard to index well.

Step-by-step thought process: 1. Open with the decision rule: how often do you need to filter or join on the fields inside? If the answer is "every read", use columns; if it's "almost never", use JSON. 2. Give the canonical good case: `user_preferences` with notification settings, theme, language — read on login, almost never queried by content, varies per user. 3. Give the canonical bad case: `order_status` shoved inside a JSON blob — you'll filter on it in every list query, and now every read pays the JSON parse cost. 4. Mention the middle ground — generated columns from a JSON field, which gets you JSON storage flexibility plus a real indexed column.

Edge cases to mention: - For high-cardinality nested arrays you read together (an order's line items), JSON can actually be faster than a separate child table because you avoid the join. - If multiple services own different slices of the document, JSON conflicts on write — relational columns with explicit ownership are cleaner. - Analytics teams hate JSON columns because BI tools can rarely shred them — if data goes to a warehouse, prefer columns.

What NOT to say: - Do not say "always normalise" — that's dogma. Modern teams happily mix. - Do not say "JSON is always faster because no joins" — querying inside JSON is slow.

Common follow-up: "Show me a hybrid design for an `orders` table." — Core columns `id`, `user_id`, `total`, `status`, `created_at`; JSON column for `metadata` (UTM, device, A/B bucket) that you rarely filter on.

```
-- Good: hot fields as columns, cold metadata as JSONB
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  total_paise BIGINT NOT NULL,
  status TEXT NOT NULL,
  metadata JSONB NOT NULL DEFAULT '{}::jsonb',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Q389 · JSON Column Anti-Patterns

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, PhonePe, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

Walk me through the worst JSON column design you've seen in production and why it was bad.

The 30-second answer: The classic one is putting hot filter fields inside JSON — `WHERE preferences->>'plan' = 'pro'` on a hundred-million-row table with no expression index. Every read parses JSON, every query does a full scan. Fix by either promoting the field to a real column or adding a generated column with a btree index on it.

Step-by-step thought process: 1. Start with the most common anti-pattern: storing fields you filter on inside JSON without an expression or generated-column index — your `EXPLAIN` plan will show a seq scan on every query. 2. Anti-pattern two: deeply nested arrays you have to unnest on every read. Each unnest is per-row work; over time it becomes the slowest query in your system. 3. Anti-pattern three: schema drift — three services writing different shapes into the same JSON column with no validation, so reads have to handle five variants. 4. Anti-pattern four: storing very large blobs (images base64-encoded, full third-party responses) — the column gets TOASTed and every read becomes an out-of-line fetch.

Edge cases to mention: - A small JSON column you only read together with the row is fine even without indexes — the problem is filtering and aggregating. - If you must filter on a JSON field, expression indexes (`CREATE INDEX ON t ((col->>'field'))`) save you, but only for exact matches. - ORMs often default to JSON for any unstructured field — review migrations before they hit production.

What NOT to say: - Do not say "JSON columns are always bad." They are great for the right use case. - Do not blame the database; the schema decision is on the developer.

Common follow-up: "How would you migrate a hot field out of JSON without downtime?" — Add a real column, dual-write for a release, backfill in batches, swap reads, then drop the JSON path.

```
-- Anti-pattern: hot filter inside JSON with no index
SELECT id FROM users WHERE preferences->>'plan' = 'pro';
-- Plan: Seq Scan on users (cost=0.00..1820000.00 rows=...)
```

Q390 · Postgres `->` vs `->>` Operators

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Cred, Zerodha · Topic: JSON

The question (interviewer's verbatim phrasing):

What is the difference between the `->` and `->>` operators in Postgres? Give me a query that uses both.

The 30-second answer: `->` returns a JSON (or JSONB) value — keeps the JSON type. `->>` returns the same field cast to text. You use `->` when you want to chain further JSON access, `->>` when you want a string you can filter on or display.

Step-by-step thought process: 1. Lead with the return-type rule: arrow keeps JSON, double-arrow gives text — that's the whole distinction. 2. Show a chaining example: `preferences->'notifications'>>'email'` walks two keys with `->` and pulls the leaf as text with `->>`. 3. Warn about a common bug — using `->` in a `WHERE` clause and comparing to a string literal returns false because you're comparing JSON to text. Use `->>` for filters. 4. Mention numeric indices: `tags->0` for the first element of a JSON array, `tags->>0` for the same element as text.

Edge cases to mention: - If the key doesn't exist, both operators return SQL `NULL` — they do not throw. Helpful for missing fields. - If the value is JSON `null`, `->` returns the JSON `null` value, while `->>` returns SQL `NULL`. Subtle but bites you. - `#>` and `#>>` are the path versions — `preferences #>> '{notifications,email}'` does the same as the chain above with a single operator.

What NOT to say: - Do not say "they're interchangeable" — they return different types. - Do not write `WHERE preferences->'plan' = 'pro'` — that comparison will silently never match.

Common follow-up: "How would you cast a JSON number to an integer?" — `(col->>'count')::int` — go through text, then cast.

```
SELECT id,
       preferences -> 'notifications'      AS notif_json,
       preferences ->> 'language'          AS lang_text,
       preferences #>> '{notifications,email}' AS notif_email
FROM users
WHERE preferences ->> 'plan' = 'pro';
```

Q391 · SQL/JSON Path — `@?`, `@@`, `jsonb_path_query`

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Goldman, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

Postgres 12+ added SQL/JSON path support. Explain `jsonb_path_query`, `@?` and `@@`, and when you'd reach for them over the `->` operators.

The 30-second answer: SQL/JSON path is a mini-language for navigating and filtering JSON in one expression. `jsonb_path_query(col, '$.items[*] ? (@.qty > 1)')` returns matching values, `@?` tests whether a path expression matches any value, and `@@` tests whether a path predicate evaluates true. Use them when you want to express a multi-step filter — especially inside nested arrays — that would be ugly with chained `->` calls.

Step-by-step thought process: 1. Start by framing path syntax: `$` is the document root, `.key` walks objects, `[*]` walks arrays, `? (@.field > 5)` filters. 2. Explain each operator: `@?` returns boolean — "does any value match this path?". `@@` evaluates a path predicate (one that returns boolean) and returns its result. `jsonb_path_query` actually returns the matching JSON values as a set. 3. Show the canonical use case — finding orders that contain a high-value line item: `payload @? '$.items[*] ? (@.amount > 10000)'`. 4. Mention indexing — GIN with the `jsonb_path_ops` opclass accelerates `@?` and `@@` significantly.

Edge cases to mention: - Path expressions use a different quoting style than JSON itself — double quotes inside the path string conflict with SQL; use the standard `$.key` form for safety. - `jsonb_path_query` returns a set; if you only want the first match, use `jsonb_path_query_first` to avoid scalar-vs-set bugs. - Lax mode (default) silently coerces missing keys; strict mode raises errors — pick strict when you want loud failures during testing.

What NOT to say: - Do not claim path syntax is "the same as JSONPath you saw on the internet" — Postgres SQL/JSON path is similar but not identical. - Do not use path query in a tight inner loop without a GIN index — it can be slower than a flat `->>`.

Common follow-up: "How does this compare to MySQL?" — MySQL implements `JSON_TABLE` and `JSON_EXTRACT` with a JSONPath-like syntax but lacks `@?` and `@@`; the path semantics are similar but not identical.

```
-- Orders with a line item over Rs 10,000
SELECT id FROM orders
WHERE payload @? '$.items[*] ? (@.amount_paise > 1000000)';
```

Q392 · `jsonb_set` and `jsonb_insert` — Updating JSON

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Cred, Razorpay · Topic: JSON

The question (interviewer's verbatim phrasing):

A user updates one notification preference. Write the query to update only that key in the `preferences` JSONB column without overwriting the whole document.

The 30-second answer: Use `jsonb_set(preferences, '{notifications,email}', 'false'::jsonb, true)` — the path is an array of keys, the new value must be JSONB, and the fourth argument `true` creates the key if it does not exist.

Step-by-step thought process: 1. Start by warning what people get wrong: a naive `UPDATE users SET preferences = '{...}'::jsonb` blows away every other field; partial update is mandatory. 2. Show `jsonb_set` signature: target, path array, new value (as jsonb), create-if-missing flag. 3. Differentiate `jsonb_insert` — it inserts at a position without overwriting; useful for adding into arrays. `jsonb_set` overwrites. 4. Mention concurrency — even with `jsonb_set`, two simultaneous updates to different keys in the same document are last-writer-wins on the whole row; use row-level locks or merge in app code if that matters.

Edge cases to mention: - The new value argument must be JSONB — string literals need `'"false"'::jsonb` (note the inner quotes for a JSON string). - For deeply nested paths the array form gets long; use `||` (concat) when you're replacing a whole subtree. - Postgres 16 added `jsonb_set_lax` which gives you control over what happens when the new value is SQL `NULL` — useful for upsert-style merges.

What NOT to say: - Do not use string concatenation to "patch" JSON — quoting nightmares plus injection risk. - Do not forget the cast — passing a raw text value to `jsonb_set` raises a type error.

Common follow-up: "How would you remove a key?" — Use the `-` operator: `preferences - 'old_key'`. For nested removal, use `#-` with a path: `preferences #- '{notifications,sms}'`.

```
UPDATE users
SET preferences = jsonb_set(
    preferences,
    '{notifications,email}',
    'false'::jsonb,
    true
)
WHERE id = 1042;
```

Q393 · `jsonb_path_exists` — Testing Without Returning

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Walmart Labs · Topic: JSON

The question (interviewer's verbatim phrasing):

What does `jsonb_path_exists` do, and how is it different from `jsonb_path_query` ?

The 30-second answer: `jsonb_path_exists(col, '$.items[*] ? (@.gst_invoice == true)')` returns a boolean — true if at least one value matches the path expression.

`jsonb_path_query` returns the matching values themselves as a set. Use `exists` in `WHERE` clauses, use `query` when you actually need the values.

Step-by-step thought process: 1. Lead with the return-type difference: boolean vs set of values. 2. Explain why you'd reach for `jsonb_path_exists` over `@?` — both do the same thing semantically, but `jsonb_path_exists` takes named parameters via the `vars` argument, so you can bind safely without string-formatting paths. 3. Show the parameter binding: `jsonb_path_exists(col, '$.items[*] ? (@.amount > $threshold)', '{"threshold": 10000} '::jsonb)` — the third argument injects values without SQL string concatenation. 4. Mention that `silent => true` suppresses errors from path syntax issues — handy for messy data.

Edge cases to mention: - Like `@?`, `jsonb_path_exists` benefits from a GIN index with `jsonb_path_ops`. - The function returns SQL `NULL` if the input column is `NULL` — not false. Wrap with `COALESCE` in filter logic. - Parameter binding via the `vars` argument is the cleanest way to build dynamic JSON queries from application code.

What NOT to say: - Do not say "`jsonb_path_exists` is faster than `@?`" — they compile to roughly the same plan. - Do not pass user input into a path string without parameter binding — same injection risk as string-built SQL.

Common follow-up: "Show me the same query using `@?` ." — `WHERE col @? '$.items[*] ? (@.amount > 10000)'` — the operator form is shorter; the function form gives you parameter binding.

```
SELECT id FROM orders
WHERE jsonb_path_exists(
    payload,
    '$.items[*] ? (@.gst_invoice == true)'
);
```

Q394 · GIN Indexes on JSONB — When and How

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Swiggy, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

You have a hundred-million-row `events` table with a `payload` JSONB column. Queries that filter by keys inside `payload` are slow. How do you index it?

The 30-second answer: Add a GIN index on `payload` — `CREATE INDEX ON events USING GIN (payload jsonb_path_ops)`. That accelerates `@>`, `@?` and `@@` operators. If you only query specific keys, an expression index on `(payload->>'key')` is smaller and faster for that one access pattern.

Step-by-step thought process: 1. Open with the two flavours of GIN: the default opclass (`jsonb_ops`) indexes every key and value, supports more operators but is larger; `jsonb_path_ops` indexes only paths, is smaller and faster but only accelerates `@>`, `@?`, `@@`. 2. State the decision: if you do many `?` and `?|` key-existence queries, default opclass. If you do containment queries (`@>`), `jsonb_path_ops` is roughly 30% smaller and faster. 3. Talk about size — GIN on a 100M row JSONB column can easily be 20-50 GB; budget accordingly. 4. Show the alternative — an expression index on the specific path: `CREATE INDEX ON events ((payload->>'event_type'))` is tiny and lightning fast for that one filter.

Edge cases to mention: - GIN indexes are slow to write — every insert updates the index for every key in the document. On write-heavy tables, batch loads suffer. - `fastupdate=on` (default) accumulates updates in a pending list; queries scan the pending list, so right after a bulk insert the first query is slow. Run `VACUUM` to merge. - For containment with `@>`, GIN is essential; without it Postgres scans every row.

What NOT to say: - Do not add a GIN index "just in case" — they're expensive on writes and disk. - Do not say "expression indexes work for all queries" — they only help for the exact expression you indexed.

Common follow-up: "How do you check if the GIN index is actually being used?" — `EXPLAIN (ANALYZE, BUFFERS)` and look for `Bitmap Index Scan on <index_name>`. If you see `Seq Scan`, the planner thinks the index isn't helpful — possibly bad statistics.

```
CREATE INDEX events_payload_path_idx
ON events USING GIN (payload jsonb_path_ops);

-- Now this uses the index
SELECT id FROM events
WHERE payload @> '{"event_type": "checkout_started"}';
```

Q395 · Expression Index on a Specific JSON Path

Tags: Difficulty: Medium · Asked at: Swiggy, Razorpay, Cred, PhonePe · Topic: JSON

The question (interviewer's verbatim phrasing):

You only ever filter by one JSON key — say `payload->>'event_type'`. Do you really need a GIN index?

The 30-second answer: No — a btree expression index on `((payload->>'event_type'))` is smaller, faster on writes, and gives equality plus range performance for that one path. GIN is overkill when you index a single known field.

Step-by-step thought process: 1. Lead with the size comparison — a btree expression index on one text field is a small fraction of a GIN index over the whole document. 2. Explain why writes are faster: btree updates one entry per row per insert; GIN updates one entry per key per row. 3. Show the index and emphasise that the query must use exactly the same expression — `payload->>'event_type'` not `payload->'event_type'`. 4. Mention the partial index trick: if you mostly filter `WHERE payload->>'status' = 'pending'`, a partial index `WHERE (payload->>'status') = 'pending'` is even smaller.

Edge cases to mention: - Expression must be `IMMUTABLE` — Postgres won't let you index a function that varies. Most JSON operators are immutable. - If you cast inside the index — `((payload->>'amount')::int)` — your query must also cast or the planner won't match. - Stats are collected for the expression value, so the planner has decent cardinality estimates after `ANALYZE`.

What NOT to say: - Do not say "GIN is always the answer for JSONB" — it's the wrong default for known single-key queries. - Do not assume the planner finds the index if your query expression is even slightly different.

Common follow-up: "Can you do a composite expression index?" — Yes:

`CREATE INDEX ON events ((payload->>'event_type'), created_at DESC)` for filter-then-sort access patterns.

```
CREATE INDEX events_type_idx
ON events ((payload->>'event_type'));

-- Uses the index for equality and range
SELECT id, created_at
FROM events
WHERE payload->>'event_type' = 'checkout_started';
```

Q396 · MySQL `JSON_EXTRACT` and `->` Shortcut

Tags: Difficulty: Easy · Asked at: Flipkart, Meesho, Wipro, Cognizant · Topic: JSON

The question (interviewer's verbatim phrasing):

How do you read a value out of a JSON column in MySQL? Show me both the function and the shortcut.

The 30-second answer: `JSON_EXTRACT(preferences, '$.language')` returns the JSON value; the `->` operator is shorthand for the same thing. To get the unquoted text use `JSON_UNQUOTE(JSON_EXTRACT(...))` or the `->>` shortcut.

Step-by-step thought process: 1. Start with the canonical function: `JSON_EXTRACT(col, '$.path')` — the path uses JSONPath syntax with `$` as root. 2. Introduce the `->` shorthand — `col->'$.path'` is the same as `JSON_EXTRACT(col, '$.path')`. 3. Explain the quoting issue: `JSON_EXTRACT` returns JSON, so a string comes back wrapped in double quotes. `JSON_UNQUOTE` strips them; `->>` does extract-and-unquote in one step. 4. Warn about a common Postgres-vs-MySQL trip-up: MySQL needs `$.field`, Postgres uses bare keys with `->>`. Different dialect, same idea.

Edge cases to mention: - MySQL has a single `JSON` type — no `json` vs `jsonb` split. Internally it's a binary representation similar to Postgres `jsonb`. - Filtering inside JSON in MySQL needs either a generated column with an index or a multi-valued index (8.0.17+) — direct path filters won't use indexes. - MySQL's `JSON` type sorts using its own internal order, which is not the same as text comparison — never rely on raw JSON comparisons for ordering.

What NOT to say: - Do not say "MySQL JSON works the same as Postgres jsonb" — operator names and indexing are different. - Do not put `'$.field'` paths inline in user input — quote them safely.

Common follow-up: "How would you index this for fast filtering?" — Add a stored generated column on `JSON_UNQUOTE(JSON_EXTRACT(col, '$.field'))` and index that column.

```
-- MySQL
SELECT id,
       preferences->'$.language' AS lang_json,
       preferences->>'$.language' AS lang_text
FROM users
WHERE preferences->>'$.plan' = 'pro';
```

Q397 · MySQL `JSON_TABLE` — Shredding to Relational

Tags: Difficulty: Hard · Asked at: Flipkart, Meesho, Walmart Labs, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

An `orders` row has a `line_items` JSON array. Write a MySQL query that returns one row per line item with the order id and item SKU.

The 30-second answer: Use `JSON_TABLE` — it turns a JSON array into a derived table you can join. Define the column mapping with `COLUMNS (sku VARCHAR(64) PATH '$.sku', qty INT PATH '$.qty')` and Cross Join it to the parent rows.

Step-by-step thought process: 1. State the goal: shredding a JSON array into row-per-element so the rest of SQL works normally. 2. Show the syntax:

`JSON_TABLE(line_items, '$[*]' COLUMNS (sku VARCHAR(64) PATH '$.sku', qty INT PATH '$.qty')) AS items`. 3. Explain `$[*]` — iterate every element of the array. `COLUMNS` extracts each leaf using its own path. 4. Mention `NESTED PATH` for inner arrays — `JSON_TABLE` can flatten two levels of array in a single call.

Edge cases to mention: - `JSON_TABLE` is MySQL 8.0+ only; older versions had no clean way to shred arrays. - If a path is missing, the column is `NULL` by default; `ERROR ON ERROR` makes it fail loudly instead. - Performance: every query re-shreds the JSON. For high-traffic reads, persist the rows into a child table.

What NOT to say: - Do not say "you can do this with `JSON_EXTRACT` in a loop" — that's application-side, not SQL. - Do not skip the `CROSS JOIN` — without it `JSON_TABLE` returns the array elements without parent context.

Common follow-up: "What's the Postgres equivalent?" — `jsonb_array_elements(line_items)` in a `LATERAL` subquery, which is the same shredding pattern.

```
-- MySQL
SELECT o.id AS order_id, items.sku, items.qty
FROM orders o,
JSON_TABLE(
  o.line_items, '$[*]'
  COLUMNS (
    sku VARCHAR(64) PATH '$.sku',
    qty INT PATH '$.qty'
  )
) AS items;
```

Q398 · SQL Server `JSON_VALUE` and `JSON_QUERY`

Tags: Difficulty: Medium · Asked at: Wells Fargo, JP Morgan, Goldman, AmEx · Topic: JSON

The question (interviewer's verbatim phrasing):

SQL Server stores JSON as text. How do you read scalar values versus nested objects from a JSON column?

The 30-second answer: `JSON_VALUE(col, '$.path')` returns a scalar (text or number) — fails for objects/arrays. `JSON_QUERY(col, '$.path')` returns a JSON fragment — fails for scalars. Pick the function that matches the expected return type.

Step-by-step thought process: 1. Start by stating SQL Server has no native JSON type — it's `NVARCHAR(MAX)` with helper functions and `ISJSON` for validation. 2. Show both functions side by side: `JSON_VALUE` for leaf strings/numbers, `JSON_QUERY` for nested structures. 3. Warn about the type mismatch trap — calling `JSON_VALUE` on an object path returns `NULL` (in lax mode) or throws (in strict mode). 4. Mention `WITH (STRICT)` mode in the path: `'strict $.field'` raises an error on missing keys instead of returning `NULL`.

Edge cases to mention: - `JSON_VALUE` returns `NVARCHAR(4000)` by default; long strings get truncated. Cast or use `JSON_QUERY` if you need the full payload. - Index a JSON path by adding a computed column and indexing it — there's no native JSON index. - `ISJSON(col)` validates a string is parseable JSON — use it as a `CHECK` constraint.

What NOT to say: - Do not say "SQL Server JSON is as fast as Postgres jsonb" — it's text-based parsing, slower for path access. - Do not use `JSON_VALUE` to extract an object — it returns `NULL` silently in lax mode and you'll debug for an hour.

Common follow-up: "How would you index a JSON path in SQL Server?" — Add `ALTER TABLE t ADD plan_path AS JSON_VALUE(preferences, '$.plan')` then `CREATE INDEX ON t(plan_path)`.

```
-- SQL Server
SELECT id,
       JSON_VALUE(preferences, '$.language')      AS lang,
       JSON_QUERY(preferences, '$.notifications') AS notif_obj
FROM users
WHERE JSON_VALUE(preferences, '$.plan') = N'pro';
```

Q399 · SQL Server `OPENJSON` — Shredding to Rows

Tags: Difficulty: Hard · Asked at: Wells Fargo, JP Morgan, Walmart Labs · Topic: JSON

The question (interviewer's verbatim phrasing):

SQL Server equivalent of `JSON_TABLE` — how do you turn a JSON array into rows?

The 30-second answer: `OPENJSON(col, '$.path')` returns a derived table. With an explicit `WITH (col_name type 'path', ...)` clause it maps JSON fields to typed columns. Cross apply it to the parent table to get row-per-element output.

Step-by-step thought process: 1. Show the basic form: `OPENJSON(line_items)` returns three columns — key, value, type — useful for ad-hoc inspection. 2. Show the typed form: `OPENJSON(line_items) WITH (sku NVARCHAR(64) '$.sku', qty INT '$.qty')` — gives you the column-mapped derived table. 3. Use `CROSS APPLY` to join with the parent — same pattern as `JSON_TABLE`. 4. Mention `AS JSON` modifier: a column path can end with `AS JSON` to return the nested fragment as JSON rather than scalar.

Edge cases to mention: - `OPENJSON` without `WITH` returns the key column as text — `[0]`, `[1]` for arrays, key names for objects. - Available since SQL Server 2016; Azure SQL fully supports it. - For deeply nested arrays, you chain `OPENJSON ... CROSS APPLY OPENJSON(value)`.

What NOT to say: - Do not say "this is like `STRING_SPLIT`" — `OPENJSON` is far richer. - Do not omit the typed `WITH` clause for production queries — you lose type safety.

Common follow-up: "Can you use `OPENJSON` to feed an `INSERT`?" — Yes, bulk JSON ingestion is a common pattern: `INSERT INTO line_items SELECT order_id, sku, qty FROM orders CROSS APPLY OPENJSON(...) WITH (...)`.

```
-- SQL Server
SELECT o.id AS order_id, items.sku, items.qty
FROM orders o
CROSS APPLY OPENJSON(o.line_items)
  WITH (
    sku NVARCHAR(64) '$.sku',
    qty INT          '$.qty'
  ) AS items;
```

Q400 · SQL Server 2022 `JSON_OBJECT` and `JSON_ARRAY`

Tags: Difficulty: Medium · Asked at: Wells Fargo, AmEx, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

SQL Server 2022 added new JSON constructor functions. What problem do they solve and how do you use them?

The 30-second answer: `JSON_OBJECT('key': value, ...)` and `JSON_ARRAY(value, ...)` build JSON values inline — no more string concatenation or `FOR JSON` workarounds. They are part of the SQL:2016 standard syntax and finally match what Postgres and Oracle have had for years.

Step-by-step thought process: 1. Start with the pain — before 2022, building JSON in SQL Server meant string concatenation or `FOR JSON PATH` which is verbose for small documents. 2. Show `JSON_OBJECT('id': id, 'name': name, 'email': email)` — keyword-style key:value pairs. 3. Show `JSON_ARRAY(sku1, sku2, sku3)` for arrays. 4. Mention `ABSENT ON NULL` and `NULL ON NULL` — controls whether NULL values produce a key or are omitted. Default is `ABSENT`.

Edge cases to mention: - Available only on SQL Server 2022 and Azure SQL — earlier versions still need `FOR JSON PATH`. - Output type is `NVARCHAR(MAX)` — same as any other JSON in SQL Server. - For aggregating into a JSON array, the standard's `JSON_ARRAYAGG` is not yet in SQL Server 2022; you still need `FOR JSON`.

What NOT to say: - Do not say "this is how SQL Server has always done JSON building" — it's a 2022 feature, candidates often miss the version. - Do not confuse `JSON_OBJECT` with `OPENJSON` — opposite directions (build vs shred).

Common follow-up: "How would you do this in Postgres?" — `json_build_object('id', id, 'name', name)` or `jsonb_build_object(...)` — same idea, comma-separated key-value pairs.

```
-- SQL Server 2022
SELECT JSON_OBJECT(
    'id': id,
    'name': name,
    'email': email ABSENT ON NULL
) AS user_doc
FROM users
WHERE id = 1042;
```

Q401 · Generated/Computed Column from JSON + Index

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Walmart Labs · Topic: JSON

The question (interviewer's verbatim phrasing):

I want fast equality filters on a key inside a JSON column but I don't want to expression-index every query path. What pattern do you reach for?

The 30-second answer: Add a generated column that extracts the JSON field, then index that column normally. Postgres calls them generated columns; MySQL calls them generated columns; SQL Server

calls them computed columns. Works in all three engines and gives you a clean, indexable, queryable field.

Step-by-step thought process: 1. Lead with the pattern: extract once at write-time, store, index, query. 2. Show Postgres: `ALTER TABLE users ADD COLUMN plan TEXT GENERATED ALWAYS AS (preferences->>'plan') STORED`. Stored means it's materialised on disk, so indexable. 3. Show MySQL: same idea — `ALTER TABLE users ADD COLUMN plan VARCHAR(32) AS (preferences->>'$.plan') STORED`. 4. Mention SQL Server: `ALTER TABLE users ADD plan AS JSON_VALUE(preferences, '$.plan') PERSISTED` — `PERSISTED` is the keyword.

Edge cases to mention: - `STORED` / `PERSISTED` means the value is computed once on write and re-stored on update — eats disk space but reads are free. - Non-stored (virtual) generated columns are computed at query time — saves disk but is slower for indexed reads. - Schema migrations on huge tables are painful — adding a stored generated column rewrites the table in most engines.

What NOT to say: - Do not say "this is the same as an expression index" — generated columns let you reference the field in queries by name, not just match an exact expression. - Do not skip `STORED` if you plan to index — virtual columns in MySQL cannot be indexed (in Postgres, expression indexes on virtual columns are fine).

Common follow-up: "What's the tradeoff vs an expression index?" — Generated columns are queryable by name everywhere; expression indexes are smaller and don't change the table layout. For one or two fields, either works; for many, prefer generated columns for readability.

```
-- Postgres
ALTER TABLE users
  ADD COLUMN plan TEXT GENERATED ALWAYS AS (preferences->>'plan') STORED;

CREATE INDEX users_plan_idx ON users (plan);

SELECT id FROM users WHERE plan = 'pro';
```

Q402 · `JSON_AGG` and `JSONB_AGG` — Building JSON Arrays

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Meesho · Topic: JSON

The question (interviewer's verbatim phrasing):

Return one row per user with a JSON array of all their order ids. Show me the Postgres query.

The 30-second answer: Use `jsonb_agg(o.id)` in a `GROUP BY` query: `SELECT u.id, jsonb_agg(o.id) AS order_ids FROM users u LEFT JOIN orders o ON o.user_id =`

`u.id GROUP BY u.id` . `jsonb_agg` collects values into a JSONB array; `json_agg` is the same for `json` .

Step-by-step thought process: 1. Open with the canonical use case: per-parent aggregation into JSON for API responses. 2. Show the aggregate: `jsonb_agg(o.id)` produces a JSONB array. Combine with `jsonb_build_object` to get arrays of objects, not just scalars. 3. Mention `FILTER (WHERE ...)` — `jsonb_agg(o.id) FILTER (WHERE o.status = 'paid')` aggregates only matching rows. 4. Warn about LEFT JOIN with no matches: `jsonb_agg` returns `[null]` if there are no rows — wrap in `COALESCE(jsonb_agg(o.id) FILTER (WHERE o.id IS NOT NULL), '[]'::jsonb)` .

Edge cases to mention: - Ordering inside the aggregate: `jsonb_agg(o.id ORDER BY o.created_at)` — predictable array order. - For very wide aggregations, the resulting JSONB document can be huge — paginate at the row level instead. - `string_agg` plus manual JSON formatting is the old-school workaround for Postgres versions before 9.5; nobody should write that today.

What NOT to say: - Do not say "use the application layer to build the array" — the database does this faster for medium-sized result sets. - Do not forget the `GROUP BY` — naked `jsonb_agg` over the whole table returns a single mega-array.

Common follow-up: "Now return objects not just ids." —

`jsonb_agg(jsonb_build_object('id', o.id, 'total', o.total_paise))` .

```
SELECT u.id,
       COALESCE(
         jsonb_agg(jsonb_build_object('id', o.id, 'total', o.total_paise)
                   ORDER BY o.created_at)
         FILTER (WHERE o.id IS NOT NULL),
         '[]'::jsonb
       ) AS orders
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id;
```

Q403 · `json_build_object` and `jsonb_build_object`

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Zerodha, Cred · Topic: JSON

The question (interviewer's verbatim phrasing):

How do you build a JSON object inline from columns in Postgres?

The 30-second answer:

`jsonb_build_object('id', id, 'name', name, 'email', email)` — alternating key, value, key, value. Returns a JSONB object. Use `json_build_object` for the text variant.

Step-by-step thought process: 1. Start with the function signature: variadic, alternating keys and values. 2. Show how it composes with `jsonb_agg` for one-shot API-response construction. 3. Contrast with `row_to_json(t)` — which converts an entire row into a JSON object using column names — useful when you want every column without listing them. 4. Mention the `ABSENT ON NULL` -style trick — Postgres has `jsonb_strip_nulls(...)` you wrap around the result to drop null keys.

Edge cases to mention: - Keys must be text; values can be any type the function knows how to serialise (numbers, booleans, arrays, other JSONB). - Odd number of arguments raises an error: every key needs a value. - For dynamic keys (column name unknown), use `jsonb_object(text[], text[])` — takes parallel arrays of keys and values.

What NOT to say: - Do not concatenate JSON strings with `||` and string functions — quoting and escaping will go wrong. - Do not assume key order in the output — `jsonb` does not preserve order; `json` does.

Common follow-up: "What's the MySQL equivalent?" — `JSON_OBJECT('id', id, 'name', name)` — same alternating form.

```
SELECT jsonb_build_object(
  'id',    id,
  'name',  name,
  'email', email
) AS user_doc
FROM users
WHERE id = 1042;
```

Q404 · JSON_OBJECTAGG — The SQL Standard Aggregator

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan, AmEx · Topic: JSON

The question (interviewer's verbatim phrasing):

What does `JSON_OBJECTAGG` do and which engines support it?

The 30-second answer: `JSON_OBJECTAGG(key VALUE value)` is the SQL:2016 standard aggregator that turns a set of key-value rows into a single JSON object. MySQL 5.7.22+, SQL Server 2022, Oracle 12c+ implement it; Postgres uses `jsonb_object_agg(key, value)` instead.

Step-by-step thought process: 1. Start with the use case: given a `settings` table with `(user_id, key, value)`, you want one JSON object per user with all their key-value pairs. 2. Show the standard syntax: `JSON_OBJECTAGG(setting_key VALUE setting_value)` grouped by user. 3. Note dialect differences — Postgres `jsonb_object_agg(setting_key, setting_value)` does the same job with comma syntax. 4. Warn about duplicate keys — behaviour varies by engine. MySQL keeps the last value; Postgres `jsonb_object_agg` keeps the last as well; SQL Server raises by default.

Edge cases to mention: - If a key is NULL the aggregator typically raises an error — filter or coalesce first. - For nested values you usually wrap each row's value in another JSON builder before aggregating. - Standard syntax (`KEY ... VALUE ...`) is recognised by very few client tools — IDE auto-format may mangle it.

What NOT to say: - Do not say "all engines support `JSON_OBJECTAGG`" — Postgres prefers its own function name and signature. - Do not assume key order — output ordering is implementation-defined.

Common follow-up: "Write the Postgres version." — `SELECT user_id, jsonb_object_agg(setting_key, setting_value) FROM settings GROUP BY user_id .`

```
-- MySQL / SQL Server 2022 / Oracle
SELECT user_id,
       JSON_OBJECTAGG(setting_key VALUE setting_value) AS settings
FROM user_settings
GROUP BY user_id;
```

Q405 · Comparing JSON Values — Semantic vs Textual Equality

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

Are these two JSON documents equal? `{"a":1,"b":2}` vs `{"b":2,"a":1}` . How does your database decide?

The 30-second answer: Semantically yes — they have the same keys with the same values. Postgres `jsonb` says yes because it stores normalised binary; `json` (text) says no because order differs. MySQL `JSON` and SQL Server `NVARCHAR` also do semantic equality via `JSON_EQUALS` (SQL Server 2022) or implicit comparison.

Step-by-step thought process: 1. Open with the rule: equality of JSON values is by semantic structure, not by string form — same keys with same values, regardless of order. 2. Show the Postgres difference: `'{"a":1,"b":2}'::jsonb = '{"b":2,"a":1}'::jsonb` is true; the same comparison on

`json` is false because it's string compare. 3. Mention MySQL `=` on `JSON` columns is semantic; SQL Server 2022 added `JSON_EQUALS` for explicit semantic compare. 4. Warn about numeric edge cases: `1` vs `1.0` vs `1e0` — Postgres `jsonb` treats them as equal numbers; MySQL also normalises numerics; some engines treat them as text-different. Always test on your engine.

Edge cases to mention: - Duplicate keys: `jsonb` keeps the last and dedupes, so `{"a":1,"a":2}` equals `{"a":2}`. `json` keeps both, so equality is by text. - Whitespace and key order do not affect `jsonb` equality but break `json` /text equality. - `NULL` JSON values vs SQL NULL — `'null'::jsonb` is a JSON null, not SQL NULL — `=` on two JSON nulls is true; SQL NULLs compare unknown.

What NOT to say: - Do not say "JSON equality is always by string" — false for the binary types. - Do not write `WHERE col::text = '{"a":1}'` — fragile to whitespace and ordering.

Common follow-up: "How would you compare just one key?" — Extract the key with `->` or `->>` and compare scalars: `WHERE col->>'plan' = 'pro'`.

```
SELECT '{"a":1,"b":2}'::jsonb = '{"b":2,"a":1}'::jsonb AS semantic_eq;
-- semantic_eq = true

SELECT '{"a":1,"b":2}'::json::text = '{"b":2,"a":1}'::json::text AS text_eq;
-- text_eq = false
```

Q406 · JSON Document Size Limits — When to Denormalize

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

A team is storing user activity history as a JSON array that keeps growing. What goes wrong as the array gets large and when do you split it out?

The 30-second answer: Once a JSON document exceeds the row-inline threshold (around 8KB in Postgres), it gets TOASTed — stored out-of-line, so every read pays an extra fetch. Past a few hundred KB, partial updates become slow because the entire document is rewritten on each `jsonb_set`. Split into a child table once you're appending more than a few hundred rows' worth of data per parent.

Step-by-step thought process: 1. Start with the practical thresholds: Postgres rows over 8KB go to TOAST; MySQL JSON has a hard limit of `max_allowed_packet` (16MB typical); SQL Server `NVARCHAR(MAX)` is 2GB but performance is awful at scale. 2. Explain the write penalty: every update of a JSONB column rewrites the full value, not just the changed bytes — even with `jsonb_set`. Bigger doc = more I/O per write. 3. Explain the read penalty: TOASTed JSONB requires a separate page fetch;

for queries that only need a few fields, that's wasted I/O. 4. Give the migration rule: if you're appending elements indefinitely (activity log, audit trail), it should be a child table from day one. JSON arrays are for bounded, semantic groupings.

Edge cases to mention: - Postgres `EXTERNAL` storage strategy keeps TOASTed values uncompressed for faster access — useful if your read path is hot but writes are rare. - Compressed (default) saves disk but adds CPU on every read; profile to decide. - For very large append-only arrays, consider `array_append` semantics in a child table with proper indexes instead.

What NOT to say: - Do not say "JSON columns scale infinitely." Practical issues hit well before any documented limit. - Do not claim TOAST is invisible — it has real read latency implications.

Common follow-up: "How would you migrate without downtime?" — Create the child table, dual-write each new event, backfill the historical array in batches, then drop the JSON column.

```
-- Bad pattern: ever-growing JSON array
UPDATE users
SET activity = activity || jsonb_build_array(jsonb_build_object('ts', now(), 'event', 'login'))
WHERE id = 1042;

-- Good pattern: child table
INSERT INTO user_events (user_id, ts, event) VALUES (1042, now(), 'login');
```

Q407 · Schema Validation on JSON — CHECK + `jsonb_path_match`

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman · Topic: JSON

The question (interviewer's verbatim phrasing):

JSON columns are schemaless by default. How do you enforce that a `preferences` JSONB always has a `language` key with one of `en`, `hi`, `ta`, `te`?

The 30-second answer: Use a `CHECK` constraint with `jsonb_path_match` or a simpler expression: `CHECK (preferences->>'language' IN ('en','hi','ta','te'))`. For richer schemas use `jsonb_path_match` with SQL/JSON path predicates, or install a JSON Schema extension like `pg_jsonschema`.

Step-by-step thought process: 1. Lead with the truth: SQL JSON support has no built-in schema. You enforce shape via `CHECK` constraints. 2. Show the simple form: `CHECK ((preferences->>'language') AND (preferences->>'language' IN ('en','hi','ta','te')))`. 3. Show the path form: `CHECK (jsonb_path_match(preferences, '$.language ? (@ == "en" || @ == "hi" || @ == "ta" || @ == "te")))`. 4. Mention extensions: `pg_jsonschema` and

`pg_jsonschema-rs` validate full JSON Schema documents inside a constraint — useful for complex shapes.

Edge cases to mention: - `CHECK` constraints fire on every write — heavy validation can hurt insert throughput. - `NOT VALID` lets you add a constraint without checking existing rows, then `VALIDATE CONSTRAINT` runs the check later in a less locking-heavy way. - For schema versioning, store a `schema_version` key inside the JSON and switch validation rules in app code or via partial check constraints.

What NOT to say: - Do not say "validate in the application" as the only answer — that breaks under concurrent writers or direct DB tools. - Do not invent a `JSON SCHEMA` keyword — it isn't standard SQL.

Common follow-up: "How would MySQL or SQL Server do this?" — Both support `CHECK` constraints. MySQL: `CHECK (JSON_VALID(col) AND JSON_EXTRACT(col, '$.language') IN ('en','hi','ta','te'))`. SQL Server: `CHECK (ISJSON(col) = 1 AND JSON_VALUE(col, '$.language') IN ('en','hi','ta','te'))`.

```
ALTER TABLE users
ADD CONSTRAINT preferences_language_valid
CHECK (
    (preferences ? 'language')
    AND (preferences->>'language' IN ('en','hi','ta','te'))
);
```

Q408 · Querying Nested JSON Arrays

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart · Topic: JSON

The question (interviewer's verbatim phrasing):

Each order document has `items: [{sku, qty, amount_paise}, ...]`. Find orders that contain at least one item with `amount_paise > 50000` AND total item count `>= 3`.

The 30-second answer: Combine `jsonb_array_length` with `jsonb_path_exists` (Postgres) — `WHERE jsonb_array_length(payload->'items') >= 3 AND jsonb_path_exists(payload, '$.items[*] ? (@.amount_paise > 50000)')`. Or shred with `jsonb_array_elements` in a `LATERAL` subquery and aggregate.

Step-by-step thought process: 1. Identify the two checks: array length and per-element predicate. 2. Show the path-based form first because it's the most direct. 3. Show the `LATERAL` form as an alternative when you need to project per-item values: `CROSS JOIN LATERAL`

`jsonb_array_elements(payload->'items') AS item` . 4. Mention indexing strategy: a GIN index with `jsonb_path_ops` accelerates the `jsonb_path_exists` filter; the count check is hard to index, accept a sequential portion.

Edge cases to mention: - `jsonb_array_length` on a non-array raises an error — guard with `jsonb_typeof(col) = 'array'` if you can't trust the shape. - `LATERAL` produces multiple output rows per parent row — remember to `DISTINCT` or aggregate if you only want parents. - MySQL: `JSON_LENGTH(payload, '$.items')` and `JSON_CONTAINS(payload->'$.items', ...)` are the analogues; for predicate-style queries, `JSON_TABLE` is the readable form.

What NOT to say: - Do not pull every row into the app and filter there — pushdown belongs in SQL. - Do not assume nested JSON is "free" — every query re-parses unless you have suitable indexes.

Common follow-up: "How would you find the highest-amount item across all orders?" — `LATERAL jsonb_array_elements(...)` plus `MAX((item->>'amount_paise')::bigint)` .

```
-- Postgres
SELECT id
FROM orders
WHERE jsonb_array_length(payload->'items') >= 3
      AND jsonb_path_exists(
          payload,
          '$.items[*] ? (@.amount_paise > 50000)'
      );
```

Q409 · Updating Array Elements Inside JSON

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, JP Morgan · Topic: JSON

The question (interviewer's verbatim phrasing):

An order's `items` JSON array has objects with `sku` and `qty` . Update only the entry where `sku = 'SKU123'` to set `qty = 5` . How do you do it without rewriting the whole array?

The 30-second answer: There's no native "update where" for JSON arrays. The clean pattern is: unnest the array with `jsonb_array_elements` , modify the target element, re-aggregate with `jsonb_agg` , and write back. Or `jsonb_set` by index if you know the position.

Step-by-step thought process: 1. Open with the honest truth: SQL JSON support does not have a "update array element matching predicate" operator. You rebuild. 2. Show the unnest-modify-aggregate pattern with `jsonb_agg(CASE WHEN elem->>'sku' = 'SKU123' THEN jsonb_set(elem, '{qty}', '5') ELSE elem END)` . 3. Show the index form: if you know the position,

`jsonb_set(items, '{2,qty}', '5')` updates the third element's `qty` directly — fast but brittle to ordering. 4. Mention the concurrency risk — two simultaneous updates to different items in the same array will conflict because the whole array is rewritten each time. Use row-level lock (`SELECT ... FOR UPDATE`) or push the items into a child table.

Edge cases to mention: - The unnest-aggregate pattern loses array order unless you carry an explicit ordinality column: `WITH ORDINALITY` plus `ORDER BY ord` in `jsonb_agg`. - For very large arrays this rewrites the whole document on every update — that's the underlying scalability ceiling. - MySQL has `JSON_SET`, `JSON_REPLACE` and `JSON_ARRAY_APPEND` but the same "filter by predicate" gap.

What NOT to say: - Do not pretend `jsonb_set` accepts a predicate — it only takes paths. - Do not loop in application code over the array — that's slower and unsafe under concurrent writes.

Common follow-up: "If items get updated frequently, what would you recommend?" — Move items to a child `order_items` table — proper row locks, indexes, partial updates without rewrites.

```
UPDATE orders
SET payload = jsonb_set(
  payload,
  '{items}',
  (
    SELECT jsonb_agg(
      CASE WHEN elem->>'sku' = 'SKU123'
      THEN jsonb_set(elem, '{qty}', '5'::jsonb)
      ELSE elem
    )
    FROM jsonb_array_elements(payload->'items') elem
  )
)
WHERE id = 9001;
```

Q410 · JSON in Primary Key — Avoid

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Goldman · Topic: JSON

The question (interviewer's verbatim phrasing):

Can you use a JSON column or a value extracted from JSON as a primary key? Should you?

The 30-second answer: Technically you can — Postgres lets you create a unique btree index on `((col->>'id'))` and use it as a logical PK; MySQL needs a generated column for the same. But you

almost certainly should not — JSON identity is fragile, indexes get bigger, foreign keys can't point inside JSON, and ORMs choke.

Step-by-step thought process: 1. Start with "can": Postgres `CREATE UNIQUE INDEX ON t ((payload->>'id'))` plus a `NOT NULL` constraint via `CHECK` gives you a logical unique key. 2. Move to "should not": foreign keys cannot reference an expression index — every downstream join becomes a manual constraint. 3. Add the practical points: JSON path keys are text not numbers, so btree comparisons are slower than `bigint`; replication and backups treat JSON columns as a single value, complicating partial sync; ORM tools don't model JSON-path PKs. 4. Show the alternative — extract the field into a real column at write time and PK that column. Cleanest design.

Edge cases to mention: - For analytical event stores where the row will never be referenced, a JSON-derived unique index for dedup is fine. - Composite keys mixing real and JSON-derived columns are extra messy; index size and update cost both balloon. - Some teams use a hashed JSON path as a surrogate key for idempotency — acceptable for dedup but not as a real PK.

What NOT to say: - Do not say "primary keys cannot contain JSON" — that's an oversimplification, the keys can exist via expression indexes. - Do not propose a JSON-derived PK in an interview — it signals weak schema instincts.

Common follow-up: "What if I really want idempotent inserts based on a JSON field?" — Use a unique expression index for dedup, but keep the actual PK as `bigserial` / `uuid`.

```
-- Possible but discouraged
CREATE UNIQUE INDEX events_event_id_unique
ON events ((payload->>'event_id'));

-- Better
ALTER TABLE events
ADD COLUMN event_id TEXT GENERATED ALWAYS AS (payload->>'event_id') STORED,
ADD CONSTRAINT events_event_id_unique UNIQUE (event_id);
```

Q411 · JSON Columns and Replication

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Walmart Labs · Topic: JSON

The question (interviewer's verbatim phrasing):

Do JSON columns play nicely with logical replication and CDC pipelines? Any gotchas?

The 30-second answer: Mostly yes, but with two gotchas: every JSON update writes the full new document to the WAL (no partial diff), and JSON's binary representation may not be byte-stable across

versions, so downstream consumers should compare semantically. Tools like Debezium handle JSON columns natively but emit the entire new value on every change.

Step-by-step thought process: 1. Start with the WAL behaviour: `UPDATE ... SET col = jsonb_set(col, ...)` writes the entire new value to WAL — the diff is reconstructed by the planner but the replication payload is the full new column. 2. Explain the impact on replication bandwidth: large JSON documents updated frequently produce large WAL volumes and long replica lag. 3. Mention CDC tooling: Debezium, AWS DMS, Fivetran all support Postgres JSONB and MySQL JSON; they typically deliver the after-image as a JSON object on the wire. 4. Warn about cross-version comparison: `jsonb` binary format is generally stable within a major version, but ordering of object keys and whitespace handling can shift; downstream comparisons must be semantic.

Edge cases to mention: - For row-level replication with very large JSON documents, consider `REPLICA IDENTITY FULL` carefully — it writes the old row too, doubling WAL volume. - Logical replication slots can fall behind during JSON-heavy write storms — monitor `pg_replication_slots.confirmed_flush_lsn`. - For sub-document change feeds (only emit changed keys), build it in app code or in the CDC consumer; the database doesn't natively diff JSON.

What NOT to say: - Do not claim JSON updates are "partial" at the storage level — they aren't. - Do not say "Debezium can't handle JSONB" — it has had first-class support for years.

Common follow-up: *"How would you reduce WAL volume from JSON-heavy writes?"* — Split frequently-updated keys into separate small columns, batch updates so each commit covers many keys at once, or move hot fields out of JSON entirely.

```
-- This logs the entire new preferences blob to WAL even though one key changed
UPDATE users
SET preferences = jsonb_set(preferences, '{theme}', '"dark"')
WHERE id = 1042;
```

Q412 · JSON Columns and ORM Mapping

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Meesho · Topic: JSON

The question (interviewer's verbatim phrasing):

How do popular ORMs handle JSON columns and what's the typical bug you see in production?

The 30-second answer: Modern ORMs map JSON columns to dicts, hashes or typed objects — Django `JSONField`, SQLAlchemy `JSON/JSONB`, Hibernate `@JdbcTypeCode(SqlTypes.JSON)`, Rails `serialize :col, JSON`. The classic bug is the ORM marking the row dirty on every read because the in-memory dict equality check differs from the database's normalised value.

Step-by-step thought process: 1. Start with the mapping landscape: every mainstream ORM has first-class JSON support today, but the type is mapped to a mutable object. 2. The big bug — mutability: you call `user.preferences['theme'] = 'dark'`, and depending on the ORM, the change may not be detected because the dict identity didn't change. SQLAlchemy needs

`MutableDict.as_mutable(JSONB)` for the change to be picked up. 3. The second big bug — equality drift: ORM reads the row, the dict has keys in one order; DB roundtrip reorders; next save thinks the value changed. Triggers spurious updates. 4. Validation pattern: define a Pydantic / Marshmallow / DTO schema in app code, validate before assignment, store as plain JSON.

Edge cases to mention: - Some ORMs serialise NaN/Infinity that JSON spec doesn't allow — the DB rejects with a syntax error. - Lazy loading large JSON columns can mask N+1 problems if the column is accessed in a loop. - For schema migrations adding fields to JSON, you don't need a DB migration — just app-side defaulting. Tradeoff: no validation at the storage layer unless you add `CHECK` constraints.

What NOT to say: - Do not claim "JSON columns don't work with ORMs" — they all support it. - Do not store JSON as text-typed `String` columns to avoid ORM issues — you lose database-side validation and query operators.

Common follow-up: "How do you avoid the mutability bug in SQLAlchemy?" — Wrap with `MutableDict.as_mutable(JSONB)` or always assign a new dict (`user.preferences = {**user.preferences, 'theme': 'dark'}`) rather than mutating in place.

```
# SQLAlchemy — explicit mutable tracking
from sqlalchemy.ext.mutable import MutableDict
from sqlalchemy.dialects.postgresql import JSONB

preferences = Column(MutableDict.as_mutable(JSONB), default={})
```

Q413 · `jsonb_set` Partial Update vs Replace-Whole

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred · Topic: JSON

The question (interviewer's verbatim phrasing):

When should you use `jsonb_set` for a partial update versus just replacing the whole document?

The 30-second answer: Use `jsonb_set` when one writer is touching a few known keys — it preserves the other keys safely. Replace the whole document when you have a single owner and want a clean, version-controlled write. Mixing both patterns from multiple writers is the lost-update trap.

Step-by-step thought process: 1. Open with the concurrency angle: two writers using `jsonb_set` on different keys race, last-writer-wins on the whole column — neither side knows the other's change. 2. Argue for `jsonb_set` when the field has multiple sub-owners with non-overlapping keys, each updates only their slice, and the document is small enough that the rewrite cost is acceptable. 3. Argue for replace-whole when there is a single owner (a settings page) that always loads, edits and saves the full document — simpler, no surprise key changes. 4. Mention the optimistic-locking pattern: store a `version` integer at the row level, increment on every write, use `WHERE ... AND version = :expected` so stale writes fail.

Edge cases to mention: - `jsonb_set` will create intermediate keys when the `create_if_missing` flag is true, which can hide schema typos — set the flag to false in strict pipelines. - For nested updates, paths get long; the `||` (concat) operator is cleaner for replacing whole subtrees: `preferences || '{"notifications": {...}}'::jsonb`. - Even with `jsonb_set`, the storage write is the full document — there's no save in I/O, only safety against other keys.

What NOT to say: - Do not say "`jsonb_set` is cheaper because it's partial" — at the storage layer it is not. - Do not advise replace-whole when you have multiple independent writers — you'll lose their writes.

Common follow-up: "How do you make concurrent JSON updates safe?" — Either move the hot subfields out into real columns with their own row locks, or use optimistic locking with a version column.

```
-- Concurrent-safe partial update with optimistic lock
UPDATE users
SET preferences = jsonb_set(preferences, '{theme}', '"dark"'),
    version      = version + 1
WHERE id = 1042
  AND version = 7;
```

Q414 · Reading JSON with NULL or Missing Keys

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Flipkart, Wells Fargo · Topic: JSON

The question (interviewer's verbatim phrasing):

What happens when you read a JSON key that doesn't exist or whose value is JSON `null` ?
Show me the SQL behaviour and how you'd handle it.

The 30-second answer: Missing keys return SQL `NULL`. JSON `null` values return SQL `NULL` via `->>` (text path) but return JSON `null` via `->` (JSON path). Use `COALESCE` to give defaults, and use `?` (Postgres) or `JSON_CONTAINS_PATH` (MySQL) to distinguish "missing" from "explicitly null".

Step-by-step thought process: 1. Lead with the SQL behaviour: both missing-key access and JSON `null` typically coerce to SQL `NULL` — they look the same in WHERE clauses. 2. Show how to distinguish: Postgres `col ? 'key'` returns true if the key exists (even if null), false if missing. MySQL `JSON_CONTAINS_PATH(col, 'one', '$.key')` is the equivalent. 3. Defaulting: `COALESCE(preferences->>'language', 'en')` — provides a fallback for both missing and null cases. 4. Warn about the silent-bug: filters like `WHERE preferences->>'plan' = 'free'` miss rows where the key is absent, even if intuitively you want to treat them as free.

Edge cases to mention: - JSON `null` vs SQL `NULL` confusion: `'null'::jsonb` is a valid JSON value (the JSON null literal); `jsonb_typeof('null'::jsonb)` returns `'null'`. Comparisons must use `IS NULL` carefully. - For "missing" semantics, prefer the explicit existence check (`?`) over relying on `IS NULL` of an extracted value. - The `?|` and `?&` operators check for "any of these keys" and "all of these keys" — handy for multi-key checks.

What NOT to say: - Do not say "missing keys throw errors" — they silently return NULL in all major engines. - Do not assume your filter handles missing keys correctly without testing — write the test.

Common follow-up: "How do you distinguish JSON null from a missing key?" — Use the existence operator: `(col ? 'key') AND (col->>'key' IS NULL)` means present-and-null; `NOT (col ? 'key')` means missing.

```
SELECT id,
       COALESCE(preferences->>'language', 'en') AS lang,
       (preferences ? 'plan')                  AS has_plan_key
FROM users;
```

Q415 · Store User Preferences — JSON Column or Separate Table?

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Meesho, Flipkart · Topic: JSON

The question (interviewer's verbatim phrasing):

Classic interview prompt — you need to store user preferences (theme, language, notification settings, feature flags). Would you use a JSON column or a separate preferences table? Defend your choice.

The 30-second answer: JSON column for user preferences — they're per-user, sparse, schema-light, almost never queried by content, and read together as one blob on login. A separate `(user_id, key, value)` table is over-engineering for this use case; a separate column-per-preference adds migration pain every time product adds a setting.

Step-by-step thought process: 1. Start with the access pattern: preferences are read in one go (login, app start) and written by the user themselves, very rarely queried by other users or aggregated. JSON column is the right tool. 2. Justify against the EAV (entity-attribute-value) table: a `user_preferences (user_id, key, value)` table looks "more normalised" but adds JOIN overhead on every read, makes "give me all preferences" a multi-row fetch, and requires app code to coerce types per key. 3. Justify against column-per-preference: every new toggle requires a DDL migration, the table grows wide with mostly-null columns, and `SELECT *` brings dead weight. JSON sidesteps the schema churn. 4. Add the operational notes: use `jsonb` (Postgres) or `JSON` (MySQL) with a `CHECK` constraint for top-level schema, a default of `'{}'`, and a generated column for any field you actually need to filter on (rarely needed for preferences).

Edge cases to mention: - If preferences ever become *shared* (team-level settings, admin overrides), the EAV table is worth reconsidering for cross-user queries. - If a single preference becomes a hot filter ("show me all pro users"), promote that one field to a real column with an index — don't fight the JSON column. - For very fine-grained audit ("who changed which key when"), a separate audit table records each change with delta — keeps the JSON simple.

What NOT to say: - Do not say "JSON is the modern way, EAV is dead" — there are real cases for EAV; the answer depends on access pattern. - Do not propose putting hot business fields (`status`, `plan`, `tier`) inside the same JSON — those belong as columns.

Common follow-up: "How do you handle backward compatibility when you rename a preference?" — Dual-write the old and new keys for a release, read both with a precedence rule, then run a background migration to copy and drop the old key.

```
-- Recommended shape
CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,
  email       TEXT NOT NULL UNIQUE,
  preferences JSONB NOT NULL DEFAULT '{} '::jsonb,
  CONSTRAINT preferences_is_object CHECK (jsonb_typeof(preferences) = 'object')
);

-- One-shot read on login
SELECT preferences FROM users WHERE id = 1042;
```

Chapter 10, Part B — MERGE, UPSERT, LATERAL & RETURNING

Modern SQL has finally caught up with what application developers were already doing in messy procedural code: atomic upserts, per-row correlated lookups, capturing what just got written. These are the DML patterns that surface heavily in product-company rounds at Razorpay, Swiggy, PhonePe, Cred and the GCC teams at Walmart and Target — partly because they cleanly separate candidates who write SQL daily from those who only remember it from college. MERGE went GA in Postgres 15 after years of users asking for it, MERGE on SQL Server still has the concurrency bugs Aaron Bertrand wrote about a decade ago, and ON CONFLICT remains the Postgres-native idiom that most senior engineers in India

actually reach for. This part of the chapter walks through every dialect's flavour, the pitfalls each one hides, and the LATERAL / RETURNING tricks that make modern SQL feel less like a 1992 standard and more like a real language.

Section A — MERGE and UPSERT

Q416 · MERGE Statement — The SQL:2003 Standard Syntax

Tags: Difficulty: Medium · Asked at: Walmart Labs, Goldman, JP Morgan, Oracle India · Topic: MERGE

The question (interviewer's verbatim phrasing):

Explain the MERGE statement. What problem does it solve and what does the basic syntax look like across dialects?

The 30-second answer: MERGE combines INSERT, UPDATE and DELETE into one atomic statement driven by a join between a target table and a source. You specify `WHEN MATCHED THEN UPDATE` for rows that exist and `WHEN NOT MATCHED THEN INSERT` for new ones — useful when syncing a staging table into a master.

Step-by-step thought process: 1. Frame the use case: nightly sync of a `staging_orders` feed into the canonical `orders` table — some rows are updates, some are brand-new. 2. Walk through the skeleton: `MERGE INTO target t USING source s ON (t.id = s.id) WHEN MATCHED THEN UPDATE SET ... WHEN NOT MATCHED THEN INSERT ...`. 3. Mention that MERGE landed in the SQL:2003 standard but adoption was patchy — Oracle had it first, SQL Server added it in 2008, Postgres only got it in version 15 (October 2022), MySQL still does not have MERGE. 4. Be ready for the follow-up about whether MERGE is atomic — yes, the whole statement is one transaction step, which is exactly why it's used for ETL.

Edge cases to mention: - MERGE on SQL Server has long-documented concurrency bugs — Aaron Bertrand's "MERGE considered harmful" post is required reading. - If the source has duplicate keys for the same target row, MERGE will error on Postgres and silently take the last row on Oracle — behaviour differs. - DELETE inside MERGE (`WHEN MATCHED THEN DELETE`) is supported on SQL Server and Postgres 15+, but the semantics around multiple matches differ.

What NOT to say: - Do not claim "MERGE is concurrency-safe by default" — on SQL Server it is not, and even Postgres needs the right isolation level. - Do not say MySQL has MERGE — it does not; the equivalent there is `INSERT ... ON DUPLICATE KEY UPDATE`.

Common follow-up: "If MERGE has bugs on SQL Server, what would you use instead?" — A serialisable transaction wrapping a separate UPDATE and INSERT, or `INSERT ... WHERE NOT EXISTS` followed by UPDATE, depending on the workload.

```
MERGE INTO orders t
USING staging_orders s ON t.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET amount = s.amount, status = s.status
WHEN NOT MATCHED THEN INSERT (order_id, user_id, amount, status)
VALUES (s.order_id, s.user_id, s.amount, s.status);
```

Q417 · MERGE in Postgres 15 — Why It Took So Long

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postgres-heavy GCCs · Topic: MERGE

The question (interviewer's verbatim phrasing):

Postgres only got MERGE in version 15. Why so late, and how does it compare with ON CONFLICT?

The 30-second answer: Postgres held out because `INSERT ... ON CONFLICT DO UPDATE` already covered the upsert case more safely and used the underlying unique-index machinery to handle concurrency. MERGE finally landed in Postgres 15 for ETL workflows and SQL standard compliance, but ON CONFLICT remains the idiom for true upserts.

Step-by-step thought process: 1. Open with the historical reason: Postgres committers were sceptical of MERGE because the standard's semantics around concurrent modification are weak — ON CONFLICT was deliberately designed to be race-safe using the unique index lock. 2. Note the GA version — Postgres 15, October 2022. Anything older does not have it. 3. Distinguish the use cases: MERGE for bulk source-to-target syncs where you want one statement; ON CONFLICT for single-row or small-batch upserts driven by application code. 4. Mention that MERGE in Postgres 15 does not yet support the full `RETURNING` clause across all branches — that came in Postgres 17.

Edge cases to mention: - Postgres MERGE errors out if two source rows map to the same target row — Oracle silently picks one. Indian product teams have been bitten by this when migrating ETL from Oracle. - Postgres 15 MERGE cannot use `WHEN NOT MATCHED BY SOURCE` — that arrived later. - If you need true atomic upsert with a returning row, ON CONFLICT is still cleaner.

What NOT to say: - Do not say "Postgres MERGE replaces ON CONFLICT" — it does not, and the project explicitly states both have their place. - Do not assume MERGE works on Postgres 14 — it does not. Check the version before writing it in an interview.

Common follow-up: "When would you actually pick MERGE over ON CONFLICT in Postgres?" — When the source is itself a SELECT from another table and you want to express both insert and update in one statement, especially for nightly ETL.


```
-- Postgres 15+
MERGE INTO inventory t
USING daily_stock_feed s ON t.sku = s.sku
WHEN MATCHED AND s.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty, updated_at = NOW()
WHEN NOT MATCHED THEN INSERT (sku, qty) VALUES (s.sku, s.qty);
```

Q418 · MERGE in SQL Server and Its Known Bugs

Tags: Difficulty: Hard · Asked at: JP Morgan, Wells Fargo, Goldman, Walmart Labs · Topic: MERGE

The question (interviewer's verbatim phrasing):

Are you aware of any issues with MERGE on SQL Server? Have you read Aaron Bertrand's article on it?

The 30-second answer: Yes — SQL Server's MERGE has a long list of known bugs around concurrency, deadlocks, filtered indexes and incorrect plans. The community consensus (Aaron Bertrand, Paul White) is to prefer separate INSERT and UPDATE inside an explicit transaction with **HOLDLOCK**.

Step-by-step thought process: 1. Lead with credibility: name the article — "MERGE Considered Harmful" — and the author, Aaron Bertrand. This signals you actually read SQL Server material. 2. List the categories of issues: concurrency races that can cause primary-key violations even with default isolation, optimizer bugs that produce wrong cardinality estimates, problems with filtered unique indexes. 3. State the alternative: **BEGIN TRAN; UPDATE ... WITH (UPDLOCK, HOLDLOCK); IF @@ROWCOUNT = 0 INSERT ...; COMMIT;** — this is the historic safe pattern. 4. Be ready for the follow-up about whether Microsoft has fixed any of these — some have been patched but the community still treats MERGE with suspicion.

Edge cases to mention: - The race is real: under READ COMMITTED with default settings, two sessions can both see "no match" and both try to INSERT — one fails on the unique constraint. - Filtered indexes on the target can confuse MERGE into producing incorrect results — KB articles document this. - Even if you fix concurrency with HOLDLOCK hints, MERGE can deadlock more often than separate statements.

What NOT to say: - Do not say "MERGE is fine on SQL Server, the bugs are old" — many are still open or only partially fixed. - Do not claim to use MERGE in production on SQL Server without mentioning the locking hints.

Common follow-up: "How would you make MERGE concurrency-safe on SQL Server?" — Add **WITH (HOLDLOCK)** on the target table reference; this escalates to a key-range lock that prevents the phantom-insert race.


```
MERGE INTO accounts WITH (HOLDLOCK) AS t
USING (VALUES (@id, @balance)) AS s(id, balance) ON t.id = s.id
WHEN MATCHED THEN UPDATE SET balance = s.balance
WHEN NOT MATCHED THEN INSERT (id, balance) VALUES (s.id, s.balance);
```

Q419 · MERGE in Oracle — The Original Implementation

Tags: Difficulty: Medium · Asked at: Oracle India, Infosys, TCS, Cognizant · Topic: MERGE

The question (interviewer's verbatim phrasing):

Oracle has had MERGE for almost two decades. What does the Oracle dialect look like and what are its quirks?

The 30-second answer: Oracle introduced MERGE in 9i (2001) and the syntax is close to the SQL standard, with two extensions — an optional `DELETE WHERE` clause attached to the WHEN MATCHED branch, and an `ERROR LOGGING` clause for batching error rows aside.

Step-by-step thought process: 1. Note Oracle's long head start — many Indian banking and telecom systems run ETL pipelines that have used Oracle MERGE since the early 2000s. 2. Walk through the dialect-specific extras: `DELETE WHERE` inside `WHEN MATCHED` deletes rows that match a secondary predicate after the UPDATE applies. 3. Mention `LOG ERRORS INTO err_table REJECT LIMIT UNLIMITED` — extremely useful in batch ETL because one bad row does not abort the whole MERGE. 4. Be ready for the duplicate-source-row question: Oracle does not error on duplicates the way Postgres does; it picks an unspecified row, which is a portability gotcha.

Edge cases to mention: - Oracle MERGE allows updating the join column itself — most other dialects disallow this. - If both branches are written but you only need one, Oracle lets you omit either WHEN MATCHED or WHEN NOT MATCHED — handy. - `LOG ERRORS` requires the error log table to be created with `DBMS_ERRLOG.CREATE_ERROR_LOG`.

What NOT to say: - Do not say "Oracle MERGE is identical to ANSI" — it has extensions that won't port. - Do not assume `DELETE WHERE` inside MERGE works on Postgres — it does not; Postgres uses a separate `WHEN MATCHED THEN DELETE` branch.

Common follow-up: "How would you migrate an Oracle MERGE with LOG ERRORS to Postgres?" — You lose the LOG ERRORS feature; the closest pattern is wrapping the MERGE in a savepoint loop or routing failed rows through a trigger.

```
-- Oracle
MERGE INTO customers t
USING new_customers s ON (t.cust_id = s.cust_id)
WHEN MATCHED THEN UPDATE SET t.email = s.email
DELETE WHERE s.status = 'closed'
WHEN NOT MATCHED THEN INSERT (cust_id, email) VALUES (s.cust_id, s.email)
LOG ERRORS INTO merge_errors REJECT LIMIT UNLIMITED;
```

Q420 · ON CONFLICT DO UPDATE — The Postgres Upsert Idiom

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postgres-heavy product teams · Topic: UPSERT

The question (interviewer's verbatim phrasing):

Walk me through `INSERT ... ON CONFLICT DO UPDATE` in Postgres. Why is it considered safer than MERGE for upserts?

The 30-second answer: `ON CONFLICT (col) DO UPDATE SET ...` tells Postgres to attempt the INSERT, and if it would violate the named unique constraint, switch to an UPDATE of the conflicting row instead. It is concurrency-safe because it piggybacks on the unique index lock that the INSERT acquires.

Step-by-step thought process: 1. Start with the syntax: `INSERT INTO t (a, b) VALUES (1, 'x') ON CONFLICT (a) DO UPDATE SET b = EXCLUDED.b`. The `EXCLUDED` pseudo-table holds the row that was about to be inserted. 2. Explain the concurrency story: two concurrent sessions inserting the same key — one wins the unique-index race, the other sees the conflict and runs the UPDATE branch. No primary-key violation can escape. 3. Mention that the conflict target must reference a unique constraint or unique index — you can also use `ON CONFLICT ON CONSTRAINT constraint_name`. 4. Be ready to discuss when DO NOTHING is preferable — silent dedup, like a log-line ingest where duplicates are ignored.

Edge cases to mention: - The conflict target must match an actual unique index, including partial indexes if applicable. - If multiple rows in the INSERT batch collide with the same target row, only one wins — the second triggers the UPDATE on the just-updated row. - `EXCLUDED.col` refers to the proposed insert value; `t.col` refers to the existing row.

What NOT to say: - Do not say it works for arbitrary CHECK constraints — only unique constraints / indexes trigger ON CONFLICT. - Do not claim DO UPDATE without a target column works — you need `ON CONFLICT (col)` or `ON CONSTRAINT name`.

Common follow-up: "How do you implement a counter increment safely?" — `INSERT INTO counters (key, n) VALUES ('foo', 1) ON CONFLICT (key) DO UPDATE SET n = counters.n + 1;` — one atomic statement, no race.

```
INSERT INTO inventory (sku, qty)
VALUES ('SKU-1024', 50)
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = NOW();
```

Q421 · ON CONFLICT DO NOTHING — Silent Upsert

Tags: Difficulty: Easy · Asked at: Swiggy, Zerodha, Meesho · Topic: UPSERT

The question (interviewer's verbatim phrasing):

When would you use `ON CONFLICT DO NOTHING` instead of `DO UPDATE`?

The 30-second answer: Use `DO NOTHING` when duplicates should be silently dropped — typical for idempotent event ingestion, append-only audit logs, or seed data where the row should not be overwritten if it already exists.

Step-by-step thought process: 1. Frame the use case: payment webhook handlers receive the same event multiple times due to retries; `DO NOTHING` makes the insert idempotent against a `(event_id)` unique index. 2. Compare with `DO UPDATE`: `DO UPDATE` rewrites the existing row; `DO NOTHING` leaves it untouched. For audit data, `DO NOTHING` is correct because you want the original row preserved. 3. Mention `RETURNING` — `INSERT ... ON CONFLICT DO NOTHING RETURNING id` returns nothing for conflict rows, which is a quirk if you need to know the existing id (see the `xmax` trick later in this chapter). 4. Be ready for the follow-up about `INSERT IGNORE` in MySQL — semantically similar but suppresses all errors, not just duplicates, which is a footgun.

Edge cases to mention: - `ON CONFLICT DO NOTHING` with no target column suppresses all unique-constraint violations across all unique indexes on the table. - It does NOT return the id of the existing row — for that you need either a separate `SELECT` or the `xmax` trick. - Performance-wise, `DO NOTHING` is slightly cheaper than `DO UPDATE` because no row is rewritten.

What NOT to say: - Do not equate `ON CONFLICT DO NOTHING` with MySQL's `INSERT IGNORE` — the latter silences all errors, including data-type errors and foreign-key violations, which is dangerous. - Do not claim `DO NOTHING` returns the existing row in `RETURNING` — it does not.

Common follow-up: "How do you get the id of the existing row when `DO NOTHING` fires?" — Either run `RETURNING` then fall back to a `SELECT`, or use a `WITH` CTE that retries, or the `xmax` trick which we cover separately.

```
INSERT INTO webhook_events (event_id, payload)
VALUES ('evt_abc123', '{"amount": 4999}')
ON CONFLICT (event_id) DO NOTHING;
```

Q422 · INSERT ... ON DUPLICATE KEY UPDATE in MySQL

Tags: Difficulty: Easy · Asked at: Flipkart, PhonePe, Meesho, TCS · Topic: UPSERT

The question (interviewer's verbatim phrasing):

MySQL doesn't have ON CONFLICT or MERGE. How do you upsert there?

The 30-second answer: MySQL has `INSERT ... ON DUPLICATE KEY UPDATE`, which attempts the INSERT and, if it would violate any unique key (primary or secondary), runs the UPDATE clause on the conflicting row instead. Use `VALUES(col)` (older) or the `AS new` row alias (8.0.19+) to reference the proposed insert values.

Step-by-step thought process: 1. Walk through the syntax: `INSERT INTO t (a, b) VALUES (1, 'x') ON DUPLICATE KEY UPDATE b = VALUES(b)`. From MySQL 8.0.19, prefer the alias form: `INSERT INTO t (a, b) VALUES (1, 'x') AS new ON DUPLICATE KEY UPDATE b = new.b`. 2. Note that ON DUPLICATE KEY fires on ANY unique-key conflict — not a specific one. That can be a footgun on tables with multiple unique indexes. 3. Mention that the affected-rows count is 1 for an INSERT, 2 for an UPDATE, and 0 if the existing row was unchanged. This is a classic MySQL trick question. 4. Be ready for the follow-up about RETURNING — MySQL does not have it, so you have to query `LAST_INSERT_ID()` or re-select.

Edge cases to mention: - If the UPDATE branch changes the primary key, replication and triggers can behave oddly — avoid it. - `INSERT ... ON DUPLICATE KEY UPDATE` consumes an auto-increment value even on the UPDATE path (until MySQL 8.0 with `innodb_autoinc_lock_mode=0`). - Multi-row INSERT with this clause runs the UPDATE for each colliding row in order — generally what you want.

What NOT to say: - Do not equate it with MERGE — MERGE is a full standard statement; ON DUPLICATE KEY UPDATE is MySQL-specific. - Do not claim it works on PostgreSQL or SQL Server.

Common follow-up: "What does `VALUES(col)` mean inside ON DUPLICATE KEY UPDATE?" — It references the value from the proposed INSERT row. Deprecated in 8.0.20 in favour of the row alias.

```
INSERT INTO product_views (product_id, view_count) VALUES (501, 1) AS new
ON DUPLICATE KEY UPDATE view_count = product_views.view_count + new.view_count;
```

Q423 · REPLACE INTO in MySQL — Why It's NOT an Upsert

Tags: Difficulty: Medium · Asked at: PhonePe, Flipkart, Walmart Labs · Topic: UPSERT

The question (interviewer's verbatim phrasing):

What does `REPLACE INTO` do in MySQL, and why do senior engineers tell juniors never to use it?

The 30-second answer: REPLACE INTO is a DELETE-then-INSERT, not an upsert. If a row with the same unique key exists, MySQL DELETES it and then INSERTS the new one — which fires DELETE triggers, breaks foreign-key children with ON DELETE CASCADE, and burns a new auto-increment id every time.

Step-by-step thought process: 1. Lead with the danger: REPLACE LOOKS like an upsert but mechanically it's destructive — the old row is gone, and any audit triggers see a DELETE followed by an INSERT. 2. The classic example: a `users` table with `user_id INT AUTO_INCREMENT PRIMARY KEY` and a unique key on `email`. REPLACE INTO users (email, name) VALUES ('a@b.com', 'New Name') deletes the existing row and creates a new one with a brand new user_id. All foreign keys pointing to the old user_id are now broken (or cascade-deleted). 3. Mention that REPLACE INTO is fine ONLY for cache-like tables with no foreign keys pointing in and no triggers — for example, a session table or a precomputed aggregate. 4. Be ready to recommend the alternative — `INSERT ... ON DUPLICATE KEY UPDATE` does the right thing.

Edge cases to mention: - Auto-increment values are consumed on every REPLACE, even when no INSERT happens conceptually — table can run out of ids surprisingly fast. - ON DELETE CASCADE foreign keys WILL cascade — silently deleting child rows. - Audit triggers see two events per REPLACE, doubling audit volume.

What NOT to say: - Do not call REPLACE "the MySQL upsert" — it is not, and using it where ON DUPLICATE KEY UPDATE was intended has caused production data loss. - Do not claim it's faster — usually it's slower because of the extra DELETE-and-INSERT work.

Common follow-up: "Is there ANY good reason to use REPLACE INTO?" — Yes, when the semantic you want is genuinely "throw away the old row, insert this one", such as cache rows where you don't care about the surrogate id.

```
-- DANGER: deletes the existing row before inserting
REPLACE INTO sessions (session_id, user_id, expires_at)
VALUES ('sess_abc', 42, NOW() + INTERVAL 1 HOUR);
```

Q424 · Detecting Whether an Upsert Inserted or Updated

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman, Postgres-heavy GCCs · Topic: UPSERT

The question (interviewer's verbatim phrasing):

After an `INSERT ... ON CONFLICT DO UPDATE`, how can you tell whether the row was newly inserted or an existing one was updated?

The 30-second answer: In Postgres, use the `xmax` system column in `RETURNING` — `xmax = 0` means a fresh insert, non-zero means an update fired. It's an undocumented but widely-used trick because there is no first-class way to ask.

Step-by-step thought process: 1. Set up the problem: you ran an upsert and you want to send a "welcome email" only for new rows, not updates. You need a boolean per returned row. 2. Walk through the `xmax` trick — every row in Postgres has hidden system columns; `xmax` is the deletion transaction id. A freshly-inserted row has `xmax = 0`. When the row is updated through `ON CONFLICT`, the old version's `xmax` gets set to the current `xid`, and the new version sees a non-zero `xmax` in `RETURNING`. Hence `(xmax = 0) AS inserted` works as a flag. 3. Mention alternatives: in MySQL, use `ROW_COUNT()` — 1 means insert, 2 means update. In SQL Server with `MERGE`, use the `OUTPUT` clause's `$action` pseudo-column. 4. Be ready for the follow-up about reliability — the `xmax` trick is not officially documented as a contract, but it has been stable across Postgres versions and is acknowledged in mailing-list discussions.

Edge cases to mention: - In Postgres, `xmax` is only zero for a row that has not been deleted or updated in the current transaction. - `MERGE` in SQL Server provides `$action` in `OUTPUT` — values are `'INSERT'`, `'UPDATE'`, `'DELETE'`. Cleaner than `xmax` but only for `MERGE`. - In MySQL `ROW_COUNT()` returns 1 per INSERT, 2 per UPDATE, 0 if the row was unchanged — confusing because "2" sounds like two rows.

What NOT to say: - Do not invent a column — beginners often write `RETURNING xmax = 0 AS was_inserted` and miscount. - Do not claim Postgres has an official boolean for this — it does not.

Common follow-up: "What's the SQL Server equivalent?" — `MERGE ... OUTPUT $action, inserted.id` — `$action` reports `'INSERT'`, `'UPDATE'` or `'DELETE'`.

```
INSERT INTO orders (order_id, amount) VALUES ('ord_42', 1999)
ON CONFLICT (order_id) DO UPDATE SET amount = EXCLUDED.amount
RETURNING order_id, (xmax = 0) AS inserted;
```

Q425 · Bulk Upsert Patterns

Tags: Difficulty: Medium · Asked at: Walmart Labs, Target, Swiggy, ETL teams · Topic: UPSERT

The question (interviewer's verbatim phrasing):

You have 100k rows to upsert nightly. How do you do it efficiently?

The 30-second answer: Bulk-load the rows into a staging table first, then run a single

`INSERT ... ON CONFLICT` from staging or a MERGE driven by staging. Avoid one-row-at-a-time round trips at all costs.

Step-by-step thought process: 1. Lead with the architecture: client-side loop calling 100k single-row upserts is the slow path — each one is a network round trip plus an index probe. 2. Stage first: COPY (Postgres) or LOAD DATA INFILE (MySQL) or BULK INSERT (SQL Server) into a temp/staging table without constraints. This is at least 10x faster than INSERTs. 3. Then merge in one statement: `INSERT INTO target SELECT * FROM staging ON CONFLICT (id) DO UPDATE SET ...`. The optimizer can use a hash join here, batch the index updates, and do everything in one transaction. 4. Be ready to discuss chunking — for tables with billions of rows, even one big MERGE can be too long; break it into chunks by primary-key ranges.

Edge cases to mention: - Disable non-essential indexes and triggers on the staging table — they slow the load without benefit. - For Postgres, `INSERT INTO ... SELECT ... ON CONFLICT` is the canonical pattern. - If the source has duplicate keys, dedup in staging first with `ROW_NUMBER() OVER (PARTITION BY id ORDER BY ts DESC)` so only one row per key reaches the upsert.

What NOT to say: - Do not propose calling upsert in an ORM `.each` loop for 100k rows — that's the anti-pattern this question is testing for. - Do not claim "one giant transaction is always best" — it can block other writers and bloat the WAL; chunking helps.

Common follow-up: "How do you handle a 100M-row upsert without blocking online traffic?" — Chunk by id range, throttle with `pg_sleep()` between batches, and consider partitioning the target.

```
INSERT INTO inventory (sku, qty, updated_at)
SELECT sku, qty, NOW() FROM staging_inventory
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = EXCLUDED.updated_at;
```

Q426 · Concurrent Upsert Race Conditions

Tags: Difficulty: Hard · Asked at: Razorpay, Goldman, Walmart Labs · Topic: UPSERT

The question (interviewer's verbatim phrasing):

Two sessions try to upsert the same key at the same time. Walk me through what each dialect does.

The 30-second answer: Postgres `ON CONFLICT` serialises on the unique index — one wins the INSERT, the other re-reads and runs the UPDATE branch with no race. SQL Server MERGE without HOLDLOCK can race and produce a unique-constraint violation. MySQL `ON DUPLICATE KEY UPDATE` is also race-safe via the unique index.

Step-by-step thought process: 1. Frame the scenario: two payment webhook handlers process the same `event_id` simultaneously. Both INSERT first; one row wins. 2. Postgres path: session A acquires the unique-index entry, session B blocks on the index lock until A commits, then session B's INSERT sees the conflict and runs the UPDATE branch. Clean, atomic, no application retry needed. 3. SQL Server path: with default isolation and no locking hints, MERGE can read "no match" in both sessions, then both INSERTs run, second one violates the constraint. Fix is `WITH (HOLDLOCK)` on the target. 4. MySQL path: `ON DUPLICATE KEY UPDATE` is essentially safe; the storage engine's unique index ensures serialization, similar to Postgres.

Edge cases to mention: - Postgres returns the inserted-or-updated row in RETURNING regardless of which branch fired — both sessions see correct results. - Deadlocks are possible if two upserts target rows in opposite order — keep insert order consistent if you bulk-upsert. - SQL Server READ COMMITTED SNAPSHOT can ameliorate but does not fully fix MERGE races.

What NOT to say: - Do not claim "the application must retry on conflict" — Postgres and MySQL handle it internally; only SQL Server MERGE without hints needs application-level retry. - Do not say "use SERIALIZABLE for everything" — way too heavy for routine upserts.

Common follow-up: "What if two sessions update the same key with different values?" — Postgres applies them serially — last-writer-wins on the column values, but both rows survive.

```
-- Race-safe in Postgres
INSERT INTO counters (key, n) VALUES ('homepage_views', 1)
ON CONFLICT (key) DO UPDATE SET n = counters.n + 1;
```

Section B — LATERAL Joins and Set-Returning Functions

Q427 · LATERAL Join Basics

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postgres-heavy GCCs · Topic: LATERAL

The question (interviewer's verbatim phrasing):

What does a LATERAL join do? When would I use it instead of a regular subquery?

The 30-second answer: LATERAL lets the subquery on the right side of a JOIN reference columns from tables on the left, evaluating the subquery once per left row. It's how you say "for each row in the outer table, run this inner query parameterised by that row".

Step-by-step thought process: 1. Open with the syntactic restriction LATERAL removes: normally a subquery in the FROM clause cannot reference outer columns. With LATERAL, it can. 2. Concrete example: for each user, fetch their three most-recent orders. Without LATERAL you'd need a window function plus a filter. With LATERAL it's a clean per-row TOP-N. 3. Mention the dialect names — Postgres and Oracle 12c+ call it `LATERAL`, SQL Server calls it `CROSS APPLY` (and `OUTER APPLY` for the LEFT JOIN variant). MySQL added it in 8.0.14. 4. Be ready for the question on what the planner does — typically a nested loop with the LATERAL subquery on the inner side, often using an index for efficiency.

Edge cases to mention: - LATERAL with an empty subquery result drops the outer row — use LEFT JOIN LATERAL to keep it. - LATERAL is great with set-returning functions like `unnest()` and `jsonb_array_elements()` — naturally per-row. - Performance depends on having a useful index on the inner-side filter column.

What NOT to say: - Do not claim LATERAL is just a subquery rename — it has different scoping rules. - Do not assume CROSS APPLY syntax works on Postgres — Postgres uses LATERAL only.

Common follow-up: "Can you do per-row TOP-N without LATERAL?" — Yes, with `ROW_NUMBER()` OVER (PARTITION BY user_id ORDER BY ts DESC) and a filter — but LATERAL is often cleaner and uses indexes better.

```
-- Per user, fetch the latest order
SELECT u.name, o.order_id, o.amount
FROM users u
LEFT JOIN LATERAL (
  SELECT order_id, amount FROM orders WHERE user_id = u.id
  ORDER BY created_at DESC LIMIT 1
) o ON TRUE;
```

Q428 · CROSS APPLY in SQL Server

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman · Topic: LATERAL

The question (interviewer's verbatim phrasing):

What is CROSS APPLY in SQL Server and how does it differ from LATERAL?

The 30-second answer: `CROSS APPLY` is SQL Server's name for LATERAL — same semantics, different keyword. It evaluates the right-side table expression once per left row using values from the left as parameters. `OUTER APPLY` is the LEFT JOIN equivalent.

Step-by-step thought process: 1. State the equivalence: `LATERAL` (Postgres / Oracle 12c+ / MySQL 8.0.14+) and `CROSS APPLY` (SQL Server, Sybase) do the same thing. 2. SQL Server uses APPLY heavily because table-valued functions (TVFs) only work in the FROM clause via CROSS APPLY — you call a TVF per row of the outer table. 3. Walk through an example: `SELECT u.id, top_orders.* FROM users u CROSS APPLY (SELECT TOP 3 * FROM orders WHERE user_id = u.id ORDER BY created_at DESC) top_orders;` . This is the classic per-row top-N. 4. Be ready for the dialect comparison: when porting SQL Server CROSS APPLY to Postgres, replace with LATERAL; replace OUTER APPLY with LEFT JOIN LATERAL ... ON TRUE.

Edge cases to mention: - CROSS APPLY drops outer rows with no inner match; OUTER APPLY keeps them with NULLs. - TVFs in SQL Server are a major reason CROSS APPLY exists — they can be parameterised by outer-row columns only via APPLY. - Older Postgres docs used the term "lateral subquery" before SQL standard adopted LATERAL.

What NOT to say: - Do not claim CROSS APPLY is a Postgres feature — it is not; Postgres uses LATERAL. - Do not confuse CROSS APPLY (per-row) with CROSS JOIN (Cartesian product).

Common follow-up: "Why use CROSS APPLY over a window function?" — APPLY can use a different filter or order per outer row trivially; window functions need the same partition/order for everyone.

```
-- SQL Server: per user, top 3 orders by amount
SELECT u.name, o.order_id, o.amount
FROM users u
CROSS APPLY (
    SELECT TOP 3 order_id, amount FROM orders
    WHERE user_id = u.id ORDER BY amount DESC
) o;
```

Q429 · LATERAL for Per-Row Top-N

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, PhonePe, Walmart · Topic: LATERAL

The question (interviewer's verbatim phrasing):

For each user, return the three most recent orders. Show me two ways — one with LATERAL and one without.

The 30-second answer: With LATERAL: `SELECT u.*, o.* FROM users u JOIN LATERAL (SELECT * FROM orders WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3) o`

`ON TRUE;` . Without LATERAL: use `ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC)` and filter for `rn <= 3` .

Step-by-step thought process: 1. Lead with LATERAL: it reads naturally — "for each user, pull three orders" — and uses the index on `(user_id, created_at DESC)` straightforwardly via nested loop. 2. Window-function alternative: a single scan of orders with ROW_NUMBER, then a filter. Good if the orders table is small or you need a single scan. 3. Performance contrast: LATERAL is faster when there are many users and few orders per user (index seek per user). Window function is faster when you'd otherwise loop millions of users with sparse data per user. 4. Be ready for the follow-up: "Which would you pick for a 10M-user table with avg 5 orders per user?" — Probably window function, one scan beats 10M seeks unless cache is hot.

Edge cases to mention: - LATERAL needs LEFT JOIN LATERAL (with `ON TRUE`) to keep users who have no orders. - The ROW_NUMBER version returns ties unpredictably — use ORDER BY with a tiebreaker. - For "top N per group with ties", use RANK() or DENSE_RANK() instead.

What NOT to say: - Do not claim LATERAL is always faster — it depends on cardinality and indexes. - Do not omit the ORDER BY in the LIMIT subquery — the result is non-deterministic.

Common follow-up: "What if N varies per user?" — LATERAL handles this trivially with `LIMIT u.preferred_count` inside the subquery. Window functions cannot do per-group variable N as cleanly.

```
SELECT u.name, o.order_id, o.created_at
FROM users u
JOIN LATERAL (
  SELECT order_id, created_at FROM orders
  WHERE user_id = u.id ORDER BY created_at DESC LIMIT 3
) o ON TRUE;
```

Q430 · LATERAL with Set-Returning Functions

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postgres-heavy teams · Topic: LATERAL

The question (interviewer's verbatim phrasing):

How does LATERAL interact with set-returning functions like `unnest()` or `jsonb_array_elements()` ?

The 30-second answer: LATERAL is the standard way to expand a set-returning function once per row of an outer table — for each row, the SRF emits N rows and you get the cartesian per-row product. Postgres actually allows omitting the LATERAL keyword in this case as a convenience, but writing it explicitly is clearer.

Step-by-step thought process: 1. Example setup: an `orders` table where each row has a `jsonb` column `items` holding an array of line-item objects. You want one row per (order, item). 2. Write the LATERAL:

`SELECT o.id, item FROM orders o, LATERAL jsonb_array_elements(o.items) item;` . Each order produces as many rows as it has items. 3. Mention the comma shortcut: `FROM orders o, jsonb_array_elements(o.items) item` works in Postgres because SRFs in FROM are implicitly lateral. 4. Be ready for the use-case follow-up: this is how you de-normalise jsonb arrays for analytical queries.

Edge cases to mention: - If `items` is NULL or empty array, the order disappears (CROSS JOIN LATERAL semantics). Use `LEFT JOIN LATERAL ... ON TRUE` to keep it. - SRFs called in the SELECT list (not FROM) have different semantics — they produce a single result column with set semantics, not a join. - `WITH ORDINALITY` (next question) gives you an index column alongside the unnested values.

What NOT to say: - Do not omit LATERAL on non-SRF subqueries — Postgres requires it for regular subqueries even though it tolerates omission for SRFs. - Do not assume MySQL's `JSON_TABLE` works the same way — it does, but the syntax is different.

Common follow-up: "How would you do this in MySQL?" —

`JSON_TABLE(items, '$[*]' COLUMNS (...))` provides equivalent functionality from MySQL 8.0.

```
SELECT o.order_id, item->>'sku' AS sku, (item->>'qty')::int AS qty
FROM orders o, LATERAL jsonb_array_elements(o.items) item;
```

Q431 · LATERAL for Unnesting Arrays Per Row

Tags: Difficulty: Medium · Asked at: Razorpay, Meesho, Postgres-heavy teams · Topic: LATERAL

The question (interviewer's verbatim phrasing):

I have a Postgres table where each row has an `int[]` column. I want one output row per (id, array element). How?

The 30-second answer: `SELECT t.id, x FROM t, LATERAL unnest(t.arr) x;` — `unnest()` is a set-returning function and LATERAL gives it access to the outer row's array. One outer row produces N output rows, one per array element.

Step-by-step thought process: 1. Set up the example: `survey_responses` table with a column `selected_options int[]` — each response picked some subset. 2. Write the unnest: `SELECT r.id, opt FROM survey_responses r, LATERAL unnest(r.selected_options) opt;` . Now each response contributes one row per selected option — ready for aggregation by option. 3. Mention that `unnest()` returns no rows for an empty array or NULL — the parent row is dropped. Use

LEFT JOIN LATERAL ... ON TRUE to keep responses with no selections. 4. Be ready to discuss `array_agg()` as the inverse — collapsing rows back into an array.

Edge cases to mention: - Multi-dimensional arrays unnest by total elements unless you specify the dimension. - `unnest(arr1, arr2)` unnests two arrays in parallel, padding the shorter with NULLs — handy for paired data. - `unnest()` is the Postgres idiom; SQL Server uses `STRING_SPLIT()` for strings, and `OPENJSON()` for JSON arrays.

What NOT to say: - Do not claim arrays are the same as JSON arrays — they are stored and indexed differently in Postgres. - Do not unnest in the SELECT list when you mean to use it in FROM — the SELECT-list usage is legacy and produces confusing results.

Common follow-up: "How do you count, per option, how many responses selected it?" — `SELECT opt, COUNT(*) FROM survey_responses, LATERAL unnest(selected_options) opt GROUP BY opt;`

```
SELECT r.response_id, opt
FROM survey_responses r, LATERAL unnest(r.selected_options) opt;
```

Q432 · generate_series() with LATERAL

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, analytics teams · Topic: LATERAL

The question (interviewer's verbatim phrasing):

Show me how to use `generate_series()` inside a LATERAL join to expand each row into multiple rows.

The 30-second answer: `generate_series(start, end)` is a set-returning function; combined with LATERAL it expands each outer row into one row per integer (or date) in the series. Classic use case: filling in missing dates for a per-day report.

Step-by-step thought process: 1. Use case: a subscriptions table with `started_on` and `ended_on`. You want one row per (subscription, day) for revenue allocation. 2. Write the lateral `generate_series`: `SELECT s.id, d::date FROM subscriptions s, LATERAL generate_series(s.started_on, s.ended_on, '1 day'::interval) d;` Now each subscription produces a row per day it was active. 3. Another use case: per-user, generate a row for each of the last 30 days, then LEFT JOIN to orders for a presence-of-orders matrix. 4. Be ready for the date-spine pattern: `generate_series(start, end, '1 day')` is the canonical Postgres way to manufacture missing dates.

Edge cases to mention: - generate_series is Postgres-specific; SQL Server uses recursive CTEs or a numbers table; MySQL 8.0+ uses recursive CTE. - For huge ranges generate_series can blow up memory — use sane bounds. - The interval form works for dates and timestamps; integer form is for plain numbers.

What NOT to say: - Do not say generate_series is in the SQL standard — it is not, it's Postgres-only. - Do not use it without LATERAL when referencing outer columns — Postgres allows it for SRFs but explicit LATERAL is clearer.

Common follow-up: "How would you do this in SQL Server?" — Recursive CTE: `WITH dates AS (SELECT @start AS d UNION ALL SELECT DATEADD(day, 1, d) FROM dates WHERE d < @end) ...` with `OPTION (MAXRECURSION 0)`.

```
SELECT s.subscription_id, d::date AS active_day
FROM subscriptions s,
     LATERAL generate_series(s.started_on, s.ended_on, '1 day') d;
```

Q433 · WITH ORDINALITY — Index from Set-Returning Functions

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, analytics-heavy GCCs · Topic: LATERAL

The question (interviewer's verbatim phrasing):

What does `WITH ORDINALITY` do, and when is it useful?

The 30-second answer: `WITH ORDINALITY` adds an integer column to a set-returning function's output, numbering the rows from 1 — you get the position of each element in the original set. Useful for preserving array ordering after `unnest`.

Step-by-step thought process: 1. Problem: you `unnest()` an array but lose the original index. If you want to know "this was the third element", `WITH ORDINALITY` gives you that. 2. Syntax: `SELECT * FROM unnest(ARRAY['a', 'b', 'c']) WITH ORDINALITY AS t(val, idx);`. You get two columns — the value and its 1-based position. 3. Real-world example: a `tags TEXT[]` column where the first tag is the primary. After `unnest` with `ORDINALITY`, filter `WHERE idx = 1` to get just the primary tag. 4. Be ready to mention this is SQL standard syntax — Postgres supports it from 9.4, Oracle 12c also supports it.

Edge cases to mention: - Ordinality starts at 1, not 0 — easy to off-by-one in code that thinks in zero-indexed terms. - `WITH ORDINALITY` works with any SRF — `generate_series`, `regexp_split_to_table`, `jsonb_array_elements`, etc. - For `jsonb` arrays, the ordinality column gives the array index — useful for joining back to the source structure.

What NOT to say: - Do not assume ROW_NUMBER() over unnest is equivalent — it works but is more verbose and may not preserve original array order across all dialects. - Do not claim it's a Postgres extension — it's SQL standard.

Common follow-up: "How would you re-create the array in original order after some filtering?" — `array_agg(val ORDER BY idx)` reconstructs the array using the ordinality column.

```
SELECT u.id, t.tag, t.idx
FROM users u, LATERAL unnest(u.interests) WITH ORDINALITY AS t(tag, idx)
WHERE t.idx <= 3; -- top 3 interests by original order
```

Section C — RETURNING, OUTPUT and DML in CTEs

Q434 · RETURNING Clause in Postgres and Oracle

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman, Postgres-heavy teams · Topic: RETURNING

The question (interviewer's verbatim phrasing):

What does the RETURNING clause do? Why does it matter for application code?

The 30-second answer: RETURNING lets INSERT, UPDATE and DELETE statements return the rows they touched, so the application doesn't need a follow-up SELECT. Crucial for capturing auto-generated ids and audit data without an extra round trip.

Step-by-step thought process: 1. Lead with the round-trip cost: without RETURNING, after an INSERT you'd run `SELECT currval()` or a separate SELECT to get the id. With RETURNING, you get the id in the same statement. 2. Show the syntax: `INSERT INTO orders (...) VALUES (...) RETURNING id, created_at;`. UPDATE and DELETE work the same — `RETURNING *` returns all columns of affected rows. 3. Mention dialect support: Postgres has full RETURNING. Oracle has `RETURNING ... INTO` (slightly different syntax, populates host variables). SQL Server uses the OUTPUT clause. MySQL does not have RETURNING. 4. Be ready for the batch-INSERT angle — RETURNING gives you all generated ids in one shot for a multi-row INSERT.

Edge cases to mention: - `RETURNING *` is fine for prototyping but list explicit columns in production — the row shape is part of the contract. - RETURNING runs after triggers fire, so trigger-modified column values are visible. - In Oracle, `RETURNING INTO` only works for single-row DML unless you use `BULK COLLECT INTO`.

What NOT to say: - Do not claim RETURNING is universal — MySQL still lacks it (MariaDB has it). - Do not write `RETURNING *` in production code without thinking about the row shape.

Common follow-up: "How do you get all 1000 generated ids in a batch INSERT?" — `INSERT INTO ... SELECT ... RETURNING id;` returns one row per inserted row.

```
INSERT INTO orders (user_id, amount) VALUES (42, 1999)
RETURNING order_id, created_at;
```

Q435 · OUTPUT Clause in SQL Server

Tags: Difficulty: Medium · Asked at: JP Morgan, Wells Fargo, Goldman, Walmart Labs · Topic: RETURNING

The question (interviewer's verbatim phrasing):

SQL Server has OUTPUT instead of RETURNING. How does it work?

The 30-second answer: `OUTPUT inserted.col, deleted.col` returns rows affected by INSERT, UPDATE, DELETE or MERGE. `inserted` holds the new row, `deleted` holds the pre-update version. You can stream OUTPUT directly to the client or `OUTPUT INTO @table` for further processing.

Step-by-step thought process: 1. Walk through the pseudo-tables: `inserted` (new values, populated by INSERT and UPDATE) and `deleted` (old values, populated by UPDATE and DELETE). MERGE populates both for updated rows plus `$action`. 2. Show two patterns: `INSERT ... OUTPUT inserted.id, inserted.created_at` streams the values back; `DELETE ... OUTPUT deleted.* INTO @audit` captures deleted rows into a table variable for audit. 3. Mention that OUTPUT is older and more powerful than RETURNING in some ways — it can capture both pre- and post-update state. For pure pre-image audit, OUTPUT is unique. 4. Be ready for the MERGE-OUTPUT combination: `MERGE ... OUTPUT $action, inserted.id, deleted.id` gives you per-row which branch fired.

Edge cases to mention: - OUTPUT INTO with a target table requires no triggers on that target (or INTO clause limitations). - `$action` is MERGE-only — `'INSERT'`, `'UPDATE'`, or `'DELETE'`. - For UPDATE without OUTPUT, you cannot get the pre-image — capture it before the UPDATE if needed.

What NOT to say: - Do not say OUTPUT works on Postgres — it does not; Postgres uses RETURNING. - Do not assume OUTPUT INTO a temp table works with triggers — there are documented restrictions.

Common follow-up: "How do you audit which `user_id` changed in an `UPDATE`?" — `UPDATE users SET ... OUTPUT deleted.user_id AS old_id, inserted.user_id AS new_id INTO audit_table` .

```
DELETE FROM orders OUTPUT deleted.order_id, deleted.amount
INTO archived_orders
WHERE status = 'cancelled' AND created_at < DATEADD(year, -2, GETDATE());
```

Q436 · RETURNING for Capturing Generated IDs in Batch INSERT

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, ETL-heavy teams · Topic: RETURNING

The question (interviewer's verbatim phrasing):

I'm batch-inserting 500 rows in Postgres. How do I get all the generated serial ids?

The 30-second answer: Use `INSERT INTO t (...) VALUES (...), (...), ... RETURNING id;` — Postgres returns one row per inserted row, in insertion order. Capture them in the application or in a CTE for follow-up DML.

Step-by-step thought process: 1. Set up the scenario: a Razorpay-style payment batch where you INSERT 500 payment rows and need each new `payment_id` to send to the downstream ledger. 2. The naive path: insert one row at a time using `currval()` or `RETURNING id` per row — 500 round trips. 3. The right path: one INSERT with 500 VALUES rows, `RETURNING id` returns 500 ids at once. Read all of them in the application. 4. Be ready for the ordering question — in Postgres, INSERT order is preserved in RETURNING output (you can rely on this in practice).

Edge cases to mention: - For very large batches, use `INSERT ... SELECT ... RETURNING` from a staging table rather than 50k VALUES literals. - The Oracle equivalent uses `RETURNING INTO ... BULK COLLECT` — different syntax but same idea. - MySQL does not support multi-row RETURNING; use `LAST_INSERT_ID()` plus knowledge of the auto-increment increment.

What NOT to say: - Do not loop single-row INSERTs and call this "batch" — that's the slow path you're being asked to avoid. - Do not claim RETURNING is unordered — in Postgres practice, order is preserved.

Common follow-up: "What about for COPY-loaded data?" — COPY does not support RETURNING directly; load into a staging table then `INSERT INTO target SELECT * FROM staging RETURNING id` .

```
WITH new_payments AS (  
  INSERT INTO payments (order_id, amount, gateway)  
  VALUES (1, 100, 'razorpay'), (2, 200, 'razorpay'), (3, 300, 'razorpay')  
  RETURNING payment_id, order_id  
)  
INSERT INTO ledger (payment_id, order_id, type)  
SELECT payment_id, order_id, 'credit' FROM new_payments;
```

Q437 · DML in CTE — Chaining Writes with RETURNING

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postgres-heavy product teams · Topic: RETURNING

The question (interviewer's verbatim phrasing):

Show me how to perform multiple DML statements in one query using CTEs and RETURNING.

The 30-second answer: Postgres allows INSERT, UPDATE and DELETE inside `WITH` clauses, each with RETURNING — chain them so output of one DML feeds the next. Classic pattern: archive rows then delete them, atomically.

Step-by-step thought process: 1. The pattern:

`WITH archived AS (DELETE FROM orders WHERE status='cancelled' RETURNING *)
INSERT INTO archived_orders SELECT * FROM archived;` . One statement, atomic — orders cannot be deleted without being archived. 2. Note that this works only in Postgres — Oracle and SQL Server require separate statements or different mechanisms (OUTPUT INTO for SQL Server). 3. All DML in a single WITH happens in one snapshot — the CTEs don't "see" each other's effects on the same rows. Be careful with overlapping row sets. 4. Be ready for the consistency follow-up — for atomic multi-table writes that need to see each other's effects, use a regular transaction with explicit statements.

Edge cases to mention: - The CTE order in the WITH list does not determine execution order — Postgres is free to interleave them. Use a final SELECT or INSERT to pin order. - Each CTE sees the table state as of the statement start, not after the other CTEs' writes. - A single statement is fully atomic — if any CTE fails, all roll back.

What NOT to say: - Do not assume DML CTEs see each other — they do not, by design. - Do not write this in MySQL — MySQL 8.0 has CTEs but not data-modifying CTEs.

Common follow-up: "How do you move rows from one table to another atomically?" — Exactly this pattern, with DELETE...RETURNING piped into INSERT...SELECT.

```
WITH expired AS (  
  DELETE FROM sessions WHERE expires_at < NOW() RETURNING *  
)  
INSERT INTO session_history SELECT * FROM expired;
```

Q438 · UPDATE ... FROM in Postgres

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postgres-heavy GCCs · Topic: DML

The question (interviewer's verbatim phrasing):

How do you update a table based on values from another table in Postgres?

The 30-second answer: Postgres extends UPDATE with a `FROM` clause: `UPDATE t SET col = s.col FROM source s WHERE t.id = s.id;` . The FROM table provides the values; the WHERE clause joins target to source.

Step-by-step thought process: 1. Set up the scenario: nightly job updates `users.tier` based on a computed `user_tiers_snapshot` table. 2. Write the UPDATE...FROM: `UPDATE users u SET tier = s.tier FROM user_tiers_snapshot s WHERE u.id = s.user_id;` . The FROM table can be a real table, a CTE, or a subquery. 3. Watch out: if the FROM source has multiple rows matching the same target row, the behaviour is unspecified (Postgres picks one). Dedup the source first. 4. Be ready for the MySQL contrast — MySQL uses `UPDATE t JOIN source ON ... SET t.col = source.col` syntax (next question).

Edge cases to mention: - Multiple matches on the source side produce unspecified results — Postgres will not raise an error but may update with any one of the matching rows. - The target table cannot appear in the FROM list (use a self-join via a subquery or CTE if you need that). - Standard SQL has `UPDATE ... SET col = (SELECT ...)` for scalar correlated updates — slower, more portable.

What NOT to say: - Do not claim UPDATE...FROM is standard SQL — it is a Postgres extension (SQL Server also supports it with similar syntax). - Do not write it in MySQL — MySQL uses UPDATE...JOIN.

Common follow-up: "How do you ensure idempotency if the same source row appears twice?" — Dedup in a CTE first: `WITH dedup AS (SELECT DISTINCT ON (user_id) ... FROM source) UPDATE users SET ... FROM dedup WHERE ...` .

```
UPDATE users u SET tier = s.tier, updated_at = NOW()  
FROM user_tiers_snapshot s  
WHERE u.id = s.user_id AND u.tier <> s.tier;
```

Q439 · UPDATE ... JOIN in MySQL

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Meesho · Topic: DML

The question (interviewer's verbatim phrasing):

Same scenario in MySQL — how do you update one table using values from another?

The 30-second answer: MySQL uses an inline JOIN in the UPDATE: `UPDATE users u JOIN user_tiers_snapshot s ON u.id = s.user_id SET u.tier = s.tier;`. Multiple tables can be joined and multiple SET targets can appear.

Step-by-step thought process: 1. Show the syntax: target table first, JOIN second, SET third, optional WHERE last. MySQL allows updating columns of multiple joined tables in one statement. 2. Multi-table example: `UPDATE orders o JOIN payments p ON p.order_id = o.id SET o.status='paid', p.settled_at=NOW() WHERE p.confirmed=1;`. Both tables are updated atomically. 3. Note that ORDER BY and LIMIT are valid with single-table UPDATE in MySQL but not with multi-table UPDATE — common gotcha. 4. Be ready for the porting question — MySQL UPDATE JOIN to Postgres becomes UPDATE...FROM.

Edge cases to mention: - LIMIT does not work with multi-table UPDATE in MySQL. - If the JOIN matches the same target row multiple times, MySQL updates it once per match — order-dependent. - For very large UPDATE JOIN, MySQL can be slow; chunk by primary key.

What NOT to say: - Do not use the Postgres `UPDATE...FROM` syntax in MySQL — it will not parse. - Do not claim multi-table UPDATE is in the SQL standard — it is a MySQL extension.

Common follow-up: "How do you simulate UPDATE...JOIN with LIMIT in MySQL?" — Use a subquery to select ids first, then UPDATE filtered by those ids.

```
UPDATE orders o
JOIN payments p ON p.order_id = o.id
SET o.status = 'paid', o.paid_at = NOW()
WHERE p.confirmed_at IS NOT NULL AND o.status <> 'paid';
```

Q440 · DELETE ... USING in Postgres

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postgres-heavy teams · Topic: DML

The question (interviewer's verbatim phrasing):

How do you delete rows from one table based on a join with another in Postgres?

The 30-second answer: Postgres uses

`DELETE FROM target USING source WHERE target.col = source.col` — `USING` introduces additional tables that the `WHERE` clause can reference, similar to `UPDATE...FROM`.

Step-by-step thought process: 1. Use case: delete orders that belong to users who are flagged as fraudulent in another table. 2. Syntax: `DELETE FROM orders o USING fraud_flags f WHERE o.user_id = f.user_id AND f.confirmed = true;`. Only the `FROM` table is deleted; `USING` is read-only. 3. Mention `NOT EXISTS` as an alternative — `DELETE FROM orders WHERE EXISTS (...)` is portable; `USING` is more concise. 4. Be ready for the parallel: just as `UPDATE` has `UPDATE...FROM`, `DELETE` has `DELETE...USING`.

Edge cases to mention: - Only the target table is deleted; `USING` tables stay untouched — common confusion. - If multiple `USING` rows match the same target row, the target is deleted once (idempotent). - For deleting child rows via FK, prefer `ON DELETE CASCADE` in the constraint definition.

What NOT to say: - Do not put the target table in `USING` — it deletes nothing. - Do not assume `DELETE...USING` works in MySQL — MySQL has its own `DELETE...JOIN` syntax.

Common follow-up: "*Is `DELETE...USING` faster than `DELETE...WHERE EXISTS`?*" — Usually similar; the planner often converts `EXISTS` to a semi-join anyway. Profile if it matters.

```
DELETE FROM orders o
USING fraud_flags f
WHERE o.user_id = f.user_id AND f.confirmed = true;
```

Q441 · DELETE ... JOIN in MySQL

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Meesho · Topic: DML

The question (interviewer's verbatim phrasing):

Same problem in MySQL — delete from one table using a join.

The 30-second answer: MySQL syntax is `DELETE t1 FROM t1 JOIN t2 ON t1.id = t2.id WHERE ...` — the `DELETE t1` part names which table to delete from, the `FROM` clause provides the join.

Step-by-step thought process: 1. Walk through the structure: `DELETE` followed by the table alias(es) to delete from, `FROM` followed by the join, then `WHERE`. 2. Show example: `DELETE o FROM orders o JOIN fraud_flags f ON o.user_id = f.user_id WHERE f.confirmed = 1;` 3. Multi-table `DELETE`: `DELETE o, p FROM orders o JOIN payments p ON p.order_id = o.id WHERE ...` deletes from both atomically. 4. Be ready for the ordering follow-up — multi-table `DELETE` in MySQL does not support `ORDER BY` or `LIMIT`.

Edge cases to mention: - The aliases in `DELETE t1, t2` must match the `FROM` clause exactly — typos here silently change behaviour. - Foreign-key cascades fire after the parent row is deleted — multi-table `DELETE` can interact with cascades unexpectedly. - For huge `DELETE`, chunk it by id range to avoid long-held locks.

What NOT to say: - Do not write `DELETE FROM orders USING ...` in MySQL — that's Postgres syntax. - Do not assume `LIMIT` works with multi-table `DELETE` — it does not.

Common follow-up: "How do you delete rows from two tables in one statement?" — `DELETE t1, t2 FROM t1 JOIN t2 ON ... WHERE ...`. Both tables lose matching rows in one atomic operation.

```
DELETE o FROM orders o
JOIN fraud_flags f ON f.user_id = o.user_id
WHERE f.confirmed = 1;
```

Q442 · Multi-Table DELETE

Tags: Difficulty: Hard · Asked at: Flipkart, PhonePe, Walmart Labs · Topic: DML

The question (interviewer's verbatim phrasing):

When would you actually use multi-table `DELETE`, and what are the risks?

The 30-second answer: Multi-table `DELETE` removes matching rows from multiple tables in one statement, useful for cleaning related rows without foreign-key cascades — for example, archiving an order and its line items together. Risks: easy to delete more than you meant if the `JOIN` matches unexpectedly.

Step-by-step thought process: 1. Use case: an `orders` table and a `line_items` table without `ON DELETE CASCADE` — when deleting old orders, you need to delete both atomically. `DELETE o, li FROM orders o JOIN line_items li ON li.order_id = o.id WHERE o.created_at < ...` 2. Why not use cascades — cascades are great but can be surprising; explicit multi-table `DELETE` makes the intent visible in the query log. 3. Risk: a typo in the `JOIN` (`o.id = li.order_id` vs `o.id = li.id`) can either delete nothing or delete too much. Test with `SELECT` first. 4. Be ready

for the audit-trail follow-up — capture pre-image with OUTPUT INTO (SQL Server) or DELETE...RETURNING (Postgres) into an audit table.

Edge cases to mention: - In Postgres, multi-table DELETE is not directly supported — use a CTE with DELETE...RETURNING and a follow-up DELETE. - In MySQL, the alias list in

`DELETE a, b FROM ...` controls which tables get rows removed. - If one of the multi-table DELETES would orphan FK children, the whole statement aborts.

What NOT to say: - Do not assume multi-table DELETE is in the SQL standard — it is not; it's a MySQL extension. - Do not skip the test SELECT — multi-table DELETE typos have caused production data loss.

Common follow-up: "How do you do this in Postgres?" — Two DELETES in one transaction, or a CTE chain: `WITH d AS (DELETE FROM line_items WHERE order_id IN (SELECT id FROM orders WHERE ...)) RETURNING order_id) DELETE FROM orders WHERE id IN (SELECT order_id FROM d);`.

```
-- MySQL
DELETE o, li FROM orders o
JOIN line_items li ON li.order_id = o.id
WHERE o.created_at < '2024-01-01';
```

Q443 · INSERT ... SELECT with Column Transformation

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Accenture, ETL teams · Topic: DML

The question (interviewer's verbatim phrasing):

How do you copy data from one table to another while transforming columns?

The 30-second answer: `INSERT INTO target (col1, col2) SELECT expr1, expr2 FROM source WHERE ...` — the SELECT can include arbitrary expressions, casts, joins and CASE statements. Standard SQL, supported everywhere.

Step-by-step thought process: 1. Use case: snapshot a `customers` table into `customer_archive` with PII masked. `INSERT INTO customer_archive (id, name_masked, email_masked) SELECT id, LEFT(name, 1) || '***', SUBSTRING(email FROM '@.*$') FROM customers WHERE active = false;` 2. Mention casting — `INSERT INTO t (i) SELECT col::int FROM src` (Postgres) or `CAST(col AS INT)` (portable). 3. Multi-table SELECT — INSERT...SELECT can pull from joins, aggregations, CTEs. 4. Be ready for the volume question — for very large copies, use `COPY` (Postgres) or `INSERT ... SELECT` in chunks to avoid bloat.

Edge cases to mention: - Column order in the INSERT list and the SELECT list must match — easy to swap by mistake. - Default values fill in for any unlisted columns. - For self-INSERT (`INSERT INTO t SELECT * FROM t WHERE ...`), make sure the WHERE clause doesn't include rows already inserted — can loop.

What NOT to say: - Do not match by name — INSERT...SELECT matches by position only. - Do not insert without the column list in production — schema changes can silently misalign columns.

Common follow-up: "How do you do this incrementally, only new rows?" — Add `WHERE NOT EXISTS (SELECT 1 FROM target WHERE target.id = source.id)` or use ON CONFLICT DO NOTHING.

```
INSERT INTO customer_archive (id, name_masked, archived_at)
SELECT id, LEFT(name, 1) || '***', NOW()
FROM customers WHERE active = false AND last_login < NOW() - INTERVAL '2 years';
```

Q444 · Atomic Increment Without Read-Then-Update Race

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Goldman, Walmart Labs · Topic: UPSERT

The question (interviewer's verbatim phrasing):

I'm maintaining a counter — page views per article. Two requests increment it at the same time. How do I avoid the race?

The 30-second answer: Never read-then-update from the application. Use a single atomic SQL statement — `UPDATE counters SET n = n + 1 WHERE key = 'foo'`. The database handles concurrent increments correctly because the UPDATE acquires a row lock and reads-modifies-writes within one statement.

Step-by-step thought process: 1. State the anti-pattern: app code reads

`SELECT n FROM counters WHERE key='foo'`, adds 1 in application memory, writes back

`UPDATE counters SET n = $new WHERE key = 'foo'`. Under concurrency, two sessions can both read N, both write N+1, losing one increment. 2. The fix is one statement:

`UPDATE counters SET n = n + 1 WHERE key = 'foo'`. The database serialises the row-level updates via row locks — no lost increments. 3. Combined with upsert: `INSERT INTO counters (key, n) VALUES ('foo', 1) ON CONFLICT (key) DO UPDATE SET n = counters.n + 1;` — works whether the row exists or not, atomically. 4. Be ready for the high-traffic follow-up — for very hot counters, even row locks bottleneck; use sharding (split the counter into N rows by random key suffix and sum at read time) or write to a queue and batch-aggregate.

Edge cases to mention: - In MySQL: `INSERT INTO counters (key, n) VALUES ('foo', 1) ON DUPLICATE KEY UPDATE n = n + 1;` — same semantics. - For very hot rows, even row-level locking is a bottleneck — consider sharded counters or Redis INCR with periodic DB flush. - If you need to know the new value after the increment, add `RETURNING n` (Postgres) — atomic read-and-modify.

What NOT to say: - Do not say "use SERIALIZABLE isolation" — way too heavy and still doesn't fix the application-side read-then-update if you're using two separate statements. - Do not say "use SELECT FOR UPDATE then UPDATE" — that works but is two statements where one would do.

Common follow-up: "At 10k QPS on the same counter, is `n = n + 1` still good?" — No, you'll bottleneck on the row lock. Shard: write `(key, shard) -> n` for 16 random shards per key, sum at read time. Standard high-scale pattern.

```
-- Atomic, race-free, even with concurrent callers
INSERT INTO counters (key, n) VALUES ('article_42', 1)
ON CONFLICT (key) DO UPDATE SET n = counters.n + 1
RETURNING n;
```

Chapter 10, Part C — Modern Aggregation & NULL Handling

This part picks up the modern aggregation toolkit — `GROUPING SETS`, `ROLLUP`, `CUBE`, the `FILTER` clause, `ANY_VALUE`, the array/JSON aggregate family — and then walks through the NULL idioms that interviewers love to set traps with. The aggregation side is where senior candidates show they can collapse three `UNION ALL` blocks into one query that the planner only scans once. The NULL side is where careers get derailed: `NOT IN` with a NULL, three-valued logic in `WHERE`, the `IS DISTINCT FROM` operator that nobody teaches but everyone tests. Twenty-eight questions, each one battle-tested in real Indian interview rounds — read them slowly, because every one of these has a "gotcha" answer that the panel is hoping you'll trip over.

Q445 · GROUPING SETS — One Query, Multiple GROUP BYs

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan India, Razorpay · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

I want one query that returns sales totals grouped by region, separately by category, and a grand total — all in one result set. How would you write it without using three UNION ALL blocks?

The 30-second answer: Use `GROUPING SETS` inside the `GROUP BY` clause — each parenthesised set defines one grouping level, and an empty `()` set produces the grand total. The optimizer scans the base table once instead of three times.

Step-by-step thought process: 1. Frame the problem: three different aggregations from the same fact table is a textbook `GROUPING SETS` use case. 2. Write `GROUP BY GROUPING SETS ((region), (category), ())` — that gives one row per region, one per category, and one grand total. 3. The columns that aren't in a particular grouping set come back as `NULL` — that's the standard SQL behaviour and it's what `GROUPING()` exists to disambiguate later. 4. Mention the win: a single sequential scan with hash aggregation, versus three full table scans for three `UNION` blocks.

Edge cases to mention: - Postgres, SQL Server, Oracle, DB2, Snowflake, BigQuery all support `GROUPING SETS`. MySQL does not — you'd fall back to `UNION ALL` there until MySQL adds it. - The `NULLs` returned for non-grouped columns can collide with real data `NULLs` — that's exactly what `GROUPING()` and `GROUPING_ID()` solve in the next questions. - `ORDER BY` on a `GROUPING SETS` result is tricky because of those `NULLs` — usually you sort with `GROUPING()` first to push subtotals to the bottom.

What NOT to say: - Do not claim `GROUPING SETS` is just syntactic sugar for `UNION ALL` — the execution plan is genuinely different and much cheaper. - Do not forget the empty `()` set — without it you don't get the grand total row.

Common follow-up: "Compare `GROUPING SETS` to running three queries and combining them in the application layer." — One round-trip, one scan, one consistent snapshot of the data versus three of each.

```
SELECT region, category, SUM(amount) AS total_sales
FROM orders
GROUP BY GROUPING SETS ((region), (category), ());
```

Q446 · ROLLUP — Hierarchical Subtotals

Tags: Difficulty: Medium · Asked at: Infosys, Cognizant, Goldman, Target · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

I want sales broken down by region, then country within region, then city within country — with subtotals at each level and a grand total. Write that in one query.

The 30-second answer: Use `GROUP BY ROLLUP(region, country, city)`. `ROLLUP` generates the hierarchical sequence (region, country, city), (region, country), (region), and `()` — exactly the four levels you need.

Step-by-step thought process: 1. Define ROLLUP: it's a shortcut for a specific GROUPING SETS pattern — strip rightmost columns one at a time until you reach the empty set. 2. `ROLLUP(a, b, c)` expands to `GROUPING SETS ((a, b, c), (a, b), (a), ())` — emphasise that it is hierarchical, not all combinations. 3. Order matters: `ROLLUP(country, region)` would not make geographic sense because country is the wider bucket — list from broadest to narrowest. 4. This is the standard tool for a financial summary: branch → region → country → grand total.

Edge cases to mention: - ROLLUP is in the SQL standard since SQL:1999 — Postgres, Oracle, SQL Server, MySQL 8, DB2, Snowflake all support it. - The grand-total row has NULL in every grouped column — same disambiguation problem as GROUPING SETS, solved with GROUPING(). - ROLLUP can be combined with regular columns: `GROUP BY year, ROLLUP(region, city)` rolls up only the geography axis.

What NOT to say: - Do not confuse ROLLUP with CUBE — ROLLUP is hierarchical, CUBE is every combination. Mixing them up tags you as a beginner. - Do not write three separate `SUM ... GROUP BY` queries — that's exactly what ROLLUP exists to replace.

Common follow-up: "What is the relationship between the number of columns in ROLLUP and the number of grouping sets generated?" — $N+1$ grouping sets for N columns.

```
SELECT region, country, city, SUM(amount) AS total
FROM orders
GROUP BY ROLLUP(region, country, city);
```

Q447 · CUBE — Every Combination of Dimensions

Tags: Difficulty: Medium · Asked at: JP Morgan India, Wells Fargo, AmEx · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

What does CUBE do, and when would you reach for it over ROLLUP?

The 30-second answer: `CUBE(a, b, c)` generates every possible grouping combination — 2^N grouping sets for N columns. Use it for BI dashboards where the analyst slices by every dimension; use ROLLUP when there's a natural hierarchy.

Step-by-step thought process: 1. Spell out the expansion: `CUBE(a, b)` is `GROUPING SETS ((a, b), (a), (b), ())` — four sets for two columns, eight for three. 2. The classic use case: an OLAP-style summary report where users want to slice sales by region, by category, by region+category, and overall — all in one trip. 3. Compare to ROLLUP: ROLLUP gives $N+1$ sets, CUBE gives 2^N — CUBE is exponentially more expensive on wide cube definitions. 4. In practice CUBE over four or more columns gets expensive fast (16+ aggregation groups) — limit dimensionality or partition the data.

Edge cases to mention: - Supported in Postgres, SQL Server, Oracle, DB2, MySQL 8.0+, Snowflake — same support matrix as ROLLUP. - A CUBE of three boolean dimensions generates 8 grouping sets, which is fine; a CUBE of seven dimensions is 128, which the planner will hate. - Mixing GROUPING SETS, ROLLUP and CUBE is allowed by the standard:

`GROUP BY a, ROLLUP(b, c), CUBE(d, e)` — useful but always check your dialect.

What NOT to say: - Do not assume CUBE is "better" than ROLLUP — they answer different questions. - Do not run CUBE over a dozen columns and complain about performance; the cost is in the spec.

Common follow-up: "For CUBE over four dimensions, how many grouping sets are generated?" — $2^4 = 16$.

```
SELECT region, category, SUM(amount) AS total
FROM orders
GROUP BY CUBE(region, category);
```

Q448 · GROUPING() — Telling Data NULL from Rollup NULL

Tags: Difficulty: Hard · Asked at: Goldman, Walmart Labs, Razorpay · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

When you use ROLLUP, the subtotal rows have NULL in the rolled-up columns. But your `region` column also has genuine NULLs for rows where region wasn't recorded. How do you tell them apart?

The 30-second answer: Use the `GROUPING(column)` function. It returns 1 when the NULL was injected by ROLLUP/CUBE/GROUPING SETS, and 0 when the NULL came from the underlying data. Wrap it in a CASE to label subtotals correctly.

Step-by-step thought process: 1. Define the ambiguity: a row with `region = NULL` could mean "data didn't have a region" or "this is the subtotal across all regions" — they collide visually. 2. Show the disambiguation pattern: `CASE WHEN GROUPING(region) = 1 THEN 'All regions' ELSE COALESCE(region, 'Unknown') END`. 3. Stress that this is what makes ROLLUP and CUBE actually usable in reports — without GROUPING(), readers can't tell totals from missing data. 4. Be ready to chain it with COALESCE for the rare third case: actual NULL data that you want to display as "Unknown".

Edge cases to mention: - GROUPING() takes one column argument; for multi-column checks use GROUPING_ID (next question). - Supported in Postgres, SQL Server, Oracle, DB2, MySQL 8.0+ — the support matrix matches ROLLUP/CUBE. - It only makes sense inside a query with GROUPING SETS, ROLLUP, or CUBE — using it elsewhere returns 0 for everything.

What NOT to say: - Do not try to detect subtotal rows by checking for NULL — that's exactly the bug GROUPING() was invented to prevent. - Do not skip GROUPING() because "my data has no NULLs" — schemas change, and reports break silently when they do.

Common follow-up: "Order the result so subtotals appear at the bottom of each group." — `ORDER BY GROUPING(region), region` puts data rows first, then subtotals.

```
SELECT
  CASE WHEN GROUPING(region) = 1 THEN 'All regions' ELSE region END AS region,
  SUM(amount) AS total
FROM orders
GROUP BY ROLLUP(region);
```

Q449 · GROUPING_ID — Bitmap Across Multiple Columns

Tags: Difficulty: Hard · Asked at: JP Morgan India, Target, Snowflake-style rounds · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

If I CUBE over three columns, how do I tell which specific combination each result row represents — without writing three separate GROUPING() calls?

The 30-second answer: `GROUPING_ID(a, b, c)` returns a single integer whose binary digits are the GROUPING bits for each column. So `GROUPING_ID=0` means all three columns are real data, `GROUPING_ID=7` (binary 111) is the grand total, and `GROUPING_ID=4` (binary 100) means a is aggregated but b and c are real.

Step-by-step thought process: 1. Show the bit layout: leftmost argument is the highest bit. `GROUPING_ID(region, country, city)` returns 0 for (R, C, Ci), 1 for (R, C, total), 2 for (R, total, Ci), and so on up to 7 for the grand total. 2. Demonstrate the typical use: filter or label subtotal rows by integer comparison instead of chained GROUPING() calls. 3. Stress that the bit order is by argument position — get it wrong and you'll mislabel every row in the report. 4. Mention dialect quirks: SQL Server uses GROUPING_ID with the same semantics; Oracle pioneered it.

Edge cases to mention: - Postgres added GROUPING_ID functionality through GROUPING(a, b, c) returning the bitmap directly — slight syntax difference but identical effect. - MySQL 8.0+ supports it as well; check your specific version. - The bitmap interpretation makes HAVING filters very clean: `HAVING GROUPING_ID(region, city) = 0` keeps only the lowest-level rows.

What NOT to say: - Do not memorise specific integers in interviews — explain the bit-position reasoning, that's what they're testing. - Do not confuse GROUPING_ID with GROUPING — the former takes multiple columns and returns an integer, the latter takes one and returns 0 or 1.

Common follow-up: *"Why is this more useful than chained GROUPING() calls?"* — Single integer comparison is cheaper to write, cheaper to index in derived tables, and unambiguous in audit reports.

```
SELECT region, country, city, GROUPING_ID(region, country, city) AS level, SUM(amount)
FROM orders
GROUP BY CUBE(region, country, city);
```

Q450 · HAVING with GROUPING SETS

Tags: Difficulty: Hard · Asked at: Walmart Labs, Wells Fargo, Razorpay · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

I want a CUBE report but only the subtotal rows, not the leaf rows. How do I filter that out?

The 30-second answer: Use `HAVING GROUPING_ID(...) > 0`, which keeps any row where at least one column has been aggregated away. Filtering on grouping-set membership belongs in HAVING because it operates on grouped output, not raw rows.

Step-by-step thought process: 1. Start with where the filter belongs — WHERE runs before grouping, HAVING after, so anything involving GROUPING/GROUPING_ID must be in HAVING. 2. Show three useful filters: `HAVING GROUPING_ID(region, city) = 0` for leaf rows only, `> 0` for subtotals only, `= 3` for grand total only. 3. Combine with normal HAVING predicates: `HAVING SUM(amount) > 100000 AND GROUPING(region) = 0` — keeps high-value rows only for real regions, not the rollup row. 4. This is the standard pattern in BI dashboards where the UI wants only "rollup rows above a threshold".

Edge cases to mention: - WHERE clause cannot reference GROUPING() because the grouping hasn't happened yet — common syntax error. - ORDER BY with GROUPING_ID is the easiest way to put subtotals at the bottom of each section. - In Postgres you can also use the boolean-returning GROUPING(col) directly in HAVING.

What NOT to say: - Do not try to filter subtotals in WHERE — the engine will reject it or worse silently drop the aggregated rows. - Do not use complex CASE expressions when GROUPING_ID does the job in one comparison.

Common follow-up: *"Show me only the rows where the grand total of amount is greater than 10 lakh."* — `HAVING GROUPING_ID(...) = 3 AND SUM(amount) > 1000000`.

```
SELECT region, city, SUM(amount) AS total
FROM orders
GROUP BY CUBE(region, city)
HAVING GROUPING_ID(region, city) > 0;
```

Q451 · FILTER Clause — Cleaner Conditional Aggregation

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postgres-heavy shops · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

Show me how the `FILTER` clause replaces `SUM(CASE WHEN ... THEN amount END)` and tell me why it's preferred.

The 30-second answer: `SUM(amount) FILTER (WHERE status = 'paid')` is the SQL standard way to write conditional aggregation. It's shorter, more readable, and on Postgres often slightly faster because the planner can push the filter down.

Step-by-step thought process: 1. Show the side-by-side: `SUM(CASE WHEN status='paid' THEN amount END)` versus `SUM(amount) FILTER (WHERE status='paid')`. Same result, half the noise. 2. Explain why interviewers care: it signals familiarity with modern standard SQL (added in SQL:2003, common since 2010). 3. The big win is in pivot-style queries with five or six different per-status totals — the `FILTER` version reads like a table of definitions, the `CASE` version reads like obfuscated C. 4. Be ready to discuss support: Postgres has it natively, SQLite added it in 3.30, Oracle and SQL Server still need the `CASE` form, MySQL 8 supports it through certain functions only.

Edge cases to mention: - `FILTER` works on any aggregate: `COUNT(*) FILTER (WHERE ...)`, `AVG(...) FILTER (WHERE ...)`, even `STRING_AGG(...) FILTER (WHERE ...)`. - The `FILTER` predicate sees raw rows (it runs before aggregation) — you cannot reference aggregate results inside it. - In dialects without `FILTER`, the equivalent rewrite is mechanical: replace `agg(x) FILTER (WHERE p)` with `agg(CASE WHEN p THEN x END)`.

What NOT to say: - Do not say `FILTER` is "Postgres-specific" — it's been SQL standard for over two decades. - Do not nest `FILTER` inside `FILTER` — it does not chain; one `FILTER` per aggregate.

Common follow-up: "What's the `FILTER` rewrite of `COUNT(DISTINCT user_id) WHERE region='South'`?" — `COUNT(DISTINCT user_id) FILTER (WHERE region='South')`.


```
SELECT
  SUM(amount) FILTER (WHERE status = 'paid') AS paid_total,
  SUM(amount) FILTER (WHERE status = 'refunded') AS refund_total,
  COUNT(*) FILTER (WHERE status = 'failed') AS failed_count
FROM orders;
```

Q452 · FILTER vs CASE in SUM — Performance and Readability

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Cred · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

Is there any real performance difference between `SUM(amount) FILTER (WHERE status='paid')` and `SUM(CASE WHEN status='paid' THEN amount END)` ?

The 30-second answer: On most engines they produce the same plan and the same speed. The win for FILTER is readability and standardisation — the planner treats both equivalently after parsing. Choose FILTER when your dialect supports it.

Step-by-step thought process: 1. Show the equivalence: both forms produce identical EXPLAIN output on Postgres 12+ for simple cases. 2. The readability argument matters in code review — six FILTER clauses for six per-status totals reads cleanly; six CASE expressions become a wall of nested conditions. 3. The honest caveat: in some older optimizers, FILTER allowed planner shortcuts that the CASE-form missed — modern optimizers have closed the gap. 4. Anticipate the gotcha: when the predicate is complex (joins, subqueries), FILTER and CASE may still differ — benchmark on your real schema.

Edge cases to mention: - For `COUNT(*) FILTER (WHERE ...)`, the equivalent CASE is `COUNT(CASE WHEN ... THEN 1 END)` — note that 1, not TRUE, because COUNT ignores NULLs and TRUE-vs-FALSE wouldn't reduce. - FILTER plays nicely with window functions: `SUM(x) FILTER (WHERE ...) OVER (...)` is supported on Postgres. - Some optimizers (Snowflake, BigQuery) only honour FILTER through partial native support — check your specific version before assuming.

What NOT to say: - Do not assert "FILTER is always faster" — measurable differences are rare on modern engines. - Do not assert "they're identical" without the caveat — they can differ on edge cases.

Common follow-up: *"When would CASE be the only option?"* — On dialects without FILTER support (Oracle, SQL Server, older MySQL).


```
-- FILTER form
SELECT region, SUM(amount) FILTER (WHERE status = 'paid') AS paid
FROM orders GROUP BY region;

-- Equivalent CASE form
SELECT region, SUM(CASE WHEN status = 'paid' THEN amount END) AS paid
FROM orders GROUP BY region;
```

Q453 · ANY_VALUE — Picking a Representative Without MIN/MAX Hack

Tags: Difficulty: Medium · Asked at: Walmart Labs, Razorpay, BigQuery shops · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

I want one row per user with their latest order's region, but `region` is not in the GROUP BY. I don't care which region — any will do. How do I write that cleanly?

The 30-second answer: Use `ANY_VALUE(region)`. It tells the engine "pick any value from the group, I don't care which" — without the MIN/MAX hack and without ONLY_FULL_GROUP_BY errors. It's in MySQL 8.0+, Postgres 16, BigQuery, Snowflake, and SQL:2023.

Step-by-step thought process: 1. Spell out the problem: in strict GROUP BY mode, you can't select a non-aggregated column unless it's a group key — but sometimes you genuinely don't care which value comes back. 2. Show the old workaround: `MIN(region)` or `MAX(region)` — which signals to readers that you're picking a deterministic value, even though you don't care. 3. ANY_VALUE expresses intent — "any one is fine" — which is documentation as well as semantics. 4. Mention performance: ANY_VALUE can be slightly cheaper than MIN/MAX because the engine doesn't need to sort or compare; it grabs the first value it sees.

Edge cases to mention: - Postgres only added ANY_VALUE in 16 — before that, the idiom was MIN/MAX or a window function with LIMIT. - BigQuery and Snowflake had it earlier than the mainstream relational engines. - ANY_VALUE on a strict aggregation function still requires that the column actually has values; on empty groups it returns NULL just like MIN.

What NOT to say: - Do not say "use MIN, it does the same thing" — it does, but ANY_VALUE is more expressive and may be cheaper. - Do not assume ANY_VALUE is deterministic — by definition it's not, and that's the point.

Common follow-up: "What happens if I use ANY_VALUE on Postgres 15?" — It doesn't exist; you'd get a function-not-found error. Use MIN/MAX as the portable fallback.

```
SELECT user_id, ANY_VALUE(region) AS some_region, SUM(amount) AS total
FROM orders
GROUP BY user_id;
```

Q454 · ARRAY_AGG, JSONB_AGG, STRING_AGG — Collecting into Composites

Tags: Difficulty: Medium · Asked at: Swiggy, Flipkart, Postgres shops · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

I want one row per customer with a list of all their order IDs. How would you collect them — array, JSON or comma-separated string — and what changes?

The 30-second answer: `ARRAY_AGG(order_id)` returns a native array, `JSONB_AGG(order_id)` returns a JSON array, `STRING_AGG(order_id::text, ',')` returns a CSV string. Pick array for further SQL processing, JSON for API payloads, string for human display.

Step-by-step thought process: 1. Lay out the three idioms — each one solves the same shape problem with a different output type. 2. `ARRAY_AGG` is the workhorse on Postgres for collecting into arrays, which you can then unnest, slice or pass to PL/pgSQL. 3. `JSONB_AGG` is what you want when the next consumer is an HTTP API — no string parsing needed downstream. 4. `STRING_AGG` is for reports and CSV exports — the separator is a required argument, and you usually cast to text first.

Edge cases to mention: - MySQL has `GROUP_CONCAT` instead of `STRING_AGG` with a slightly different syntax: `GROUP_CONCAT(col SEPARATOR ',')`. SQL Server got `STRING_AGG` in 2017. - All three skip NULLs by default — to include NULLs as 'NULL' literally, COALESCE them inside the aggregate. - `STRING_AGG` has a hard length limit on MySQL's `GROUP_CONCAT` (default 1024 bytes) — bump `group_concat_max_len` or move to a different aggregate.

What NOT to say: - Do not concatenate manually with `||` and a subquery — that's an antipattern when `STRING_AGG` exists. - Do not use `STRING_AGG` for data that another query will need to parse back into rows; use `ARRAY_AGG` or JSON instead.

Common follow-up: "What if I want each customer's orders as an array of (id, amount) tuples?" — `JSONB_AGG(JSONB_BUILD_OBJECT('id', id, 'amount', amount))`.

```
SELECT
  customer_id,
  ARRAY_AGG(order_id) AS order_ids,
  STRING_AGG(order_id::text, ',') AS order_csv
FROM orders
GROUP BY customer_id;
```

Q455 · ARRAY_AGG ORDER BY — Controlling Element Order

Tags: Difficulty: Medium · Asked at: Postgres shops, Razorpay, Swiggy · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

Your ARRAY_AGG result comes back in random order. The API contract says it must be sorted by order date descending. How do you guarantee that?

The 30-second answer: Use `ARRAY_AGG(order_id ORDER BY order_date DESC)` — the ORDER BY clause goes inside the aggregate. Without it the order is implementation-defined and can change between runs.

Step-by-step thought process: 1. State the rule: in SQL standard, aggregate-internal ORDER BY is the only way to guarantee element order in ARRAY_AGG, JSONB_AGG, STRING_AGG. 2. Without ORDER BY, the planner picks whatever order is convenient — often insertion order, but never guaranteed. 3. The ORDER BY can use any column, not just the aggregated one: `ARRAY_AGG(order_id ORDER BY order_date DESC)` orders the array elements by a different column entirely. 4. Combine with FILTER for conditional ordered aggregation: `ARRAY_AGG(order_id ORDER BY order_date DESC) FILTER (WHERE status='paid')`.

Edge cases to mention: - MySQL's GROUP_CONCAT has its own `ORDER BY` syntax inside the function — same idea, slightly different keyword placement. - ORDER BY inside an aggregate is allowed by the SQL standard since SQL:2003 and supported on Postgres, SQL Server (STRING_AGG WITHIN GROUP), Oracle (LISTAGG WITHIN GROUP), DB2. - Window aggregates with frame clauses give you another way to order results, but inside ARRAY_AGG the internal ORDER BY is the right tool.

What NOT to say: - Do not rely on the order of rows fed into the aggregate — that's the "input order is preserved" myth, and it's not guaranteed by the standard. - Do not sort after aggregation with `array_sort(ARRAY_AGG(...))` when the internal ORDER BY does the job in one step.

Common follow-up: "Now sort by amount descending, breaking ties by order date." — `ARRAY_AGG(order_id ORDER BY amount DESC, order_date)`.

```
SELECT customer_id, ARRAY_AGG(order_id ORDER BY order_date DESC) AS recent_orders
FROM orders
GROUP BY customer_id;
```

Q456 · ARRAY_AGG DISTINCT — Deduplicating Inside the Aggregate

Tags: Difficulty: Easy · Asked at: Postgres shops, Cred, Meesho · Topic: Modern Aggregation

The question (interviewer's verbatim phrasing):

Each customer might order the same category multiple times. I want one row per customer with a list of distinct categories. How?

The 30-second answer: `ARRAY_AGG(DISTINCT category)` does it in one step — `DISTINCT` is a modifier on the aggregate, not a subquery construct. Same idea works with `STRING_AGG`, `JSONB_AGG`, `COUNT`, `SUM`.

Step-by-step thought process: 1. Explain that `DISTINCT` inside an aggregate eliminates duplicate input values before collecting — much cleaner than aggregating an already-deduped subquery. 2. Show the side-by-side: `ARRAY_AGG(DISTINCT category)` versus `(SELECT ARRAY_AGG(category) FROM (SELECT DISTINCT category ...) sub)` — the inline form is far shorter and the planner usually generates the same hashing. 3. Combining `DISTINCT` with `ORDER BY`: the standard requires that the `ORDER BY` expression be in the `DISTINCT` list — `ARRAY_AGG(DISTINCT category ORDER BY category)` is fine, `ORDER BY some_other_column` is not. 4. Useful with `COUNT` too: `COUNT(DISTINCT user_id)` is the classic distinct-count idiom.

Edge cases to mention: - `DISTINCT` inside the aggregate can be expensive on large groups — it needs to maintain a hash set per group. - Some engines (BigQuery) have approximate alternatives like `APPROX_COUNT_DISTINCT` for billion-row distinct counts. - `DISTINCT` with `FILTER`: `COUNT(DISTINCT user_id) FILTER (WHERE region='South')` — works on engines that support both.

What NOT to say: - Do not write `ARRAY_AGG(category) FROM (SELECT DISTINCT ...)` as your first answer — the inline `DISTINCT` is shorter and idiomatic. - Do not assume `DISTINCT` preserves order — without `ORDER BY` it doesn't.

Common follow-up: "How does this perform on a customer with a million orders?" — Worst case $O(n \log n)$ for sort-based `DISTINCT`, $O(n)$ for hash-based; check `EXPLAIN` to see which the planner chose.

```
SELECT customer_id, ARRAY_AGG(DISTINCT category ORDER BY category) AS categories
FROM orders
GROUP BY customer_id;
```

Q457 · IS DISTINCT FROM — NULL-Safe Comparison

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan India, Postgres shops · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

`WHERE a <> b` doesn't return rows where one side is NULL. How do you write "different value or one is NULL" in pure SQL?

The 30-second answer: Use `a IS DISTINCT FROM b`. It returns TRUE when values differ, and treats two NULLs as equal (so it returns FALSE), and one NULL versus a value as different (TRUE). It's NULL-safe by definition.

Step-by-step thought process: 1. Start with the problem: standard `<>` returns UNKNOWN whenever a NULL is involved, and UNKNOWN is filtered out by WHERE. 2. So `WHERE old_value <> new_value` silently misses every row where one of them is NULL — that's the change-tracking bug that catches every junior dev. 3. `IS DISTINCT FROM` is the SQL standard NULL-safe inequality: `NULL IS DISTINCT FROM 5` → TRUE, `NULL IS DISTINCT FROM NULL` → FALSE, `3 IS DISTINCT FROM 5` → TRUE. 4. The opposite `IS NOT DISTINCT FROM` is NULL-safe equality — covered in the next question.

Edge cases to mention: - Supported in Postgres, SQL Server 2022, Oracle (since 12c), DB2, SQLite 3.32+. MySQL uses the `<=>` operator with inverted semantics (`<=>` is NULL-safe equality, so NOT (a <=> b) is the equivalent of IS DISTINCT FROM). - Inside CHECK constraints and partial indexes, IS DISTINCT FROM is often the only correct comparison. - For trigger-based audit tables, `IF OLD.x IS DISTINCT FROM NEW.x THEN ...` is the textbook pattern.

What NOT to say: - Do not pad with COALESCE to dodge the NULL problem — `COALESCE(a, '') <> COALESCE(b, '')` breaks when '' is a real value in your domain. - Do not say `<>` "ignores NULLs" — it returns UNKNOWN, which the WHERE clause then filters out. The distinction matters when interviewers probe deeper.

Common follow-up: "How do you spell IS DISTINCT FROM in MySQL?" — `NOT a <=> b`, where `<=>` is NULL-safe equals.

```
-- Find rows where the new value differs from the old (including NULL transitions)
SELECT id FROM audit
WHERE old_status IS DISTINCT FROM new_status;
```

Q458 · IS NOT DISTINCT FROM — NULL-Safe Equality

Tags: Difficulty: Medium · Asked at: Postgres shops, Goldman, Walmart Labs · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

I'm joining on a nullable column and I want NULL=NULL to be a match. How do I write that in a join condition?

The 30-second answer: `ON a.col IS NOT DISTINCT FROM b.col`. This is NULL-safe equality: two NULLs match, two equal values match, mismatched values don't, NULL and non-NULL don't.

Step-by-step thought process: 1. Restate the SQL default: `a.col = b.col` returns UNKNOWN when either side is NULL, so the join never matches NULL-to-NULL. 2. Sometimes that's wrong: in a "category mapping" table where NULL means "any category", you genuinely want NULL on both sides to match. 3. `IS NOT DISTINCT FROM` is the SQL standard way. It's a single operator, no COALESCE tricks needed. 4. In MySQL it's spelled `a.col <=> b.col` — same semantics, different syntax.

Edge cases to mention: - Joining on `IS NOT DISTINCT FROM` is usually a hint that your schema has too many nullable join columns — consider making them NOT NULL with a sentinel value if matching is part of the contract. - The optimizer in Postgres can use indexes on `IS NOT DISTINCT FROM` joins, but only in newer versions (PG 16+); older versions fall back to a full scan. - Partial unique indexes often use this idiom: `CREATE UNIQUE INDEX ... WHERE deleted_at IS NULL` — that handles NULL-safe uniqueness without IS DISTINCT FROM in the condition.

What NOT to say: - Do not write `ON COALESCE(a.col, -1) = COALESCE(b.col, -1)` as your default — it works but breaks the moment -1 is a real value. - Do not assert that `=` joins NULLs — it never does in standard SQL.

Common follow-up: "What's the difference between IS NOT DISTINCT FROM and `=`?" — They're identical for non-NULL values; they differ only when either side is NULL.

```
SELECT a.id, b.id
FROM table_a a
JOIN table_b b ON a.region IS NOT DISTINCT FROM b.region;
```

Q459 · COALESCE — First Non-NULL, Multiple Args, Short-Circuit

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Accenture · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

What does COALESCE do and how is it different from NVL or IFNULL?

The 30-second answer: `COALESCE(a, b, c, ...)` returns the first non-NULL argument. It takes any number of arguments (NVL and IFNULL take exactly two), is in the SQL standard, and short-circuits — once it finds a non-NULL, it stops evaluating the rest.

Step-by-step thought process: 1. Cover the basics: any number of arguments, returns the first non-NULL, returns NULL only if all arguments are NULL. 2. Stress short-circuiting: in

`COALESCE(cheap_col, expensive_subquery())`, the subquery isn't run when `cheap_col` is not NULL — important for performance. 3. Compare with vendor-specific functions: NVL (Oracle) is exactly two args, IFNULL (MySQL/SQLite) is also two, ISNULL (SQL Server) is two, COALESCE is the portable n-ary option. 4. The "default value" use case: `COALESCE(discount_pct, 0)` so that downstream math doesn't propagate NULL.

Edge cases to mention: - All arguments must be type-compatible — `COALESCE('abc', 5)` errors on strict engines. - COALESCE with side-effecting functions (sequences, random) is a known footgun because of short-circuiting; the un-evaluated ones don't fire. - For default columns in INSERT, prefer the DEFAULT clause over COALESCE in application code — the DB enforces it consistently.

What NOT to say: - Do not say COALESCE is "the same as IFNULL but with more arguments" — that's only mostly true; the standard guarantees apply only to COALESCE. - Do not call it "ko-uh-LESS" — it's "co-uh-LESS" but pronunciation rarely matters; what matters is correct usage.

Common follow-up: "Write a query that gives every order a display name: order's nickname if set, else customer's name, else 'Anonymous'." — `COALESCE(order_nickname, customer_name, 'Anonymous')`.

```
SELECT id, COALESCE(discount_pct, 0) AS discount, COALESCE(notes, comments, 'N/A') AS notes
FROM orders;
```

Q460 · NULLIF — Opposite of COALESCE

Tags: Difficulty: Easy · Asked at: Infosys, Wipro, Walmart Labs · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

When would you reach for NULLIF, and what does it actually do?

The 30-second answer: `NULLIF(a, b)` returns NULL if `a = b`, otherwise returns `a`. Classic use case: avoid divide-by-zero with `amount / NULLIF(quantity, 0)` — when quantity is zero you get NULL instead of an error.

Step-by-step thought process: 1. Open with the divide-by-zero idiom — that's the textbook NULLIF use. 2. The general pattern: NULLIF normalises a sentinel value to NULL. `NULLIF(country, '')`

turns empty strings into proper NULLs before aggregation. 3. Pair it with COALESCE for clean defaults: `COALESCE(NULLIF(notes, ''), 'No notes')` — empty string becomes NULL, then COALESCE substitutes the default. 4. SQL standard since SQL-92, supported everywhere.

Edge cases to mention: - `NULLIF` requires both arguments to be type-compatible — same rule as COALESCE. - It's an inverse of sorts to COALESCE: COALESCE turns NULL into a default, NULLIF turns a default into NULL. - For string normalisation across data imports, `NULLIF(TRIM(col), '')` is a one-liner that catches both whitespace-only and empty cells.

What NOT to say: - Do not confuse NULLIF with IFNULL — IFNULL is MySQL's COALESCE shortcut, NULLIF is the standard "return NULL if equal" function. - Do not use NULLIF for floats with `NULLIF(x, 0.0)` and then expect it to catch values like 0.0000001 — equality is exact.

Common follow-up: "What's the result of `NULLIF(NULL, NULL)` ?" — NULL, because NULL = NULL is UNKNOWN, not TRUE.

```
-- Avoid divide-by-zero in average price per unit
SELECT order_id, amount / NULLIF(quantity, 0) AS unit_price
FROM order_lines;
```

Q461 · NVL, IFNULL, ISNULL, COALESCE — Vendor Comparison

Tags: Difficulty: Easy · Asked at: Multi-DB shops, GCCs, panel rounds · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

Walk me through the difference between NVL, IFNULL, ISNULL and COALESCE — which one should I use in new code?

The 30-second answer: Use `COALESCE` — it's SQL standard, takes any number of arguments, and works on every engine. NVL is Oracle-only (two args), IFNULL is MySQL/SQLite (two args), ISNULL is SQL Server (two args). They all do the "first non-NULL" job for two arguments, but COALESCE wins on portability and flexibility.

Step-by-step thought process: 1. Lay out the four side by side: same semantics for two args, different syntax, different vendor support. 2. Show the portability angle: COALESCE works everywhere, vendor functions don't survive a migration. 3. SQL Server's ISNULL is a footgun: it returns the type of the first argument, while COALESCE returns the highest-precedence type — `ISNULL('abc', 5)` errors differently than `COALESCE('abc', 5)`. 4. NVL has a sibling NVL2 (Oracle): `NVL2(expr, val_if_not_null, val_if_null)` — handy but Oracle-specific.

Edge cases to mention: - IFNULL in MySQL is just **COALESCE** with exactly two arguments — same result, fewer characters. - ISNULL in SQL Server is distinct from the **IS NULL** predicate — they share a name but do completely different things. - For type-strict engines, COALESCE applies standard type unification; NVL/IFNULL/ISNULL apply vendor-specific rules.

What NOT to say: - Do not say "they're identical" — type handling differs, and SQL Server's ISNULL behaviour bites in real production code. - Do not use vendor-specific functions in new code unless you're locked to that vendor — COALESCE is the portable default.

Common follow-up: "What does NVL2 do?" — Oracle-only three-arg variant: returns the second arg if the first is not NULL, the third if it is.

```
-- Portable
SELECT COALESCE(discount, 0) FROM orders;

-- Oracle
SELECT NVL(discount, 0) FROM orders;

-- MySQL
SELECT IFNULL(discount, 0) FROM orders;

-- SQL Server
SELECT ISNULL(discount, 0) FROM orders;
```

Q462 · Three-Valued Logic — TRUE, FALSE, UNKNOWN

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan India, Razorpay · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

SQL has three-valued logic. What's the third value and how does it propagate through AND, OR and NOT?

The 30-second answer: The third value is **UNKNOWN**, returned whenever a comparison involves NULL. **TRUE AND UNKNOWN** = UNKNOWN. **FALSE AND UNKNOWN** = FALSE. **TRUE OR UNKNOWN** = TRUE. **NOT UNKNOWN** = UNKNOWN. WHERE clauses keep only TRUE rows, so UNKNOWN rows are silently dropped.

Step-by-step thought process: 1. State the three values: TRUE, FALSE, UNKNOWN — UNKNOWN is what **x = NULL** returns, what **5 > NULL** returns, what every comparison with NULL returns. 2. AND truth table: TRUE and TRUE = TRUE; TRUE and FALSE = FALSE; TRUE and UNKNOWN = UNKNOWN; FALSE dominates AND. 3. OR truth table: FALSE or FALSE = FALSE; FALSE or TRUE = TRUE; FALSE or

UNKNOWN = UNKNOWN; TRUE dominates OR. 4. NOT UNKNOWN = UNKNOWN — that's the one most candidates forget.

Edge cases to mention: - WHERE keeps only TRUE rows. HAVING also keeps only TRUE rows. CHECK constraints accept TRUE and UNKNOWN — both pass the check. - `IF NOT (status = 'paid')` will not include rows with NULL status, because NULL becomes UNKNOWN, NOT UNKNOWN is UNKNOWN, which isn't TRUE. - IS NULL and IS NOT NULL always return TRUE or FALSE, never UNKNOWN — they're the safe predicates.

What NOT to say: - Do not say "NULL is falsy" — it's not, it's UNKNOWN, and that's a meaningfully different behaviour. - Do not write `WHERE status = NULL`; it's always UNKNOWN, never matches. Use `IS NULL`.

Common follow-up: "Why does `NOT (NULL = NULL)` not return TRUE?" — Because NULL = NULL is UNKNOWN, and NOT UNKNOWN is UNKNOWN, not TRUE.

```
-- Returns no rows even if every order has paid status (because of one NULL)
SELECT * FROM orders WHERE status != 'pending'; -- misses NULL rows

-- Safer
SELECT * FROM orders WHERE status IS DISTINCT FROM 'pending';
```

Q463 · NULL in WHERE — The Silent Exclusion Trap

Tags: Difficulty: Medium · Asked at: Cognizant, Wipro, Wells Fargo, Cred · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

If `status` can be NULL and I write `WHERE status != 'paid'`, what rows do I get back?

The 30-second answer: You get rows where status is exactly any value other than 'paid' — but NOT rows where status is NULL. The comparison `NULL != 'paid'` evaluates to UNKNOWN, and WHERE drops everything except TRUE. The fix is `WHERE status IS DISTINCT FROM 'paid'` or `WHERE status != 'paid' OR status IS NULL`.

Step-by-step thought process: 1. Name the trap: developers often think `!=` is "any other value, including missing", but SQL is strict — NULL is filtered out. 2. The pattern is so common that audit reports built on `WHERE state != 'closed'` silently miss every NULL-state row. 3. Three fixes: `IS DISTINCT FROM` (SQL standard, NULL-safe), `OR col IS NULL` (verbose but portable), or make the column NOT NULL with a sentinel like 'unknown' (best long-term). 4. The general lesson: every WHERE filter on a nullable column needs a NULL-handling decision.

Edge cases to mention: - Same trap exists in HAVING, ON clauses, CASE WHEN, anywhere the engine evaluates a boolean. - Some engines have a "strict NULL" pragma that loudly errors on these — most don't. - Code reviewers should treat every `WHERE col != value` as suspect until the nullability of `col` is known.

What NOT to say: - Do not assume `!=` "includes NULLs" — it never does in standard SQL. - Do not solve it with COALESCE to a sentinel value unless that sentinel is impossible in the real data domain.

Common follow-up: "What about `WHERE status NOT IN ('paid', 'refunded')`?" — Same trap; NULL is excluded silently.

```
-- Misses NULL rows
SELECT * FROM orders WHERE status != 'paid';

-- Correct, NULL-safe
SELECT * FROM orders WHERE status IS DISTINCT FROM 'paid';
```

Q464 · NULL in NOT IN — The Empty Result Trap

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Goldman, JP Morgan India · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

I wrote `WHERE user_id NOT IN (SELECT user_id FROM blocked_users)` and got zero rows back even though most users aren't blocked. What's the bug?

The 30-second answer: Your `blocked_users.user_id` has at least one NULL. `x NOT IN (1, 2, NULL)` evaluates to `x != 1 AND x != 2 AND x != NULL`, and `x != NULL` is UNKNOWN, so the whole AND chain is UNKNOWN, which WHERE drops. Switch to `NOT EXISTS` or filter NULLs out of the subquery.

Step-by-step thought process: 1. This is the most famous NULL trap in SQL — every senior interviewer asks it at some point. 2. Walk through the logic: `5 NOT IN (1, 2, NULL)` expands to `5 != 1 AND 5 != 2 AND 5 != NULL`. The last conjunct is UNKNOWN, dragging the whole AND down to UNKNOWN. 3. The fix: `WHERE NOT EXISTS (SELECT 1 FROM blocked_users b WHERE b.user_id = u.id)`. EXISTS uses correlated lookup, which handles NULLs sensibly — a NULL in `blocked_users.user_id` simply doesn't match anything, it doesn't poison the predicate. 4. Alternative fix: `WHERE user_id NOT IN (SELECT user_id FROM blocked_users WHERE user_id IS NOT NULL)`. Works, but you must remember to write it every time.

Edge cases to mention: - The trap doesn't fire for `IN` (only `NOT IN`) — `x IN (1, 2, NULL)` returns TRUE for 1 and 2, UNKNOWN for everything else; UNKNOWN is dropped by WHERE, but the

matches still come through. - `LEFT JOIN ... WHERE right.col IS NULL` is the third safe anti-join idiom — slower in some plans but always correct. - If the subquery column is declared NOT NULL, `NOT IN` is safe — but don't rely on schema you can't see.

What NOT to say: - Do not say "this is a Postgres bug" — it's the SQL standard, every conforming engine behaves this way. - Do not paper over it with COALESCE — `NOT IN (SELECT COALESCE(user_id, -1) FROM ...)` is brittle and depends on the sentinel being impossible.

Common follow-up: "Rewrite this anti-join using NOT EXISTS and explain why it's safer." — NOT EXISTS uses a per-row correlated check, so a NULL on the right just doesn't match; it doesn't propagate UNKNOWN.

```
-- Trap: returns 0 rows if blocked_users has a NULL user_id
SELECT id FROM users WHERE id NOT IN (SELECT user_id FROM blocked_users);

-- Safe
SELECT id FROM users u WHERE NOT EXISTS (
  SELECT 1 FROM blocked_users b WHERE b.user_id = u.id
);
```

Q465 · NULL in Foreign Key — Allowed but Tricky

Tags: Difficulty: Medium · Asked at: Schema-design rounds, Goldman, AmEx · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

Can a foreign key column be NULL? If yes, what does that mean?

The 30-second answer: Yes — a FK column can be NULL unless you also declare it NOT NULL. A NULL FK means "no parent assigned yet" and is allowed by the FK constraint (NULLs don't need to match a parent row). Interviewers love this because juniors often think every FK needs a parent.

Step-by-step thought process: 1. State the rule: the FK constraint checks "does this value exist in the parent table OR is it NULL". NULL gets a free pass. 2. Use a realistic example: `orders.coupon_id` is FK to `coupons.id`. Most orders have no coupon, so `coupon_id` is NULL — that's intentional and correct. 3. To force every order to have a coupon, add `NOT NULL` to the column itself — the FK alone won't enforce mandatoriness. 4. Composite FKs: if any component is NULL, the entire FK check is skipped by default in standard SQL (MATCH SIMPLE behaviour) — there are MATCH FULL and MATCH PARTIAL options, but most engines only implement SIMPLE.

Edge cases to mention: - ON DELETE SET NULL requires the FK column to be nullable — that's a common reason to keep it NULL-able. - Some ORMs (Hibernate, ActiveRecord) default to making FKs nullable, which then bites you in joins where you forgot to handle the NULL case. - Designing for "always

has a parent" usually means NOT NULL + FK + DEFAULT to a system row like 'no-coupon' to keep joins predictable.

What NOT to say: - Do not say "FKs can't be NULL" — that's wrong unless the column is also NOT NULL. - Do not assume MATCH FULL is the default — it isn't; MATCH SIMPLE is, and it accepts any NULL in a composite key.

Common follow-up: *"Why might I make a FK NOT NULL even if ORM defaults are NULL?"* — Forces every child row to have a parent at insert time; prevents orphans by design rather than by code discipline.

```
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  customer_id BIGINT NOT NULL REFERENCES customers(id),  
  coupon_id BIGINT REFERENCES coupons(id) -- nullable: most orders have no coupon  
);
```

Q466 · NULL in Unique Constraint — Multiple NULLs Usually OK

Tags: Difficulty: Hard · Asked at: SQL Server shops, schema-design rounds, Walmart Labs · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

If I have a UNIQUE constraint on a nullable column, can multiple rows be NULL?

The 30-second answer: On Postgres, Oracle, MySQL, DB2, SQLite — yes, NULL is treated as "unknown, distinct from all other NULLs", so any number of NULLs are allowed. On SQL Server (default) — no, two NULLs collide. The difference is a notorious portability gotcha.

Step-by-step thought process: 1. Lead with the standard view: SQL standard treats two NULLs as distinct for uniqueness purposes — that's the Postgres/Oracle/MySQL behaviour. 2. SQL Server is the outlier: its UNIQUE constraint treats NULL as a single value, so two NULL rows violate the constraint. Workaround: filtered index `WHERE col IS NOT NULL`. 3. Postgres 15 added the `NULLS NOT DISTINCT` option to UNIQUE constraints — opt in if you want SQL Server's behaviour on Postgres. 4. The implication: if you're migrating from SQL Server to Postgres (or vice versa), every UNIQUE on a nullable column needs review.

Edge cases to mention: - Composite uniqueness has the same NULL-distinct rule per row: `(a, b) UNIQUE` allows `(1, NULL)` twice on standard engines. - Partial indexes on `WHERE col IS NOT NULL` are a portable workaround for SQL Server semantics. - ORMs sometimes generate "uniqueness validations" in application code that don't match DB behaviour — always test at the DB level.

What NOT to say: - Do not say "UNIQUE allows multiple NULLs" without the SQL Server caveat — that's exactly the trap. - Do not assume an ORM-level uniqueness check is equivalent to a DB-level UNIQUE — race conditions and NULL handling diverge.

Common follow-up: "How do I get SQL Server's behaviour on Postgres 15?" — `UNIQUE (col) NULLS NOT DISTINCT`.

```
-- Standard SQL (Postgres default): multiple NULLs allowed
CREATE TABLE customers (email VARCHAR(255) UNIQUE);
INSERT INTO customers VALUES (NULL), (NULL); -- OK on Postgres, fails on SQL Server
```

Q467 · NULLS FIRST / NULLS LAST — Controlling NULL Sort Position

Tags: Difficulty: Easy · Asked at: Reporting-heavy shops, Razorpay, Swiggy · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

When I `ORDER BY amount DESC`, where do the NULL amounts end up — top or bottom?

The 30-second answer: It depends on the engine. Postgres puts NULLs last in ASC and first in DESC by default. Oracle does the same. MySQL puts NULLs first in ASC and last in DESC — the opposite. SQL Server puts NULLs first in ASC by default. Use the explicit `NULLS FIRST` or `NULLS LAST` to control it portably.

Step-by-step thought process: 1. Acknowledge the inconsistency upfront — interviewers love this one because the defaults differ across vendors. 2. The standard `ORDER BY col [ASC|DESC] [NULLS FIRST|NULLS LAST]` lets you spell it out. 3. The most common ask in reports: "show NULL values at the bottom regardless of sort direction" — `ORDER BY amount DESC NULLS LAST` does that on Postgres, Oracle, SQLite. 4. MySQL didn't support `NULLS FIRST/LAST` syntax until version 8.0 — the workaround is `ORDER BY col IS NULL, col` (or `IS NOT NULL, col` for the inverse).

Edge cases to mention: - Index ordering: if you have an index on `col DESC NULLS LAST`, Postgres can use it for that exact sort but not for `col DESC NULLS FIRST`. - Combining sort directions: `ORDER BY priority ASC NULLS LAST, created_at DESC` — mix and match per column. - For paginated lists with `ORDER BY created_at DESC`, putting NULLs last keeps the "still being processed" rows at the bottom of every page consistently.

What NOT to say: - Do not assume NULLs sort to a specific position — different engines disagree. - Do not rely on the default ordering for production reports — always spell it out.

Common follow-up: "How would you sort by amount descending, NULLs at the bottom, on MySQL 5.7?" — `ORDER BY amount IS NULL, amount DESC` — the first key pushes NULLs after non-NULL rows.

```
SELECT id, amount FROM orders
ORDER BY amount DESC NULLS LAST;
```

Q468 · NULL in JOIN — Only Matches Explicit IS NULL

Tags: Difficulty: Medium · Asked at: Most product companies, Razorpay, Swiggy · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

Two tables both have a `region` column that can be NULL. If I `JOIN ON a.region = b.region`, do rows with NULL on both sides match?

The 30-second answer: No. `NULL = NULL` evaluates to UNKNOWN, and JOIN ON keeps only TRUE rows. Two NULL regions don't match each other. If you want them to match, use `IS NOT DISTINCT FROM` (Postgres/standard) or `<=>` (MySQL).

Step-by-step thought process: 1. State the rule clearly: standard `=` comparisons return UNKNOWN whenever either side is NULL, including when both are NULL. 2. The fix depends on dialect: `JOIN ON a.region IS NOT DISTINCT FROM b.region` for standard SQL, `JOIN ON a.region <=> b.region` for MySQL. 3. Often the right answer is "don't try to join on NULLs" — model the data so the join column is NOT NULL, with a sentinel like 'unspecified' if needed. 4. Also relevant: LEFT JOIN preserves the left row even when the right doesn't match, so the right-side column comes back NULL — that's a different NULL source than the data NULLs.

Edge cases to mention: - Composite join keys: even one NULL in the composite means the row doesn't match anything in standard SQL. - Outer joins generate NULLs in the unmatched-side columns; downstream filters need to handle both kinds. - Index usage on `IS NOT DISTINCT FROM` was historically limited — modern Postgres handles it; older versions fall back to seq scans.

What NOT to say: - Do not claim NULLs join to each other "because they're both NULL" — that intuition is wrong in SQL. - Do not use COALESCE-to-sentinel in JOIN ON without thinking through whether the sentinel can be a real value elsewhere.

Common follow-up: "What does a LEFT JOIN with no match produce on the right side?" — NULLs in every right-side column for that row.

```
-- These NULLs don't match
SELECT * FROM table_a a JOIN table_b b ON a.region = b.region;

-- These NULLs do match
SELECT * FROM table_a a JOIN table_b b ON a.region IS NOT DISTINCT FROM b.region;
```

Q469 · DEFAULT vs NULL — When Each Is the Right Call

Tags: Difficulty: Medium · Asked at: Schema-design rounds, GCC platform teams, Walmart Labs · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

When designing a schema, when should a column be NULLable and when should it have a DEFAULT?

The 30-second answer: NULL means "this value is genuinely unknown or not applicable". DEFAULT means "this value has a sensible starting value if none is supplied". Use NULL for optional facts (notes, end_date), DEFAULT for mandatory facts with conventional starting values (created_at=CURRENT_TIMESTAMP, is_active=TRUE).

Step-by-step thought process: 1. The rule of thumb: if your business logic says "every row must have a meaningful value here", use NOT NULL with DEFAULT. If the value can legitimately be missing, use NULL. 2. Common DEFAULT patterns: `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`, `status VARCHAR DEFAULT 'pending'`, `retry_count INT DEFAULT 0`. 3. Common NULL patterns: `end_date` (still ongoing), `discount_pct` (no discount), `cancelled_reason` (not cancelled). 4. Boolean columns are the classic case study:

`is_active BOOLEAN NOT NULL DEFAULT TRUE` is almost always better than `is_active BOOLEAN` with NULL meaning "unknown" — because "unknown active state" usually doesn't have a business meaning.

Edge cases to mention: - A column can be both NOT NULL AND have a DEFAULT — those compose: every row gets a value either from the INSERT or from the DEFAULT. - DEFAULT expressions can call functions: `created_by INT DEFAULT current_user_id()` if your DB supports session-scoped defaults. - NULL plus DEFAULT 0 is contradictory at the schema design level — pick one. If the value can be missing meaningfully, NULL it; if it has a default, NOT NULL DEFAULT.

What NOT to say: - Do not make every column NULL "just in case" — it makes queries harder and indexes weaker. - Do not skip DEFAULT and rely on application code to set it — DB-level defaults are the safety net.

Common follow-up: "What about timestamps? `created_at` should be `DEFAULT CURRENT_TIMESTAMP`. What about `updated_at`?" — Same `DEFAULT`, plus a trigger or `ON UPDATE` clause to refresh on modification.

```
CREATE TABLE orders (  
  id BIGINT PRIMARY KEY,  
  status VARCHAR(20) NOT NULL DEFAULT 'pending',  
  cancelled_reason TEXT, -- NULL = not cancelled  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Q470 · NULL in JSON — SQL NULL vs JSON null

Tags: Difficulty: Hard · Asked at: Postgres-heavy shops, Razorpay, Swiggy · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

A JSON column has a field set to `null`. Is that the same as the field being missing, or as the row's column itself being `NULL`?

The 30-second answer: Three distinct things: (1) the whole JSON column is SQL `NULL` — no JSON document at all; (2) the JSON document exists but the field is missing — the key isn't present; (3) the JSON document has the field with explicit JSON null. They behave differently in queries — extracting a missing field returns SQL `NULL`, extracting an explicit `null` returns a JSON null.

Step-by-step thought process: 1. Lay out the three states with examples. Column = `NULL`: `SELECT data FROM t` returns `NULL`. Field missing: `data->'discount'` returns SQL `NULL`. Field explicitly null: `data->'discount'` returns JSON null, which Postgres distinguishes from SQL `NULL`. 2. In Postgres, `data ? 'discount'` tests whether the key exists at all (catches difference between cases 2 and 3). 3. `data->>'discount' IS NULL` returns `TRUE` for both missing keys and explicit JSON null — be deliberate about which one you want. 4. Filtering practical example: rows with `data->>'status' = 'paid'` will miss both the missing-key rows and the JSON-null rows — those need separate handling.

Edge cases to mention: - `JSONB_TYPEOF(data->'discount')` returns `'null'` for explicit JSON null versus `NULL` (SQL `NULL`) for missing keys — that's how you tell them apart. - API contracts often differ: "field absent" might mean "no opinion", "field is null" might mean "explicit no value". Treat them differently in your queries. - Storing JSON null as a way of saying "no value" is usually inferior to omitting the key — keeps the document smaller and the semantics cleaner.

What NOT to say: - Do not treat JSON null the same as SQL NULL; modern engines distinguish them, and so should you. - Do not assume `data->'x' IS NULL` covers all the missing/null cases — it covers some but conflates others.

Common follow-up: "How do you find rows where the JSON has an explicit `discount: null` (not just missing)?" — `WHERE JSONB_TYPEOF(data->'discount') = 'null'`.

```
-- Distinguishing JSON null from missing key
SELECT id,
CASE
  WHEN data IS NULL THEN 'no document'
  WHEN NOT (data ? 'discount') THEN 'key missing'
  WHEN JSONB_TYPEOF(data->'discount') = 'null' THEN 'explicit null'
  ELSE 'has value'
END AS state
FROM orders;
```

Q471 · Aggregates and NULL — COUNT(col) Ignores Them

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant, Accenture · Topic: NULL Handling

The question (interviewer's verbatim phrasing):

Out of a million order rows where 100 have NULL `amount`, what does `COUNT(*)` return, what does `COUNT(amount)` return, and what does `SUM(amount)` do with the NULLs?

The 30-second answer: `COUNT(*)` returns 1,000,000 (counts rows, NULL or not). `COUNT(amount)` returns 999,900 (counts non-NULL values only). `SUM(amount)` ignores NULLs and sums the rest — but if every value were NULL, SUM would return NULL, not 0.

Step-by-step thought process: 1. State the rule: aggregate functions (SUM, AVG, MIN, MAX, COUNT(col)) skip NULL inputs. COUNT() is the only one that counts rows including NULL. 2. AVG is the famous one: `AVG(amount)` divides SUM by COUNT(amount), not by COUNT() — so 100 NULL rows don't drag the average toward zero. 3. Empty groups: SUM, AVG, MIN, MAX of zero non-NULL rows is NULL, not 0. COUNT of zero non-NULL rows is 0. 4. For "treat NULL as zero", wrap with COALESCE inside or outside the aggregate depending on intent: `SUM(COALESCE(amount, 0))` versus `COALESCE(SUM(amount), 0)`.

Edge cases to mention: - `COUNT(DISTINCT col)` also ignores NULL — distinct counts are over non-NULL values only. - For "ratio of non-NULL to total": `COUNT(amount) * 1.0 / COUNT(*)` — gives fraction of rows with values. - In window aggregates, NULL is still skipped inside the frame.

What NOT to say: - Do not say `SUM` returns 0 on empty input — it returns NULL. - Do not confuse `COUNT(*)` and `COUNT(col)` — they answer different questions.

Common follow-up: "If I have 100 rows with NULL amount and want the average amount including them as 0, how do I write it?" — `SUM(COALESCE(amount, 0)) / COUNT(*)`, not `AVG(COALESCE(amount, 0))`. The COALESCE inside AVG also works but the explicit form documents intent better.

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(amount) AS rows_with_amount,
  SUM(amount) AS sum_amount,
  AVG(amount) AS avg_of_non_null,
  COALESCE(SUM(amount), 0) AS sum_or_zero
FROM orders;
```

Q472 · Putting It All Together — Modern Reporting Query

Tags: Difficulty: Hard · Asked at: BI rounds, GCC analytics teams, Walmart Labs, Target · Topic: Modern Aggregation + NULL Handling

The question (interviewer's verbatim phrasing):

Write one query that produces a region-by-category sales report with subtotals at each level and a grand total. Use FILTER to break out paid versus refunded amounts, COALESCE to label rollup rows cleanly, and ANY_VALUE for the most-recent currency seen per group.

The 30-second answer: Combine `GROUP BY ROLLUP(region, category)` for the hierarchy, `SUM(amount) FILTER (WHERE status='paid')` and `FILTER (WHERE status='refunded')` for the conditional totals, `ANY_VALUE(currency)` for the representative currency, and `CASE WHEN GROUPING(region)=1 THEN 'All regions' ELSE COALESCE(region, 'Unknown') END` to label both subtotal and missing-data rows.

Step-by-step thought process: 1. Start with the GROUP BY: `ROLLUP(region, category)` gives you region+category leaves, region subtotals, and a grand total. 2. Add the FILTER aggregates: one per status you want to break out — paid, refunded, failed. 3. Add ANY_VALUE for any column where you need a representative value (currency, primary language, primary timezone). 4. Use GROUPING() to label subtotal versus data rows; use COALESCE for the actual NULL-in-data case. Both wrapped in a single CASE expression. 5. Order with `GROUPING()` first so subtotals follow their group, and the grand total lands at the very bottom.

Edge cases to mention: - On dialects without FILTER, swap to `SUM(CASE WHEN status='paid' THEN amount END)` — identical semantics. - On dialects without ANY_VALUE (Postgres <16), swap to `MIN(currency)` or `MAX(currency)` with a comment explaining intent. - Watch out for currency mixing: if rows have different currencies, the SUM is meaningless — usually you'd group by currency first or convert to one currency in a subquery.

What NOT to say: - Do not write three separate queries and union them — the whole point is to do it in one scan. - Do not skip the GROUPING-vs-NULL distinction; reports look professional when subtotals are clearly labelled.

Common follow-up: *"How would you extend this to year-over-year comparison?"* — Add a CTE with previous-year totals and join on (region, category), then compute the percentage difference.

```
SELECT
  CASE
    WHEN GROUPING(region) = 1 THEN 'All regions'
    ELSE COALESCE(region, 'Unknown')
  END AS region,
  CASE
    WHEN GROUPING(category) = 1 THEN 'All categories'
    ELSE COALESCE(category, 'Unknown')
  END AS category,
  SUM(amount) FILTER (WHERE status = 'paid') AS paid_total,
  SUM(amount) FILTER (WHERE status = 'refunded') AS refund_total,
  ANY_VALUE(currency) AS sample_currency,
  GROUPING_ID(region, category) AS level
FROM orders
GROUP BY ROLLUP(region, category)
ORDER BY GROUPING(region), region, GROUPING(category), category;
```

Chapter 10, Part D — Generated Columns, Identity, Partitioning & Materialized Views

This is the closer for the Modern SQL Features chapter, and it is the section that most reliably separates a senior candidate from a mid one. The earlier parts covered expression-level features — windows, JSON, CTEs, recursive queries. This part is about table-level declarative constructs: generated columns that compute themselves on every insert, identity columns that issue keys without a separate sequence call, declarative partitioning that lets the planner skip whole chunks of data, and materialized views that pre-compute reports for dashboards. Interviewers ask these because the syntax varies wildly across Postgres, MySQL, SQL Server and Oracle, and because the right answer almost always involves a tradeoff — STORED vs VIRTUAL, IDENTITY vs UUID, COMPLETE vs CONCURRENTLY refresh. Read each Q twice: once for the syntax, once for the tradeoff buried in the edge cases.

Section A — Generated Columns

Q473 · GENERATED ALWAYS AS — Cross-Vendor Syntax

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, JP Morgan India, Wells Fargo · Topic: Generated columns

The question (interviewer's verbatim phrasing):

What is a generated column? Show me the syntax in Postgres, SQL Server and MySQL, and tell me what the differences are.

The 30-second answer: A generated column is a column whose value is computed from an expression over other columns in the same row — you never insert into it, the engine fills it automatically. Postgres only supports STORED (materialised on disk), SQL Server supports both PERSISTED and non-persisted, and MySQL supports both STORED and VIRTUAL.

Step-by-step thought process: 1. Lead with the one-liner: "computed from other columns, you never write to it directly." 2. Postgres syntax: `amount_paise INT GENERATED ALWAYS AS (amount_rupees * 100) STORED`. STORED is mandatory; Postgres has not implemented VIRTUAL yet. 3. SQL Server syntax: `amount_paise AS (amount_rupees * 100) PERSISTED` — note SQL Server uses `AS` without the `GENERATED ALWAYS` clause, and PERSISTED is the equivalent of STORED. 4. MySQL syntax: `amount_paise INT GENERATED ALWAYS AS (amount_rupees * 100) STORED` or `... VIRTUAL`. MySQL is the only one of the three with a true virtual variant — computed at SELECT time, no disk cost. 5. Be ready to explain the use case: derived totals, normalised search keys, JSON field extraction. Anything you would otherwise do in every SELECT.

Edge cases to mention: - The expression must be deterministic — no `NOW()`, no `RANDOM()`, no calls to non-deterministic functions. Postgres rejects these at table creation time. - You cannot reference another generated column in the same table in Postgres; SQL Server allows chained computed columns if the dependency is acyclic. - INSERTs that try to write a value into a generated column either error (with `GENERATED ALWAYS`) or are silently ignored — vendor-specific.

What NOT to say: - Do not call it a "computed column" in Postgres or a "generated column" in SQL Server — get the vendor terminology right. - Do not claim Postgres supports VIRTUAL — it does not, as of the current major version, and that is the single most common interview gotcha.

Common follow-up: "If the expression changes, what happens to existing rows?" — In Postgres and SQL Server you have to drop and re-add the column; the engine does not retroactively recompute STORED columns when you `ALTER` the expression.

```
-- Postgres
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  amount_rupees NUMERIC(10,2) NOT NULL,
  amount_paise BIGINT GENERATED ALWAYS AS (amount_rupees * 100) STORED
);
```

Q474 · STORED vs VIRTUAL Generated Columns

Tags: Difficulty: Medium · Asked at: Flipkart, PhonePe, Walmart Labs · Topic: Generated columns

The question (interviewer's verbatim phrasing):

In MySQL you can mark a generated column as STORED or VIRTUAL. When would you pick each one, and what does Postgres force you into?

The 30-second answer: STORED writes the computed value to disk on every INSERT or UPDATE — costs space and write time, but reads are free and the column is indexable like any other. VIRTUAL computes on every SELECT — saves disk but pays CPU on every read, and indexing rules are more restrictive. Postgres supported STORED only for generated columns through version 17; VIRTUAL generated columns were added in Postgres 18 (September 2025).

Step-by-step thought process: 1. Frame it as a classic space-vs-time tradeoff: STORED pays at write time, VIRTUAL pays at read time. 2. Pick STORED when the expression is expensive (string normalisation, JSON extraction, hashing) or when you need to index the column. SQL Server requires PERSISTED before you can index a non-deterministic computed column. 3. Pick VIRTUAL when the expression is cheap (simple arithmetic on existing columns), when storage is constrained, or when you need to add the column to a huge existing table without rewriting the heap. 4. Mention that VIRTUAL columns in MySQL can still be indexed — the index itself stores the computed value, so you are paying the storage cost in the index rather than the row.

Edge cases to mention: - Adding a STORED column to a billion-row table is a full table rewrite — schedule the migration carefully, or use a pt-online-schema-change style tool. - A VIRTUAL column that gets read in every query may end up costing more CPU than just storing it — measure before you assume VIRTUAL is cheaper. - Some replication setups (older MySQL row-based replication) ship the computed value over the wire for STORED columns but recompute for VIRTUAL — bandwidth implications.

What NOT to say: - Do not say "VIRTUAL is always cheaper" — it is cheaper on disk, not in CPU. - Do not say "you cannot index VIRTUAL columns in MySQL" — you can, since MySQL 5.7.

Common follow-up: "If you index a VIRTUAL column, where does the computed value actually live?" — In the index B-tree leaf entries. The row itself does not store it, but the index does, so the storage cost moves to the index pages.

```
-- MySQL: VIRTUAL with an index
CREATE TABLE products (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  name_lower VARCHAR(200) GENERATED ALWAYS AS (LOWER(name)) VIRTUAL,
  INDEX idx_name_lower (name_lower)
);
```

Q475 · Generated Column from JSON Field

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Meesho · Topic: Generated columns + JSON

The question (interviewer's verbatim phrasing):

I have an `events` table with a JSONB `payload` column and I keep filtering on `payload->>'event_type'`. The query is slow. What is the idiomatic fix?

The 30-second answer: Add a generated column that extracts the JSON field, then put a B-tree index on the generated column. The engine maintains the extracted value automatically, the index works like any normal index, and your query stays readable.

Step-by-step thought process: 1. Diagnose: filtering on `payload->>'event_type'` without an index forces a full scan plus JSON parsing per row. 2. The blunt fix is a functional index — `CREATE INDEX ON events ((payload->>'event_type'))` — which works but the column does not exist as a real attribute, so you cannot select on it cheaply or add a foreign key. 3. The senior answer is a STORED generated column: `event_type TEXT GENERATED ALWAYS AS (payload->>'event_type') STORED`. Now you have a real column, you can index it, you can constrain it, you can join on it. 4. Mention that this is the standard pattern for shredding hot JSON keys out of a JSONB blob — a hybrid model where the blob holds the long tail and generated columns expose the hot fields.

Edge cases to mention: - If `payload` is NULL or the key is missing, the generated column is NULL — make sure your downstream queries handle that. - The expression must be marked IMMUTABLE in Postgres for the column to be valid; `->>` is immutable, but a custom JSON parser function may not be. - Backfilling a generated column on an existing huge table rewrites every row — do it during a maintenance window or use a partitioned table where you migrate one partition at a time.

What NOT to say: - Do not suggest moving the field out of JSON entirely as the only option — generated columns let you keep schema flexibility while getting query speed. - Do not forget the index — the generated column by itself does not speed up the filter, it just makes the data structurally visible.

Common follow-up: *"What if you also need a GIN index on the whole JSONB?"* — Keep both. The generated column with a B-tree handles equality on the hot key; the GIN index on the JSONB handles ad-hoc containment queries on everything else.

```
-- Postgres
ALTER TABLE events
  ADD COLUMN event_type TEXT
  GENERATED ALWAYS AS (payload->>'event_type') STORED;

CREATE INDEX idx_events_type ON events (event_type);
```

Q476 · Generated Column for Case-Insensitive Search

Tags: Difficulty: Easy · Asked at: Infosys, Cognizant, Wipro · Topic: Generated columns

The question (interviewer's verbatim phrasing):

Users search for customer names case-insensitively, but `WHERE LOWER(name) = ?` is slow. Solve it without changing the application.

The 30-second answer: Add a generated column `name_lower` computed as `LOWER(name)`, index it, and rewrite the query — or let the application keep calling `LOWER(name) = ?` and add a functional index on `LOWER(name)`. Generated column is cleaner because the lowercased value is a first-class attribute.

Step-by-step thought process: 1. Acknowledge the symptom: the function call on the left side of `=` prevents the planner from using a normal index on `name`. 2. Two fixes — functional index or generated column. Both work; the generated column gives you a real attribute you can also SELECT, ORDER BY and use in joins. 3. The generated column should be STORED (or PERSISTED in SQL Server) so the engine can index it and the read path does no extra work. 4. For Postgres, you could alternatively use a case-insensitive collation (ICU) or the `citext` extension — mention these as the more modern alternative, especially if name search is pervasive.

Edge cases to mention: - `LOWER` on Unicode strings is locale-dependent — Turkish dotless `i` is the canonical example. Use a deterministic ICU collation if you care about correctness across locales. - The generated column doubles your storage for the name field — fine for one column, painful if you do it for ten. - If you also need accent-insensitive search, layer `unaccent(LOWER(name))` — both functions must be IMMUTABLE for Postgres to accept the expression.

What NOT to say: - Do not suggest a trigger that maintains a parallel column — generated columns exist precisely to avoid that anti-pattern. - Do not say "just store everything lowercased" — you lose the original casing for display, which is a UX regression.

Common follow-up: "What if the search is partial — `WHERE name_lower LIKE 'rahul%'` ?" — B-tree on `name_lower` still works for prefix matches because the leading anchor is a constant. For arbitrary substring matches you need a trigram index (`pg_trgm` in Postgres).

```
ALTER TABLE customers
  ADD COLUMN name_lower TEXT
  GENERATED ALWAYS AS (LOWER(name)) STORED;

CREATE INDEX idx_customers_name_lower ON customers (name_lower);
```

Q477 · Generated Column for Derived Totals

Tags: Difficulty: Easy · Asked at: TCS, Accenture, Walmart Labs · Topic: Generated columns

The question (interviewer's verbatim phrasing):

An `order_items` table has `quantity` and `unit_price`. Every report computes `line_total = quantity * unit_price`. Is there a better way?

The 30-second answer: Yes — add `line_total` as a generated column `GENERATED ALWAYS AS (quantity * unit_price) STORED`. Reports now read the column directly, you cannot drift between application and database calculations, and you can index `line_total` if you need to filter on it.

Step-by-step thought process: 1. Frame the problem: when the same arithmetic happens in many places, you eventually get a bug where one report uses pre-tax and another uses post-tax. A generated column is the single source of truth. 2. STORED is the right pick here because the multiplication is cheap but you might index `line_total` for queries like "all line items above ₹10,000". 3. Mention that this composes — you can have `line_total = quantity * unit_price` and then `taxed_total = line_total * (1 + tax_rate)` if the engine allows chained generated columns (SQL Server does, Postgres does not). 4. Be ready for the question about updates: if `quantity` or `unit_price` is updated, the engine recomputes `line_total` automatically on UPDATE.

Edge cases to mention: - If `quantity` is INT and `unit_price` is NUMERIC, the result type is NUMERIC — set the generated column type explicitly to avoid implicit-cast surprises in downstream queries. - A generated column cannot be the target of a foreign key in some engines — usually not a concern for derived totals, but worth knowing. - Discount logic that depends on customer tier cannot live in a generated column because it needs another table — use a view for that, not a generated column.

What NOT to say: - Do not say "just compute it in the application" — that loses the constraint that the database enforces consistency for every read path, including ad-hoc analyst queries. - Do not over-engineer with triggers — triggers are the pre-generated-column workaround and they are strictly worse.

Common follow-up: "What if quantity can be NULL?" — `NULL * anything = NULL`, so `line_total` will be NULL. Either constrain `quantity` to NOT NULL, or wrap the expression in `COALESCE(quantity, 0) * unit_price`.

```
CREATE TABLE order_items (  
  id BIGSERIAL PRIMARY KEY,  
  order_id BIGINT NOT NULL,  
  quantity INT NOT NULL,  
  unit_price NUMERIC(10,2) NOT NULL,  
  line_total NUMERIC(12,2) GENERATED ALWAYS AS (quantity * unit_price) STORED  
);
```

Q478 · Indexing Generated Columns

Tags: Difficulty: Medium · Asked at: Goldman, PhonePe, Cred · Topic: Generated columns + indexing

The question (interviewer's verbatim phrasing):

Can you index a generated column? Does it matter whether it is STORED or VIRTUAL?

The 30-second answer: Yes — STORED generated columns are indexable everywhere. VIRTUAL columns are indexable in MySQL (the computed value lives in the index), not directly in SQL Server unless persisted, and not at all in Postgres because Postgres does not support VIRTUAL. The index works exactly like any other B-tree, hash or GIN index.

Step-by-step thought process: 1. Open with the rule: STORED is always indexable; VIRTUAL depends on the engine. 2. In MySQL, indexing a VIRTUAL column writes the computed value into the index leaves — the row stays slim, but the index pays the storage cost. The optimizer can use that index for queries that reference the original expression too. 3. In SQL Server, a non-persisted computed column is indexable only if the expression is deterministic and precise — otherwise you need PERSISTED. The engine refuses to create the index if these checks fail. 4. In Postgres, since only STORED exists, indexing is straightforward. The functional-index alternative (`CREATE INDEX ON tbl ((expr))`) does the same thing without a column — pick generated column when you also want the value accessible as an attribute.

Edge cases to mention: - Multi-column indexes can mix generated and regular columns — useful when you want a composite key like `(status, line_total)` for fast filtering plus ordering. - A unique constraint on a generated column enforces uniqueness on the computed value — handy for normalised

forms (e.g. lowercase email). - Index maintenance overhead applies to UPDATES that change any input to the generated expression — measure the write penalty if you index aggressively.

What NOT to say: - Do not claim you can index a VIRTUAL column in Postgres — Postgres has no VIRTUAL. - Do not assume an index on a generated column will speed up the original expression query — in Postgres, only `WHERE generated_col = ?` uses it; the planner does not automatically rewrite `WHERE LOWER(name) = ?` to use an index on a `name_lower` column.

Common follow-up: *"How do you make Postgres use the generated-column index when the application keeps querying the raw expression?"* — Either change the application to query the generated column directly, or add a functional index on the expression too. The generated column is for structure, the functional index is for query rewriting.

```
-- Postgres: composite index on generated + regular columns
CREATE INDEX idx_orders_status_total
ON orders (status, line_total);
```

Q479 · Constraints on Generated Columns

Tags: Difficulty: Medium · Asked at: Walmart Labs, JP Morgan India, Wells Fargo · Topic: Generated columns + constraints

The question (interviewer's verbatim phrasing):

Can you put CHECK or NOT NULL or UNIQUE constraints on a generated column? Are there gotchas?

The 30-second answer: Yes to all three, with caveats. CHECK and NOT NULL work the same as on regular columns. UNIQUE works on STORED generated columns. Foreign keys are partially supported — Postgres allows them on STORED generated columns since version 12; SQL Server has more limits. The main gotcha is that constraint violations surface on the INSERT, not on the expression definition.

Step-by-step thought process: 1. Start with NOT NULL — straightforward, but if the underlying expression can produce NULL, your INSERT fails. This is actually a feature: it forces you to think about NULL inputs. 2. CHECK constraints work and are useful for narrowing the domain — e.g. `CHECK (line_total >= 0)` ensures the multiplication did not overflow into a negative. 3. UNIQUE on a generated column enforces uniqueness on the computed form — perfect for normalised email columns, slug columns, or canonicalised phone numbers. 4. Foreign keys: Postgres 12+ allows a FK on a STORED generated column. SQL Server requires PERSISTED and a few additional rules. Mention this only if asked — it is a niche but slick pattern.

Edge cases to mention: - A CHECK on a generated column is evaluated after the expression is computed — so if the expression itself errors (division by zero), you get the expression error, not the CHECK error. - UNIQUE plus a NULL-producing expression behaves per standard: multiple NULLs are allowed in most engines, which may or may not be what you want. - If you DROP the column that the generated column depends on, the generated column goes too — there is no auto-fix.

What NOT to say: - Do not claim generated columns cannot be the parent side of a foreign key in any engine — that was true historically, but Postgres relaxed it. - Do not use CHECK constraints to validate cross-table invariants — they are row-local; cross-table needs triggers or assertions.

Common follow-up: "If a CHECK constraint on a generated column suddenly starts failing, what is the most likely cause?" — A change in the input data distribution. The expression is deterministic, so the only way it newly violates a CHECK is because the inputs now produce values that did not exist before.

```
CREATE TABLE customers (  
  id BIGSERIAL PRIMARY KEY,  
  email TEXT NOT NULL,  
  email_lower TEXT GENERATED ALWAYS AS (LOWER(email)) STORED,  
  UNIQUE (email_lower),  
  CHECK (LENGTH(email_lower) > 3)  
);
```

Section B — Identity, Sequences and Auto-Increment

Q480 · GENERATED BY DEFAULT AS IDENTITY

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Wells Fargo · Topic: Identity columns

The question (interviewer's verbatim phrasing):

What is the standard SQL way to declare an auto-incrementing primary key, and how does it differ from the older `SERIAL` syntax?

The 30-second answer: The standard form is `id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`. It is the SQL standard syntax adopted by Postgres in version 10 and by Oracle in 12c. SQL Server does not support `GENERATED ... AS IDENTITY` — it uses its proprietary `IDENTITY(seed, increment)` syntax. It replaces vendor-specific things like `SERIAL` (Postgres), `AUTO_INCREMENT` (MySQL) and `IDENTITY(1,1)` (older SQL Server).

Step-by-step thought process: 1. Lead with the canonical form: `GENERATED BY DEFAULT AS IDENTITY` lets you omit the column on INSERT and the engine assigns the next value, but also lets you

override with an explicit value if you need to. 2. Contrast with `SERIAL` in Postgres — `SERIAL` is just shorthand for "create a sequence and set the default to nextval(...)". It works but it is not the standard, and you lose some metadata (the sequence is owned by but not bound to the column). 3. Mention `GENERATED ALWAYS AS IDENTITY` as the stricter variant — it disallows manual inserts unless you use `OVERRIDING SYSTEM VALUE`. 4. Be ready to explain why `GENERATED BY DEFAULT` is the more practical default for application tables: you can still seed data, replay logs and import dumps without fighting the engine.

Edge cases to mention: - Identity columns in Postgres are backed by a sequence under the hood — you can still query `pg_sequences` and manipulate it directly. - After a `COPY` or bulk insert with explicit IDs, you must `ALTER SEQUENCE ... RESTART WITH` to avoid duplicate-key errors on the next auto-generated value. - `BIGINT` is the right type — `INT` runs out in a few years on a busy production system, and the schema migration to widen the key is painful.

What NOT to say: - Do not use `SERIAL` in new Postgres schemas — it is the legacy spelling and the docs themselves recommend `GENERATED ... AS IDENTITY`. - Do not say "identity columns are slower than sequences" — they use the same mechanism.

Common follow-up: "What is the practical difference between `BY DEFAULT` and `ALWAYS`?" — `BY DEFAULT` lets you write `INSERT ... VALUES (5, ...)` and the engine accepts the 5. `ALWAYS` rejects it unless you say `OVERRIDING SYSTEM VALUE`. `ALWAYS` is safer for systems where IDs must come from the database.

```
-- Postgres / SQL Server / Oracle
CREATE TABLE orders (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id BIGINT NOT NULL,
  amount NUMERIC(10,2) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Q481 · GENERATED ALWAYS AS IDENTITY — Preventing Manual Inserts

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan India, Walmart Labs · Topic: Identity columns

The question (interviewer's verbatim phrasing):

When would you use `GENERATED ALWAYS AS IDENTITY` instead of `GENERATED BY DEFAULT`? What happens if someone tries to insert an explicit value?

The 30-second answer: Use `ALWAYS` when the ID must come from the database and never from the application — typically for surrogate keys in tables where consistency depends on monotonic, gap-

tolerant generation. An attempt to insert an explicit value raises an error, unless the inserting session uses the `OVERRIDING SYSTEM VALUE` escape hatch.

Step-by-step thought process: 1. Explain the contract: ALWAYS means the engine is the authority on the value. The application cannot accidentally collide with an existing ID by passing one in. 2. The most common scenario where this matters: tables that are being replicated or where the ID is used in business contracts. If an application accidentally inserts `id = 999999999`, the next auto-generated value clashes and inserts start failing. 3. The escape hatch — `INSERT INTO t (id, ...) OVERRIDING SYSTEM VALUE VALUES (...)` — is intentionally verbose so that bulk loads, restores and migrations can override but accidental inserts cannot. 4. Mention that ALWAYS does NOT prevent you from updating the column after insert — it only governs insertion. If you also want to lock the value, add a separate trigger or use a constraint.

Edge cases to mention: - Restoring a logical dump into a table with ALWAYS often needs `OVERRIDING SYSTEM VALUE` plus a sequence restart — pg_dump handles this, but custom dump scripts may not. - Some ORM's do not know how to issue `OVERRIDING SYSTEM VALUE` — test your migration tooling before flipping to ALWAYS. - In SQL Server the equivalent is `SET IDENTITY_INSERT t ON` — the spirit is the same but the syntax is procedural rather than per-statement.

What NOT to say: - Do not claim ALWAYS is "more secure" in a vague hand-wavy way — be specific about the consistency guarantee. - Do not blanket-recommend ALWAYS for every table — for staging or replay tables where bulk-loading with explicit IDs is routine, BY DEFAULT is friendlier.

Common follow-up: "What error do you get on an explicit insert into an ALWAYS column?" — Postgres: `ERROR: cannot insert into column "id"` with a hint to use `OVERRIDING SYSTEM VALUE`. Knowing the exact wording shows the interviewer you have actually hit it.

```
CREATE TABLE payments (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id BIGINT NOT NULL,
  amount NUMERIC(10,2) NOT NULL
);

-- This errors out:
INSERT INTO payments (id, order_id, amount) VALUES (1, 100, 500);

-- This is the escape hatch:
INSERT INTO payments (id, order_id, amount)
  OVERRIDING SYSTEM VALUE VALUES (1, 100, 500);
```

Q482 · Explicit SEQUENCES vs IDENTITY

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Flipkart · Topic: Sequences

The question (interviewer's verbatim phrasing):

When would you create a standalone SEQUENCE instead of using an IDENTITY column?

The 30-second answer: Create a standalone sequence when the generated number is used outside a single column — shared across multiple tables, used as an order number visible to the user, or pre-fetched in batches by the application. IDENTITY is the right default; standalone sequences are the escape hatch for these specific scenarios.

Step-by-step thought process: 1. State the default: IDENTITY first. Standalone sequences are not "more advanced"; they are a different tool with different ergonomics. 2. Use case one — shared sequence across tables. If `invoice_number` must be globally unique across `invoices_2025` and `invoices_2026`, a shared sequence is the cleanest way. 3. Use case two — pre-fetch. The application calls `nextval('order_seq')` once per batch and uses the values internally before inserting. Identity columns do not let you pre-fetch easily. 4. Use case three — business-visible IDs. Order numbers shown to customers often want gaps avoided, monotonic, maybe even with a prefix; a sequence with custom CACHE and CYCLE settings gives you full control.

Edge cases to mention: - Sequences are not transactional — `nextval` advances even if the transaction rolls back. You will see gaps. Document this for stakeholders who expect contiguous order numbers. - `CACHE n` on a sequence pre-allocates n values per session — improves throughput, increases gap size on crash. - Sequences are session-aware: `currval` returns the last value generated in the current session, useful for getting the just-assigned ID without a round-trip.

What NOT to say: - Do not claim sequences are gap-free — they explicitly are not. - Do not use a sequence where IDENTITY would do the same job — you are inviting future maintainers to wonder why.

Common follow-up: *"How do you handle the gap problem when stakeholders demand contiguous invoice numbers?"* — Either accept the gaps and explain that gap-free is incompatible with concurrent inserts, or use a separate gap-fill process that allocates numbers in a single-writer service. Trying to solve it with `SELECT MAX(...) + 1` is a race-condition trap.

```
CREATE SEQUENCE invoice_number_seq START 100001 INCREMENT 1 CACHE 50;

INSERT INTO invoices (invoice_number, customer_id, amount)
VALUES (nextval('invoice_number_seq'), 42, 1500.00);
```

Q483 · AUTO_INCREMENT (MySQL) vs IDENTITY

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Cognizant · Topic: Identity columns

The question (interviewer's verbatim phrasing):

In MySQL the syntax is `AUTO_INCREMENT`. How is it different from the standard `GENERATED ... AS IDENTITY` you would use in Postgres or SQL Server?

The 30-second answer: Functionally similar — both assign a monotonic integer to a column on insert. `AUTO_INCREMENT` is MySQL-specific, lives in the column definition, and lets you read and reset it via `LAST_INSERT_ID()` and `ALTER TABLE ... AUTO_INCREMENT = n`. Standard `IDENTITY` is portable across most modern engines and has finer-grained control over `BY DEFAULT` vs `ALWAYS`.

Step-by-step thought process: 1. Start with the syntactic difference: `id BIGINT AUTO_INCREMENT PRIMARY KEY` in MySQL vs `id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY` in Postgres or SQL Server. 2. MySQL has only one mode — it is equivalent to `BY DEFAULT` in the standard. You cannot configure it to reject explicit inserts the way `GENERATED ALWAYS` does. 3. Resetting the next value differs: `ALTER TABLE t AUTO_INCREMENT = 1000` in MySQL versus `ALTER SEQUENCE seq_name RESTART WITH 1000` in Postgres. 4. Cross-engine portability: if you write generic SQL, prefer the standard form. `AUTO_INCREMENT` is a deal-breaker for anyone migrating to Postgres or SQL Server.

Edge cases to mention: - MySQL's `AUTO_INCREMENT` counter is in memory in older InnoDB versions and can reset to `MAX(id)+1` after a server restart — leading to ID reuse after a delete. MySQL 8 fixed this by persisting the counter. - `LAST_INSERT_ID()` returns the first ID of a multi-row insert, not the last — a sharp-edge that bites people. - `AUTO_INCREMENT` works only with integer types; the standard `IDENTITY` also requires integer in most engines but the spelling is more forgiving.

What NOT to say: - Do not say `AUTO_INCREMENT` is "older" or "worse" — it is just MySQL's idiom. Worse and better depends on context. - Do not assume the counter is persisted across restarts in old MySQL versions — that bug bit a lot of production systems.

Common follow-up: "What does `LAST_INSERT_ID()` return after a 10-row `INSERT`?" — The auto-generated ID of the first row. To find the last, you do `LAST_INSERT_ID() + ROW_COUNT() - 1`. This is exactly the kind of trivia that catches mid candidates off-guard.

```
-- MySQL
CREATE TABLE orders (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  amount DECIMAL(10,2) NOT NULL
);

-- Reset the counter
ALTER TABLE orders AUTO_INCREMENT = 100001;
```


Q484 · IDENTITY Columns vs UUID — Distributed Systems

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred, Walmart Labs · Topic: Identity vs UUID

The question (interviewer's verbatim phrasing):

For a new payments microservice running on three database shards, would you use IDENTITY columns or UUIDs for the primary key? Defend your choice.

The 30-second answer: UUIDs (specifically UUIDv7 or ULIDs, which are time-ordered) are the better default for distributed systems because they let any node generate keys without coordination. IDENTITY columns require either a single sequence service, shard-offset schemes, or interleaved ranges — all of which are operationally fragile compared to a 128-bit time-ordered identifier.

Step-by-step thought process: 1. State the constraint that drives the decision: with multiple writers (shards, regions, edge nodes), a monotonic integer requires global coordination — and global coordination defeats the point of sharding. 2. Three options for sharded integer IDs: a central sequence service (single point of failure), shard offsets (`shard1: 1, 4, 7...` , `shard2: 2, 5, 8...` — clever but ugly), or per-shard sequences with a shard prefix (works but the application has to assemble the key). 3. UUIDv7 / ULID give you time-ordered, locally-generated 128-bit identifiers — no coordination, sortable by creation time, and index-friendly because new rows cluster together. 4. The classic objection — "UUIDs are 16 bytes, BIGINT is 8" — matters in tight inner loops but is rarely the bottleneck. Storage is cheap; coordination overhead is expensive.

Edge cases to mention: - Random UUIDv4 fragments B-tree indexes because every insert lands in a random place — UUIDv7 fixes this by encoding the timestamp in the leading bytes. - Some clients still serialize UUIDs as strings — switch to binary representation (16 bytes vs 36) for the storage savings. - If the table is small and single-master, the UUID overhead may not pay for itself — IDENTITY is simpler. Pick based on the actual architecture, not future plans.

What NOT to say: - Do not say "UUIDs are always better" — they have real costs, especially in inner-loop joins and on cache-bound workloads. - Do not propose UUIDv4 for a primary key without acknowledging the B-tree fragmentation — that is the senior-vs-mid signal.

Common follow-up: *"How does UUIDv7 fix the B-tree problem?"* — The first 48 bits are a Unix timestamp, so consecutive inserts land in nearby pages of the B-tree, behaving much more like an IDENTITY column for cache efficiency. This is why UUIDv7 has become the recommended UUID variant for primary keys.

```
-- Postgres with UUIDv7 (extension or generated client-side)
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT uuidv7(),
  order_id UUID NOT NULL,
  amount NUMERIC(10,2) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Q485 · Resetting and Restarting Identity Columns

Tags: Difficulty: Medium · Asked at: Wipro, Accenture, Wells Fargo · Topic: Identity management

The question (interviewer's verbatim phrasing):

After truncating a table or migrating data, how do you reset the identity column so the next insert starts from 1?

The 30-second answer: Postgres: `ALTER TABLE t ALTER COLUMN id RESTART WITH 1` or `ALTER SEQUENCE seq_name RESTART WITH 1`. SQL Server: `DBCC CHECKIDENT('t', RESEED, 0)`. MySQL: `ALTER TABLE t AUTO_INCREMENT = 1`. TRUNCATE by itself resets the counter in Postgres and SQL Server but not always in MySQL.

Step-by-step thought process: 1. Lead with TRUNCATE behaviour because it surprises people: `TRUNCATE TABLE t RESTART IDENTITY` in Postgres explicitly resets the sequence. Plain `TRUNCATE` does not, and MySQL's TRUNCATE resets AUTO_INCREMENT to 1 only sometimes (storage-engine-dependent). 2. For Postgres identity columns, the per-column syntax is `ALTER TABLE t ALTER COLUMN id RESTART WITH 1`. If you used `SERIAL` instead of IDENTITY, you need `ALTER SEQUENCE t_id_seq RESTART WITH 1`. 3. For SQL Server, `DBCC CHECKIDENT('t', RESEED, 0)` sets the seed so the next inserted value is 1. Note the documented quirk: if you reseed below MAX(id), you will get duplicate-key errors on the next insert. 4. Always reset to MAX(id) + 1 after a bulk load with explicit IDs — otherwise the next normal insert collides. This is the single most common production incident around identity columns.

Edge cases to mention: - Restarting an identity sequence does not delete existing rows — you can end up with duplicate keys if existing IDs are $\geq$ the new restart value. - In replication setups, restarting a sequence on the primary does not replicate to replicas in some configurations — verify your setup. - For Postgres, `setval(seq_name, n)` is the function-level equivalent and is what most migration tools use under the hood.

What NOT to say: - Do not just use `DELETE FROM t` expecting the identity to reset — DELETE never resets, only TRUNCATE does (and only with the right options). - Do not reset to 1 on a production table without checking MAX(id) first — that is a fast path to a duplicate-key outage.

Common follow-up: "After a logical restore from `pg_dump`, the next insert fails with a duplicate-key error. Why?" — The dump restored rows with explicit IDs, but the sequence still points to its old position. Run `SELECT setval('seq_name', (SELECT MAX(id) FROM t))` to fix it.

```
-- Postgres
ALTER TABLE orders ALTER COLUMN id RESTART WITH 1;
-- or
TRUNCATE TABLE orders RESTART IDENTITY;

-- SQL Server
DBCC CHECKIDENT('orders', RESEED, 0);

-- MySQL
ALTER TABLE orders AUTO_INCREMENT = 1;
```

Section C — Declarative Table Partitioning

Q486 · Postgres Declarative Partitioning — RANGE, LIST, HASH

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan India, Walmart Labs · Topic: Partitioning

The question (interviewer's verbatim phrasing):

Walk me through declarative partitioning in Postgres. What are the three strategies and when do you pick each?

The 30-second answer: Postgres 10 introduced declarative RANGE and LIST partitioning; HASH partitioning was added in Postgres 11. RANGE for time-series (one partition per month), LIST for discrete categories (one partition per country code), HASH for even distribution when the access pattern does not have a natural key — typically used for sharding without natural shard keys.

Step-by-step thought process: 1. RANGE: `PARTITION BY RANGE (created_at)`, then `CREATE TABLE orders_2026_01 PARTITION OF orders FOR VALUES FROM ('2026-01-01') TO ('2026-02-01')`. This is the workhorse — almost every "we have a huge events table" answer ends in monthly RANGE partitions. 2. LIST: `PARTITION BY LIST (region)` with one partition per known region. Good when partition boundaries are categorical and stable (state codes, payment status). 3. HASH: `PARTITION BY HASH (user_id) ... WITH (MODULUS 8, REMAINDER 0)`. Distributes load evenly when there is no natural range or list key — typically used when you want parallelism within a single server. 4. Mention that declarative partitioning (Postgres 10+) replaced the old inheritance + trigger approach — much faster, much less code, and the planner is partition-aware.

Edge cases to mention: - The partition key must be part of every primary key and unique constraint — you cannot have a globally unique constraint that ignores the partition key. - Foreign keys to a partitioned table are supported in modern Postgres but each partition must accept the FK — there are some still-evolving constraints. - Cross-partition updates that change the partition key are slow because they delete from one partition and insert into another — design to avoid them.

What NOT to say: - Do not confuse declarative partitioning with the old inheritance approach — they look similar but the new one is what every Postgres tutorial today shows. - Do not partition unless you have a real reason — partitioning a 10-million row table adds complexity without benefit.

Common follow-up: *"At what table size does partitioning start to pay off?"* — Rule of thumb: when a single index no longer fits in memory and queries naturally scope to a sub-range. Often hundreds of millions of rows for time-series. Small tables are slower with partitioning, not faster.

```
CREATE TABLE orders (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY,  
  user_id BIGINT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL,  
  amount NUMERIC(10,2) NOT NULL,  
  PRIMARY KEY (id, created_at)  
) PARTITION BY RANGE (created_at);  
  
CREATE TABLE orders_2026_05 PARTITION OF orders  
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
```

Q487 · Partition Pruning

Tags: Difficulty: Medium · Asked at: Flipkart, Goldman, PhonePe · Topic: Partitioning + planner

The question (interviewer's verbatim phrasing):

What is partition pruning and when does the planner actually use it?

The 30-second answer: Partition pruning is the planner's ability to skip partitions whose key range cannot possibly satisfy the WHERE clause. It works when the WHERE clause references the partition key with a constant or a value the planner can evaluate at plan time — and modern Postgres also supports execution-time pruning for parameterised queries.

Step-by-step thought process: 1. Start with the example: `SELECT * FROM orders WHERE created_at >= '2026-05-01'` on a table partitioned by month. The planner reads only the May 2026 partition. 2. Two flavours of pruning — plan-time (the planner knows the partition at plan time) and run-time (the value is a parameter, but the executor still skips at execution time). Run-time pruning was added in Postgres 11 and is essential for prepared statements. 3. Pruning fails when the WHERE clause

uses a function the planner cannot reason about —

`WHERE date_trunc('month', created_at) = '2026-05-01'` may not prune because `date_trunc` is not a simple comparison. 4. Always verify with EXPLAIN — look for "Subplans Removed" or "Partitions Removed" in the plan output. If you do not see it, the query is hitting every partition.

Edge cases to mention: - Pruning happens after the planner sees the WHERE clause — if you wrap the partition key in a function or apply implicit casts that the planner cannot push through, pruning silently fails. - JOINS across partitioned tables can prune partitions on both sides if the join condition uses the partition key — but only with `enable_partitionwise_join` on (it is off by default). - Default partition (the catch-all) is always scanned unless the planner can prove no value in the WHERE clause maps to it.

What NOT to say: - Do not assume pruning happens automatically for every query — write the EXPLAIN and read the plan. - Do not partition a table just to get parallelism — without WHERE clauses that prune, you get full scans of every partition.

Common follow-up: *"Why does*

WHERE created_at >= CURRENT_DATE - INTERVAL '7 days' sometimes not prune?" — In older Postgres versions, this could be evaluated at runtime rather than plan time, defeating plan-time pruning. Modern Postgres handles it via run-time pruning, but if you see surprising plans, check the version and consider rewriting to use a literal constant.

```
EXPLAIN
SELECT *
FROM orders
WHERE created_at >= '2026-05-01' AND created_at < '2026-06-01';
-- Look for: "Subplans Removed: N" indicating pruning is active
```

Q488 · Partition-Wise Joins

Tags: Difficulty: Hard · Asked at: Goldman, JP Morgan India, Walmart Labs · Topic: Partitioning + joins

The question (interviewer's verbatim phrasing):

What is a partition-wise join and when is it actually worth turning on?

The 30-second answer: A partition-wise join is when the planner joins two partitioned tables partition-by-partition — for each pair of matching partitions on each side, it does a smaller join and unions the results. It is worth turning on when both tables are partitioned by the same key with matching bounds, the partitions are large enough that the per-partition join is non-trivial, and you have parallelism available.

Step-by-step thought process: 1. Set the stage: you have `orders` and `order_items` both partitioned by `created_at` with monthly ranges. Without partition-wise join, the planner joins the whole `orders` table to the whole `order_items` table. 2. With `enable_partitionwise_join = on`, the planner generates plans like "join orders_2026_05 to order_items_2026_05, then union with the next month, and so on". Each sub-join is smaller and can run in parallel. 3. Wins are biggest when the per-partition data fits in memory and the join keys have selective indexes. Losses appear when partitions are tiny and the per-partition planning overhead dominates. 4. Mention partition-wise aggregation as the sibling feature — `enable_partitionwise_aggregate = on` lets the planner aggregate per partition and combine results, which is huge for time-series analytics.

Edge cases to mention: - Partitions must be bound-compatible — same key, same boundaries. If `orders` is monthly and `order_items` is weekly, partition-wise join does not work. - The planner has to consider many more plans, so query planning time goes up — for short OLTP queries this can hurt, for analytical queries it is worth it. - Memory pressure: each per-partition join needs `work_mem`, so eight parallel partition joins can use eight times the memory of a single join.

What NOT to say: - Do not turn it on globally without measuring — it adds planner overhead even when it does not produce a better plan. - Do not assume it works on heterogeneous partition layouts — it really does require matching bounds.

Common follow-up: *"What if the two tables are partitioned by the same key but with slightly different boundaries — say monthly vs every 28 days?"* — Partition-wise join does not apply. You either re-partition one side to match, or accept the non-partition-wise plan.

```
SET enable_partitionwise_join = on;
SET enable_partitionwise_aggregate = on;

EXPLAIN
SELECT o.user_id, SUM(oi.line_total)
FROM orders o JOIN order_items oi
  ON oi.order_id = o.id AND oi.created_at = o.created_at
WHERE o.created_at >= '2026-01-01'
GROUP BY o.user_id;
```

Q489 · Adding Partitions for New Time Ranges — Automation

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Meesho · Topic: Partition automation

The question (interviewer's verbatim phrasing):

You have a table partitioned by month. How do you make sure next month's partition exists before the first insert hits it?

The 30-second answer: Automate it. Either schedule a job that runs on the 25th of every month to create next month's partition, use an extension like `pg_partman` that manages the lifecycle for you, or set up a default partition as a safety net so unexpected dates do not error out.

Step-by-step thought process: 1. Start with the failure mode: if `2026-06-01` arrives and there is no `orders_2026_06` partition, every INSERT for June fails. This is a 3 AM page nobody wants. 2. The classic fix is a cron job that runs `CREATE TABLE orders_YYYY_MM PARTITION OF orders FOR VALUES FROM (...) TO (...)` for the upcoming month. Run it well in advance — at least a week. 3. `pg_partman` is the Postgres extension that does this declaratively — you tell it the partition interval and retention period, it handles create-new-partition and drop-old-partition. 4. Always have a default partition as the safety net — `CREATE TABLE orders_default PARTITION OF orders DEFAULT`. New rows that do not match any explicit partition land there, you get alerted on a monitoring metric, and you fix it before the volume becomes huge.

Edge cases to mention: - Creating a partition takes a brief lock on the parent table — fast on idle systems, can wait behind long transactions on busy ones. - `pg_partman` also handles partition retention — old partitions can be dropped or detached and archived to cold storage. - Default partition should be small in normal operation; if it grows, your automation has failed and you have a backlog of misplaced data to redistribute.

What NOT to say: - Do not say "we will just create them manually when needed" — that is fine until the on-call person is on vacation. - Do not skip the default partition because "it should never happen" — defensive design is exactly the kind of thing senior interviewers test for.

Common follow-up: *"What happens to rows in the default partition when you later add the missing explicit partition?"* — They do not move automatically. You have to detach the default, redistribute the rows, and reattach. Plan a clean-up window if the default partition has accumulated meaningful data.

```
-- Default partition as safety net
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

-- Automate next month's creation
DO $$
DECLARE
    next_month DATE := date_trunc('month', NOW() + INTERVAL '1 month')::DATE;
    next_next DATE := date_trunc('month', NOW() + INTERVAL '2 months')::DATE;
BEGIN
    EXECUTE format(
        'CREATE TABLE IF NOT EXISTS orders_%s PARTITION OF orders FOR VALUES FROM (%L) TO (%L)',
        to_char(next_month, 'YYYY-MM'), next_month, next_next);
END $$;
```

Q490 · Default Partition — The Catch-All

Tags: Difficulty: Easy · Asked at: TCS, Cognizant, Wipro · Topic: Partitioning

The question (interviewer's verbatim phrasing):

What is a default partition in Postgres and why would you use one?

The 30-second answer: A default partition is the catch-all that receives any row whose partition key does not match any explicit partition. It prevents insert failures when an out-of-range value arrives — useful as a safety net during automation gaps, but never as the primary storage strategy.

Step-by-step thought process: 1. Define it crisply: `CREATE TABLE orders_default PARTITION OF orders DEFAULT`. Any row that does not match an explicit partition goes here. 2. The primary use case is operational safety — your monthly partition automation missed a month, the next morning's inserts still succeed, and your monitoring alerts you that the default partition has data. 3. The secondary use case is LIST partitioning with a long tail — you partition by `region` for India, US, UK, and put everything else in default. Cleaner than enumerating fifty more regions. 4. Mention the cost: the planner always has to consider the default partition during pruning, because it cannot prove that no satisfying row is in there. This is small in practice but it does prevent some partition pruning optimizations.

Edge cases to mention: - You can add an explicit partition later that covers some of the default partition's range, but Postgres requires the default to be empty for that range — you must move the rows out first. - Default partitions inherit the same indexes as the parent — they are not a place to dump unindexed data. - For HASH partitioning, you cannot create a default partition because hash partitions always cover the full space.

What NOT to say: - Do not say "you can just dump everything in default" — defeats the purpose of partitioning. - Do not omit the default partition from your design out of misplaced strictness — it is the safety net that buys you debugging time.

Common follow-up: *"Can you add a new explicit partition that overlaps with rows currently in the default?"* — Not directly. You have to detach the default, move the conflicting rows, attach the new explicit partition, and reattach the default. Plan a maintenance window.

```
-- LIST partition with default for long-tail regions
CREATE TABLE customers (
    id BIGINT,
    region TEXT NOT NULL,
    name TEXT NOT NULL
) PARTITION BY LIST (region);

CREATE TABLE customers_in PARTITION OF customers FOR VALUES IN ('IN');
CREATE TABLE customers_us PARTITION OF customers FOR VALUES IN ('US');
CREATE TABLE customers_other PARTITION OF customers DEFAULT;
```


Q491 · Sub-Partitioning

Tags: Difficulty: Hard · Asked at: Walmart Labs, JP Morgan India, Goldman · Topic: Partitioning

The question (interviewer's verbatim phrasing):

What is sub-partitioning and when does it actually help?

The 30-second answer: Sub-partitioning is partitioning a partition — for example, partition `orders` by month at the top level, then sub-partition each month by `region`. It helps when both dimensions independently scope your queries and each individual sub-partition is still big enough to justify the per-partition overhead.

Step-by-step thought process: 1. Frame the problem: you have `orders` and your queries always filter by month and by region. Partitioning only by month means scanning all regions within the month; partitioning only by region means scanning all months within the region. 2. Sub-partition: top-level RANGE by month, second-level LIST by region. Each leaf partition is `orders_2026_05_in`, holding May 2026 Indian orders only. 3. The win is double pruning — the planner can drop entire months and entire regions independently. Combined with partition-wise joins, this gets very fast for analytical queries. 4. Acknowledge the cost: you now have $N \times M$ partitions instead of $N + M$. If you had 36 months and 5 regions, you go from 36 partitions to 180. Operational overhead climbs.

Edge cases to mention: - Postgres declarative partitioning supports sub-partitioning since version 10 — every partition can itself be `PARTITION BY ...`. - Default partitions at the sub-level are allowed, giving you a per-month catch-all. Useful for new regions. - Index management becomes harder — each leaf has its own index, which means more files on disk and more vacuum work.

What NOT to say: - Do not sub-partition speculatively — the operational overhead is real and you should have measured the access pattern first. - Do not sub-partition by HASH inside RANGE just for parallelism — single-server parallelism within a partition is usually a better lever.

Common follow-up: *"What is the simplest signal that you are about to over-partition?"* — When a typical query touches dozens of partitions and the per-partition fixed cost (open file, look at index root) starts to dominate the actual scan time. Run EXPLAIN ANALYZE and look at the planning time — if it grows linearly with partition count, you are over-partitioning.

```
CREATE TABLE orders (  
  id BIGINT,  
  created_at TIMESTAMPTZ NOT NULL,  
  region TEXT NOT NULL,  
  amount NUMERIC(10,2) NOT NULL  
) PARTITION BY RANGE (created_at);  
  
CREATE TABLE orders_2026_05 PARTITION OF orders  
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01')  
  PARTITION BY LIST (region);  
  
CREATE TABLE orders_2026_05_in PARTITION OF orders_2026_05 FOR VALUES IN ('IN');  
CREATE TABLE orders_2026_05_us PARTITION OF orders_2026_05 FOR VALUES IN ('US');
```

Q492 · MySQL Partitioning (Legacy Approach)

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Cognizant · Topic: MySQL partitioning

The question (interviewer's verbatim phrasing):

How does MySQL partitioning differ from Postgres declarative partitioning?

The 30-second answer: MySQL has had partitioning since version 5.1 with its own syntax — `PARTITION BY RANGE/LIST/HASH/KEY` defined inside the CREATE TABLE statement. It is functional but less flexible than Postgres declarative partitioning — no default partition, the partition key must be part of every unique key, sub-partitioning is limited to HASH or KEY at the second level, and partition-wise joins are not supported.

Step-by-step thought process: 1. Show the syntax:

`CREATE TABLE orders (...) PARTITION BY RANGE (YEAR(created_at)) (PARTITION p2025 VALUES LESS THAN (2026), PARTITION p2026 VALUES LESS THAN (2027), PARTITION pmax VALUES LESS THAN MAXVALUE)`. 2. The MAXVALUE partition is MySQL's equivalent of a default partition — anything beyond explicit ranges lands there. You can split it later but it is not as ergonomic as Postgres default. 3. MySQL supports RANGE, LIST, HASH, KEY (hash on the primary key), and RANGE COLUMNS / LIST COLUMNS (multi-column partition keys). Sub-partitioning is allowed only as RANGE→HASH or LIST→HASH — not the freeform composition Postgres allows. 4. Crucial constraint: every unique key (including the primary key) must contain the partition key. This forces a lot of schemas to add the partition key column to keys that semantically did not need it.

Edge cases to mention: - Foreign keys are not supported on partitioned tables in MySQL — a hard limitation that often disqualifies partitioning for transactional schemas. - Partition pruning works for simple expressions on the partition key — the optimizer is less aggressive than Postgres in pushing

through function calls. - ALTER TABLE to add or drop partitions takes a metadata lock — generally fast, but can stall during heavy concurrent writes.

What NOT to say: - Do not treat MySQL and Postgres partitioning as syntactic variants of the same thing — the feature sets differ meaningfully. - Do not suggest MySQL partitioning for a schema that uses foreign keys — the lack of FK support kills the design.

Common follow-up: *"If you cannot use foreign keys, how do you maintain referential integrity in a partitioned MySQL table?"* — Application-level checks plus careful transaction discipline. Or you keep small reference tables unpartitioned and only partition the high-volume fact tables that did not need FKs to begin with.

```
-- MySQL
CREATE TABLE events (
  id BIGINT AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  created_at DATETIME NOT NULL,
  payload JSON,
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (YEAR(created_at)) (
  PARTITION p2025 VALUES LESS THAN (2026),
  PARTITION p2026 VALUES LESS THAN (2027),
  PARTITION pmax VALUES LESS THAN MAXVALUE
);
```

Q493 · SQL Server Partitioning — Functions and Schemes

Tags: Difficulty: Medium · Asked at: JP Morgan India, Wells Fargo, AmEx · Topic: SQL Server partitioning

The question (interviewer's verbatim phrasing):

Explain SQL Server's partition function and partition scheme model.

The 30-second answer: SQL Server splits partitioning into two objects: a partition function that defines the boundary values, and a partition scheme that maps partitions to filegroups. You then create the table or index `ON scheme_name(partition_column)`. The model is more verbose than Postgres but it gives you fine control over which storage goes to which partition — useful for moving cold partitions to slow disks.

Step-by-step thought process: 1. Start with the two-object split: `CREATE PARTITION FUNCTION pf_monthly (DATETIME2) AS RANGE RIGHT FOR VALUES ('2026-01-01', '2026-02-01', ...)`. 2. Then the scheme: `CREATE PARTITION SCHEME ps_monthly AS PARTITION pf_monthly TO (fg_2025, fg_2026_01, fg_2026_02, ...)`. The scheme binds

each partition to a filegroup. 3. The table: `CREATE TABLE events (...) ON ps_monthly(created_at)`. SQL Server now stores each row in the partition (and the filegroup) that the function maps it to. 4. The killer feature is partition exchange via `ALTER TABLE ... SWITCH PARTITION`, which is a metadata-only operation — practically instant — used to load and unload data in bulk.

Edge cases to mention: - `RANGE RIGHT` vs `RANGE LEFT` determines whether the boundary value belongs to the right or left partition — confusing but matters at the boundary. - Splitting an existing partition can be expensive — it physically moves rows. Plan splits during maintenance windows. - Alignment: indexes should be "aligned" (partitioned the same way as the table) to benefit from SWITCH operations.

What NOT to say: - Do not confuse partition functions with table-valued functions — completely different objects. - Do not skip the filegroup design — the partition scheme is most of why people pick SQL Server partitioning in the first place.

Common follow-up: "How would you move data from a staging table into a specific partition of a production table?" — Load into a staging table with the same structure and filegroup as the target partition, then `ALTER TABLE staging SWITCH TO production PARTITION n`. Metadata-only, takes milliseconds.

```
-- SQL Server
CREATE PARTITION FUNCTION pf_monthly (DATETIME2)
AS RANGE RIGHT FOR VALUES
('2026-01-01', '2026-02-01', '2026-03-01');

CREATE PARTITION SCHEME ps_monthly
AS PARTITION pf_monthly
TO (fg_old, fg_2026_01, fg_2026_02, fg_2026_03);

CREATE TABLE events (
  id BIGINT IDENTITY PRIMARY KEY,
  created_at DATETIME2 NOT NULL,
  payload NVARCHAR(MAX)
) ON ps_monthly(created_at);
```

Q494 · Partition Exchange — Fast Bulk Load Pattern

Tags: Difficulty: Hard · Asked at: Walmart Labs, Goldman, JP Morgan India · Topic: Partition operations

The question (interviewer's verbatim phrasing):

You need to load 200 million rows into a partitioned table every night. The straight INSERT is too slow. What is the idiomatic pattern?

The 30-second answer: Load the data into a staging table that has the same structure as one partition, build all the indexes on the staging table, then attach the staging table as a partition (Postgres: `ATTACH PARTITION` ; SQL Server: `SWITCH PARTITION`). The exchange is metadata-only, so the production table sees the new data instantaneously.

Step-by-step thought process: 1. Frame the problem: straight INSERT updates indexes row-by-row, blows out the buffer cache, and competes with read traffic. For huge nightly loads this is a dealbreaker. 2. The pattern — build a standalone table with identical schema, bulk-load it (COPY in Postgres, BULK INSERT in SQL Server), create indexes on it in parallel, validate it, then attach. 3. In Postgres: `ALTER TABLE parent ATTACH PARTITION staging FOR VALUES FROM (...) TO (...)` . The planner checks that the data in staging matches the bounds — if it does, attachment is metadata-only. 4. In SQL Server: `ALTER TABLE staging SWITCH TO production PARTITION n` — even faster, because SQL Server has explicit partition exchange semantics.

Edge cases to mention: - For Postgres attach to be fast, the staging table must already have all the same indexes as the parent, and a CHECK constraint matching the partition bounds (`CHECK (created_at >= '2026-05-01' AND created_at < '2026-06-01')`) — Postgres uses the CHECK to skip the scan. - For SQL Server SWITCH, both tables must be on the same filegroup and have identical schema, including all indexes. - The reverse — `DETACH PARTITION` in Postgres or `SWITCH PARTITION OUT` in SQL Server — is how you cleanly archive a cold partition to a separate table that you can then dump or drop.

What NOT to say: - Do not propose plain INSERT for the nightly load — interviewers test specifically whether you know this pattern exists. - Do not skip the index build on the staging table — if you attach without indexes, the parent's queries will be slow for that partition.

Common follow-up: "What goes wrong if the staging table contains a row whose partition key falls outside the declared bounds?" — Postgres rejects the ATTACH outright. SQL Server's SWITCH also validates and rejects. Always run a sanity query (`SELECT MIN/MAX of the partition key`) before attaching.

```
-- Postgres bulk-load pattern
CREATE TABLE events_2026_05_staging (LIKE events INCLUDING ALL);

COPY events_2026_05_staging FROM '/data/may_2026.csv' WITH (FORMAT csv);

ALTER TABLE events_2026_05_staging
  ADD CONSTRAINT chk_may
  CHECK (created_at >= '2026-05-01' AND created_at < '2026-06-01');

ALTER TABLE events ATTACH PARTITION events_2026_05_staging
  FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');
```

Section D — Materialized Views

Q495 · Materialized View Basics — Postgres and Oracle

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Goldman · Topic: Materialized views

The question (interviewer's verbatim phrasing):

What is a materialized view and how does it differ from a regular view?

The 30-second answer: A regular view is a saved query — every SELECT against it executes the underlying query. A materialized view physically stores the query result on disk, so reads are as fast as reads from a table. The cost is that the stored data is a snapshot — it goes stale until you refresh it.

Step-by-step thought process: 1. Start with the dichotomy: regular view = virtual, materialized view = physical. The application queries both the same way; the difference is where the work happens. 2. Use case: a sales-by-month dashboard runs a 30-second aggregation query. Wrap it in a materialized view, refresh nightly, and the dashboard reads in 50 ms. 3. Mention that Postgres added materialized views in version 9.3 — the syntax is `CREATE MATERIALIZED VIEW name AS SELECT ...`. Oracle has had them forever and has more advanced refresh modes. 4. Be ready for the refresh-strategy question — that is where the depth lies.

Edge cases to mention: - A materialized view in Postgres can have indexes, just like a table — and those indexes are maintained during refresh. - Refresh is a full re-execute of the query in vanilla Postgres — there is no incremental refresh built in (Oracle has FAST refresh; Postgres relies on extensions). - The view is empty immediately after creation if you use `WITH NO DATA` — you have to refresh once before queries return anything.

What NOT to say: - Do not call a materialized view "a cached query" — that minimises the operational implications. It is a real table that needs space, indexing and refresh strategy. - Do not assume materialized view data is automatically up-to-date — that is the single biggest source of bugs in dashboards built on them.

Common follow-up: *"What is the difference between a materialized view and a summary table?"* — A materialized view has a declarative query attached, so the engine knows how to refresh it. A summary table is whatever you populated it with — usually maintained by your own ETL code. Materialized views are summary tables with batteries included.

```
-- Postgres
CREATE MATERIALIZED VIEW sales_by_month AS
SELECT date_trunc('month', created_at) AS month,
       region,
       SUM(amount) AS total
FROM orders
GROUP BY 1, 2;

CREATE INDEX idx_sales_by_month_region ON sales_by_month (region, month);
```

Q496 · Refresh Strategies — COMPLETE vs CONCURRENTLY vs FAST

Tags: Difficulty: Hard · Asked at: PhonePe, Cred, Walmart Labs · Topic: Materialized view refresh

The question (interviewer's verbatim phrasing):

Walk me through the refresh options for materialized views in Postgres and Oracle.

The 30-second answer: Postgres has two modes: **REFRESH MATERIALIZED VIEW** (rebuilds and takes an ACCESS EXCLUSIVE lock — blocks reads) and **REFRESH MATERIALIZED VIEW CONCURRENTLY** (rebuilds into a new copy and swaps — reads keep working but the view needs a unique index). Oracle has COMPLETE (rebuild) and FAST (incremental, requires materialized view logs on the base tables).

Step-by-step thought process: 1. Postgres COMPLETE (the default): the engine truncates the view, re-runs the defining query, and rebuilds the indexes. Fast for small views, terrible for big ones because the table is locked for the duration. 2. Postgres CONCURRENTLY (added in 9.4): the engine computes the diff using the unique index and applies INSERT/UPDATE/DELETE to bring the view in sync. Reads see consistent state throughout. Mandatory prerequisite: at least one unique index on the view. 3. Oracle COMPLETE: same as Postgres — full rebuild. 4. Oracle FAST: incremental refresh using materialized view logs (essentially triggers on the base tables that record changes). The view applies only the deltas — fast for big views with small change rates, but the logs add write overhead on the base tables.

Edge cases to mention: - Postgres CONCURRENTLY is slower than the non-concurrent version because it has to compute the diff — pick concurrent only when read availability matters more than refresh speed. - Oracle FAST refresh has many restrictions on the defining query — joins, aggregates and subqueries must all be in the supported subset. Read the Oracle docs carefully. - pg_ivm and similar Postgres extensions are starting to bring incremental refresh to Postgres — worth mentioning if asked about the future direction.

What NOT to say: - Do not call `REFRESH MATERIALIZED VIEW` "non-blocking" — without `CONCURRENTLY` it takes an `ACCESS EXCLUSIVE` lock that blocks every reader. - Do not promise Oracle-style `FAST` refresh in vanilla Postgres — it does not exist yet in the core product.

Common follow-up: "Why does `CONCURRENTLY` need a unique index?" — Because the diff algorithm joins old and new rows on the unique key to figure out which rows are insertions, deletions, and updates. Without a unique key it cannot identify rows.

```
-- Postgres
CREATE UNIQUE INDEX idx_sales_by_month_pk ON sales_by_month (month, region);

REFRESH MATERIALIZED VIEW CONCURRENTLY sales_by_month;
```

Q497 · Indexed Views in SQL Server (Equivalent)

Tags: Difficulty: Hard · Asked at: JP Morgan India, Wells Fargo, AmEx · Topic: Indexed views

The question (interviewer's verbatim phrasing):

SQL Server does not call them materialized views — it calls them indexed views. How do you create one and what are the restrictions?

The 30-second answer: You create a view with `WITH SCHEMABINDING`, then create a unique clustered index on the view. SQL Server then physically materialises the view and keeps it up to date automatically as the base tables change. The restrictions are extensive — deterministic `SELECT`, two-part names, no outer joins, no subqueries in many positions, and the right `SET` options on the session.

Step-by-step thought process: 1. The recipe: `CREATE VIEW v WITH SCHEMABINDING AS SELECT ...`, then `CREATE UNIQUE CLUSTERED INDEX idx ON v(...)`. `SCHEMABINDING` prevents the underlying tables from being altered in ways that break the view. 2. Once the clustered index exists, SQL Server automatically maintains the view on every base-table change — no manual refresh. This is the key difference from Postgres materialized views. 3. The cost: every `INSERT`, `UPDATE`, `DELETE` on the base tables now also writes to the indexed view. For heavy-write tables this can double or triple write cost. 4. The restrictions are real: deterministic, schema-bound, certain aggregates only (`COUNT_BIG`, `SUM`, but not `COUNT` directly), referenced tables must be in the same database. Read the official list before designing.

Edge cases to mention: - Required session `SET` options for the connection that creates the index include `ANSI_NULLS`, `ANSI_PADDING`, `ANSI_WARNINGS`, `ARITHABORT`, `CONCAT_NULL_YIELDS_NULL`, `QUOTED_IDENTIFIER` all `ON`. Forgetting one breaks the index creation. - The clustered index on the view must be unique — same constraint as Postgres `CONCURRENTLY`. - The Enterprise edition optimizer can use indexed views automatically even when the query references

the base tables directly (query rewrite). Standard edition does not — you must reference the view by name with `WITH (NOEXPAND)`.

What NOT to say: - Do not claim SQL Server indexed views are "just like Postgres materialized views" — the auto-maintenance changes the entire operational model. - Do not forget `SCHEMABINDING` — without it, the index creation fails.

Common follow-up: "*Why do you need SCHEMABINDING?*" — Because the indexed view materialisation depends on the schema of the base tables. `SCHEMABINDING` prevents `ALTER` or `DROP` of those tables in ways that would invalidate the view.

```
-- SQL Server
CREATE VIEW sales_by_month_v
  WITH SCHEMABINDING AS
SELECT region,
       CAST(YEAR(created_at) AS INT) AS yr,
       CAST(MONTH(created_at) AS INT) AS mo,
       SUM(amount) AS total,
       COUNT_BIG(*) AS row_count
FROM dbo.orders
GROUP BY region, YEAR(created_at), MONTH(created_at);

CREATE UNIQUE CLUSTERED INDEX idx_sbm
  ON sales_by_month_v (region, yr, mo);
```

Q498 · Materialized View vs Denormalization vs Cache

Tags: Difficulty: Hard · Asked at: Razorpay, Flipkart, Cred · Topic: Architecture tradeoffs

The question (interviewer's verbatim phrasing):

A dashboard needs aggregated metrics that take 30 seconds to compute. You could materialize the result, denormalize the underlying schema, or cache the response in Redis. How do you choose?

The 30-second answer: Materialize when the underlying query is complex but stable and the data can be slightly stale. Denormalize when you need real-time accuracy and you control all the writers. Cache when freshness windows are very short, the data is identical to what you return to the user, and you want minimal database changes. They are not exclusive — many production systems use all three at different layers.

Step-by-step thought process: 1. Materialized view sits in the database, refreshed on a schedule. Pros: declarative, queryable with SQL, indexable. Cons: stale until refresh, refresh load on the database, no incremental refresh in vanilla Postgres. 2. Denormalization moves aggregates into the base tables —

`users.total_orders` updated on every order insert. Pros: always fresh. Cons: every writer has to maintain the column, race conditions if multiple writers, schema couples reads to writes. 3. Cache (Redis, Memcached) sits outside the database. Pros: very low read latency, no database load, easy to invalidate. Cons: another piece of infrastructure, stale-after-update window, harder to query slices of. 4. Pick by access pattern: dashboards with daily refresh → materialized view. Per-user counters → denormalized. Hot per-page response data → cache. Often you layer: materialized view feeds the cache feeds the dashboard.

Edge cases to mention: - Materialized view refresh becomes a load spike — schedule it carefully. CONCURRENTLY helps but does not eliminate the spike. - Denormalized counters are race-prone — use atomic UPDATE (`SET count = count + 1`), not read-modify-write. - Cache invalidation is the second hardest problem in computer science — pick TTL-based or write-through deliberately.

What NOT to say: - Do not declare one approach universally best — interviewers want the multi-axis reasoning. - Do not skip the operational story (refresh load, cache eviction, schema migrations) — that is what separates an architecture answer from an implementation answer.

Common follow-up: *"Walk me through a layered architecture for the dashboard."* — Base tables → materialized view refreshed every 15 minutes → cache layer with 60-second TTL → API. Each layer absorbs a different latency budget, and a failure of one layer degrades gracefully.

```
-- Hybrid: materialized view as the truth, app caches reads
CREATE MATERIALIZED VIEW dashboard_metrics AS
SELECT region, date_trunc('day', created_at) AS day,
       SUM(amount) AS revenue, COUNT(*) AS orders
FROM orders
GROUP BY 1, 2;

CREATE UNIQUE INDEX idx_dm ON dashboard_metrics (region, day);
```

Q499 · Stale Materialized View Detection

Tags: Difficulty: Medium · Asked at: Goldman, JP Morgan India, Walmart Labs · Topic: Materialized view operations

The question (interviewer's verbatim phrasing):

How do you know whether a materialized view is up to date, and how do you alert when it has become stale?

The 30-second answer: Track the last refresh time as metadata — either in a small audit table you write to after every refresh, or by reading from system catalogs (`pg_stat_user_tables.last_analyze`

is a rough proxy; better is a custom audit table). Alert when `NOW() - last_refresh` exceeds your SLO. Some teams also compare row counts between base tables and the view to detect refresh lag.

Step-by-step thought process: 1. Postgres does not store the last refresh time on the materialized view itself — the catalog has no `last_refreshed_at` column. You have to record it yourself. 2. The simple pattern: a `mv_refresh_log` table with `mv_name`, `refreshed_at`, `duration_ms`, `rows_after`. Insert a row at the end of every refresh job. Your monitoring queries this. 3. Detect lag with `SELECT NOW() - MAX(refreshed_at) AS lag FROM mv_refresh_log WHERE mv_name = 'sales_by_month'`. Alert if lag exceeds the SLO — e.g. 90 minutes for a 60-minute refresh schedule. 4. Mention freshness queries — comparing the latest event timestamp in the base table to the latest in the view tells you the data lag, not just the wall-clock lag.

Edge cases to mention: - Wall-clock lag and data lag are different — the view might refresh on time but the source data ETL is backed up. Track both. - Refresh failures should alert separately — a refresh that errored out leaves the view at the last successful state, which silently goes stale. - For very large views, the refresh duration itself can exceed the refresh interval — instrument to catch this.

What NOT to say: - Do not assume the database has a built-in "is this view stale" function — Postgres does not. - Do not poll the materialized view itself to check freshness — query a metadata table, the view is what you are trying to test.

Common follow-up: *"What if the refresh job runs but the data inside the view is somehow wrong?"* — That is a data quality problem, not a freshness problem. Add a separate validation step — e.g. compare aggregate sums between the view and a sample re-aggregation of the base table.

```
-- Audit table + refresh wrapper
CREATE TABLE mv_refresh_log (
  mv_name TEXT NOT NULL,
  refreshed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  duration_ms INT,
  rows_after BIGINT
);

-- Alert query
SELECT mv_name, NOW() - MAX(refreshed_at) AS lag
FROM mv_refresh_log
GROUP BY mv_name
HAVING NOW() - MAX(refreshed_at) > INTERVAL '90 minutes';
```

Q500 · Materialized Views for Reporting — Common Patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart · Topic: Reporting patterns

The question (interviewer's verbatim phrasing):

Give me three concrete patterns where materialized views are the right tool for reporting.

The 30-second answer: First, daily aggregates — `sales_by_day` refreshed nightly powers every revenue dashboard. Second, expensive joins denormalized once — `enriched_orders` that joins orders, customers, products and payments. Third, rolling windows — `last_30_days_active_users` recomputed hourly. All three replace a slow query with a fast table read, accepting controlled staleness in exchange.

Step-by-step thought process: 1. Pattern one — pre-aggregated time series. Base table `orders` with millions of rows, materialized view `sales_by_day` with hundreds. Dashboard queries are sub-second. Refresh nightly with CONCURRENTLY so dashboards stay available. 2. Pattern two — flattened wide table for BI tools. Joins five reference tables to produce a fact table that Tableau or Metabase can query directly without complex SQL. Refresh hourly or whenever upstream tables change. 3. Pattern three — rolling-window metrics. `WHERE created_at >= NOW() - INTERVAL '30 days'` is expensive to compute on every dashboard load; materialize it and refresh hourly. Acceptable lag for cohort analytics. 4. Mention what is NOT a good fit — anything that needs real-time accuracy (transaction confirmations, current account balance), anything where the underlying data changes too fast to make any refresh window viable.

Edge cases to mention: - For dashboards with multiple users, the staleness perception is per-user — show a "last updated at" timestamp so users know they are not looking at live data. - If two materialized views feed the same dashboard and refresh independently, they can briefly disagree — schedule their refreshes together or document the inconsistency window. - Some materialized views naturally evolve in size — `sales_by_day` adds one row per day per region. Plan for retention and archiving.

What NOT to say: - Do not propose materialized views for ledger or balance queries — staleness is unacceptable there. - Do not use materialized views as a substitute for proper schema design — if your base tables are unfit, no view can save them.

Common follow-up: *"How do you handle a materialized view whose source data backfills retroactively — e.g. someone corrects a record from three weeks ago?"* — Schedule a deeper refresh window that goes back further than the natural change horizon. Or use a partition-aware refresh that rebuilds only the affected time slices. This is exactly the sort of operational nuance that signals senior experience to the interviewer.

```
-- Pattern: rolling 30-day active users
CREATE MATERIALIZED VIEW active_users_30d AS
SELECT user_id, COUNT(*) AS event_count, MAX(created_at) AS last_seen
FROM events
WHERE created_at >= NOW() - INTERVAL '30 days'
GROUP BY user_id;

CREATE UNIQUE INDEX idx_au30_user ON active_users_30d (user_id);

-- Refresh hourly via cron
REFRESH MATERIALIZED VIEW CONCURRENTLY active_users_30d;
```